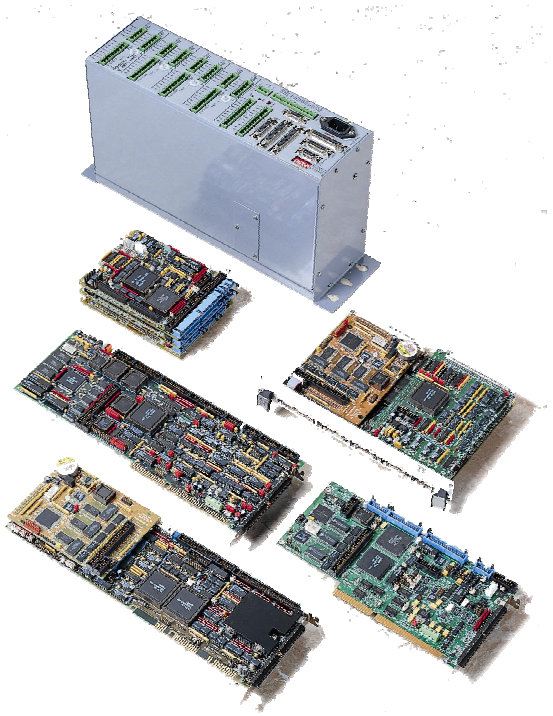
SOFTWARE MANUAL

PMAC / PMAC2



Programmable Multi-Axis Controller

3Ax-602204-xSxx

February 19, 2003

DELTA TAU
Data Systems, Inc.
NEW IDEAS IN MOTION ...

Single Source Machine Control *Power // Flexibility // Ease of Use*
21314 Lassen Street Chatsworth, CA 91311 // Tel. (818) 998-2095 Fax. (818) 998-7807 // www.deltatau.com

Contents

Chapter Number	Chapter/Paragraph Title	Page Number
1.	**PMAC Summary Information**	**1-1**
	PMAC Command Summary	**1-1**
	Notes	1-1
	Definitions	1-1
	On-Line Commands	1-2
	On-line Global Commands	1-2
	On-line Coordinate System Commands	1-5
	On-line Motor Commands	1-6
	Motion Program Commands	1-8
	PLC Program Commands	1-12
	PMAC I-Variable Summary	1-13
	Global I-Variables	1-13
	Motor I-Variables	1-13
	Motor Definition I-Variables	1-13
	Motor Safety I-Variables	1-14
	Motor Movement I-Variables	1-14
	Motor Basic Servo Control I-Variables	1-14
	Motor Servo Loop Modifications	1-15
	Motor Commutation I-Variables	1-15
	Further Motor I-Variables	1-16
	Coordinate System I-Variables	1-16
	Encoder/Flag Setup I-Variables	1-17
	For Encoder n (n = 1 -> 16)	1-17
	Macro Support I-Variables	1-17
	PMAC Error Code Summary	1-18
	PMAC Syntax Notes	1-18
2.	**PMAC I-Variable Specification**	**2-1**
	I-Variable Functions	2-1
	Global I-Variables	2-2
	I0 PMAC2 Card Number (PMAC2 Only)	2-2

I1 Serial Port Mode ... 2-3
I2 Control Panel Disable ... 2-3
I3 I/O Handshake Control ... 2-4
I4 Communications Integrity Mode ... 2-6
I5 PLC Programs On/Off ... 2-8
I6 Error Reporting Mode ... 2-8
I7 In-Position Number of Cycles ... 2-9
I8 Real Time Interrupt Period ... 2-10
I9 Full/Abbreviated Program Listing Form ... 2-10
I10 Servo Interrupt Time (PMAC[1] Specific) ... 2-13
I10 Servo Interrupt Time (PMAC2 Specific) ... 2-14
I11 Programmed Move Calculation Time ... 2-16
I12 Jog-to-Position Calculation Time ... 2-16
I13 Programmed Move Segmentation Time ... 2-17
I14 Auto Position Match on Run Enable ... 2-17
I15 Degree/Radian Control for User Trig Functions ... 2-18
I16 Rotary Buffer Request On Point ... 2-18
I17 Rotary Buffer Request Off Point ... 2-20
I18 Fixed Buffer Full Warning Point ... 2-21
Data Gathering I-Variables ... 2-21
I19 Data Gathering Period (in Servo Cycles) ... 2-21
I20 Data Gathering Selection Mask ... 2-21
I21 Data Gathering Source 1 Address ... 2-22
I22--I44 Data Gathering Source 2 thru 24 Addresses ... 2-23
I45 Data Gathering Buffer Location and Mode ... 2-23
I46 (Reserved For Future Use) ... 2-24
I47 Address of Pointer for Control-W Command ... 2-24
I48 DPRAM Servo Data Enable ... 2-25
I49 DPRAM Background Data Enable ... 2-25
I50 Rapid Move Mode Control ... 2-26
I51 Leadscrew Compensation Enable ... 2-26
I52 '\' Program Hold Slew Rate ... 2-26
I53 Program Step Mode Control ... 2-27
I54 Serial Baud Rate (PMAC2 Only) ... 2-28
I55 DPRAM Background Variable Buffers Enable ... 2-28
I56 DPRAM ASCII Communications Interrupt Enable (PMAC[1] Specific) ... 2-29

I56 DPRAM ASCII Communications Interrupt Enable (PMAC2 Specific) 2-29
I57 DPRAM Binary Rotary Buffer Enable 2-30
I58 DPRAM ASCII Communications Enable 2-30
I59 DPRAM Buffer Maximum Motor/C.S. Number 2-30
I60 Auto-Converted ADC Register Address PMAC[1] Specific) 2-31
I61 Number of Auto-Converted ADC Registers (PMAC[1] Specific) 2-32
I60, I61 Analog-to-Digital Conversion Support (Deleted) 2-33
I62 Internal Message Carriage Return Control (PMAC2 Specific) 2-33
I8x Motor x Third-Resolver Gear Ratio 2-34
I89 Cutter Comp Outside Corner Break Point 2-34
I9x Motor x Second-Resolver Gear Ratio 2-36
I99 Backlash Hysteresis 2-36
Motor I-Variables 2-38
Motor Definition I-Variables 2-38
Ix00 Motor x Activate 2-38
Ix01 Motor x PMAC-Commutation Enable 2-38
Ix02 Motor x Command Output (DAC) Address 2-38
Ix02 Motor x Command Output Address 2-41
Ix03 Motor x Position Loop Feedback Address 2-43
Ix04 Motor x Velocity Loop Feedback Address 2-44
Ix05 Motor x Master (Handwheel) Position Address 2-46
Ix06 Motor x Master (Handwheel) Following Enable 2-47
Ix07 Motor x Master (Handwheel) Scale Factor 2-47
Ix08 Motor x Position Scale Factor 2-48
Ix09 Motor x Velocity Loop Scale Factor 2-48
Ix10 Motor x Power-On Servo Position Address 2-49
Motor Safety I-Variables 2-51
Ix11 Motor x Fatal (Shutdown) Following Error Limit 2-51
Ix12 Motor x Warning Following Error Limit 2-52
Ix13 Motor x Positive Software Position Limit 2-53
Ix14 Motor x Negative Software Position Limit 2-53
Ix15 Motor x Deceleration Rate on Position Limit or Abort 2-54
Ix16 Motor x Maximum Permitted Motor Program Velocity 2-54
Ix17 Motor x Maximum Permitted Motor Program Acceleration 2-54
Ix19 Motor x Maximum Permitted Motor Jog/Home Acceleration 2-57
Motor Movement I-Variables 2-57
Ix20 Motor x Jog/Home Acceleration Time 2-57

Ix21 Motor x Jog/Home S-Curve Time 2-58
Ix22 Motor x Jog Speed 2-58
Ix23 Motor x Homing Speed and Direction 2-59
Ix24 (Reserved For Future Use) 2-59
Ix25 Motor x Limit/Home Flag/Amp Flag Address (PMAC[1] Specific) 2-59
Ix25 Motor x Flag Address (PMAC2 Specific) 2-63
Ix26 Motor x Home Offset 2-65
Ix27 Motor x Position Rollover Range 2-66
Ix28 Motor x In-position Band 2-66
Ix29 Motor x Output - or First Phase - DAC Bias (PMAC[1] Specific) 2-68
Ix29 Motor x Output/First Phase Offset (PMAC2 Specific) 2-68
Servo Control I-Variables 2-69
Ix30 Motor x PID Proportional Gain 2-70
Ix31 Motor x PID Derivative Gain 2-70
Ix32 Motor x PID Velocity Feedforward Gain 2-71
Ix33 Motor x PID Integral Gain 2-71
Ix34 Motor x PID Integration Mode 2-72
Ix35 Motor x PID Acceleration Feedforward Gain 2-72
Ix36 Motor x PID Notch Filter Coefficient N1 2-73
Ix37 Motor x PID Notch Filter Coefficient N2 2-73
Ix38 Motor x PID Notch Filter Coefficient D1 2-73
Ix39 Motor x PID Notch Filter Coefficient D2 2-74
Ix40 - Ix56 Motor x Extended Servo Algorithm I-Variables 2-74
Motor Servo Loop Modifiers 2-74
Ix57 Motor x Continuous Current Limit (PMAC[1] Specific) 2-74
Ix57 Motor x Continuous Current Limit (PMAC2 Specific) 2-75
Ix58 Motor x Integrated Current Limit (PMAC[1] Specific) 2-75
Ix58 Motor x Integrated Current Limit (PMAC2 Specific) 2-77
Ix59 Motor x User-Written Servo/Phase Enable 2-78
Ix60 Motor x Servo Cycle Period Extension (PMAC[1] Only) 2-78
Ix61 Motor x Current Loop Integral Gain (PMAC2 Only) 2-79
Ix62 Motor x Current Loop Proportional Gain (Forward Path) (PMAC2 Only) 2-79
Ix63 Motor x Integration Limit 2-80
Ix64 Motor x Deadband Gain Factor 2-81
Ix65 Motor x Deadband Size 2-83
Ix66 Motor x PWM Scale Factor (PMAC2 Only) 2-83
Ix67 Motor x Linear Position Error Limit 2-83

Ix68 Motor x Friction Feedforward 2-84
Ix69 Motor x Output Command (DAC) Limit (PMAC[1] Specific) 2-84
Ix69 Motor x Output Command Limit (PMAC2 Specific) 2-85
Commutation I-Variables 2-86
Ix70 Motor x Number of Commutation Cycles (N) (PMAC[1] Specific) 2-86
Ix70 Motor x Number of Commutation Cycles (N) (PMAC2 Specific) 2-86
Ix71 Motor x Encoder Counts per N Commutation Cycles (PMAC[1] Specific) 2-87
Ix71 Motor x Counts per N Commutation Cycles (PMAC2 Specific) 2-87
Ix72 Motor x Commutation Phase Angle (PMAC[1] Specific) 2-88
Ix72 Motor x Commutation Phase Angle (PMAC2 Specific) 2-88
Ix73 Motor x Phase Finding Output Value (PMAC[1] Specific) 2-89
Ix73 Motor x Phase Finding Output Value (PMAC2 Specific) 2-90
Ix74 Motor x Phase Finding Time (PMAC[1] Specific) 2-91
Ix74 Motor x Phase Finding Time (PMAC2 Specific) 2-92
Ix75 Motor x Power-On Phase Position Offset 2-92
Ix76 Motor x Velocity Phase Advance Gain PMAC[1] Specific) 2-93
Ix76 Motor x Current-Loop Proportional Gain (Back Path) (PMAC2 Specific) 2-94
Ix77 Motor x Induction Motor Magnetization Current (PMAC[1] Specific) 2-94
Ix77 Motor x Magnetization Current (PMAC2 Specific) 2-95
Ix78 Motor x Induction Motor Slip Gain (PMAC[1] Specific) 2-95
Ix78 Motor x Slip Gain (PMAC2 Specific) 2-95
Ix79 Motor x A Phase Output (DAC) Bias (PMAC[1] Specific) 2-96
Ix79 Second Phase Offset (PMAC2 Specific) 2-96
Ix80 Motor x Power-Up Mode (PMAC[1] Specific) 2-97
Ix80 Motor x Power-Up Mode (PMAC2 Specific) 2-99
Ix81 Motor x Power-On Phase Position Address (PMAC[1] Specific) 2-100
Ix81 Motor x Power-On Phase Position Address (PMAC2 Specific) 2-101
Ix82 Current loop feedback address (PMAC 2 Only) 2-104
Ix83 Motor x Ongoing Phasing Position Address (PMAC[1] Specific) 2-106
Ix83 Motor x Commutation Position Address (PMAC2 Specific) 2-107
Further Motor I-Variables 2-108
Ix84 Current-Loop Feedback Mask Word (PMAC2 Only) 2-108
Ix85 Motor x Backlash Take-up Rate 2-108
Ix86 Motor x Backlash Size 2-109
Coordinate System I-Variables 2-109
Ix87 Coordinate System x Default Program Acceleration Time 2-110
Ix88 Coordinate System x Default Program S-Curve Time 2-111

Ix8 Coordinate System x Default Program Feedrate / Move Time 2-111
Ix90 Coordinate System x Feedrate Time Units 2-112
Ix91 Coordinate System x Default Working Program Number 2-112
Ix92 Coordinate System x Move Blend Disable 2-113
Ix93 Coordinate System x Time Base Control Register Address 2-113
Ix94 Coordinate System x Time Base Slew Rate (and Limit) 2-114
Ix95 Coordinate System x Feed Hold Deceleration Rate 2-115
Ix96 Coordinate System x Circle Error Limit 2-115
PMAC[1] Encoder/Flag Setup I-Variables 2-117
I900, I905, ... I975 Encoder n Decode Control "Encoder I-Variable 0" 2-117
I901, I906, ... I976 Encoder n Filter Disable "Encoder I-Variable 1" 2-118
I902, I907, ... I977 Encoder n Position Capture Control "Encoder I-Variable 2" 2-119
I903, I908, ... I978 Encoder n Flag Select Control "Encoder I-Variable 3" 2-121
I904, I909, ... I979 – (Reserved For Future Use) 2-121
I1000 MACRO Node Auxiliary Register Enable 2-121
I1001 MACRO Ring Check Period 2-122
Global Gate Array I-variables (PMAC2 Only) 2-123
I900 MaxPhase and PWM 1-4 Frequency Control 2-123
I901 Phase Clock Frequency Control 2-124
I902 Servo Clock Frequency Control 2-125
I903 Hardware Clock Control Channels 1-4 2-126
I904 PWM 1-4 Deadtime/PFM 1-4 Pulse Width Control 2-128
I905 DAC 1-4 Strobe Word 2-129
I906 PWM 5-8 Frequency Control 2-129
I907 Hardware Clock Control Channels 5-8 2-131
I908 PWM 5-8 Deadtime / PFM 5-8 Pulse Width Control 2-132
Channel-Specific Gate Array I-variables (For Channel n, where n = 1 to 8) (PMAC2 Specific) 2-133
I9n0 Encoder/Timer n Decode Control 2-133
I9n1 Position Compare n Channel Select 2-134
I9n2 Encoder n Capture Control 2-135
I9n3 Capture n Flag Select Control 2-136
I9n4 Encoder n Gated Index Select 2-138
I9n5 Encoder n Index Gate State 2-139
I9n6 Output n Mode Select 2-139
I9n7 Output n Invert Control 2-140
I9n8 Output n PFM Direction Signal Invert Control 2-141
I9n9 Reserved for future use 2-141

DSPGATE2 I-Variables (PMAC2 Only) 2-141
I990 Handwheel 1 Decode Control 2-141
I991 Handwheel 2 Decode Control 2-142
I992 MaxPhase and PWM 1*-2* Frequency Control 2-143
I993 Hardware Clock Control Channels 1*-2* 2-144
I994 PWM 1*-2* Deadtime / PFM 1* Pulse Width Control 2-146
I995 MACRO Ring Configuration/Status 2-147
I996 MACRO Node Activate Control 2-148
I997 Phase Clock Frequency Control 2-150
I998 Servo Clock Frequency Control 2-150
I999 Reserved for future use 2-151
MACRO Control I-Variables (PMAC2 Only) 2-152
I1000 MACRO Node Auxiliary Register Enable 2-152
I1001 MACRO Ring Check Period 2-153
I1002 MACRO Node Protocol Type Control 2-153
I1003 MACRO Type 1 Auxiliary Communications Timeout 2-153
PMAC2 Default Conversion Tables 2-154
Non-Ultralite PMAC2 Boards 2-154
PMAC2 Ultralite Boards 2-154

3. PMAC On-Line Command Specification 3-1
On-Line Command Syntax and Function 3-1
< 3-1
<CONTROL-B> 3-2
<CONTROL-C> 3-3
<CONTROL-D> 3-3
<CONTROL-E> 3-4
<CONTROL-F> 3-5
<CONTROL-G> 3-5
<CONTROL-H> 3-6
<CONTROL-I> 3-6
<CONTROL-K> 3-7
<CONTROL-L> 3-7
<CONTROL-M> 3-9
<CONTROL-N> 3-9
< 3-10
<CONTROL-P> 3-10
< 3-11

<
<CONTROL-S>
<CONTROL-T>
<CONTROL-U>
<CONTROL-V>
<CONTROL-W>
<CONTROL-X>
<CONTROL-Y>
<CONTROL-Z>
#
#{constant}
#
#
#
$
$$$
$
%
%{constant}
&{constant}
&
/
?
??
???
@
@{*card*}
\
A
ABS
{axis}={constant}
B{constant}
CLEAR
CLOSE
{constant}
DATE
D

D 3-51
D 3-55
DEFINE GATHER 3-57
DEFINE ROTARY 3-58
D 3-59
D 3-60
DEFINE UBUFFER 3-60
D 3-62
D 3-62
DELETE GATHER 3-63
DELETE PLCC 3-64
DELETE ROTARY 3-65
D 3-65
DELETE TCOMP 3-66
DELETE TRACE 3-66
DISABLE PLC 3-67
DISABLE PLCC 3-68
ENABLE PLC 3-69
ENABLE PLCC 3-70
ENDGATHER 3-71
F 3-71
FRAX 3-72
GATHER 3-73
H 3-74
HOME 3-74
HOMEZ 3-75
I{constant} 3-76
I{constant}={expression} 3-77
I{constant}=* 3-79
INC 3-79
J 3-80
J+ 3-81
J- 3-82
J/ 3-82
J:{constant} 3-83
J 3-83
J= 3-84

J={constant} 3-86
J 3-86
J 3-87
J^{constant} 3-88
J 3-89
{ 3-90
K 3-91
LEARN 3-92
LIST 3-93
LIST COMP 3-94
L 3-94
LIST GATHER 3-95
LIST LDS 3-97
LIST LINK 3-97
LIST PC 3-97
LIST PE 3-99
LIST PLC 3-100
LIST PROGRAM 3-100
M{constant} 3-102
M{constant}={expression} 3-103
M{constant}-> 3-103
M{constant}->* 3-104
M{constant}->D:{address} 3-105
M{constant}->DP:{address} 3-105
M{constant}->F:{address} 3-107
M{constant}->L:{address} 3-108
M 3-109
M 3-109
M 3-111
M 3-112
M{constant}->X/Y:{address} 3-115
MACROAUX 3-117
MACROAUXREAD 3-117
MACROAUXWRITE 3-119

MFLUSH 3-120
O{constant} 3-120
OPEN PLC 3-121
OPEN PROGRAM 3-122
O 3-123
P 3-124
P{constant} 3-125
P{constant}={expression} 3-126
PASSWORD={string} 3-126
PC 3-128
PE 3-128
PMATCH 3-129
PR 3-130
Q 3-131
Q{constant} 3-131
Q 3-132
R 3-132
R[H]{*address*} 3-134
S 3-134
SAVE 3-135
SIZE 3-136
TYPE 3-137
UNDEFINE 3-138
UNDEFINE ALL 3-139
V 3-139
VERSION 3-140
W{address} 3-141
Z 3-142

4. PMAC Program Command Specification 4-1
Buffered Program Commands 4-1
{axis}{data}[{axis}{data}...] 4-1
{ 4-2
{axis}{data}^{data}[{axis}{data}^{data}...] 4-3
{ 4-5
A{data} 4-7
ABS 4-7

ADDRESS 4-8
ADIS{constant} 4-9
AND ({condition}) 4-10
AROT{constant} 4-11
B{data} 4-11
BLOCKSTART 4-12
BLOCKSTOP 4-13
C{data} 4-14
CALL 4-14
CC0 4-15
CC1 4-16
CC2 4-17
CCR{data} 4-18
CIRCLE1 4-18
CIRCLE2 4-19
COMMAND"{command}" 4-20
COMMAND^{letter} 4-22
D{data} 4-23
DELAY{data} 4-24
DISABLE PLC {constant}[,{constant}...] 4-24
DISABLE PLCC {constant}[,{constant}...] 4-25
DISPLAY [{constant}] "{message}" 4-26
DISPLAY ... {variable} 4-28
DWELL 4-29
ELSE 4-30
ENABLE PLC 4-32
ENABLE PLCC 4-32
ENDIF 4-33
ENDWHILE 4-34
F{data} 4-35
FRAX 4-36
G{data} 4-37
GOSUB 4-38
GOTO 4-38
HOME 4-39
HOMEZ 4-40
I{data} 4-41

I{constant}={expression} 4-42
IDIS{constant} 4-43
IF ({condition}) 4-44
INC 4-46
IROT{constant} 4-47
J{data} 4-48
K{data} 4-48
Linear 4-49
M{constant}={expression} 4-50
M{constant}=={expression} 4-51
M{constant}&={expression} 4-52
M{constant}|={expression} 4-53
M{constant}^={expression} 4-53
M{data} 4-54
MACROAUXREAD 4-55
MACROAUXWRITE 4-55
N{constant} 4-56
NORMAL 4-57
O{constant} 4-58
OR({condition}) 4-58
P{constant}={expression} 4-60
PRELUDE 4-61
PSET 4-64
PVT{data} 4-65
Q{constant}={expression} 4-66
R{data} 4-67
RAPID 4-68
READ 4-68
RETURN 4-71
S{data} 4-72
SEND 4-73
SEND^{letter} 4-74
SPLINE1 4-76
SPLINE2 4-77
STOP 4-78
T{data} 4-78
TA{data} 4-79

TINIT 4-80
TM{data} 4-80
TS{data} 4-82
TSELECT{constant} 4-82
U{data} 4-83
V{data} 4-84
W{data} 4-85
WAIT 4-85
WHILE({condition}) 4-86
X{data} 4-88
Y{data} 4-88
Z{data} 4-89

5. PMAC I/O and Memory Map 5-1
Introduction 5-1
Global Servo Calculation Registers 5-2
Motor Calculation Registers (PMAC[1] Specific) 5-4
Motor Calculation Registers (PMAC2 Specific) 5-6
Encoder Conversion (Interpolation) Table (PMAC[1] Specific) 5-9
Encoder Conversion (Interpolation) Table (PMAC2 Specific) 5-10
General Global Registers 5-10
General Global Registers (PMAC[1] Specific) 5-11
General Global Registers (PMAC2 Specific) 5-11
Motor and Coordinate System Status and Control Registers 5-12
Buffer Management Registers (PMAC[1] Specific) 5-16
Buffer Management Registers (PMAC2 Specific) 5-17
Gate Array Registers (PMAC[1] Specific) 5-18
DSPGATE1 Registers (PMAC2 Specific) 5-20
DSPGATE2 Registers (PMAC2 Specific) 5-25
Dual-Ported RAM (Option 2 Required) 5-35
VME-Bus Registers (PMAC-VME and PMAC2-VME only) 5-35
I/O Control Registers (PMAC2 Specific) 5-36
Inputs and Outputs (PMAC-PC, PMAC-VME, and PMAC-Lite Only) 5-37
Inputs and Outputs (PMAC-STD Only) 5-38
Inputs and Outputs (PMAC1.5-STD Only) 5-41
Expansion I/O (PMAC-PC, PMAC-VME, and PMAC-Lite Only) 5-48

6. Example Programs 6-1
PMAC Example Programs 6-1
Example 1: Suggested M-variable Definitions (PMAC[1] Specific) 6-1
Example 1: Suggested M-Variable Definitions (PMAC2 Specific) 6-13
Example 2: A Simple Move 6-26
Example 3: A More Complex Move 6-27
Example 4: Conditional Branching 6-27
Example 5: Linear and Circular interpolation 6-28
Example 6: A Simple G-Code Program 6-29
Example 7: A Robotic Application 6-30
Example 8: Rotary Tangent to X-Y System 6-31
Example 9: PLC Input Programming 6-31
Example 10: PLC Display Programming 6-32
Example 11: Precision Slaving 6-32
Example 12: Matching Positions on the Fly 6-33
Example 13: Spindle Operation 6-34
Example 14: Torque-limited Velocity Move 6-36
Example 15: A Torque Limited Screwing Operation 6-36
Example 16: Altering Destination in Moves 6-37
Example 17: Changing Speed 6-37
Example 18: Circular Interpolation with Linear Moves 6-38
Example 19: Sending Information to a Display 6-40
Example 20: In Position Confirming Routines 6-42
Example 21: Encoder Modulo Check 6-43
Example 22: Store The Home-Position Encoder Offset 6-44
Example 23: Pure Position Step 6-45
Example 24: Pulse Output with Motors 6-45
Example 25: PLC I/O 6-46
Example 26: Circular Interpolation 6-47
Example 27: Proportional Voltage Output 6-48
Example 28: Pulsing a Machine Output 6-49
Example 29: Jogging Motors From A PLC Program 6-49
Example 30: Open-loop Movement 6-50
Example 31: Inverse Kinematics 6-52
Example 32: Orientation of a Rotary Head 6-54
Example 33: Time Base Following 6-55

Example 34: Move Until Trigger 6-57
Example 35: Linear and Circular Moves 6-58
Example 36: Motion Program Execution 6-58
Example 37: G-code Programming 6-60
Example 38: Delayed Program Execution 6-62
Example 39: Integrated Current Limiting 6-63
Host Example Programs 6-64
PMACPOLL.C 6-64
PMACINT.C 6-69
PMACPROT.C 6-75
PMACIROT.C 6-80
PMACLERN.C 6-85
PMACRAM.C 6-89

7. PMAC PROM Software Update Listings 6-1
Instructions to Change a PROM 6-1
Backing Up Your Data 6-1
Standard and Option 5 PMACs 6-1
PMACs with Options 4A, 5A, and 5B 6-2
From V1.08 to V1.09 (2/13/91) 6-4
Additions 6-4
From V1.09 to V1.11 (7/30/91) 6-4
Changes 6-4
Additions 6-5
From V1.11 to V1.12 (4/13/92) 6-6
Changes 6-6
Fixes 6-6
Additions 6-6
From V1.12 to V1.13 (10/21/92) 6-8
Additions 6-8
Update Summary: From V1.13 To V1.14 (July 93) 6-9
Changes 6-9
Additions 6-10
Removals 6-11
Update Summary: From V1.14 To V1.15 (April 94) 6-12
Changes 6-12
Additions 6-13
Update Summary: From V1.15 to V1.16 (July 96) 6-15

Changes 6-15
Additions 6-15



PMAC Summary Information

PMAC Command Summary

Notes

PMAC syntax is not case sensitive.

Spaces are not important in PMAC syntax, except where noted

`{}` *-- item in* `{}` *can be replaced by anything fitting definition*

`[]` *-- item in* `[]` *is optional to syntax*

`[{item}...]` *-- indicates previous item may be repeated in syntax*

`[..{item}]` *-- the periods are to be included in the syntax to specify a range*

`[..{item}]` *-- the periods are to be included in the syntax to specify a range*

`()` *-- parentheses are to be included in syntax as they appear*

Definitions

constant - numerically specified non-changing value

variable - entity that holds a changeable value

I-variable - variable of fixed meaning for card setup and personality (1 of 1024)

P-variable - global variable for programming use (1 of 1024)

Q-variable - local var. (in coord. sys.) for programming use (1 of 1024)

M-variable - variable assigned to memory location for user use (1 of 1024)

pre-defined variable - mnemonic that has fixed meaning in card

function - SIN,COS,TAN,ASIN,ACOS,ATAN,ATAN2,LN,EXP,SQRT,ABS,INT

operator - for arithmetic or bit-by-bit logical combination of two values: +, -, *, /, % (mod), & (and), | (or), ^ (xor)

expression - grouping of const's, var's, fn's, and operators

data - constant w/out parentheses, or expression w/ parentheses

comparator - evaluates relationship between two values: =, !=, >, !>, <, !<, ~, !~

condition - evaluates as true or false based on comparator(s)

simple condition - {expression} {comparator} {expression}

compound condition - logical combination of simple conditions

motor - element of control for hardware setup; specified by num

coordinate system - collections of motors working synchronously

axis - element of a coordinate system; specified by letter chosen from X, Y, Z, A, B, C, U, V, W

buffer - space in user memory for program or list; contains up to 256 motion programs and 32 PLC blocks

On-Line Commands

On-Line Commands are executed immediately upon receipt by PMAC.

On-line Global Commands

Addressing Mode Commands

```
@n  --  Address card n (n is hex digit 0 to f); serial host only
@   --  Report currently addressed card to host; serial host only
#n  --  make motor n currently addressed motor
#   -- report currently addressed motor number to host
&n  --  make coord. sys. n the currently addressed coord. sys.
&   --  report currently addressed coordinate system to host
```

Communications Control-Characters

<CTRL-H> -- Erase last character from host (backspace)

<CTRL-I> -- Repeat last command from host (tab)

<CTRL-M> -- End of command line (carriage return)

<CTRL-N> -- Report checksum of current command line

<CTRL-T> -- Toggle serial communications full/half duplex

<CTRL-W> -- Execute ASCII command from DPRAM buffer

<CTRL-X> -- Abort current PMAC command and response strings

<CTRL-Y> -- Report last command to host; ready to repeat to card

<CTRL-Z> -- Make serial port the communications port

General Global Commands

$$$ -- global reset: including all motors and coord. systems

$$$* --** reset and re-initialize entire card

PASSWORD={string} -- set/confirm password for PROG1000-32767,PLC0-15

SAVE -- save I-variables into EEPROM

UNDEFINE ALL -- erase definition of all coordinate systems

Global Action Commands

<CTRL-A> -- Abort all motion programs and moves

<CTRL-D> -- Disable all PLC and PLCC programs

<CTRL-K> -- Kill outputs for all motors

<CTRL-L> -- Close rotary program buffer

<CTRL-O> -- Do feed hold on all coordinate systems

<CTRL-Q> -- Quit all programs at end of calculated moves

<CTRL-R> -- Run working programs in all coordinate systems

<CTRL-S> -- Step working programs in all coordinate systems

<CTRL-U> -- Open rotary program buffer

Global Status Commands

<CTRL-B> -- Report all motor status words to host

<CTRL-C> -- Report all coordinate system status words to host

<CTRL-E> -- Report address contents in binary according to data gathering configuration

<CTRL-F> -- Report all motor following errors (unscaled)

<CTRL-G> -- Report global status words in binary form

<CTRL-P> -- Report all motor positions (unscaled)

<CTRL-V> -- Report all motor velocities (unscaled)

??? -- Report global status words in hex ASCII

DATE -- Report date of firmware version used

LIST [{buffer}] -- Report contents of open [or specified] buffer

SIZE -- Report size of open memory in words (sub-blocks)

TYPE -- Report type of PMAC

VERSION -- Report firmware revision level

Register Access Commands

R{address}[,{constant}] -- Report contents of specified memory word address [or specified number of addresses] in decimal

RH{address}[,{constant}] -- Report contents of specified memory word address [or specified number of addresses] in hex

W{address},{constant} [,{constant}..] -- Write value to specified memory word address[or values to range]

PLC Control Commands

ENABLE PLC{constant}[,{constant}...] -- Enable operation of specified interpreted PLC program[s]

DISABLE PLC{constant}[,{constant}...] -- Disable operation of specified interpreted PLC program[s]

ENABLE PLCC{constant}[,{constant}...] -- Disable operation of specified compiled PLC program[s]

DISABLE PLCC{constant}[,{constant}...] -- Disable operation of specified compiled PLC program[s]

Global Variable Commands

{constant} -- Equivalent to P0={constant} if no unfilled table; otherwise value entered into table

I{constant}={expression} -- Expression value assigned to I-variable

I{constant}[..{constant}]=* -- Set I-variable[s] to default[s]

P{constant}[..{constant}]={expression} -- Expression value assigned to P-variable(s)

M{constant}[..{constant}]={expression} -- Expression value assigned to M-variable(s)

M{constant}->{definition} -- M-variable defined as **specified**

M{constant}->* -- M-variable defined as non-pointer variable

I{constant}[..{constant}] -- report I-variable value(s) to host

P{constant}[..{constant}] -- report P-variable value(s) to host

M{constant}[..{constant}] -- report M-variable value(s) to host

M{constant}[..{constant}]-> -- report M-variable definition(s) to host

Buffer Control Commands

OPEN PROG {constant} -- open specified motion program buffer for entering/editing

OPEN ROT -- open all defined rotary program buffer for entry

OPEN PLC{constant} -- open specified PLC program buffer for entry

CLOSE -- close currently opened buffer

CLEAR -- erase contents of opened buffer

DEFINE GATHER [{constant}] -- set up a data gathering buffer using all open memory [or of specified size]

DELETE GATHER -- erase the data gathering buffer

GATHER [TRIGGER] -- Start data gathering [on external trigger]

ENDGATHER -- Stop data gathering

DELETE PLCC{constant} -- Erase specified compiled PLC program

DELETE TRACE -- Erase the program trace buffer (no action taken; kept for backward compatability)

DEFINE TBUF{constant} -- set up specified number of axis transformation matrices

DELETE TBUF -- Erase all axis transformation matrices

DEFINE UBUFFER{constant} -- set up a user buffer of specified number of words

On-line Coordinate System Commands

On-line Coordinate System Commands act immediately on currently addressed coordinate system.

Axis Definition Commands

#n->[{constant}]{axis}[+{constant}] -- define axis in terms of motor #, scale factor, and offset.

Examples: ..... **#1->X**

#4->2000A+500

#n>[{constant}]{axis}[+[{constant}]{axis}[+[{constant}]{axis}]][+{constant}] -- define 2 or 3 axes in terms of motor #, scale factors, and offset. Valid only within XYZ or UVW groupings.

Examples: ..... **#1->8660X-5000Y**

#2->5000X+8660Y+5000

#n-> -- report axis definition of motor n in this C. S.

UNDEFINE -- erase definition of all axes in this C. S.

General coordinate-system commands

?? -- report coordinate system status in hex ASCII form

%{constant} -- specify feedrate override value

% -- report current feedrate override value to host

Program Control Commands

R -- Run current program

S -- Do one step of current program

B[{constant}] -- Set program counter to specified location

H -- Feed hold for coordinate system

A -- Abort present program or move starting immediately

Q -- Halt program; stop moves at end of last calculated program command

/ -- Halt program execution at end of currently executing move

**** -- Do program hold that permits jogging while in hold mode

MFLUSH -- Erase contents of synchronous M-variable stack without executing

Coordinate-System Variable Commands

Q{constant}[..{constant}]={expression} -- Assign expression value to Q-variable(s)

Q{constant}[..{constant}] -- report Q-variable value(s) to host

Axis Attribute Commands

{axis}={expression} -- Change value of commanded axis position

Z -- Make present actual position of all axes in coordinate system equal to zero.

INC[({axis}[,{axis}...])] -- Make all [or specified] axes do their moves incrementally

ABS[({axis}[,{axis}...])] -- Make all [or specified] axes do their moves absolute

FRAX({axis}[,{axis}...]) -- Make specified axes to be used in vector feedrate calculations

PMATCH -- Re-match coordinate system axis positions to motor commanded positions (used in case axis definition or motor position changed since last *axis* move); not valid if there is axis-motor cross-coupling.

Buffer Control Commands

PC -- report next program number and line (offset) to be executed to host

LIST PC[,[{constant}] -- List next line of working program [and specified lines afterward] to be calculated

PE -- report working program number and line (offset) currently executing to host

LIST PE[,[{constant}] -- List currently executing line of working program [and specified lines afterward]

DEFINE ROT {constant} -- Establish rotary motion program buffer of specified word size for the addressed coordinate system

DELETE ROT -- Erase rotary motion program buffer for addressed coordinate system

PR -- Report number of lines between executing point and last loaded line in rotary program buffer.

LEARN -- Read present commanded positions and add as axis commands to open program buffer

On-line Motor Commands

These act immediately on the currently addressed motor. Except for the reporting commands, these commands are rejected if the motor is in a coordinate system that is currently running a motion program.

General Motor Commands

$ -- reset motor -- feedback device(s) and phasing

HM -- perform homing routine for motor

HMZ -- perform zero-move homing routine for motor

K -- kill output for motor

O{constant} -- open-loop output of specified magnitude

Jogging Commands

J+ -- jog motor indefinitely in positive direction

J- -- jog motor indefinitely in negative direction

J/ -- stop jogging motor; also restore to position control

J= -- jog motor to last pre-jog or pre-handwheel position

J={constant} -- jog motor to specified position

J=* -- variable jog to position

J:{constant} -- jog motor specified distance from current commanded position

J:* -- variable incremental jog from current commanded position

J^{constant} -- jog motor specified distance from current actual position

J^* -- variable incremental jog from current actual position

{jog command}^{constant} -- jog until trigger, final value specifies distance from trigger position to stop

Reporting Commands

P -- report position of motor

V -- report velocity of motor

F -- report following error of motor

? -- report status words for motor in hex ASCII form

Buffer Control Commands

DEFINE BLCOMP {entries},{count length} -- establish backlash compensation table for motor; to be filled by specified number of values .

DELETE BLCOMP -- erase backlash compensation table for motor.

DEFINE COMP {entries},[#{source},[#{target},]],{count length} -- establish leadscrew compensation table for motor; to be filled by specified number of values.

DEFINE COMP {rows}.{columns},[#{source1},[#{source2}, [#{target},]]],{count length1},{count length2} -- establish two-dimensional leadscrew compensation table for motor; to be filled by specified number of values.

DELETE COMP -- erase leadscrew compensation table for motor.

DEFINE TCOMP {entries},{count length} -- establish torque compensation table for motor; to be filled by specified number of values .

DELETE TCOMP -- erase torque compensation table for motor.

Motion Program Commands

Move Commands

{axis}{data}[{axis}{data}...] -- simple movement statement; can be used in blended, transition-point, or splined modes.

Example: **X1000 Y(P1) Z(P2*P3)**

{axis}{data}:{data}[{axis}{data}:{data}...] -- pos and vel move; to be used only in PVT mode.

Example: **X5000:750 Y3500:(P3) A(P5+P6):100**

{axis}{data}^{data}[{axis}{data}^{data}...] -- move until trigger, variant of RAPID mode.

{axis}{data}[{axis}{data}...][{vector}{data}...] -- circle move; to be used only in circular mode; vector is to circle center

Example: **X2000 Y3000 Z1000 I500 J300 K500**

DWELL{data} -- keep same commanded position; fixed time base

DELAY{data} -- keep same commanded position; variable time base

HOME{constant}[,{constant}...] -- home specified *motor(s)*

HOMEZ{constant}[,{constant}...] -- do zero-move homing of specified *motor(s)*

Move Mode Commands

LINEAR -- blended linear interpolation move mode

RAPID -- mode where all axes move a maximum velocity and accel.

CIRCLE1 -- clockwise circular interpolation move mode

CIRCLE2 -- counterclockwise circular interpolation move mode

PVT{data} -- position/velocity/time transition-point move mode (parabolic velocity profiles)

SPLINE1 -- uniform cubic spline move mode

SPLINE2 -- non-uniform cubic spline move mode

CC0 -- turns off cutter radius compensation

CC1 -- turns on cutter radius compensation left

CC2 -- turns on cutter radius compensation right

Axis Attribute Commands

ABS[({axis}[,{axis},...])] -- makes all [or specified] axes in absolute move mode

INC[({axis}[,{axis},...])] -- makes all [or specified] axes in incremental move mode

FRAX[({axis}[,{axis}...])] -- specifies feedrate axes

NORMAL{vector}{data}[{vector}{data}...] -- specifies normal vector to plane for circular moves and cutter compensation

PSET{axis}{data}[{axis}{data}...] -- sets axis position values

R{data} -- spec.'s circle radius; neg. val. is long arc

CCR{data} -- specifies cutter radius compensation value (modal)

TSEL{data} -- selects specified axis transformation matrix

TINIT -- initializes selected axis transformation matrix as identity matrix

ADIS{data} -- sets displacement vector of selected matrix to values starting with specified Q-variable

IDIS{data} -- increments displacement vector of selected matrix to values starting with specified Q-variable

AROT{data} -- sets rotation/scaling portion of selected matrix to values starting with specified Q-variable

IROT{data} -- incrementally changes rotation/scaling portion of selected matrix by multiplying it with values starting with specified Q-variable

Move Attribute Commands

TM{data} -- specifies move time (modal)

F{data} -- specifies move speed (modal)

TA{data} -- specifies move acceleration time (modal)

TS{data} -- specifies acceleration S-curve time (modal)

Variable Assignment Commands

I{constant}={expression} -- assigns expression value to specified I-variable

P{constant}={expression} -- assigns expression value to specified P-variable(s)

Q{constant}={expression} -- assigns expression value to specified Q-variable(s)

M{constant}={expression} -- assigns expression value to specified M-variable(s)

M{constant}==={expression} -- assigns expression synchronous with start of next move

M{constant}&={expression} -- Mvar ANDed w/exp. synchronously

M{constant}|={expression} -- Mvar ORed w/exp. synchronously

M{constant}^={expression} -- Mvar XORed w/exp. synchronously

Program Logic Control

N{constant} -- line label

O{constant} -- alternate line label

GOTO{data} -- jump to specified N-label; no return

GOSUB{data}[{letter}{axis}...] -- jump to specified N-label and return [with arguments]

CALL{data}[.{data}][{letter}{axis}...] -- jump to specified program [and label] [with arguments] and return.

RETURN -- return program operation to most recent GOSUB or CALL

READ({letter} [,{letter}...]) -- Allows subprogram or subroutine to take arguments

G{data} -- Gnn[.mmm] interpreted as CALL 1000.nnmmm (PROG 1000 provides subroutines for desired G-Code actions.)

M{data} -- Mnn[.mmm] interpreted as CALL 1001.nnmmm (PROG 1001 provides subroutines for desired M-Code actions.)

T{data} -- Tnn[.mmm] interpreted as CALL 1002.nnmmm (PROG 1002 provides subroutines for desired T-Code actions.)

D{data} -- Dnn[.mmm] interpreted as CALL 1003.nnmmm (PROG 1003 provides subroutines for desired D-Code actions.)

S{data} -- sets Q127 to value of {data}

PRELUDE1{call command} --for modal execution of call command before subsequent moves

PRELUDE0 -- de-activates modal PRELUDE calls

IF({condition}){action} -- conditionally execute action

IF({condition}) -- conditionally execute following statements

ELSE {action} -- execute action on previous false condition

ELSE -- execute following statements on previous false condition

ENDIF -- follows last of conditionally executed statements

WHILE({condition}){action} -- do action as long as condition true

WHILE({condition}) -- do following statements as long as true

ENDWHILE -- follows last of conditionally executed statements

BLOCKSTART -- so all commands until BLOCKSTOP to execute on Step

BLOCKSTOP -- end of stepped statements starting on BLOCKSTART

STOP -- halts program execution; ready to resume

WAIT -- used with WHILE to halt execution while condition true

Miscellaneous commands

COMMAND "{command}" -- issue command as if it came from host

COMMAND^{letter} -- issue control character command

SEND "{message}" -- transmit message over host interface

SENDS"{message}" -- transmit message over serial interface

SENDP"{message}" -- transmit message over parallel interface

DISPLAY [{constant}] "{message}" -- send message to LCD display starting at specified location]

DISPLAY {constant}, {constant}.{constant}, {variable} -- send variable value to LCD using specified location and format

ENABLE PLC{constant}[,{constant}...] -- Enable operation of specified interpreted PLC program[s]

DISABLE PLC{constant}[,{constant}...] -- Disable operation of specified interpreted PLC program[s]

ENABLE PLCC{constant}[,{constant}...] -- Enable operation of specified compiled PLC program[s]

DISABLE PLCC{constant}[,{constant}...] -- Disable operation of specified compiled PLC program[s]

PLC Program Commands

Conditions

IF ({condition}) -- evaluates condition to determine which branch to enter

WHILE ({condition}) -- conditional loop start; if true, holds up operation of PLC in the **WHILE** loop

AND ({condition}) -- forms compound condition w/ **IF** or **WHILE**

OR ({condition}) -- forms compound condition w/ **IF** or **WHILE**

ELSE -- starts false branch of **IF**

ENDIF -- closes out the actions dependent on an **IF** statement; used after, not before, an **ELSE** statement.

ENDWHILE -- closes out the actions dependent on a **WHILE** statement

Actions

{variable}={expression} -- value given to variable

COMMAND "{command}" -- issue command as if from host

COMMAND^{letter} -- issue control character command

SEND "{message}" -- send message to host interface (ser. or par.)

SENDS "{message}" -- send message to serial interface

SENDP "{message}" -- send message to parallel (bus) interface

DISPLAY [{constant}] "{message}" -- display message on LCD display, starting at specified character .

DISPLAY {constant}, {constant}.{constant}, {variable} -- send variable value to LCD using specified location and format.

ENABLE PLC{constant}[,{constant}...] -- Enable operation of specified PLC program[s]

DISABLE PLC{constant}[,{constant}...] -- Disable operation of specified PLC program[s]

PMAC I-Variable Summary

I0 -- I99	General card setup global)	**I800 -- I886**	Motor #8 setup
I100 -- I186	Motor #1 setup	**I887 -- I899**	Coordinate System 8 setup
I187 -- I199	Coordinate System 1 setup	**I900 -- I979**	Encoder 1 - 16 setup (in groups of 5)
I200 -- I286	Motor #2 setup	**I980 -- I1023**	Reserved for future use
I287 -- I299	Coordinate System 2 setup		

...

Global I-Variables

I0 PMAC2 Card Number
I1 Serial Port Communications Mode
I2 Control Panel Disable
I3 I/O Handshake Mode
I4 Communications Checksum Enable
I5 PLC Programs On/Off
I6 Error Reporting Mode
I7 In-position number of cycles
I8 Real Time Interrupt Period
I9 Full/Abbreviated Program Listing Form
I10 Servo Interrupt Time
I11 Programmed Move Calculation Time
I12 Jog-to-Position Calculation Time
I13 Programmed Move Segmentation Time
I14 Auto Position Match on Run
I15 Degree/Radian Control For User Trig Functions
I16 Rotary Buffer Request On Point
I17 Rotary Buffer Request Off Point
I18 Fixed Buffer Full Warning Point
I19 Data Gathering Period (in Servo Cycles)
I20 Data Gathering Selection Mask
I21--I44 Data Gathering Source 1--24 Address
I45 Data Gathering Buffer Location and Mode
I47 Address of Pointer to **<CTRL-W>** Command
I48 DPRAM Servo Data Enable
I49 DPRAM Background Data Enable
I50 Rapid Move Velocity Mode
I51 Leadscrew Compensation Enable
I52 '\' Program Hold Slew Rate
I53 Program Step Mode Control
I54 PMAC2 Serial Baud Rate
I55 DPRAM Background Buffer Control
I56 DPRAM ASCII Communications Interrupt Enable
I57 DPRAM Binary Rotary Buffer Enable
I58 DPRAM ASCII Communications Enable
I59 DPRAM Buffer Maximum Motor/C.S. Number
I60 Auto-Converted ADC Register Address
I61 Number of Auto-Converted ADC Registers
I62 Internal Message Carriage Return Control
I8x Motor x Third-Resolver Gear Ratio
I89 Cutter Comp Outside Angle Break Point
I9x Motor x Second-Resolver Gear Ratio
I99 Backlash Hysteresis

Motor I-Variables

X = Motor Number (#x, x = 1 To 8)

Motor Definition I-Variables

Ix00 Motor x Activate

Ix01 Motor x PMAC-Commutate Enable

Ix02 Motor x Command Output (DAC) Address (PMAC[1] Specific)

Ix02 Motor x Command Output Address (PMAC2 Specific)

Ix03 Motor x Position-Loop Feedback Address

Ix04 Motor x Velocity-Loop Feed-

Ix05 Motor x Master (Handwheel) Position Address

Ix06 Motor x Master (Handwheel) Following Enable

Ix07 Motor x Master (Handwheel) Scale Factor

Ix08 Motor x Position-Loop Scale Factor

Ix09 Motor x Velocity-Loop Scale

back Address

Factor

Ix10 Motor x Power-On Servo Position Address

Motor Safety I-Variables

Ix11 Motor x Fatal (Shutdown) Following Error Limit

Ix12 Motor x Warning Following Error Limit

Ix13 Motor x Positive Software Position Limit

Ix14 Motor x Negative Software Position Limit

Ix15 Motor x Deceleration Rate on Posi- tion Limit or Abort

Ix16 Motor x Maximum Permitted Program Velocity

Ix17 Motor x Maximum Permitted Program Accel.

Ix19 Motor x Maximum Permitted Jog Accel.

Motor Movement I-Variables

Ix20 Motor x (Jogging and Homing)

Acceleration Time

Ix21 Motor x (Jogging and Homing) S- Curve Time

Ix22 Motor x Jog Speed

Ix23 Motor x Homing Speed and Direction

Ix25 Motor x Limit/Home Flag/Amp Flag Address (PMAC[1] Specific)

Ix25 Motor x Flag Address (PMAC2 Specific)

Ix26 Motor x Home Offset

Ix27 Motor x Position Rollover Range

Ix28 Motor x In-position Band

Ix29 Motor x Output - or First Phase - DAC Bias (PMAC[1] Specific)

Ix29 Motor x Output/First Phase Offset (PMAC2 Specific)

Motor Basic Servo Control I-Variables

Ix30 Motor x PID Proportional Gain

Ix31 Motor x PID Derivative Gain

Ix32 Motor x PID Velocity Feedforward Gain

Ix36 Motor x Notch Filter Coefficient N1

Ix37 Motor x Notch Filter Coefficient N2

Ix38 Motor x Notch Filter Coef-

Ix33 Motor x PID Integral Gain

Ix34 Motor x PID Integration Mode

Ix35 Motor x PID Acceleration Feedfor- ward Gain

ficient D1

Ix39 Motor x Notch Filter Coefficient D2

Ix40 - Ix56 Motor x Extended Servo Algorithm I-variables

Motor Servo Loop Modifications

Ix57 Motor x Continuous Current Limit

Ix58 Integrated Current Fault Level (PMAC[1] Specific)

Ix58 Integrated Current Limit (PMAC2 Specific)

Ix59 Motor x User-Written Servo Enable

Ix60 Motor x Servo Cycle Extension Pe- riod

Ix61 Motor x Current Loop Integral Gain (PMAC2 Only)

Ix62 Motor x Current Loop Proportional Gain (Forward Path) (PMAC2 Only)

Ix63 Motor x Integration Limit

Ix64 Motor x "Deadband Gain Factor"

Ix65 Motor x Deadband

Ix66 Motor x PWM Scale Factor (PMAC2 Only)

Ix67 Motor x Linear Position Error ("Big Step") Limit

Ix68 Motor x Friction Feedforward

Ix69 Motor x Output Command (DAC) Limit

Motor Commutation I-Variables

Ix70 Motor x Number of Commutation Cycle (N) for Cycle Size Definition

Ix71 Motor x Encoder Counts per N Commu- tation Cycles

Ix72 Motor x Commutation Phase Angle

Ix73 Motor x Phase Finding Output (DAC) Value

Ix74 Motor x Phase Finding Time

Ix75 Motor x Phasing Offset

Ix76 Motor x Velocity Phase Advance Gain (PMAC[1] Specific)

Ix76 Motor x Current Loop Proportional Gain (Back Path) (PMAC2 Specific)

Ix77 Motor x Induction Motor Magnetiza-tion Current

Ix78 Motor x Induction Motor Slip Gain

Ix79 Motor x Second Phase Output(DAC) Bias (PMAC[1] Specific)

Ix79 Motor x Second Phase Offset (PMAC2 Specific)

Ix80 Motor x Power-Up Mode

Ix81 Motor x Power-On Phase Position Address

Ix82 Current Loop Feedback Address (PMAC2 Only)

Ix83 Motor x Ongoing Phasing Position Address (PMAC[1] Specific)

Ix83 Motor x Commutation Position Address (PMAC2 Specific)

Ix84 Current Loop Feedback Mask Word (PMAC2 Only)

Further Motor I-Variables

Ix85 Motor x Backlash Take-up Rate

Ix86 Motor x Backlash Size

Coordinate System I-Variables



x = Coordinate System Number (&x, x = 1 to 8)

Ix87 Coordinate System x Default Program Acceleration Time

Ix88 Coordinate System x Default Program S-Curve Time

Ix89 Coordinate System x Default Program Feedrate

Ix90 Coordinate System x Feedrate Time Units

Ix91 Coordinate System x Default Working Program Number

Ix92 Coordinate System x Move Blend Dis-able

Ix93 Coordinate System x Time Base Con- trol Register Address

Ix94 Coordinate System x Time Base Slew Rate (and Limit)

Ix95 Coordinate System x Feed Hold Decel- eration Rate

Ix96 Coordinate System x Maximum "Circle" Error Limit

Encoder/Flag Setup I-Variables

For Encoder n (n = 1 -> 16)

```
I900 - I904  -- Encoder 1                  I901, I906,  -- (Encoder I-Variable 1)  ...
I905 - I909  -- Encoder 2                  I911, I916,
I910 - I914  -- Encoder 3                  I921, I926,
I915 - I919  -- Encoder 4                  I931, I936,
   ...                                     I941, I946,
I970 - I974  -- Encoder 15                 I951, I956,
I975 - I979  -- Encoder 16                 I961, I966,
I900, I905, -- (Encoder I-Variable 0)   ...I971, I976
I910, I915,                                I902, I907,  -- (Encoder I-Variable 2)  ...
I920, I925,                                I912, I917,
I930, I935,                                I922, I927,
I940, I945,                                I932, I937,
I950, I955,                                I942, I947,
I960, I965,                                I952, I957,
I962, I967,                                I933, I938,
I972, I977                                 I943, I948,
I903, I908, -- (Encoder I-Variable 3)  .....I953, I958,
I913, I918,                                I963, I968,
I923, I928,                                I973, I978
I970, I975
```

Macro Support I-Variables

I1000 MACRO Node Auxiliary Register En- able

I1001 MACRO Ring Check Time Period

PMAC Error Code Summary



Variable I6 controls whether and how these error messages are sent

PMAC can report the following error messages in response to commands:

ERROR	PROBLEM	SOLUTION
ERR001	Command not allowed during program execution	(should halt program execution before issuing command)
ERR002	Password error	(should enter the proper password)
ERR003	Data error or unrecognized command	(should correct syntax of command)
ERR004	Illegal character: bad value (>127 ASCII) or serial parity/framing error	(should correct the character and or check for noise on the serial cable)
ERR005	Command not allowed unless buffer is open	(should open a buffer first)
ERR006	No room in buffer for command	(should allow more room for buffer -- DELETE or CLEAR other buffers)
ERR007	Buffer already in use	(should CLOSE currently open buffer first)
ERR009	Program structural error (e.g. ENDIF without IF)	(should correct structure of program)
ERR010	Both overtravel limits set for a motor in the CS	(should correct or disable limits)
ERR011	Previous move not completed	(should Abort it or allow it to complete)
ERR012	A motor in the coordinate system is open-loop	(should close the loop on the motor)
ERR013	A motor in the coordinate system is not activated	(should set Ix00 to 1 or remove motor from CS)
ERR014	No motors in the coordinate system	(should define at least one motor in CS)
ERR015	Not pointing to valid program buffer	(should use B command first, or clear out scrambled buffers)
ERR016	Running improperly structured program (e.g. missing ENDWHILE)	(should correct structure of program)
ERR017	Trying to resume after / or \ with motors out of stopped position	(should use J= to return motor[s] to stopped position)

PMAC Syntax Notes

0: PMAC syntax is not case-sensitive. That is, it does not matter whether an upper case or lower-case letter is used in any command or statement. PMAC commands are shown in this document in all upper-case letters to help distinguish them better from the explanatory text.

```
Example: X1000 and x1000 are identical statements to PMAC.
```

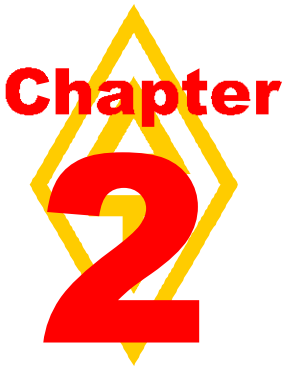
1: In syntax definitions, an item in squiggly brackets, such as **{data}**, means the user has the flexibility to put what he wishes into that part of the syntax, subject to the defined limitations of that item.

Example: If the syntax is **X{data}**, the user can put **X1000**, **X(P1)**, or **X(P2*P3+50)**, because **1000**, **(P1)**, and **(P2*P3+50)** all fit in the defined limitations for **{data}**.

2: In syntax definitions, items contained within square brackets are optional to the syntax. If there is an ellipsis (...) within the square brackets, items contained within the square brackets can be repeated.

Example: If the syntax definition is **{axis}{data} [{axis}{data} ...]**, the user can put **X1000**, X1000Y1000, or X1000Y1000Z1000.

3: Spaces are not important in PMAC syntax, except where specifically noted.



PMAC I-Variable Specification

I-Variable Functions

On PMAC, I-variables (Initialization, or Set-up, Variables) determine the "personality" of the controller for a given application. They are at fixed locations in memory and have pre-defined meanings. Most are integer values, and their range varies depending on the particular variable. There are 1024 I- variables, from I0 to I1023, and they are organized as follows:

I-VARIABLE	FUNCTION
I0 -- I75	General card setup (global)
I76 -- I99	Dual-speed resolver setup
I100 -- I186	Motor #1 setup
I187 -- I199	Coordinate System 1 setup
I200 -- I286	Motor #2 setup
I287 -- I299	Coordinate System 2 setup
I800 -- I886	Motor #8 setup
I887 -- I899	Coordinate System 8 setup
I900 -- I979	Encoder 1 - 16 setup (in groups of 5)
I980 -- I1023	Reserved for future use

Global I-Variables

I0 PMAC2 Card Number (PMAC2 Only)

Range: ..... $0 to $F (0 to 15)

Default: ..... **$**0

Units: ..... none

Remarks



RS-422 serial interfaces must be used on all PMAC2 boards for multi-drop serial communications; this will not work with RS-232 interfaces. If the RS-422 interface is not present as a standard feature on the PMAC2 board, the Option 9L serial converter module must be purchased. It is possible to use an RS-232 interface on the host computer, connected to the RS-422 ports on the PMAC2 boards.



Any board with jumper E1 ON is expecting its SERVO and PHASE clock signals externally from a Card 0. If it does not receive these clock signals, the watchdog timer will immediately shut down the board and the red LED will light.

I0 controls the PMAC2 card number for software addressing purposes on a multi-drop serial communications cable. If I2 is set to 2, the PMAC2 must be addressed with the @n command, where n matches the value of I0 on the board, before it will respond. If the PMAC2 receives the @n command, where n does not match I0 on the board, it will stop responding to commands on the serial port. No two boards on the same serial cable may have the same value of I0.

If the @@ command is sent over the serial port, all boards on the cable will respond to action commands. However, only the board with I0 set to 0 will respond to the host with handshake characters and/or data responses. All boards on the cable will respond to control-character action commands such as <CTRL-R>, regardless of the current addressing.

Typically multiple PMAC2 boards on the same serial cable will share servo and phase clock signals over the serial port cable for tight synchronization. If the servo and phase clock lines are connected between multiple PMAC2s, only one of the PMAC2 boards can be set up to output these clocks (E1 jumper OFF). All of the other boards in the chain must be set up to input these clocks (E1 jumper ON).

If the PMAC2 has E1 ON to receive external SERVO and PHASE clock signals for synchronization purposes, but is not using multi-drop serial communications, I0 does not need to be changed from 0.

To set up a board to communicate as Card 1 to Card 15 on a multi-drop serial cable, first communicate with the board as Card 0. Set I0 to specify the card number (software address) that the board will have on the multi-drop cable. Also, set I1 to 2 to enable the serial software addressing. Store these values to the non-volatile flash memory with the **SAVE** command. Then turn off power; if the board is to input its clocks, put a jumper on E1; connect the multi-drop cable; restore power to the system.

I1 Serial Port Mode

Range: ..... 0 .. 3
Default: 0
Units: ..... none

Remarks

This parameter controls two aspects of how PMAC uses its serial port. The first aspect is whether PMAC uses the CS (CTS) handshake line to decide if it can send a character out the serial port. The second aspect is whether PMAC will require software card addressing, permitting multiple cards to be daisychained on a single serial line. There are four possible values of I1, covering all the possible combinations:

SETTING	MEANING
0	CS handshake used; no software card address required
1	CS handshake not used; no software card address required
2	CS handshake used; software card address required
3	CS handshake not used; software card address required

When CS handshaking is used (I1 is 0 or 2), PMAC waits for CS to go true before it will send a character. This is the normal setting for real serial communications to a host; it allows the host to hold off PMAC messages until it is ready.
When CS handshaking is not used (I1 is 1 or 3), PMAC disregards the state of the CS input and always sends the character immediately. This mode permits PMAC to "output" messages, values, and acknowledgments over the serial port even when there is nothing connected, which can be valuable in stand-alone and PLC-based applications where there are **SEND** and **CMD** statements in the program. If these strings cannot be sent out the serial port, they can "back up", stopping program execution.
When software addressing is not used (I1 is 0 or 1), PMAC assumes that it is the only card on the serial line, so it always acts on received commands, sending responses back over the line as appropriate.
When software addressing is used (I1 is 2 or 3), PMAC assumes that there are other cards on the line, so it requires that it be addressed (with the `@{card}` command) before it responds to commands. The `{card}` number in the command must match the card number set up in hardware on the card with jumpers or DIP switches.

See Also

Serial Port, Multiple-Card Applications (Talking to PMAC)
I-variable I6
Program Commands **SEND**, **CMD**
Connectors J4 (PMAC-PC, -Lite, -VME), J1, J3 (PMAC-STD)
Jumpers E40-E43 (PMAC-PC, -Lite, -VME)
DIP-switches SW1-1 -- SW1-4 (PMAC-STD)

I2 Control Panel Disable

Range: ..... 0 .. 3

Default: 0

Units: ..... none

Remarks

This parameter allows the enabling and disabling of the control panel discrete inputs (on the JPAN connector). I2=0 enables these control panel functions; I2=1 disables them. When disabled, these inputs can be used as general purpose I/O. The reset, handwheel, and wiper inputs on the JPAN connector are not affected by I2.
When I2=0, the IPOS, EROR and F1ER status lines to JPAN and the Programmable Interrupt Controller (PIC), and the BREQ status line to the PIC, reflect the hardware-selected coordinate system (by BCD-coded lines FPDn/ on JPAN); when I2=1, they reflect the software-addressed coordinate system (&n).
When I2=3, the discrete inputs on the JPAN connector are disabled, and the dual-ported RAM control panel functions are enabled. Refer to the descriptions of DPRAM functions for more detail.

See Also

Using Interrupts (Writing a Host Communications Program)

I-variables I16-I18

Custom Inputs Example `(JOGSWTCH.PMC)`

Connector JPAN (J2)

DPRAM Control Panel Functions

I3 I/O Handshake Control

Range: .....

Default: .....

Units: .....

With checksum enabled (I4=1), checksum bytes are added after the handshake character bytes.

When I58=1 to enable DPRAM ASCII communications, I3 is forced to 0 or 2 from 1 or 3, respectively.

Remarks

This parameter controls what characters, if any, are used by PMAC to delimit a transmitted line, and whether PMAC issues an acknowledgment (handshake) of a command.
Valid values of I3 and the modes they represent are:

0: PMAC does not acknowledge receipt of a valid command. It returns a **<BELL>** character on receipt of an invalid command. Messages are sent without beginning or terminating **<LF>** (line feed); simply as **DATA <CR>** (carriage return).

1: PMAC acknowledges receipt of a valid **<CR>**-terminated command with a **<LF>**; of an invalid command with a **<BELL>** charact **Ix05**.....Motor x Master (Handwheel) Position Address er. Messages are sent as **<LF> DATA <CR> [<LF> DATA <CR> ...] <LF>**. (The final **<LF>** is the acknowledgment of the host command; it is not sent with a message initiated from a PMAC program [**SEND** or **CMD**]). This setting is good for communicating with terminal display programs, such as the PMAC Executive program.

2: PMAC acknowledges receipt of a valid **<CR>**-terminated command with an **<ACK>**; of an invalid command with a **<BELL>** character. Messages are sent as **DATA <CR> [DATA <CR> ...] <ACK>**. (The final **<ACK>** is the acknowledgment of the host command; it is not sent with a message initiated from a PMAC program [**SEND** or **CMD**]). This is probably the best setting for fast communications with a host program without terminal display.

3: PMAC acknowledges receipt of a valid **<CR>**-terminated command with an **<ACK>**; of an invalid command with a **<BELL>** character. Messages are sent as **<LF> DATA <CR> [<LF> DATA <CR> ...] <ACK>**. (The final **<ACK>** is the acknowledgment of the host command; it is not sent with a message initiated from a PMAC program [**SEND** or **CMD**]).

Example

With I3=0:

```
#1J+<CR>     ; Valid command not requiring data response

   ; No acknowledging character

UUU<CR> ; Invalid command

<BELL>  ; PMAC reports error

P1..3<CR>    ; Valid command requiring data response

25<CR>50<CR>75<CR>          ; PMAC responds with requested data
```

With I3=1:

```
#1J+<CR>     ; Valid command not requiring data response

<LF>    ; Acknowledging character

UUU<CR> ; Invalid command

<BELL>  ; PMAC reports error

P1..3<CR>    ; Valid command requiring data response

<LF>25<CR><LF>50<CR><LF>75<CR><LF>    ; PMAC responds with re-
quested data
```

With I3=2:

```
#1J+<CR>      ; Valid command not requiring data response

<ACK>   ; Acknowledging character

UUU<CR> ; Invalid command

<BELL>  ; PMAC reports error

P1..3<CR>     ; Valid command requiring data response

25<CR>50<CR>75<CR><ACK>      ; PMAC responds with requested data
```

With I3=3:

```
#1J+<CR>      ; Valid command not requiring data response

<ACK>   ; Acknowledging character

UUU<CR> ; Invalid command

<BELL>  ; PMAC reports error

P1..3<CR>     ; Valid command requiring data response

<LF>25<CR><LF>50<CR><LF>75<CR><ACK>   ; PMAC responds with re-
quested data
```

See Also

Talking to PMAC
Writing a Host Communications Program
I-variables I4, I6, I58

I4 Communications Integrity Mode

The on-line command ***<CTRL-N>*** *can be used to verify the checksum of a command line before the* ***<CR>*** *has been sent. The use of* ***<CTRL-N>*** *does not affect how I4 causes PMAC to report a checksum after the* ***<CR>*** *has been sent.*

Range: .....

Default: .....

Units: .....

Remarks

This parameter permits PMAC to compute checksums of the communications bytes (characters) sent either way between the host and PMAC, and also controls how PMAC reacts to serial character errors (parity and framing), if found. Parity checking is only enabled if jumper E49 is OFF for PMAC-PC, -Lite, -VME; or ON for PMAC-STD. The possible settings of I4 are:

SETTING	MEANING

0	Checksum disabled, serial errors reported immediately
1	Checksum enabled, serial errors reported immediately
2	Checksum disabled, serial errors reported at end of line
3	Checksum enabled, serial errors reported at end of line

Communications Checksum

With I4=1 or 3, PMAC computes the checksum for communications in either direction and sends the checksum to the host. It is up to the host to do the comparison between the PMAC checksum and the checksum it computed itself. PMAC does not do this comparison. The host should never send a checksum byte to PMAC.

Host-to-PMAC Checksum

PMAC will compute the checksum of a communications line sent from the host to PMAC. The checksum does not include any control characters sent (not even the final Carriage-Return). The checksum is sent to the host immediately following the acknowledging handshake character (<LF> or <ACK>), if any. Notice that this acknowledging and handshake comes after any data response to the command (and its checksum!). If PMAC detects an error in the line through its normal syntax checking, it will respond with the <BELL> character, but will not follow this with a checksum byte.

PMAC-to-Host Checksum

PMAC will compute the checksum of any communications line it sends to the host. This checksum includes control characters sent with the line, including the final **<carriage-return>**. The checksum is sent immediately following this **<carriage-return>**. On a multiple-line response, one checksum is sent for each line. Notice that this checksum is sent before the checksum of the command line that caused the response. For more details on checksum, refer to the Writing a Host Communications Program section of the manual.

Serial character errors

If PMAC detects a serial character error, it will set a flag so that the entire command line will be rejected as having a syntax error after the **<CR>** is sent. With I4=0 or 1, it will also send a **<BELL>** character to the host immediately on detecting the character error. Notice that this mode will catch a character error on the **<CR>** as well, whereas in the I4=2 or 3 mode, the host would have to catch an error on the **<CR>** character by the fact that PMAC would not respond (because it never saw a **<CR>**).

See Also

Communications Checksum (Writing a Host Communications Program)

I-variables I3, I6

On-line command **<CTRL-N>**

Jumper E49

I5 PLC Programs On/Off

Range: ..... 0 .. 3
Default: 0
Units: ..... none

Remarks

This parameter controls which PLC programs may be enabled. There are two types of PLC programs: the foreground program (PLC 0), which operates at the end of servo interrupt calculations, with a repetition rate determined by I8 (PLC 0 should be used only for time-critical tasks and should be short); and the background programs (PLC 1 to PLC 31) which cycle repeatedly in background as time allows. I5 controls these as follows:

SETTING	MEANING
0	Foreground PLC off; background PLC off
1	Foreground PLC on; background PLC off
2	Foreground PLC off; background PLC on
3	Foreground PLC on; background PLC on

Notice that an individual PLC program still needs to be enabled to run -- a proper value of I5 merely permits it to be run. Any PLC program that exists at power-up or reset is automatically enabled (even if the saved value of I5 does not permit it to run immediately); also, the **ENABLE PLC n** command enables the specified program(s). A PLC program is disabled either by the **DISABLE PLC n** command, or by the **OPEN PLC n** command. A **CLOSE** command does not automatically re-enable the PLC program -- it must be done explicitly.

See Also

Running PLC Programs (Writing a PLC Program)

On line commands ENABLE PLC n, DISABLE PLC n, OPEN PLC n, CLOSE, <CTRL-D>, $$$.

I6 Error Reporting Mode

Range: ..... 0 .. 3
Default: 3
Units: ..... none

Remarks

This parameter reports how PMAC reports errors in command lines. When I6 is set to 0 or 2, PMAC reports any error only with a **<BELL>** character. When I6 is 0, the **<BELL>** character is given for invalid commands issued both from the host and from PMAC programs (using **CMD"{command}"**).

When I6 is 2, the **<BELL>** character is given only for invalid commands from the host; there is no response to invalid commands issued from PMAC programs. (In no mode is there a response to valid commands issued from PMAC programs.

When I6 is set to 1 or 3, an error number message can be reported along with the **<BELL>** character. The message comes in the form of **ERRnnn<CR>**, where **nnn** represents the three-digit error number. If I3 is set to 1 or 3, there is a **<LF>** character in front of the message.

When I6 is set to 1, the form of the error message is **<BELL>{error message}**. This setting is the best for interfacing with host-computer driver routines. When I6 is set to 3, the form of the error message is

<BELL><CR>{error message}. This setting is appropriate for use with the PMAC Executive Program in terminal mode.
Currently, the following error messages can be reported:

ERROR	PROBLEM	SOLUTION
ERR001	Command not allowed during program execution	(should halt program execution before issuing command)
ERR002	Password error	(should enter the proper password)
ERR003	Data error or unrecognized command	(should correct syntax of command)
ERR004	Illegal character: bad value (>127 ASCII) or serial parity/framing error	(should correct the character and or check for noise on the serial cable)
ERR005	Command not allowed unless buffer is open	(should open a buffer first)
ERR006	No room in buffer for command	(should allow more room for buffer -- DELETE or CLEAR other buffers)
ERR007	Buffer already in use	(should CLOSE currently open buffer first)
ERR009	Program structural error (e.g. ENDIF without IF)	(should correct structure of program)
ERR010	Both overtravel limits set for a motor in the CS	(should correct or disable limits)
ERR011	Previous move not completed	(should Abort it or allow it to complete)
ERR012	A motor in the coordinate system is open-loop	(should close the loop on the motor)
ERR013	A motor in the coordinate system is not activated	(should set Ix00 to 1 or remove motor from CS)
ERR014	No motors in the coordinate system	(should define at least one motor in CS)
ERR015	Not pointing to valid program buffer	(should use B command first, or clear out scrambled buffers)
ERR016	Running improperly structured program (e.g. missing ENDWHILE)	(should correct structure of program)
ERR017	Motor(s) in CS not at halted position to restart after / or \ command	(should move motor(s) back to halted position with J=)

See Also

Talking to PMAC
Writing a Host Communications Program
I-variables I3, I4
On-line commands R, S

I7 In-Position Number of Cycles

Range: ..... 0 .. 255
Default: 0
Units: ..... Background computation cycles (minus one)

Remarks

This parameter permits the user to define the number of consecutive scans that PMAC motors must satisfy all "in-position" conditions before the motor in-position bit is set true. This permits the user to ensure that the motor is truly settled in the end position before executing the next operation, on or off PMAC. The number of consecutive scans required is equal to I7 + 1.

PMAC scans for the in-position condition of each active motor during the "housekeeping" part of every background cycle, which occurs between each scan of each enabled uncompiled background PLC (PLC 1-31). All motors in a coordinate system must have true in-position bits for the coordinate-system in-position bit to be set true.

See Also

Control Panel Port (Connecting PMAC to the Machine)

Using Interrupts (Writing a Host Communications Program)

I-variable Ix28

On-line commands **?**,**??**

Suggested M-variable definitions Mx40

Memory registers Y:$0814, Y:$08D4, etc., Y:$0817, Y:$08D7, etc.

DPRAM Control Panel Functions

JPAN connector

I8 Real Time Interrupt Period

Range: ..... 0 .. 255
Default: 2
Units: ..... Servo Interrupt Cycles

Remarks



A large PLC 0 with a small value of I8 can cause severe problems, because PMAC will attempt to execute the PLC program every I8 cycle. This can starve background tasks, including communications, background PLCs, and even updating of the watchdog timer, for time, leading to erratic performance or possibly even shutdown.

This parameter controls how often certain time-critical tasks, such as PLC 0 and checking for motion program move planning, are performed. A value of 2 means that they are performed after every third servo interrupt, 3 means every fourth interrupt, and so on. The vast majority of users can leave this at the default value. In some advanced applications that push the PMAC speed capabilities, tradeoffs between performance of these tasks and the calculation time they take may have to be evaluated in setting this parameter.

In multiple-card PMAC applications where it is very important that motion programs on the two cards start as closely together as possible, I8 should be set to 0. In this case, no PLC 0 should be running when the cards are awaiting a Run command. At other times I8 may be set greater than 0 and PLC 0 re-enabled.

See Also

How PMAC Executes a Motion Program (Writing a Motion Program)

PLC 0 (Writing a PLC Program)

I9 Full/Abbreviated Program Listing Form

Range: ..... 0 .. 3
Default: 2
Units: ..... none

Remarks

SETTING	MEANING
0	Short form, decimal address I-variable return
1	Long form, decimal address I-variable return
2	Short form, hex address I-variable return
3	Long form, hex address I-variable return

When this parameter is 0 or 2, programs are sent back in abbreviated form for maximum compactness, and when I-variable values or M-variable definitions are requested, only the values or definitions are returned, not the full statements. When this parameter is 1 or 3, programs are sent back in full form for maximum readability. In addition, I-variable values and M-variable definitions are returned as full command statements, which is useful for archiving and later downloading.
When this parameter is 0 or 1, I-variable values that specify PMAC addresses are returned in decimal form. When it is 2 or 3, these values are returned in hexadecimal form (with the '$' prefix). You are always free to send any I-variable values to PMAC either in hex or decimal, regardless of the I9 setting. This does not affect how I-variable assignment statements inside PMAC motion and PLC programs are reported when the program is listed.

Example

With I9=0:

```
I125    ; Request address I-variable value
49152   ; PMAC reports just value, in decimal
M101->  ; Request M-variable definition
X:$C001,24,S ; PMAC reports just definition
LIST PROG 1  ; Request listing of program
LIN     ; PMAC reports program short form
X10
DWE1000
RET
```

With I9=1:

```
I125    ; Request address I-variable value
I125=49152   ; PMAC reports whole statement, in decimal
M101->  ; Request M-variable definition
M101->X:$C001,24,S          ; PMAC reports whole statement
LIST PROG 1  ; Request listing of program
LINEAR  ; PMAC reports program long form
```

```
X10

DWELL1000

RETURN
```

With I9=2:

```
I125    ; Request address I-variable value

$C000   ; PMAC reports just value, in hexadecimal
```

With I9=3:

```
I125    ; Request address I-variable value

I125=$C000   ; PMAC reports whole statement, in hexadecimal
```

See Also

Talking to PMAC
On-line commands I{constant}, M{constant}->, LIST
I-Variables I19-I44, I47, Ix02-Ix05, Ix25, Ix83, Ix93

I10 Servo Interrupt Time (PMAC[1] Specific)

Range: .....
Default:.....
Units: .....

Even if Ix60 (servo cycle extension) has been changed from its default value of 0 for any or all motors, the value of I10 should reflect the time between servo interrupts*, not between consecutive servo cycle calculations*

Remarks

This parameter tells the PMAC how much time there is between servo interrupts (which is controlled by hardware), so that the software knows how much time to increment each servo interrupt. Although this parameter can be changed at any time and immediately effects how the present feedrate override value (**%** command) is reported, the new value will only take effect at the next power-up, reset, or **%{constant}** command.
The fundamental equation for I10 is:

$$I10 = \frac{8{,}388{,}608}{ServoFrequency(kHz)} = 8{,}388{,}608 * ServoTime(m\,\text{sec})$$

The value of I10 for a given hardware jumper configuration can be determined by the following formula:

$$I10 = \frac{8{,}388{,}608 * N}{PhaseFrequency(kHz)}$$

where the phase frequency is determined by the jumpers E29-E33 (only one of which may be ON at once) and the crystal clock frequency (not necessarily the CPU clock frequency:

		19.6608 MHZ MASTER CLOCK	29.4912 MHZ MASTER CLOCK (OPT 5 ONLY)
Phase Frequency:	**E29 ON:**	2.258824 kHz	3.388235 kHz
	E30 ON:	4.517647 kHz	6.776471 kHz
	E31 ON:	9.035294 kHz	13.552941 kHz
	E32 ON:	18.070588 kHz	27.105882 kHz
	E33 ON:	36.141176 kHz	54.211765 kHz

and N is determined by the jumpers E3-E6 (it is the number of times the phase clock is divided down to provide the servo clock) using the table in the Jumper Description section or the following equation:

$$N = 1 + E3 + 2 \cdot E4 + 4 \cdot E5 + 8 \cdot E6$$

in which En = 0 if the jumper is ON, and En = 1 if the jumper is OFF.

The actual frequencies of the servo and phase clocks can be checked on the SERVO and PHASE pins of the serial connector, using an oscilloscope.

I10 is used to provide the delta-time value in the position update calculations, scaled such that 2^{23} -- 8,388,608 -- means one millisecond. Delta-time in these equations is I10 * (%value/100). The % (feedrate override) value can be controlled in any of several ways: with the on-line '%' command, with a direct write to the command '%' register, with an analog voltage input, or with a digital input frequency. The default % value is 100, and many applications can always leave it at 100.

Example

If Jumper E98 has been changed from its default setting connecting pins 1-2 to instead connect pins 2-3 (this would be done to provide a slower conversion clock for better analog-to-digital conversion from ACC-23 or ACC-28), all phase frequencies -- and therefore servo frequencies -- will be exactly one-half of that listed in the above table.

Some commonly used examples follow -- the first one being the default configuration as shipped (these all assume a master clock rate of 19.6608 MHz; the optional 29.4912 MHz clock provides a period 2/3 as long as the standard for the same jumper settings):

PHASE CLOCK JUMPER	PHASE RATE (KHZ)	E3	E4	E5	E6	N	SERVO PERIOD (μSEC)	PROPER I10 VALUE
E31	9.04	OFF	OFF	ON	ON	4	442.7	3,713,707
E31	9.04	ON	ON	ON	ON	1	110.7	928,427
E30	4.52	OFF	OFF	ON	ON	4	885.4	7,427,413

See Also

Setting the Servo Update Time (Servo Features)

Jumpers E3-E6, E29-E33, E98

Connector J4 Pins 21-24 (PMAC-PC, -VME), J4 Pins 1 & 8 (PMAC-Lite), J3 Pins 5-8 (PMAC-STD).

I10 Servo Interrupt Time (PMAC2 Specific)

Range: .....0 to 8,388,607

Units: ..... 1 / 8,388,608 msec

Default: ..... 3,713,707 (=442 μsec)

Remarks

This parameter tells PMAC2 how much time there is between servo interrupts (which is controlled by hardware circuitry), so that the software knows how much time to increment each servo interrupt.

On PMAC2, the servo interrupt time is determined by hardware control I-variables I900, I901, and I902; or on PMAC2 Ultralite boards, by I992, I997, and I998. The proper setting of I10 can be determined from these variables by the formula:

$$I10 = \frac{640}{9}(2 * I900 + 3)(I901 + 1)(I902 + 1)$$

or on the PMAC2 Ultralite boards:

$$I10 = \frac{640}{9}(2 * I992 + 3)(I997 + 1)(I998 + 1)$$

I10 is used to provide the delta-time value in the position update calculations, scaled such that 2^{23} -- 8,388,608 -- means one millisecond. Delta-time in these equations is I10*(%value/100). The % (feedrate override) value can be

controlled in any of several ways: with the on-line '%' command, with a direct write to the command '%' register, with an analog voltage input, or with a digital input frequency. The default % value is 100, and many applications can always leave it at 100.

Example



If Jumper E98 has been changed from its default setting connecting pins 1-2 to instead connect pins 2-3 (this would be done to provide a slower conversion clock for better analog-to-digital conversion from ACC-23 or ACC-28), all phase frequencies -- and therefore servo frequencies -- will be exactly one-half of that listed in the above table.

Some commonly used examples follow -- the first one being the default configuration as shipped (these all assume a master clock rate of 19.6608 MHz; the optional 29.4912 MHz clock provides a period 2/3 as long as the standard for the same jumper settings):

PHASE CLOCK JUMPER	PHASE RATE (KHZ)	E3	E4	E5	E6	N	SERVO PERIOD (μSEC)	PROPER I10 VALUE
E31	9.04	OFF	OFF	ON	ON	4	442.7	3,713,707
E31	9.04	ON	ON	ON	ON	1	110.7	928,427
E30	4.52	OFF	OFF	ON	ON	4	885.4	7,427,413

See Also

Setting the Servo Update Time (Servo Features)

Jumpers E3-E6, E29-E33, E98

Connector J4 Pins 21-24 (PMAC-PC, -VME), J4 Pins 1 & 8 (PMAC-Lite), J3 Pins 5-8 (PMAC-STD).

I11 Programmed Move Calculation Time

If I11 is greater than zero, defining a definite time for calculations, and PMAC cannot complete the calculations for the first move of a sequence by the end of the I11 time, PMAC will terminate the running of the program with a run-time error.

Range: .....
Default:.....
Units: .....

Remarks

This parameter controls the delay from when the run signal is taken (or the move sent if executing immediately) and when the first programmed move starts. If several PMACs need to be run synchronously, I11 should be set the same on all of the cards. If I11 is set to zero, the first programmed move starts as soon as the calculation is complete.
This calculation time delay is also used after any break in the continuous motion of a motion program : a **DWELL**, a **PSET**, a **WAIT**, or each move if Ix92=1 (a **DELAY** is technically a zero-distance move, and so does not constitute a break).
The actual delay time varies with the time base (e.g. at a % value of 50, the actual delay time will be twice the number defined here), which keeps it as a fixed *distance* of the master in an external time base application. If it is desired to have the slave coordinate system start up immediately with the master, I11 should be set to zero, and the program commanded to run *before* the master starts to move.

See Also

External Time Base (Synchronizing PMAC to External Events)

I-variables I12, I13

Program commands **DWELL**, **DELAY**.

I12 Jog-to-Position Calculation Time

Range: .....1 .. 8,388,607
Default:.....10
Units: .....
..... msec

Remarks

This parameter controls how much time is allotted to calculate any jog move to a final position (J=, J={constant}, J/, J^{constant}, J:{constant}, or a motion program RAPID move). If a motor is currently jogging, it will continue on its present course during that time. If it is currently sitting still, it will continue to sit for this time.
This parameter should rarely need to be changed from the default. It should not be set to 0 for any reason, or PMAC will not be able to perform any of these types of moves. The minimum practical value for this parameter is 2 or 3.

See Also

I-variables I11, I13

Program command RAPID

On-line commands J=, J={constant}, J/, J^{constant}, J:{constant}

I13 Programmed Move Segmentation Time



When I13=0, moves are done without this ongoing spline technique, and ***CIRCLE*** *mode moves are done as* ***LINEAR*** *mode moves.*

Range: .....
Default: .....
Units: .....

Remarks

When greater than zero, this parameter puts PMAC into a mode ("segmentation mode") where all **LINEAR** and **CIRCLE** moves are done as a continuous cubic spline in which the move segments are of the time length specified by the parameter in this variable (this is not the same thing as **SPLINE** mode moves). This mode is required for applications using **CIRCLE** mode moves.

Segmentation mode (I13 greater than 0) is required to support any of the following PMAC features:

Circular interpolation

Cutter radius compensation

/ Program stop command

**** Program hold command

Rotary buffer blend on-the-fly

If none of these features is required, it is usually best to leave I13 at 0.

Typical values of I13 for segmentation mode are 5 to 10 msec. The smaller the value, the tighter the fit to the true curve, but the more computation is required for the moves, and the less is available for background tasks. If I13 is set too low, PMAC will not be able to do all of its move calculations in the time allotted, and it will stop the motion program with a run-time error.

See Also

Circular Interpolation, Cutter Radius Compensation (Writing a Motion Program)

On-line commands **/**, ****

Program commands **{axis}{data}{vector}{data}, CIRCLE1**, **CIRCLE2, CC0, CC1, CC2**

I14 Auto Position Match on Run Enable

Range: ..... 0 .. 1
Default: 1
Units: .....
..... none

Remarks

This parameter, when set to 1, performs an automatic re-matching of motor and axis *starting* position registers to current motor *commanded* positions whenever a motion program is started. A mismatch can occur whenever a mo-

tor move (jog, open-loop, abort, or limit) changes the motor's target position without letting the axis position "know" of the change, or on power-up when an absolute position sensor starts up with a position other than zero.
With I14=1, PMAC will execute the **PMATCH** function on any Run or Step command to make sure that the axes in the motion program have the proper starting-position information. The only users who would not want this function, setting I14 to 0, are those who cannot afford the extra millisecond (approximately) of calculation time this requires.
With I14=0, PMAC uses the last *motion program target position* as the starting point for the calculations of the next move, even if these do not match the positions currently commanded for the motors assigned to the axes.

See Also

Axis-Motor Position Re-Matching (Setting Up a Coordinate System)

On-line command PMATCH

Suggested M-variable definitions Mx61, Mx63, Mx64 Mx65

I15 Degree/Radian Control for User Trig Functions

Range: ..... 0 .. 1
Default: ..... 0 (degrees)
Units: ..... none

Remarks

This parameter controls whether the angle values for trigonometric functions in user programs (motion and PLC) and on-line commands are expressed in degrees (I15=0) or radians (I15=1).

See Also

SIN, **COS**, **TAN**, **ASIN**, **ACOS**, **ATAN**, **ATAN2** (Computational Features)

I16 Rotary Buffer Request On Point



The BREQ line to the interrupt controller reflects the status of the hardware-selected coordinate system (by JPAN pins FPDn/) if the control-panel inputs are enabled (I2=0); it represents the status of the software-host-addressed coordinate system if the control-panel inputs are disabled (I2=1). In virtually all applications using this feature, the user will want to set I2 to 1 so the BREQ line reflects the status of the coordinate system to which he is currently talking.

Range: .....
Default: .....
Units: .....

Remarks

This parameter controls the point at which an executing rotary program will signal that it is ready to take more command lines (BREQ line taken high, coordinate system *Rotary Buffer Full* status bit -- Y:$0817 bit 11 -- taken low). This occurs when the executing point in the program has caught up to within fewer lines behind the last line

sent to PMAC than the value in this parameter. This can be detected as an interrupt to the host or be checked by the host on a polled basis.

Example

With I17=10 and I16=5, as program lines are sent to PMAC, PMAC will keep requesting more lines (BREQ goes high, *Rotary Buffer Full* bit stays 0) until there are 10 lines in the buffer ahead of the executing line. BREQ will then be held low and *Rotary Buffer Full* bit stays 1 until enough program lines have executed so that there are less than 5 lines in the buffer ahead of the execution point. At this time, BREQ will be set high again, and *Rotary Buffer Full* will become 0.

See Also

Using Interrupts (Writing a Host Communications Program)

Rotary Motion Program Buffers (Writing a Motion Program)

Coordinate-system *Rotary Buffer Full* status bit (Y:$0817, etc., bit 16)

On-line commands **PR**, **??**

I-variables I2, I17, I18

I17 Rotary Buffer Request Off Point

Range: .....
Default: .....
Units: .....

The BREQ line to the interrupt controller reflects the status of the hardware-selected coordinate system (by JPAN pins FPDn/) if the control-panel inputs are enabled (I2=0); it represents the status of the software-host-addressed coordinate system if the control-panel inputs are disabled (I2=1). In virtually all applications using this feature, the user will want to set I2 to 1 so the BREQ line reflects the status of the coordinate system to which he is currently talking.

Remarks

This parameter controls how many lines ahead of the executing line the host can provide a PMAC rotary motion program buffer before it signals that it is not ready for more lines (BREQ line held low, coordinate system status bit *Rotary Buffer Full* becomes 1). This status information can be detected either by polling (**??** or **PR**) or by using the interrupt line to the host.

If you send a program line to the rotary buffer, the BREQ line will be taken low (at least momentarily). If there are still fewer than I17 number of lines in the buffer ahead of the executing line, the BREQ line will be taken high again (giving the ability to generate an interrupt), and the *Rotary Buffer Full* status bit will stay 0. If there are greater than or equal to I17 lines in the buffer ahead of the executing line, the BREQ line will be left low, and the *Rotary Buffer Full* status bit will become 1. Normally at this point, the host will stop sending program lines (although this is not required) and wait for program execution to catch up to within I16 lines and take BREQ high again.

See Also

Program Using Interrupts (Writing a Host Communications Program)

Rotary Motion Program Buffers (Writing a Motion Program)

Coordinate-system "buffer-full" status bit (Y:$0817, etc., bit 16)

On-line commands **PR**, **??**

I-variables I2, I16, I18

I18 Fixed Buffer Full Warning Point

Range: .....0 .. 8,388,607
Default:.....10
Units: .....
.....Long Memory Words

Remarks

This parameter sets the level of open memory below which BREQ (Buffer Request) will not go true (global status bit *Fixed Buffer Full* will become 0) during the entry of a fixed (non-rotary) buffer.
Every time a command line is downloaded to an open fixed buffer (PROG or PLC), the BREQ line will be taken low (at least momentarily). If there are more than I18 words of open memory left, the BREQ line will be taken high again (giving the ability to generate an interrupt), and *Fixed Buffer Full* will stay at 0. If there are I18 words or less, the BREQ line will be left low, and *Fixed Buffer Full* will become 1.
The number of available words of memory can be found using the **SIZE** command.

See Also

Using Interrupts (Writing a Host Communications Program)
Global Fixed Buffer Full status bit (Y:$0003 bit 11),
I-variables I16, I17.
On-line command SIZE

Data Gathering I-Variables

I19 Data Gathering Period (in Servo Cycles)

Normally this parameter is controlled automatically by the PMAC Executive Program's Gathering and Tuning routines.

Range: .....
Default: .....
Units:.....

Remarks

This parameter controls how often data gathering is performed, in numbers of servo interrupt cycles. If I19 is 0, data gathering is performed only once per command.

See Also

Data Gathering (Analysis Features)
I-variables; I20-I44.
On-line commands GATHER, ENDGATHER

I20 Data Gathering Selection Mask

Normally this parameter is controlled automatically by the PMAC Executive Program's Gathering and Tuning routines.

Range: $000

Default: .....

Units:.....

Remarks

This 24-bit parameter controls which of the 24 potential data sources (as specified by I21 to I44) will be gathered when gathering is performed. If bit 0 (least significant bit) is 1, the 1st source (specified by I21) will be gathered; if it is 0, it will not be. Bit 1 controls the 2nd source (I22), and so on, to bit 23, which controls the 24th source (I44). With I9 at 2 or 3, the value of this variable will be reported back to the host in hexadecimal form, which is the more convenient form for understanding the value.

Example

With I20=7, only the addresses specified by I21, I22, & I23 will be gathered
With I20=$FF (255), the addresses specified by I21-I28 will be gathered
With I20=$300 (768), the addresses specified by I29 and I30 will be gathered
With I20=$FFFFFF (8,388,607), the addresses specified by I21-I44 will be gathered

See Also

Data Gathering (Analysis Features)

On-line commands GATHER, ENDGATHER, <CTRL-E>

I-variables; I19, I21-I44.

I21 Data Gathering Source 1 Address

Normally this parameter is controlled automatically by the PMAC Executive Program's Gathering and Tuning routines.

Range:.....$000

Default:.....

Units:.....

Remarks

This parameter specifies the address of the first data item to be gathered. This address is usually given in hexadecimal (i.e. preceded by a '$').
The specification is twenty-four bits -- six hex digits. The lowest 16 bits -- 4 hex digits -- represent the actual word address in the PMAC memory and I/O space. The highest two bits specify which part of the double word at that address is to be gathered (the Motorola DSP56000 has a double memory space -- X and Y -- to supply its dual data buses). If both of these bits are 0 -- first hex digit is 0 -- the Y word will be gathered; if the higher bit is 0 and the second bit is 1 -- first hex digit is 4 -- the X word will be gathered; if the higher bit is 1 -- first hex digit is 8 or greater -- both the X and Y words will be gathered as a long (double) word. In the case of a long word, it does not matter to PMAC what the second bit is, but the PMAC PC-Executive Program uses this bit to note whether this word is a fixed-point (0) or floating-point (1) value.
With I9 at 2 or 3, the value of this variable will be reported back to the host in hexadecimal form, which is the more convenient form for understanding the value.

Example

If the word address were $0720 (1824 decimal), I21=$000720 would denote gathering of the Y word; I21=$400720 would denote gathering of the X word; I21=$800720 or I21=$C00720 would cause gathering of both words. (You may specify this parameter in decimal form, but it is much more tricky.)

See Also

Data Gathering (Analysis Features)

On-line commands **GATHER**, **ENDGATHER**, **<CTRL-E>**

I-variables I19, I21-I44.

I22--I44 Data Gathering Source 2 thru 24 Addresses

Normally these parameters are controlled automatically by the PMAC Executive Program's Gathering and Tuning routines.

Range: $000
Default: .....
Units: .....

Remarks

These are the addresses of the second thru twenty-fourth data items to be gathered. See I21 for more details.

See Also

Data Gathering (Analysis Features)

On-line commands GATHER, ENDGATHER, <CTRL-E>

I-variables I19, I21-I44.

I45 Data Gathering Buffer Location and Mode

Wraparound feature not supported by PMAC Executive program data gathering and tuning routines.

Range: .....
Default: .....
Units: .....

Remarks

This variable controls

1. Locate buffer in regular RAM. Do not permit wrap-around (stop

where the data gather buffer will be located when it is defined, and whether it will wrap around when it is filled.

gathering when end of buffer is reached).

2. Locate buffer in regular RAM. Permit wraparound upon reaching end of buffer.
3. Locate buffer in dual-ported RAM (PMAC Option 2 required). Do not permit wraparound. Not very useful.
4. Locate buffer in dual-ported RAM (PMAC Option 2 required). Permit wraparound upon reaching end of buffer (usual mode for dual-ported RAM).

When I45 is set to 2 or 3, the gather buffer starts at PMAC address $D200 -- host address [base + $0800] -- and occupies the number of PMAC addresses specified in the **DEFINE GATHER** command. The DPRAM locations used by PMAC for gathering are as follows;

ADDRESS	DESCRIPTION
0x07FC (Y:$D1FF)	Data Gather Buffer Size.
0x07FC (X:$D1FF)	PMAC Data Gather Buffer Storage Address. If I45 = 2 and the buffer's end has been reached (this index is greater than or equal to the size), the DEFINE GATHER command must be issued again to allow gathering to restart.
0x0800 ($D200)	Start of Data Gather Buffer (not changeable).

See Also

Data Gathering (Analysis Features)

Option 2 Dual-Ported RAM

On-line commands **DEFINE GATHER**, **GATHER**, **ENDGATHER**, **DELETE GATHER**, **LIST GATHER**

I46 (Reserved For Future Use)

I47 Address of Pointer for Control-W Command

*The **<CONTROL-W>** function is now effectively obsolete. The newer bi-directional DPRAM ASCII communications feature enabled by I58 is superior and should be used instead*

Range:.....
Default:.....
Units:.....

Remarks

This parameter specifies the address of the register that tells the **<CONTROL-W>** command where to pick up its command string. The **<CONTROL-W>** command permits the host to load command strings into dual-ported RAM (Option 2 required), instead of the normal command interface, then cause the command to be accepted by sending a single byte (ASCII 23D is **<CTRL-W>**) to the command interface.

Example

For instance, if I47 is set to $D200, PMAC will look to its memory register Y:$D200 (wherever it sits in the host memory space) on receipt of a **<CTRL-W>** to see where to look for the command string. If Y:$D200 holds a value of $D700, PMAC will take the command string starting at register Y:$D700, incrementing addresses until it finds the null character (value 0).

See Also

Option 2 Dual-Ported RAM Manual

Memory-map registers $D000-$DFFF.

I-variables I56, I58

On-line command **<CONTROL-W>**

I48 DPRAM Servo Data Enable

Range: ..... 0 .. 1
Default: 0
Units: ..... none

Remarks

This parameter enables or disables the dual-ported RAM (DPRAM) servo data reporting function. When I48=1 and the **GATHER** command has been issued, PMAC copies key data from the servo control registers to fixed registers in the DPRAM every I19 servo cycles for easy access by the host computer. Servo data for motors up to the number specified by I59 are reported. Regular data gathering is disabled in this mode.
When I48=0, the DPRAM servo data reporting function is disabled. Regular data gathering can be enabled in this mode. Refer to the description of DPRAM functions for more information.

See Also

DPRAM Servo Data Reporting (Option 2 DPRAM Manual)

I-variables I19, I49, I59

On-line commands **GATHER**, **ENDGATHER**

I49 DPRAM Background Data Enable

Range: ..... 0 .. 1
Default: 0
Units: ..... none

Remarks

This parameter enables or disables the dual-ported RAM (DPRAM) background data reporting function. When I49=1, PMAC copies key data from the background information registers to fixed registers in the DPRAM for easy

access by the host computer. Each time the host computer reads these registers and signals it is done, PMAC will copy the data again. Data for motors and coordinate systems up to the number specified by I59 are reported.
When I49=0, the DPRAM background data reporting function is disabled.
Refer to the description of DPRAM functions for more information.

See Also

DPRAM Background Data Reporting (Option 2 DPRAM Manual)

I-variables I48, I59

I50 Rapid Move Mode Control

Range: ..... 0 .. 1
Default: 1
Units: ..... none

Remarks

This parameter determines which variables are used for speed of **RAPID** mode moves. When I50 is set to 0, the jog parameter for each motor (Ix22) is used. When I50 is set to 1, the maximum velocity parameter for each motor (Ix16) is used instead. Regardless of the setting of I50, the jog acceleration parameters Ix19-Ix21 control the acceleration.

See Also

RAPID mode moves (Writing a Motion Program)

I-variables Ix16, Ix19-Ix22

Program command **RAPID**.

I51 Leadscrew Compensation Enable

Range: ..... 0 .. 1
Default: 0
Units: ..... none

Remarks

This parameter permits the enabling and disabling of the leadscrew compensation tables. When I51 is 0, all tables are disabled and there is no correction performed. When I51 is 1, all tables are enabled and corrections are performed as specified in the tables.

See Also

Leadscrew Compensation (Setting Up a Motor)

On-line commands DEFINE COMP, DELETE COMP, LIST COMP DEF.

Compensation Position Registers

Suggested M-variables

I52 '\' Program Hold Slew Rate

Range: ..... 0 .. 8,388,607
Default: 37,137
Units: ..... I10 units / segmentation period

Remarks

This parameter controls the slew rate to a stop on a '****' program hold command, and the slew rate back up to speed on a subsequent **R** command, for all coordinate systems, provided PMAC is in a segmented move (**LINEAR** or **CIRCLE** mode with I13>0). If PMAC is not in a segmented move (I13=0, or other move mode), the '****' command acts just like an '**H**' feed hold command, with Ix95 controlling the slew rate.
The units of I52 are the units of I10 (1/8,388,608 msec) per segmentation period (I13 msec). To calculate how long it takes to stop on a **** command, and to restart on the next R command, use the formula :

$$T(msec) = I10 \cdot \frac{I13}{I52}$$

To calculate the value of I52 for a given start/stop time, use the formula :

$$I52 = I10 \cdot \frac{I13}{T(msec)}$$

Example

To execute a full stop in one second with the default servo update time (I10 = 3,713,707) and a move segmentation time of 10 msec, I52 should set to 3,713,707 * 10 / 1000 = 37,137.

See Also

Stop Commands (Making Your Application Safe)
I-variables I13, Ix95
On-line commands \, H

I53 Program Step Mode Control

Range: ..... 0 .. 1
Default: 0
Units: ..... none

Remarks

This parameter controls the action of a Step (**S**) command in any coordinate system on PMAC. At the default I53 value of zero, a Step command causes program execution through the next move, **DELAY**, or **DWELL** command in the program, even if this takes multiple program lines.
When I53 is set to 1, a Step command causes program execution of only a single program line, even if there is no move or **DWELL** command on that line. If there is more than one **DWELL** or **DELAY** command on a program line, a single Step command will only execute one of the **DWELL** or **DELAY** commands.
Regardless of the setting of I53, if program execution on a Step command encounters a **BLOCKSTART** statement in the program, execution will continue until a **BLOCKSTOP** statement is encountered.

See Also

Control Panel Port STEP/ Input (Connecting PMAC to the Machine)
On-line commands **<CTRL-R>**, **<CTRL-S>**, **Q**, **R**, **S**
Program commands **BLOCKSTART**, **BLOCKSTOP**

I54 Serial Baud Rate (PMAC2 Only)

Range: ..... 0 to 15
Units: .....
.....None
Default: ..... 12 (38400 baud)

Remarks

I54 controls the baud rate for communications on the serial port. PMAC2 uses I54 only at power-up/reset to set up the frequency of the clocking circuit for the serial port. To change the baud rate, it is necessary to change the value of I54, store this value to non-volatile flash memory with the **SAVE** command, and reset the card. At this time, PMAC2 will establish the new baud rate
The possible settings of I54 and the baud rates they define are:

I54	BAUD RATE FOR 40 MHZ CPU	BAUD RATE FOR 60 MHZ CPU
0	600	DISABLED
1	900* (-0.05%)	900
2	1200	1200
3	1800* (-0.1%)	1800
4	2400	2400
5	3600* (-0.19%)	3600
6	4800	4800
7	7200* (-0.38%)	7200
8	9600	9600
9	14,400*(-0.75%)	14,400
10	19,200	19,200
11	28.800*(-1.5%)	28,800
12	38,400	38,400
13	57,600*(-3.0%)	57,600
14	76,800	76,800
15	DISABLED	115,200

* Not an exact baud rate

Because of the nature of the clock generation circuitry, odd values of I54 on a PMAC2 with a CPU operating at 40 MHz (Jumper E2 OFF) produce non-exact baud rates. The error in baud rate is small enough that communications should still be valid.
If your host computer baud rate cannot be made to match the PMAC2's baud rate, either PMAC2's baud rate must be changed through the bus communications port, or the PMAC2 must be re-initialized by resetting or powering up with the E3 jumper ON. This forces the PMAC2 to the default baud rate of 38,400.

I55 DPRAM Background Variable Buffers Enable

Range: ..... 0 .. 1
Default: 0
Units: .....
..... none

Remarks

This parameter enables or disables the dual-ported RAM (DPRAM) background variable read and write buffer function. When I55 is 0, this function is disabled. When I55 is 1, this function is enabled. When enabled, the user can specify up to 128 PMAC registers to be copied into DPRAM each background cycle to be read by the host (background variable read) and up to 128 PMAC registers to be copied each background cycle from values written into the DPRAM by the host (background variable write).

See Also

DPRAM Background Variable Read Buffer (Option 2 DPRAM Manual)
DPRAM Background Variable Write Buffer (Option 2 DPRAM Manual)

I56 DPRAM ASCII Communications Interrupt Enable (PMAC[1] Specific)

Range: ..... 0 .. 1
Default: 0
Units: .....
..... none

Remarks

This parameter enables or disables the interrupt feature for the dual-ported RAM (DPRAM) ASCII communications function. When I56=1, PMAC will generate an interrupt to the host computer each time it loads a line into the DPRAM ASCII buffer for the host to read. When I56=0, it will not generate this interrupt.
For PMAC-PC and PMAC-Lite, the interrupt line used is the EQU4 interrupt. For this to reach the host, the E55 jumper must be ON, and E54, E56, and E57 must be OFF (E54 and E56 do not exist on PMAC-Lite).
For PMAC-VME, the interrupt line used is the normal communications interrupt (the only interrupt available). This line -- IRQn on the VME bus, is determined by the VME setup value in register X:$0788. The interrupt vector provided to the host is one greater than the value in setup register X:$0789. For example, if the value in X:$0789 is the default of $A1, this interrupt will provide an interrupt vector of $A2.

See Also

DPRAM ASCII Communications (Option 2 DPRAM Manual)
I-variables I48, I49, I55, I57, I58, I59

I56 DPRAM ASCII Communications Interrupt Enable (PMAC2 Specific)

Range: 0 to 1

Units: .....
.....None
Default: .....0 (disabled)
This parameter enables or disables the interrupt feature for the dual-ported RAM (DPRAM) ASCII communications function enabled by I58=1. When I56=1, PMAC will generate an interrupt to the host computer each time it loads a line into the DPRAM ASCII buffer for the host to read. When I56=0, it will not generate this interrupt.
For the regular PC-bus versions of PMAC2 (PMAC2-PC and PMAC2-Lite), the interrupt line used is the EQU1 interrupt. When using this feature, do not use the EQU1 line for any other purpose, including position compare.
For the VME-bus versions of PMAC2 (PMAC2-VME and PMAC2-VME Ultralite), the interrupt line used is the normal communications interrupt (the only interrupt available). This line -- IRQn on the VME bus, is determined by the VME setup value in register X:$0788. The interrupt vector provided to the host is one greater than the value in

setup register X:$0789. For example, if the value in X:$0789 is the default of $A1, this interrupt will provide an interrupt vector of $A2.
For the PMAC2-PC Ultralite, this feature is not presently supported with the standard hardware.

See Also

DPRAM ASCII Communications (Option 2 DPRAM Manual)

I-variables I48, I49, I55, I57, I58, I59

I57 DPRAM Binary Rotary Buffer Enable

Range: ..... 0 .. 1
Default: 0
Units: ..... none

Remarks

This parameter enables or disables the dual-ported RAM (DPRAM) binary rotary buffer function. When I57=1, this function is enabled and the host computer can download motion program data to the PMAC through the DPRAM in binary form for maximum possible throughput. When I57=0, this function is disabled.

See Also

DPRAM Binary Rotary Buffer (Binary Rotary Buffer)

I-variables I48, I49, I55, I56, I58, I59

I58 DPRAM ASCII Communications Enable

Range: ..... 0 .. 1
Default: 0
Units: ..... none

Remarks

This parameter enables or disables the dual-ported RAM (DPRAM) ASCII communications function. When I58=1, this function is enabled and the host computer can send ASCII command lines to the PMAC through the DPRAM and receive ASCII responses from PMAC through the DPRAM. When I58=0, this function is disabled.
With I58=1, the PMAC response port on the bus will be whichever port (normal I/O-mapped bus port, or DPRAM) has received the most recent command.
The **<CTRL-Z>** command, which causes the PMAC response port to revert to the serial port, automatically sets I58 to 0.
When I58 is set to 1, I3 is automatically forced to 0 or 2, if it has been at 1 or 3, respectively.
If I56 is also equal to 1, PMAC will provide an interrupt to the host computer when it provides a response string.

See Also

DPRAM ASCII Communications (Option 2 DPRAM Manual)

I-variables I3, I48, I49, I55, I56, I57, I59

On-line command <CTRL-Z>

I59 DPRAM Buffer Maximum Motor/C.S. Number

Range: ..... 0 .. 8
Default: 0

Units: ..... none

Remarks

This parameter determines the highest-numbered motor and/or coordinate system for which servo data is reported in the DPRAM when servo data reporting is active (I48=1) and/or background data reporting is active (I49=1). If I59=0, no data is reported even if one or both reporting functions are active. If I59>0, data for both motor and coordinate system of numbers up to the value of I59 are reported -- there is not separate control of maximum motor and coordinate system numbers.

See Also

DPRAM Servo Data Reporting, DPRAM Background Data Reporting

I60 Auto-Converted ADC Register Address PMAC[1] Specific)



It is obviously easier to specify this parameter in hexadecimal form ($ prefix). If I9 is set to 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.

Range: .....
Default: .....
Units: .....

Remarks

This parameter permits the user to specify the address of one ACC-36 analog-to-digital converter (ADC) board whose values will automatically be copied into the PMAC memory at a high rate so that they can be used as servo feedback. This can also be used to make user program access to these ADCs more convenient, but it is not required for this purpose.

There are 24 legal addresses for an ACC-36 in the PMAC memory and I/O space: even values from $FFD0 to $FFFE. If you have more than one ACC-36 connected to PMAC, only one board may be used in this manner. All other boards must be accessed in user programs. For the ACC-36 board automatically converted using I60 and I61, the board must never be accessed in user programs, but user programs may read the memory registers in PMAC to which the ADC values are copied.

If I60 is set to 0, no automatic conversion will take place. If the first two hex digits of I60 are set to anything except $FF, PMAC will automatically change them to $FF.

ADCs 1 to 8 are copied into the low 12 bits of registers Y:$0708 to Y:$070F, respectively. ADCs 9 to 16, if they exist on the addressed board, are copied into the low 12 bits of registers X:$0708 to X:$070F. These registers should be treated as signed registers.

Example

A PMAC system has an ACC-14D at address $FFD0, and an ACC-36 at address $FFD8. It is desired to automatically convert all 8 registers on the ACC-36. I60 is set to $FFD8, and I61 is set to 7.

See Also

Parallel Position Feedback Conversion (Setting Up a Motor)

I-variables I61, Ix10

Memory and I/O Map registers $FFD0 to $FFFE

ACC-36 User's Manual

I61 Number of Auto-Converted ADC Registers (PMAC[1] Specific)

Range: ..... 0 .. 7
Default: 0
Units: .....
.....Number of registers minus 1

Remarks

This parameter permits the user to specify the number of analog-to-digital converter (ADC) registers on the ACC-36 specified by I60 that will be automatically converted and copied into PMAC memory. There are two 12-bit converters per 24-bit register. The number of registers converted automatically is equal to I61 + 1.

Each phase cycle (9 kHz default), PMAC copies the contents of an ACC-36 register into RAM, then selects the next register, so the conversion can start and the results will be ready for the next phase cycle. PMAC will cycle through the first I61+1 registers on the ACC-36 in this fashion. If I61 is set to 0, PMAC will cycle through all 8 registers on the ACC-36 (equivalent to I61=7).

If you have more than one ACC-36 connected to PMAC, only one board may be used in this manner. All other boards must be accessed in user programs. For the ACC-36 board automatically converted using I60 and I61, the board must never be accessed in user programs, but user programs may read the memory registers in PMAC to which the ADC values are copied.

ADCs 1 to 8 are copied into the low 12 bits of registers Y:$0708 to Y:$070F, respectively. ADCs 9 to 16 are copied into the low 12 bits of registers X:$0708 to X:$070F. These registers should be treated as signed values.

Example

The system has 8 axes with analog feedback. There are 4 phase cycles per servo cycle, and it is important to have new feedback values every servo cycle. Therefore an ACC-36 with Option 1 is ordered, so there are 2 ADCs per register, and I61 is set to 3 to convert the first 4 registers in a cyclic fashion. ADCs 1 to 4 are copied into Y:$0708 to Y:$070B, respectively; ADCs 9 to 12 are copied into X:$0708 to X:$070B, respectively.

See Also

Parallel Position Feedback Conversion (Setting Up a Motor)

I-variables I60, Ix10

Memory and I/O Map registers $FFD0 to $FFFE

ACC-36 User's Manual

I60, I61 Analog-to-Digital Conversion Support (Deleted)

These PMAC I-variables have been replaced with a converter set-up table on the PMAC2. Refer to the description of PMAC2 General Purpose I/O in the PMAC2 User's Guide.

I62 Internal Message Carriage Return Control (PMAC2 Specific)

Do not set I62 to1 if using dual-ported RAM ASCII communications (I58=1).

Range: .....
Default: .....
Units: .....

Remarks

This parameter permits the user to control whether internally generated messages sent from PMAC to the host computer are terminated with the carriage return (**<CR>**) character or not. It affects only those messages generated by a **CMD**, **SEND**, **SENDP**, or **SENDS** statement in a PMAC motion or PLC program. The ability to suppress the **<CR>** provides more flexibility in controlling the format display of a terminal window or printer.

If I62 is set to the default value of 0, these messages are terminated with a **<CR>** . If I62 is set to 1, the **<CR>** is suppressed. With I62 set to 1, if a PMAC program is desired to cause a **<CR>** to be sent, the **SEND^M** command must be used (the carriage return character is <CTRL-M>).

Example

With program code:

```
I62=1       ; Suppress <CR> on SEND

SEND "THE VALUE OF P1 IS "  ; String sent with no <CR>

CMD "P1"    ; Response string follows on same line, no <CR>

SEND^M      ; Send a <CR>
```

PMAC responds with: .....

```
THE VALUE OF P1 IS 42
```

See Also

Program Commands CMD, SEND, SENDS, and SENDP

I63--I79 (Reserved For Future Use)

I8x Motor x Third-Resolver Gear Ratio

Range: ..... 0 .. 4095
Default: 0
Units: .....
..... Second-resolver turns per third-resolver turn

Remarks

This parameter tells PMAC the gear ratio between the second (medium) and third (coarse) resolvers for a triple-resolver setup for Motor x. It is expressed as the number of turns (electrical cycles) the second resolver makes in one full turn (electrical cycle) of the third resolver.
This parameter is used only during the PMAC power-up/reset cycle to establish absolute power-on servo position. Therefore, the parameter must be set, the value stored in EAROM with the **SAVE** command, and the card reset before it takes effect.
If there is no geared third resolver on Motor x, or if absolute power-on position is not desired, I8x should be set to zero. If either Ix10 (for the primary resolver) or I9x (for the secondary resolver) is set to zero, I8x is not used.
The third resolver must be connected to the next higher numbered R/D converter at the same multiplexer address than the second resolver, which must be connected to the next higher numbered converter at the same multiplexer address than the primary resolver. There can be up to eight R/D converters on two ACC-8D Option 7 boards at one multiplexer address.

Example

Motor 3 has a triple resolver, with each resolver geared down by a ratio of 16:1 from the resolver before it. The fine resolver is connected to R/D converter 4 at multiplexer address 0 (the first R/D converter on the second ACC-8D Option 7 at address 0). The medium resolver is connected to R/D converter 5 at this address, and the coarse resolver is connected to R/D converter 6. The following I-variable values should be used:

```
I310=$040100   ; The 0100 in the low 16 bits specifies
     ; multiplexer address 0; the 4 in the high 8 bits
     ; specifies R/D converter 4 at this address.
I93=16    ; Specifies 16:1 ratio between medium and fine
I83=16    ; Specifies 16:1 ratio between coarse and medium
```

See Also

Selecting the Position Loop Feedback (Setting Up a Motor)
I-Variables I9x, Ix03, Ix10, Ix81
ACC-8D Option 7 (R/D Converter) Manual

I89 Cutter Comp Outside Corner Break Point

Range: .....-1.0 -- 1.0
Default: ..... 0.99848 (cos 1°)
Units: .....
..... cos $\Delta\theta$

Remarks

This parameter controls the threshold between outside corner angles for which an extra arc move is added in cutter compensation, and those for which the incoming and outgoing moves are directly blended together.
I89 is expressed as the cosine of the change in directed angle between the incoming and outgoing moves. As such, it can take a value between -1.0 and +1.0. If the two moves have the same directed angle at the move boundary (i.e. they are moving in the same direction), the change in directed angle is 0, and the cosine is 1.0. As the change in di-

rected angle increases, the corner gets sharper, and the cosine of the change in directed angle decreases. For a total reversal, the change in directed angle is 180°, and the cosine is -1.0.

If the cosine of the change in directed angle of an outside corner is less than I89 (a large change in directed angle; a sharp corner), PMAC will automatically add an arc move with a radius equal to the cutter radius to join the incoming and outgoing moves. This prevents the cutter from moving too far out when going around the outside of a sharp corner.

If the cosine of the change in directed angle of an outside corner is greater than I89 (a small change in directed angle; a gradual corner), PMAC will directly blend the incoming and outgoing moves with its normal blending algorithms. This can provide increased speed on small angle changes, because an extra segment of minimum TA or 2*TS time is not added.

I89 does not affect the behavior at inside corners, where the incoming and outgoing moves are always blended directly together, regardless of the change in directed angle.

Before V1.16 firmware, an arc was added to an outside corner if the change in directed angle were greater than 1°.

Example

If it is desired that an arc only be added if the change in directed angle is greater than 45°, then I89 should be set to 0.707, because $\cos \Delta\theta = \cos 45^o = 0.707$

See Also

Cutter Radius Compensation (Writing a Motion Program)

I9x Motor x Second-Resolver Gear Ratio

Range: ..... 0 .. 4095
Default: 0
Units: .....
.....Primary-resolver turns per second-resolver turn

Remarks

This parameter tells PMAC the gear ratio between the first (fine, or primary) and second (coarse or medium) resolvers for a double- or triple-resolver setup for Motor x. It is expressed as the number of turns (electrical cycles) the first resolver makes in one full turn (electrical cycle) of the second resolver.
This parameter is used only during the PMAC power-up/reset cycle to establish absolute power-on servo position. Therefore, the parameter must be set, the value stored in EAROM with the **SAVE** command, and the card reset before it takes effect.
If there is no geared second resolver on Motor x, or if absolute power-on position is not desired, I9x should be set to zero. If Ix10 (for the primary resolver) is set to zero, I9x is not used. In a triple-resolver system, I9x must be set greater than zero in order for I8x (third-resolver gear ratio) to be used.
The second resolver must be connected to the next higher numbered R/D converter at the same multiplexer address than the first resolver. If there is a third resolver, it must be connected to the next higher numbered converter at the same multiplexer address than the second resolver. There can be up to eight R/D converters on two ACC-8D Option 7 boards at one multiplexer address.
If Ix10 is set up for an ACC-8D Option 9 Yaskawa encoder converter, I9x represents the counts per revolution (including x2 or x4 quadrature decode, if used) of the encoder; effectively it is the "gear ratio" between the encoder and the revolution counter.

Example

Motor 1 has a double resolver with the fine resolver connected to the R/D converter at location 2 on an ACC-8D Option 7 board set to multiplexer address 4, and the coarse resolver, geared down at a 36:1 ratio from the fine resolver, connected to the R/D converter at location 3 on the same board. The following I-variable settings should be used:

```
I110=$020004   ; Value of $0004 in low 16 bits specifies multiplexer address 4

     ; $02 in high 8 bits specifies R/D at location 2 of this address

I91=36    ; Specify 36 turns of fine resolver per turn of coarse resolver

     ; R/D must ;be at location 3 of multiplexer address 4

I81=0     ; No third resolver
```

See Also

Selecting the Position Loop Feedback (Setting Up a Motor)
I-Variables I8x, Ix03, Ix10, Ix81
ACC-8D Option 7 (R/D Converter) Manual

I99 Backlash Hysteresis

Range: .....0 .. 8,388,607
Default: .....64 (= 4 counts)

Units: ..... 1/16 count

Remarks

This parameter controls the size of the direction reversal in motor commanded position that must occur on any motor before PMAC starts to add the programmed backlash (Ix86) in the direction of motion. The purpose of this variable is to allow the customer to ensure that a very small direction reversal (e.g. from the dithering of a master encoder) does not cause the backlash to "kick in". I99 thus provides a hysteresis in the backlash function.

The units of I99 are 1/16 of a count. Therefore this parameter must hold a value 16 times larger than the number of counts reversal at which backlash is introduced. For example, if backlash is to be introduced after 5 counts of reversal, I99 should be set to 80.

Before I99 was implemented, the backlash hysteresis was fixed at 4 counts, equivalent to the default I99 value of 64.

Example

With a system in which one count of the master encoder creates 10 counts of movement in the slave motor, it is desired that a single count reversal of the master not trigger backlash reversal. Therefore the backlash hysteresis is set to 15 counts, and I99 is set to 15*16=240.

See Also

Backlash Compensation (Setting Up a Motor)

I-Variables Ix85, Ix86

Motor I-Variables

x = Motor Number (#x, x = 1 to 8)

Motor Definition I-Variables

Ix00 Motor x Activate

Range: ..... 0 .. 1
Default: ..... I100=1; I200 .. I800=0
Units: ..... none

Remarks

This parameter determines whether the motor is de-activated (=0) or activated (=1). If activated, position, servo, and trajectory calculations are done for the motor. An "activated" motor may be "enabled" -- either in open or closed loop -- or "disabled" (killed), depending on commands or events.
If Ix00 is 0, not even the position calculations for that motor are done, so a **P** command would not reflect position changes. Any PMAC motor not used should be de-activated, so PMAC does not waste time doing calculations for that motor. The fewer motors are activated, the faster the servo update time can be.

See Also

On-line commands **K**, **<CTRL-K>**, **A**, **<CTRL-A>**, **J/**

Ix01 Motor x PMAC-Commutation Enable

Range: ..... 0 .. 1
Default: 0
Units: ..... none

Remarks

This parameter determines whether PMAC will perform commutation calculations for the motor and provide two analog outputs (Ix01=1), or not perform commutation, and only provide one analog output (Ix01=0). If a multi-phase motor is used, but is commutated in the amplifier, Ix01 should be set to 0.

See Also

Setting Up PMAC Commutation
I-variables Ix70-Ix83.

Ix02 Motor x Command Output (DAC) Address

With I9 at 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.

Range: Extended legal PMAC "X and Y" addresses

Default:

MOTOR	I-VARIABLE	HEX	DECIMAL	DAC
Motor #1:	I102:	$C003	49155	(=DAC1)
Motor #2:	I202:	$C002	49154	(=DAC2)
Motor #3:	I302:	$C00B	49163	(=DAC3)

Motor #4:	I402:	$C00A	49162	(=DAC4)
Motor #5:	I502:	$C013	49171	(=DAC5)
Motor #6:	I602:	$C012	49170	(=DAC6)
Motor #7:	I702:	$C01B	49179	(=DAC7)
Motor #8:	I802:	$C01A	49178	(=DAC8)

Units: Extended legal PMAC "X and Y" addresses

Remarks

This parameter tells the PMAC where (what address) to put the output command for motor x. The address may be specified as either a decimal or hexadecimal value. Usually the output is directed towards a DAC register.
Non-PMAC-Commutated Motors: If PMAC is not performing the commutation for motor x, Ix02 should point directly to the DAC register in the DSP-GATE. Typically DACx is used for motor x, but this is not required. The addresses of DAC1 -- DAC8 are given in the default table above. The addresses for DAC9 -- DAC16 (which are on the Accessory 24 Axis Expansion Board) are:

DAC #	HEX	DECIMAL
DAC 9	$C023	(49187)
DAC10	$C022	(49186)
DAC11	$C02B	(49195)
DAC12	$C02A	(49194)
DAC13	$C033	(49203)
DAC14	$C032	(49202)
DAC15	$C03B	(49211)
DAC16	$C03A	(49210)

PMAC-Commutated Motors: If motor x is commutated by PMAC, Ix02 must specify the lower address of a pair of adjacent DACs that are being used to command the phases of the motor.

HEX	DECIMAL	DAC PAIR
$C002	49154	DAC1 and DAC2
$C00A	49162	DAC3 and DAC4
$C012	49170	DAC5 and DAC6
$C01A	49178	DAC7 and DAC8
$C022	49186	DAC9 and DAC10
$C02A	49194	DAC11 and DAC12
$C032	49202	DAC13 and DAC14
$C03A	49210	DAC15 and DAC16

Extended Addressing: The destination address of the output command occupies bits 0 to 15 of Ix02 (range $0000 to $FFFF, or 0 to 65535). With bit 16 equal to zero -- the normal case -- the output is signed: a negative output for a negative value, and a positive output for a positive value. Setting bit 16 to 1 provides a couple of interesting output options, as explained below. In the extended version, it is obviously easier to specify this parameter in hexadecimal form.

Magnitude and Direction Output: However, if bit 16 of Ix02 -- value 65536 -- equals 1, and Ix01=0 (no PMAC commutation), then the output is the absolute value (magnitude) of what is calculated, and the sign (direction) bit is output on the AENAn/DIRn line of the set of flags pointed to by Ix25 (polarity is determined by jumper E17). In this case, bit 16 of Ix25 should also be set to 1 to disable the amplifier-enable function for that line.
This magnitude-and-direction mode is suited for driving servo amplifiers that expect this type of input, and for driving voltage-to-frequency (V/F) converters, such as the PMAC ACC-8D Option 2 board, for running stepper motor drivers. For example, if you were using PMAC and an ACC-8D Option 2 to run a four-axis stepper systems, you would set up your variables in the following way:

```
I102=$1C003 .....
I125=$1C000

I202=$1C002 .....
I225=$1C004

I302=$1C00B .....
I325=$1C008

I402=$1C00A .....
I425=$1C00C
```

Direct Microstep Output: If bit 16 of Ix02 -- value 65536 -- equals 1, and Ix01=1 (PMAC commutation), then the output is set is set up for direct microstepping phase control using the PMAC commutation algorithms. Just as in the closed-loop commutation case (see above), bits 0-15 should point to the low address of an adjacent pair of DACs.
X-register Output: If bit 19 of Ix02 is set to 1, the command output(s) is (are) written to the X-register(s) of the specified address instead of the Y-register(s). If bit 19 is at the default of 0, the command output(s) is (are) written to the normal Y-register(s). Writing to X-registers has two main uses. First, some MACRO nodes are in X-registers. Second, for cascaded loops, the output of one loop can become the input to another loop, and master or feedback inputs are expected in X-registers.

See Also

Selecting the Output (Setting Up a Motor)

I-variables Ix01, Ix25, Ix70-Ix83

Memory-I/O registers Y:$C000-Y:$C03F

Ix02 Motor x Command Output Address

Range/Units: ..... Extended PMAC Addresses

Default values (non-Ultralite PMAC2 boards):

MOTOR	I-VARIABLE	HEX	DAC	PWM
Motor #1:	I102	$C002	DAC1A, DAC1A&B	PWM1A&B&C
Motor #2:	I202	$C00A	DAC2A, DAC2A&B	PWM2A&B&C
Motor #3:	I302	$C012	DAC3A, DAC3A&B	PWM3A&B&C
Motor #4:	I402	$C01A	DAC4A, DAC4A&B	PWM4A&B&C
Motor #5:	I502	$C022	DAC5A, DAC5A&B	PWM5A&B&C
Motor #6:	I602	$C02A	DAC6A, DAC6A&B	PWM6A&B&C
Motor #7:	I702	$C032	DAC7A, DAC7A&B	PWM7A&B&C
Motor #8:	I802	$C03A	DAC8A, DAC8A&B	PWM8A&B&C

Default values (PMAC2 Ultralite boards):

MOTOR	I-VARIABLE	HEX	MACRO NODE	REGISTER
Motor #1:	I102	$C0A0	0	0
Motor #2:	I202	$C0A0	1	0
Motor #3:	I302	$C0A0	4	0
Motor #4:	I402	$C0A0	5	0
Motor #5:	I502	$C0A0	0	0
Motor #6:	I602	$C0A0	1	0
Motor #7:	I702	$C0A0	4	0
Motor #8:	I802	$C0A0	5	0

Remarks

Ix02 tells Motor x which register or registers to which it writes its command output values. Bits 0 to 15 of Ix02 contain the address of this register or the first of these multiple registers. This determines which output lines transmit the command output signals.

If PMAC2 is not commutating Motor x (Ix01=0), only one command output value is calculated, which is written to the register at the address specified in Ix02. If PMAC2 is commutating Motor x (Ix01=1), but not closing its current loop (Ix82=0), two command output values are calculated, which are written to the register at the address specified in Ix02, plus the register at the next higher address. If PMAC2 is commutating Motor x (Ix01=1) and closing its current loop (Ix82>0), three command output values are calculated, which are written to the registers at the address specified in Ix02, plus the registers at the next two higher addresses.

The default values listed above are suitable for most applications. They can be used when Motor x uses the machine interface channel of the same number, whether for single analog output (velocity or torque mode), double analog output, or triple PWM output.

One common application type for which the default value of Ix02 cannot be used is the direct pulse-and-direction output for stepper motor drives. This mode uses the 'C' output register alone for each channel. In this case, the following values should be used:

Channel 1: $C004 (PULSE1, DIR1)
Channel 2: $C00C (PULSE2, DIR2)
Channel 3: $C014 (PULSE3, DIR3)

Channel 4: $C01C (PULSE4, DIR4)

Channel 5: $C024 (PULSE5, DIR5)
Channel 6: $C02C (PULSE6, DIR6)

Channel 7: $C034 (PULSE7, DIR7)

Channel 8: $C03C (PULSE8, DIR8)

If bit 19 of Ix02 is set to 1, then the command output is written to a PMAC2 X-register rather than the typical Y-register. The main use for this is to be able to write to those Type 0 MACRO registers that are mapped into X-registers. This feature is only usable if PMAC2 is not performing the commutation for the motor -- if PMAC2 is commutating the motor through MACRO registers, only those MACRO nodes mapped into Y-registers (0, 1, 4, 5, 8, 9, 12, 13) can be used.

To write command outputs to MACRO registers for Type 0 MACRO drives such as the Performance Controls FLX Drive, the following values of Ix02 should be used:

MACRO NODE #	3-PHASE (DIRECT PWM)	2-PHASE (SINEWAVE)	1-PHASE (VELOCITY OR TORQUE)
Node 0	$C0A1	$C0A2	$C0A3
Node 1	$C0A5	$C0A6	$C0A7
Node 2	N/A	N/A	$8C0A3
Node 3	N/A	N/A	$8C0A7
Node 4	$C0A9	$C0AA	$C0AB
Node 5	$C0AD	$C0AE	$C0AF
Node 6	N/A	N/A	$8C0AB
Node 7	N/A	N/A	$8C0AF
Node 8	$C0B1	$C0B2	$C0B3
Node 9	$C0B5	$C0B6	$C0B7
Node 10	N/A	N/A	$8C0B3
Node 11	N/A	N/A	$8C0B7
Node 12	$C0B9	$C0BA	$C0BB
Node 13	$C0BD	$C0BE	$C0BF
Node 14	N/A	N/A	$8C0BB
Node 15	N/A	N/A	$8C0BF

N/A = not available

The 3-phase outputs are used for direct PWM outputs when PMAC2 is performing commutation and digital current loop (Ix01=1 and Ix82>0); the 2-phase outputs are used for sinusoidal phase current commands when PMAC2 is performing commutation but not digital current loop (Ix01=1 and Ix82=0); the 1-phase outputs are used for velocity or torque outputs when PMAC2 is not performing either commutation or current loop (Ix01=0 and Ix82=0).
To write commands to Type 1 MACRO registers, as with the Delta Tau Compact MACRO Station, the following values of Ix02 should be used:

MACRO NODE #	IX25 VALUE
Node 0	$C0A0
Node 1	$C0A4
Node 4	$C0A8
Node 5	$C0AC
Node 8	$C0B0
Node 9	$C0B4
Node 12	$C0B8
Node 13	$C0BC

Ix03 Motor x Position Loop Feedback Address

In the extended version, it is obviously easier to specify this pa-

Range: ..... E

Default:

VARIABLE	HEX	DECIMAL	ENCODER

rameter in hexadecimal form. With I9 at 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.

I103:	$0720	(1824)	(=converted ENC1)
I203:	$0721	(1825)	(=converted ENC2)
I303:	$0722	(1826)	(=converted ENC3)
I403:	$0723	(1827)	(=converted ENC4)
I503:	$0724	(1828)	(=converted ENC5)
I603:	$0725	(1829)	(=converted ENC6)
I703:	$0726	(1830)	(=converted ENC7)
I803:	$0727	(1831)	(=converted ENC8)

Units: ..... E

Remarks

This parameter tells the PMAC where to look for its feedback to close the position loop for motor x. Usually it points to an entry in the "Encoder Conversion Table", where the values from the encoder counter registers have been processed at the beginning of each servo cycle (possibly to include sub-count data). This table starts at address $0720 (1824 decimal). It is shipped from the factory configured as shown in the default table above.

For a motor with dual feedback (motor and load), use Ix03 to point to the encoder on the load, and Ix04 to point to the encoder on the motor.

If the position loop feedback device is the same device as is used for commutation (with PMAC doing the commutation), then it must also be specified for commutation with Ix83. However, Ix83 should specify the address of the encoder counter itself, not the converted data of the table.

Hardware Home Position Capture: The source address of the position information occupies bits 0 to 15 of Ix03 (range $0000 to $FFFF, or 0 to 65535). With bit 16 equal to zero -- the normal case -- position capture on homing is done with the hardware capture register associated with the flag inputs pointed to by Ix25. In this case, it is important to match the encoder number, the address pointed to with Ix03, with the flag number, the address pointed to with Ix03 (e.g. ENC1 -- CHA1 & CHB1 -- with HMFL1 and LIM1).

Software Home Position Capture: If bit 16 (value 65536) is set to one, the position capture on homing is done through software, and the position source does not have to match the input flag source. This is particularly important for parallel-data position feedback, such as from a laser interferometer (which is incremental data and requires homing). For example, if motor #1 used parallel feedback from a laser interferometer processed as the first (triple) entry in the conversion table, the key I-variables would be:

I103=$10722.....
I125=$C000

This would permit homing on interferometer data with HMFL1 triggering.

Capture on following error: If bit 17 of Ix03 is set to 1, then the trigger for position capture of this motor is a true state on the warning following error status bit for the motor. If bit 17 is at the default of 0, the trigger for position capture is the capture flag of the flag registers as set by Ix25. The trigger is used in two types of moves: homing search moves and programmed move-until-triggers. If bit 17 is set to 1, the triggered position must be software captured, so bit 16 must also be set to 1 to specify software captured bit position.

See Also

Selecting the Position Loop Feedback (Setting Up a Motor)

Encoder Conversion Table (Setting Up a Motor)

I-variables Ix04, Ix05, Ix25, Ix83.

Ix04 Motor x Velocity Loop Feedback Address

When planning which channels to use when connecting the position and velocity encod-

Range: .....
Default: .....
Units: .....

ers, remember that the channel pointed to by Ix25 is used for the Overtravel, Amplifier Fault, and Home Flag inputs.

Remarks

This parameter holds the address of the *position* feedback device that PMAC uses for its velocity-loop feedback information. For a motor with only a single feedback device (the usual case), this must be the same as Ix03. For a motor with dual feedback (motor and load), use Ix04 to point to the encoder on the motor, and Ix03 to point to the encoder on the load.

If the velocity-loop feedback device is the same device as is used for commutation (if PMAC is doing the commutation), then both Ix04 and Ix83 (commutation feedback address) must reference the same device. However, Ix04 typically points to the "converted" data -- a register in the Encoder Conversion table -- while Ix83 must point directly to the DSPGATE encoder register.

The instructions for setting this parameter are identical to those for Ix03, except that there are no address extension bits.

See Also

Selecting the Velocity-Loop Feedback (Setting Up a Motor)

Encoder Conversion Table (Setting Up a Motor)

I-variables Ix03, Ix05, Ix25, Ix83.

Ix05 Motor x Master (Handwheel) Position Address

Range: .....

Default: .....$073F (1855) (= zero r

Units: .....

In the extended version, it is obviously easier to specify this parameter in hexadecimal form. With I9 at 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.
It is important not to have the same source be both the master and the feedback for an individual motor. If this is the case, with Ix06=1 to enable following, the motor will run away (it is like a puppy chasing its tail -- it cannot catch up to its desired position, because its desired position keeps moving ahead of it). This case can easily occur for motor 2 with the default values of I203 and I205 specifying the same address.

Remarks

This parameter tells the PMAC where to look for the position of the master, or handwheel, encoder for motor x. Usually this is an entry in the Encoder Conversion Table that holds processed information from an encoder channel. The instructions for setting this parameter are identical to those for Ix03, except the extended bits mean different things. The default value permits handwheel input from the JPAN connector (jumpered into the ENC2 counter with E22 and E23).

Following Modes: The source address of the position information occupies bits 0 to 15 of Ix05 (range $0000 to $FFFF, or 0 to 65535). With bit 16 equal to zero -- the normal case -- position following is done with the actual position reported for the motor reflecting the change due to the following. With bit 16 -- value 65536 -- equal to one, the actual position reported for the motor does not reflect the change due to the following ("offset" mode). This mode can be useful for part offsets, and for superimposing programmed and following moves. For example, to have motor #1 following encoder 2 in offset mode, I105 should be set to $10721.

If you want to ensure that following cannot occur by accident, you may want to change Ix05 so it points to a register that cannot change. This way, even if the following function is turned on, for instance by the motor selector inputs on the JPAN connector, no following can occur. The best registers to use for this purpose are the unused ones at the end of the conversion table. With the default table setup, you can choose any register between $072A and $073F (1834 to 1855 decimal). If you extend the table, choose a register between the end of the table and $073F.

See Also

Selecting the Master Position Source (Setting Up a Motor)

Encoder Conversion Table (Setting Up a Motor)

Position Following (Synchronizing PMAC to External Events)

I-variables Ix03, Ix04, Ix06, Ix08, Ix09.

Ix06 Motor x Master (Handwheel) Following Enable

This parameter can be changed on-line through hardware inputs on the JPAN connector. The FPDn/ motor/ coordinate-system select lines (low-true BCD-coded) can turn Ix06 on and off. On power-up or reset, if I2 was saved as zero, Ix06 for the selected motor is set to one and Ix06 for all other motors is set to zero regardless of the values that were saved. When the select switch is changed, Ix06 for the de-selected motor is set to zero and Ix06 for the selected motor is set to 1.

Range: .....
Default: .....
Units: .....

Remarks

This parameter disables or enables motor x's position following function. A value of 0 means disabled; a value of 1 means enabled. Following mode is specified by high bits of Ix05.

See Also

Position Following (Synchronizing PMAC to External Events)

I-variables I2, Ix05, Ix07, Ix08

Control Panel Inputs (Connecting PMAC to the Machine)

Ix07 Motor x Master (Handwheel) Scale Factor

Range: ..... -8,388,608 .. 8,388,607
Default: 96
Units: .....
..... none

Remarks

This parameter controls with what scaling the master (handwheel) encoder is extended into the full-length register. In combination with Ix08, it also controls the following ratio of motor x (delta-motor-x = [Ix07/Ix08] * delta-handwheel-x) for position following (electronic gearing). For following, Ix07 and Ix08 can be thought of as the number of teeth on meshing gears in a mechanical coupling. Ix07 can be changed on the fly to permit real-time changing of the following ratio, but Ix08 may not.

See Also

I-variables Ix05, Ix06, Ix08

Position Following (Synchronizing PMAC to External Events)

Ix08 Motor x Position Scale Factor

When changing this parameter, ensure the motor is disabled. Otherwise, a sudden jump will occur because the internal position registers will have changed. This means that this parameter should not be changed in the middle of an application. If a real-time change in the position-following "gear ratio" is desired, Ix07 should be changed.

Range: .....

Default: .....

Units: .....

Remarks

This parameter controls how the position encoder counter is extended into the full-length register. For most purposes, this is transparent to the user and does not need to be changed from the default.

There are two reasons that the user might want to change this from the default value. First, because it is involved in the "gear ratio" of the position following function -- the ratio is Ix07/Ix08 -- this might be changed (usually raised) to get a more precise ratio.

The second reason to change this parameter (usually lowering it) is to prevent internal saturation at very high gains or count rates (velocity). the PMAC filter will saturate when the velocity in counts/sec multiplied by Ix08 exceeds 768M (805,306,368). *This only happens in very rare applications -- the count rate must exceed 8.3 million counts per second before the default value of Ix08 gives a problem.*

In most practical cases, Ix08 should not be set above 1000 because higher values can make the servo filter saturate too easily. If Ix08 is changed, Ix30 should be changed inversely to keep the same servo performance (e.g. if Ix08 is doubled, Ix30 should be halved).

See Also

Position Following (Synchronizing PMAC to External Events)

I-variables Ix05, Ix06, Ix07, Ix09, Ix30

Ix09 Motor x Velocity Loop Scale Factor

Range: .....0 .. 8,388,607
Default:.....96
Units: .....
..... none

Remarks

This parameter controls how the encoder counter used to close the velocity servo loop is extended into the full-length register. For most purposes, this is transparent to the user and does not need to be changed from the default. This parameter should not be changed in the middle of an application, because it scales many internal values. If the same sensor is used to close both the position and velocity loops (Ix03=Ix04), Ix09 should be set equal to Ix08. If different sensors are used, Ix09 should be set such that the ratio of Ix09 to Ix08 is inversely proportional to the ratio of the velocity sensor resolution (at the load) to the position sensor resolution.

Example

If a 5000 line/inch (20,000 cts/in) linear encoder is used for position feedback, and a 500 line/rev (2000 cts/rev) rotary encoder is used for velocity loop feedback, and there is a 5-pitch screw, the effective resolution of the velocity encoder is 10,000 cts/in (2000x5), half of the position sensor resolution, so Ix09 should be set to twice Ix08.
If the value computed this way for Ix09 does not come to an integer, use the nearest integer value.

See Also

I-variables Ix03, Ix04, Ix08, Ix31

Dual-Feedback Systems (Setting Up a Motor)

Ix10 Motor x Power-On Servo Position Address



It is obviously easier to specify this parameter in hexadecimal form ($ prefix). If I9 is set to 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.

Range: .....Extended PM
Default: .....
Units: .....

Remarks

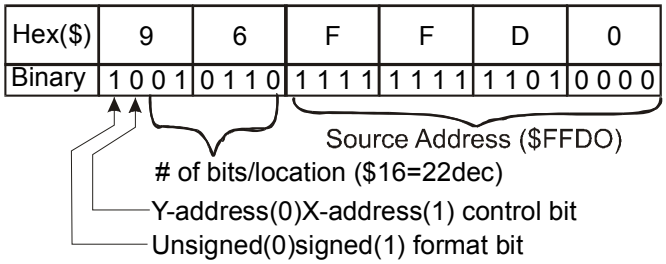
This parameter tells PMAC what address to read, and how to read it, for absolute power-on servo position information, if such information must come from a different address than the ongoing position information, which is specified by Ix03. It is set to zero if no special power-on position reading is desired, as is the case for an incremental encoder. Ix10 is used only at a full-board power-up/reset.
Ix10 consists of two parts. The low 16 bits, represented by four hex digits, contain the address of the register containing the power-on position information, either a PMAC memory-I/O address, or an address on the multiplexer (thumbwheel) port. The high 8 bits, represented by two hex digits, specify how to read the information at this address.
Resolver Feedback: If bits 16-21 contain a value from 0 to 7, the address specified in the low 16 bits is a multiplexer-port address, with a valid range of $0002 to $0100 (2 to 256 decimal), evenly divisible by 2. (The actual multiplexer port addresses have a range of 0 to 254 decimal, but in Ix10 a value of 256 must be used to represent address 0.) The multiplexer port is used for devices like the resolver-to-digital converters on ACC-8D Option 7. The value in bits 16-21 specifies the number (location) of the device at that multiplexer address; there are potentially 8 devices at each multiplexer address.
Parallel Feedback: If bits 16-21 contain a value from 8 to 48 ($08 to $30), the address specified in the low 16 bits is a PMAC memory-I/O address, usually the address of an ACC-14 register containing absolute encoder position. The value in bits 16-21 specifies the number of bits, starting at bit 0, to be used for the absolute power-on position information.
For bit counts of 8 through 24, the entire power-on position is contained in a single data word. For bit counts of 25 through 48, the most significant word must be at the next higher address, with bit 24 of the position information coming in at bit 0 of the next word. Since the position information comes from two separate data words read in sequence, a robust power-on procedure requires that the motor be physically constrained from moving at this time. Bit counts greater than 43 are not useful, because PMAC is limited to at most 43 bits of integer position data, even with the Ix08 position scale factor reduced to 1.
ACC-28 Analog Input: If bits 16 to 21 contain a value of 49 ($31), and bit 22 is 0, PMAC looks for parallel feedback in the *high* 16 bits of the specified PMAC Y-address. This is typically used for analog-to-digital converters from an ACC-28. An ACC-28A has signed data, so bit 23 should also be set (see below), making the first two hex digits of Ix10 equal to $B1. An ACC-28B has unsigned data, so bit 23 is 0 and the first two hex digits should be $31.

Yaskawa Absolute Encoder: If bits 16 to 21 contain a value of 49($31), and bit 22 is 1, PMAC looks for data from an ACC-8D Opt 9 Yaskawa encoder converter at the specified thumbwheel multiplexer port address. In this case, I9x must be set to the bits (n in 2^n counts) per revolution.

X/Y Address Bit: If bit 22 of Ix10 is 0, PMAC looks for the parallel sensor in its Y address space. The is the standard choice, since all I/O ports map into the Y address space. If this bit is 1, PMAC looks for the parallel sensor in its X address space.

Signed/Unsigned Bit: If the most significant bit (MSB -- bit 23) of Ix10 is 0, the value read from the absolute sensor is treated as an unsigned quantity. If the MSB is 1, which adds $80 to the high 8 bits of Ix10, the value read from the sensor is treated as a signed, two's-complement quantity.

Summarizing the format of the variable through an example, which specifies a 22-bit parallel encoder at Y:$FFD0, treated as a signed quantity:



For Ix10 to be useful, your sensor must be absolute over the entire range you are concerned with. This usually requires a multi-turn resolver or absolute encoder. If you only have a single-turn absolute sensor and you simply want absolute position within one revolution to eliminate the need for a power-on phasing search, you will utilize Ix81 for this purpose. Since a homing search is still required if the motor can go multiple turns in the application, Ix10 can be set to zero.

For a multi-turn absolute encoder, the data lines from the encoders must be combined into a single binary word so it appears to PMAC as if it came from a single encoder. For a multi-turn resolver, parameter I9x is used to specify the use of the second resolver, and I8x is used if there is a third resolver.

Example

Motor 1 has a single resolver at location 0 of an ACC-8D Opt.7 R/D converter board at multiplex address 0; no homing search is permitted; position is to be an unsigned value: I110=$000100 ($100=256dec, representing multiplex address 0). For a signed position value, I110=$800100.

Motor 2 has a single resolver at location 6 of an ACC-8D Opt 7 board at multiplex address 4; no homing search is permitted; unsigned position value: I210=$060004. For a signed position value, I210=$860004.

Motor 3 has a double resolver at locations 2 and 3 of an ACC-8D Opt 7 board at multiplex address 8, with a 10:1 gear ratio between them; unsigned position value: I310=$020008; I93=10. For a signed position value, I310=$820008, I93=10.

Motor 4 is a permanent-magnet brushless motor that has a single resolver at location 7 of an ACC-8D Opt 7 board at multiplex address 16; no phasing search is permitted, but a homing search will be required: I110=0, Ix81=$070010

Motor 5 has a 20-bit absolute encoder at Port A of the first ACC-14 (address Y:$FFD0), unsigned position value: I510=$14FFD0 ($14=20dec). For a signed position value, I510=$94FFD0

Motor 6 has a 32-bit absolute encoder with the low 24 bits on Port A of the second ACC-14 (Y:$FFD2), and the high 8 bits on the low 8 bits of Port B (Y:$FFD3; unsigned position value: I610=$20FFD2 ($20=32dec). For a signed position value, I610=$A0FFD2.

Motor 7 has an LVDT analog sensor read through an ACC-28A board at ADC7 (address Y:$C01E). I710=$B1C01E.

Motor 8 uses MLDT feedback through an ACC-29 at Timer 9 (Y:$C020) that can provide up to a 17-bit timer value. I810=$11C020.

See Also

Selecting the Position Loop Feedback (Setting Up a Motor)

I-Variables I8x, I9x, Ix03, Ix81

ACC-8D Option 7 (R/D Converter) Manual

Motor Safety I-Variables

Ix11 Motor x Fatal (Shutdown) Following Error Limit



The units of Ix11 are 1/16 of a count. Therefore this parameter must hold a value 16 times larger than the number of counts at which the limit will occur. For example, if the limit is to be 1000 counts, Ix11 should be set to 16,000.

Range: .....
Default: .....
Units: .....

Remarks

This parameter sets the magnitude of the following error for motor x at which operation will shut down. When the magnitude of the following error exceeds Ix11, motor x is disabled (killed). If the motor's coordinate system is executing a program at the time, the program is aborted. It is optional whether other PMAC motors are disabled when this motor exceeds its following error limit; bits 21 and 22 of Ix25 control what happens to the other motor (the default is that all PMAC motors are disabled).

A status bit for the motor, and one for the coordinate system (if the motor is in one) are set. If this coordinate system is hardware-selected on JPAN (with I2=0), or software-addressed by the host (with I2=1), the ERLD/ output on JPAN, and the EROR input to the interrupt controller (except for PMAC-VME) are triggered.

Setting Ix11 to zero disables the fatal following error limit for the motor. This may be desirable during initial development work, but it is *strongly discouraged* in an actual application. A fatal following error limit is a very important protection against various types of faults, such as loss of feedback, that cannot be detected directly, and that can cause severe damage to people and equipment.

See Also

I-variables I2, Ix12, Ix25

Following Error Limits, Amplifier Fault (Making Your Application Safe)

Control Panel Outputs (Connecting PMAC to the Machine)

Using Interrupts (Writing a Host Communications Program)

Memory registers Y:$0814, Y:08D4, etc., Y:$0817, Y:$08D7, etc.

On-line commands **?**, **??**.

Ix12 Motor x Warning Following Error Limit



The units of Ix12 are 1/16 of a count. Therefore this parameter must hold a value 16 times larger than the number of counts at which the limit will occur. For example, if the limit is to be 1000 counts, Ix12 should be set to 16,000.

Range: .....
Default: .....
Units: .....

Remarks

This parameter sets the magnitude of the following error for motor x at which a warning flag goes true. If this limit is exceeded, status bits are set for the motor and the motor's coordinate system (if any). The coordinate system status bit is the logical OR of the status bits of all the motors in the coordinate system.

Setting this parameter to zero disables the warning following error limit function. If this parameter is set greater than the *fatal* following error limit (Ix11), the warning status bit will never go true, because the fatal limit will disable the motor first.

If bit 17 of Ix03 is set to 1, the motor can be "triggered" for homing search moves, jog-until-trigger moves, and motion program move-until-trigger moves when the following error exceeds Ix12. This is known as "torque-mode" triggering, because the trigger will occur at a torque level corresponding to the Ix12 limit.

At any given time, one coordinate system's status bit can be output to several places; which system depends on what coordinate system is hardware-selected on the panel input port if I2=0, or what coordinate system is software-addressed from the host (&n) if I2=1. The outputs that work in this way are F1LD/ (pin 23 on connector J2), F1ER (line IR3 into the programmable interrupt controller (PIC) on PMAC-PC, line IR6 into the PIC on PMAC-STD) and, if E28 connects pins 1 and 2, FEFCO/ (on the JMACH connectors).

See Also

Control-panel (JPAN) output F1LD/, Interrupt line F1ER/

Following Error Limits (Making Your Application Safe)

Control Panel Outputs (Connecting PMAC to the Machine)

Torque-Mode Triggering (Basic Motor Moves)

Using Interrupts (Writing a Host Communications Program)

Memory registers Y:$0814, Y:08D4, etc., Y:$0817, Y:$08D7, etc.

I-variables I2, Ix11, Ix25;

On-line commands ?, ??, HOME, {jog}^{constant}.

Motion Program commands {axis}{data}^{data}

Ix13 Motor x Positive Software Position Limit

Range: ..... +/- 2^{47}

Default: 0

Units: .....
.....Encoder Counts

Remarks

This parameter sets the position for motor x which if exceeded in the positive direction causes a deceleration to a stop (controlled by Ix15) and allows no further positive position increments or positive output commands as long as the limit is exceeded. If this value is set to zero, there is no positive software limit (if you want 0 as a limit, use 1). This limit is automatically de-activated during homing search moves, until the home trigger is found. It is active during the post-trigger move.
Starting in firmware 1.15, bit 17 of Ix25 does not de-activate the software limits. Permanent de-activation is done by setting the value of the software limit to zero.
This limit is referenced to the most recent power-up zero position or homing move zero position. The physical position at which this limit occurs is not affected by axis offset commands (e.g. **`PSET, X=`**), although these commands will change the *reported* position value at which the limit occurs.

See Also

Hardware Overtravel Limits, Software Overtravel Limits (Making Your Application Safe)

I-variables Ix14, Ix15.

Ix14 Motor x Negative Software Position Limit

Range: ..... +/- 2^{47}

Default: ..... 0 (Disabled)

Units: .....
.....Encoder Counts

Remarks

This parameter sets the position for motor x which if exceeded in the negative direction causes a deceleration to a stop (controlled by Ix15) and allows no further negative position increments or negative output commands as long as the limit is exceeded. If this value is set to zero, there is no negative software limit (if you want 0 as a limit, use -1). This limit is automatically de-activated during homing search moves, until the trigger is found. It is active during the post-trigger move.
Starting in firmware 1.15, bit 17 of Ix25 does not de-activate the software limits. Permanent de-activation is done by setting the value of the software limit to zero.
This limit is referenced to the most recent power-up zero position or homing move zero position. The physical position at which this limit occurs is not affected by axis offset commands (e.g. **`PSET, X=`**), although these commands will change the *reported* position value at which the limit occurs.

See Also

Hardware Overtravel Limits, Software Overtravel Limits (Making Your Application Safe)

I-variables Ix13, Ix15.

Ix15 Motor x Deceleration Rate on Position Limit or Abort



Do not set this parameter to zero, or the motor will continue indefinitely after an abort or limit.

Range: .....

Default: .....

Units: .....

Remarks

This parameter sets the *rate* of deceleration that motor x will use if it exceeds a hardware or software limit, or has its motion aborted by command (**A or <CONTROL-A>**). This value should usually be set to a value near the maximum physical capability of the motor. It is not a good idea to set this value past the capability of the motor, because doing so increases the likelihood of exceeding the following error limit, which stops the braking action, and could allow the axis to coast into a hard stop.

Example

Suppose your motor had 125 encoder lines (500 counts) per millimeter, and you wished it to decelerate at 4000 mm/sec^2. You would set Ix15 to 4000 mm/sec^2 *500 cts/mm * sec^2/1,000,000 msec^2 = 2 cts/msec^2.

See Also

On-line commands A, <CONTROL-A>

Hardware Overtravel Limits, Software Overtravel Limits (Making Your Application Safe)

Ix16 Motor x Maximum Permitted Motor Program Velocity



If the PMAC circular interpolation function is used at all, then I13 must be greater than zero, and Ix16 will not be active as a velocity limit.

Range: .....

Default: .....

Units: .....

Remarks

This parameter sets a limit to the allowed velocity for **LINEAR** mode programmed moves for motor x, provided I13 equals zero (no move segmentation). If a blended move command in a motion program requests a higher velocity of this motor, all motors in the coordinate system are slowed down proportionately so that motor x will not exceed this parameter, yet the path will not be changed. This limit does not affect transition-point, circular, or splined moves. The calculation does not take into account any feedrate override (% value other than 100).

This parameter also sets the speed of a programmed **RAPID** mode move for the motor, if variable I50 is set to 1 (if I50 is set to 0, jog speed parameter Ix22 is used instead). This happens regardless of the setting of I13.

See Also

I-variables I13, I50, Ix17, Ix22

Velocity Limits (Making Your Application Safe)

LINEAR, RAPID-mode moves (Writing a Motion Program)

Ix17 Motor x Maximum Permitted Motor Program Acceleration



When moves are broken into small pieces and blended to-

Range: .....

gether, this limit can affect the velocity, because it limits the calculated deceleration for each piece, even if that deceleration is never executed, because it blends into the next piece.

If the PMAC circular interpolation function is used at all, then I13 must be greater than zero, and Ix17 will not be active as an acceleration limit.

Default: .....

Units:.....

Remarks

This parameter sets a limit to the allowed acceleration in **LINEAR**-mode blended programmed moves for motor x, provided I13 equals zero (no move segmentation). If a **LINEAR** move command in a motion program requests a higher acceleration of this motor given its TA and TS time settings, the acceleration for all motors in the coordinate system is stretched out proportionately so that motor x will not exceed this parameter, yet the path will not be changed.

Because PMAC cannot look ahead through an entire move sequence, it sometimes cannot anticipate enough to keep acceleration within this limit. Refer to LINEAR-mode trajectories in *Writing a Motion Program*.

It is possible to have this limit govern the acceleration for all LINEAR-mode moves by setting very low TA and TS times. *Do not set both the TA and TS times to zero, or a division-by-zero error will occur in the move calculations, possibly causing erratic movement.* The minimum acceleration time settings that should be used are TA1 with TS0.

This limit does not affect **PVT**, **CIRCLE**, **RAPID**, or **SPLINE** moves. The calculation does not take into account any feedrate override (% value other than 100).

Example

Given axis definitions of **#1->10000X**, **#2->10000Y**, an Ix17 for each motor of 0.25, and the following motion program segment:

```
INC F10 TA200 TS0

X20

Y20
```

the rate of acceleration from the program at the corner for motor #2 (X) is ((0-10)units/sec * 10000 cts/unit * sec/1000msec) / 200 msec = -0.5 cts/msec^2. The acceleration of motor #2 (Y) is +0.5 cts/msec^2. Since this is twice the limit, the acceleration will be slowed so that it takes 400 msec.

With the same setup parameters, and the following program segment:

```
INC F10 TA200 TS0

X20 Y20

X-20 Y20
```

the rate of acceleration from the program at the corner for motor #1 (X) is ((-7.07-7.07)units/sec * 10000 cts/unit * sec/1000msec) / 200 msec = -0.707 cts/msec^2. The acceleration of motor #2 (Y) is 0.0. Since motor #1 exceeds its limit the acceleration time will be lengthened to 200 * 0.707/0.25 = 707 msec.

Notice that in the second case, the acceleration time is made longer (the corner is made larger) for what is an identically shaped corner (90°). In a contouring XY application, this parameter should not be relied upon to produce consistently sized corners.

See Also

Acceleration Limits (Making Your Application Safe)

LINEAR-mode moves (Writing a Motion Program)
I-variables I13, I50, Ix16, Ix19, Ix22

Ix19 Motor x Maximum Permitted Motor Jog/Home Acceleration

Range: ..... positive. floating point
Default: 0.015625
Units: ..... counts/msec^2

Remarks

This parameter sets a limit to the commanded acceleration magnitude for jog and home moves, and for **RAPID**-mode programmed moves, of motor x. If the acceleration times in force at the time (Ix20 and Ix21) request a higher rate of acceleration, this rate of acceleration will be used instead. The calculation does not take into account any feedrate override (% value other than 100).

Since jogging moves are usually not coordinated between motors, many people prefer to specify jog acceleration by rate, not time. To do this, simply set Ix20 and Ix21 low enough that the Ix19 limit is always used. *Do not set both Ix20 and Ix21 to 0, or a division-by-zero error will result in the move calculations, possibly causing erratic operations*. The minimum acceleration *time* settings that should be used are Ix20=1 and Ix21=0.

The default limit of 0.015625 counts/msec^2 is quite low and will probably limit acceleration to a lower value than is desired in most systems; most users will eventually raise this limit. This low default was used for safety reasons.

Example

With Ix20 (accel time) at 100 msec, Ix21 (S-curve time) at 0, and Ix22 (jog speed) at 50 counts/msec, a jog command from stop would request an acceleration of (50 cts/msec) / 100 msec, or 0.5 cts/msec^2. If Ix19 were set to 0.25, the acceleration would be done in 200 msec, not 100 msec.

With the same parameters in force, an on-the-fly reversal from positive to negative jog would request an acceleration of (50-(-50) cts/msec) / 100 msec, or 1.0 cts/msec^2. The limit would extend this acceleration period by a factor of 4, to 400 msec.

See Also

Jogging and Homing Moves (Basic Motor Moves)

RAPID-mode moves (Writing a Motion Program)

I-variables I50, Ix16, Ix20, Ix21, Ix22

On-line commands **HM, J+, J-, J=, J^, J:, J/**

Motion program commands **HOME**, **RAPID**

Motor Movement I-Variables

Ix20 Motor x Jog/Home Acceleration Time

Do not set both Ix20 and Ix21 to 0 simultaneously, even if you are relying on Ix19 to limit your acceleration, or a division-by-zero error will occur in the jog move calculations, possibly resulting in erratic motion.

Range: .....
Default: .....
Units: .....

Remarks

This parameter establishes the time spent in acceleration in a jogging, homing, or programmed **RAPID**-mode move (starting, stopping, and changing speeds). However, if Ix21 (jog/home S-curve time) is greater than half this parameter, the total time spent in acceleration will be 2 times Ix21. Therefore, if Ix20 is set to 0, Ix21 alone controls the acceleration time in "pure" S-curve form. In addition, if the maximum acceleration rate set by these times exceeds what is permitted for the motor (Ix19), the time will be increased so that Ix19 is not exceeded.

A change in this parameter will not take effect until the next move command. For instance, if you wanted a different deceleration time from acceleration time in a jog move, you would specify the acceleration time, command the jog, change the deceleration time, then command the jog move again (e.g. **J=**), or at least the end of the jog (**J/**).

See Also

I-variables I50, Ix16, Ix19, Ix21, Ix22, Ix23, Ix87, Ix88

Jogging and Homing Moves (Basic Motor Moves)

RAPID-mode moves (Writing a Motion Program)

On-line commands HM, J+, J-, J=, J^, J:, J/

Motion program command HOME, RAPID

Ix21 Motor x Jog/Home S-Curve Time



Do not set both Ix20 and Ix21 to 0 simultaneously, even if you are relying on Ix19 to limit your acceleration, or a division-by-zero error will occur in the jog move calculations, possibly resulting in erratic motion.

Range: .....
Default: .....
Units: .....

Remarks

This parameter establishes the time spent in each "half" of the "S" for S-curve acceleration in a jogging, homing, or **RAPID**-mode move (starting, stopping, and changing speeds). If this parameter is more than half of Ix20, the total acceleration time will be 2 times Ix21, and the acceleration time will be "pure" S-curve (no constant acceleration portion). If the maximum acceleration rate set by Ix20 and Ix21 exceeds what is permitted for the motor (Ix19), the time will be increased so that Ix19 is not exceeded.

A change in this parameter will not take effect until the next move command. For instance, if you wanted a different deceleration time from acceleration time in a jog move, you would specify the acceleration time, command the jog, change the deceleration time, then command the jog move again (e.g. **J=**), or at least the end of the jog (**J/**)

See Also

I-variables I50, Ix16, Ix19, Ix20, Ix22, Ix23, Ix87, Ix88

Jogging and Homing Moves (Basic Motor Moves)

RAPID-mode moves (Writing a Motion Program)

On-line commands **HM**, **J+**, **J-**, **J=**, **J^**, **J:**, **J/**

Motion program commands **HOME**, **RAPID**

Ix22 Motor x Jog Speed

Range: ..... positive floating point
Default: 32.0
Units: .....
..... Counts / msec

Remarks

This parameter establishes the commanded speed of a jog move, or a programmed **RAPID**-mode move (if I50=0) for motor x. Direction of the jog move is controlled by the jog command.

A change in this parameter will not take effect until the next move command. For instance, if you wanted to change the jog speed on the fly, you would start the jog move, change this parameter, and then issue a new jog command.

See Also

I-variables I50, Ix19-Ix21
Jogging Moves (Basic Motor Moves)
RAPID-Mode Moves (Writing a Motion Program)
On-line commands J+, J-, J=, J^, J:, J/.
Program command RAPID

Ix23 Motor x Homing Speed and Direction

Range: ..... floating point
Default: 32.0
Units: ..... Counts / msec

Remarks

This parameter establishes the commanded speed and direction of a homing-search move for motor x. Changing the sign reverses the direction of the homing move -- a negative value specifies a home search in the negative direction; a positive value specifies the positive direction.

See Also

Homing Moves (Basic Motor Moves)
I-variables Ix19, Ix20 Ix21, Ix22, Ix25, Ix26
Encoder/Flag I-Variables 2 and 3
On-line command HM
Motion program command HOME.

Ix24 (Reserved For Future Use)

Ix25 Motor x Limit/Home Flag/Amp Flag Address (PMAC[1] Specific)

To use the PMAC Hardware Position Capture for homing search moves, the channel number of the flags specified by Ix25 must match the channel number of the encoder specified by Ix03 for position-loop feedback.
In the extended versions, it is obviously easier to specify this parameter in hexadecimal form. With I9 at 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.

Range: Extended legal PMAC 'X' addresses

Default:

VARIABLE	HEX	DECIMAL	LIMIT AND FLAG SET
I125	$C000	(49152)	(LIM1, HMFL1...)
I225	$C004	(49156)	(LIM2, HMFL2...)
I325	$C008	(49160)	(LIM3, HMFL3...)
I425	$C00C	(49164)	(LIM4, HMFL4...)
I525	$C010	(49168)	(LIM5, HMFL5...)
I625	$C014	(49172)	(LIM6, HMFL6...)
I725	$C018	(49176)	(LIM7, HMFL7...)
I825	$C01C	(49180)	(LIM8, HMFL8...)

Units: Extended legal PMAC 'X' addresses

Remarks

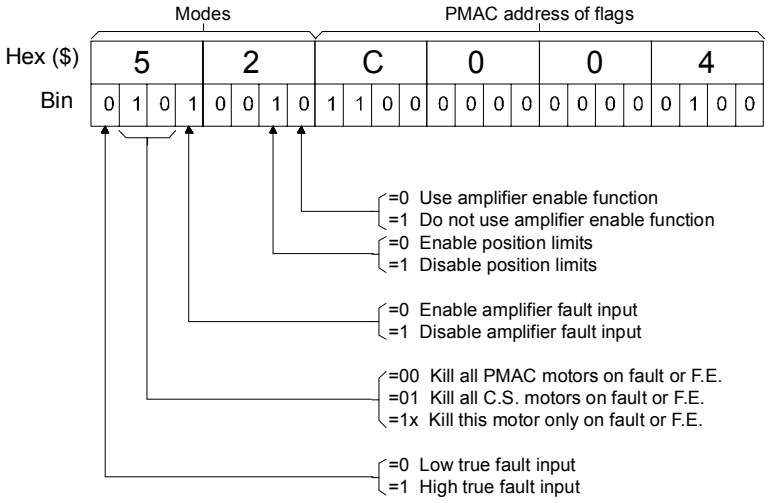
This parameter tells PMAC what set of flags it will look to for motor x's overtravel limit switches, home flag, amplifier-fault flag, amplifier-enable output, and index channel. Typically, these are the flags associated with an encoder input; specifically, those of the position feedback encoder for the motor. If dual-loop feedback is used (Ix03 and Ix04 are different) Ix25 should be set to match the position-loop encoder, not the velocity-loop.
The addresses for the first eight sets of inputs are given in the default table, above. The addresses for input sets 9 through 16, which are on the Accessory 24 Axis Expansion Board, are given below:

LIMIT AND FLAG SET	HEX	DECIMAL
LIM9, HMFL9	$C020	(49184)
LIM10, HMFL10	$C024	(49188)
LIM11, HMFL11	$C028	(49192)
LIM12, HMFL12	$C02C	(49196)
LIM13, HMFL13	$C030	(49200)
LIM14, HMFL14	$C034	(49204)
LIM15, HMFL15	$C038	(49208)
LIM16, HMFL16	$C03C	(49212)

The overtravel-limit inputs specified by this parameter must be held low in order for motor x to be able to command movement. The polarity of the amplifier-fault input is determined by a high-order bit of this parameter (see below). The polarity of the home-flag input is determined by the Encoder/Flag I-Variables 2 and 3 for the specified encoder. The polarity of the amplifier-enable output is determined by Jumper E17.
Extended Addressing: The source address of the flag information occupies bits 0 to 15 of Ix25 (range $0000 to $FFFF, or 0 to 65535). If this is all that is specified -- that is, all higher bits are zero -- then all of the flags are used, and used in the "normal" mode (low-true FAULT, disabling all motors). If higher bits are set to one, some of the flags are not used, or used in an alternate manner, as documented below.
Amplifier Enable Use Bit: With bit 16 equal to zero -- the normal case -- the AENAn/DIRn output is used as an amplifier-enable line: off when the motor is "killed", on when it is enabled. Voltage polarity is determined by jumper(s) E17.
If bit 16 (value $10000, or 65536) is set to one (e.g. I125=$1C000), this output is not used as an amplifier-enable line. This permits use of the line as a direction bit for applications requiring magnitude-and direction outputs, such as driving steppers through voltage-to-frequency converters. (Setting bit 16 of Ix02 to 1 enables use of this output as a direction bit.) General-purpose use of this output is also possible by assigning an M-variable to it.
Overtravel Limit Use Bit: With bit 17 equal to zero -- the normal case -- the +/-LIMn inputs must be held low to permit commanded motion in the appropriate direction. If there are not actual (normally closed or normally conducting) limit switches, the inputs must be hardwired to ground.
If bit 17 (value $20000, or 131072) is set to one (e.g. I125=$2C000), motor x does not use these inputs as overtravel limits. This can be done temporarily, as when using a limit as a homing flag. If the limit function is not used at all, these inputs can be used as general-purpose inputs by assigning M-variables to them.
Starting in firmware 1.15, bit 17 of Ix25 does not effect the software overtravel limits. Activation of the software overtravel limits is done by setting the value of Ix13 and or Ix14 to a non-zero value. De-activation is done by setting their values to zero.

The direction sense of the limit inputs is the opposite of what many people consider intuitive. That is, the +LIMn input, when taken high (opened), stops commanded motion in the negative direction; the -LIMn input, when taken high, stops commanded motion in the positive direction. It is important to confirm the direction sense of your limit inputs in actual operation.

Ix25 - Motor x Flag Address and Modes



Amplifier Fault Use Bit: If bit 20 of Ix25 is 0, the amplifier-fault input function through the FAULTn input is enabled. If bit 20 (value $100000, or 1,048,576) is 1 (e.g. I125=$10C000), this function is disabled. General-purpose use of this input is then possible by assigning an M-variable to the input.

Action-on-Fault Bits: Bits 21 (value $200000, or 2,097,152) and 22 (value $400000, or 4,194,344) of Ix25 control what action is taken on an amplifier fault for the motor, or on exceeding the fatal following error limit (Ix11) for the motor:

BIT 22	BIT 21	FUNCTION
Bit 22=0	Bit 21=0:	Kill all PMAC motors
Bit 22=0	Bit 21=1:	Kill all motors in same coordinate system
Bit 22=1	Bit 21=0:	Kill only this motor
Bit 22=1	Bit 21=1:	Kill only this motor

Regardless of the setting of these bits, a program running in the coordinate system of the offending motor will be halted on an amplifier fault or the exceeding of a fatal following error limit.

Amplifier-Fault Polarity Bit: Bit 23 (value 8,388,608) of Ix25 controls the polarity of the amplifier fault input. A zero in this bit means a low-true input (low means fault); a one means high-true (high means fault). The input is pulled high internally, so if no line is attached to the input, and bit 20 of Ix25 is zero (enabling the fault function), bit 23 of Ix25 must be zero to permit operation of the motor**.**

First Hex Digit: In the hexadecimal form, bits 20 to 23 combine to form a single hexadecimal digit. For reference, the possible values and their meanings are:

HEX DIGIT	FUNCTION
$0:	Low-true amp fault enabled; all motors killed on fault or excess following error (default)
$1:	Amp fault disabled; all motors killed on excess following error
$2:	Low-true amp fault enabled: coordinate system motors killed on fault or excess following error
$3:	Amp fault disabled; coordinate system motors killed on excess following error

HEX DIGIT	FUNCTION
$4:	Low-true amp fault enabled; only this motor killed on fault or excess following error
$5:	Amp fault disabled; only this motor killed on excess following error
$6:	Low-true amp fault enabled; only this motor killed on fault or excess following error
$7:	Amp fault disabled; only this motor killed on excess following error
$8:	High-true amp fault enabled; all motors killed on fault or excess following error (default)
$9:	Amp fault disabled; all motors killed on excess following error
$A:	High-true amp fault enabled: coordinate system motors killed on fault or excess following error
$B:	Amp fault disabled; coordinate system motors killed on excess following error
$C:	High-true amp fault enabled; only this motor killed on fault or excess following error
$D:	Amp fault disabled; only this motor killed on excess following error
$E:	High-true amp fault enabled; only this motor killed on fault or excess following error
$F:	Amp fault disabled; only this motor killed on excess following error

Example

1.....M
otor 1 using flags 1 with amp-enable output, and low-true amp fault disabling all motors: I125=$00C000 or I125=$C000

2.....M
otor 1 using flags 1 with direction output, and low-true amp fault disabling all motors: I125=$01C000

3.....M
otor 1 using flags 1 with amp-enable output, and low-true amp fault disabling only coordinate system motors: I125=$20C000

4.....M
otor 1 using flags 1 with direction output, and amp-fault disabled, with excess F.E. disabling all C.S. motors: I125=$31C000

5.....M
otor 1 using flags 5 with amp-enable output, and high-true amp fault disabling only this motor: I125=$C0C010

See Also

Selecting the Flag Register (Setting Up a Motor)

Homing Moves (Basic Motor Moves)

I-variables Ix02, Ix03 Ix11

Encoder/Flag I-Variables 2 and 3

Jumper(s) E17; JMACH connector flag I/O pins

Ix25 Motor x Flag Address (PMAC2 Specific)

Range/Units: Extended PMAC Addresses

Default values:

VARIABLE	NON-ULTRALITE	ULTRALITE
I125:	$C000 (Flags1)	$840F70 (Node 0)
I225:	$C008 (Flags2)	$840F71 (Node 1)
I325:	$C010 (Flags3)	$840F74 (Node 4)
I425:	$C018 (Flags4)	$840F75 (Node 5)
I525:	$C020 (Flags5)	$840F78 (Node 8)
I625:	$C028 (Flags6)	$840F79 (Node 9)
I725:	$C030 (Flags7)	$840F7C (Node 12)
I825:	$C038 (Flags8)	$840F7D (Node 13)

Remarks

This parameter tells PMAC2 what registers it will look to for motor x's overtravel limit inputs, home flag input, amplifier-fault input, and amplifier-enable output. Typically, these are the flags associated with an encoder input; specifically, those of the position feedback encoder for the motor. If you want to use the hardware capture feature for homing the motor, the flag register specified with Ix25 must have the same channel number as the encoder counter specified in Ix03 for position-loop (not necessarily velocity-loop) feedback.
The addresses for the eight standard flag registers are given in the default table, above.



In the extended versions, The overtravel-limit inputs specified by this parameter must read 0 in order for motor x to be able to command movement. With Delta Tau ACC-8x interface

it is obviously easier to specify this parameter in hexadecimal form. With I9 at 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.

boards, this means that they must be drawing current through them. The polarity of the amplifier-fault input is determined by bit 23 of Ix25 (see below). The polarity of the home-flag input is determined by I9n2 for Channel n. The polarity of the amplifier-enable output must be determined with the hardware interface.

Extended Addressing: The source address of the flag information occupies bits 0 to 15 of Ix25 (range $0000 to $FFFF, or 0 to 65535). If this is all that is specified -- that is, all higher bits are zero -- then all of the flags are used, and used in the "normal" mode (low-true FAULT, disabling all motors). If higher bits are set to one, some of the flags are not used, or used in an alternate manner, as documented below.

Amplifier Enable Use Bit: With bit 16 equal to zero -- the normal case -- the AENAn/DIRn output is used as an amplifier-enable line: off when the motor is "killed", on when it is enabled

If bit 16 (value $10000, or 65536) is set to one (e.g. I125=$1C000), this output is not used as an amplifier-enable line. General-purpose use of this output is then possible by assigning an M-variable to it.

Overtravel Limit Use Bit: With bit 17 equal to zero -- the normal case -- the PLIMn and MLIMn inputs must read 0 (drawing current) to permit commanded motion in the appropriate direction. If there are not actual (normally closed or normally conducting) limit switches, the inputs must be hardwired to ground.

If bit 17 (value $20000, or 131072) is set to one (e.g. I125=$2C000), motor x does not use these inputs as overtravel limits. This can be done temporarily, as when using a limit as a homing flag. If the limit function is not used at all, these inputs can be used as general-purpose inputs by assigning M-variables to them.

Macro Node Use Bit: When bit 18 of Ix25 is set to 1, PMAC2 will expect that the flag register be a MACRO auxiliary register. If a MACRO auxiliary node *n* is used for the flag register, then bit *n* of I1000 must be set so PMAC2 performs auxiliary node update functions for the node. Also, bit 23 of Ix25 must be set to 1 to designate a high-true amplifier fault, which is the MACRO standard. This makes the first two hex digits of Ix25 equal to $84.

When using a MACRO auxiliary register for the flags, the address part of Ix25 should contain the address of a holding register in RAM, not the actual MACRO register. The address of the holding register is $0F7n for node *n*. PMAC firmware automatically copies between the holding registers and the MACRO registers as enabled by I1000. When the flag information uses MACRO nodes, the following settings of Ix25 should be used:

NODE #	IX25	NODE #	IX25
Node 0	$840F70	Node 8	$840F78
Node 1	$840F71	Node 9	$840F79
Node 2	$840F72	Node 10	$840F7A
Node 3	$840F73	Node 11	$840F7B
Node 4	$840F74	Node 12	$840F7C
Node 5	$840F75	Node 13	$840F7D
Node 6	$840F76	Node 14	$840F7E
Node 7	$840F78	Node 15	$840F7F

Note that bit *n* of I1002 must be set properly to determine whether the Type 0 or Type 1 MACRO protocol is being used on MACRO node *n*. A value of 0 in I1002 bit *n* sets Type 0 protocol; a value of 1 sets Type 1.

Amplifier Fault Use Bit: If bit 20 of Ix25 is 0, the amplifier-fault input function through the FAULTn input is enabled. If bit 20 (value $100000, or 1,048,576) is 1 (e.g. I125=$10C000), this function is disabled. General-purpose use of this input is then possible by assigning an M-variable to the input.

Action-on-Fault Bits: Bits 21 (value $200000, or 2,097,152) and 22 (value $400000, or 4,194,344) of Ix25 control what action is taken on an amplifier fault for the motor, or on exceeding the fatal following error limit (Ix11) for the motor:

BIT 22	BIT 21	FUNCTION
Bit 22=0	Bit 21=0:	Kill all PMAC motors

Bit 22=0	Bit 21=1:	Kill all motors in same coordinate system
Bit 22=1	Bit 21=0:	Kill only this motor
Bit 22=1	Bit 21=1:	Kill only this motor

Regardless of the setting of these bits, a program running in the coordinate system of the offending motor will be halted on an amplifier fault or the exceeding of a fatal following error.

Amplifier-Fault Polarity Bit: Bit 23 (value 8,388,608) of Ix25 controls the polarity of the amplifier fault input. A zero in this bit means a low-true input (low means fault); a one means high-true (high means fault). The input is pulled high internally, so if no line is attached to the input, and bit 20 of Ix25 is zero (enabling the fault function), bit 23 of must be zero to permit operation of the motor.

See Also

Selecting the Flag Register (Setting Up a Motor)

Homing Moves (Basic Motor Moves)

I-variables Ix02, Ix03 Ix11

Encoder/Flag I-Variables 2 and 3

Jumper(s) E17; JMACH connector flag I/O pins

Ix26 Motor x Home Offset



The units of this parameter are 1/16 of a count, so the value should be 16 times the number of counts between the trigger position and the home zero position.

Range: .....
Default: .....
Units: .....

Remarks

This is the relative position of the end of the homing cycle to the position at which the home trigger was made. That is, the motor will command a stop at this distance from where it found the home flag(s), and call this commanded location as motor position zero.

This permits the motor zero position to be at a different location from the home trigger position, particularly useful when using an overtravel limit as a home flag (offsetting out of the limit before re-enabling the limit input as a limit). If large enough (greater than 1/2 times home speed times accel time) it permits a homing move without any reversal of direction.

Example

If you wish your motor zero position to be 500 counts in the negative direction from the home trigger position, you would set Ix26 to -500 * 16 = -8000.

See Also

Homing Moves (Basic Motor Moves)

I-variables Ix23, Ix25

Encoder I-Variables 2 and 3

On-line command HM

Program command HOME

Ix27 Motor x Position Rollover Range

Range: .....0 .. 8,388,607

Default:..... 0

Units: .....
..... Counts

Remarks

This parameter permits position rollover on a PMAC rotary axis by telling PMAC how many encoder counts are in one revolution of the rotary axis. This lets PMAC handle rollover properly. When Ix27 is greater than zero, and motor x is assigned to a rotary axis (A, B, or C), rollover is active. With rollover active, for a programmed axis move in Absolute (**ABS**) mode, the motor will take the shortest path around the circular range defined by Ix27 to get to the destination point.

Axis moves in Incremental (**INC**) mode are not affected by rollover. When Ix27 is set to 0, there is no rollover. Rollover should not be attempted for axes other than A, B, or C. Jog moves are not affected by rollover. Reported motor position is not affected by rollover. (To obtain motor position information "rolled over" to within one motor revolution, use the modulo (remainder) operator, either in PMAC or in the host computer: e.g.

```
P4=(M462/(I408*32))%I427)
```

Example

Motor #4 drives a rotary table with 36,000 counts per revolution. It is defined to the A-axis with **#4->100A** (A is in units of degrees). I427 is set to 36000. With motor #4 at zero counts (A-axis at zero degrees), an **A270** move in a program is executed in Absolute mode. Instead of moving the motor from 0 to 27,000 counts, which it would have done with I427=0, PMAC moves the motor from 0 to -9,000 counts, or -90 degrees, which is equivalent to +270 degrees on the rotary table.

See Also

On-line commands INC, ABS

Program commands INC, ABS, A{data}, B{data}, C{data}

Axis Types (Setting Up a Coordinate System)

Ix28 Motor x In-position Band



The units of this parameter are 1/16 of a count, so the value should be 16 times the number of counts in the in-position band.

Range: .....
Default: .....
Units:.....

Remarks

This is the magnitude of the maximum following error at which motor x will be considered in position when not performing a move. Several things happen when the motor is "in-position". First, a status bit in the motor status word is set. Second, if all other motors in the same coordinate system are also "in-position", a status bit in the coordinate system status word is set. Third, for the hardware-selected (FPD0/-FPD3/) coordinate system -- if I2=0 -- or for the software addressed (&n) coordinate system -- if I2=1 -- outputs to the control panel port and to the interrupt controller are set.

Technically, five conditions must be met for a motor to be considered "in-position":

5. the motor must be in closed-loop control

6. the desired velocity must be zero;

7. the magnitude of the following error must be less than this parameter;

8. the move timer must not be active;

9. the above four conditions must all be true for (I7+1) consecutive scans

The move timer is active during any programmed or non-programmed move, including **DWELL**s and **DELAY**s in a program -- if you wish this bit to come true during a program, you must do an indefinite wait between some moves by keeping the program trapped in a **WHILE** loop that has no moves or **DWELL**s. More sophisticated in-position functions (for instance, ones that require several consecutive scans within the band) can be implemented using PLC programs. See the program examples section.

Example

```
M140->Y:$0814,0; Motor 1 in-position bit
...
WHILE (M140=0) WAIT ; Delay indefinitely until in-position is true
M1=1 ; Set output once in-position
```

See Also

Control Panel Port (Connecting PMAC to the Machine)
Using Interrupts (Writing a Host Communications Program)
I-variable I7
On-line commands ?, ??
Suggested M-variable definitions Mx40, Mx87
Memory Registers Y:$0814, Y:$08D4, etc., Y:$0817, Y:$08D7, etc.
JPAN connector

Ix29 Motor x Output - or First Phase - DAC Bias (PMAC[1] Specific)

Range: ..... -32,768 .. 32,767

Default: 0

Units: .....
.....DAC Bits

Remarks

This parameter is the digital equivalent of an offset potentiometer on the analog output. It can be used to correct for differences in zero-reference between the PMAC analog output and the amplifier's analog input. This offset is active in both closed-loop and open-loop modes, even when the motor is killed.

For a motor not commutated by PMAC (Ix01=0), this is the value that is added onto the output of the servo algorithm or the open loop output value (including the zero output when the motor is killed) before it is sent to the DAC.

If the analog output is unidirectional (bit 16 of Ix02 is 1), this bias term is added before the absolute value function is performed. It is used if there is a directional bias on the motor. In this type of motor, Ix79 (offset after absolute value) is used to control output deadband or dithering.

For a PMAC-commutated motor (Ix01=1), this is the value that is added onto the B-phase output of the commutation algorithm. This is the DAC with the lower address (higher-numbered, i.e. DAC 2 of a DAC 1 and DAC 2 pair) of the adjacent DAC pair used for commutation. Ix79 is added onto the other phase output (higher-addressed, lower-numbered DAC) of the commutation algorithm. In addition to the primary use of compensating for analog offsets, it can be used in certain phasing search or phasing direction algorithms for permanent-magnet brushless motors, because it drives the motor like a stepper motor.

Ix29 can be used to create a torque offset for a motor not commutated by PMAC. For a motor commutated by PMAC, use the "Output Offset" register Y:$0045, etc., instead (it is also suitable for a motor not commutated by PMAC).

See Also

Permanent Magnet Brushless Motor (Setting Up PMAC Commutation)
I-variables Ix01, Ix02, Ix79.

Ix29 Motor x Output/First Phase Offset (PMAC2 Specific)

Range: ..... -32,768 - 32,767

Units: ..... 16-bit DAC/ADC bit equivalent

Default: 0

Ix29 serves as an output or feedback offset for Motor x; its exact use depends on the mode of operation as described below. In any of the modes, it effectively serves as the digital equivalent of an offset pot.

Mode 1: When PMAC2 is not commutating Motor x (Ix01=0), Ix29 serves as the offset for the single command output value, usually a DAC command. Ix29 is added to the output command value before it is written to the command output register.

Mode 2: When PMAC2 is commutating Motor x (Ix01=1) but not closing the current loop (Ix82=0), Ix29 serves as the offset for the first of two phase command output values (Phase A), for the address specified by Ix02; Ix79 serves the same purpose for the second phase. Ix29 is added to the output command value before it is written to the command output register.

As an output command offset, Ix29 is always in units of a 16-bit register, even if the actual output device is of a different resolution. For example, if a value of 60 had to be written into an 18-bit DAC to create a true zero command, this would be equivalent to a value of 60/4=15 in a 16-bit DAC, so Ix29 would be set to 15 to cancel the offset.

Mode 3: When PMAC2 is commutating (Ix01=1) and closing the current loop for Motor x (Ix82>0), Ix29 serves as an offset that is added to the phase current reading from the ADC for the first phase (Phase A), at the address specified by Ix82 minus 1. Ix79 performs the same function for the second phase. The sum of the ADC reading and Ix29 is used in the digital current loop algorithms.

Ix29 is always in units of a 16-bit ADC, even if the actual ADC is of a different resolution. For example, if a 12-bit ADC reported a value of -5 when no current was flowing in the phase, this would be equivalent to a value of -5*16=-80 in a 16-bit ADC, so Ix29 would be set to 80 to compensate for this offset.

Servo Control I-Variables



For a PMAC with Option 6, refer to the manual for Option 6 for descriptions of the variables in this range.

The servo control variables in the range Ix30-Ix69 have different meanings on a PMAC with the Option 6 Extended Servo Algorithm

Ix30 Motor x PID Proportional Gain



Changing the sign of Ix30 on a motor that has been closing a stable servo loop will cause an unstable servo loop, leading to a probable runaway condition.



The default value of 2000 for this parameter is exceedingly "weak" for most systems (all but the highest resolution velocity-loop systems), causing sluggish motion and/or following error failure. Most users will immediately want to raise this parameter significantly even before starting serious tuning.

Range: .....
Default: .....
Units: .....

Remarks

This term provides a control output proportional to the position error (commanded position minus actual position) of motor x. It acts effectively as an electronic spring. The higher Ix30 is, the stiffer the "spring" is. Too low a value will result in sluggish performance. Too high a value can cause a "buzz" from constant over-reaction to errors.

If Ix30 is set to a negative value, this has the effect of inverting the command output polarity for motors not commutated by PMAC, when compared to a positive value of the same magnitude. This can eliminate the need to exchange wires to get the desired polarity. On a motor that is commutated by PMAC, changing the sign of Ix30 has the effect of changing the commutation phase angle by 180°. Negative values of Ix30 cannot be used with the autotuning programs in the PMAC Executive program.

This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

If the servo update time is changed, Ix30 will have the same effect for the same numerical value. However, smaller update times (faster update rates) should permit higher values of Ix30 (stiffer systems) without instability problems.

See Also

PID Servo Filter (Closing the Servo Loop)

I-variables Ix31-Ix39

Tuning Instructions (PMAC Executive Program manual)

Ix31 Motor x PID Derivative Gain

Range: .....-8,388,608 .. 8,388,607
Default:..... 1280
Units: .....
.....(Ix30*Ix09)/2^{26} DAC bits/(Counts/cycle)

Remarks

This term subtracts an amount from the control output proportional to the measured velocity of motor x. It acts effectively as an electronic damper. The higher Ix31 is, the heavier the damping effect is.

If the motor is driving a properly tuned tachometer-loop (velocity) amplifier, the amplifier will provide sufficient damping, and Ix31 should be set to zero. If the motor is driving a current-loop (torque) amplifier, or if PMAC is commutating the motor, the amplifier will provide no damping, and Ix31 must be greater than zero to provide damping for stability.

On a typical system with a current-loop amplifier and the PMAC default servo update time (~440 μsec), an Ix31 value of 2000 to 3000 will provide a critically damped step response.

If the servo update time is changed, Ix31 must be changed proportionately in the opposite direction to keep the same damping effect. For instance, if the servo update time is cut in half, from 440 μsec to 220 μsec, Ix31 must be doubled to keep the same effect.

This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

See Also

I-variables Ix30, Ix32-Ix39

PID Servo Filter (Closing the Servo Loop)

Tuning Instructions (PMAC Executive Program manual)

Ix32 Motor x PID Velocity Feedforward Gain

Range: .....0 .. 8,388,607
Default:..... 1280
Units: .....
.....(Ix30*Ix08)/2^{26} DAC bits/(counts/cycle)

Remarks

This term adds an amount to the control output proportional to the desired velocity of motor x. It is intended to reduce tracking error due to the damping introduced by Ix31, analog tachometer feedback, or physical damping effects.

If the motor is driving a current-loop (torque) amplifier, Ix32 will usually be equal to (or slightly greater than) Ix31 to minimize tracking error. If the motor is driving a tachometer-loop (velocity) amplifier, Ix32 will typically be substantially greater than Ix31 to minimize tracking error.

If the servo update time is changed, Ix32 must be changed proportionately in the opposite direction to keep the same effect. For instance, if the servo update time is cut in half, from 440 μsec to 220 μsec, Ix32 must be doubled to keep the same effect.

This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

See Also

PID Servo Filter (Closing the Servo Loop)

I-variables Ix30-Ix31, Ix32-Ix39

Tuning Instructions (PMAC Executive Program manual)

Ix33 Motor x PID Integral Gain

Range: .....0 .. 8,388,607
Default:..... 0
Units: .....
.....(Ix30*Ix08)/2^{42} DAC bits/(counts*cycles)

Remarks

This term adds an amount to the control output proportional to the time integral of the position error for motor x. The magnitude of this integrated error is limited by Ix63. With Ix63 at a value of zero, the contribution of the integrator to the output is zero, regardless of the value of Ix33.

No further errors are added to the integrator if the output saturates (if output equals Ix69), and, if Ix34=1, when a move is being commanded (when desired velocity is not zero). In both of these cases, the contribution of the integrator to the output remains constant.

If the servo update time is changed, Ix33 must be changed proportionately in the same direction to keep the same effect. For instance, if the servo update time is cut in half, from 440 μsec to 220 μsec, Ix33 must be cut in half to keep the same effect.

This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

See Also

PID Servo Filter (Closing the Servo Loop)

I-variables Ix30-Ix32, Ix34-Ix39, Ix63, Ix69

Tuning Instructions (PMAC Executive Program manual)

Ix34 Motor x PID Integration Mode

Range: ..... 0 .. 1
Default: 1
Units: ..... none

Remarks

This parameter controls when the position-error integrator is turned on. If it is 1, position error integration is performed only when PMAC is not commanding a move (when desired velocity is zero). If it is 0, position error integration is performed all the time.
If Ix34 is 1, it is the *input* to the integrator that is turned off during a commanded move, which means the *output* control effort of the integrator is kept constant during this period (but is generally not zero). This same action takes place whenever the total control output saturates at the Ix69 value.
This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. When performing the feedforward tuning part of that utility, it is important to set Ix34 to 1 so the dynamic behavior of the system may be observed without integrator action. Ix34 may be changed on the fly at any time to create types of adaptive control.

See Also

PID Servo Filter (Closing the Servo Loop)

I-variables Ix30-Ix33, Ix35-Ix39, Ix63, Ix69

Tuning Instructions (PMAC Executive Program manual)

Ix35 Motor x PID Acceleration Feedforward Gain

Range: ..... 0 .. 8,388,607
Default: 0
Units: ..... (Ix30*Ix08)/2^{26} DAC bits/(counts/cycle^2)

Remarks

This term adds an amount to the control output proportional to the desired acceleration for motor x. It is intended to reduce tracking error due to inertial lag.
If the servo update time is changed, Ix35 must be changed by the inverse square to keep the same effect. For instance, if the servo update time is cut in half, from 440 μsec to 220 μsec, Ix35 must be quadrupled to keep the same effect.
This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

See Also

PID Servo Filter (Closing the Servo Loop)

I-variables Ix30-Ix34, Ix36-Ix39

Tuning Instructions (PMAC Executive Program manual)

Ix36 Motor x PID Notch Filter Coefficient N1

Range: ..... -2.0 .. +2.0
Default: 0
Units: ..... none (actual z-transform coefficient)

Remarks

This term, along with Ix37-Ix39, is part of the notch filter for motor x, whose purpose is to damp out a resonant mode in the system. This parameter can be set according to instructions in the Servo Loop Features section of the manual.
The notch filter parameters Ix36-Ix39 are 24-bit variables, with 1 sign bit, 1 integer bit, and 22 fractional bits, providing a range of -2.0 to +2.0.
The equation for the notch filter is:

$$F(z) = \frac{1 + N1z^{-1} + N2z^{-2}}{1 + D1z^{-1} + D2z^{-2}}$$

This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

See Also

Notch Filter (Closing the Servo Loop)
I-variables Ix30-Ix35, Ix37-Ix39
Tuning Instructions (PMAC Executive Program manual)

Ix37 Motor x PID Notch Filter Coefficient N2

Range: ..... -2.0 .. +2.0
Default: 0
Units: .....
..... none (actual z-transform coefficient)

Remarks

This term is part of the notch filter for motor x. See Ix36 and the Servo Loop Features section of the manual for details.
This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

See Also

Notch Filter (Closing the Servo Loop)
I-variables Ix30-Ix36, Ix38-Ix39
Tuning Instructions (PMAC Executive Program manual)

Ix38 Motor x PID Notch Filter Coefficient D1

Range: ..... -2.0 .. +2.0
Default: 0
Units: .....
..... none (actual z-transform coefficient)

Remarks

This term is part of the notch filter for motor x. See Ix36 and the Servo Loop Features section of the manual for details.

This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

See Also

Notch Filter (Closing the Servo Loop)

I-variables Ix30-Ix37, Ix39

Tuning Instructions (PMAC Executive Program manual)

Ix39 Motor x PID Notch Filter Coefficient D2

Range: ..... -2.0 .. +2.0
Default: 0
Units: .....
..... none (actual z-transform coefficient)

Remarks

This term is part of the notch filter for motor x. See Ix36 and the Servo Loop Features section of the manual for details.
This parameter is usually set initially using the Tuning utility in the PMAC Executive Program. It may be changed on the fly at any time to create types of adaptive control.

See Also

Notch Filter (Closing the Servo Loop)

I-variables Ix30-Ix38

Tuning Instructions (PMAC Executive Program manual)

Ix40 - Ix56 Motor x Extended Servo Algorithm I-Variables

These variables are only used the Option 6 Extended Servo Algorithm. Refer to the manual for the Extended Servo Algorithm and the ACC-25 Servo Evaluation Program for details.

Motor Servo Loop Modifiers

Ix57 Motor x Continuous Current Limit (PMAC[1] Specific)

Range: .....0 .. 32,767
Default: 0
Units: .....
..... Bits of a 16-bit DAC

Remarks

This parameter sets the maximum continuous current limit for the PMAC I^2T integrated current limiting function, when that function is active (Ix58 must be greater than 0 for I^2T to be active). If the commanded current from PMAC is above this value for a significant period, as set by Ix58, PMAC will trip this motor on an I^2T amplifier fault condition.
Ix57 is in units of a 16-bit DAC (maximum possible value of 32,767), even if the actual output device has a different resolution. Typically Ix57 will be set to between 1/3 and 1/2 of the Ix69 (instantaneous) output limit. Consult your amplifier and motor documentation for their specifications on instantaneous and continuous current limits.

See Also

Integrated Current Protection (Making Your Application Safe)
I-Variables Ix58, Ix69

Ix57 Motor x Continuous Current Limit (PMAC2 Specific)

Range: ..... 0 - 32,767
Units: .....
..... 16-bit DAC/ADC bit equivalent
Default: 0

Remarks

This parameter sets the maximum continuous current limit for PMAC's I^2T integrated current limiting function, when that function is active (Ix58 must be greater than 0 for I^2T to be active). If PMAC2 is closing a digital current loop for the motor, it uses actual current measurements for this function; otherwise it uses commanded current values If the actual or commanded current level from PMAC is above Ix57 for a significant period of time, as set by Ix58, PMAC will trip this motor on an I^2T amplifier fault condition.
Ix57 is in units of a 16-bit DAC (maximum possible value of 32,767), even if the actual output device has a different resolution. Typically Ix57 will be set to between 1/3 and 1/2 of the Ix69 (instantaneous) output limit. Consult your amplifier and motor documentation for their specifications on instantaneous and continuous current limits.

See Also

Integrated Current Protection (Making Your Application Safe)
I-Variables Ix58, Ix69

Ix58 Motor x Integrated Current Limit (PMAC[1] Specific)

Range: .....0 .. 8,388,607

Default:..... 0

Units: .....
..... 2^{30} (DAC bits)^2 • servo cycles

{bits of a 16-bit DAC}

Remarks

This parameter sets the maximum integrated current limit for the PMAC I2T integrated current limiting function. If Ix58 is 0, the I2T limiting function is disabled. If Ix58 is greater than 0, PMAC will compared the time-integrated difference between the squares of commanded current and the Ix57 continuous current limit to Ix58. If the integrated value exceeds Ix58, then PMAC faults the motor just as it would for receiving an amplifier fault signal, setting both the amplifier-fault and the I2T-fault motor status bits.
The Ix58 limit is typically set by taking the relationship between the instantaneous current limit (Ix69 on PMAC, in units of a 16-bit DAC), the magnetization current (Ix77; typically 0 except for vector control of induction motors) and the continuous current limit (Ix57 on PMAC, in units of a 16-bit DAC) and multiplying by the time permitted at the instantaneous limit. The formula is:

$$Ix58 = \frac{Ix69^2 + Ix77^2 - Ix57^2}{32768^2} * ServoUpdateRate(Hz) * PermittedTime(\text{sec})$$

Refer to the section Making Your Application Safe in the User's Guide for a more detailed explanation of I^2T protection.

Example

With the instantaneous current limit Ix69 at 32,767, the magnetization current Ix77 at 0, the continuous current limit Ix57 at 10,589 (1/3 of max), the time permitted with maximum current is at 1 minute, and the servo update rate at the default of 2.25 kHz, Ix58 would be set as

$$Ix58 = (1.0^2 + 0.0^2 - 0.33^2) * 2250 * 60 = 120000$$

See Also

Integrated Current Protection (Making Your Application Safe)

Ix58 Motor x Integrated Current Limit (PMAC2 Specific)

Range: ..... 0 - 8,388,607
Units: .....
.....2^{30} (DAC bits)^2 • servo cycles
.....{
.....bits of a 16-bit DAC}
Default: 0

Remarks

This parameter sets the maximum integrated current limit for PMAC's I^2T integrated current limiting function. If Ix58 is 0, the I^2T limiting function is disabled. If Ix58 is greater than 0, PMAC will compare the time-integrated difference between the squares of commanded or actual current and the Ix57 continuous current limit to Ix58. If the integrated value exceeds Ix58, then PMAC faults the motor just as it would for receiving an amplifier fault signal, setting both the amplifier-fault and the I^2T-fault motor status bits.

The Ix58 limit is typically set by taking the relationship between the instantaneous current limit (Ix69 on PMAC, in units of a 16-bit DAC), the magnetization current (commanded by Ix77; typically 0 except for vector control of induction motors) and the continuous current limit (Ix57 on PMAC, in units of a 16-bit DAC) and multiplying by the time permitted at the instantaneous limit. The formula is:

$$Ix58 = \frac{Ix69^2 + Ix77^2 - Ix57^2}{32768^2} * ServoUpdateRate(Hz) * PermittedTime(\text{sec})$$

Refer to the section Making Your Application Safe in the User's Guide for a more detailed explanation of I^2T protection.

Example

With the instantaneous current limit Ix69 at 32,767, the magnetization current Ix77 at 0, the continuous current limit Ix57 at 10,589 (1/3 of max), the time permitted with maximum current is at 1 minute, and the servo update rate at the default of 2.25 kHz, Ix58 would be set as

..... $Ix58 = (1.0^2 + 0.0^2 - 0.33^2) * 2250 * 60 = 120000$

See Also

Integrated Current Protection (Making Your Application Safe)

Ix59 Motor x User-Written Servo/Phase Enable

Range: ..... 0 .. 3

Default: 0

Units: ..... none

Remarks

This parameter controls whether the built-in servo and commutation routines, or user-written servo and commutation routines, are used for motor x.

Ix59	SERVO ALGORITHM	COMMUTATION ALGORITHM
0	Built-in	Built-in
1	User-written	Built-in
2	Built-in	User-written
3	User-written	User-written

Any user-written servo or commutation (phase) algorithms will have been coded and cross-assembled in a host computer, and downloaded into the PMAC program memory. These algorithms are retained by the battery on battery-backed RAM versions, or saved into flash memory on flash-backed versions.

Ix00 must be 1 in order for the user-written servo to execute. Ix01 must be 1 in order for the user-written commutation to execute. The servo algorithm can be changed immediately between the built-in algorithm and a user-written algorithm by changing Ix59. PMAC only selects the phasing algorithm to be used at power-on reset, so in order to change the commutation algorithm, Ix59 must be changed, this new value stored to non-volatile memory with the **SAVE** command, and the board reset.

It is possible to use the user-written algorithms for purposes other than servo or commutation, making them essentially very fast and efficient PLC programs. This is very useful for fast, position-based outputs. Simply load the code, activate an extra "motor" with Ix00 and/or Ix01, and set Ix59 for this pseudo-motor to use this algorithm.

See Also

User-Written Servo Instructions (Closing the Servo Loop)

User-Written Commutation Instructions (Setting Up PMAC Commutation)

I-Variables Ix00, Ix01

Ix60 Motor x Servo Cycle Period Extension (PMAC[1] Only)

Range: ..... 0 .. 255

Default: 0

Units: ..... Servo Interrupt Periods

Remarks

This parameter permits an extension of the servo update time for motor x beyond the servo interrupt period, which is controlled by hardware (E3-E6, E29-E33, E98, and master clock). The servo loop will be closed every (Ix60 + 1) servo interrupts. With the default value of zero, the loop will be closed every servo interrupt.
Other update times, including trajectory update, and phase update, are not affected by Ix60. I10 does not need to be changed with Ix60.
The velocity values reported for a motor with the **V** or **<CTRL-V>** command, and the actual velocity registers in regular memory or DPRAM, are affected by Ix60. They are reported in counts per servo loop closure, not counts per servo interrupt.

See Also

Servo Update Rate (Closing the Servo Loop)

On-line commands **<CTRL-V>**, **V**

M-Variable Mx66

Jumpers E3-E6, E29-E33, E98; I10.

Ix61 Motor x Current Loop Integral Gain (PMAC2 Only)

Range: ..... 0.0 - 1.0 (24-bit resolution)
Default: 0
Units: .....
..... Output = 8 * Ix61 * Sum [i=0 to n] (Icmd[i]-Iact[i])

Remarks

Ix61 is the integral gain term of the digital current loops, multiplying the difference between the commanded and actual current levels and adding the result into a running integrator that adds into the command output. It is only used if Ix82>0 to activate digital current loop execution.
Ix61 can be used with either Ix62 forward-path proportional gain, or Ix76 back-path proportional gain. If used with Ix62, the value can be quite low, because Ix62 provides the quick response, and Ix61 just needs to correct for biases. If used with Ix76, Ix61 is the only gain that responds directly to command changes, and it must be significantly higher to respond quickly. Ix61 is typically set using the current loop autotuner or interactive tuner in the PMAC Executive Program. Typical values of Ix61 are 0.02.

Ix62 Motor x Current Loop Proportional Gain (Forward Path) (PMAC2 Only)

Range: ..... 0.0 - 1.0 (24-bit resolution)
Default: 0
Units: .....
..... Output = 4 * Ix62 * (I_{cmd} - I_{act})

Remarks

Ix62 is the proportional gain term of the digital current loops that is in the "forward path" of the loop, multiplying the difference between the commanded and actual current levels. Either Ix62 or Ix76 (back path proportional gain) must be used to close the current loop. Generally, only one of these proportional gain terms is used, although both can be. Ix62 is only used if Ix82>0 to activate digital current loop execution.
Ix62 can provide more responsiveness to command changes from the position/velocity loop servo, and therefore a higher current loop bandwidth, than Ix76. However, if the command value is very noisy, which can be the case with a low-resolution position sensor, using Ix76 instead can provide better filtering of the noise.

Summary

Ix62 is typically set using the current loop autotuner or interactive tuner in the PMAC Executive Program. . Typical values of Ix62, when used, are around 0.5.

Ix63 Motor x Integration Limit

Range: .....-8,388,608 .. 8,388,607

Default:.....4,194,304

Units: .....
.....1/16 count

Remarks

This parameter limits the magnitude of the integrated position error (the *output* of the integrator), which can be useful for "anti-windup" protection, when the servo loop output saturates. The default value of Ix63 provides essentially no limitation. (The integral gain Ix33 controls how fast the error is integrated.)
A value of zero here forces a zero output of the integrator, effectively disabling the integration function in the PID filter. This can be useful during periods when you are applying a constant force and are expecting a steady-state position error. (In contrast, setting Ix33 to 0 prevents further inputs to the integrator, but maintains the output.)

The Ix63 integration limit can also be used to create a fault condition for the motor. If Ix63 is set to a negative number, then PMAC will also check as part of its following error safety check whether the magnitude of integrated following error has saturated at the magnitude of Ix63. With Ix63 negative, if the integrator has saturated, PMAC will trip (kill) the motor with a following error fault. Both the normal fatal following error motor status bit and the integrated following error status bit are set when this fault occurs. If Ix63 is 0 or positive, the motor cannot trip on integrated following error fault.
To set Ix63 to a value such that the integrator saturates at the same point that its contribution to the command output causes saturation at the Ix69 level, use the following formula:

$$Ix63 = \pm\left(\frac{Ix69*2}{Ix08*Ix}$$

.....

To cause trips, the magnitude of Ix63 must be set to less than this value due to other potential contributions to the output. Remember that the integrator stops increasing when the output saturates at Ix69.

See Also

PID Servo Filter (Closing the Servo Loop)

I-variables Ix33, Ix67, Ix69

Ix64 Motor x Deadband Gain Factor



Values of Ix64 less than -16 will cause negative gain inside the "deadband", making it impossible for the system to settle inside the band. These settings have no known useful function.

Range: .....

Default: .....

Units: .....

Remarks

This parameter is part of the PMAC feature known as "deadband compensation", which can be used to create or cancel deadband. It controls the effective gain within the deadband zone (see Ix65). When the following error is less than the value of Ix65, the proportional gain (Ix30) is multiplied by (Ix64+16)/16. At a value of -16, Ix64 provides true deadband. Values between -16 and 0 yield reduced gain within the "deadband". Ix64 = 0 disables any deadband effect.

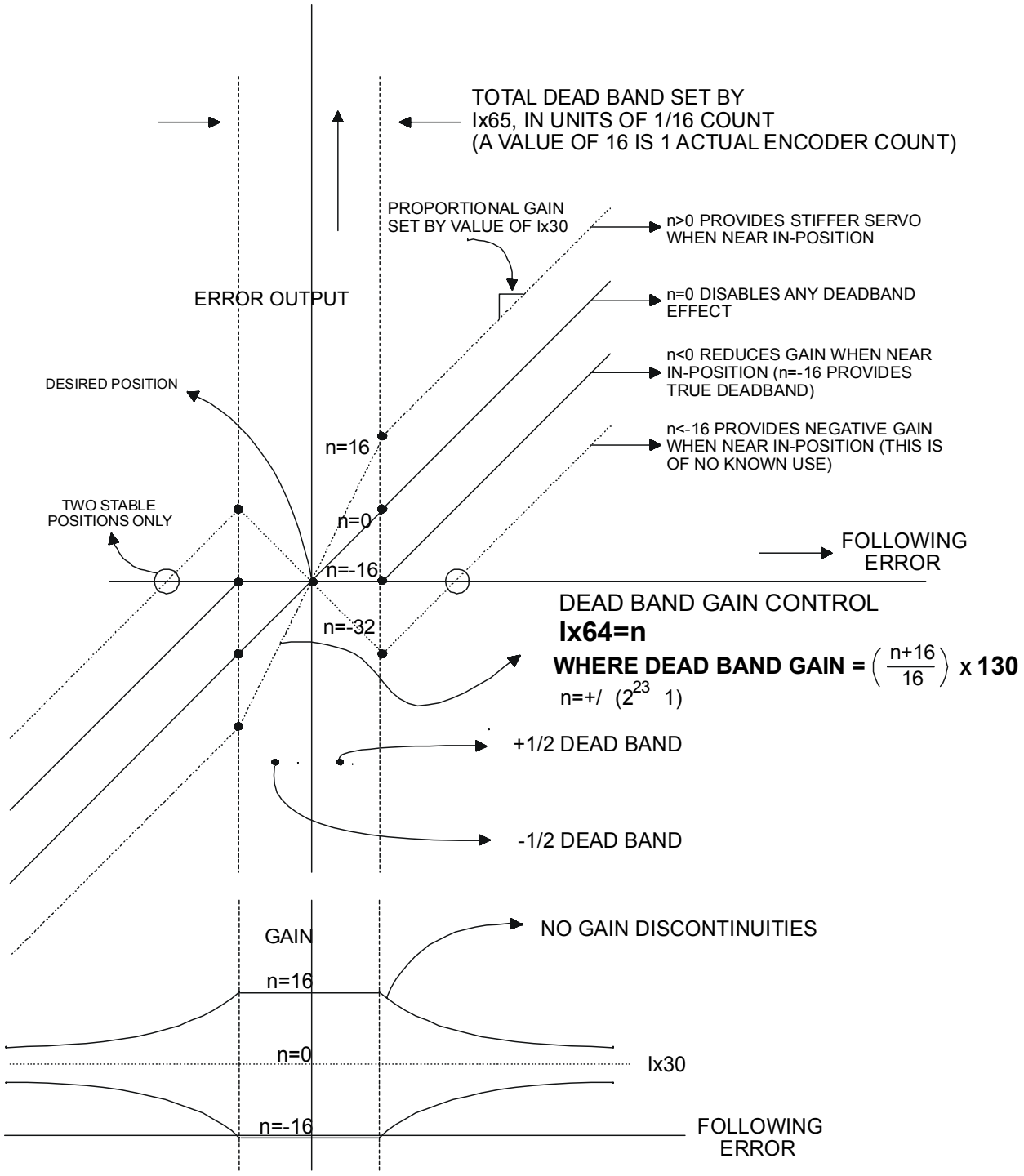
Values of Ix64 greater than 0 yield increased gain within the "deadband"; a value of 16 provides double gain in the "deadband". A small band of increased gain can be used to reduce errors while holding position, without as much of a threat to make the system unstable. It is also useful in compensating for physical deadband in the system.

Outside the deadband, gain asymptotically approaches Ix30 as the following error increases.

See Also

I-variables Ix30, Ix65

Closing The Servo Loop


TOTAL DEAD BAND SET BY
Ix65, IN UNITS OF 1/16 COUNT
(A VALUE OF 16 IS 1 ACTUAL ENCODER COUNT)
PROPORTIONAL GAIN
SET BY VALUE OF Ix30
n>0 PROVIDES STIFFER SERVO
WHEN NEAR IN-POSITION
ERROR OUTPUT
n=0 DISABLES ANY DEADBAND
EFFECT
n<0 REDUCES GAIN WHEN NEAR
IN-POSITION (n=-16 PROVIDES
TRUE DEADBAND)
DESIRED POSITION
n=16
n<-16 PROVIDES NEGATIVE GAIN
WHEN NEAR IN-POSITION (THIS IS
OF NO KNOWN USE)
TWO STABLE
POSITIONS ONLY
n=0
FOLLOWING
ERROR
n=-16
DEAD BAND GAIN CONTROL
Ix64=n
n=-32
WHERE DEAD BAND GAIN = (n+16 / 16) x 130
n=+/ (2^23 1)
+1/2 DEAD BAND
-1/2 DEAD BAND
GAIN
NO GAIN DISCONTINUITIES
n=16
n=0
Ix30
n=-16
FOLLOWING
ERROR

Ix65 Motor x Deadband Size

The units of this parameter are 1/16 of a count, so the value should be 16 times the number of counts in the deadband. For example, if modified gain is desired in the range of +/- 5 counts of following error, Ix65 should be set to 80.

Range: .....

Default: .....

Units: .....

Remarks

This term defines the size of the position error band, measured from zero error, within which there will be changed or no control effort, for the PMAC feature known as "deadband compensation". Ix64 controls the effective gain relative to Ix30 within the "deadband".

See Also

Deadband (Closing the Servo Loop)

I-variables Ix30, Ix64

Ix66 Motor x PWM Scale Factor (PMAC2 Only)

Range: ..... 0 - 32767

Default:

Units: .....

..... PWM_CLK cycles

Remarks

Ix66 multiplies the output of the digital current loops for Motor x (which are values between -1.0 and 1.0) before they are written to the PWM output registers. As such, it determines the maximum value that can be written to the PWM output register. Ix66 is only used if Ix82>0 to activate digital current loop execution.

The PWM output value for each phase is compared digitally to the PWM up-down counter, which increments or decrements once per PWM_CLK cycle to determine whether the outputs are on or off. The limits of the up-down counter are I900+1 and -I900-2 for channels 1 to 4; I906+1 and -I906-2 for channels 5 to 8.

Generally, Ix66 is set to about 10% above I900 (or I906) for motors commutated by PMAC2. This permits a full-on command of the phase for a substantial fraction of the commutation cycle, providing maximum possible utilization of the power devices at maximum command. If Ix66 is set to a smaller value than I900 or I906, it serves as a voltage limit for the motor (Vmax = [Ix66 / I900]*VDC). Ix69 serves as the current limit.

Ix67 Motor x Linear Position Error Limit

The units of this parameter are 1/16 of a count, so the value should be 16 times the number of counts in the limit

Range: .....

Default: .....

Units: .....

Remarks

This term defines the biggest position error that will be allowed into the servo filter. This is intended to keep extreme conditions from upsetting the stability of the filter. However, if it is set too low, it can limit the response of the system to legitimate commands (this situation can particularly be noticed on very fine resolution systems).

This parameter is not to be confused with Ix11 or Ix12, the following error limits. Those parameters act outside the servo loop based on the real (before limiting) following error.

See Also

I-variables Ix11, Ix12, Ix68

Ix68 Motor x Friction Feedforward

This direction-sensitive bias term is independent of the constant bias introduced by Ix29 and/or Ix79.

Range: .....

Default: .....

Units: .....

Remarks

This parameter adds a bias term to the servo loop output of motor x that is proportional to the *sign* of the commanded velocity. That is, if the commanded velocity is positive, Ix68 is added to the output. If the commanded velocity is negative, Ix68 is subtracted from the output. If the commanded velocity is zero, no value is added to or subtracted from the output.

This parameter is intended primarily to help overcome errors due to mechanical friction. It can be thought of as a "friction feedforward" term. Because it is a feedforward term that does not utilize any feedback information, it has no direct effect on system stability. It can be used to correct the error resulting from friction, especially on turn-around, without the time constant and potential stability problems of integral gain.

If PMAC is commutating this motor, the correction is applied before the commutation algorithm and will affect the magnitude of both analog outputs.

Example

Starting with a motor at rest, if Ix68 = 1600, then as soon as a commanded move in the positive direction is started, a value of +1600 (~0.5V) is added to the servo loop output. As soon as the commanded velocity goes negative, a value of -1600 is added to the output. When the commanded velocity becomes zero again, no bias is added to the servo output because of this term.

See Also

Closing the Servo Loop

I-Variables Ix01, Ix29, Ix32, Ix35, Ix79

Ix69 Motor x Output Command (DAC) Limit (PMAC[1] Specific)

When using PMAC to do internal open-loop microstepping (using its own commutation algorithms, not external V/F converters), the servo loop is writing to an internal register, not directly to the DACs. In this case we can allow more than a +/-32K limit. The value of Ix69 that should be used for this microstepping is 524,287 (2^{19}-1).

Range: .....

0 .. 524,287 (direct microstepping mode only)

Default: .....

Units: .....

Remarks

This parameter defines the magnitude of the largest output that can be sent from the control loop. If a larger value is calculated, it is clipped to this number. The analog outputs on PMAC are 16-bit DACs, which map a numerical range of -32,768 to +32,767 into a voltage range of -10V to +10V relative to analog ground (AGND).

If you are using differential outputs (DAC+ and DAC-), the voltage between the two outputs is twice the voltage between an output and AGND. (If you wish to limit the voltage between DAC+ and DAC- to 10V, Ix69 should be 16,384.)
This parameter provides a torque limit in systems with current- loop amplifiers, or a velocity limit with tachometer-based amplifiers. Notice that if this limit "kicks in" for any amount of time, the following error will start increasing. When Ix69 is actually limiting the output, the integrator in the PID loop will turn off for anti-windup protection.

See Also

I-variables Ix63, Ix67

Ix69 Motor x Output Command Limit (PMAC2 Specific)

Range: ..... 0 .. 32,767 (0 to 10V or equivalent)
0 .. 524,287 when in direct microstepping mode

Default: ..... 20,480 (6.25V or equivalent)
Units: ..... 16-bit DAC bits

Remarks

If the Ix77 magnetization current for the motor is set to a value other than 0, Ix69 should be set such that

Ix69 defines the magnitude of the largest output that can be sent from PMAC2's PID position/velocity servo loop. If the servo loop computes a larger value, PMAC2 clips it to this value. Ix69 is always in units of a 16-bit DAC, even if the actual output device is of a different resolution, or the command value is used for PMAC2's own internal current loop commands.
If you are using differential analog outputs (DAC+ and DAC-), the voltage between the two outputs is twice the voltage between an output and AGND, so the Ix69 value should be set to half of what it would be for a single-ended analog output.

When using PMAC2 to do internal direct open-loop microstepping (using its own commutation algorithms, not the PFM pulse generators) with Ix01=1 and Ix02 bit 16 = 1, then Ix69 should be set to 524,287 (2^{19}-1).

This parameter provides a torque (current) limit in systems with current-loop amplifiers, or when using PMAC's internal commutation; it provides a velocity limit with velocity-mode amplifiers. Note that if this limit "kicks in" for any amount of time, the following error will start increasing. When Ix69 is actually limiting the output, the integrator in the PID loop will turn off for anti-windup protection.

$$\sqrt{Ix69^2 + Ix77^2} \leq 32{,}767$$

Examples:

With DACs of any resolution that have a +/-10V range with respect to AGND, to limit the voltage between DAC+ and DAC- to +/-10V, each DAC signal must be limited to +/-5V and Ix69 should be set to 16,384.

See Also

I-variables Ix63, Ix67

Commutation I-Variables

Ix70 Motor x Number of Commutation Cycles (N) (PMAC[1] Specific)

Range: ..... 0 .. 255

Default: 1

Units: .....
..... Commutation cycles

Remarks

For a PMAC-commutated motor (Ix01=1), this parameter is used in combination with Ix71 to define the size of the commutation cycle, in encoder counts, as Ix71/Ix70. Usually, this is set to one, and Ix71 represents the number of counts in a single commutation cycle. Ix70 only needs to be set greater than one if the number of counts in a single cycle is not an integer.

A commutation cycle, or electrical cycle, consists of two poles (one pole pair) of a multiphase motor.

Example

A 6-pole brushless motor has three commutation cycles per mechanical revolution. If a feedback device is used with 4096 counts per mechanical revolution (a number not divisible by three), Ix70 should be set to 3, and Ix71 to 4096.

See Also

I-variables Ix01, Ix71-Ix83
Setting Up PMAC Commutation

Ix70 Motor x Number of Commutation Cycles (N) (PMAC2 Specific)

Range: ..... 0 .. 255
Default: 1
Units: .....
..... Commutation Cycles

Remarks

For a PMAC-commutated motor (Ix01=1), this parameter is used in combination with Ix71 to define the size of the commutation cycle. Usually, this is set to one, and represents the number of counts in a single commutation cycle. Ix70 only needs to be set greater than one if the number of counts in a single cycle is not an integer.
A commutation cycle, or electrical cycle, consists of two poles (one pole pair) of a multiphase motor.

Example

A 6-pole brushless motor has three commutation cycles per mechanical revolution. If a feedback device is used with 4096 counts per mechanical revolution (a number not divisible by three), Ix70 should be set to 3, and Ix71 to 4096.

See Also

I-variables Ix01, Ix71-Ix83

Setting Up PMAC Commutation

Ix71 Motor x Encoder Counts per N Commutation Cycles (PMAC[1] Specific)

Range: .....0 .. 8,388,607
Default:..... 1000
Units: .....
..... Counts

Remarks

For a PMAC-commutated motor, this parameter defines the size of a commutation cycle in conjunction with Ix70 (counts/cycle = Ix71/Ix70). The meaning of a "count" used in this parameter is defined by the encoder-decode variable for the commutation feedback device (Encoder I-Variable 0; I900, I905, etc.). If a "times-4" decode is used, a *count* is one-fourth of an encoder *line*. A commutation cycle, or electrical cycle, consists of two poles (one pole pair) of a multiphase motor.

Example

A four-pole brushless motor with a 1000-line-per-revolution encoder and "times-4" decode has 2 commutation cycles per revolution and 4000 counts per revolution. Therefore, either Ix70=2 and Ix71=4000 could be used, or Ix70=1 and Ix71=2000.

See Also

I-variables Ix01, Ix70, Ix72-Ix83
Setting Up PMAC Commutation

Ix71 Motor x Counts per N Commutation Cycles (PMAC2 Specific)

Range: .....0 .. 8,388,607
Default: 1000
Units: .....
..... counts

Remarks

For a PMAC2-commutated motor, this parameter defines the size of a commutation cycle in conjunction with Ix70 (counts/cycle = Ix71/Ix70). The meaning of a "count" used in this parameter is defined by the encoder-decode variable I9n0 for the commutation feedback device. If a "times-4" decode is used, a *count* is one-fourth of an encoder *line*.
A commutation cycle, or electrical cycle, consists of two poles (one pole pair) of a multiphase motor.

Example

A four-pole brushless motor with a 1000-line-per-revolution encoder and "times-4" decode has 2 commutation cycles per revolution and 4000 counts per revolution. Therefore, either Ix70=2 and Ix71=4000 could be used, or Ix70=1 and Ix71=2000.

See Also

I-variables Ix01, Ix70, Ix72-Ix83
Setting Up PMAC Commutation

Ix72 Motor x Commutation Phase Angle (PMAC[1] Specific)

Range: ..... 0 .. 255
Default: ..... 85 (=120° e)
Units: .....
..... 360/256 elec. deg. (1/256 commutation cycle)

Remarks

This parameter sets the angular distance between the phases of a multiphase motor. The usual values to be used are:
..... 3-phase: 85 or 171 (+/- 120°e)
..... 2- or 4-phase: 64 or 192 (+/- 90°e)

Changing between the two values for a given phase number has the same effect as exchanging motor leads, which changes the motor's direction of rotation for a given sign of PMAC torque command.
The output direction sense determined by this parameter and the motor wiring must match the feedback direction sense as determined by the encoder-decode variable (Encoder I-variable 0; I900, I905, etc.) and the encoder wiring. If the senses do not match, the motor will lock in rather than generate continuous torque, and proper commutation and servo control will be impossible.

See Also

I-variables Ix70, Ix71

Encoder I-Variable 0

Setting Up PMAC Commutation

Getting Started Section, Setting Up A PMAC-Commutated Motor

Ix72 Motor x Commutation Phase Angle (PMAC2 Specific)

Range: ..... 0 .. 255
Default: ..... 85 (=120°e)
Units: .....
..... 360/256 elec. deg. (1/256 commutation cycle)

Remarks

For a PMAC2-commutated motor, this parameter set the angular distance between the phases of a multiphase motor. The usual values to be used are:
..... 3-phase:
..... 85 or 171 (+/- 120°e)
..... 2- or 4-phase:
..... 64 or 192 (+/- 90°e)

PMAC2 Performing Commutation, But Not Current Loop

When PMAC2 *is not* performing digital current loop closure for Motor x (Ix82=0), the output direction sense determined by this parameter and the motor and amplifier phase wiring must match the feedback direction sense as determined by the encoder-decode variable I9n0 and the encoder wiring. If the direction senses do not match proper commutation and servo control will be impossible; the motor will lock into a given position.
For these systems, changing between the two values for a given number of phases has the same effect as exchanging motor leads, which changes the motor's direction of rotation for a given sign of a PMAC2 torque command.

Refer to the section *Setting Up PMAC2 Commutation* for tests to determine the proper Ix72 setting. For systems without PMAC2 digital current loop closure, once this commutation/feedback polarity has been properly matched, the servo/feedback polarity will automatically be properly matched.

PMAC2 Performing Commutation And Current Loop

When PMAC2 *is* performing digital current loop closure for Motor x (Ix82>0), the output direction sense determined by this parameter must match the polarity of the phase current sensors and the analog-to-digital conversion (ADC) circuitry that brings this data into PMAC2. It is independent of motor or amplifier phase wiring, encoder wiring, and PMAC2 encoder-decode direction sense.

CAUTION *Do not attempt to close the digital current loops on PMAC2 (O commands or closing the position loop) until you are sure of the proper sense of the Ix72 setting. An Ix72 setting of the wrong sense will cause positive feedback in the current loop, leading to saturation of the PMAC outputs and possible damage to the motor and or amplifier.*

For these systems with a PMAC2 digital current loop, if the phase-current ADC registers report a positive value for current flowing *into* the phase (i.e. the PWM voltage command value and the current feedback value have the same sign), Ix72 must be set to a value greater than 128 (usually 171 for a 3-phase motor, or 192 for a 2- or 4-phase motor). If the phase-current ADC registers report a positive value for current flowing *out of* the phase (i.e. the PWM voltage command value and the current feedback value have opposite signs), Ix72 must be set to a value less than 128 (usually 85 for a 3-phase motor, or 64 for a 2- or 4-phase motor).

For systems with PMAC2 digital current loop closure, the commutation/feedback polarity match is independent of the servo/feedback polarity. Once Ix72 has been set for proper commutation/feedback polarity, the proper position-loop servo/feedback polarity must still be established.

Ix73 Motor x Phase Finding Output Value (PMAC[1] Specific)

Range: .....
Default:.....
Units: .....

WARNING *An unreliable phasing search method can lead to a runaway condition. Test your phasing search method carefully to make sure it works properly under all conceivable conditions. Make sure your Ix11 fatal following error limit is active and as tight as possible so the motor will be killed quickly in case of a serious phasing search error.*

Remarks

This parameter defines the magnitude of the open-loop output to be used if a power-on phasing search is done for a PMAC-commutated motor (Ix01=1). A phasing search is required for a synchronous motor (Ix78=0) such as a permanent-magnet brushless motor with no absolute position sensor (Ix81=0). The phasing search is done automatically as part of the power-on phasing search if Ix80 is 1 or 3; if Ix80 is 0 or 2, the on-line **$** command must be used must be used to initiate the phasing search.

If Ix80 is 0 or 1, the “two-guess” phasing search is used, and Ix73 controls the “vector” magnitude of the open-loop output that is distributed among the phases according to the guessed phasing angle. If Ix80 is 2 or 3, the “stepper-motor” phasing search is used, and Ix73 controls the magnitude of current forced into individual phase(s) to lock the motor to a position like a stepper motor.

Values of 2000 to 6000 are typically used for Ix73 in either method.

See Also

Power-Up Phasing Search (Setting Up PMAC Commutation)

I-Variables Ix01, Ix74, Ix78, Ix80, Ix81

Ix73 Motor x Phase Finding Output Value (PMAC2 Specific)

Range: ..... -32,768 .. 32,767
Default: 0
Units: .....
..... bits of 16-bit DAC

Remarks



An unreliable phasing search method can lead to a runaway condition. Test your phasing search method carefully to make sure it works properly under all conceivable conditions. Make sure your Ix11 fatal following error limit is active and as tight as possible so the motor will be killed quickly in the event of a serious phasing search error.

This parameter defines the magnitude of the open-loop output to be used if a power-on phasing search is done for a PMAC2-commutated motor (Ix01=1). A phasing search is required for a synchronous motor (Ix78=0) such as a permanent-magnet brushless motor with no absolute position sensor (Ix81=0). The phasing search is done automatically as part of the power-on phasing search if Ix80 is 1 or 3; if Ix80 is 0 or 2, the on-line **$** command must be used must be used to initiate the phasing search.

Ix73 is in units of a 16-bit DAC, so that 32,767 represents full current command to the phases, even if a different output device and/or different resolution is used.

If Ix80 is 0 or 1, the "two-guess" phasing search is used, and Ix73 controls the "vector" magnitude of the open-loop output that is distributed among the phases according to the guessed phasing angle.

If Ix80 is 2 or 3, the "stepper-motor" phasing search is used, and Ix73 controls the magnitude of current forced into individual phase(s) to lock the motor to a position like a stepper motor. In this method, if the PMAC2 is not performing current loop closure for the motor (Ix82=0) and Ix72>128, then Ix73 should be set to a negative number of the desired magnitude. In all other cases it should be set to a positive number. If the sign of Ix73 is wrong for your setup, the motor will run away when the loop is closed.

Values of magnitude 2000 to 6000 are typically used for Ix73 in either method.

See Also

Power-Up Phasing Search (Setting Up PMAC Commutation)

I-Variables Ix01, Ix74, Ix78, Ix80, Ix81

Ix74 Motor x Phase Finding Time (PMAC[1] Specific)



An unreliable phasing search method can lead to a runaway condition. Test your phasing search method carefully to make sure it works properly under all conceivable conditions. Make sure your Ix11 fatal following error limit is active and as tight as possible so the motor will be killed quickly in case of a serious phasing search error.

Range: .....

Default: .....

Units: .....

or

Servo Interrupt Cycles * 256 (for Ix80 = 2 or 3)

Remarks

This parameter defines the time that an open-loop output is to be used if a power-on phasing search is done for a PMAC-commutated motor (Ix01=1). A phasing search is required for a synchronous motor (Ix78=0) such as a permanent-magnet brushless motor with no absolute position sensor (Ix81=0). The phasing search is done automatically as part of the power-on phasing search if Ix80 is 1 or 3; if Ix80 is 0 or 2, the on-line **$** command must be used must be used to initiate the phasing search.

If Ix80 is 0 or 1, the "two-guess" phasing search is used; Ix74 has units of servo cycles and controls the time for the open-loop command at each "guess" of the phase angle. Typical values are 3 to 10 servo cycles; 5 is a good starting point. If Ix80 is 2 or 3, the "stepper-motor" phasing search is used; Ix74 has units of (servo cycles*256) and controls the time current is forced into each phase and PMAC waits for the motor to settle into the "step" position. With the default servo cycle rate of 2.25 kHz, each unit of Ix74 represents about 0.1 seconds in this mode; typical values are 10 to 20.

See Also

Power-Up Phasing Search (Setting Up PMAC Commutation)
I-Variables Ix01, Ix73, Ix78, Ix80, Ix81

Ix74 Motor x Phase Finding Time (PMAC2 Specific)



An unreliable phasing search method can lead to a runaway condition. Test your phasing search method carefully to make sure it works properly under all conceivable conditions. Make sure your Ix11 fatal following error limit is active and as tight as possible so the motor will be killed quickly in case of a serious phasing search error.

Range: .....

Default:.....

Units: .....

or
Servo Interrupt Cycles * 256 (for Ix80 = 2 or 3)

Remarks

This parameter defines the time that an open-loop output is to be used if a power-on phasing search is done for a PMAC-commutated motor (Ix01=1). A phasing search is required for a synchronous motor (Ix78=0) such as a permanent-magnet brushless motor with no absolute position sensor (Ix81=0). The phasing search is done automatically as part of the power-on phasing search if Ix80 is 1 or 3; if Ix80 is 0 or 2, the on-line **$** command must be used must be used to initiate the phasing search.

If Ix80 is 0 or 1, the "two-guess" phasing search is used; Ix74 has units of servo cycles and controls the time for the open-loop command at each "guess" of the phase angle. Typical values are 3 to 10 servo cycles; 5 is a good starting point.

If Ix80 is 2 or 3, the "stepper-motor" phasing search is used; Ix74 has units of (servo cycles*256) and controls the time current is forced into each phase and PMAC waits for the motor to settle into the "step" position. With the default servo cycle rate of 2.25 kHz, each unit of Ix74 represents about 0.1 seconds in this mode; typical values are 10 to 20.

See Also

Power-Up Phasing Search (Setting Up PMAC Commutation)
I-Variables Ix01, Ix73, Ix78, Ix80, Ix81

Ix75 Motor x Power-On Phase Position Offset

Range: ..... -8,388,608 -- 8,388,607

Default:..... 0

Units: .....
.....Encoder counts * Ix70

Remarks

This parameter tells PMAC the distance between the zero position of an absolute sensor used for power-on phase position (specified by Ix81) and the zero position of the PMAC commutation cycle. It is used to reference the phas-

ing algorithm for a PMAC-commutated motor with an absolute sensor (Ix81 > 0). If Ix80 is 1, this is done automatically during the power-up/reset cycle. It will also be done in response to a **$** command to the motor.
The proper value for this parameter can be found with a simple procedure that should be done with an *unloaded* motor, after satisfactory operation has been achieved using a power-on phasing search. Define an M-variable to the absolute sensor (TWR form for a resolver, Y form for an absolute encoder). Give the motor an **O0** command Put a bias on the B phase (higher-numbered DAC of a pair) by setting Ix29; use a positive bias if Ix72=171 or 192 (2000 is usually a good value); use a negative bias if Ix72=85 or 64. Also put a bias in the opposite direction of the same magnitude on the A phase by setting Ix79. The motor should lock in on a position like a stepper motor.
Now remove the B-phase bias by setting Ix29 back to zero, or at least to the value you have found to force zero current in the phase, and the motor should lock in on another position. This position is the zero position of the phasing cycle. After you are sure the motor has settled, read the position of the absolute sensor by querying its M-variable value. Take the negative of this value, multiply it by Ix70, and put the resulting value in Ix75. Now, with Ix79 returned to zero or the proper bias, and Ix81 pointing to the absolute sensor, give the motor a **$** command. The motor should be properly phased. Remember to save these variable values before doing a full reset on the card.

Example

On a brushless motor #1 commutated from PMAC with Ix70 =1 and Ix72 = 171, using an R/D converter at location 0 of a board at multiplexer address 0, the following on-line commands can be used to set Ix75:

```
M171->TWR:0,0  ;Resolver position
#1O0 ;Open-loop zero command
I129=2000 ;Pos bias on first phase output (Neg. if Ix72=85 or 64)
I179=-2000;Neg. bias on second phase output (Pos if Ix72=85 or
64)
I129=0    ;Remove bias from first phase output
M171 ;Query sensor position
223  ;PMAC responds
I175=-223 ;Set phasing position offset (223*-1*Ix70)
I179=0    ;Remove bias from second phase
I181=$000100   ;Set power-on position address
I173=0    ;Make sure no phasing search move is done
I174=0    ;Make sure no phasing search move is done
SAVE ;Store I-variables in non-volatile memory
$    ;Try phasing from absolute position sensor
```

See Also

Phasing Referenced to Absolute Sensor (Setting Up PMAC Commutation)
I-Variables I8x, I9x, Ix03, Ix10, Ix81, Ix83
ACC-8D Option 7 (R/D Converter) Manual

Ix76 Motor x Velocity Phase Advance Gain PMAC[1] Specific)

Range: ..... 0 .. 8,388,607
Default: 0

Units: ..... Angle/Vel

Remarks

This parameter advances the phasing angles by an amount proportional to the measured velocity of the motor. This compensates for the lag in the electrical circuits of the phases, and for calculation delays. It should be set to zero for induction motors.
This parameter is best set experimentally by running the motor at high speeds, and finding the setting that minimizes the current draw of the motor.

See Also

Setting Up PMAC Commutation

I-variables Ix70-Ix75

Ix76 Motor x Current-Loop Proportional Gain (Back Path) (PMAC2 Specific)

Range: ..... 0.0 - 1.0 (24-bit resolution)
Default: 0.0
Units: .....
.....PWMout = -4 * Ix62 * (Iact)

Remarks

Ix76 is the proportional gain term of the digital current loop that is in the "back path" of the loop, multiplying the acutal current level, and subtracting the result from the command output. Either Ix76 or Ix62 (forward path proportional gain) must be used to close the current loop. Generally, only one of these proportional gain terms is used, although both can be.
If Ix76 is used as the only proportional gain term, only the Ix61 integral gain term reacts directly to command changes. The act of integration acts as a low-pass filter on the command, which eliminates a lot of noise, but lowers the responsiveness to real changes. Generally Ix76 is only used when the command value from the position/velocity loop servo have high noise levels (usually due to low position resolution), and the actual current measurements have low noise levels.
Ix76 is typically set using the current loop autotuner or interactive tuner in the PMAC Executive Program. Typical values of Ix76, when used, are around 0.5.

Summary

Ix76 is only used if Ix82>0 to activate digital current loop execution.

Ix77 Motor x Induction Motor Magnetization Current (PMAC[1] Specific)

Range: ..... -32,768 .. 32,767
Default:..... 0
Units: .....
..... DAC bits

Remarks

This parameter is used in induction motors to provide a stator current component parallel to the estimated rotor magnetic field (the "direct" current -- the control loop determines the magnitude of the "quadrature" current perpendicular to this component). This should be set to zero for non-induction motors. The proper value for an induction motor is system dependent, but 2500 is a good starting value for most motors. Refer to the *Setting Up PMAC Commutation* section of the manual for instructions in optimizing the setting of this parameter.

See Also

Setting Induction Motor Parameters (Setting Up PMAC Commutation)
I-variables Ix01, Ix70-Ix72, Ix78

Ix77 Motor x Magnetization Current (PMAC2 Specific)

Range: ..... -32,768 .. 32,767
Units: .....
..... Bits of a 16-bit DAC
Default: 0

Remarks

This parameter is used in induction motors to provide a stator current component parallel to the estimated rotor magnetic field (the "direct" current -- the control loop determines the magnitude of the "quadrature" current perpendicular to this component). This should generally be set to zero for non-induction motors, unless advanced "field weakening" algorithms are desired.
The proper value for an induction motor is system dependent, but 2500 is a good starting value for most motors. Refer to the *Setting Up Commutation* section of the manual for instructions in optimizing the setting of this parameter. The "P2SETUP" expert system program for IBM PC and compatible computers is typically used to set the proper value of Ix77 for induction motors

See Also

Setting Induction Motor Parameters (Setting Up PMAC Commutation)
I-variables Ix01, Ix70-Ix72, Ix78

Ix78 Motor x Induction Motor Slip Gain (PMAC[1] Specific)

Range: .....0 .. 8,388,607
Default:..... 0
Units: .....
..... 2^{38} (electrical cycles/update)/DAC bit

Remarks

This parameter is used in induction motors to estimate how much the rotor magnetic field "slips" with regard to the rotor for a given amount of torque. It is proportional to the L/R time constant of the rotor. This should be set to zero for non- induction motors. If this parameter is greater than zero, no power-on phasing search will be done (because the rotor field is not fixed to the rotor). The proper value of this variable is system dependent, but 1200 is a good starting value for most motors. Refer to the *Setting Up PMAC Commutation* section of the manual.

See Also

Setting Induction Motor Parameters (Setting Up PMAC Commutation)
I-Variables Ix01, Ix70-Ix72, Ix77

Ix78 Motor x Slip Gain (PMAC2 Specific)

Range: ..... 0.0 - 1.0 (24-bit resolution)

Units: .....
.....none (ratio of times)
Default: 0.0

Remarks

Ix78 controls the relationship between the torque command and the slip frequency of magnetic field on the rotor of an AC asynchronous (induction) motor. While it is usually set experimentally, it can be calculated as the ratio between the phase update period and the rotor (not stator) L/R electrical time constant.
PMAC2 computes the slip frequency each phase update by multiplying the torque command from the position/velocity-loop servo (or O-command magnitude) by Ix78 and dividing by the magnetization current value controlled by Ix77.
Ix78 is typically set through use of the "P2SETUP" expert system program running on IBM PC compatible. P2SETUP excites the motor and analyzes its response to derive an optimum Ix78 value.
Ix78 can also be set experimentally by giving the motor an O-command and watching the velocity response, probably with the data gathering feature. As the velocity saturates because the back EMF reaches the supply voltage, the velocity should fall back about 5% to reach a steady-state value. If it falls back more than this, the slip time constant is too high; if it falls back less than this, or not at all, the slip time constant is too low.
0.00015 is a typical value of Ix78 for an standard induction motor at a phase update rate of about 9 kHz.
Ix78 is only active if Ix01 is set to 1 to specify PMAC2 commutation of Motor x. It should be set to 0 for AC synchronous motors such as permanent-magnet brushless motors and switched (variable) reluctance motors.

Ix79 Motor x A Phase Output (DAC) Bias (PMAC[1] Specific)

Range: ..... -32,768 .. 32,767
Default:..... 0
Units: .....
..... DAC bits

Remarks

For a PMAC-commutated motor (Ix01=1), the value of this parameter is added to the A-phase output of the commutation algorithm for motor x. This is the higher addressed DAC (lower numbered i.e. DAC 2 of a DAC 1 and DAC 2 pair) of the adjacent DAC pair used for commutation. It is meant to compensate for amplifier offsets. The first-phase bias parameter, Ix29, is added onto the B phase output (higher addressed DAC) of the commutation algorithm.
For a motor with sign-and-magnitude output from PMAC (Ix01=0; bit 16 of Ix02=1), Ix79 provides an offset to the output after the absolute value of the command is taken. This can be used to compensate for deadband in the output (if there is deadband, change Ix79 in the positive direction), or for dithering (if there is dithering, change Ix79 in the negative direction). On this type of motor, Ix29 is used to compensate for directional bias. When PMAC is used to command stepper motor drivers through the ACC-8D Option 2 V-to-F Converter board, this output format is used.

See Also

I-variable Ix29

Accessory 8D Option 2 V-to-F Converter Manual.

Ix79 Second Phase Offset (PMAC2 Specific)

Range: .....-32,768 - 32,767
Units: .....
..... 16-bit DAC/ADC bit equivalent
Default: 0

Remarks

Ix79 serves as an output or feedback offset for Motor x; its exact use depends on the mode of operation as described below:
Mode 1: When PMAC2 is not commutating Motor x (Ix01=0), Ix79 is not used Ix29 is the offset for this mode.

Mode 2: When PMAC2 is commutating Motor x (Ix01=1) but not closing the current loop (Ix82=0), Ix79 serves as the offset for the second of two phase command output values (Phase B), for the address specified by Ix02 plus 1; Ix29 serves the same purpose for the second phase. Ix79 is added to the output command value before it is written to the command output register.
As an output command offset, Ix79 is always in units of a 16-bit register, even if the actual output device is of a different resolution. For example, if a value of 60 had to be written into an 18-bit DAC to create a true zero command, this would be equivalent to a value of 60/4=15 in a 16-bit DAC, so Ix29 would be set to 15 to cancel the offset.
Mode 3: When PMAC2 is commutating (Ix01=1) and closing the current loop for Motor x (Ix82>0), Ix79 serves as an offset that is added to the phase current reading from the ADC for the second phase (Phase B), at the address specified by Ix82. Ix29 performs the same function for the first phase. The sum of the ADC reading and Ix79 is used in the digital current loop algorithms.

Summary

Ix79 is always in units of a 16-bit ADC, even if the actual ADC is of a different resolution. For example, if a 12-bit ADC reported a value of -5 when no current was flowing in the phase, this would be equivalent to a value of -5*16=-80 in a 16-bit ADC, so Ix79 would be set to 80 to compensate for this offset.

Ix80 Motor x Power-Up Mode (PMAC[1] Specific)



An unreliable phasing search method can lead to a runaway condition. Test your phasing search method carefully to make sure it works properly under all conceivable conditions. Make sure your Ix11 fatal following error limit is active and as tight as possible so the motor will be killed quickly in case of a serious phasing search error.

Range: .....

Default: .....

Units: .....

Remarks

This parameter controls the power-up mode for motor x. It controls whether the motor is enabled or killed on power-up/reset (P/R), and if the motor is commutated by PMAC (Ix01=1) and requires a phasing search (Ix78=0; Ix81=0), it controls which type of phasing search is done. The possible values of Ix80, and the effects they have, are:

0	Killed on P/R	"Two guess" phasing search (on $ command only)
1	Enabled on P/R	"Two guess" phasing search (automatically on P/R)
2	Killed on P/R	"Stepper motor" phasing search (on $ command only)
3	Enabled on P/R	"Stepper motor" phasing search (automatically on P/R)

With Ix80=0 or 2, a command must be given to enable the motor. For a PMAC-commutated motor, the **$** command must be given to start up the commutation algorithms, performing the phasing search if necessary, then leaving the motor in closed-loop servo control at zero commanded velocity. For non-PMAC-commutated motors, a **J/** (jog stop) or **$** (motor reset) command (for the motor), an **A** (abort) command (for all motors in the coordinate system), or a **<CTRL-A>** (abort all) command (for all PMAC motors) must be given to put the motor in closed-loop servo control.
If Ix80 is 1 or 3, the motor is enabled automatically at power-up/reset and put in closed-loop servo control at zero commanded velocity. If a phasing search is required, it is done automatically during the power-up/reset cycle.
If Ix80 is 0 or 1 and a phasing search is required, PMAC will use the "two-guess" phasing search method, which is very quick and requires little movement, but is not as reliable in the presence of significant external loads such as friction and gravity.

If Ix80 is 2 or 3 and a phasing search is required, PMAC will use the "stepper-motor" phasing search method, which is takes more time and causes more movement, but is more reliable in the presence of significant external loads.
If Ix80 is 1 or 3, and the motor is disabled right after the power-up/reset cycle, the motor is either being killed by an automatic PMAC safety feature (fatal following error, amplifier fault, or phasing search error) or by a "kill" command from a PLC program.

See Also

Power-Up Phasing Search (Setting Up PMAC Commutation)

I-Variables Ix01, Ix73, Ix74, Ix78, Ix81

Ix80 Motor x Power-Up Mode (PMAC2 Specific)



An unreliable phasing search method can lead to a run-away condition. Test your phasing search method carefully to make sure it works properly under all conceivable conditions. Make sure your Ix11 fatal following error limit is active and as tight as possible so the motor will be killed quickly in the event of a serious phasing search error.

Range: .....
Units: .....
Default: .....

Remarks

This parameter controls the power-up mode for motor x. It controls whether the motor is enabled or killed on power-up/reset (P/R), and if the motor is both commutated by PMAC2 (Ix01=1) and requires a phasing search (Ix78=0; Ix81=0), it controls which type of phasing search is done. The possible values of Ix80, and the effects they have, are:

0	Killed on P/R	"Two guess" phasing search (on $ command only)
1	Enabled on P/R	"Two guess" phasing search (automatically on P/R)
2	Killed on P/R	"Stepper motor" phasing search (on $ command only)
3	Enabled on P/R	"Stepper motor" phasing search (automatically on P/R)



If PMAC2 is commutating across a MACRO ring, proper operation of the drives cannot be guaranteed while PMAC2 is executing its power-up/reset cycle. Therefore, Ix80 should be set to 0 or 2 in these systems

With Ix80=0 or 2, a command must be given to enable the motor. For a PMAC2-commutated motor, the **$** command must be given to start up the commutation algorithms, performing the phasing search if necessary, then leaving the motor in closed-loop servo control at zero commanded velocity. For non-PMAC2-commutated motors, a **J/** (jog stop) or **$** (motor reset) command (for the motor), an **A** (abort) command (for all motors in the coordinate system), or a **<CTRL-A>** (abort all) command (for all PMAC motors) must be given to put the motor in closed-loop servo control.

If Ix80 is 1 or 3, the motor is enabled automatically at power-up/reset and put in closed-loop servo control at zero commanded velocity. If a phasing search is required, it is done automatically during the power-up/reset cycle.

If Ix80 is 0 or 1 and a phasing search is required, PMAC2 will use the "two-guess" phasing search method, which is very quick and requires little movement, but is not as reliable in the presence of significant external loads such as friction and gravity.

If Ix80 is 2 or 3 and a phasing search is required, PMAC2 will use the "stepper-motor" phasing search method, which is takes more time and causes more movement, but is more reliable in the presence of significant external loads.

If Ix80 is 1 or 3, and the motor is disabled right after the power-up/reset cycle, the motor is either being killed by an automatic PMAC safety feature (fatal following error, amplifier fault, or phasing search error) or by a "kill" command from a PLC program.

See Also

Power-Up Phasing Search (Setting Up PMAC Commutation)
I-Variables Ix01, Ix73, Ix74, Ix78, Ix81

Ix81 Motor x Power-On Phase Position Address (PMAC[1] Specific)

Ix81 is used for the PMAC commutation algorithms alone, to locate position within one electrical cycle of the motor. It is not used for any servo loop position information, even for power-up. Ix10 is used for that purpose.

It is easier to specify this parameter in hexadecimal form ($ prefix). If I9 is set to 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.

Range: ..... Extended PM
Default: .....
Units: .....

Remarks

This parameter tells PMAC what address to read, and how to read it, for absolute power-on phase position information, if such information must come differently than the ongoing position information, which is specified by Ix83. It is set to zero if no special power-on phase position reading is desired, as is the case for an incremental encoder. If this variable is set to zero, a power-on phasing routine is required for "fixed-field" brushless motors (permanent magnet, and switched reluctance); those that have a slip gain (Ix78) of zero.

Ix81 consists of two parts. The low 16 bits contain the address of the register containing the power-on position information, either a PMAC memory-I/O address, or an address on the multiplexer ("thumbwheel") port. The high 8 bits specify how to read the information at this address.

Resolver Feedback: If the high 8 bits contain a value from 0 to 7, the address specified in the low 16 bits is a multiplexer-port address, with a valid range of $0002 to $0100 (2 to 256 decimal), evenly divisible by 2. (The actual multiplexer port addresses have a range of 0 to 254 decimal, but in Ix81 a value of 256 must be used to represent address 0.) The multiplexer port is used for devices like the resolver-to-digital converters on ACC-8D Option 7. The value in the high 8 bits specifies the number (location) of the device at that multiplexer address; there are potentially 8 devices at each multiplexer.

Parallel Feedback: If the high 8 bits contain a value from 8 to 24, the address specified in the low 16 bits is a PMAC memory-I/O address, usually the address of an ACC-14 register containing absolute encoder position. The value in the high 8 bits specifies the number of bits, starting at bit 0, to be used for the absolute power-on position information.

Because phase position needs only to be known within a single revolution, any geared-down secondary absolute sensors are not relevant for this purpose. They may still be used for power-on position information for the servo loop, with Ix10, I9x, and I8x.

The zero position of the absolute sensor will not, in general, be the same as the zero position of the commutation cycle. Parameter Ix75 is used to hold the offset between these two reference positions.

Hall-Effect Feedback: If bit 23 of Ix81 is set, the address specified in the low 16 bits is a PMAC memory-I/O address, usually the address of a flag register. In this case, PMAC will use bits 20, 21, and 22 of the address as W, V, and U "hall-effect" commutation signals, respectively, for the absolute power-up phase position. In a PMAC flag register, the inputs for these bits are HMFLn, -LIMn, and +LIMn, respectively. In a typical PMAC application, these inputs are used from the "spare" flag register matching the second DAC channel used for the commutation. The PMAC addresses most commonly used for this are:

Flags 2:	$C004	**Flags 4:**	$C00C
Flags 6:	$C014	**Flags 8:**	$C01C
Flags 10:	$C024	**Flags 12:**	$C02C
Flags 14:	$C034	**Flags 16:**	$C03C

Bit 22 of Ix81 allows for reversal of the sense of the hall-effect sensors. If W (bit 20 of the register; HMFLn) leads V (bit 21; -LIMn), and V leads U (bit 22; +LIMn) as the commutation cycle counts up, then bit 22 of Ix81 should be set to 0. If U leads V and V leads W as the commutation cycle counts up, then bit 22 of Ix81 should be set to 1.

Bits 16 to 21 of Ix81 together form an offset value from 0 to 63 representing the difference between the PMAC commutation cycle zero and the hall-effect sensor zero position, which is defined as the transition of the V signal when U is low. This offset has units of 1/64 of a commutation cycle, or 5.625^{o}e. Typically, one of the transitions will be at the PMAC commutation zero point, so the desired offset values will be 0°, 60°, 120°, 180°, 240°, and 300°, approximated by values of 0, 11($0B), 21($15), 32($20), 43($2B), and 53($35).

This operation can handle hall-effect sensors separated by 120°e. The following tables give the Ix81 settings for bits 16 to 23 for all of the common cases of hall-effect settings as they relate to the PMAC commutation cycle (*xxxx* represents the address portion of Ix81).

120°e Spacing:

0 TO 60 DEG	60 TO 120DEG	120 TO 180 DEG	180 TO -120 DEG	-120 TO -60 DEG	-60 TO 0 DEG	Ix81
011	010	110	100	101	001	$80xxxx
001	011	010	110	100	101	$8Bxxxx
101	001	011	010	110	100	$95xxxx
100	101	001	011	010	110	$A0xxxx
110	100	101	001	011	010	$ABxxxx
010	110	100	101	001	011	$B5xxxx
001	101	100	110	010	011	$C0xxxx
011	001	101	100	110	010	$CBxxxx
010	011	001	101	100	110	$D5xxxx
110	010	011	001	101	100	$E0xxxx
100	110	010	011	001	101	$EBxxxx
101	100	110	010	011	001	$F5xxxx

Example

Motor 1 has a single resolver at location 0 of an ACC-8D Opt.7 R/D converter board at multiplex address 0; no phasing search is permitted, but a homing search is required: I181=$000100 ($100=256dec, representing multiplex address 0), I110=0.

Motor 2 has a single resolver at location 6 of an ACC-8D Opt 7 board at multiplex address 4; no phasing search is permitted, but a homing search is required: I281=$060004; I210=0.

Motor 3 has a double geared resolver at locations 2 and 3 of an ACC-8D Opt 7 board at multiplex address 6, with a 10:1 gear ratio between them; no phasing search or homing search is permitted: I381=$020006; I310=$020006; I93=10

Motor 4 has a 20-bit single-turn absolute encoder at Port A of the first ACC-14 (address Y:$FFD0): I481=$14FFD0 ($14=20dec)

Motor 5 is a brush motor with a double geared resolver at locations 0 and 1 of an ACC-8D Opt 7 board at multiplex address 2; no homing search is permitted: I581=0 (no phasing required); I510=$000002

Motor 6 uses hall-effect sensors wired into the flags on Channel 12 for power-up phase referencing. The zero point of the hall effect is at 60°e, and the direction is "standard", not reversed. I610= $8BC034.

See Also

Phasing Referenced to Absolute Sensor (Setting Up PMAC Commutation)

I-Variables I8x, I9x, Ix03, Ix10, Ix75, Ix83

ACC-8D Option 7 (R/D Converter) Manual

Ix81 Motor x Power-On Phase Position Address (PMAC2 Specific)

Range: ..... $000000 - $FFFFFF

Units: .....
..... Extended PMAC2 or multiplexer-port addresses

Default: 0

Remarks

This parameter tells PMAC what address to read, and how to read it, for absolute power-on phase position information, if such information is present. This can be a different address from that of the ongoing phase position information, which is specified by Ix83. Ix81 is set to zero if no special power-on phase position reading is desired, as is the case for an incremental encoder. If this variable is set to zero, a power-on phasing routine is required for synchronous "fixed-field" brushless motors (permanent magnet, and switched reluctance); those that have a slip gain (Ix78) of zero.



Ix81 is used for PMAC's commutation algorithms alone, to locate position within one electrical cycle of the motor. It is not used for any servo loop position information, even for power-up. Ix10 is used for that purpose.



It is obviously easier to specify this parameter in hexadecimal form ($ prefix). If I9 is set to 2 or 3, the value of this variable will be reported back to the host in hexadecimal form.

Ix81 consists of two parts. The low 16 bits contain the address of the register containing the power-on position information, either a PMAC memory-I/O address, or an address on the multiplexer ("thumbwheel") port. The high 8 bits specify how to read the information at this address.

If the high 8 bits contain a value from 0 to 7, the address specified in the low 16 bits is a multiplexer-port address, with a valid range of $0002 to $0100 (2 to 256 decimal), evenly divisible by 2. (The actual multiplexer port addresses have a range of 0 to 254 decimal, but in Ix81 a value of 256 must be used to represent address 0.) The multiplexer port is used for devices like the resolver-to-digital converters on ACC-8D Option 7. The value in the high 8 bits specifies the number (location) of the device at that multiplexer address; there are potentially 8 devices at each multiplexer.

If the high 8 bits contain a value from 8 to 24, the address specified in the low 16 bits is a PMAC memory-I/O address, usually the address of an ACC-14 register containing absolute encoder position. The value in the high 8 bits specifies the number of bits, starting at bit 0, to be used for the absolute power-on position information.

If bit 23 of Ix81 is set, the address specified in the low 16 bits is a PMAC memory-I/O address, usually the address of a flag register. In this case, PMAC will use bits 22, 21, and 20 of the address as U, V, and W “hall-effect” commutation signals, respectively, for the absolute power-up phase position. In a PMAC2 flag register, the input flags for these bits are CHUn, CHVn, and CHWn, respectively. In a typical PMAC application, these inputs are used from the same flag register addressed by Ix25 for the main flags. The PMAC2 addresses most commonly used for this are:

.....Flags 1:
.....$C000
.....Flags 2:
.....$C008
.....Flags 3:
.....$C010
.....Flags 4:
.....$C018
.....Flags 5:
.....$C020
.....Flags 6:
.....$C028
.....Flags 7:
.....$C030

.....Flags 8:
.....$C038

Bit 22 of Ix81 allows for reversal of the sense of the hall-effect sensors. If W (bit 20 of the register; HMFLn) leads V (bit 21; -LIMn), and V leads U (bit 22; +LIMn) as the commutation cycle counts up, then bit 22 of Ix81 should be set to 0. If U leads V and V leads W as the commutation cycle counts up, then bit 22 of Ix81 should be set to 1.

With bit 23 set, bits 16 to 21 of Ix81 together form an offset value from 0 to 63 representing the difference between PMAC's commutation cycle zero and the hall-effect sensor zero position, which is defined as the transition of the V signal when U is low. This offset has units of 1/64 of a commutation cycle, or 5.625°e. Typically, one of the transitions will be at PMAC's commutation zero point, so the desired offset values will be 0°, 60°, 120°, 180°, 240°, and 300°, approximated by values of 0, 11($0B), 21($15), 32($20), 43($2B), and 53($35).

This operation can handle hall-effect sensors separated by 120°e. The following tables give the Ix81 settings for bits 16 to 23 for all of the common cases of hall-effect settings as they relate to the PMAC commutation cycle (*xxxx* represents the address portion of Ix81).

120°e Spacing:

0 TO 60 DEG	60 TO 120DEG	120 TO 180 DEG	180 TO -120 DEG	-120 TO - 60 DEG	-60 TO 0 DEG	Ix81
011	010	110	100	101	001	$80xxxx
001	011	010	110	100	101	$8Bxxxx
101	001	011	010	110	100	$95xxxx
100	101	001	011	010	110	$A0xxxx
110	100	101	001	011	010	$ABxxxx
010	110	100	101	001	011	$B5xxxx
001	101	100	110	010	011	$C0xxxx
011	001	101	100	110	010	$CBxxxx
010	011	001	101	100	110	$D5xxxx
110	010	011	001	101	100	$E0xxxx
100	110	010	011	001	101	$EBxxxx
101	100	110	010	011	001	$F5xxxx

Because phase position needs only to be known within a single revolution, any geared-down secondary absolute sensors are not relevant for this purpose. They may still be used for power-on position information for the servo loop, with Ix10, I9x, and I8x.
The zero position of the absolute sensor will not, in general, be the same as the zero position of the commutation cycle. Parameter Ix75 is used to hold the offset between these two reference positions.

Examples

Motor 1 has a single resolver at location 0 of an ACC-8D Opt.7 R/D converter board at multiplex address 0; no phasing search is permitted, but a homing search is required: I181=$000100 ($100=256dec, representing multiplex address 0), I110=0.
Motor 2 has a single resolver at location 6 of an ACC-8D Opt 7 board at multiplex address 4; no phasing search is permitted, but a homing search is required: I281=$060004; I210=0.
Motor 3 has a double geared resolver at locations 2 and 3 of an ACC-8D Opt 7 board at multiplex address 6, with a 10:1 gear ratio between them; no phasing search or homing search is permitted: I381=$020006; I310=$020006; I93=10
Motor 4 has a 20-bit single-turn absolute encoder at Port A of the first ACC-14 (address Y:$FFD0): I481=$14FFD0 ($14=20dec)
Motor 5 is a brush motor with a double geared resolver at locations 0 and 1 of an ACC-8D Opt 7 board at multiplex address 2; no homing search is permitted: I581=0 (no phasing required); I510=$000002;

See Also

Phasing Referenced to Absolute Sensor (Setting Up PMAC Commutation)

I-Variables I8x, I9x, Ix03, Ix10, Ix75, Ix83

ACC-8D Option 7 (R/D Converter) Manual

Ix82 Current loop feedback address (PMAC 2 Only)

Range/.....Units: Legal PMAC Y-addresses
Default: 0

Remarks

Ix82 tells PMAC2 which addresses to read to get its current feedback values for motor x if PMAC2 is closing the current loop for this motor. PMAC must be performing the commutation for the motor (Ix01=1) if it is to close the current loop as well.

A zero value for Ix82 tells PMAC2 not to close the current loop for this motor. In this case, PMAC either outputs one velocity or torque command value (Ix01=0), or two phase-current command values (Ix01=1), usually represented as analog voltages.

A non-zero value for Ix82 automatically triggers current loop execution in the phase interrupt, using the current value(s) found in the register(s) specified by Ix82. Typically these registers are analog-to-digital converter (ADC) registers in the PMAC2 ASIC, or MACRO feedback registers containing copies of ADC registers in a MACRO Station

When Ix01 is set to 1, PMAC2 performs the phase commutation for this motor, computing two phase current commands based on the position/velocity servo command and the magnetization current value. If Ix82>0, these commands are compared to the two actual current values read from the address specified by Ix82, and the next *lower* address. It executes a PI filter on the current loops and outputs three voltage command values to the address specified by Ix02 and the next two higher addresses. These are typically the PWM commands for the three half-bridges of a brushless motor power stage.

When the digital current loop is used for drives connected directly to the PMAC2, the typical values for Ix82 are:

I182:
$C006
(Address of ADC1B; specifies ADC1A & B)

I282:
$C00E
(Address of ADC2B; specifies ADC2A & B)

I382:
$C016
(Address of ADC3B; specifies ADC3A & B)

I482:
$C01E
(Address of ADC4B; specifies ADC4A & B)

I582:
$C026
(Address of ADC5B; specifies ADC5A & B)

I682:
$C02E
(Address of ADC6B; specifies ADC6A & B)

I782:
$C036
(Address of ADC7B; specifies ADC7A & B)

I882:
$C03E
(Address of ADC8B; specifies ADC8A & B)

When the digital current loop is used for drives connected to the PMAC2 through a MACRO station, the typical values for Ix82 are:

I182:
$C0A2
(Address of Node 0 Register 2; specifies Reg. 1 & 2)

I282:
$C0A6
(Address of Node 1 Register 2; specifies Reg. 1 & 2)

I382:
$C0AA
(Address of Node 4 Register 2; specifies Reg. 1 & 2)
I482:
$C0AE
(Address of Node 5 Register 2; specifies Reg. 1 & 2)
I582:
$C0B2
(Address of Node 8 Register 2; specifies Reg. 1 & 2)
I682:
$C0B6
(Address of Node 9 Register 2; specifies Reg. 1 & 2)
I782:
$C0BA
(Address of Node 12 Register 2; specifies Reg. 1 & 2)
I882:
$C0BE
(Address of Node 13 Register 2; specifies Reg. 1 & 2)

If Ix82>0, the following variables must be set properly for correct operation of the digital current loop:
Ix61: Current-Loop Integral Gain
Ix62: Current-Loop Forward-Path Proportional Gain
Ix66: PWM Scale Factor
Ix72: Commutation Phase Angle
Ix76: Current-Loop Back-Path Proportional Gain
Ix84: Current-Loop Feedback Mask Word

Ix83 Motor x Ongoing Phasing Position Address (PMAC[1] Specific)



Ix83 points directly to the DSPGATE encoder registers, scaled in raw counts, but Ix03, Ix04, and Ix05 typically point to encoder conversion table registers, holding interpolated count data..

Range: Legal PMAC 'X' and 'Y' addresses

Default:

VARIABLE	HEX	DECIMAL	ENCODER
I183	$C001	49153	Encoder 1
I283	$C009	49161	Encoder 3
I383	$C011	49169	Encoder 5
I483	$C019	49177	Encoder 7
I583	$C021	49185	Encoder 9
I683	$C029	49193	Encoder 11
I783	$C031	49201	Encoder 13
I883	$C039	49209	Encoder 15

Units:

Remarks

For a PMAC-commutated motor (Ix01=1), this parameter tells PMAC which register to get its commutation (phasing) information from for motor x on an ongoing basis. This can be different from the power-on phasing position register address, which is determined by Ix81. Typically, Ix83 points to the "phase position" encoder register in the DSPGATE. Default values for each motor x use the phase position register for Encoder (2x - 1).
If the motor is performing open-loop microstepping control inside PMAC (Ix01=1, bit 16 of Ix02=1), this parameter must contain the address of the motor's "phase advance" register (X:$0042, X:$007E, etc.) instead of an encoder register.
If bit 19 of Ix83 is set to 1, PMAC uses the Y-register of the specified address instead of the X-register. If bit 19 is at the default of 0, PMAC uses the normal X-register. Commutation feedback will only appear in a Y-register on some MACRO nodes.

See Also

I-variables Ix01, Ix02, Ix03, Ix04, Ix81

Setting Up PMAC Commutation

I/O-Memory Map registers X:$C001, X:$C005, etc., X:$0042, X:$007E, etc.

Ix83 Motor x Commutation Position Address (PMAC2 Specific)

Range/Units: .....
Modified PMAC addresses

Default values:

I-VAR.	NON-ULTRALITE PMAC2	PMAC2 ULTRALITE
I183:	$C001 (Encoder 1)	$8C0A0 (MACRO Node 0)
I283:	$C009 (Encoder 2)	$8C0A4 (MACRO Node 1)
I383:	$C011 (Encoder 3)	$8C0A8 (MACRO Node 4)
I483:	$C019 (Encoder 4)	$8C0AC (MACRO Node 5)
I583:	$C021 (Encoder 5)	$8C0B0 (MACRO Node 8)
I683:	$C029 (Encoder 6)	$8C0B4 (MACRO Node 9)
I783:	$C031 (Encoder 7)	$8C0B8 (MACRO Node 12)
I883:	$C039 (Encoder 8)	$8C0BC (MACRO Node 13)

Remarks

For a motor commutated by PMAC2, this parameter tells PMAC2 where to read its commutation (phasing) position information for motor *x* every commutation cycle. This can be a different address from that used for power-on/reset phasing position, which is determined by Ix81.
Bits 0 to 15 of Ix83 contain the 16-bit address of the register to be read. Bit 19 of Ix83 tells whether the register has an 'X' address or a 'Y' address; a 0 value specifies X, and a 1 value (which makes the hexadecimal digit have an 8 value) specifies Y.
For PMAC2 boards with on-board encoder circuitry, Ix83 typically contains the address of the "phase position" encoder register for encoder *x*; this is the default. Since these registers have 'X' addresses, bit 19 is 0.
For PMAC2 Ultralite boards, Ix83 typically contains the address of a MACRO node's position feedback register; this is the default. Since PMAC2 can only commutate over MACRO using nodes with 'Y' addresses, bit 19 must be set to 1 in these cases.

Summary

Ix83 performs exactly the same function on PMAC2 as it does on PMAC. However, because of new memory-mapped hardware in the PMAC2, the addresses that Ix83 contains may be different in the PMAC2.

Further Motor I-Variables

Ix84 Current-Loop Feedback Mask Word (PMAC2 Only)

Range: ..... $000000 - $FFFFFF

Units: ..... Bit mask

Default: ..... $FFF000 (12-bit ADCs)

Remarks

Ix84 tells PMAC2 what bits of the 24-bit current feedback word(s) to use as actual the actual current value in the current loop equations. It is only used if Ix82>0, enabling current loop closure in the PMAC2.

PMAC2 supports interface to serial analog-to-digital converters of many resolutions through its "DSPGATE1" ASIC. The data is received in 18-bit shift registers in the ASIC, which are read as the high end of a 24-bit word, with the number "left-justified" to the most significant bit.

Ix84 specifies a 24-bit mask word that is combined with the feedback word through a logical AND operation to produce the value that is used in the current loop equations. There should be a 1 in every bit that is used, and a 0 in every bit that is not. Since the data is left-justified, Ix84 should start with 1s and end with 0s. Usually Ix84 is represented as a hexadecimal number, with 4 bits per digit, and a total of six digits

Some amplifiers will transmit status and fault information on the end of the serial data stream for the ADC, and it is important to mask out these values from the current loop equations.

Examples

For a 10-bit ADC: Ix84=$FFC000

For a 12-bit ADC: Ix84=$FFF000

For a 16-bit ADC: Ix84=$FFFF00

Ix85 Motor x Backlash Take-up Rate

Range: ..... 0 .. 8,388,607

Default: 0

Units: ..... (1/16 Counts) / Background Cycle

Remarks

This parameter determines how fast backlash is "taken up" on direction reversal. The size of the backlash is determined by Ix86, and possibly the backlash compensation table for the motor. PMAC will "take up" the backlash at the Ix85 rate whenever the commanded or Master Handwheel position for the motor reverses more than 4 encoder counts. If Ix85 is zero, backlash is effectively disabled. Ix85 is usually set as high as possible without creating dynamic problems.
Variable I99, Backlash Hysteresis, determines the amount of reversal in desired position that is required before backlash will start to be introduced or removed.

See Also

I-variables I99, Ix64, Ix65, Ix68,Ix86

On-line commands DEFINE BLCOMP, DELETE BLCOMP

Backlash Compensation (Setting Up a Motor)

Ix86 Motor x Backlash Size

The units of this parameter are 1/16 of a count so the value should be 16 times the number of counts of backlash compensation desired.

Range: .....
Default:.....
Units: .....

Remarks

This parameter allows PMAC to compensate for backlash in the motor's coupling by adding or subtracting (depending on the new direction) the amount specified in the parameter to the commanded position on direction reversals (this offset will not appear when position is queried or displayed). A value of zero means no backlash. The rate at which this backlash is added or subtracted ("taken up") is determined by Ix85.

Variable I99, Backlash Hysteresis, determines the amount of reversal in desired position that is required before backlash will start to be introduced or removed. If backlash tables are used, Ix86 represents the backlash at motor zero position; values in the table should represent the difference between the backlash at a given position and Ix86.

Example

If you find that you have a backlash on reversal of motor direction of 7.5 encoder counts, you would set Ix86 to 7.5 * 16 = 120.

See Also

I-variables I99, Ix64, Ix65, Ix68, Ix85

On-line commands DEFINE BLCOMP, DELETE BLCOMP

Backlash Compensation (Setting Up a Motor)

Coordinate System I-Variables

x = Coordinate System Number (&x, x = 1 to 8)

Ix87 Coordinate System x Default Program Acceleration Time

Range: .....
Default:.....
Units: .....

Remarks

This parameter sets the default time for commanded acceleration for programmed blended **LINEAR** and **CIRCLE** mode moves in coordinate system x. It also provides the default segment time for **SPLINE** mode moves. The first use of a **TA** statement in a program overrides this value.

If the specified S-curve time (see Ix88, below) is greater than half the specified acceleration time, the time used for commanded acceleration in blended moves will be twice the specified S-curve time.
The acceleration time is also the minimum time for a blended move; if the distance on a feedrate-specified (F) move is so short that the calculated move time is less than the acceleration time, or the time of a time-specified (TM) move is less than the acceleration time, the move will be done in the acceleration time instead. This will slow down the move

Even though this parameter makes is possible not to specify acceleration time in the motion program, you are strongly encouraged to use **TA** *in the program and not rely on this parameter, unless you must keep to a syntax standard that does not support this (e.g. RS-274 "G-Codes"). Specifying acceleration time in the program along with speed and move modes makes it much easier for later debugging.*

The acceleration time will be extended automatically when any motor in the coordinate system is asked to exceed its maximum acceleration rate (Ix17) for a programmed LINEAR-mode move with I13=0 (no move segmentation).

*Make sure that the specified acceleration time (Ix87 or 2*Ix88) is greater than zero, even if you are planning to rely on the maximum acceleration rate parameters. A specified acceleration time of zero will cause a divide-by-zero error. The minimum specified time should be Ix87=1, Ix88=0.*

See Also

Acceleration Limits (Making Your Application Safe)

I-variables I13, Ix17, Ix88

Program Commands **TA**, **TS**

Ix88 Coordinate System x Default Program S-Curve Time

Range: .....
Default:.....
Units: .....

Even though this parameter makes is possible not to specify acceleration time in the motion program, you are strongly encouraged to use ***TS*** *in the program and not rely on this parameter, unless you must keep to a syntax standard that does not support this (e.g. RS-274 "G-Codes"). Specifying acceleration time in the program along with speed and move modes makes it much easier for later debugging.*

The acceleration time will be extended automatically when any motor in the coordinate system is asked to exceed its maximum acceleration rate (Ix17) for a programmed LINEAR-mode move with I13=0 (no move segmentation).

*Make sure the specified acceleration time (TA or 2*TS) is greater than zero, even if you are planning to rely on the maximum acceleration rate parameters (Ix17). A specified acceleration time of zero will cause a divide-by-zero error. The minimum specified time should be* ***TA1 TS0****.*

Remarks

This parameter set the default time in each "half" of the "S" in S-curve acceleration for programmed blended **LINEAR** and **CIRCLE** mode moves in coordinate system x. It does not affect **SPLINE**, **PVT**, or **RAPID** mode moves. The first use of a **TS** statement in a program overrides this value.
If Ix88 is zero, the acceleration is constant throughout the Ix87 time and the velocity profile is trapezoidal. If Ix88 is greater than zero, the acceleration will start at zero and linearly increase through Ix88 time, then stay constant (for time TC) until Ix87-Ix88 time, and linearly decrease to zero at Ix87 time (that is Ix87=2*Ix88 - TC). If Ix88 is equal to Ix87/2, the entire acceleration will be spec in S-curve form (Ix88 values greater than Ix87/2 override the Ix87 value; total acceleration time will be 2*Ix88).

See Also

Acceleration Limits (Making Your Application Safe)
I-variables I13, Ix17, Ix87
Program Commands TA, TS

Ix8 Coordinate System x Default Program Feedrate / Move Time

Range: .....
Default:.....
Units: .....
msec for move time

CAUTION *You are strongly encouraged* not *to rely on this parameter and to declare your feedrate in the program. This will keep your move parameters with your move commands, lessening the chances of future errors, and making debugging easier.*

Remarks

This parameter sets the default feedrate (commanded speed) for programmed **LINEAR** and **CIRCLE** mode moves in coordinate system x. The first use of an **F** or **TM** statement in a motion program overrides this value. The velocity

units are defined by the position and time units, as defined by axis definition statements and Ix90. After power-up/reset, the coordinate system is in feedrate mode, not move time mode.
When polled, Ix89 will report the value from the most recently executed **F** or **TM** command in that coordinate system.

See Also

Axis Definition Statements (Setting Up a Coordinate System)
LINEAR and CIRCLE mode blended moves (Writing a Motion Program)
I-variables Ix87, Ix88, Ix90
Program commands F, TM

Ix90 Coordinate System x Feedrate Time Units

Range: ..... positive floating point
Default:.....1000.0 (velocity time units are seconds)
Units: .....
.....msec

Remarks

This parameter defines the time units used in commanded velocities (feedrates) in motion programs executed by coordinate system x. Velocity units are comprised of length units divided by time units. The length units are determined in the axis definition statements for the coordinate system. Ix90 sets the time units. Ix90 itself has units of milliseconds, so if Ix90 is 60,000, the time units are 60,000 milliseconds, or minutes. The default value of Ix90 is 1000 msec, specifying velocity time units of seconds.
This affects two type of motion program values: **F** values (feedrate) for **LINEAR**- and **CIRCLE**-mode moves; and the velocities in the actual move commands for **PVT**-mode moves.

Example

If position units have been set as centimeters by the axis definition statements, and it is desired that feedrate values be specified in cm/sec, this parameter would be set to 1000.0 (time units = sec).
If position units have been set as degrees by the axis definition statements, and it is desired that feedrate values be specified in deg/min, this parameter would be set to 60,000.0 (time units = minutes).
If a spindle is rotating at 4800 rpm, with a linear axis specified in inches, and it is desired that linear speed be specified in inches per spindle revolution, Ix90 would be set to 12.5 ([1 min/4800 rev] * [60,000 msec/ min] = 12.5 msec/rev).

See Also

Axis Definition Statements (Setting Up a Coordinate System)
Velocity-Specified Moves, PVT-Mode Moves (Writing a Motion Program)
Motion program commands **F{data}**, **{axis}{data}:{data}**.

Ix91 Coordinate System x Default Working Program Number

Range: .....0 .. 32,767
Default:..... 0
Units: .....
.....Motion Program Numbers

Remarks

This parameter tells PMAC which motion program to run in this coordinate system when commanded to run from the control-panel input (START/ or STEP/ line taken low). It performs the same function for a hardware "run" command as the **B** command does for a software "run" command (**R**). It is intended primarily for stand-alone PMAC applications. The first use of a **B** command from a host computer for this coordinate system overrides this parameter.

See Also

Control-Panel Port Inputs (Connecting PMAC to the Machine)

On-line commands **B{constant}**, **R**, **S**.

Ix92 Coordinate System x Move Blend Disable

Range: ..... 0 .. 1
Default: 0
Units: ..... none

Remarks

If this parameter set to 0, programmed blended moves -- **LINEAR**, **SPLINE**, and **CIRCLE**-mode -- are blended together with no intervening stop. Upcoming moves are calculated during the current moves. If this parameter is set to 1, there is a brief stop in between each programmed move (it effectively adds a **DWELL 0** command), during which the next move is calculated. The calculation time for the next move is determined by I11.
This parameter is only acted upon when the **R** or **S** command is given to start program execution. To change the mode of operation while the program is running the "continuous motion request" coordinate system status bit (bit 4 of X:$0818 etc.) must be changed. The polarity of this bit is opposite that of Ix92.

See Also

LINEAR- and CIRCLE-Mode Blended Moves (Writing a Motion Program)

How PMAC Executes a Motion Program (Writing a Motion Program)

I-variable I11

Ix93 Coordinate System x Time Base Control Register Address

Range: .....

Ix93 contains the address *of the register that holds the time-base value (it is a pointer to that register). Ix93 does not contain the time-base value itself.*

Default:

VARIABLE	HEX	DECIMAL	REGISTER
I193	$0806	2054	C.S.1 '%' cmd reg
I293	$08C6	2246	C.S.2 '%' cmd reg
I393	$0986	2438	C.S.3 '%' cmd reg
I493	$0A46	2630	C.S.4 '%' cmd reg
I593	$0B06	2822	C.S.5 '%' cmd reg
I693	$0BC6	3014	C.S.6 '%' cmd reg
I793	$0C86	3206	C.S.7 '%' cmd reg
I893	$0D46	3398	C.S.8 '%' cmd reg

Units: .....

Remarks

This parameter tells coordinate system x where to look for its time base control ("feedrate override") information by specifying the address of the register that will be used. The default value of this parameter for each coordinate system (see above) specifies the register that responds to on-line `%` commands. If the time base is left alone, or is under host or programmatic control, this parameter should be left at the default.
Alternatively, if the time base is controlled externally from a frequency or voltage, the register containing the time-base information will usually be in the conversion table (which starts at address $720 [1824 decimal]). With the default conversion table, there is a time-base register at $0729 (1833) related to the frequency into the Encoder 4 counter. This frequency can be controlled by an input voltage on the WIPER pin of the Control Panel Port if jumpers E72 and E73 are ON. If another register is to be used for the time base, it must have the units of I10 so that 8388608 (2^{23}) indicates 1 msec between servo interrupts. See instructions for using an external time base, under Synchronizing PMAC to External Events.

See Also

Time-Base Control (Synchronizing PMAC to External Events)

Control Panel Port Inputs (Connecting PMAC to the Machine)

I-variables I10, Ix93, Ix95

On-line commands %, %{constant}.

Jumpers E72, E73

Ix94 Coordinate System x Time Base Slew Rate (and Limit)



The default Ix94 value of 1644, when used on a card set up with the default servo cycle time of 442 µsec, provides a transition time between %0 and %100 of one second.

Range: .....
Default: .....
Units: .....

Remarks

This parameter controls the rate of change of the coordinate system's time base. It effectively works in two slightly different ways, depending on the source of the time base information. If the source of the time base is the "%" command register, then Ix94 defines the rate at which the "%" (actual time base) value will slew to a newly commanded value. If the rate is too high, and the % value is changed while axes in the coordinate system are moving, there will be a virtual step change in velocity. For these type of applications, Ix94 is set relatively low (often 1000 to 5000) to provide smooth changes.
If there is a hardware source (as defined by Ix93), the commanded time-base value changes every servo cycle, and the rate of change of the commanded value is typically limited by hardware considerations (e.g. inertia). In this case, Ix94 effectively defines the maximum rate at which the "%" value can slew to the new hardware-determined value, and the actual rate of change is determined by the hardware. If you wish to keep synchronous to a hardware input frequency, as in a position-lock cam, Ix94 should be set high enough that the limit is never activated. However, following motion can be smoothed significantly with a lower limit if total synchronicity is not required.

See Also

Time-Base Control (Synchronizing PMAC to External Events)

I-variables I10, Ix93, Ix95

On-line commands **`%{constant}`**, `%`

Ix95 Coordinate System x Feed Hold Deceleration Rate

The default Ix95 value of 1644, when used on a card set up with the default servo cycle time of 442 μsec, provides a transition time between %100 and %0 (feed hold) of one second.

Range: .....
Default:.....
Units: .....

Remarks

This parameter controls the rate at which the axes of the coordinate system stop if a feed hold command (**H**) is given, and the rate at which they start up again on a succeeding run command (**R** or **S**). A feed hold command is equivalent to a **%0** command except that it uses Ix95 for its slew rate instead of Ix94. Having separate slew parameters for normal time-base control and for feed hold commands allows both responsive ongoing time-base control (Ix94 relatively high) and well-controlled holds (Ix95 relatively low).

See Also

Stop Commands (Making Your Application Safe)

Time-Base Control (Synchronizing PMAC to External Events)

I-variables I10, Ix93, Ix94

On-line commands **H**, **<CONTROL-O>**, **R**, **<CONTROL-R>**, **S**, **<CONTROL-S>**, **%**.

Ix96 Coordinate System x Circle Error Limit

Range: ..... positive floating point

Default:..... 0 (function disabled)

Units: .....
..... User length units

Remarks

In a circular arc move, a move distance that is more than twice the specified radius will cause a computation error because a proper path cannot be found. Sometimes, due to round-off errors, a distance slightly larger than twice the radius is given (for a half-circle move), and it is desired that this not create an error condition.
This parameter allows the user to set an error limit on the amount the move distance is greater than twice the radius. If the move distance is greater than 2R, but by less than this limit, the move is done in a spiral fashion to the endpoint, and no error condition is generated. If the distance error is greater than this limit, a run-time error will be generated, and the program will stop. If this variable is set to 0 the error generation is disabled and any move distance greater than 2R is done in a spiral fashion to the endpoint.

Example

Given the program segment

```
INC CIRCLE1 F2

X7.072 Y7.072 R5
```

technically no circular arc path can be found, because the distance is SQRT(7.072^2+7.072^2) = 10.003, which is greater than twice the radius of 5. However as long as Ix96 is greater than 0.003, PMAC will create a near-circular path to the end point.

See Also

Circular Blended Moves (Writing a Motion Program)

Program commands CIRCLE1, CIRCLE2, {axis}{data}{vector}{data}

PMAC[1] Encoder/Flag Setup I-Variables

Encoder/Flag Setup I-Variables pertain to the PMAC[1] only. For PMAC2 refer to the Global Gate Array I-Variables, Channel-Specific Gate Array I-Variables, DSPGATE2 I-Variables, and MACRO Control I-Variables sections below.

One PMAC can have up to 16 incremental encoder channels -- 4 per gate array IC. Each encoder and its related flags and registers are set up using (up to) 5 I-variables. The encoders and their flags are numbered 1 to 16, matching the numbers of their pin outs (e.g. CHA1, CHB1, and CHC1 belong to encoder 1.) The encoder I-variables are assigned to the different encoders as follows:

```
I900 - I904  -- Encoder 1...
I905 - I909  -- Encoder 2I970 - I974  -- Encoder 15
I910 - I914  -- Encoder 3I975 - I979  -- Encoder 16
I915 - I919  -- Encoder 4
```

An encoder is assigned to a motor for position, velocity (feedback), handwheel (master), or feedpot (frequency control) by using the appropriate motor I-variables (see above).

I900, I905, ... I975 Encoder n Decode Control "Encoder I-Variable 0"

Changing the direction sense of the encoder decode for a motor that is servoing properly will result in unstable positive feedback and a dangerous runaway condition in the absence of other changes (for motors not commutated by PMAC from the same encoder). The output polarity must be changed as well to re-establish polarity match for stable negative feedback.

Range: .....

Default:.....

Units: .....

Remarks

This parameter controls how the input signal for Encoder n is decoded into counts. As such, this defines the sign and magnitude of a "count". The following settings may be used to decode an input signal.

VALUE	MEANING
0	pulse and direction CW
1	x1 quadrature decode CW
2	x2 quadrature decode CW

3	x4 quadrature decode CW
4	pulse and direction CCW
5	x1 quadrature decode CCW
6	x2 quadrature decode CCW
7	x4 quadrature decode CCW

In any of the quadrature decode modes, PMAC is expecting two input waveforms on CHAn and CHBn, each with approximately 50% duty cycle, and approximately one-quarter of a cycle out of phase with each other. "Times-one" (x1) decode provides one count per cycle; x2 provides two counts per cycle; and x4 provides four counts per cycle. The vast majority of users select x4 decode to get maximum resolution.
The "clockwise" (CW) and "counterclockwise" (CCW) options simply control which direction counts up. If you get the wrong direction sense, simply change to the other option (e.g. from 7 to 3 or vice versa).
In the pulse-and-direction decode modes, PMAC is expecting the pulse train on CHAn, and the direction (sign) signal on CHBn. If the signal is unidirectional, the CHBn input can be tied high (to +5V) or low (to GND), or, if set up by E18-E21, E24-E27 for single-ended (non-differential) input, left to float high.
Any spare encoder counters may be used as fast and accurate timers by setting this parameter in the 8 to 15 range. In this range, any input signal is ignored. The following settings may be used in timer mode:

SETTING	MEANING
8	Timer counting up at SCLK/10
9	Timer counting up at SCLK/10
10	Timer counting up at SCLK/5
11	Timer counting up at SCLK/2.5
12	Timer counting down at SCLK/10
13	Timer counting down at SCLK/10
14	Timer counting down at SCLK/5
15	Timer counting down at SCLK/2.5

These timers are particularly useful when the related capture and compare registers are utilized for precise event marking and control, including triggered time base. The SLCK frequency is determined by the crystal clock frequency and E34-E38.

See Also

Triggered Time Base (Synchronizing PMAC to External Events)

I-variables Ix03-Ix05, Ix93

Jumpers E18-E21, E24-E27, E34-E38.

I901, I906, ... I976 Encoder n Filter Disable "Encoder I-Variable 1"

Generally, the only people to disable this filter are those using the special interpolated parallel data format. These people should disable the filters both

Range: .....
Default:.....
Units: .....

on the encoder for their quadrature signals and the encoder matching their parallel data input.

Remarks

This parameter controls whether the encoder channel enables or disables its digital delay filter. The options are:
0 = Encoder n digital delay filter enabled
1 = Encoder n digital delay filter disabled (bypassed)
The filter is a 3-stage digital delay filter with best-2-of-3 voting to help suppress noise spikes on the input lines. It does introduce a small delay into the signal, which can be unacceptable if the motor is using interpolated sub-count parallel data input, because of loss of synchronization between the quadrature and parallel data signals.
The sampling frequency for the filter is that of the SCLK signal, which is set by the master clock frequency and jumpers E34-E38. The higher the frequency of SCLK, the higher the possible count rate, but the narrower the pulse that can be filtered out. SCLK should be set to allow the maximum expected encoder frequency, but no faster, in order to provide the maximum noise protection.

See Also

Digital Delay Filter (Connecting PMAC to the Machine)
Parallel Sub-Count Interpolation (Setting Up a Motor)
Jumpers E34-E38

I902, I907, ... I977 Encoder n Position Capture Control "Encoder I-Variable 2"

Range: ..... 0 .. 15

Default: 1

Units: ..... none

Remarks

This parameter determines which signal or combination of signals (and which polarity) triggers a position capture of the counter for encoder n. If a flag input (home, limit, or fault) is used, I903 (etc.) determines which flag. Proper setup of this variable is essential for a successful home search, which depends on the position-capture function. The following settings may be used:
Notice that several of these values are redundant. To do a software-controlled position capture, preset this parameter to 0 or 8; when the parameter is then changed to 4 or 12, the capture is triggered (this is not of much practical use, but can be valuable for testing the capture function).

SETTING	MEANING
0	Software Control
1	Rising edge of CHCn (third channel)
2	Rising edge of Flag n (as set by Flag Select)
3	Rising edge of [CHCn AND Flag n] -- Low true index, high true Flag
4	Software Control
5	Falling edge of CHCn (third channel)

SETTING	MEANING
6	Rising edge of Flag n (as set by Flag Select)
7	Rising edge of [CHCn/ AND Flag n] -- Low true index, high true Flag
8	Software Control
9	Rising edge of CHCn (third channel)
10	Falling edge of Flag n (as set by Flag Select)
11	Rising edge of [CHCn AND Flag n/] -- High true index, low true Flag
12	Software Control
13	Falling edge of CHCn (third channel)
14	Falling edge of Flag n (as set by Flag Select)
15	Rising edge of [CHCn/ AND Flag n/] -- Low true index, low true Flag

See Also

Position Capture (Synchronizing PMAC to External Events)

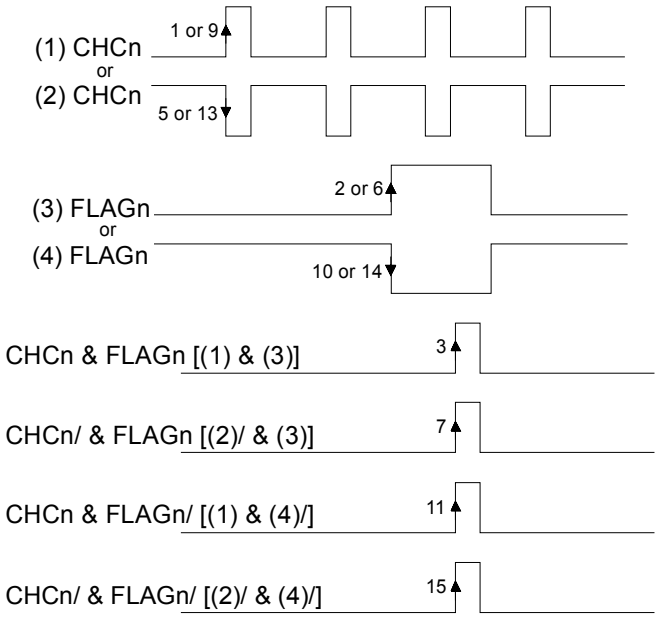
Homing Moves (Basic Motor Moves)

I-variables Ix25, Encoder I-Variable 3

I902, I907, ... , I977

ENCODER POSITION CAPTURE CONTROL

Used for homing and registration



I903, I908, ... I978 Encoder n Flag Select Control "Encoder I-Variable 3"

Range: .....
Default:.....
Units: .....

The direction sense of the limit inputs is the opposite of what many people consider intuitive. That is, the +LIMn input, when taken high (opened), stops commanded motion in the negative direction; the -LIMn input, when taken high, stops commanded motion in the positive direction. It is important to confirm the direction sense of your limit inputs in actual operation.

Remarks

This parameter determines which of the "Flag" inputs will be used for position capture (if one is used -- see I902 etc.):

SETTING	MEANING
0	HMFLn (Home Flag n)
1	-LIMn (Positive Limit Signal n)
2	+LIMn (Negative Limit Signal n)
3	FAULTn (Amplifier Fault Signal n)

This parameter is typically set to zero, because in actual use, the +/-LIMn and FAULTn flags create other effects that usually interfere with what is trying to be accomplished by the position capture. If you wish to capture on the +/-LIMn or FAULTn flags, you must either disable their normal functions with Ix25, or use a channel n where none of the flags is used for the normal axis functions.

See Also

I-variables Ix25, I902

Position Capture (Synchronizing PMAC to External Events)

Homing Moves (Basic Motor Moves)

I904, I909, ... I979 – (Reserved For Future Use)

I1000 MACRO Node Auxiliary Register Enable

Range: .....0 .. $FFFF (0 .. 65,535)

Default:..... 0

Units: .....
..... none

Remarks

This parameter controls which MACRO nodes PMAC performs automatic copying into and out of the auxiliary registers. Enabling this function for a node is required to use the auxiliary register as the flag register for a motor.

I1000 is a 16-bit variable. Bits 0 to 15 control the enabling of this copying function for MACRO nodes 0 to 15, respectively. A bit value of 1 means the copying function is enabled; a bit value of 0 means the copying function is disabled.

If the copying function is enabled for Node n (where n = 0 to F hex or 0 to 15 decimal), during each background "housekeeping" software cycle, PMAC copies the contents of Y:$0F7n to the Node n auxiliary write register, and copies the contents of the Node n auxiliary read register into X:$0F7n.

The copying function enabled by I1000 permits PLC and on-line-command auxiliary read and write functions plus use of the auxiliary registers for command and status flags.

See Also

MACRO Setup

I-Variables Ix25, I995, I996

On-line commands

I1001 MACRO Ring Check Period

Range: ..... 0 .. 255
Default: 0
Units: .....
..... servo cycles

Remarks

I1001 determines the period for PMAC2 to evaluate whether there has been a MACRO ring failure. If I1001 is greater than 0, PMAC2 must receive the "sync node" packet (as specified by I996) twice within I1001 servo cycles. It also must detect no more than one ring error (byte violation error, packet parity error, packet overflow error, or packet underflow error) in this same period. If either of these conditions is not met, PMAC2 will consider this a ring fault, and it will disable all motors.
If I1000 is 0, PMAC2 does not perform these checks, even if MACRO is active.
PMAC2 performs these checks during its background "housekeeping" cycle, executed once between each scan of each background uncompiled PLC (all compiled background PLCs execute a scan each between each housekeeping cycle). It is vital that I1001 be set large enough that at least 2, and preferably 3, housekeeping cycles can be executed in the allotted time.

Example

With the default servo update of 2.25 kHz (440 usec), an I1001 value of 10 sets the check period at 4.4 msec. An I1001 value of 20 sets the check period at 8.8 msec.

Global Gate Array I-variables (PMAC2 Only)

I-variables in the range I900 to I909 control global aspects of the hardware setup using the first "DSPGATE1" ASIC on the PMAC2. On PMAC2 Ultralite boards, there are no DSPGATE1 ASICs on board, so these functions are implemented in the DSPGATE2 ASIC, which is controlled by variables in the I990s.

I900 MaxPhase and PWM 1-4 Frequency Control

Range: ..... 0 - 32767

Units: .....

..... MaxPhase Frequency = 117,964.8 kHz / [2*I900+3]

PWM Frequency = 117,964.8 kHz / [4*I900+6]

Default: 6527

MaxPhase Frequency = 117,964.8 / 13057 = 9.0346 kHz

.....

..... PWM Frequency = 117,964.8 / 26114 = 4.5173 kHz

Remarks

I900 controls the "maximum phase" clock frequency for the PMAC2, and the PWM frequency for machine interface channels 1-4. It does this by setting the limits of the PWM up-down counter, which increments and decrements at the PWMCLK frequency of 117,964.8 kHz (117.9648 MHz).

The actual phase clock frequency is divided down from the maximum phase clock according to the setting of I901. On the falling edge of the phase clock, PMAC2 samples any serial analog-to-digital converters connected to its ASICs (as for phase current measurement), and interrupts the processor to start any necessary phase commutation and digital current-loop algorithms. Even if phasing and current-loop algorithms are not used, the MaxPhase and PhaseClock frequencies are important because the servo clock is derived from the phase clock.

The PWM frequency determines the actual switching frequency of amplifiers connected to any of PMAC2's first four machine interface channels with the direct PWM command. It is only important if the direct PWM command signal format is used.

The maximum value that can be written into the PWM command register without full saturation is I900+1 on the positive end, and -I900-2 on the negative end. Generally, the "PWM scale factor" Ix66 for Motor, which determines the maximum PWM command magnitude, is set to I900 + 10%.

To set I900 for a desired PWM frequency, the following formula can be used:

$$I900 = \frac{117{,}964.8(kHz)}{4 * PWM_Freq(kHz)} - 1 \quad \textit{(rounded down)}$$

To set I900 for a desired "maximum phase" clock frequency, the following formula can be used:

$$..... I900 = \frac{117{,}964.8(kHz)}{2 * MaxPhaseFreq(kHz)} - 1 \quad \textit{(rounded down)}$$

Example:

To set a PWM frequency of 10 kHz and therefore a MaxPhase clock frequency of 20 kHz:

```
I900 = (117,964.8 kHz / [4*10 kHz]) - 1 = 2948
```

To set a PWM frequency of 7.5 kHz and therefore a MaxPhase clock frequency of 15 kHz:

```
I900 = (117,964.8 kHz / [4*7.5 kHz]) - 1 = 3931
```

See Also:

I901, I902, I905, I906, I992

I901 Phase Clock Frequency Control

Range: .....
Units: .....
Default: .....
.....
.....

CAUTION *If the phase clock frequency is set too high, lower priority tasks such as communications can be starved for time. If the background tasks are completely starved, the watchdog timer will trip, shutting down the board. If a normal reset of the board does not re-establish a state where the watchdog timer has not tripped and communications works well, it will be necessary to re-initialize the board by powering up with the E3 re-initialization jumper on. This restores default settings, so communication is possible, and I900 and I901 can be set to supportable values.*

Remarks

If jumper E1 is ON, PMAC2 gets its PHASE clock signal externally from a serial-port input, and I901 is not used.

I901, in conjunction with I900, determines the frequency of the PHASE clock on PMAC2 (except for PMAC2 Ultralites, which use I992 and I997 for this). Each cycle of the PHASE clock, motor phase commutation and digital current-loop algorithms are performed for specified motors.

Specifically, I901 controls how many times the PHASE clock frequency is divided down from the "maximum phase" clock, whose frequency is set by I900. The PHASE clock frequency is equal to the "maximum phase" clock frequency divided by (I901+1). I901 has a range of 0 to 15, so the frequency division can be by a factor of 1 to 16. The equation for I901 is:

$$I901 = \frac{MaxPhaseFreq(kHz)}{PhaseFreq(kHz)} - 1$$

The ratio of MaxPhase Freq. to PHASE Clock Freq. must be an integer.

Example

With a 20 kHz MaxPhase Clock frequency established by I900, and a desired 6.67 kHz PHASE clock frequency, the ratio between MaxPhase and PHASE is 3:

..... I901 = (20 / 6.67) - 1 = 3 -1 = 2

See Also

I900, I902, I997

I902 Servo Clock Frequency Control

CAUTION *If the phase clock frequency is set too high, lower priority tasks such as communications can be starved for time. If the background tasks are completely starved, the watchdog timer will trip, shutting down the board. If a normal reset of the board does not re-establish a state where the watchdog timer has not tripped and communications works well, it will be necessary to re-initialize the board by powering up with the E3 re-initialization jumper on. This restores default settings, so communication is possible, and I900 and I901 can be set to supportable values.*

Range: .....
Units: .....
Default: .....
.....
.....

Remarks

If jumper E1 is ON, PMAC2 gets its SERVO clock signal externally from a serial-port input, and I902 is not used.

I902, in conjunction with I901 and I900, determines the frequency of the SERVO clock on PMAC2 (except for PMAC2 Ultralites, which use I992, I997, and I998 for this). Each cycle of the SERVO clock, PMAC2 updates the commanded position for each activated motor, and executes the servo algorithm to compute the command output to the amplifier.

Specifically, I902 controls how many times the SERVO clock frequency is divided down from the PHASE clock, whose frequency is set by I900 and I901. The SERVO clock frequency is equal to the PHASE clock frequency divided by (I902+1). I902 has a range of 0 to 15, so the frequency division can be by a factor of 1 to 16. The equation for I902 is:

$$I902 = \frac{PhaseFreq(kHz)}{ServoFreq(kHz)} -$$

The ratio of PHASE Clock Freq. to SERVO Clock Freq. must be an integer.
For execution of trajectories at the proper speed, I10 must be set properly to tell the trajectory generation software what the SERVO clock cycle time is. The formula for I10 is:

$$I10 = \frac{8{,}388{,}608}{ServoFreq(k}$$

In terms of the variables that determine the SERVO clock frequency on a (non-Ultralite) PMAC2 board, the formula for I10 is:

$$I10 = \frac{640}{9}(2 * I900 + 3)(I901 + 1)(I902 + 1)$$

At the default servo clock frequency, I10 should be set to 3,713,707 in order that PMAC's interpolation routines use the proper servo update time.

Example

With a 6.67 kHz PHASE Clock frequency established by I900 and I901, and a desired 3.33 kHz SERVO Clock frequency:

```
I902 = (6.67 / 3.33) - 1 = 2 - 1 = 1
```

See Also

I10, I900, I901, I998

I903 Hardware Clock Control Channels 1-4

Range: ..... 0 - 4095

Units: .....

.....I903 = Encoder SCLK Divider
+ 8 * PFM_CLK Divider
.....
.....+ 64 * DAC_CLK Divider
.....
.....+ 512 * ADC_CLK Divider
.....
.....where:
.....
..... Encoder SCLK Frequency = 39.3216 MHz / (2 ^ Encoder SCLK Divider)
.....
..... PFM_CLK Frequency = 39.3216 MHz / (2 ^ PFM_CLK Divider)
.....
..... DAC_CLK Frequency = 39.3216 MHz / (2 ^ DAC_CLK Divider)
.....
..... ADC_CLK Frequency = 39.3216 MHz / (2 ^ ADC_CLK Divider)
.....
.....Default: 2258 = 2 + (8 * 2) + (64 * 3) + (512 * 4)
.....
..... Encoder SCLK Frequency = 39.3216 MHz / (2 ^ 2) = 9.8304 MHz
.....
.....PFM_CLK Frequency = 39.3216 MHz / (2 ^ 2) = 9.8304 MHz
.....
..... DAC_CLK Frequency = 39.3216 MHz / (2 ^ 3) = 4.9152 MHz
.....
..... ADC_CLK Frequency = 39.3216 MHz / (2 ^ 4) = 2.4576 MHz

Remarks

I903 controls the frequency of four hardware clock frequencies -- SCLK, PFM_CLK, DAC_CLK, and ADC_CLK -- for the first four machine interface channels on PMAC2. It is a 12-bit variable consisting of four independent 3-bit controls, one for each of the clocks. Each of these clock frequencies can be divided down from a starting 39.3216 MHz frequency by powers of 2, 2^N, from 1 to 128 times (N=0 to 7). This means that the possible frequency settings for each of these clocks are:

FREQUENCY	DIVIDE BY	DIVIDER N IN $1/2^N$
39.3216 MHz	1	0
19.6608 MHz	2	1
9.8304 MHz	4	2

4.9152 MHz	8	3
2.4576 MHz	16	4
1.2288 MHz	32	5
611.44 kHz	64	6
305.72 kHz	128	7

Very few PMAC2 users will be required to change the setting of I903 from the default value.



If jumper E13 is ON in either setting, PMAC2 uses an external SCLK signal for encoder sampling and digital delay filter clocking; in this case, this part of I903 is not used.

The encoder sample clock signal SCLK controls how often PMAC2's digital hardware looks at the encoder and flag inputs. PMAC2 can take at most one count per SCLK cycle, so the SCLK frequency is the absolute maximum encoder count frequency. SCLK also controls the signal propagation through the digital delay filters for the encoders and flags; the lower the SCLK frequency, the greater the noise pulse that can be filtered out. The SCLK frequency should optimally be set to the lowest value that can accept encoder counts at the maximum possible rate.

The pulse-frequency-modulation clock PFM_CLK controls the PFM circuitry that is commonly used for stepper drives. The maximum pulse frequency possible is 1/4 of the PFM_CLK frequency. The PFM_CLK frequency should optimally be set to the lowest value that can generate pulses at the maximum frequency required.

The DAC_CLK controls the serial data frequency into D/A converters. If these converters are on Delta Tau-provided accessories, the DAC_CLK setting should be left at the default value.

The ADC_CLK controls the serial data frequency from A/D converters. If these converters are on Delta Tau-provided accessories, the ADC_CLK setting should be left at the default value.

To determine the clock frequencies set by a given value of I903, use the following procedure:

1. Divide I903 by 512 and round down to the nearest integer. This value N1 is the ADC_CLK divider.

2. Multiply N1 by 512 and subtract the product from I903 to get I903'. Divide I903' by 64 and round down to the nearest integer. This value N2 is the DAC_CLK divider.

3. Multiply N2 by 64 and subtract the product from I903' to get I903". Divide I903" by 8 and round down to the nearest integer. This value N3 is the PFM_CLK divider.

4. Multiply N3 by 8 and subtract the product from I903". The resulting value N4 is the SCLK divider.

Examples:

The maximum encoder count frequency in the application is 800 kHz, so the 1.2288 MHz SCLK frequency is chosen. A pulse train up to 500 kHz needs to be generated, so the 2.4576 MHz PFM_CLK frequency is chosen. The default serial DACs and ADCs provided by Delta Tau are used, so the default DAC_CLK frequency of 4.9152 MHz and the default ADC_CLK frequency of 2.4576 MHz are chosen. From the table:

```
SCLK Divider N: 5

PFM_CLK Divider N: 4

DAC_CLK Divider N: 3

ADC_CLK Divider N: 4
```

```
     I903 = 5 + (8 * 4) + (64 * 3) + (512 * 4) = 5 + 32 + 192 +
2048 = 2277
```

I903 has been set to 3429. What clock frequencies does this set?

```
     N1 = INT (3429/512) = 6        ADC_CLK = 611.44 kHz

     I903' = 3429 - (512*6) = 357

     N2 = INT (357/64) = 5               DAC_CLK = 1.2288 MHz

     I903'' = 357 - (64*5) = 37

     N3 = INT (37/8) = 4            PFM_CLK = 2.4576 MHz

     N4 = 37 - (8*4) = 5            SCLK = 1.2288 MHz
```

See Also

I907, I993

I904 PWM 1-4 Deadtime/PFM 1-4 Pulse Width Control

Range: ..... 0 - 255

Units: .....

.....PWM Deadtime = [16 / PWM_CLK (MHz)] * I904 = 0.135 usec * I904

PFM Pulse Width = [1 / PFM_CLK (MHz)] * I904 = PFM_CLK_period (usec) * I904

Default: 15

.....

..... PWM Deadtime = 0.135 usec * 15 = 2.03 usec

.....

.....PFM Pulse Width = [1 / 9.8304 MHz] * 15 = 1.526 usec (with default I903)

Remarks

I904 controls the deadtime period between top and bottom on-times in PMAC2's automatic PWM generation for machine interface channels 1-4. In conjunction with I903, it also controls the pulse width for PMAC2's automatic pulse-frequency modulation generation for machine interface channels 1-4.

The PWM deadtime, which is the delay between the top signal turning off and the bottom signal turning on, and vice versa, is specified in units of 16 PWM_CLK cycles. This means that the deadtime can be specified in increments of 0.135 usec. The equation for I904 as a function of PWM deadtime is:

$$I904 = \frac{DeadTime(\mu se}{0.135\mu \text{sec}}$$

The PFM pulse width is specified in PFM_CLK cycles, as defined by I903. The equation for I904 as a function of PFM pulse width and PFM_CLK frequency is:

$$I904 = PFM_CLK_Freq(MHz) * PFM_Pulse_Width(\mu \text{sec})$$

In PFM pulse generation, the minimum off time between pulses is equal to the pulse width. This means that the maximum PFM output frequency is

$$PFM_Max_Freq(MHz) = \frac{PFM_CLK_Freq(MHz)}{2 * I904}$$

Examples

A PWM deadtime of approximately 1 microsecond is desired:

```
I904 ≅ 1 usec / 0.135 usec ≅ 7
```

With a 2.4576 MHz PFM_CLK frequency, a pulse width of 0.4 usec is desired:

```
I904 ≅ 2.4576 MHz * 0.4 usec ≅ 1
```

See Also

I908, I994

I905 DAC 1-4 Strobe Word

Range: ..... $000000 - $FFFFFF
Units: .....
..... Serial Data Stream (MSB first, starting on rising edge of phase clock)
Default: $7FFFC0

Remarks

I905 controls the DAC strobe signal for machine interface channels 1-4. The 24-bit word set by I905 is shifted out serially on lines DAC_STROB1-4, MSB first, one bit per DAC_CLK cycle starting on the rising edge of the phase clock. The value in the LSB is held until the next phase clock cycle.
The default I905 value of $7FFFC0 is suitable for the 18-bit DACs on the ACC-8E Analog Interface Board. I905 should not be changed from the default unless different DACs are used.

I906 PWM 5-8 Frequency Control

Range: ..... 0 - 32767
Units: .....
..... PWM Frequency = 117,964.8 kHz / [4*I906+6]
Default: 6257
.....
..... PWM Frequency = 117,964.8 / 26114 = 4.5163 kHz

Remarks

I906 controls the PWM frequency for machine interface channels 5-8. It does this by setting the limits of the PWM up-down counter, which increments and decrements at the PWMCLK frequency of 117,964.8 kHz (117.9648 MHz). The PWM frequency determines the actual switching frequency of amplifiers connected to any of PMAC2's first four machine interface channels with the direct PWM command. The value of I906 is only important if the direct PWM command signal format is used on channels 5 to 8.
Generally, I906 is set to the same value as I900, which controls the frequency of channels 1 to 4. If a different PWM frequency is desired for channels 5 to
8, I906 should be set so that:

$$\frac{2 * PWM[5-8]Freq(kHz)}{PhaseFreq} = \{Integer\}$$

This will keep the PWM hardware on channels 5-8 in synchronization with the software algorithms driven by the PHASE clock, which is set by I900, I901, and I902. For example if the phase frequency is 10 kHz, the PWM frequency for channels 5 to 8 can be 5, 10, 15, 20, (etc.) kHz.
To set I906 for a desired PWM frequency, the following formula can be used:

$$I906 = \frac{117{,}964.8(kHz)}{4 * PWM_Freq(kHz)} - 1 \quad \textit{(rounded down)}$$

Example:

A 30 kHz PWM frequency is desired for Channels 5-8:

```
I906 = (117,964.8 / [4 * 30]) - 1 = 982
```

See Also

I900, I992

I907 Hardware Clock Control Channels 5-8

Range: ..... 0 - 4095
Units: .....
.....I907 = Encoder SCLK Divider
.....
..... + 8 * PFM_CLK Divider
.....
.....+ 64 * DAC_CLK Divider
.....
.....+ 512 * ADC_CLK Divider
.....
.....where:
.....
..... Encoder SCLK Frequency = 39.3216 MHz / (2 ^ Encoder SCLK Divider)
.....
..... PFM_CLK Frequency = 39.3216 MHz / (2 ^ PFM_CLK Divider)
.....
..... DAC_CLK Frequency = 39.3216 MHz / (2 ^ DAC_CLK Divider)
.....
..... ADC_CLK Frequency = 39.3216 MHz / (2 ^ ADC_CLK Divider)
.....
.....Default: 2258 = 2 + (8 * 2) + (64 * 3) + (512 * 4)
.....
..... Encoder SCLK Frequency = 39.3216 MHz / (2 ^ 2) = 9.8304 MHz
.....
.....PFM_CLK Frequency = 39.3216 MHz / (2 ^ 2) = 9.8304 MHz
.....
..... DAC_CLK Frequency = 39.3216 MHz / (2 ^ 3) = 4.9152 MHz
ADC_CLK Frequency = 39.3216 MHz / (2 ^ 4) = 2.4576 MHz

I907 controls the frequency of four hardware clock frequencies for the second group of four machine interface channels on PMAC2 (channels 5-8). It is a 12-bit variable consisting of four independent 3-bit controls, one for each of the clocks. Each of these clock frequencies can be divided down from a starting 39.3216 MHz frequency by powers of 2, from 1 to 128 times.
This means that the possible frequency settings for each of these clocks are:

FREQUENCY	DIVIDE BY	DIVIDER N IN 1/2^N
39.3216 MHz	1	0
19.6608 MHz	2	1
9.8304 MHz	4	2
4.9152 MHz	8	3
2.4576 MHz	16	4
1.2288 MHz	32	5
611.44 kHz	64	6
305.72 kHz	128	7

Very few PMAC2 users will be required to change the setting of I907 from the default value
The encoder sample clock signal SCLK controls how often PMAC2's digital hardware looks at the encoder and flag inputs. PMAC2 can take at most one count per SCLK cycle, so the SCLK frequency is the absolute maximum en-

coder count frequency. SCLK also controls the signal propagation through the digital delay filters for the encoders and flags; the lower the SCLK frequency, the greater the noise pulse that can be filtered out. The SCLK frequency should optimally be set to the lowest value that can accept encoder counts at the maximum possible rate.
The pulse-frequency-modulation clock PFM_CLK controls the PFM circuitry that is commonly used for stepper drives. The maximum pulse frequency possible is 1/4 of the PFM_CLK frequency. The PFM_CLK frequency should optimally be set to the lowest value that can generate pulses at the maximum frequency required.
The DAC_CLK controls the serial data frequency into D/A converters. If these converters are on Delta Tau-provided accessories, the DAC_CLK setting should be left at the default value.
The ADC_CLK controls the serial data frequency from A/D converters. If these converters are on Delta Tau-provided accessories, the ADC_CLK setting should be left at the default value.

Example

See I903 Example

See Also

I903, I993

I908 PWM 5-8 Deadtime / PFM 5-8 Pulse Width Control

Range: ..... 0 - 255
Units: .....
..... PWM Deadtime = 0.135 usec * I908
PFM Pulse Width = [1 / PFM_CLK (MHz)] * I908 = PFM_CLK_period (usec) * I908

Default: 15
PWM Deadtime = 0.135 usec * 15 = 2.03 usec
.....
..... PFM Pulse Width = [1 / 9.8304 MHz] * 15 = 1.526 usec (with default I907)

Remarks

I908 controls the deadtime period between top and bottom on-times in PMAC2's automatic PWM generation for machine interface channels 5-8. In conjunction with I907, it also controls the pulse width for PMAC2's automatic pulse-frequency modulation generation for machine interface channels 5-8.
The PWM deadtime, which is the delay between the top signal turning off and the bottom signal turning on, and vice versa, is specified in units of 16 PWM_CLK cycles. This means that the deadtime can be specified in increments of 0.135 usec. The equation for I908 as a function of PWM deadtime is:

$$I904 = \frac{DeadTime(\mu se}{0.135\mu \text{sec}}$$

The PFM pulse width is specified in PFM_CLK cycles, as defined by I907. The equation for I908 as a function of PFM pulse width and PFM_CLK frequency is:

$$I908 = PFM_CLK_Freq(MHz) * PFM_Pulse_Width(\mu \text{sec})$$

In PFM pulse generation, the minimum off time between pulses is equal to the pulse width. This means that the maximum PFM output frequency is

$$PFM_Max_Freq(MHz) = \frac{PFM_CLK_Freq(MHz)}{2 * I904}$$

Example

See I904 Example.

See Also

I904, I994I909 DAC 5-8 Strobe Word

Range: ..... $000000 - $FFFFFF

Units: .....
Serial Data Stream (MSB first, starting on rising edge of phase clock)

Default: $7FFFC0

Remarks

I909 controls the DAC strobe signal for machine interface channels 5-8. The 24-bit word set by I909 is shifted out serially on lines DAC_STROB1-4, MSB first, one bit per DAC_CLK cycle starting on the rising edge of the phase clock. The value in the LSB is held until the next phase clock cycle.

The default I909 value of $7FFFC0 is suitable for the 18-bit DACs on the ACC-8E Analog Interface Board. I909 should not be changed from the default unless different DACs are used.

See Also

I905

Channel-Specific Gate Array I-variables (For Channel n, where n = 1 to 8) (PMAC2 Specific)



In almost all cases, the machine interface channel n used for motor x will be of the same number as the motor number (that is, n = x*). However, this does not necessarily have to be the case, so it is a good idea to keep a clear distinction between the software motor functions and the hardware channel functions.*

I-Variables in the I910s through I980s control the hardware aspects of the "DSPGATE1" ASICs that provide the machine interface for channels 1 through 8. Each DSPGATE1 ASIC controls four channels. On an 8-channel PMAC2 (one that includes Option 1), I-variables for all 8 channels can be used. On a 4-channel PMAC2 (PMAC2-Lite or other PMAC2 without Option 1), only the I-variables for the first 4 channels can be used. On a PMAC2 Ultralite, there are no local machine interface channels, so none of the I-variables in this range may be used.

I9n0 Encoder/Timer n Decode Control

Range: .....0 - 15 (Bits 0-3 of channel control word)
Units: .....
.....None
Default: 7

Remarks

I9n0 controls how the input signal for Encoder n is decoded into counts. As such, this defines the sign and magnitude of a "count". The following settings may be used to decode an input signal.

0: Pulse and direction CW

1: x1 quadrature decode CW

2: x2 quadrature decode CW

3: x4 quadrature decode CW

4: Pulse and direction CCW

5: x1 quadrature decode CCW

6: x2 quadrature decode CCW

7: x4 quadrature decode CCW

8: Internal pulse and direction

9: Not used

10: Not used

11: Not used

12: MLDT pulse timer control (internal pulse resets timer; external pulse latches timer)

13: Not used

14: Not used

15: Not used

In any of the quadrature decode modes, PMAC2 is expecting two input waveforms on CHAn and CHBn, each with approximately 50% duty cycle, and approximately one-quarter of a cycle out of phase with each other. "Times-one" (x1) decode provides one count per cycle; x2 provides two counts per cycle; and x4 provides four counts per cycle. The vast majority of users select x4 decode to get maximum resolution.
The "clockwise" (CW) and "counterclockwise" (CCW) options simply control which direction counts up. If you get the wrong direction sense, simply change to the other option (e.g. from 7 to 3 or vice versa).
Important note: Changing the direction sense of the decode for the feedback encoder of a motor that is operating properly will result in unstable positive feedback and a dangerous runaway condition in the absence of other changes. The output polarity must be changed as well to re-establish polarity match for stable negative feedback.

In the pulse-and-direction decode modes, PMAC2 is expecting the pulse train on CHAn, and the direction (sign) signal on CHBn. If the signal is unidirectional, the CHBn line can be allowed to pull up to a high state, or it can be hardwired to a high or low state.

If I9n0 is set to 8, the decoder inputs the pulse and direction signal generated by Channel n's pulse frequency modulator (PFM) output circuitry. This permits the PMAC2 to create a phantom closed loop when driving an open-loop stepper system. *No jumpers or cables are needed to do this; the connection is entirely within the ASIC.* The counter polarity automatically matches the PFM output polarity.

If I9n0 is set to 12, the timer circuitry is set up to read magnetostrictive linear displacement transducers (MLDTs) such as Temposonics™. In this mode, the timer is cleared when the PFM circuitry sends out the excitation pulse to the sensor on PULSEn, and it is latched into the memory-mapped register when the excitation pulse is received on CHAn.

I9n1 Position Compare n Channel Select

Range: ..... 0 - 1

Units: .....
.....None

Default: 0

.....
..... 0: Use channel n encoder counter for position compare function

1: Use first encoder counter on IC (encoder 1 for channels 1 to 4; encoder 5 for channels 5 to 8) for position compare function

Remarks

When I9n1 is set to 0, the channel's position compare register is tied to the channel's own encoder counter, and the position compare signal appears only on the EQUn output.

When I9n1 is set to 1, the channel's position compare register is tied to the first encoder counter on the ASIC -- Encoder 1 for channels 1-4, or Encoder 5 for channels 5-8 -- and the position compare signal appears both on EQUn, and combined into the EQU output for the first channel on the IC (EQU1 or EQU5); executed as a logical OR.

I911 and I951 perform no effective function, so are always 1. They cannot be set to 0.

I9n2 Encoder n Capture Control

Range: .....0 - 15 (Bits 4-7 of channel control word)

Units: .....
none

Default:..... 1

Remarks

This parameter determines which input signal or combination of signals for channel n, and which polarity, triggers a hardware position capture of the counter for encoder n. If a flag input (home, limit, or user) is used, I9n3 determines which flag. Proper setup of this variable is essential for a successful home search, which depends on the position-capture function. The following settings may be used:

0: Capture under software control (armed)

1: Capture on Index (CHCn) high

2: Capture on Flag n high

3: Capture on (Index high AND Flag n high)

4: Capture under software control (latched)

5: Capture on Index (CHCn) low

6: Capture on Flag n high

7: Capture on (Index low AND Flag n high)

8: Capture under software control (armed)

9: Capture on Index (CHCn) high

10: Capture on Flag n low

11: Capture on (Index high AND Flag n low)

12: Capture under software control (latched)

13: Capture on Index (CHCn) low

14: Capture on Flag n low

15: Capture on (Index low AND Flag n low)

Note that only flags and index inputs of the same channel number as the encoder may be used for hardware capture of that encoder's position. This means that to use the hardware capture feature for the homing search move, Ix25 must use flags of the same channel number as the encoder that Ix03 uses for position-loop feedback.

To do a software-controlled position capture, preset this parameter to 0 or 8; when the parameter is then changed to 4 or 12, the capture is triggered (this is not of much practical use).

The trigger is armed when the position capture register is read. After this, as soon as PMAC2 sees that the specified input lines are in the specified states, the trigger will occur -- it is level-triggered, not edge-triggered.

I9n3 Capture n Flag Select Control

Range: ..... 0 - 3 (Bits 8-9 of channel control word)

Units: ..... none

Default: 0

Remarks

This parameter determines which of the "Flag" inputs will be used for position capture (if one is used -- see I902 etc.):

16: HMFLn (Home Flag n)

17: PLIMn (Positive End Limit Flag n)

18: MLIMn (Negative End Limit Flag n)

19: USERn (User Flag n)

This parameter is typically set to zero, because in actual use, the LIMn flags create other effects that usually interfere with what is trying to be accomplished by the position capture. If you wish to capture on the PLIMn or MLIMn flags, you probably will want to disable their normal functions with Ix25, or use a channel n where none of the flags is used for the normal axis functions.

I9n4 Encoder n Gated Index Select

Range: ..... 0 - 1

Units: ..... none

0 = Use ungated index for encoder position capture

1 = Use index gated by quadrature channels for position capture

Default: 0

Remarks



If I9n4 is set to 1, but I9n2 bit 0 is set to 0, so the index is not used in the position capture, then the encoder position is captured on the first edge of any of the U, V, or W flag inputs for the channel. In this case, bits 0, 1, and 2 of the channel status word tell what hall-state edge caused the capture.

When I9n4 is set to 0, the encoder index channel input (CHCn) is passed directly into the position capture circuitry.

When I9n4 is set to 1, the encoder index channel input (CHCn) is logically combined with ("gated by") the quadrature signals of Encoder n before going to the position capture circuitry. The intent is to get a "gated index" signal exactly one quadrature state wide. This provides a more accurate and repeatable capture, and makes the use of the capture function to confirm the proper number of counts per revolution very straightforward.

In order for the gated index capture to work reliably, the index pulse must reliably span one, but only one, "high-high" or "low-low" AB quadrature state of the encoder. I9n5 allows you to select which of these two possibilities is used.

I9n5 Encoder n Index Gate State

Range: ..... 0 - 1

Units: .....
..... none

.....
.....0 = Gate index with "high-high" quadrature state (GI = A & B & C)

.....
.....1 = Gate index with "low-low" quadrature state (GI = A/ & B/ & C)

Default: 0

Remarks

When using the "gated index" feature of the PMAC2 for more accurate position capture, I9n5 specifies whether the raw index-channel signal fed into PMAC2 is passed through to the position capture signal only on the "high-high" quadrature state, or only on the "low-low" quadrature state. If I9n5 is set to 0, it is passed through only on the "high-high" state; if I9n5 is set to 1, it is passed through only on the "low-low" state.

I9n6 Output n Mode Select

Range: ..... 0 - 3

Units: .....
..... none

0 = Outputs A & B are PWM; Output C is PWM

.....
..... 1 = Outputs A & B are DAC; Output C is PWM

.....
.....2 = Outputs A & B are PWM; Output C is PFM

.....
.....3 = Outputs A & B are DAC; Output C is PFM

Default: 0

Remarks

I9n6 controls what output formats are used on the command output signal lines for machine interface channel n. If a three-phase direct PWM command format is desired, I9n6 should be set to 0. If signal outputs for (external) digital-to-analog converters are desired, I9n6 should be set to 1 or 3. In this case, the C output can be used as a supplemental (non-servo) output in either PWM or PFM form. For example, it can be used to excite an MLDT sensor (e.g. Temposonics^{TM}) in PFM form.

I9n7 Output n Invert Control

Range: ..... 0 - 3
Units: .....
..... none

0 = Do not invert Outputs A & B; Do not invert Output C

.....
.....1 = Invert Outputs A & B; Do not invert Output C

.....
.....2 = Do not invert Outputs A & B; Invert Output C

.....
..... 3 = Invert Outputs A & B; Invert Output C

Default: 0

Remarks

I9n7 controls the high/low polarity of the command output signals for Channel n. The default non-inverted outputs are high true.



If the high/low polarity of the PWM signals is wrong for a particular amplifier, what was intended to be deadtime between top and bottom on-states as set by I904 and I908 becomes overlap. If the amplifier input circuitry does not lock this out properly, this causes an effective momentary short circuit between bus power and ground. This would destroy the power transistors very quickly



Changing the high/low polarity of the digital data to the DACs has the effect of inverting the voltage sense of the DACs' analog outputs. This changes the polarity match between output and feedback. If the feedback loop had been stable with negative feedback, this change would create destabilizing positive feedback, resulting in a dangerous runaway condition that would only be stopped when the motor exceeded Ix11 fatal following er-

For PWM signals on Outputs A, B, and C, this means that the transistor-on signal is high. Delta Tau PWM-input amplifiers, and most other PWM-input amplifiers, expect this non-inverted output format. For such a 3-phase motor drive, I9n7 should be set to 0.

For PFM signals on Output C, non-inverted means that the pulse-on signal is high (direction polarity is controlled by I9n8). During a change of direction, the direction bit will change synchronously with the leading edge of the pulse, which in the non-inverted form is the rising edge. If the drive requires a set-up time on the direction line before the rising edge of the pulse, the pulse output can be inverted so that the rising edge is the trailing edge, and the pulse width (established by I904 or I908) is the set-up time.

For DAC signals on Outputs A and B, non-inverted means that a 1 value to the DAC is high. DACs used on Delta Tau accessory boards, as well as all other known DACs always expect non-inverted inputs, so I9n7 should always be set to 0 or 2 when using DACs on Channel n.

ror.

I9n8 Output n PFM Direction Signal Invert Control

Range: ..... 0 - 1

Units: none

.....
.....0 = Do not invert direction signal (+ = low; - = high)
.....
.....1 = Invert direction signal (- = low; + = high)

Default:..... 0

Remarks

I9n8 controls the polarity of the direction output signal in the pulse-and-direction format for Channel n. It is only active if I9n6 has been set to 2 or 3 to use Output C as a pulse-frequency-modulated (PFM) output.

If I9n8 is set to the default value of 0, a positive direction command provides a low output; if I9n8 is set to 1, a positive direction command provides a high output.

I9n9 Reserved for future use

DSPGATE2 I-Variables (PMAC2 Only)

I-Variables numbered in the I990s control hardware aspects of the "DSPGATE2" ASIC. This IC controls operation of the MACRO ring on all PMAC2 boards. On the Ultralite versions of the PMAC2, this IC also controls the frequency of the clock signals on the board, because the "DSPGATE1" ICs are not present. On all of these boards, I990 and I991 control the decode of the handwheel encoder inputs on the JHW port.

I990 Handwheel 1 Decode Control

Range: .....0 - 15 (Bits 0-3 of channel control word)

Units: .....
.....None

Default: 7

Remarks

I990 controls how the input signal for Handwheel 1 on the JHW port is decoded into counts. As such, this defines the sign and magnitude of a "count". The following settings may be used to decode an input signal.

0: Pulse and direction CW

1: x1 quadrature decode CW

2: x2 quadrature decode CW

3: x4 quadrature decode CW

4: Pulse and direction CCW

5: x1 quadrature decode CCW

6: x2 quadrature decode CCW

7: x4 quadrature decode CCW

8-15: Not used

In any of the quadrature decode modes, PMAC2 is expecting two input waveforms on HWA1 and HWB1, each with approximately 50% duty cycle, and approximately one-quarter of a cycle out of phase with each other. "Times-one" (x1) decode provides one count per cycle; x2 provides two counts per cycle; and x4 provides four counts per cycle. The vast majority of users select x4 decode to get maximum resolution.

The "clockwise" (CW) and "counterclockwise" (CCW) options simply control which direction counts up. If you get the wrong direction sense, simply change to the other option (e.g. from 7 to 3 or vice versa)

Important note: Changing the direction sense of the decode for the feedback encoder of a motor that is operating properly will result in unstable positive feedback and a dangerous runaway condition in the absence of other changes. The output polarity must be changed as well to re-establish polarity match for stable negative feedback.

In the pulse-and-direction decode modes, PMAC2 is expecting the pulse train on HWA1, and the direction (sign) signal on HWB1. If the signal is unidirectional, the HWB1 line can be allowed to pull up to a high state, or it can be hardwired to a high or low state.

I991 Handwheel 2 Decode Control

Range: .....0 - 15 (Bits 0-3 of channel control word)
Units: .....None
Default: 7

Remarks

I991 controls how the input signal for Handwheel 2 is decoded into counts. As such, this defines the sign and magnitude of a "count". The following settings may be used to decode an input signal.

0: Pulse and direction CW

1: x1 quadrature decode CW

2: x2 quadrature decode CW

3: x4 quadrature decode CW

4: Pulse and direction CCW

5: x1 quadrature decode CCW

6: x2 quadrature decode CCW

7: x4 quadrature decode CCW

8-15: Not used

In any of the quadrature decode modes, PMAC2 is expecting two input waveforms on HWA2 and HWB2, each with approximately 50% duty cycle, and approximately one-quarter of a cycle out of phase with each other. "Times-one" (x1) decode provides one count per cycle; x2 provides two counts per cycle; and x4 provides four counts per cycle. The vast majority of users select x4 decode to get maximum resolution.
The "clockwise" (CW) and "counterclockwise" (CCW) options simply control which direction counts up. If you get the wrong direction sense, simply change to the other option (e.g. from 7 to 3 or vice versa)
Important note: Changing the direction sense of the decode for the feedback encoder of a motor that is operating properly will result in unstable positive feedback and a dangerous runaway condition in the absence of other changes. The output polarity must be changed as well to re-establish polarity match for stable negative feedback.

In the pulse-and-direction decode modes, PMAC2 is expecting the pulse train on HWA2, and the direction (sign) signal on HWB2. If the signal is unidirectional, the HWB2 line can be allowed to pull up to a high state, or it can be hardwired to a high or low state.

I992 MaxPhase and PWM 1*-2* Frequency Control

On PMAC2 boards that are not "Ultralite", I992 does not control the MaxPhase frequency; I900 does. On all PMAC2 boards, the PWM 1-2* frequency is only important if you are using supplemental PWM channels.*

Range: 0 - 32767

Units: MaxPhase Frequency = 117,964.8 kHz / [2*I992+3]

PWM Frequency = 117,964.8 kHz / [4*I992+6]

Default: 6527

MaxPhase Frequency = 117,964.8 / 13057 = 9.0346 kHz

PWM Frequency = 117,964.8 / 26114 = 4.5173 kHz

Remarks

I992 controls the "maximum phase" clock frequency for the PMAC2 Ultralite, and the PWM frequency for supplementary machine interface channels 1* and 2*. It does this by setting the limits of the PWM up-down counter, which increments and decrements at the PWMCLK frequency of 117,964.8 kHz (117.9648 MHz).
The actual phase clock frequency is divided down from the maximum phase clock according to the setting of I997. On the falling edge of the phase clock, PMAC2 Ultralite starts transmission of a set of MACRO ring data and interrupts the processor to start any necessary phase commutation and digital current-loop algorithms. Even if phasing and current-loop algorithms are not used, the MaxPhase and Phase clock frequencies are important because the servo clock is derived from the phase clock.

To set I992 for a desired "maximum phase" clock frequency, the following formula can be used:
.....I992 = (117,964.8 kHz / [2*MaxPhase (kHz)]) - 1 (rounded down)
On PMAC2 boards that are not “Ultralite”, I992 is generally set to the same value as I900, which controls the maximum phase frequency, and the PWM frequency of channels 1 to 4. If a different PWM frequency is desired for channels 1* and 2*, I992 should be set so that

$$\frac{2 * PWM[1*-2*]Freq(kHz)}{PhaseFreq} = \{Integer\}$$

Examples

To set a PWM frequency of 10 kHz and therefore a MaxPhase clock frequency of 20 kHz:

```
I992 = (117,964.8 kHz / [4*10 kHz]) - 1 = 2948
```

To set a PWM frequency of 7.5 kHz and therefore a MaxPhase clock frequency of 15 kHz:

```
I992 = (117,964.8 kHz / [4*7.5 kHz]) - 1 = 3931
```

I993 Hardware Clock Control Channels 1*-2*

Range: ..... - 4095
Units: .....
.....I993 = Encoder SCLK Divider
+ 8 * PFM_CLK Divider
.....
.....+ 64 * DAC_CLK Divider
.....
.....+ 512 * ADC_CLK Divider

.....
.....where:
.....
..... Encoder SCLK Frequency = 39.3216 MHz / (2 ^ Encoder SCLK Divider)
.....
..... PFM_CLK Frequency = 39.3216 MHz / (2 ^ PFM_CLK Divider)
.....
..... DAC_CLK Frequency = 39.3216 MHz / (2 ^ DAC_CLK Divider)
.....
..... ADC_CLK Frequency = 39.3216 MHz / (2 ^ ADC_CLK Divider)
.....
.....Default: 2258 = 2 + (8 * 2) + (64 * 3) + (512 * 4)
.....
..... Encoder SCLK Frequency = 39.3216 MHz / (2 ^ 2) = 9.8304 MHz
.....
.....PFM_CLK Frequency = 39.3216 MHz / (2 ^ 2) = 9.8304 MHz
.....
..... DAC_CLK Frequency = 39.3216 MHz / (2 ^ 3) = 4.9152 MHz
.....
..... ADC_CLK Frequency = 39.3216 MHz / (2 ^ 4) = 2.4576 MHz

Remarks

I993 controls the frequency of three hardware clock frequencies -- SCLK, PFM_CLK, and ADC_CLK -- for the supplemental machine interface channels 1* and 2* on PMAC2 or PMAC2 Ultralite (there is no DAC_CLK on the supplemental channels, but it is referred to here for consistency with I903 and I907). It is a 12-bit variable consisting of four independent 3-bit controls (the 3 bits for DAC_CLK are "don't care"), one for each of the clocks. Each

of these clock frequencies can be divided down from a starting 39.3216 MHz frequency by powers of 2, 2^N, from 1 to 128 times (N=0 to 7). This means that the possible frequency settings for each of these clocks are:

FREQUENCY	DIVIDE BY	DIVIDER N IN $1/2^N$
39.3216 MHz	1	0
19.6608 MHz	2	1
9.8304 MHz	4	2
4.9152 MHz	8	3
2.4576 MHz	16	4
1.2288 MHz	32	5
611.44 kHz	64	6
305.72 kHz	128	7

Very few PMAC2 users will be required to change the setting of I993 from the default value.
The encoder sample clock signal SCLK controls how often PMAC2's digital hardware looks at the handwheel encoder inputs. PMAC2 can take at most one count per SCLK cycle, so the SCLK frequency is the absolute maximum encoder count frequency. SCLK also controls the signal propagation through the digital delay filters for the encoders and flags; the lower the SCLK frequency, the greater the noise pulse that can be filtered out. The SCLK frequency should optimally be set to the lowest value that can accept encoder counts at the maximum possible rate.
The pulse-frequency-modulation clock PFM_CLK controls the PFM circuitry that can create pulse and direction outputs on the JHW connector. The maximum pulse frequency possible is 1/4 of the PFM_CLK frequency. The PFM_CLK frequency should optimally be set to the lowest value that can generate pulses at the maximum frequency required.
The ADC_CLK controls the serial data frequency from A/D converters. These can only be accessed as the alternate use of general-purpose I/O pins.
To determine the clock frequencies set by a given value of I993, use the following procedure:

5. Divide I993 by 512 and round down to the nearest integer. This value N1 is the ADC_CLK divider.

6. Multiply N1 by 512 and subtract the product from I993 to get I993'. Divide I993' by 64 and round down to the nearest integer. This value N2 is the DAC_CLK divider (not relevant here).

7. Multiply N2 by 64 and subtract the product from I993' to get I993". Divide I993" by 8 and round down to the nearest integer. This value N3 is the PFM_CLK divider.

8. Multiply N3 by 8 and subtract the product from I993". The resulting value N4 is the SCLK divider.

Examples

The maximum encoder count frequency in the application is 800 kHz, so the 1.2288 MHz SCLK frequency is chosen. A pulse train up to 500 kHz needs to be generated, so the 2.4576 MHz PFM_CLK frequency is chosen. ADCs and DACs are not used, so the default DAC_CLK frequency of 4.9152 MHz and the default ADC_CLK frequency of 2.4576 MHz are chosen. From the table:

```
SCLK Divider N: 5

PFM_CLK Divider N: 4
```

```
     DAC_CLK Divider N: 3

     ADC_CLK Divider N: 4

     I993 = 5 + (8 * 4) + (64 * 3) + (512 * 4) = 5 + 32 + 192 +
2048 = 2277

I993 has been set to 3429.  What clock frequencies does this set?

     N1 = INT (3429/512) = 6       ADC_CLK = 611.44 kHz

     I993' = 3429 - (512*6) = 357

     N2 = INT (357/64) = 5         DAC_CLK = 1.2288 MHz

     I993'' = 357 - (64*5) = 37

     N3 = INT (37/8) = 4           PFM_CLK = 2.4576 MHz

     N4 = 37 - (8*4) = 5           SCLK = 1.2288 MHz
```

I994 PWM 1*-2* Deadtime / PFM 1* Pulse Width Control

Range: ..... 0 - 255
Units: .....
.....PWM Deadtime = [16 / PWM_CLK (MHz)] * I994 = 0.135 usec * I994
PFM Pulse Width = [1 / PFM_CLK (MHz)] * I994 = PFM_CLK_period (usec) * I994

Default: 15
.....
..... PWM Deadtime = 0.135 usec * 15 = 2.03 usec
.....
.....PFM Pulse Width = [1 / 9.8304 MHz] * 15 = 1.526 usec (with default I993)

Remarks

I994 controls the deadtime period between top and bottom on-times in PMAC2's automatic PWM generation for supplemental machine interface channels 1* and 2*. In conjunction with I993, it also controls the pulse width for PMAC2's automatic pulse-frequency modulation generation for supplemental machine interface channel 1*.
The PWM deadtime, which is the delay between the top signal turning off and the bottom signal turning on, and vice versa, is specified in units of 16 PWM_CLK cycles. This means that the deadtime can be specified in increments of 0.135 usec. The equation for I994 as a function of PWM deadtime is:

$$I994 = \frac{DeadTime(\mu\text{sec})}{0.135\mu\text{sec}}$$

The PFM pulse width is specified in PFM_CLK cycles, as defined by I993. The equation for I994 as a function of PFM pulse width and PFM_CLK frequency is:

$$. \; I994 = PFM_CLK_Freq(MHz) * PFM_Pulse_Width(\mu\text{sec})$$

In PFM pulse generation, the minimum off time between pulses is equal to the pulse width. This means that the maximum PFM output frequency is:

$$..... \; PFM_Max_Freq(MHz) = \frac{PFM_CLK_Freq(MHz)}{2 * I994}$$

Examples

A PWM deadtime of approximately 1 microsecond is desired:

```
I994 ≅ 1 usec / 0.135 usec ≅ 7
```

With a 2.4576 MHz PFM_CLK frequency, a pulse width of 0.4 usec is desired:

```
I994 ≅ 2.4576 MHz * 0.4 usec ≅ 1
```

I995 MACRO Ring Configuration/Status

Range: ..... $0000 - $FFFF (0 - 65,535)
Units: ..... none
Default: 0

Remarks



On prototype PMAC2 boards that did not support multi-master MACRO rings, I995 was simply 1 for a master station, or 0 for a slave station.

I995 contains configuration and status bits for MACRO ring operation of the PMAC2. There are 11 configuration bits and 5 status bits, as follows:

BIT #	VALUE	TYPE	FUNCTION
0	1($1)	Status	Data Overrun Error (cleared when read)
1	2($2)	Status	Byte Violation Error (cleared when read)
2	4($4)	Status	Packet Parity Error (cleared when read)
3	8($8)	Status	Packet Underrun Error (cleared when read)
4	16($10)	Config	Master Station Enable
5	32($20)	Config	Synchronizing Master Station Enable
6	64($40)	Status	Sync Node Packet Received (cleared when read)
7	128($80)	Config	Sync Node Phase Lock Enable
8	256($100)	Config	Node 8 Master Address Check Disable
9	512($200)	Config	Node 9 Master Address Check Disable
10	1024($400)	Config	Node 10 Master Address Check Disable
11	2048($800)	Config	Node 11 Master Address Check Disable
12	4096($1000)	Config	Node 12 Master Address Check Disable
13	8192($2000)	Config	Node 13 Master Address Check Disable
14	16384($4000)	Config	Node 14 Master Address Check Disable
15	32768($8000)	Config	Node 15 Master Address Check Disable

In most applications, the only important configuration bits are bits 4, 5, and 7. In every MACRO ring, there must be one and only one synchronizing master. On this card, bits 4 and 5 should be set (1), but bit 7 should be clear (0). On this card, I995 should be set to $30, or $xx30 if any of the high bits are to be set.

If there are more that one PMAC2 acting as a masters on the ring, these should not be synchronizing masters, but they should enable "sync node phase lock" to stay synchronized with the synchronizing master. One these cards, bit 4 should be set, bit 5 should be clear, and bit 7 should be set, so I995 should be set to $90, or $xx90 if any of the high bits are to be set.

Bits 8-15 can be set individually to disable the "master address check" for their corresponding node numbers. This capability is for multi-master broadcast and synchronization. If the master address check is disabled, only the slave node number part of the packet address must match for a packet to be latched in. In this way, the synchronizing master can send the same data packet to multiple other master and slave stations. This common packet can be used to keep multiple stations synchronized using the sync lock function enabled with bit 7 of I995; the packet number is specified in I996 (packet 15 is suggested for this purpose).

I996 MACRO Node Activate Control

Range: .....$000000 to $FFFF (0 to 8,388,607)

Units: ..... none

Default: ..... $0 (all nodes de-activated)

Remarks



If the use of an activated node n includes auxiliary register functions, including servo flags, bit n of I1000 must also be set to 1.

I996 controls which of the 16 MACRO nodes on the card are activated. It also controls the master station number, and the node number of the packet that creates a synchronization signal. The bits of I996 are arranged as follows:

Bits 0 to 15 are individual control bits for the matching node number 0 to 15. If the bit is set to 1, the node is activated; if the bit is set to 0, the node is de-activated.
If the PMAC2 is a master station (likely) as determined by I995, it will send out a packet for each activated node every ring cycle (every phase cycle). When it receives a packet for an activated node, it will latch in that packet and not pass anything on.
If the PMAC2 is a slave station (unlikely but possible) as determined by I995, when it receives a packet for an activated node, it will latch in the contents of that packet into its read registers for that node address, and automatically substitute the contents of its write registers into the packet.
If a node is disabled, the PMAC2, whether master or slave, will still latch in the contents of a packet it receives, but it will also pass on the packet unchanged. This feature is particularly useful for the MACRO broadcast feature, in which multiple stations need to receive the same packet.

BIT #	VALUE	TYPE	FUNCTION
0	1($1)	Config	Node 0 Activate
1	2($2)	Config	Node 1 Activate
2	4($4)	Config	Node 2 Activate
3	8($8)	Config	Node 3 Activate
4	16($10)	Config	Node 4 Activate
5	32($20)	Config	Node 5 Activate

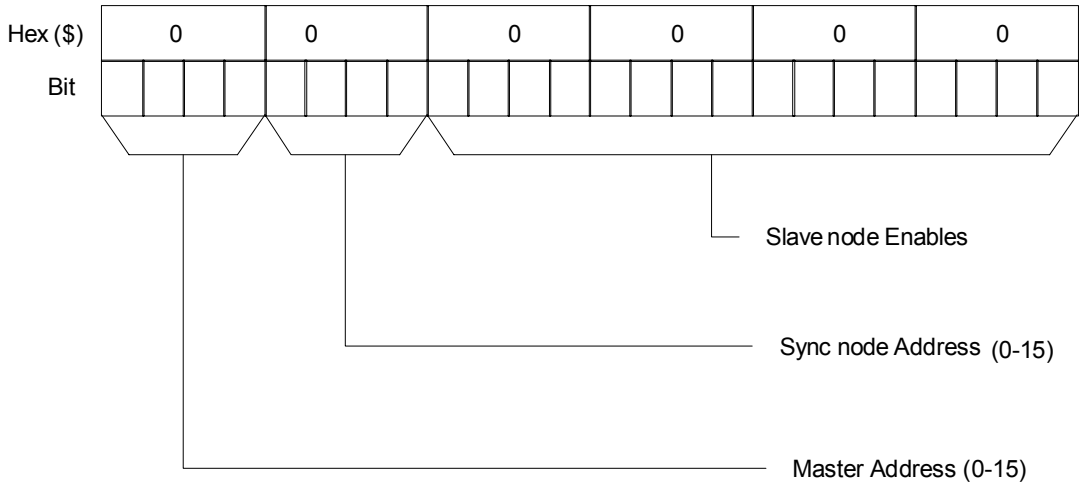
6	64($40)	Config	Node 6 Activate
7	128($80)	Config	Node 7 Activate
8	256($100)	Config	Node 8 Activate
9	512($200)	Config	Node 9 Activate
10	1024($400)	Config	Node 10 Activate
11	2048($800)	Config	Node 11 Activate
12	4096($1000)	Config	Node 12 Activate
13	8192($2000)	Config	Node 13 Activate
14	16384($4000)	Config	Node 14 Activate
15	32768($8000)	Config	Node 15 Activate
16-19	$X0000	Config	Packet Sync Node Slave Address (0 - 15)
20-23	$X00000	Config	Master Station Number (0-15)

On prototype PMAC2 boards that did not support multi-master MACRO rings, I996 only contained bits 0-15.)

Bits 16-19 together specify the slave number part of the packet address (0-15) that will cause a sync lock pulse on the card, if this function is enabled by I995. This function is useful for a PMAC2 that is a slave or non-synchronizing master on the ring, to keep it locked to the synchronizing master. If the master address check for this node is disabled with I995, only the slave number must match to create the sync lock pulse. If the master address check is left enabled, the master number part of the packet address must match the master number for the card, as set in bits 20-23 of I996.

If this card is the synchronizing master, this function is not enabled, so the value of these bits does not matter; they can be left at the default of 0.

Bits 20-23 specify the master number for the card (0-15). The number must be specified whether the card is a master station or a slave station.



Examples

Master number 0; Sync node address 0

Activated nodes 0-5; De-activated nodes 6-15:

```
I996 =0000 0000 0000 0000 0011 1111 (binary) = $00003F
```

Master number 1; Sync node address 15 ($F)

Activated nodes 0, 2, 4, 6, 8, 10, 12; other nodes de-activated:

```
I996 = 0001 1111 0001 0101 0101 0101 (binary) = $1F1555
```

I997 Phase Clock Frequency Control

On PMAC2 boards that are not "Ultralite", I997 does not control the Phase Clock frequency; I901 does. I997 has no effect on non-Ultralite versions of the PMAC2.

Range: .....

Units: PHASE Clock Frequency = MaxPhase Frequency / (I997+1)

Default: .....

PHASE Clock Frequency = 9.0346 kHz / 1 = 9.0346 kHz
(with default value of I992)

Remarks

If jumper E1 is ON, PMAC2 Ultralite gets its PHASE clock signal externally from a serial-port input, and I997 is not used.

I997, in conjunction with I992, determines the frequency of the PHASE clock on PMAC2 Ultralite. Each cycle of the PHASE clock, a set of MACRO ring information is transmitted, and any required motor phase commutation and digital current-loop algorithms are performed for specified motors.

If the phase clock frequency is set too high, lower priority tasks such as communications can be starved for time. If the background tasks are completely starved, the watchdog timer will trip, shutting down the board. If a normal reset of the board does not re-establish a state where the watchdog timer has not tripped and communications works well, it will be necessary to re-initialize the board by powering up with the E3 re-initialization jumper on. This restores default settings, so communication is possible, and I992 and I997 can be set to supportable values.

Specifically, I997 controls how many times the PHASE clock frequency is divided down from the "maximum phase" clock, whose frequency is set by I992. The PHASE clock frequency is equal to the "maximum phase" clock frequency divided by (I997+1). I997 has a range of 0 to 15, so the frequency division can be by a factor of 1 to 16. The equation for I997 is:

..... *I*997

The ratio of MaxPhase Freq. to PHASE Clock Freq. must be an integer.

Example

With a 20 kHz MaxPhase Clock frequency established by I992, and a desired 6.67 kHz PHASE clock frequency, the ratio between MaxPhase and PHASE is 3:

```
I997 = (20 / 6.67) - 1 = 3 -1 = 2
```

I998 Servo Clock Frequency Control

On PMAC2 boards that

Range: .....

are not "Ultralite", I997 does not control the Phase Clock frequency; I901 does. I997 has no effect on non-Ultralite versions of the PMAC2.

Units: Servo Clock Frequency = PHASE Clock Frequency / (I998+1)

Default: .....

PHASE Clock Frequency = 9.0346 kHz / (3+1) = 2.2587 kHz (with default values of I992 and I997)

Remarks

If jumper E1 is ON, PMAC2 Ultralite gets its SERVO clock signal externally from a serial-port input, and I998 is not used.

I998, in conjunction with I997 and I992, determines the frequency of the SERVO clock on PMAC2 Ultralite. Each cycle of the SERVO clock, PMAC2 Ultralite updates the commanded position for each activated motor, and executes the servo algorithm to compute the command output to the amplifier.

Specifically, I998 controls how many times the SERVO clock frequency is divided down from the PHASE clock, whose frequency is set by I992 and I997. The SERVO clock frequency is equal to the PHASE clock frequency divided by (I998+1). I998 has a range of 0 to 15, so the frequency division can be by a factor of 1 to 16. The equation for I998 is:

$$I998 = \frac{PhaseFreq(kHz)}{ServoFreq(kHz)}$$

The ratio of PHASE Clock Freq. to SERVO Clock Freq. must be an integer.

For execution of trajectories at the proper speed, I10 must be set properly to tell the trajectory generation software what the SERVO clock cycle time is. The formula for I10 is:

$$I10 = \frac{8{,}388{,}6}{ServoFreq}$$

In terms of the variables that determine the SERVO clock frequency on a PMAC2 Ultralite board, the formula for I10 is:

$$I10 = \frac{640}{9}(2 * I992 + 3)(I997 + 1)(I998 + 1)$$

Example

With a 6.67 kHz PHASE Clock frequency established by I900 and I997, and a desired 3.33 kHz SERVO Clock frequency:

```
I998 = (6.67 / 3.33) - 1 = 2 - 1 = 1
```

I999 Reserved for future use

MACRO Control I-Variables (PMAC2 Only)

I1000 MACRO Node Auxiliary Register Enable

Range: ..... 0 .. $FFFF (0 .. 65,535)
Units: ..... none
Default: 0

Remarks

This parameter controls which MACRO nodes PMAC performs automatic copying into and out of the auxiliary registers. Enabling this function for a node is required to use the auxiliary register as the flag register for a motor.
I1000 is a 16-bit variable. Bits 0 to 15 control the enabling of this copying function for MACRO nodes 0 to 15, respectively. A bit value of 1 means the copying function is enabled; a bit value of 0 means the copying function is disabled.
If the copying function is enabled for Node n (where n = 0 to F hex or 0 to 15 decimal), during each background "housekeeping" software cycle, PMAC copies the contents of Y:$0F7n to the Node n auxiliary write register, and copies the contents of the Node n auxiliary read register into X:$0F7n.
The copying function enabled by I1000 permits the use of the auxiliary registers for command and status flags plus Type 0 auxiliary read and write functions in PLC programs and on-line commands.
For each node whose auxiliary functions are enabled by I1000, I1002 must correctly specify for the node whether the Type 0 or Type 1 MACRO protocol is used.
If a value of I1003 greater than 0 has been saved into PMAC's non-volatile memory to enable Type 1 MACRO auxiliary communications with Node 15, then at subsequent power-up/resets, bit 15 of I1000 is automatically forced to 0 by PMAC firmware, regardless of the value saved for I1000. The reserves Node 15 for the Type 1 auxiliary communications alone.

See Also

MACRO Setup

I-Variables Ix25, I995, I996, I1002, I1003

I1001 MACRO Ring Check Period

Range: ..... 0 .. 255
Units: .....
.....servo cycles
Default: 0

Remarks

I1001 determines the period for PMAC2 to evaluate whether there has been a MACRO ring failure. If I1001 is greater than 0, PMAC2 must receive the "sync node" packet (as specified by I996) twice within I1001 servo cycles. It also must detect no more than one ring error (byte violation error, packet parity error, packet overflow error, or packet underflow error) in this same period. If either of these conditions is not met, PMAC2 will consider there to be a ring fault, and it will disable all motors.

If I1001 is 0, PMAC2 does not perform these checks, even if MACRO is active.
If I1001 is greater than 0, activating this check function, bit 7 of I995 (Sync Node Phase Lock Enable) must be set to 1, and bits 16 to 19 of I996 (Sync Packet Number) must specify the number of a packet that is being received by this card.

I1002 MACRO Node Protocol Type Control

Range: ..... 0 .. $FFFF (0 .. 65,535)
Units: .....
..... none
Default: 0

Remarks

I1002 controls for each MACRO node (0 - 15) whether the Type 0 or Type 1 MACRO protocol is used on that node. I1002 is a 16-bit value; each bit 0 -15 controls the protocol type for the MACRO node of the same number. A value of 0 in the bit selects the Type 0 protocol for the matching MACRO node; a value of 1 in the bit selects the Type 1 protocol for the node.
The key difference between Type 0 and Type 1 protocols is in which node register is used for control and status flags. In the Type 0 protocol, the 1st register (24 bits) is used for the flags; in the Type 1 protocol, the 4th register (16 bits) is used for the flags. The bits of I1002 must be set properly for any node whose auxiliary flag function is enabled by I1000.
The Type 0 protocol is generally used for single-node MACRO devices, such as the Performance Controls FLX Drive. The Type 1 protocol is generally used for multi-node MACRO devices, such as Delta Tau's Compact MACRO Station.

I1003 MACRO Type 1 Auxiliary Communications Timeout

Range: ..... 0 .. 255
Units: .....
..... Servo Cycles
Default: 0

Remarks

I1003 permits the enabling of MACRO Type 1 auxiliary communications using Node 15, which are executed with the **MS**, **MSR**, and **MSW** commands. If I1003 is set to 0, these communications are disabled. If I1003 is set to a value greater than 0, these communications are enabled, and the value of I1003 sets the "timeout" value for the auxiliary response, in PMAC servo cycles.
If PMAC has not received a response to the MACRO auxiliary communications command within I1003 servo cycles, it will stop waiting and register a "MACRO Auxiliary Communications Error", setting Bit 5 of global status register X:$0003. A value of 32 for I1003 is suggested

Bit 15 of I1000 must be set to 0 to disable Node 15's Type 0 (node-specific) auxiliary communications if I1003 is greater than 0. If a value of I1003 greater than 0 has been saved into PMAC's non-volatile memory, then at subsequent power-up/resets, bit 15 of I1000 is automatically forced to 0 by PMAC firmware, regardless of the value saved for I1000.

PMAC2 Default Conversion Tables

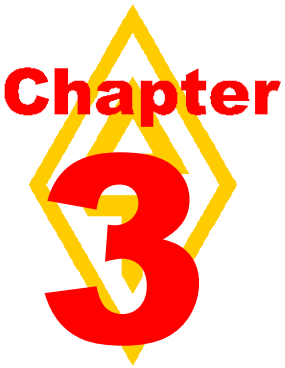
Non-Ultralite PMAC2 Boards

ADDRESS	SETTING	MEANING
Y:$0720	$00C000	(1/T Conversion of Encoder 1)
Y:$0721	$00C008	(1/T Conversion of Encoder 2)
Y:$0722	$00C010	(1/T Conversion of Encoder 3)
Y:$0723	$00C018	(1/T Conversion of Encoder 4)
Y:$0724	$00C020	(1/T Conversion of Encoder 5)
Y:$0725	$00C028	(1/T Conversion of Encoder 6)
Y:$0726	$00C030	(1/T Conversion of Encoder 7)
Y:$0727	$00C038	(1/T Conversion of Encoder 8)
Y:$0728	$00C090	(1/T Conversion of Handwheel 1)
Y:$0729	$00C098	(1/T Conversion of Handwheel 2)
Y:$072A	$400729	(Time Base Conversion of Handwheel 2)
Y:$072B	$000295	(Time Base Scale Factor)
Y:$072C	$000000	(End of Table)

PMAC2 Ultralite Boards

ADDRESS	SETTING	MEANING
Y:$0720	$28C0A0	(Unshifted parallel conversion of Node 0 Register 0)
Y:$0721	$FFFFFF	(Use all 24 bits; converted value in X:$0721)
Y:$0722	$28C0A4	(Unshifted parallel conversion of Node 1 Register 0)
Y:$0723	$FFFFFF	(Use all 24 bits; converted value in X:$0723)
Y:$0724	$28C0A8	(Unshifted parallel conversion of Node 4 Register 0)
Y:$0725	$FFFFFF	(Use all 24 bits; converted value in X:$0725)
Y:$0726	$28C0AC	(Unshifted parallel conversion of Node 5 Register 0)
Y:$0727	$FFFFFF	(Use all 24 bits; converted value in X:$0727)
Y:$0728	$28C0B0	(Unshifted parallel conversion of Node 8 Register 0)
Y:$0729	$FFFFFF	(Use all 24 bits; converted value in X:$0729)
Y:$072A	$28C0B4	(Unshifted parallel conversion of Node 9 Register 0)
Y:$072B	$FFFFFF	(Use all 24 bits; converted value in X:$072B)

Y:$072C	$28C0B8	(Unshifted parallel conversion of Node 12 Register 0)
Y:$072D	$FFFFFF	(Use all 24 bits; converted value in X:$072D)
Y:$072E	$28C0BC	(Unshifted parallel conversion of Node 13 Register 0)
Y:$072F	$FFFFFF	(Use all 24 bits; converted value in X:$072F)
Y:$0730	$0000000	(End of table)



PMAC ON-LINE COMMAND SPECIFICATION

On-Line Command Syntax and Function

PMAC has a large set of on-line commands. On-line commands are executed immediately by PMAC, and then discarded. Except for the most recently issued command, on-line commands are not retained by the PMAC. Some on-line commands are motor-specific – acting on the addressed motor (#n) – some are coordinate-system-specific – acting on the addressed coordinate system (&n) – and some are global – the command's action is independent of which motor or coordinate system is addressed. In the specification for each command in this section, the "Scope" of the command – motor specific, coordinate system specific, or global – is listed.

<CONTROL-A>

Function:.....Abort all programs and moves.
Scope: .....
.....Global
Syntax: .....`ASCII Value 1D;  $01`

Remarks

This command aborts all motion programs and stops all non-program moves on the card. It also brings any disabled or open-loop motors to an enabled zero-velocity closed-loop state. Each motor will decelerate at a rate defined by its own motor I-variable I*x*15. However, a multi-axis system may not stay on its programmed path during this deceleration.

A **<CTRL-A>** stop to a program is not meant to be recovered from gracefully, because the axes will in general not stop at a programmed point. The next programmed move will not behave properly unless a **PMATCH** command is given or I14 is set to 1 (these cause PMAC to use the aborted position as the move start position). Alternately, an on-line **J=** command may be issued to each motor to cause it to move to the end point that was programmed when the abort occurred. Then the program(s) can be resumed with an **R** (run) command.

To stop a motion sequence in a manner that can be recovered from easily, use instead the Quit (**Q** or **<CTRL-Q>**) or the Hold (**H** or **<CTRL-O>**) command.

When PMAC is set up to power on with all motors killed (I*x*80 = 0), this command can be used to enable all of the motors (provided that they are not commutated by PMAC -- in that case, each motor should be enabled with the **$** command).

For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

See Also

Stop Commands (Making Your Application Safe)

On-line commands **A**, **$**, **J=**, **PMATCH**, **H**, **<CTRL-O>**, **Q**, **<CTRL-Q>**

I-variables **I14**, **Ix15**, **Ix80**.

<CONTROL-B>

Function: ..... Report status word for all motors.
Scope: ..... Global
Syntax: ..... **ASCII Value 2D; $02**

Remarks

This command causes PMAC to report the status words for all of the motors to the host in hexadecimal ASCII form, 12 characters per motor starting with motor #1, with the characters for each motor separated by spaces. The characters reported for each motor are the same as if the **?** command had been issued for that motor.
The detailed meanings of the individual status bits are shown under the **?** command description.
For multiple cards on a single serial daisy-chain, this command affects only the card currently addressed in software (**@n**).

Example

<CTRL-B>

```
812000804001 812000804001 812000A04001 812000B04001 050000000000
050000000000 050000000000 050000000000<CR>
```

See Also

On-line commands **<CTRL-C>**, **<CTRL-G>**, **?**, **@*n***

Memory-map registers X:$003D, X:$0079, etc., Y:$0814, Y:$08D4;

Suggested M-Variable definitions Mx30-Mx45.

<CONTROL-C>

Function: ..... Report all coordinate system status words
Scope: ..... Global
Syntax: ..... **ASCII Value 3D, $03**

Remarks

This command causes PMAC to report the status words for all of the coordinate systems to the host in hexadecimal ASCII form, 12 characters per coordinate system starting with coordinate system 1, with the characters for each coordinate system separated by spaces. The characters reported for each coordinate system are the same as if the **??** command had been issued for that coordinate system.
The detailed meanings of the individual status bits are shown under the **??** command description.
For multiple cards on a single serial daisy-chain, this command affects only the card currently addressed in software (by the **@n** command).

Example

<CTRL-C>

```
A80020020000 A80020020000 A80020020000 A80020020000 A80020000000
A80020000000 A80020000000 A80020000000<CR>
```

See Also

On-line commands **<CTRL-B>**, **<CTRL-G>**, **??**;
Memory-map registers Y:$0817, Y:$08D7, etc., X:$0818, X:$08D8, etc.;
Suggested M-variable definitions Mx80-Mx90.

<CONTROL-D>

Function: ..... Disable all PLC programs.
Scope: ..... Global
Syntax: ..... **ASCII Value 4D; $04**

Remarks

This command causes all PLC programs to be disabled (i.e. stop executing). This is the equivalent of **DISABLE PLC 0..31** and **DISABLE PLCC 0..31**. It is especially useful if a **CMD** or **SEND** statement in a PLC has run amok.
For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

Example

```
TRIGGER FOUND

TRIGTRIGER FOTRIGGER FOUND

TRTRIGTRIGGER FOUND (Out-of-control SEND message from PLC)

<CTRL-D>  (Command to disable the PLCs)

     (No more messages; can now edit PLC)
```

See Also

On-line commands DISABLE PLC, ENABLE PLC, DISABLE PLCC, ENABLE PLCC, OPEN PLC

Program commands DISABLE PLC, ENABLE PLC, DISABLE PLCC, ENABLE PLCC, COMMAND, SEND

<CONTROL-E>



This command cannot be used over the VME bus.

Function: ..... Report con

Scope: .....

Syntax: .....

Remarks

This command causes PMAC to report back to the host the contents of those addresses specified by the data gathering I-variables (I20-I44). This is much like a one-shot data gather and report, except the values are reported back *in binary form*, not ASCII.

Each short word is 3 bytes (LSByte first); each long word is 6 bytes (LSByte first). On the serial port or VME bus, the bytes are reported sequentially, LSByte first. On the PCbus or STDbus, the bytes are sent out three at a time, with the LSByte placed at *base address* + 7 (the usual address), the middle byte placed at b*ase address* + 6, and the MSByte placed at *base address* + 5. There are no preceding or following handshake characters; it is your responsibility to know the number of bytes that he will get back, and write your receiving routine accordingly.

For multiple cards on a single serial daisy-chain, this command affects only the card currently addressed in software (by the **@n** command). The Example below shows how you can test this command by first writing to open memory registers, then reading back the contents with **<CTRL-E>**

Example

```
I20=7<CR>

I21=$0007F0<CR>

I22=$4007F1<CR>

I23=$8007F2<CR>

WY$07F0,$001122<CR>

WX$07F1,$334455<CR>

WL$07F2,$024680135791<CR>

<CTRL-E>

<$22><$11>$<$00><$55><$44><$33><$91><$57><$13>

<$80><$46><$02>
```

See Also

I-Variables I20-I44

On-line commands GATHER, LIST GATHER

<CONTROL-F>

Function: Report following errors for all motors.

Scope: Global.

Syntax: **ASCII Value 6D; $06**

Remarks

This command causes PMAC to report the following errors of all motors to the host. The errors are reported in an ASCII string, each error scaled in counts, rounded to the nearest tenth of a count. A space character is returned between the reported error for each motor.

Refer to the on-line **F** command for more detail as to how the following error is calculated.

For multiple cards on a single serial daisy-chain, this command affects only the card currently addressed in software (by the **@n** command).

Example

```
<CTRL-F>

0.5 7.2 -38.3 1.7 0 0 0 0<CR>
```

See Also

I-variables I*x*11, I*x*12

On-line commands **F**, **<CTRL-P>**, **<CTRL-V>**

<CONTROL-G>

Function: Report global status word.

Scope: ..... Global

Syntax: ..... **ASCII Value 7D; $07**

Remarks

This command causes PMAC to report the global status words to the host in hexadecimal ASCII form, using 12 characters. The characters sent are the same as if the **???** command had been sent, although no command acknowledgement character (<ACK> or <LF>, depending on I3) is sent at the end of the response.
The detailed meanings of the individual status bits are shown under the **???** command description.
For multiple cards on a single serial daisy-chain, this command affects only the card currently addressed in software (by the **@n** command).

Example

<CTRL-G>

003000400000<CR>

See Also

On-line commands <CTRL-B>, <CTRL-C>, ???
Memory-map registers X:$0003, Y:$0003.

<CONTROL-H>

Function: ..... Erase last character.

Scope: ..... Global

Syntax: ..... **ASCII Value 8D; $08 (<BACKSPACE>)**

Remarks

This character, usually entered by typing the **<BACKSPACE>** key when talking to PMAC in terminal mode, causes the most recently entered character in the PMAC command-line-receive buffer to be erased.

See Also

Talking to PMAC
On-line command <CTRL-O> (Feed Hold All)

<CONTROL-I>



Most versions of the PMAC Executive Program "trap" a ***<CTRL-I>*** *or* ***<TAB>*** *for their own purposes, and do not send it on to PMAC, even when in terminal mode*

Function: .....

Scope: .....

Syntax: ..... **ASCII**

Remarks

This character, sometimes entered by typing the **<TAB>** key, causes the most recently sent alphanumeric command line to PMAC to be re-commanded. It provides a convenient way to quicken a repetitive task, particularly when

working interactively with PMAC in terminal mode. Other control-character commands cannot be repeated with this command.

Example

This Example shows how the tab key can be used to look for some event:

```
PC<CR>

P1:10<CR>

<TAB>

P1:10<CR>

<TAB>

P1:10<CR>

<TAB>

P1:11<CR>
```

See Also

On-line command <CONTROL-Y>.

<CONTROL-K>

Function:.....Kill all motors.

Scope: .....
.....Global

Syntax: .....**ASCII Value 11D; $0B**

Remarks

This command kills all motor outputs by opening the servo loop, commanding zero output, and taking the amplifier enable signal (AENA*n*) *false* (polarity is determined by jumper E17) for all motors on the card. If any motion programs are running, they will automatically be aborted. (For the motor-specific **K** (kill) command, if the motor is in a coordinate system that is executing a motion program, the program execution must be stopped with either an **A** (abort) or **Q** (quit) command before PMAC will accept the **K** command.)

For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

See Also

On-line commands K, A, <CONTROL-A>.

<CONTROL-L>

Function:..... Close open rotary buffer.

Scope: ..... Global

Syntax: ..... **ASCII Value 12D; $0C**

Remarks

This character causes PMAC to close the open rotary program buffer on PMAC. It is exactly equivalent in effect to the **CLOSE** command, but it is faster to send. The primary use of **<CTRL-L>** is when the rotary buffer needs to be opened and closed repeatedly. After closing the rotary buffer, there is no chance that an on-line command can be mistaken for a buffer command.

Example

```
<CTRL-U>        ; Open rotary buffer

X10 Y20 F5 M3        ; Put program line in buffer

X30 Y40 F5      ; Put program line in buffer

<CTRL-L>        ; Close rotary buffer

M1          ; On-line command for value of M1

1           ; PMAC responds
```

See Also

Rotary Motion Program Buffers (Writing a Motion Program)
On-line commands <CTRL-U>, OPEN ROT, CLOSE

<CONTROL-M>

Function: ..... Enter command line.
Scope: ..... Gobal
Syntax: ..... **ASCII Value 13D; $0D (<CR>)**

Remarks

This character, commonly known as **<CR>** (carraige return), causes the alphanumeric characters in the the PMAC command-line-receive buffer to be interpreted and acted upon. (Control-character commands do not require a **<CR>** character to execute.)
Notice that for multiple PMACs daisy-chained together on a serial interface, this will act on all cards simultaneously, not just the software-addressed card. For simultaneous action on multiple cards, it is best to load up the command-line-receive buffers on all cards before issuing the **<CR>** character.

Example

```
#1J+<CR>
P1<CR>
@0&1B1R@1&1B7R<CR>        (This causes card 0 on the serial daisy-chain to have  its CS 1 execute PROG 1 and card 1 to have its CS 1    execute PROG 7 simultaneously.)
```

See Also

Talking to PMAC

<CONTROL-N>

*The PMAC Executive Program terminal mode will not display the checksum values resulting from a **<CTRL-N>** command.*

Function: .....
Scope: .....
Syntax: .....

Remarks

This character causes PMAC to calculate and report the checksum of the alphanumeric characters of the present command lines (i.e. since the most recent carriage-return character).
As typically used, the host computer would send the entire command line up to, but not including, the carriage return. It would then send the **<CTRL-N>** character, and PMAC would return the checksum value. If this value agreed with the host's internally calculated checksum value, the host would then send the **<CR>** and PMAC would execute the command line. If the values did not agree, the host would send a **<CTRL-X>** command to erase the command line, then resend the line, repeating the process.

Example

```
With I4=1 and I3=2:
Host sends:     J+<CTRL-N>
PMAC sends:     <117dec>  (117=74[J] + 43[+]; correct)
Host sends:     <CR>
```

```
PMAC sends:    <ACK><117dec>  (handshake and checksum again)
Host sends:    J/<CTRL-N>
PMAC sends:    <122dec>  (122 != 74[J] +47[/]; incorrect)
Host sends:    <CTRL-X>  (Erase the incorrect command)
     J/<CTRL-N>(Send the command again)
PMAC sends:    <121dec>  (121 = 74[J] + 47[/]; correct)
Host sends:    <CR>
PMAC sends:    <ACK><121dec>  (handshake and checksum again)
```

See Also

Communications Checksum (Writing a Host Communications Program)

I-variables I3, I4

On-line commands <CTRL-M> (<CR>), <CTRL-X>

<CONTROL-O>

Function: ..... Feed hold on all coordinate systems.
Scope: .....
..... Global
Syntax: ..... **ASCII Value 15D; $0F**

Remarks

This command causes all coordinate systems in PMAC to undergo a feed hold. A feed hold is much like a **%0** command where the coordinate system is brought to a stop without deviating from the path it was following, even around curves. However, with a feed hold, the coordinate system slows down at a slew rate determined by I*x*95, and can be started up again with an **R** (run)command. The system then speeds up at the rate determined by I*x*95, until it reaches the desired **%** value (from internal *or* external timebase). From then on, any timebase changes occur at a rate determined by I*x*94.

For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

On a flash memory PMAC that is in bootstrap mode (powered up with E51 ON), the **<CTRL-O>** command puts PMAC into its firmware reload command. All subsequent characters sent to PMAC are interpreted as bytes of machine code for the PMAC operational firmware, overwriting the existing operational firmware in flash memory.

See Also

Resetting PMAC (Talking to PMAC)

I-variables I52, I*x*94, I*x*95

On-line commands **<CTRL-H>** (backspace) **H** (feedhold), **R** (run), **%** (feedrate override), **** (program hold).

Jumper E51

<CONTROL-P>

Function: ..... Report positions of all motors.
Scope: .....
..... Global
Syntax: ..... **ASCII Value 16D; $10**

Remarks

This command causes the positions of all motors to be reported to the host. The positions are reported as an ASCII string, scaled in counts, rounded to the nearest tenth of a count, with a space character in between each motor's position.
The position window in the PMAC Executive program works by repeatedly sending the **<CTRL-P>** command and rearranging the repsonse into the window.
PMAC reports the value of the *actual position register* plus the *position bias register* plus the *compensation correction register*, and if bit 16 of Ix05 is 1 (handwheel offset mode), minus the *master postion register*.
For multiple cards on a single serial daisy-chain, this command affects only the card currently addressed in software (by the **@n** command).

Example

```
<CTRL-P>

9999.5 10001.2 5.7 -2.1 0 0 0 0<CR>
```

See Also

On-line commands **P**, **<CTRL-V>**, **<CTRL-F>**.

<CONTROL-Q>

Function: ..... Quit all executing motion programs.
Scope: .....
..... Global
Syntax: ..... **ASCII Value 17D; $11**

Remarks

This command causes any and all motion programs running in any coordinate system to stop executing after the moves that have already been calculated are finished. Program execution may be resumed from this point with the **R** (run) or **S** (step) commands.
For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

See Also

On-line commands **<CTRL-A>, <CTRL-K>, <CTRL-O>, <CTRL-R>, <CTRL-S>, Q**

Motion-program command **STOP**.

<CONTROL-R>

Function: ..... Begin execution of motion programs in all coordinate systems.

Scope: .....
..... Global

Syntax: ..... **ASCII Value 18D; $12**

Remarks

This command is the equivalent of issuing the **R** (run) command to all coordinate systems in PMAC. Each active coordinate system (i.e. one that has at least one motor assigned to it) that is to run a program must already be pointing to a motion program (initially this is done with a **B{*prog num*}** command).
For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

For a flash memory PMAC that is in bootstrap mode (powered up with E51 ON), the <CTRL-R> command puts PMAC into normal operational mode, but with factory default I-variables, conversion table settings, and VME/DPRAM addresses.

Example

```
&1B1&2B500<CR>

<CTRL-R>
```

See Also

Executing a Motion Program (Writing a Motion Program)

Resetting PMAC (Talking to PMAC)

On-line commands **R**, **B**.

Jumper E51

<CONTROL-S>

Function:.....Step working motion programs in all coordinate systems.

Scope: .....
.....Global

Syntax: .....**ASCII Value 19D; $13**

Remarks

This command is the equivalent of issuing an S (step) command to all of the coordinate systems in PMAC. Each active coordinate system (i.e. one that has at least one motor assigned to it) that is to run a program must already be pointing to a motion program (initially this is done with a B{*prog num*} command).

A program that is not running will execute all lines down to and including the next motion command (move or dwell), or if it encounters a **BLOCKSTART** command first, all lines down to and including the next **BLOCKSTOP** command.

If a program is already running in continuous execution mode (from an **R** (run) command), an **S** command will put the program in single-step mode, stopping execution after the next motion command). In this situation, it has exactly the same effect as a **Q** (quit) command.

For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

See Also

On-line commands **<CTRL-A>, <CTRL-O>, <CTRL-Q>, <CTRL-R>, A, H, O, Q, R, S;**

Motion-program commands **BLOCKSTART, BLOCKSTOP, STOP.**

Control-panel port (JPAN) input STEP/.

<CONTROL-T>

Function:.....Toggle serial port half/full duplex mode.
Scope: .....
.....Global
Syntax: .....**ASCII Value 20D; $14**

Remarks

This causes *serial* port communications to toggle between half duplex (PMAC will not echo character back to host) and full duplex (PMAC will echo character back to host). The power-up default is half duplex.
This command is invalid when multiple PMACs are daisy-chained on a single serial interface.

See Also

Data Integrity Checks (Writing a Host Communications Program)

On-line command **<CTRL-Z>**

<CONTROL-U>

Function: ..... Open rotary program buffer(s).
Scope: .....
.....Global
Syntax: .....**ASCII Value 21D; $15**

Remarks

This character causes PMAC to open all existing rotary motion program buffers for entry. It is exactly equivalent in effect to the **OPEN ROTARY** command, but it is faster to send. Along with the **<CTRL-L>** command, it permits rapid opening and closing of the rotary buffer, so that on-line commands can be sent with the buffer closed without chance that they will be mistaken for buffer commands.

Example

```
<CTRL-L>  ; Close rotary buffer
M1    ; On-line command for value of M1
1     ; PMAC responds
<CTRL-U>  ; Open rotary buffer
X10 Y20 F5 M3  ; Put program line in buffer
X30 Y40 F5; Put program line in buffer
```

See Also

Rotary Motion Program Buffers (Writing a Motion Program)

On-line commands **<CTRL-L>**, **OPEN ROT**, **CLOSE**

<CONTROL-V>



The velocity values reported here are obtained by subtracting positions of consecutive servo cycles. As such, they can be very noisy. For purposes of display, it is probably better to use averaged velocity values held in registers Y:$082A, Y:$08EA, etc., accessed with M-variables

Function: .....

Scope: .....

Syntax: .....

Remarks

This command causes PMAC to report the velocities of all motors to the host. The velocity units are in encoder counts per servo cycle, rounded to the nearest tenth. The <F7> velocity window in the PMAC Executive program works by repeatedly issuing the <CTRL-V> command and displaying the response on the screen.
To scale these values into counts/msec, multiply the response by 8,388,608*(Ix60+1)/I10 (servo cycles/msec).
For multiple cards on a single serial daisy-chain, this command affects only the card currently addressed in software (**@n**).

See Also

I-variable I10, Ix60

On-line commands <CTRL-F>, <CTRL-P>, V

Memory registers X:$0033, X:$006F, etc.

Suggested M-variable definitions Mx66

<CONTROL-W>

This method of DPRAM communications has largely been superseded by the bidirectional ASCII DPRAM buffers enabled by I58.

Function: ..... Take com

Scope: .....

Syntax: .....

Remarks

This command causes PMAC to take an ASCII command line from registers in the dual-port RAM (Option 2 is required). This permits the command itself to be sent through the dual-ported RAM, so only the <CONTROL-W> character itself needs to be sent through the communications port.
The ASCII command can be in any unused memory registers, but it is mainly useful in the dual-ported RAM. The **<CTRL-W>** can come through any port, but it is mainly useful through the bus port (PC or VME).
There is a two-stage addressing procedure that tells PMAC where to get the command when it receives a **<CTRL-W>**. PMAC I-variable I47 contains the PMAC memory address of a register that itself contains the memory address of the start of the command. That is, I47 contains the address of the *pointer* to the command, not the address of the command itself. The pointer register is usually in DPRAM itself. This double-indirect addressing allows many commands to be pre-loaded into the DPRAM, so that only the contents of the pointer register need to be changed for a new command.
When PMAC receives a **<CTRL-W>** command, it first looks at the Y memory address specified by I47. It next looks to the address specified by the contents of this register. It will then read the command line from memory, starting with the LSByte (bits 0-7) of the Y register of the specified address. The second character will come from the MSByte (bits 8-15) of the same register; the 3rd character will come from the LSByte of the X register at the same address; the 4th character will come from the MSByte of that register; the 5th character will come from the LSByte of the Y register of the next higher address; and so on, until a <*NULL*> character (ASCII value of zero) is found. A command line must have a <*CR*> character (ASCII value of 13) before the <*NULL*> in order to be acted upon .
PMAC will still respond to any command received this way through the selected host port, not the DPRAM.
For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

Example

To try this command, it can be useful to work first entirely from the PMAC side of DPRAM. This can be done very easily in the terminal mode of the PMAC Executive program.
First write the command string(s) into DPRAM. To write the command **P1=2<CR>** into DPRAM starting at address $D201, and the command **P1=0<CR>** starting at $D203, use the following commands:

```
WY$D201,$3150  ; 50 is P, 31 is 1
```

```
WX$D201,$323D  ; 3D is =, 32 is 2

WY$D202,$000D  ; 0D is <CR>, 00 is <NULL>

WY$D203,$3150  ; 50 is P, 31 is 1

WX$D203,$303D  ; 3D is =, 30 is 0

WY$D204,$000D  ; 0D is <CR>, 00 is <NULL>
```

Next, set I47 to $D200 (type **I47=$D200**), and point to the first command with **WY$D200,$D201**. Now issue the **<CTRL-W>** command. Variable P1 should now be set to 2. Verify this by typing **P1**.
To point to the next command, type **WY$D200,$D203**. Issue the **<CTRL-W>** command, and P1 should change back to 0. Verify this by typing **P1** again.
Of course, in actual use, you will write to DPRAM through the host side, not the PMAC side. But the above Example is good to understand and debug the technique.

See Also

Dual-Ported RAM Description (Writing a Host Communications Program)

I-variable I47, I58.

<CONTROL-X>



This command empties the command queue in PMAC RAM, but it cannot erase the 1 or 2 characters already in the response port. A robust algorithm for clearing responses would include 2 character read commands that can time-out if necessary.

Function: C

Scope: .....

Syntax: .....

Remarks

This command causes the PMAC to stop sending any messages that it had started to send, even multi-line messages. This also causes PMAC to empty the command queue from the host, so it will erase any partially sent commands. It can be useful to send this before sending a query command for which you are expecting an exact response format, if you are not sure what PMAC has been doing before, because it makes sure nothing else comes through before the expected response. As such, it is often the first character sent to PMAC from the host when trying to establish initial communications.

For multiple cards on a single serial daisy-chain, this command affects all cards on the chain, regardless of the current software addressing.

See Also

On-line commands **<CTRL-H>**, **<CTRL-Z>**

<CONTROL-Y>

Function: ..... Report last command line.

Scope: .....
..... Global

Syntax: ..... **ASCII Value 25D; $19**

Remarks

This causes PMAC to report the last command line to the host (with no trailing <*CR*>) and to re-enter the line into the command queue ready to execute upon the next receipt of <*CR*>. This allows a user communicating with PMAC in terminal mode to recall the last command and to be able to edit it with the backspace and typing in desired changes. The command will be re-executed when the host sends a <*CR*>.

Example

```
P123=5<CR>Set the first value

P124=7<CR>Set the second value

P123<CR>  Query the first value

5     PMAC responds with value

<CTRL-Y>  Tell PMAC to report last command

P123 PMAC reports last command
```

```
<BACKSPACE>4<CR>     Modify to P124 and send

7    PMAC tells value of P124
```

See Also

On-line command **<CONTROL-I>**.

<CONTROL-Z>

Function:.....Set PMAC in serial port communications mode.

Scope: .....
.....Global

Syntax: .....**ASCII Value 26D; $1A**

Remarks

This command causes the the PMAC serial port to become the active communications output port. All PMAC responses directed to the host will be sent over the serial port. This mode will continue until a command is received over the bus (parallel) port, which will make the bus port the active communications output port. PMAC powers up/resets with the serial port the active port.

If you are trying to establish communications with PMAC over the serial port, it is a good idea to send this character before any query commands to make sure PMAC will try to respond over the serial port.

Regardless of which is the active output port, PMAC can accept commands over either port. It is the user's responsibility not to garble commands by simultaneously commanding over both ports.

Example

```
Serial host sends:       P1

PMAC responds to serial port: 12

Bus host sends:P1=P1+1

Serial host sends:       P1

PMAC responds to bus port:    13

(Serial host gets no response)

Serial host sends:       <CTRL-Z>P1

PMAC responds to serial port: 13
```

See Also

On-line commands **<CTRL-T>**, **<CTRL-X>**;

Jumpers E44-E47.

# #

Function: ..... Report currently addressed motor
Scope: ..... Global
Syntax: ..... **#**

Remarks

This causes PMAC to return the number of the motor currently addressed by the host -- the one which acts upon motor-specific commands from the host.
Notice that a different motor may be hardware selected from the control panel port for motor-specific control panel inputs, and that different motors may be addressed from programs within PMAC for **COMMAND** statements.

Example

```
#     Ask PMAC which motor is addressed

2     PMAC reports that motor 2 is addressed
```

See Also

Control-Panel Port Inputs (Connecting PMAC to the Machine)

On-line commands **#{constant}**, **&**, **&{constant}**, **@{constant}**

Program commands **ADDRESS**, **COMMAND**

#{constant}

A different motor may simultaneously be hardware selected from the control panel port for motor-specific control panel

Function: .....
Scope: .....
Syntax: .....

inputs, and that different motors may be addressed from programs within PMAC for ***COMMAND*** *statements.*

{constant} is an integer from 1 to 8, representing the number of the motor to be addressed

Remarks

This command makes the motor specified by **{constant}** the addressed motor (the one on which on-line motor commands will act). The addressing is modal, so all further motor-specific commands will affect this motor until a different motor is addressed. At power-up/reset, Motor 1 is addressed.

Example

```
#1J+ Command Motor 1 to jog positive

J-   Command Motor 1 to jog negative

#2J+ Command Motor 2 to jog positive

J/   Command Motor 2 to stop jogging
```

See Also

Control-Panel Port Inputs (Connecting PMAC to the Machine)

Addressing commands (Talking to PMAC)

Program commands **COMMAND**, **ADDRESS**

On-line commands **#**, **&**, **&{constant}**, **@{constant}**

#{constant}->

Function:.....Report the specified motor's coordinate system axis definition.
Scope: .....
.....Coordinate-system specific
Syntax: ..... **#{constant}->**

Where:

{constant} is an integer from 1 to 8 representing the number of the motor whose axis definition is requested

Remarks

This command causes PMAC to report the current axis definition of the specified motor *in the currently addressed coordinate system*. If the motor has not been defined to an axis in the currently addressed system, PMAC will return a **0** (even if the motor has been assigned to an axis in another coordinate system). A motor can have an axis definition in only one coordinate system at a time.

Example

```
&1   ; Address Coordinate System 1

#1-> ; Request Motor 1 axis definition in C.S. 1

10000X    ; PMAC responds with axis definition

&2   ; Address Coordinate System 2

#1-> ; Request Motor 1 axis definition in C.S. 2
```

```
0     ; PMAC shows no definition in this C.S.
```

See Also

Axes, Coordinate Systems (Setting Up a Coordinate System)

On-line commands **#{constant}->0**, **#{constant}->{axis definition}**, **UNDEFINE**, **UNDEFINE ALL**.

#{constant}->0

Function: ..... Clear axis definition for specified motor.
Scope: ..... Coordinate-system specific
Syntax: ..... **#{constant}->0**

Where

{constant} is an integer from 1 to 8 representing the number of the motor whose axis definition is to be cleared

Remarks

This command clears the axis definition for the specified motor *if* the motor has been defined to an axis in the currently addressed coordinate system. If the motor is defined to an axis in another coordinate system, this command will not be effective. This allows the motor to be redefined to another axis in this coordinate system or a different coordinate system.

Compare this command to **UNDEFINE**, which erases all the axis definitions in the addressed coordinate system, and to **UNDEFINE ALL**, which erases all the axis definitions in all coordinate systems.

Example

This Example shows how the command can be used to move a motor from one coordinate system to another:

```
&1    ; Address C.S. 1

#4->  ; Request definition of #4

5000A      ; PMAC responds

#4->0      ; Clear definition

&2    ; Address C.S. 2

#4->5000A ; Make new definition in C.S. 2
```

See Also

Axes, Coordinate Systems (Setting Up a Coordinate System)

On-line commands **UNDEFINE**, **UNDEFINE ALL**, **#{constant}->{axis definition}**.

#{constant}->{axis definition}

Function: ..... Assign an axis definition for the specified motor.
Scope: ..... Coordinate-system specific
Syntax: ..... **#{constant}->{axis definition}**

Where

{constant} is an integer from 1 to 8 representing the number of the motor whose axis definition is to be made;

{axis definition} consists of:

1 to 3 sets of **[{scale factor}]{axis}**, separated by the **+** character, in

Which:

the optional **{scale factor}** is a floating-point constant representing the number of motor counts per axis unit (engineering unit); if none is specified, PMAC assumes a value of 1.0;

{axis} is a letter (X, Y, Z, A, B, C, U, V, W) representing the axis to which the motor is to be matched;

[+{offset}] (optional) is a floating-point constant representing the difference between axis zero positon and motor zero (home) position, in motor counts; if none is specified, PMAC assumes a value of 0.0

Notice that no space is allowed between the motor number and the "arrow" double character.

Remarks

This command assigns the specified motor to a set of axes in the addressed coordinate system. It also defines the scaling and starting offset for the axis or axes.

In the vast majority of cases, there is a one-to-one matching between PMAC motors and axes, so this axis definition statment only uses one axis name for the motor.

A scale factor is typically used with the axis character, so that axis moves can be specified in standard units (e.g. millimeters, inches, degrees). *This number is what defines what the user units will be for the axis.* If no scale factor is specified, a user unit for the axis is one motor count. Occasionally an offset parameter is used to allow the axis zero position to be different from the motor home position. (This is the starting offset; it can later be changed in several ways, including the **PSET**, **{axis}=**, **ADIS**, and **IDIS** commands).

If the specified motor is currently assigned to an axis in a different coordinate system, PMAC will reject this command (reporting an ERR003 if I6=1 or 3). If the specified motor is currently assigned to an axis in the addressed coordinate system, the old definition will be overwritten by this new one.

To undo a motor's axis definition, address the coordinate system in which it has been defined, and use the command **#{constant}->0**. To clear all of the axis definitions within a coordinate system, address the coordinate system and use the **UNDEFINE** command. To clear all axis definitions in *all* coordinate systems, use **UNDEFINE ALL**.

For more sophisticated systems, two or three cartesian axes may be defined as a linear combination of the same number of motors. This allows coordinate system rotations and orthogonality corrections, among other things. One to three axes may be specified (if only one, it amounts to the simpler definition above). All axes specified in one definition must be from the same triplet set of cartesian axes: XYZ or UVW. If this multi-axis definition is used, a command to move an axis will result in multiple motors moving.

Example

```
#1->X      ; User units = counts

#4->2000 A; 2000 counts/user unit

#8->3333.333Z-666.667     ; Non-integers OK

#3->Y      ; 2 motors may be assigned to the same axis;

#2->Y      ; both motors move when a Y move is given

#1->8660X-5000Y;This provides a 30o rotation of X and Y...

#2->5000X+8660Y;with 10000 cts/unit -- this rotation does

#3->2000Z-6000 ;not involve Z, but it could have
```

This Example corrects for a Y axis 1 arc minute out of square:

```
#5->100000X     ;100000 cts/in

#6->-29.1X+100000Y        ;sin and cos of 1/60
```

See Also

Axes, Coordinate Systems (Setting Up a Coordinate System)

On-line commands **#{constant}->**, **#{constant}->0**, **UNDEFINE**, **UNDEFINE ALL**.

$

Function:.....Reset motor
Scope: .....
..... Motor specific
Syntax: .....**$**

Remarks

This command causes PMAC to initialize the addressed motor, performing any required commutation phasing and full reading of an absolute position sensor, leaving the motor in a closed-loop zero-velocity state. (For a non-commutated motor with an incremental encoder, the **J/** command may also be used.)
This command is necessary to initialize a PMAC-commutated motor after power-up/reset if Ix80 for the motor is set to 0. If Ix80 is 1, the initialization will be done automatically during the power-up/reset cycle.
This command will not be accepted if the motor is executing a move.

Example

```
I180 ; Request value of #1 power-on mode variable
0    ; PMAC responds with 0; powers on unphased and killed
$$$  ; Reset card; motor is left in killed state
#1$  ; Initialize motor, phasing and reading as necessary
```

See Also

Absolute Sensors (Setting Up a Motor)
Power-on Phasing (Setting Up PMAC Commutation)
I-variables Ix10, Ix73, Ix74, Ix75, Ix80, Ix81
On-line commands **$$$**, **J/**

$$$

Function:.....Full card reset.
Scope: .....
.....Global
Syntax: .....**$$$**

Remarks

This command causes PMAC to do a full card reset. The effect of **$$$** is equivalent to that of cycling power on PMAC, or taking the INIT/ line low, then high.
With jumper E51 in its default state (OFF for PMAC-PC, -Lite, -VME, ON for PMAC-STD), this command does a standard reset of the PMAC. On the PMAC without the Option CPU section (not option 4A, 5A, or 5B), I-variable values, conversion-table settings, and DPRAM and VMEbus addresses stored in permanent memory (EAROM) by the last **SAVE** command are reloaded into active memory (RAM). All information stored in battery backed RAM such as P-variable and Q-variable values, M-variable definitions, and motion and PLC programs are not changed by this command.
On the PMAC with the Option CPU section (option 4A, 5A, or 5B), PMAC copies the contents of the flash memory into active memory during a normal reset cycle, overwriting any current contents. *This means that anything changed in the PMAC active memory that was not saved to flash memory will be lost.* Even the last saved P-variable

and Q-variable values, M-variable definitions, and motion and PLC programs are copied from flash to RAM during the reset cycle.
With jumper E51 in non-default state (ON for PMAC-PC, -Lite, -VME, OFF for PMAC-STD), this command does a reset and re-initialization of the PMAC. On the PMAC without the Option CPU section (not option 4A, 5A, or 5B), factory default I-variable values, conversion-table settings, and DPRAM and VMEbus addresses stored in the firmware (EPROM) are copied into active memory (RAM). (Values stored in EAROM are not lost; they are simply not used.)
On the PMAC with the Option CPU section (option 4A, 5A, or 5B), PMAC enters a special re-initialization mode called "bootstrap mode" that permits the downloading of new firmware (see PMAC PROM SOFTWARE UPDATE LISTING for details of this mode). In this bootstrap mode, there are very few command options. To bypass the download operation in this mode, send a **<CONTROL-R>** character to PMAC. This puts PMAC in the normal operational mode with the existing firmware. Factory default values for I-variables, conversion table settings, and bus addresses for DPRAM and VME are copied from the firmware section of flash memory into active memory. The saved values of these values are not used, but they are still kept in the user section of flash memory.
Because this command immediately causes PMAC to enter its power-up/rest cycle, there is no acknowledging character (**<ACK>** or **<LF>**) returned to the host.

Example

```
I130=60000; Change #1 proportional gain

SAVE ; SAVE I-variables to EAROM

I130=80000; Change gain again

$$$  ; Reset card

I130 ; Request value of parameter

60000     ; PMAC reports current value, which is SAVEd value

(Put E51 on)

$$$  ; Reset card

I130 ; Request value of parameter

2000 ; PMAC reports current value, which is default
```

See Also

Resetting PMAC (Talking to PMAC)
PMAC PROM SOFTWARE UPDATE LISTING
Control-Panel Port INIT/ Input (Connecting PMAC to the Machine)
On-line command $$$***
I-variables I5, Ix80
JPAN Connector Pin 15
Jumper E51.

$$$***

Function:..... Global card reset and reinitialization.

Scope: ..... Global

Syntax: ..... **$$$*****

Remarks

This command performs a full reset of the card and reinitializes the memory. All programs and other buffers are erased. All I-variables, encoder conversion table entries, and VME and DPRAM addressing parameters are returned to their factory defaults. (Previously **SAVE**d values for these parameters are still held in EAROM, and can be brought into active memory with a subsequent **$$$** command). It will also recalculate the firmware checksum reference value and eliminate any PASSWORD that might have been entered.

M-variable definitions, P-variable values, Q-variable values, and axis definitions are not affected by this command. They can be cleared by separate commands (e.g. **M0..1023->***, **P0..1023=0**, **Q0..1023=0**, **UNDEFINE ALL**).

This command is particularly useful if the program buffers have become corrupted. It clears the buffers and buffer pointers so the files can be re-sent to PMAC. Regular backup of parameters and programs to the disk of a host computer is strongly encouraged so this type of recovery is possible. The PMAC Executive program has "Save Full PMAC Configuration" and "Restore Full PMAC Configuration" Functions to make this process easy.

With jumper E51 in non-default state (ON for PMAC-PC, -Lite, -VME, OFF for PMAC-STD), A PMAC with the Option CPU section (option 4A, 5A, or 5B) enters a special re-initialization mode called "bootstrap mode" when this command is given. This mode permits the downloading of new firmware (see PMAC PROM SOFTWARE UPDATE LISTING for details of this mode). In this mode, there are very few command options. To bypass the download operation in this mode, send a **<CONTROL-R>** character to PMAC. This puts PMAC in the normal operational mode with the existing firmware. Factory default values for I-variables, conversion table settings, and bus addresses for DPRAM and VME are copied from the firmware section of flash memory into active memory. The saved values of these values are not used, but they are still kept in the user section of flash memory.

Example

```
I130=60000; Set #1 proportional gain
SAVE ; Save to non-volatile memory
$$$***    ; Reset and re-initialize card
I130 ; Request value of I130
2000 ; PMAC reports current value, which is default
$$$  ; Normal reset of card
I130 ; Request value of I130
60000     ; PMAC reports current value, which is SAVEd value
```

See Also

On-line command **$$$**;

PMAC PROM SOFTWARE UPDATE LISTING

Jumper E51

PMAC Executive Program Save/Restore Full Configuration.

%

Function: ..... Report the addressed coordinate system's feedrate override value.

Scope: ..... Coordinate-system specific

Syntax: ..... **%**

Remarks

This command causes PMAC to report the present feedrate-override (time-base) value for the currently addressed coordinate system. A value of 100 indicates "'real time"'; i.e. move speeds and times occur as specified.
PMAC will report the value in response to this command, regardless of the source of the value (even if the source is not the **%{constant}** command).

Example

```
%     ; Request feedrate-override value
100   ; PMAC responds: 100 means real time
H     ; Command a feed hold
%     ; Request feedrate-override value
0     ; PMAC responds: 0 means all movement frozen
```

See Also

Time-Base Control (Synchronizing PMAC to External Events)

I-Variables I10, Ix93, Ix94, Ix95

On-line commands **%{constant}**, **H**

Memory map registers X:$0808, X:$08C8, etc.

%{constant}

Function: ..... Set the addressed coordinate system's feedrate override value.
Scope: .....
.....Coordinate-system specific
Syntax: ..... **{constant}**

Where:

{constant} is a non-negative floating point value specifying the desired feedrate override (time-base) value (100 represents real-time)

Remarks

This command specifies the feedrate override value for the currently addressed coordinate system. The rate of change to this newly specified value is determined by coordinate system I-variable Ix94.
I-variable Ix93 for this coordinate system must be set to its default value (which tells to coordinate system to take its time-base value from the %-command register) in order for this command to have any effect.

The maximum % value that PMAC can implement is equal to (2^{23}/I10)*100 or the (servo update rate in kHz)*100. If you specify a value greater than this, PMAC will saturate at this value instead.
If you want to control the time base based on a variable value, you should assign an M-variable (suggested M197) to the commanded time base register (X:$0806, X:$08C6, etc.), then assign a variable value to the M-variable. The value assigned here should be equal to the desired % value times (I10/100).

Example

```
%0    ; Command value of 0, stopping motion

%33.333    ; Command 1/3 of real-time speed

%100 ; Command real-time speed

%500 ; Command too high a value

%    ; Request current value

225.88230574    ; PMAC responds; this is max allowed value

M197->X:$0806,24          ; Assign variable to C.S. 1 % command
reg.

M197=P1*I10/100; Equivalent to &1%(P1)
```

See Also

Time-Base Control (Synchronizing PMAC to External Events)

I-Variables I10, Ix93, Ix94, Ix95

On-line commands **%**, **H**

Memory map registers X:$0806, X:$08C6, etc.

&{constant}

Function:.....Address a coordinate system.
Scope: .....
.....Global
Syntax: ..... **&{constant}**

Where:

{constant} is an integer from 1 to 8, representing the number of the coordinate system to be addressed

Remarks

This command makes the coordinate system specified by **{constant}** the addressed coordinate system (the one on which on-line coordinate-system commands will act). The addressing is modal, so all further coordinate-system-specific commands will affect this coordinate system until a different coordinate system is addressed. At power-up/reset, Coordinate System 1 is addressed.
Notice that a different coordinate system may simultaneously be hardware selected from the control panel port for coordinate-system-specific control panel inputs, and that different coordinate systems may be addressed from programs within PMAC for **COMMAND** statements.
If the control-panel inputs are disabled by I2=1, the host-addressed coordinate system also controls the indicator lines for the in-position, warning-following-error, and fatal-following-error **Function:..** s. These indicator lines connect to both control-panel port outputs (all PMAC versions), and to the interrupt controller (PMAC-PC, PMAC-Lite, PMAC-STD). (If I2=0, the hardware-selected coordinate system controls these lines.)

Example

```
&1B4R      ; C.S.1 point to Beginning of Prog 4 and Run
Q     ; C.S.1 Quit running program
&3B6R      ; C.S.3 point to Beginning of Prog 5 and Run
A     ; C.S.3 Abort program
```

See Also

I-variable I2

On-line commands **#**, **#{motor number}**, **&**

Program commands **ADDRESS**, **COMMAND**

&

Function: ..... Report currently addressed coordinate system.
Scope: ..... Global
Syntax: ..... **&**

Remarks

This command causes PMAC to return the number of the coordinate system currently addressed by the host. Notice that a different coordinate system may be hardware selected from the control panel port for coordinate-system-specific control panel inputs, and that different coordinate systems may be addressed from programs within PMAC for **COMMAND** statements.

Example

```
&     ; Ask PMAC which C.S. is addressed

4     ; PMAC reports that C.S. 4 is addressed
```

See Also

I-variable I2
On-line commands **#, #{motor number},&{C.S. number}**;
Program commands **ADDRESS, COMMAND**;

/

Function: ..... Halt program execution at end of currently executing move
Scope: ..... Coordinate-system specific
Syntax: ..... **/**

Remarks

This command causes PMAC to halt the execution of the motion program running in the currently addressed coordinate system at the end of the currently executing move, provided PMAC is in segmentation mode (I13>0). If PMAC is not in segmentation mode (I13=0), the **/** command has the same effect as the **Q** command, halting execution at the end of the latest *calculated* move, which can be 1 or 2 moves past the currently *executing* move.
Once halted at the end of the move, program execution can be resumed with the **R** command. In the meantime, the individual motors may be jogged way from this point, but they must all be returned to this point using the **J=** command before program execution may be resumed. An attempt to resume program execution from a different point will result in an error (ERR017 reported if I6 = 1 or 3). If resumption of this program from this point is not desired, the **A** (abort) command should be issued before other programs are run.

Example

```
&1B5R     ; Command C.S. 1 to start PROG 5

/     ; Halt execution of program

#1J+ ; Jog Motor 1 positive
```

```
J/    ; Stop jogging

J=    ; Return to prejog position

R     ; Resume execution of PROG 5

/     ; Halt program execution

#2J-  ; Jog Motor 2 negative

J/    ; Stop jogging

R     ; Try to resume execution of PROG 5

<BELL>ERR017   ; PMAC reports error; not at position to resume

J=    ; Return to prejog position

R     ; Resume execution of PROG 5
```

See Also

I-variables I6, I13
On-line commands R, J=, Q, A, \, H

?

Function: ..... Report motor status
Scope: ..... Motor specific
Syntax: ..... **?**

Remarks

This command causes PMAC to report the motor status bits as an ASCII hexadecimal word. PMAC returns twelve characters, representing two status words. Each character represents four status bits. The first character represents Bits 20-23 of the first word; the second shows Bits 16-19; and so on, to the sixth character representing Bits 0-3. The seventh character represents Bits 20-23 of the second word; the twelfth character represents Bits 0-3. The value of a bit is 1 when the condition is true; 0 when it is false. The meaning of the individual bits is:

FIRST WORD RETURNED (X:$003D, X:$0079, etc.):

First character returned:

Bit 23 - Motor Activated: This bit is 1 when Ix00 is 1 and the motor calculations are active; it is 0 when Ix00 is 0 and motor calculations are deactivated.
Bit 22 - Negative End Limit Set: This bit is 1 when motor actual position is less than the software negative position limit (Ix14), or when the hardware limit on this end (+LIMn -- note!) has been tripped; it is 0 otherwise. If the motor is deactivated (bit 23 of the first motor status word set to zero) or killed (bit 14 of the second motor status word set to zero) this bit is not updated.
Bit 21 - Positive End Limit Set: This bit is 1 when motor actual position is greater than the software positive position limit (Ix13), or when the hardware limit on this end (-LIMn -- note!) has been tripped; it is 0 otherwise. If the motor is deactivated (bit 23 of the first motor status word set to zero) or killed (bit 14 of the second motor status word set to zero) this bit is not updated.

Bit 20 - Handwheel Enabled: This bit is 1 when Ix06 is 1 and position following for this axis is enabled; it is 0 when Ix06 is 0 and position following is disabled.

Second character returned:

Bit 19 - Phased Motor: This bit is 1 when Ix01 is 1 and this motor is being commutated by PMAC; it is 0 when Ix01 is 0 and this motor is not being commutated by PMAC.
Bit 18 - Open Loop Mode: This bit is 1 when the servo loop for the motor is open, either with outputs enabled or disabled (killed). (Refer to Amplifier Enabled status bit to distinguish between the two cases.) It is 0 when the servo loop is closed (under position control, always with outputs enabled).
Bit 17 - Running Definite-Time Move: This bit is 1 when the motor is executing any move with a predefined end-point and end-time. This includes any motion program move dwell or delay, any jog-to-position move, and the portion of a homing search move after the trigger has been found. It is 0 otherwise. It changes from 1 to 0 when execution of the *commanded* move finishes.
Bit 16 - Integration Mode: This bit is 1 when Ix34 is 1 and the servo loop integrator is only active when desired velocity is zero. It is 0 when Ix34 is 0 and the servo loop integrator is always active.

Third character returned:

Bit 15 - Dwell in Progress: This bit is 1 when the motor's coordinate system is executing a **DWELL** instruction. It is 0 otherwise.
Bit 14 - Data Block Error: This bit is 1 when move execution has been aborted because the data for the next move section was not ready in time. This is due to insufficient calculation time. It is 0 otherwise. It changes from 1 to 0 when another move sequence is started. This is related to the *Run Time Error* Coordinate System status bit.
Bit 13 - Desired Velocity Zero: This bit is 1 if the motor is in closed-loop control and the commanded velocity is zero (i.e. it is trying to hold position). It is zero either if the motor is in closed-loop mode with non-zero commanded velocity, or if it is in open-loop mode.
Bit 12 - Abort Deceleration: This bit is 1 if the motor is decelerating due to an Abort command, or due to hitting hardware or software position (overtravel) limits. It is 0 otherwise. It changes from 1 to 0 when the *commanded* deceleration to zero velocity finishes.

Fourth character returned:

Bit 11 - Block Request: This bit is 1 when the motor has just entered a new move section, and is requesting that the upcoming section be calculated. It is 0 otherwise. It is primarily for internal use.
Bit 10 - Home Search in Progress: This bit is set to 1 when the motor is in a move searching for a trigger: a homing search move, a jog-until trigger, or a motion program move-until-trigger. It becomes 1 as soon as the calculations for the move have started, and becomes zero again as soon as the trigger has been found, or if the move is stopped by some other means. This is *not* a good bit to observe to see if the full move is complete, because it will be 0 during the post-trigger portion of the move. Use the Home Complete and Desired Velocity Zero bits instead.
Bits 8-9 - - These bits are used to store a pointer to the next data block for motor calculations. They are primarily for internal use.

Fifth and sixth characters returned:

Bits 0-7 - - These bits are used to store a pointer to the next data block for motor calculations. They are primarily for internal use.

SECOND WORD RETURNED (Y:$0814, Y:$08D4, etc.):

Seventh character returned:

Bit 23 - Assigned to C.S.: This bit is 1 when the motor has been assigned to an axis in any coordinate system through an axis definition statement. It is 0 when the motor is not assigned to an axis in any coordinate system.
Bits 20-22 (C.S. - 1) Number: These three bits together hold a value equal to the (Coordinate System number minus one) to which the motor is assigned. Bit 22 is the MSB, and bit 20 is the LSB. For instance,

if the motor is assigned to an axis in C. S. 6, these bits would hold a value of 5: bit 22 =1, bit 21 = 0, and bit 20 = 1.

Eighth character returned:

Bits 16-19 - (Reserved for future use)

Ninth Character Returned:

Bit 15 - - (Reserved for future use)
Bit 14 - Amplifier Enabled: This bit is 1 when the outputs for this motor's amplifier are enabled, either in open-loop or closed-loop mode (refer to Open-Loop Mode status bit to distinguish between the two cases). It is 0 when the outputs are disabled (killed).
Bits 12-13 - (Reserved for future use)

Tenth Character Returned:

Bit 11 - Stopped on Position Limit: This bit is 1 if this motor has stopped because of either a software or a hardware position (overtravel) limit, *even if the condition that caused the stop has gone away*. It is 0 at all other times, even when into a limit but moving out of it.
Bit 10 - Home Complete: This bit, set to 0 on power-up or reset, becomes 1 when the homing move *successfully* locates the home trigger. At this point in time the motor is usually decelerating to a stop or moving to an offset from the trigger determined by Ix26. If a second homing move is done, this bit is set to 0 at the beginning of the move, and only becomes 1 again if that homing move *successfully* locates the home trigger. Use the *Desired Velocity Zero* bit and/or the *In Position* bit to monitor for the end of motor motion.
Bit 9 - - (Reserved for future use)
Bit 8 - Phasing Search Error: This bit is set to 1 if the phasing search move for a PMAC-commutated motor has failed due to amplifier fault, overtravel limit, or lack of detected motion. It is set to 0 if the phasing search move did not fail by any of these conditions (not an absolute guarantee of a successful phasing search).

Eleventh Character Returned:

Bit 7 - Trigger Move: This bit is set to 1 at the beginning of a jog-until-trigger or motion program move-until-trigger. It is set to 0 at the end of the move if the trigger has been found, but remains at 1 if the move ends with no trigger found. This bit is useful to determine whether the move was successful in finding the trigger.
Bit 6 - Integrated Fatal Following Error: This bit is 1 if this motor has been disabled due to an integrated following error fault, as set by Ix11 and Ix63. The fatal following error bit (bit 2) will also be set in this case. Bit 6 is zero at all other times, becoming 0 again when the motor is re-enabled.
Bit 5 - I^2T Amplifier Fault Error: This bit is 1 if this motor has been disabled by an integrated current fault. The amplifier fault bit (bit 3) will also be set in this case. Bit 5 is 0 at all other times, becoming 0 again when the motor is re-enabled.
Bit 4 - Backlash Direction Flag: This bit is 1 if backlash has been activated in the negative direction. It is 0 otherwise.

Twelfth Character Returned:

Bit 3 - Amplifier Fault Error: This bit is 1 if this motor has been disabled because of an amplifier fault signal, *even if the amplifier fault signal has gone away*, or if this motor has been disabled due to an I^2T integrated current fault (in which case bit 5 is also set). It is 0 at all other times, becoming 0 again when the motor is re-enabled.
Bit 2 - Fatal Following Error: This bit is 1 if this motor has been disabled because it exceeded its fatal following error limit (Ix11) or because it exceeded its integrated following error limit (Ix63; in which case bit 6 is also set). It is 0 at all other times, becoming 0 again when the motor is re-enabled.
Bit 1 - Warning Following Error: This bit is 1 if the following error for the motor exceeds its warning following error limit (Ix12). It stays at 1 if the motor is killed due to fatal following error. It is 0 at all other times, changing from 1 to 0 when the motor's following error reduces to under the limit, or if killed, is re-enabled.
Bit 0 - In Position: This bit is 1 when five conditions are satisfied: the loop is closed, the desired velocity zero bit is 1 (which requires closed-loop control and no commanded move); the program timer is off (not currently executing any move, **DWELL**, or **DELAY**), the magnitude of the following error is smaller than Ix28.and the first four conditions have been satisfied for (I7+1) consecutive scans.

Example

```
#1?  ; Request status of Motor 1
812000804401   ; PMAC responds with 12 hex digits representing 48 bits
     ; The following bits are true (all others are false)
     ; Word 1 Bit 23: Motor Activated
     ;   Bit 16: Integration Mode
     ;   Bit 13: Desired Velocity Zero
     ; Word 2 Bit 23: Assigned to Coordinate System
     ;   (Bits 20-22 all 0 -- assigned to C.S.1)
     ;   Bit 14: Amplifier Enabled
     ;   Bit 10: Home Complete
     ;   Bit 0: In Position
```

See Also

On-line commands **<CTRL-B>**, **??**, **???**

Memory map registers X:$003D, X:$0079, etc. Y:$0814, Y:$08D4, etc.

Suggested M-variable definitions Mx30-Mx45

??

Function: ..... Report the status words of the addressed coordinate system.
Scope: .....
.....Coordinate-system specific
Syntax: ..... **??**

Remarks

This causes PMAC to report status bits of the addressed coordinate system as an ASCII hexadecimal word. PMAC returns twelve characters, representing two status words. Each character represents four status bits. The first character represents bits 20-23 of the first word; the second shows bits 16-19; and so on, to the sixth character representing bits 0-3. The seventh character represents bits 20-23 of the second word; the twelfth character represents its 0-3. The value of a bit is 1 when the condition is true; 0 when it is false. The meanings of the individual bits are:

FIRST WORD RETURNED (X:$0818, X:$08D8, etc.)

First character returned:

Bit 23 - Z-Axis Used in Feedrate Calculations: This bit is 1 if this axis is used in the vector feedrate calculations for F-based moves in the coordinate system; it is 0 if this axis is not used. See the **FRAX** command.
Bit 22 - Z-Axis Incremental Mode: This bit is 1 if this axis is in incremental mode -- moves specified by distance from the last programmed point. It is 0 if this axis is in absolute mode -- moves specified by end position, not distance. See the **INC** and **ABS** commands.
Bit 21 - Y-Axis Used in Feedrate Calculations: (See bit 23 description.)
Bit 20 - Y-Axis Incremental Mode: (See bit 22 description.)

Second character returned:

Bit 19 - X-Axis Used in Feedrate Calculations: (See bit 23 description.)
Bit 18 - X-Axis Incremental Mode: (See bit 22 description.)
Bit 17 - W-Axis Used in Feedrate Calculations: (See bit 23 description.)
Bit 16 - W-Axis Incremental Mode: (See bit 22 description.)

Third character returned:

Bit 15 - V-Axis Used in Feedrate Calculations: (See bit 23 description.)
Bit 14 - V-Axis Incremental Mode: (See bit 22 description.)
Bit 13 - U-Axis Used in Feedrate Calculations: (See bit 23 description.)
Bit 12 - U-Axis Incremental Mode: (See bit 22 description.)

Fourth character returned:

Bit 11 - C-Axis Used in Feedrate Calculations: (See bit 23 description.)
Bit 10 - C-Axis Incremental Mode: (See bit 22 description.)
Bit 9 - B-Axis Used in Feedrate Calculations: (See bit 23 description.)
Bit 8 - B-Axis Incremental Mode: (See bit 22 description.)

Fifth character returned:

Bit 7 - A-Axis Used in Feedrate Calculations: (See bit 23 description.)
Bit 6 - A-Axis Incremental Mode: (See bit 22 description.)
Bit 5 - Radius Vector Incremental Mode: This bit is 1 if circle move radius vectors are specified incrementally (i.e. from the move start point to the arc center). It is 0 if circle move radius vectors are specified absolutely (i.e. from the XYZ origin to the arc center). See the **INC(R)** and **ABS(R)** commands.
Bit 4 - Continuous Motion Request: This bit is 1 if the coordinate system has requested of it a continuous set of moves (e.g. with an **R** command). It is 0 if this is not the case (e.g. not running program, Ix92=1, or running under an **S** command).

Sixth character returned:

Bit 3 - Move-Specified-by-Time Mode: This bit is 1 if programmed moves in this coordinate system are currently specified by time (TM or TA), and the move speed is derived. It is 0 if programmed moves in this coordinate system are currently specified by feedrate (speed; F) and the move time is derived.
Bit 2 - Continuous Motion Mode: This bit is 1 if the coordinate system is in a sequence of moves that it is blending together without stops in between. It is 0 if it is not currently in such a sequence, for whatever reason.
Bit 1 - Single-Step Mode: This bit is 1 if the motion program currently executing in this coordinate system has been told to "step" one move or block of moves, or if it has been given a Q (Quit) command. It is 0 if the motion program is executing a program by a R (run) command, or if it is not executing a motion program at all.
Bit 0 - Running Program: This bit is 1 if the coordinate system is currently executing a motion program. It is 0 if the C.S. is not currently executing a motion program. Notice that it becomes 0 as soon as it has calculated the last move and reached the final **RETURN** statement in the program, even if the motors are still executing the last move or two that have been calculated. Compare to the motor Running Program status bit.

SECOND WORD RETURNED (Y:$0817, Y:$08D7, etc.)

Seventh character returned:

Bit 23 - Program Hold Stop: This bit is 1 when a motion program running in the currently addressed Coordinate System is stopped using the ' \ ' command from a segmented move (LINEAR or CIRCLE mode with I13 > 0).
Bit 22 - Run-Time Error: This bit is 1 when the coordinate system has stopped a motion program due to an error encountered while executing the program (e.g. jump to non-existent label, insufficient calculation time, etc.)
Bit 21 - Circle Radius Error: This bit is 1 when a motion program has been stopped because it was asked to do an arc move whose distance was more than twice the radius (by an amount greater than Ix96).
Bit 20 - Amplifier Fault Error: This bit is 1 when any motor in the coordinate system has been killed due to receiving an amplifier fault signal. It is 0 at other times, changing from 1 to 0 when the offending motor is re-enabled.

Eighth character returned:

Bit 19 - Fatal Following Error: This bit is 1 when any motor in the coordinate system has been killed due to exceeding its fatal following error limit (Ix11). It is 0 at other times. The change from 1 to 0 occurs when the offending motor is re-enabled.
Bit 18 - Warning Following Error: This bit is 1 when any motor in the coordinate system has exceeded its warning following error limit (Ix12). It stays at 1 if a motor has been killed due to fatal following error limit. It is 0 at all other times. The change from 1 to 0 occurs when the offending motor's following error is reduced to under the limit, or if killed on fatal following error as well, when it is re-enabled.
Bit 17 - In Position: This bit is 1 when all motors in the coordinate system are "in position". Five conditions must apply for all of these motors for this to be true:, the loops must be closed, the desired velocity must be zero for all motors, the coordinate system cannot be in any timed move (even zero distance) or DWELL, all motors must have a following error smaller than their respective Ix28 in-position bands, and the above conditions must have been satisfied for (I7+1) consecutive scans.
Bit 16 - Rotary Buffer Request: This bit is 1 when a rotary buffer exists for the coordinate system and enough program lines have been sent to it so that the buffer contains at least I17 lines ahead of what has been calculated. Once this bit has been set to 1 it will not be set to 0 until there are less than I16 program lines ahead of what has been calculated. The 'PR' command may be used to find the current number of program lines ahead of what has been calculated.

Ninth character returned:

Bit 15 - Delayed Calculation Flag: (for internal use)
Bit 14 - End of Block Stop: This bit is 1 when a motion program running in the currently addressed Coordinate System is stopped using the ' / ' command from a segmented move (Linear or Circular mode with I13 > 0).
Bit 13 - Synchronous M-variable One-Shot: (for internal use)
Bit 12 - Dwell Move Buffered: (for internal use)

Tenth character returned:

Bit 11 - Cutter Comp Outside Corner: This bit is 1 when the coordinate system is executing an added outside corner move with cutter compensation on. It is 0 otherwise.
Bit 10 - Cutter Comp Move Stop Request: This bit is 1 when the coordinate system is executing moves with cutter compensation enabled, and has been asked to stop move execution. This is primarily for internal use.
Bit 9 - Cutter Comp Move Buffered: This bit is 1 when the coordinate system is executing moves with cutter compensation enabled, and the next move has been calculated and buffered. This is primarily for internal use.
Bit 8 - Pre-jog Move Flag: This bit is 1 when any motor in the coordinate system is executing a jog move to "pre-jog" position (**J=** command). It is 0 otherwise.

Eleventh character returned:

Bit 7 - Segmented Move in Progress: This bit is 1 when the coordinate system is executing motion program moves in segmentation mode (I13>0). It is 0 otherwise. This is primarily for internal use.
Bit 6 - Segmented Move Acceleration: This bit is 1 when the coordinate system is executing motion program moves in segmentation mode (I13>0) and accelerating from a stop. It is 0 otherwise. This is primarily for internal use.
Bit 5 - Segmented Move Stop Request: This bit is 1 when the coordinate system is executing motion program move in segmentation mode (I13>0) and it is decelerating to a stop. It is 0 otherwise. This is primarily for internal use.
Bit 4 - PVT/SPLINE Move Mode: This bit is 1 if this coordinate system is in either PVT move mode or SPLINE move mode. (If bit 0 of this word is 0, this means PVT mode; if bit 0 is 1, this means SPLINE mode.) This bit is 0 if the coordinate system is in a different move mode (LINEAR, CIRCLE, or RAPID). See the table below.

Twelfth character returned:

Bit 3 - Cutter Compensation Left: This bit is 1 if the coordinate system has cutter compensation on, and the compensation is to the left when looking in the direction of motion. It is 0 if compensation is to the right, or if cutter compensation is off.
Bit 2 - Cutter Compensation On: This bit is 1 if the coordinate system has cutter compensation on. It is 0 if cutter compensation if off.
Bit 1 - CCW Circle\Rapid Mode: When bit 0 is 1 and bit 4 is 0, this bit is set to 0 if the coordinate system is in CIRCLE1 (clockwise arc) move mode and 1 if the coordinate system is in CIRCLE2 (counter-clockwise arc) move mode. If both bits 0 and 4 are 0, this bit is set to 1 if the coordinate system is in RAPID move mode. Otherwise this bit is 0. See the table below.
Bit 0 - CIRCLE/SPLINE Move Mode: This bit is 1 if the coordinate system is in either CIRCLE or SPLINE move mode. (If bit 4 of this word is 0, this means CIRCLE mode; if bit 4 is 1, this means SPLINE mode.) This bit is 0 if the coordinate system is in a different move mode (LINEAR, PVT, or RAPID.). See the table below.
The states of bits 4, 1, and 0 in the different move modes are summarized in the following table:

MODE	BIT 4	BIT 1	BIT 0
LINEAR	0	0	0
RAPID	0	1	0
SPLINE	1	0	1
CIRCLE1	0	0	1
CIRCLE2	0	1	1
PVT	1	1	0

Example

```
??    ; Request coordinate system status words
A8002A020010    ; PMAC responds; the following bits are true:
      ; Word 1 Bit 23: Z-axis used in feedrate calcs
      ;   Bit 21: Y-axis used in feedrate calcs
      ;   Bit 19: X-axis used in feedrate calcs
      ;   Bit 5: Radius vector incremental mode
      ;   Bit 3: Move specified by time
      ;   Bit 1: Single-step mode
      ; Word 2 Bit 17: In-position
      ;   Bit 4: PVT/Spline mode
```

See Also

On-line commands **<CONTROL-C>**, **?**, **???**

Memory-map registers X:$0818, X:$08D8, etc.; Y:$0817, Y:$08D7, etc.

Suggested M-variable definitions Mx80-Mx90

???

Communications can only happen outside of the real-time interrupt, so polling Bit 22 will always return a value of 0. Bit 22 is for internal use.

Function: .....
Scope: .....
Syntax: .....

Remarks

This command causes PMAC to return the global status bits in ASCII hexadecimal form. PMAC returns twelve characters, representing two status words. Each character represents four status bits. The first character represents Bits 20-23 of the first word, the second shows Bits 16-19; and so on, to the sixth character representing Bits 0-3. The seventh character represents Bits 20-23 of the second word; the twelfth character represents Bits 0-3 of the second word.

A bit has a value of 1 when the condition is true; 0 when false. The meaning of the individual status bits is:

FIRST WORD RETURNED (X:$0003)

First character returned:

Bit 23Real-Time Interrupt Active: This bit is 1 if PMAC is currently executing a real-time interrupt task (PLC 0 or motion program move planning). It is 0 if PMAC is executing some other task.

Bit 22Real-Time Interrupt Re-entry: This bit is 1 if a real-time interrupt task has taken long enough so that it was still executing when the next real-time interrupt came (I8+1 servo cycles later). It stays at 1 until

the card is reset, or until this bit is manually changed to 0. If motion program calculations cause this it is not a serious problem. If PLC 0 causes this (no motion programs running) it could be serious.
Bit 21Servo Active: This bit is 1 if PMAC is currently executing servo update operations. It is 0 if PMAC is executing other operations. Notice that communications can only happen outside of the servo update, so polling this bit will always return a value of 0. This bit is for internal use.
Bit 20Servo Error: This bit is 1 if PMAC could not properly complete its servo routines. This is a serious error condition. It is 0 if the servo operations have been completing properly.

Second character returned:

Bit 19 - Data Gathering Function: On: This bit is 1 when the data gathering function is active; it is 0 when the function is not active.
Bit 18 - Data Gather to Start on Servo: This bit is 1 when the data gathering Function: - is set up to start on the next servo cycle. It is 0 otherwise. It changes from 1 to 0 as soon as the gathering function actually starts.
Bit 17 - Data Gather to Start on Trigger: This bit is 1 when the data gathering Function: - is set up to start on the rising edge of Machine Input 2. It is 0 otherwise. It changes from 1 to 0 as soon as the gathering function actually starts.
Bit 16: (Reserved for future use)

Third character returned:

Bit 15: (Reserved for future use)
Bit 14 - Leadscrew Compensation On: This bit is 1 if leadscrew compensation is currently active in PMAC. It is 0 if the compensation is not active
Bit13 - Any Memory Checksum Error: This bit is 1 if a checksum error has been detected for either the PMAC firmware or the user program buffer space. Bit 12 of this word distinguishes between the two cases.
Bit12 - PROM Checksum Error: This bit is 1 if a firmware checkum error has been detected in the PMAC memory. It is 0 if a user program checksum error has been detected, or if no memory checksum error has been detected. Bit 13 distinguishes between these two cases.

Fourth character returned:

Bit 11 - DPRAM Error: This bit is 1 if PMAC has detected an error in DPRAM communications. It is 0 otherwise.
Bit 10 - EAROM Error: This bit is 1 if PMAC detected a checksum error in reading saved data from the EAROM (in which case it replaces this with factory defaults). It is 0 otherwise.
Bits 8-9: (for internal use)

fifth character returned:

Bit 7: (for internal use)
Bit 6 - TWS Variable Parity Error: This bit is 1 if the most recent TWS-format M-variable read or write operation with a device supporting parity had a parity error; it is 0 if the operation with such a device had no parity error. The bit status is indeterminate if the operation was with a device that does not support parity.
Bit 5 - MACRO Auxiliary Communications Error: This bit is 1 if the most recent MACRO auxiliary read or write command has failed. It is set to 0 at the beginning of each MACRO auxiliary read or write command.
Bit 4: (Reserved for future use)

Sixth character returned:

Bits 2-3: (Reserved for future use)
Bit 1 - All Cards Addressed: This bit is set to 1 if all cards on a serial daisychain have been addressed simultaneously with the `@@` command. It is 0 otherwise.
Bit 0 - This Card Addressed: This bit is set to 1 if this card is on a serial daisychain and has been addressed with the `@n` command. It is 0 otherwise.

SECOND WORD RETURNED (Y:$0003)

Seventh character returned:

Bit 23: (For internal use)
Bit 22 - Host Communication Mode: This bit is 1 when PMAC is prepared to send its communications over the "host port" (PC bus or STD bus). It is 0 when PMAC is prepared to send its communications over the VMEbus or the serial port. It changes from 0 to 1 when it receives an alphanumeric command over the host port. It changes from 1 to 0 when it receives a **<CTRL-Z>** over the serial port.
Bits 20-21: (For Internal Use)

Eighth character returned:

Bit 19 - Motion Buffer Open: This bit is 1 if any motion program buffer (PROG or ROT) is open for entry. It is 0 if none of these buffers is open.
Bit 18 - Rotary Buffer Open: This bit is 1 if the rotary motion program buffer(s) (ROT) is (are) open for entry. It is 0 if this is (these are) closed.
Bit 17 - PLC Buffer Open: This bit is 1 if a PLC program buffer is open for entry. It is 0 if none of these buffers is open.
Bit 16 - PLC Command: This bit is 1 if PMAC is processing a command issued from a PLC or motion program through a CMD" " statement. It is 0 otherwise. It is primarily for internal use.

Ninth character returned:

Bit 15 - VME Communication Mode: This bit is 1 when PMAC is prepared to send its communications over the VME bus "mailbox" port. It is 0 when PMAC is prepared to send its communications over the "host port" (PC bus or STD bus) or the serial port. It changes from 0 to 1 when it receives an alphanumeric command over the VME bus mailbox port. It changes from 1 to 0 when it receives a **<CTRL-Z>** over the serial port.
Bits 12-14: (For Internal use)

Tenth character returned:

Bit 11 - Fixed Buffer Full: This bit is 1 when no fixed motion (PROG) or PLC buffers are open, or when one is open but there are less than I18 words available. It is 0 when one of these buffers is open and there are more than I18 words available.
Bits 8-10: (Internal use)

Eleventh and twelfth characters returned:

Bits 0-7: (Reserved for future use)

Example

```
???  ; Ask PMAC for global status words
003000400000   ; PMAC returns the global status words
     ; 1st word bit 13 (Any checksum error) is true;
     ; 1st word bit 12 (PROM checksum error) is true;
     ; 2nd word bit 23 (for internal use) is true;
     ; All other bits are false
```

See Also

On-line commands ?, ??, <CTRL-G>
Memory registers X:$0003, Y:$0003.

@

Function: ..... Report currently addressed card on serial daisychain
Scope: ..... Global
Syntax: ..... **@**

Remarks

This command causes the addressed PMAC on a serial daisychain to report its number to the host. If all cards are addressed, card @0 will return an **@** character.
I1 must be set to 2 or 3 for this command to be accepted. Otherwise, ERR003 is reported.

Example

```
@     Ask PMAC chain which card is addressed
4     PMAC @4 reports that it is addressed
```

See Also

Addressing Commands (Talking to PMAC)
Multiple-Card Applications (Synchronizing PMAC to External Events)
I-variable I1
On-line commands #, #{constant}, &, &{constant}, @{constant}
Jumpers E40-E43 (PMAC-PC, -Lite, -VME)
Switches SW1-1 to SW1-4 (PMAC-STD)

@{card}

Function: ..... Address a card on the serial daisychain.
Scope: ..... Global
Syntax: ..... `@{card}`

Where:
`{card}` is a hexadecimal digit (0 to 9, A to F), representing the number of the card on the serial daisychain to be addressed; or the @ character, denoting that all cards are to be addressed simultaneously.

Remarks

This command makes the PMAC board specified by `{card}` the addressed board on the serial daisychain. (the one on which subsequent commands will act). The addressing is modal, so all further commands will affect this board until a different board is addressed. At power-up/reset, Board @0 is addressed.
I1 must be set to 2 or 3 for this command to be accepted. Otherwise, ERR003 is reported.
To address all cards simultaneously, use the `@@` command. Query commands (those requiring a data response) will be rejected in this mode.
This command should only be used when multiple PMAC cards are connected on a single serial cable. In this case, I-variable I1 should be set to 2 or 3 on all boards. A board's card number is selected by jumpers E40-E43 (PMAC-PC, -Lite, -VME) or switches SW1-1 to SW1-4 (PMAC-STD).

Example

```
I1=2@0    ; This sequence can be used the first time talking to
          multiple cards on a chain to put them in the proper
          configuration.
@0#1J+    ; Jog motor 1 of Card 0.
@5P20     ; Request the value of P20 on card @5
@@R  ; All cards, addressed C.S. run active program
```

See Also

Addressing Commands (Talking to PMAC)
Multiple-Card Applications (Synchronizing PMAC to External Events)
I-variable I2
On-line commands **#**, **&**, **&{constant}**, **@**
Jumpers E40-E43 (PMAC-PC, -Lite, -VME)
Switches SW1-1 to SW1-4 (PMAC-STD)

\

Function: ..... Do a program hold (permitting jogging while in hold mode)
Scope: ..... Coordinate-system specific
Syntax: ..... ****

Remarks

This command causes PMAC to do a program hold of the currently addressed coordinate system in a manner that permits jogging of the motors in the coordinate system while in hold mode, provided PMAC is in a segmented move (**LINEAR** or **CIRCLE** mode with I13>0). If PMAC is in segmentation mode (I13=0, or other move mode), the **** command has the same effect as the **H** command, bringing the motors to a stop in the same way, but not permitting any moves while in feed hold mode.

The rate of deceleration to a stop in feed hold mode, and from a stop on the subsequent **R** command, is controlled by I-variable I52. This is a global I-variable that controls the rate for all coordinate systems.

Once halted in hold mode, program execution can be resumed with the **R** command. In the meantime, the individual motors may be jogged way from this point, but they must all be returned to this point using the **J=** command before program execution may be resumed. An attempt to resume program execution from a different point will result in an error (ERR017 reported if I6 = 1 or 3). If resumption of this program from this point is not desired, the **A** (abort) command should be issued before other programs are run.

Example

```
&1B5R      ; Command C.S. 1 to start PROG 5

\     ; Command feed hold of program

#1J+  ; Jog Motor 1 positive

J/    ; Stop jogging (examine part here)

J=    ; Return to prejog position

R     ; Resume execution of PROG 5

\     ; Halt program execution

#2J-  ; Jog Motor 2 negative

J/    ; Stop jogging

R     ; Try to resume execution of PROG 5

<BELL>ERR017   ; PMAC reports error; not at position to resume

J=    ; Return to prejog position

R     ; Resume execution of PROG 5
```

See Also

Stop Commands (Making Your Application Safe)
I-variables I6, I13, I52, Ix95
On-line commands **R**, **J=**, **Q**, **A**, **/**, **H**

A

Function:..... Abort all programs and moves in the currently addressed coordinate system.
Scope: .....
..... Coordinate-system specific
Syntax: .....A

Remarks

This command causes all axes defined in the current coordinate system to begin immediately to decelerate to a stop, aborting the currently running motion program (if any). It also brings any disabled (killed) or open-loop motors (defined in the current coordinate system) to an enabled zero-velocity closed-loop state.
If moving, each motor will decelerate its commanded profile at a rate defined by its own motor I-variable Ix15. If there is significant following error when the **A** command is issued, it may take a long time for the actual motion to stop. Although the command trajectory is brought to a stop at a definite rate, the actual position will continue to "catch up" to the commanded position for a longer time.
Notice that a multi-axis system may not stay on its programmed path during this deceleration.
Abort commands are not meant to be recovered from gracefully. If you wish to resume easily, us the **H**, **Q**, **/**, or **** command instead.

Motion program execution may resume (if a motion program was in fact aborted) by issuing either an **R** or **S** command, but two factors must be considered. First, the starting positions for calculating the next move will be the original end positions of the aborted move unless the **PMATCH** command is issued or I14=1. Second, the move from the aborted position to the next move end position may not be possible or desirable. The **J=** command may be used to jog each motor in the coordinate system to the original end position of the aborted move, provided I13 is 0 (no segmentation mode).

Example

```
B1R  ; Start Motion Program 1

A    ; Abort the program

#1J=#2J=  ; Jog motors to original move-end position

R    ; Resume program with next move
```

See Also

Stop Commands (Making Your Application Safe)

Control-Panel Port STOP/ Input (Connecting PMAC to the Machine)

I-variables I13, I14, Ix15

On-line commands **<CONTROL-A>**, **H**, **J/**, **K**, **Q**

JPAN connector pin 10

ABS

Function:..... Select absolute position mode for axes in addressed coordinate system.

Scope: .....
..... Coordinate-system specific

Syntax: .....ABS

ABS({axis}[,{axis}...])

Where:

{axis} is a letter (X, Y, Z, A, B, C, U, V, W) representing the axis to be specified, or the character R to specify radial vector mode

Remarks

This command, without any arguments, causes all subsequent positions for all axes in the coordinate system in motion commands to be treated as absolute positions (this is the default condition). An **ABS** command with arguments causes the specified axes to be in absolute mode, and all others to remain unchanged.

If R is specified as one of the 'axes', the I, J, and K terms of the circular move radius vector specification will be specified in absolute form (i.e. as a vector from the origin, not from the move start point). An **ABS** command without any arguments does not affect this vector specification. The default radial vector specification is incremental.

If a motion program buffer is open when this command is sent to PMAC, the command will be entered into the buffer for later execution.

Example

```
ABS(X,Y)  ; X & Y made absolute -- other axes
     ; and radial vector left unchanged
ABS  ; All axes made absolute --
     ; radial vector left unchanged
ABS(R)    ; Radial vector made absolute --
     ; all axes left unchanged
```

See Also

Circular Moves (Writing a Motion Program)

On-line command **INC**

Program commands **ABS**, **INC**

{axis}={constant}

Function: ..... Re-define the specified axis position.

Scope: ..... Coordinate-system specific

Syntax: ..... **{axis}={constant}**

Where:

{axis} is a letter from the set (X, Y, Z, U, V, W, A, B, C) specifying the axis whose present position is to be re-named;

{constant} is a floating-point value representing the new "name" value for the axis' present position

Remarks

This command re-defines the current axis position to be the value specified in **{constant}**, in user units (as defined by the scale factor in the axis definition). It can be used to relocate the origin of the coordinate system. This does not cause the specified axis to move; it simplly assigns a new value to the position..
Internally, a position bias register is written to which creates this new position offset. **PSET** is the equivalent motion program command.

Example

```
X=0  ; Call axis X's current position zero

Z=5000    ; Re-define axis Z's position as 5000
```

See Also

Axes, Coordinate Systems (Setting Up a Coordinate System)

On-line command **Z**

Program commands **PSET**, **ADIS**, **IDIS**.

B{constant}

Function: ..... Point the addressed coordinate system to a motion program.
Scope: ..... Coordinate-system specific
Syntax: ..... **B{constant}**

Where:
{constant} is a floating point value from 0.0 to 32767.99999 representing the program and location to point the coordinate system to; with the integer part representing the program number, and the fractional part multiplied by 100,000 representing the line label (zero fractional part means the top of the program).

Remarks

This command causes PMAC to set the program counter of the addressed coordinate system to the specified motion program and location. It is usually used to set the program counter to the *B*eginning of a motion program. The next R or S command will start execution at this point.
If **{constant}** is an integer, the program counter will point to the beginning of the program whose number matches **{constant}**. Fixed motion program buffers (PROG) can have numbers from 1 to 32,767. The rotary motion program carries program number 0 for the purpose of this command.
If **{constant}** is not an integer, the fractional part of the number represents the line label (**N** or **O**) in the program to which to point. The fractional value multiplied by 100,000 determines the number of the line label to which to point (it fills the fraction to 5 decimal places with zeros).
If a motion program buffer (including ROTARY) is open when this command is sent to PMAC, the command is entered into the buffer for later execution, to be interpreted as a B-axis move command.

Example

```
B7   ;points to the top of PROG 7

B0   ;points to the top of the rotary buffer

B12.6     ;points to label N60000 of PROG 12

B3.025R   ;points to label N2500 of PROG 3 and runs
```

See Also

On-line commands **DEFINE ROT**,**R**, **S**

Program commands **B{data}**, **N{constant}**, **O{constant}**.

CLEAR

Function: ..... Erase currently opened buffer.
Scope: .....
..... Global
Syntax: ..... **CLEAR**
CLR

Remarks

This command empties the currently opened program, PLC, rotary, etc. buffer Typically, as you create a buffer file in your host computer, you will start with the **OPEN {buffer}** and **CLEAR** commands (even though these lines are technically not part of the buffer), and follow with your actual contents. This will allow you to easily edit buffers from your host and repeatedly download the buffers, erasing the old buffer's contents in the process.

Example

```
OPEN PROG 1    ; Open motion program buffer 1
CLEAR     ; Clear out this buffer
F1000     ; Program really starts here!
X2500     ;...and ends on this line!
CLOSE     ; This closes the program buffer
OPEN PLC 3 CLEAR CLOSE   ; This erases PLC 3
```

See Also

Program Buffers (Talking to PMAC)

On-line commands **OPEN**, **CLOSE**, **DELETE**.

CLOSE

Function: ..... Close the currently opened buffer.
Scope: .....
..... Global
Syntax: ..... **CLOSE**
..... **CLS**

Remarks

This closes the currently **OPEN**ed buffer. This should be used immediate after the entry of a motion, PLC, rotary, etc. buffer. If the buffer is left open, subsequent statements that are intended as on-line commands (e.g. **P1=0**) will get entered into the buffer instead. It is good practice to have **CLOSE** at the beginning and end of any file to be downloaded to PMAC.

When PMAC receives a **CLOSE** command, it automatically appends a **RETURN** statement to the end of the open program buffer.

If *any* PROGRAM or PLC in PMAC is improperly structured (e.g. no **ENDIF** or **ENDWHILE** to match an **IF** or **WHILE**), PMAC will report an ERR003 at the **CLOSE** command for any buffer until the problem is fixed.

Example

```
CLOSE     ; This makes sure all buffers are closed
OPEN PROG 1    ; Open motion program buffer 1
```

```
CLEAR      ; Clear out this buffer
F1000      ; Program actually starts here!...
X2500      ;...and ends on this line!
CLOSE      ; This closes the program buffer
LIST PROG 1     ; Request listing of closed program
F1000      ; PMAC starts listing
X2500
RETURN     ; This was appended by the CLOSE command
```

See Also

Program Buffers (Talking to PMAC)

On-line commands **OPEN**, **CLEAR**, **<CTRL-L>**, **<CTRL-U>**

{constant}



If a special table on PMAC (e.g. STIMULUS, COMP) has been defined but not filled, a constant value will be entered into this table, not into P0.

Function: .....Assign valı

Scope: .....

Syntax: .....

Where:

{constant} is a floating point value

Remarks

This command is the equivalent of **P0={constant}**. That is, a value entered by itself on a command line will be assigned to P-variable P0. This allows simple operator entry of numeric values through a dumb terminal interface. Where the value goes is hidden from the operator; the PMAC user program must take P0 and use it as appropriate.

Example

In a motion program:

```
P0=-1     ; Set P0 to an "illegal" value
SEND"Enter number of parts in run:"
     ; Prompt operator at dumb terminal
     ; Operator simply needs to type in number
WHILE (P0<1) WAIT   ; Hold until get legal response
P1=0 ; Initialize part counter
WHILE (P0<P1)  ; Loop once per part
       P1=P1+1
       ...
```

See Also

On-line commands **OPEN COMP, OPEN STIMULUS, P{constant}={expression}**

DATE

Function:.....Report PROM firmware revision date.

Scope: .....
.....Global

Syntax: .....**DATE**

.....
.....**DAT**

Remarks

This command causes PMAC to report the revision date of the PROM firmware revision it is using. The date is reported in the American style: mm/dd/yy (month/day/year).

Example

```
DATE Ask PMAC for firmware revision date

07/22/92  PMAC responds with July 22, 1992
```

See Also

On-line command **VERSION**, **TYPE**

DEFINE BLCOMP

Function:.....Define backlash compensation table

Scope: ..... Motor specific

Syntax: .....`DEFINE BLCOMP {entries},{count length}`

.....
.....`DEF BLCOMP {entries},{count length}`

Where:

`{entries}` is a positive integer constant representing the number of values in the table;

`{count length}` is a positive integer representing the span of the table in encoder counts of the motor.

Remarks

This command establishes a backlash compensation table for the addressed motor. The next **`{entries}`** constants sent to PMAC will be placed into this table. The last item on the command line **`{count length},`** specifies the span of the backlash table in encoder counts of the motor. In use, if the motor position goes outside of the range 0 to count-length, the position is "rolled over" to within this range before the compensation is computed. The spacing between entries in the table is **`{count length}`** divided by **`{entries}.`**

On succeeding lines will be given the actual entries of the table as constants separated by spaces and or carriage return characters. The units of these entries are 1/16 count, and the entries must be integer values. The first entry is the correction at one spacing from the motor zero position (as determined by the most recent home search move or power-up/reset), the second entry is the correction two spacings away, and so on. The correction from the table at motor zero position is zero by definition.

The correction is the amount *subtracted* (added in the negative direction) from the nominal commanded position when the motor is moving in the negative direction to get the corrected position. The correction from the backlash table is added to the Ix86 "constant" backlash parameter before adjusting the motor position. Correction from any leadscrew compensation tables that have this motor as the target motor are always active in both directions.

The last entry in the table represents the correction at **`{count length}`** distance from the motor's zero position. Since the table has the capability to roll over, this entry also represents the correction at the motor's zero position. For this reason, the last entry should virtually always be set to zero.

PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any BLCOMP buffer exists for a lower numbered motor, or if any TBUF, ROTARY, or GATHER buffer exists. Any of these buffers must be DELETEd first. BLCOMP buffers must be DEFINEd from high-numbered motor to low-numbered motor, and DELETEd from low-numbered motor to high-numbered motor.

I51 must be set to 1 to enable the table.

See Also

Backlash Compensation (Setting Up a Motor)

Leadscrew Compensation (Setting Up a Motor)

I-variables I99, Ix85, Ix86

On-line commands **`DEFINE COMP`**, **`DELETE BLCOMP`**

DEFINE COMP (one-dimensional)

Function: ..... Define Leadscrew Compensation Table

Scope: ..... Motor specific

Syntax: `DEFINE COMP {entries},[#{source},[#{target},]]{count length}`

Where:

`{entries}` is a positive integer constant representing the number of numbers in the table;

{source} (optional) is a constant from 1 to 8 representing the motor whose position controls which entries in the table are used for the active correction; if none is specified, PMAC assumes the source is the addressed motor;
{target} (optional) is a constant from 1 to 8 representing the motor that receives the correction; if none is specified, PMAC assumes the target is the addressed motor;
{count length} is a positive integer representing the span of the table in encoder counts of the source motor

Remarks

This command establishes a leadscrew compensation table assigned to the addressed motor. The next **{entries}** constants sent to PMAC will be placed into this table. Once defined the tables are enabled and disabled with the variable I51.
The table "belongs" to the currently addressed motor, and unless otherwise specified in the command line, it will use the addressed motor both for source position data and as the target for its corrections. Each motor can only have one table that "belongs" to it (for a total of 8 tables in one PMAC), but it can act as a source or a target for multiple motors.
PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any COMP buffer exists for a lower numbered motor, or if any TCOMP, BLCOMP, TBUF, ROTARY, or GATHER buffer exists. Any of these buffers must be DELETEd first. COMP buffers must be DEFINEd from high-numbered motor to low-numbered motor, and deleted from low-numbered motor to high-numbered motor.

It is possible to directly specify a source motor (with **#{source}**), or source and target motors (with **#{source},#{target}**), in this command. Either or both of them may be different from the motor to which the table "belongs". (In other words, just because a table belongs to a motor does not necessarily mean that it affects or is affected by that motor's position.)
The last item on the command line, **{count length}**, specifies the span of the compensation table in encoder counts of the source motor. In use, if the source motor position goes outside of the range 0 to count-length, the source position is "rolled over" to within this range before the correction is computed. The spacing between entries in the table is **{count length}** divided by **{entries}.**
On succeeding lines will be given the actual entries of the table. The units of these entries are 1/16 count, and the entries must be integer values. The first entry is the correction at one spacing from the motor zero position (as determined by the most recent home search move or power-up/reset), the second entry is the correction two spacings away, and so on. The correction is the amount *added* to the nominal position to get the corrected position. The correction at the zero position is zero by definition.
The last entry in the table represents the correction at **{count length}** distance from the source motor's zero position. Since the table has the capability to roll over, this entry also represents the correction at the source motor's zero position. For this reason, the last entry should virtually always be set to zero.

Example

```
#1 DEFINE COMP 4,2000    ; Create table for motor 1

ERR003    ; PMAC rejects this command

DELETE GATHER  ; Clear other buffers to allow loading

DELETE ROTARY

#8DEFINE COMP 500,20000  ; Uses motor 8 as source and target,

#7DEFINE COMP 256,#3,32768    ; 500 entries, spacing of 40 counts

     ; Belongs to motor 7, uses motor 3

     ; as source, motor 7 as target,

#6 DEFINE COMP 400,#5,#4,40000; 256 entries, spacing of 128
counts

     ; "Belongs" to motor 6,

     ; uses #5 as source, #4 as target,

#5 DEFINE COMP 200,#1,#1,30000; 400 entries, spacing of 100
counts

     ; "Belongs" to motor 5,

     ; uses #1 as source and target,

I51=1     ; 200 entries, spacing of 150 count

     ; Enable compensation tables
```

See Also

Leadscrew compensation (Setting Up a Motor)

I-variable I51

On-line commands **{constant}**, **LIST COMP**, **LIST COMP DEF**, **DELETE COMP**, **DELETE GATHER**, **DELETE ROTARY**, **SIZE**

DEFINE COMP (two-dimensional)

Function: ..... Define two-dimensional leadscrew compensation table.
Scope: ..... Motor specific
Syntax: ..... **DEFINE COMP {entries1}.{entries2}, #{source1}, [#{source2}, [#{target}]],{length1},{length2}**
DEF COMP...

Where:

{entries1} is a positive integer constant representing the number of specified locations of the first source motor;

{entries2} is a positive integer constant representing the number of specified locations of the second source motor;

{source1} is an integer constant from 1 to 8 representing the number of the first source motor;

{source2} is an integer constant from 1 to 8 representing the number of the second source motor; defaults to addressed motor;

{target} is an integer constant from 1 to 8 representing the number of the target motor; defaults to addressed motor;

{length1} is the length of the table, in counts, along the first source motor's travel;

{length2} is the length of the table, in counts, along the second source motor's travel

Remarks

This command command establishes a two-dimensional compensation table assigned to the addressed motor. The next *(entries1+1)*(entries2+1)-1* constants sent to PMAC will be placed into this table. This type of table is usually used to correct a third motor position (e.g. Z-axis) as a function of the planar position of two other motors (e.g. X and Y axes). Once defined the tables are enabled and disabled with the variable I51.

The table "belongs" to the currently addressed motor, and unless otherwise specified in the command line, it will use the addressed motor both as the second-motor source position data and as the target for its corrections. Each motor can only have one table that belongs to it (for a total of 8 tables in one PMAC), but it can act as a source and a target for multiple tables.

PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any COMP buffer exists for a lower numbered motor, or if any TCOMP, BLCOMP, TBUF, ROTARY, or GATHER buffer exists. Any of these buffers must be DELETEd first. COMP buffers must be DEFINEd from high-numbered motor to low-numbered motor, and DELETEd from low-numbered motor to high-numbered motor.

The first source motor must be specified in the command line with **#{source1}**. The second source motor may be specified in the command line with **#{source2}**; if it is not specified, PMAC assumes that the second source motor is the currently addressed motor. The target motor may be specified with **#{target}**; if it is not specified, PMAC assumes that the target motor is the currently addressed motor.

In other words, if only one motor is specified in the command line, it is the first source motor, and the second source and target motors default to the addressed motor. If two motors are specified in the command line, the first one specified is the first source motor, the second is the second source motor, and the target motor defaults to the addressed motor. If three motors are specified, the first is the first source motor, the second is the second source motor, and the third is the target motor. None of these motors is required to be the addressed motor.

The last two items on the command line, **{length1}** and **{length2}**, specify the span of the compensation table for the first and second source motors, respectively, expressed in encoder counts of those motors. In use, if the source motor position goes outside of the range 0 to **{length}**, the source position is "rolled over" to within this range before the correction is computed. The spacing between entries in the table for each source motor is **{length}** divided by **{entries}**.

On succeeding lines will be given the actual entries of the table. The units of these entries are 1/16 count, and the entries must be integer values. The first entry is the correction at one spacing from the zero position of the first source motor, and at the zero position of the second source motor (as determined by the most recent home search

move or power-up/reset), the second entry is the correction at two spacings of the first source motor and at zero of the second motor, and so on, to the **{entries1}** entry, which is at **{length1}** of the first source motor and zero of the second motor (this entry should be zero if you wish to use the table along the edge, to match the implied zero correction at the origin).
The next entry is at the zero position of the first source motor and one spacing from the zero position of the second source motor. The following entries index along the position of the first source motor, to entry number 2***{entries1}**+1, which is at **{length1}** of the first source motor and one spacing from zero of the second source motor. This sequence is repeated row by row of the table, finishing at entry number (**{entries1}**+1)*(**{entries2}**+1)-1, which is the correction at (**{length1}**,**{length2}**) -- this entry should be zero if you wish to use the table along the edge, to match the implied zero correction at the origin.

Example

```
#1 DEFINE COMP 30.40,#1,#2,#3,300000,400000
     ; Create table belonging to Motor 1
ERR007    ; PMAC rejects this command
DELETE GATHER  ; Clear other buffers to allow loading
DELETE TRACE
&1 DELETE ROTARY
&2 DELETE ROTARY
#2 DELETE COMP
#3 DELETE COMP
#4 DEFINE COMP 30.40,#1,#2,#3,300000,400000
     ; Create same table, now belonging to
     ; Motor 4 -- #1 & #2 are sources, #3 is
     ; target; 30x40, spacing of 10,000 counts
(1270 entries) ; (30+1)*(40+1)-1 entries of constants
#3 DEFINE COMP 20.25,#2,#3,#1,200000,250000
     ; Create table belonging to Motor 3 -- #2
     ; and #3 are sources, #1 is target; 20x25,
     ; spacing of 10,000 counts
(545 entries)  ;(20+1)*(25+1)-1 entries of constants
#2 DEFINE COMP 10.10,#1,#4,10000,20000
     ; Create table belonging to Motor 2 -- #1
     ; and #4 are sources, #2 (default) is
     ; target; 10x10, spacing of 1000 & 2000
     ; counts
(120 entries)  ; (10+1)*(10+1)-1 entries of constants
#1 DEFINE COMP 10.12,#4,1280,1200
     ; Create table belonging to Motor 1 -- #4
     ; and #1 (default) are sources, #1
```

```
     ; (default) is target; 10x12; spacing of
     ; 128 & 100 counts
(142 entries)  ; (10+1)*(12+1)-1 entries of constants
I51=1     ; Enable compensation tables
```

See Also

Leadscrew compensation (Setting Up a Motor)

I-variable I51

On-line commands **{constant}**, **LIST COMP**, **LIST COMP DEF**, **DELETE COMP**, **DELETE GATHER**, **DELETE ROTARY**, **DELETE TRACE**, **SIZE**

DEFINE GATHER

Function: .....

Scope: .....

Syntax: .....**DEF**

DEF GAT [{constant}]

Where:

{constant} is a positive integer representing the number of words of memory to be reserved for the buffer

If I45 = 2 or 3 the ***{size}*** *requested in the* ***DEFINE GATHER {size}*** *command refers to a DPRAM long word (32-bits). If the* ***{size}*** *is smaller than required to hold an even multiple of the requested data, the actual data storage will go beyond the requested* ***{size}*** *to the next higher multiple of memory words required to hold the data. For Example, if you are gathering one 24-bit value and one 48-bit value you will need 3 DPRAM long words of memory. If the* ***{size}*** *you specify is 4000 words (not a multiple of 3), the actual storage size will be 4002 words (the next higher multiple of 3).*

Remarks

This command reserves space in the PMAC memory or in DPRAM depending upon the setting of I45 for the data gathering buffer and prepares it for collecting data at the beginning of the buffer. If a data gathering buffer already exists, its contents are not erased but the Data Gather Buffer Storage address (Y:$0F20) is reinitialized to the Data Gather Buffer Start address (X:$0F20) and the **LIST GATHER** command will no longer Function. Data collection will again start at the beginning of the buffer when the command **GATHER** is issued. If Data Gathering is in progress (an **ENDGATHER** command has not been issued and the gather buffer has not been filled up) PMAC will report an error on receipt of this command.
The optional **{constant}** specifies the number of long words to be reserved for the data gathering buffer, leaving the remainder of the PMAC memory available for other buffers such as motion and PLC programs. If **{constant}** is not specified, all of the PMAC unused buffer memory is reserved for data gathering. Until this buffer is deleted (with the **DELETE GATHER** command), no new motion or PLC programs may be entered into PMAC.
The number of long words of unused buffer memory can be found by issuing the **SIZE** command.

Example

```
DEFINE GATHER

DEFINE GATHER 1000
```

See Also

I-variables I19-I45

On-line commands **GATHER**, **DELETE GATHER**, **<CTRL-G>**, **SIZE**

DEFINE ROTARY

Function: ..... Define a rotary motion program buffer

Scope: ..... Coordinate-system specific

Syntax: `DEFINE ROTARY{constant}`

`DEF ROT{constant}`

Where:

{constant} is a positive integer representing the number of long words of memory to be reserved for the buffer

Remarks

This command causes PMAC to create a rotary motion program buffer for the addressed coordinate system, allocating the specified number of long words of memory. Rotary buffers permit the downloading of program lines during the execution of the program.
The buffer should be large enough to allow it to hold safely the number of lines you anticipate downloading to PMAC ahead of the executing point. Each long word of memory can hold one sub-block of a motion program (i.e. **X1000 Y1000** is considered as two sub-blocks). The allocated memory for this coordinate system's rotary buffer remains resident until the buffer is deleted with **DELETE ROT**.
PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any ROTARY buffer exists for a lower numbered coordinate system, or if a GATHER buffer exists. Any of these buffers must be DELETEd first. ROTARY

buffers must be DEFINEd from high-numbered coordinate system to low-numbered coordinate system, and DELETEd from low-numbered coordinate system to high-numbered coordinate system.

Example

```
DELETE GATHER  ; Ensure open memory

&2DEFINE ROT 100    ; Create buffer for C.S. 2

&1DEFINE ROT 100    ; Create buffer for C.S. 1

&1B0 &2B0 ; Point to these buffers

OPEN ROT  ; Open these buffers for entry

   ...    ; enter program lines here
```

See Also

Rotary Program Buffers (Writing a Motion Program)

On-line commands **OPEN ROTARY**, **DELETE ROTARY**, **DELETE GATHER**

DEFINE TBUF

Function: Create a buffer for axis transformation matrices.

Scope: Global

Syntax: `DEFINE TBUF {constant}`

`DEF TBUF {constant}`

Where:

{constant} is a positive integer representing the number of transformation matrices to create

Remarks

This command reserves space in the PMAC memory for one or more axis transformation matrices. These matrices can be used for real-time translation, rotation, scaling, and mirroring of the X, Y, and Z axes of any coordinate system. A coordinate system selects which matrix to use with the **TSELn** command, where n is an integer from 1 to the number of matrices created here.

PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any ROTARYor GATHER buffer exists. Any of these buffers must be DELETEd first.

The number of long words of unused buffer memory can be found by issuing the **SIZE** command. Each defined matrix takes 21 words of memory.

Example

```
DELETE GATHER

DEF TBUF 1

DEFINE TBUF 8
```

See Also

Axis Transformation Matrices (Writing a Motion Program)

On-line commands **DELETE TBUF**, **DELETE GATHER**, **SIZE**.

Program commands **TSEL**, **ADIS**, **AROT**, **IDIS**, **IROT**, **TINIT**

DEFINE TCOMP

Function: Define torque compensation table
Scope: Motor specific
Syntax: **DEFINE TCOMP {entries},{count length}**
DEF TCOMP {entries},{count length}

Where:
{entries} is a positive integer constant representing the number of values in the table;
{count length} is a positive integer representing the span of the table in encoder counts of the motor.

Remarks

This command establishes a torque compensation table for the addressed motor. The next **{entries}** constants sent to PMAC will be placed into this table. The last item on the command line **{count length},** specifies the span of the torque compensation table in encoder counts of the motor. In use, if the motor position goes outside of the range 0 to count-length, the position is "rolled over" to within this range before the compensation is computed. The spacing between entries in the table is **{count length}** divided by **{entries}.**
On succeeding lines will be given the actual entries of the table as constants separated by spaces and or carriage return characters. The units of these entries are bits of a 16-bit DAC (even if the actual output device has some other resolution), and the entries must be integer values. The first entry is the correction at one spacing from the motor zero position (as determined by the most recent home search move or power-up/reset), the second entry is the correction two spacings away, and so on. PMAC computes corrections for positions between the table entries by a first-order interpolation between adjacent entries The correction from the table at motor zero position is zero by definition.
The correction is the magnitude added to the PMAC servo loop output at that position. If the PMAC command is positive, a positive value from the table will increase the magnitude of the output; a negative value will decrease the magnitude of the output. If the PMAC command is negative, a positive value from the table will increase the magnitude of the output in the negative direction; a negative value will decrease the magnitude of the output.
The last entry in the table represents the correction at **{count length}** distance from the motor's zero position. Since the table has the capability to roll over, this entry also represents the correction at the motor's zero position. For this reason, the last entry should virtually always be set to zero.
PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any TCOMP buffer exists for a lower numbered motor, or if any BLCOMP, TBUF, ROTARY, or GATHER buffer exists. Any of these buffers must be DELETEd first. TCOMP buffers must be DEFINEd from high-numbered motor to low-numbered motor, and DELETEd from low-numbered motor to high-numbered motor.
I51 must be set to 1 to enable the table.

See Also

Torque Compensation (Setting Up a Motor)

I-variables I51

On-line command DELETE TCOMP

DEFINE UBUFFER

Function: Create a buffer for user variable use.

Scope: ..... Global

Syntax: ..... **DEFINE UBUFFER {constant}**

DEF UBUF {constant}

Where:

{constant} is a positive integer representing the number of 48-bit words of PMAC memory to reserve for this purpose

Remarks

This command reserves space in the PMAC memory for the user's discretionary use. This memory space will be untouched by any PMAC automatic Functions. User access to this buffer is through M-variables, or possibly through on-line W (write) and R (read) commands.

The buffer starts at PMAC memory address $9FFF and continues back toward the beginning of memory ($0000) for the number of long (48-bit) words specified by {constant}. This memory space can be subdivided any way the user sees fit. On PMACs with battery backup, the values in the buffer at power-down will still be there at power-up. On PMACs with flash backup, the values in the buffer at the last SAVE command will be copied from the flash memory into the buffer at power-up or reset.

All other buffers except for fixed motion programs (PROG) and PLC programs must be deleted before PMAC will accept this command. There can be no rotary motion program, leadscrew compensation table, transformation matrix, data gathering or stimulus buffers in PMAC memory (any buffer created with a **DEFINE** command) for this command to be accepted. It is usually best to reinitialize the card with a **$$$***** command before sending the **DEFINE UBUFFER** command

The address of the end of unreserved memory is held in register X:$0F3F. This register must hold the address $A000, signifying no defined buffers, in order for PMAC to be able to create a user buffer. Immediately after the user buffer has successfully been defined, this register will hold the address of the start of the buffer (the end of the user buffer is always at $9FFF). However, after other buffers have been defined, the end of unreserved memory will not match the beginning of the user buffer.

To free up this memory for other uses, the **DEFINE UBUFFER 0** command should be used (there is no **DELETE UBUFFER** command). It may be more convenient simply to re-initialize the board with a **$$$*** command.

Example

```
RHX:$0F3F ; Look for end of unreserved memory

008A3D    ; Reply indicates some reserved

$$$***    ; Re-initialize card to clear memory

RHX:$0F3F ; Check end of unreserved memory

00A000    ; Reply indicates none reserved

DEFINE UBUFFER 256  ; Reserve memory for buffer

RHX:$0F3F ; Check for beginning of buffer

009F00    ; Reply confirms 256 words reserved

M1000->D:$9F00 ; Define M-var to first word

M1010->Y:$9F80,12,1 ; Define M-var to a middle word

M1023->X:$9FFF,24,S ; Define M-var to last word
```

See Also

User Buffer, M-Variables (Computational Features)

On-line commands **$$$***, **R[H]{address}**, **W{address}**

DELETE BLCOMP

Function: ..... Erase backlash compensation table
Scope: ..... Motor specific
Syntax: ..... **DEFINE BLCOMP**
. . . . . .
. . . . . **DEF BLCOMP**

Remarks

This command causes PMAC to erase the compensation table for the addressed motor, freeing that memory for other use.

PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any BLCOMP buffer exists for a lower numbered motor, or if any TBUF, ROTARY, or GATHER buffer exists. Any of these buffers must be DELETEd first. BLCOMP buffers must be DEFINEd from high-numbered motor to low-numbered motor, and DELETEd from low-numbered motor to high-numbered motor.

Example

```
#2 DEL BLCOMP  ; Erase table belonging to Motor 2

ERR003    ; PMAC rejects this command

#1 DEL BLCOMP  ; Erase table belonging to Motor 1

#2 DEL BLCOMP  ; Erase table belonging to Motor 2
```

See Also

Backlash Compensation (Setting Up a Motor)

I-variables I99, Ix85, Ix86

On-line command **DEFINE BLCOMP**

DELETE COMP

Function: ..... Erase leadscrew compensation table
Scope: ..... Motor specific
Syntax: ..... **DELETE COMP**
. . . . . .
. . . . . **DEL COMP**

Remarks

This command causes PMAC to erase the compensation table belonging to the addressed motor, freeing that memory for other use.

PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any COMP buffer exists for a lower numbered motor, or if any TCOMP, BLCOMP, TBUF, ROTARY, or GATHER buffer exists. Any of these buffers must be DELETEd first. COMP buffers must be DEFINEd from high-numbered motor to low-numbered motor, and DELETEd from low-numbered motor to high-numbered motor.

Remember that a compensation table *belonging* to a motor does not necessarily affect that motor or is not necessarily affected by that motor. The command **LIST COMP DEF** will tell which motors it affects and is affected by.

Example

```
#2DEL COMP; Erase table belonging to Motor 2
ERR003    ; PMAC rejects this command
#1 DELETE COMP ; Erase table belonging to Motor 1
#2 DELETE COMP ; Erase table belonging to Motor 2
```

See Also

Leadscrew compensation (Setting Up a Motor)
I-variable I51
On-line commands **{constant}**, **LIST COMP**, **LIST COMP DEF**, **DEFINE COMP**

DELETE GATHER

Function: ..... Erase the data gather buffer.
Scope: ..... Global
Syntax: ..... **DELETE GATHER**
..... **DEL GAT**

Remarks

This command causes the data gathering buffer to be erased. The memory that was reserved is now deallocated and is available for other buffers (motion programs, PLC programs, compensation tables, etc.). If Data Gathering is in progress (an **ENDGATHER** command has not been issued and the gather buffer has not been filled up) PMAC will report an error on receipt of this command.
the PMAC Executive Program automatically inserts this command at the top of a file when it uploads a buffer from PMAC into its editor, so the next download will not be hampered by an existing gather buffer. It is strongly recommended that you use this command as well when you create a program file in the editor (see Examples, below).
When the executive program's data gathering Function operates, it automatically reserves the entire open buffer space for gathered data. When this has happened, no additional programs or program lines may be entered into the PMAC buffer space until the **DELETE GATHER** command has freed this memory.

Example

```
CLOSE     ; Make sure no buffers are open

DELETE GATHER  ; Free memory

OPEN PROG 50   ; Open new buffer for entry

CLEAR     ; Erase contents of buffer

...  ; Enter new contents here
```

See Also

Buffered Commands (Talking to PMAC)
On-line commands **GATHER**, **DEFINE GATHER**, **SIZE**

DELETE PLCC

Function: ..... Erase specified compiled PLC program

Scope: ..... Global

Syntax: ..... **DELETE PLCC {constant}**

..... **DEL PLCC {constant}**

Where:

{constant} is an integer from 0 to 31, representing the program number

Remarks

This command causes PMAC to erase the specified compiled PLC program. Remember that because all of the compiled PLC programs must be downloaded to PMAC together, the only way to restore this PLC is to download the entire set of compiled PLCs. Also notice that
Only one PLCC program can be deleted in one command. Ranges and lists of PLCC program numbers are not permitted in this command.
To perform the same Function for an uncompiled PLC program. the command sequence would be OPEN PLC n CLEAR CLOSE.

Example

```
DELETE PLCC 5  ; Erase compiled PLC program 5

DEL PLCC 0; Erase compiled PLC program 0
```

See Also

Compiled PLCs (Writing a PLC Program)

I-variable I5

On-line commands **DISABLE PLCC**, **ENABLE PLCC**, **CLEAR**

DELETE ROTARY

Function: Delete rotary motion program buffer of addressed coordinate system

Scope: Coordinate-system specific

Syntax: **DELETE ROTARY**

DEL ROT

Remarks

This command causes PMAC to erase the rotary buffer for the currently addressed coordinate system and frees the memory that had been allocated for it.
PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if the ROTARY buffer for this coordinate system is open or executing, or if any ROTARY buffer exists for a lower numbered coordinate system, or if a GATHER buffer exists. Any of these buffers must be DELETEd first. ROTARY buffers must be DEFINEd from high-numbered coordinate system to low-numbered coordinate system, and DELETEDd from low-numbered motor to high-numbered motor.

Example

```
&2 DELETE ROTARY    ; Try to erase C.S. 2's rotary buffer

ERR003    ; PMAC rejects this; C.S. 1 still has a rotary

&1 DELETE ROTARY    ; Erase C.S. 1's rotary buffer

&2 DELETE ROTARY    ; Erase C.S. 2's rotary buffer; OK now
```

See Also

Rotary Program Buffers (Writing a Motion Program)

On-line commands **DEFINE ROTARY**, **OPEN ROTARY**.

DELETE TBUF

Function: Delete buffer for axis transformation matrices.
Scope: Global
Syntax: **DELETE TBUF**

DEL TBUF

Remarks

This command frees up the space in the PMAC memory that was used for axis transformation matrices. These matrices can be used for real-time translation, rotation, scaling, and mirroring of the X, Y, and Z axes of any coordinate system.
PMAC will reject this command, reporting an ERR007 if I6=1 or 3, if any ROTARYor GATHER buffer exists. Any of these buffers must be DELETEd first.

Example

```
DEL TBUF

DELETE TBUF
```

See Also

Axis Transformation Matrices (Writing a Motion Program)

On-line commands **DEFINE TBUF**

Program commands **TSEL, ADIS, AROT, IDIS, IROT, TINIT**

DELETE TCOMP

Function: ..... Erase torque compensation table
Scope: ..... Motor specific
Syntax: ..... **DEFINE TCOMP**
..... **DEF TCOMP**

Remarks

This command causes PMAC to erase the torque compensation table for the addressed motor, freeing that memory for other use.
PMAC will reject this command, reporting an ERR003 if I6=1 or 3, if any TCOMP buffer exists for a lower numbered motor, or if any BLCOMP, TBUF, ROTARY, or GATHER buffer exists. Any of these buffers must be DELETEd first. TCOMP buffers must be DEFINEd from high-numbered motor to low-numbered motor, and DELETEd from low-numbered motor to high-numbered motor.

Example

```
#2 DEL TCOMP    ; Erase table belonging to Motor 2
ERR003     ; PMAC rejects this command
#1 DEL TCOMP    ; Erase table belonging to Motor 1
#2 DEL TCOMP    ; Erase table belonging to Motor 2
```

See Also

Torque Compensation (Setting Up a Motor)

I-variables I51

On-line command **DEFINE TCOMP**

DELETE TRACE



The TRACE buffer feature on PMAC has been removed. DELETE TRACE is still a valid command and will not cause an error when sent to PMAC, but it causes no operation to be performed.

Function: ..... Formerly: Eras
Scope: .....
Syntax: `DELETE TRACE`
`DEL TRAC`

Example

```
CLOSE     ; Make sure no buffers are open
DELETE GATHER DELETE TRACE    ; Free memory
OPEN PLC 17    ; Open new buffer for entry
CLEAR     ; Erase contents of buffer
...  ; Enter new contents here
```

See Also

On-line commands **DELETE GATHER**.

DISABLE PLC

Function: ..... Disable specified PLC program(s).
Scope: .....
..... Global
Syntax: ..... **DISABLE PLC {constant}[,{constant}]**

.....
..... DIS PLC {constant}[,{constant}]
. . . . . .
. . . . . **DISABLE PLC {constant}..{constant}**
. . . . . .
. . . . . **DIS PLC {constant}..{constant}**
.....
..... Where:
. . . . . .
. . . . . **{constant}** is an integer from 0 to 31, representing the program number

Remarks

This command causes PMAC to disable (stop executing) the specified PLC program or programs. PLC programs are specified by number, and may be specified in a command singularly, in a list (separated by commas), or in a range of consecutively numbered programs. PLC programs can be re-enabled by using the **ENABLE PLC** command.
If a motion or PLC program buffer is open when this command is sent to PMAC, the command will be entered into that buffer for later execution.

Example

```
DISABLE PLC 1

DIS PLC 5

DIS PLC 3,4,7

DISABLE PLC 0..31
```

See Also

I-variable I5

On-line commands **ENABLE PLC**, **OPEN PLC**, **DISABLE PLCC**, **ENABLE PLCC**, **<CONTROL-D>**.

Program commands **DISABLE PLC**, **ENABLE PLC**, **DISABLE PLCC**, **ENABLE PLCC**

DISABLE PLCC

Function: ..... Disable compiled PLC program(s).

Scope: ..... Global

Syntax:

```
DISABLE PLCC {constant}[,{constant}]

DIS PLCC {constant}[,{constant}]

DISABLE PLCC {constant}..{constant}

DIS PLCC {constant}..{constant}
```

Where:

{constant} is an integer from 0 to 31, representing the compiled PLC program number

Remarks

This command causes PMAC to disable (stop executing) the specified compiled PLC program or programs. Compiled PLC programs are specified by number, and may be specified in a command singularly, in a list (separated by commas), or in a range of consecutively numbered programs. PLC programs can be re-enabled by using the **ENABLE PLCC** command.
If a motion or PLC program buffer is open when this command is sent to PMAC, the command will be entered into that buffer for later execution.

Example

```
DISABLE PLCC 1
```

```
DIS PLCC 5
DIS PLCC 3,4,7
DISABLE PLCC 0..31
```

See Also

I-variable I5

On-line commands **DISABLE PLC**, **ENABLE PLC**, **ENABLE PLCC**, **OPEN PLC**, **<CONTROL-D>**.

Program commands **DISABLE PLC**, **DISABLE PLCC**, **ENABLE PLC**, **ENABLE PLCC**

ENABLE PLC

Function: ..... Enable specified PLC program(s).
Scope: ..... Global

Syntax:
```
ENABLE PLC {constant}[,{constant}]
ENA PLC {constant}[,{constant}]
ENABLE PLC {constant}..{constant}
ENA PLC {constant}..{constant}
```

Where:
{constant} is an integer from 0 to 31, representing the program number

Remarks

This command causes PMAC to enable (start executing) the specified PLC program or progams. PLC programs are specified by number, and may be used singularly in this command, in a list (separated by commas), or in a range of consecutively numbered programs.
If a motion or PLC program buffer is open when this command is sent to PMAC, the command will be entered into that buffer for later execution.
I-variable I5 must be in the proper state to allow the PLC program(s) specified in this command to execute.
This command *must* be used to start operation of a PLC program after it has been entered or edited, because the **OPEN PLC** command automatically disables the program, and **CLOSE** does not re-enable it.

Example

```
ENABLE PLC 1
ENA PLC 2,7
ENABLE PLC 3,21
ENABLE PLC 0..31
```

This Example shows the sequence of commands to download a very simple PLC program and have it enabled automatically on the download:

```
OPEN PLC 7 CLEAR
P1=P1+1
CLOSE
ENABLE PLC 7
```

See Also

I-variable I5

On-line commands ENABLE PLC, OPEN PLC, <CONTROL-D>.

ENABLE PLCC

Function: ..... Enable specified compiled PLC program(s).
Scope: ..... Global

Syntax: `ENABLE PLCC {constant}[,{constant}]`

`ENA PLCC {constant}[,{constant}]`

`ENABLE PLCC {constant}..{constant}`

`ENA PLCC {constant}..{constant}`

..... Where:

`{constant}` is an integer from 0 to 31, representing the program number

Remarks

This command causes PMAC to enable (start executing) the specified compiled PLC program or progams. Compiled PLC programs are specified by number, and may be used singularly in this command, in a list (separated by commas), or in a range of consecutively numbered programs.
If a motion or PLC program buffer is open when this command is sent to PMAC, the command will be entered into that buffer for later execution.
I-variable I5 must be in the proper state to allow the compiled PLC program(s) specified in this command to execute.

Example

```
ENABLE PLCC 1
ENA PLCC 2,7
ENABLE PLCC 3,21
ENABLE PLCC 0..31
```

See Also

I-variable I5

On-line commands **DISABLE PLC, DISABLE PLCC, ENABLE PLC, OPEN PLC, <CONTROL-D>.**

Program commands **DISABLE PLC, DISABLE PLCC, ENABLE PLC, ENABLE PLCC**

ENDGATHER

Function: ..... Stop data gathering.
Scope: ..... Global
Syntax: ..... **ENDGATHER**
ENDG

Remarks

This command causes data gathering to cease. Data gathering may start up again (without overwriting old data) with another **GATHER** command.
This command is usually used in conjunction with the data gathering and plotting Functions of the PMAC Executive Program.
If a motion or PLC program buffer is open when this command is sent to PMAC, the command will be entered into that buffer for later execution.

Examples

```
GAT B1R   ; Start gathering and run program 1
ENDG ; Stop gathering -- give this command when moves of interest
     ;are done
OPEN PROG2 CLEAR
X10
DWELL1000
CMD"GATHER"    ; Program issues start command here
X20  ; Move of interest
DWELL50
CMD"ENDG" ; Program issues stop command here
CLOSE
```

See Also

Data Gathering Function (Analysis Features)

I-variables I19-I44

On-line commands **DEFINE GATHER**, **GATHER**, **LIST GATHER**, **DELETE GATHER**

Gathering and Plotting (PMAC Executive Program Manual)

F

Function: ..... Report motor following error

Scope: ..... Motor specific

Syntax: ..... **F**

Remarks

This command causes PMAC to report the present motor following error (in counts, rounded to the nearest tenth of a count) to the host. Following error is the difference between motor desired and measured position at any instant. When the motor is open-loop (killed or enabled), following error does not exist and PMAC reports a value of 0.

Example

```
F     ; Ask for following error of addressed motor

12    ; PMAC responds

#3F   ; Ask for following error of Motor 3

-6.7  ; PMAC responds
```

See Also

Following Error (Servo Features)

I-variables Ix11, Ix12, Ix67

On-line commands **<CTRL-F>**, **P**, **V**

Suggested M-variable definitions Mx61, Mx62

Memory map registers D:$0028, D:$002C, etc.; D:$0840, etc.

FRAX

Function: ..... Specify the coordinate system's feedrate axes.

Scope: ..... Coordinate-system specific

Syntax: ..... **FRAX**

```
FRAX({axis}[,{axis}...])
```

Where:

{axis} (optional) is a character (X, Y, Z, A, B, C, U, V, W) specifying which axis is to be used in the vector feedrate calculations

Remarks

This command specifies which axes are to be involved in the vector-feedrate (velocity) calculations for upcoming feedrate-specified (**F**) moves. PMAC calculates the time for these moves as the vector distance (square root of the sum of the squares of the axis distances) of all the feedrate axes divided by the feedrate. Any non-feedrate axes commanded on the same line will complete in the same amount of time, moving at whatever speed is necessary to cover the distance in that time.

Vector feedrate has obvious geometrical meaning only in a Cartesian system, for which it results in constant tool speed regardless of direction, but it is possible to specify for non-Cartesian systems, and for more than three axes.

If only non-feedrate axes are commanded to move in a feedrate-specified move, PMAC will compute the vector distance, and therefore the move time, as zero, and will attempt to do the move in the acceleration time (TA or 2*TS), possibly limited by the maximum velocity and/or acceleration parameters for the motor(s). This will probably be much faster than intended.

If a motion program buffer is open when this command is sent to PMAC, it will be entered into the buffer for later execution.
For instance, in a Cartesian XYZ system, if you use **FRAX(X,Y)**, all of your feedrate-specified moves will be at the specified vector feedrate in the XY-plane, but not necessarily in XYZ-space. If you use **FRAX(X,Y,Z)** or **FRAX**, your feedrate-specified moves will be at the specified vector feedrate in XYZ-space. Default feedrate axes for a coordinate system are X, Y, and Z.

Example

```
FRAX ; Make all axes feedrate axes

FRAX(X,Y) ; Make X and Y axes only the feedrate axes

FRAX(X,Y,Z)     ; Make X, Y, and Z axes only the feedrate axes
```

See Also

Feedrate-Specified Moves (Writing a Motion Program)

Program commands **F{data}**, **FRAX**.

GATHER

Function: ..... Begin data gathering.
Scope: ..... Global
Syntax: ..... **GATHER [TRIGGER]**
GAT [TRIG]

Remarks

This command causes data gathering to commence according to the configuration defined in I-variables I19-I45. If **TRIGGER** is not used in the command, gathering will start on the next servo cycle. If **TRIGGER** is used, gathering will start on the first servo cycle after machine input MI2 goes true.
Gathering will proceed at the frequency set by I19 (in number of servo interrupt cycles). If I19 is 0, only one set of data will be gathered per **GATHER** command. If PMAC is already gathering data, **GATHER** will cause resynchronization of the gathering cycle to the next servo cycle.
Gathering will continue until PMAC receives an **ENDGATHER** command, or until the buffer created by the **DEFINE GATHER** command is full.

This command is usually used in conjunction with the data gathering and plotting **Function:** s of the PMAC Executive Program.
If a motion or PLC program buffer is open when this command is sent to PMAC, the command will be entered into that buffer for later execution.

Example

```
GAT B1R   ; Start gathering and run program 1

ENDG ; Stop gathering -- give this command when moves of interest are
done

OPEN PROG2 CLEAR

X10

DWELL1000

CMD"GATHER"     ; Program issues start command here
```

```
X20  ; Move of interest

DWELL50

CMD"ENDG" ; Program issues stop command here

CLOSE
```

See Also

Data Gathering Function (Analysis Features)
I-variables I19-I45
On-line commands **DEFINE GATHER**, **GATHER**, **LIST GATHER**, **DELETE GATHER**
Gathering and Plotting (PMAC Executive Program Manual)

H

Function: ..... Perform a feedhold

Scope: .....
..... Coordinate-system specific

Syntax: ..... **H**

Remarks

This causes the currently addressed coordinate system to suspend execution of the program starting immediately by bringing its time base value to zero, decelerating along its path at a rate defined by the coordinate system I-variable Ix95. Technically the program is still executing after an **H** command, but at zero speed. This means that the motors defined in the coordinate system cannot be moved while performing the feed hold.
To do a hold of the currently addressed coordinate system in a manner that permits jogging of the motors in the coordinate system while in feed hold mode, refer to the **** "program hold" command.
The **H** command is very similar in effect to a **%0** command, except that deceleration is controlled by Ix95, not Ix94, and execution can be resumed with an **R** or an **S** command, instead of a **%100** command. In addition, **H** works under external time base, whereas a **%0** command does not.
Full speed execution along the path will commence again on an **R** or **S** command. The ramp up to full speed will also take place at a rate determined by Ix95 (full time-base value, either internally or externally set). Once the full speed is reached, Ix94 determines any time-base changes.

See Also

Stopping Commands (Making Your Application Safe)
Control-Panel Port HOLD/ Input (Connecting PMAC to the Machine)
Time-Base Control (Synchronizing PMAC to External Events)
I-variables I52, Ix93, Ix94, Ix95
On-line commands **<CTRL-O>**, **%**, **%{constant}**, **A**, **K**, ****, **Q**
JPAN Connector Pin 12

HOME

Function: ..... Start Homing Search Move

Scope: .....
..... Motor specific

Syntax: ..... **HOME**

HM

Remarks

This command causes the addressed motor to perform a homing search routine. The characteristcs of the homing search move are controlled by motor I-variables Ix03 and Ix19-Ix26, plus encoder I-variables 2 and 3 for that motor's position encoder.

The on-line home command simply starts the homing search routine. PMAC provides no automatic indication that the search has completed (although the In-Position interrupt could be used for this purpose) or whether the move completed successfully. Polling, or a combination of polling and interrupts, is generally used to determine completion and success.

By contrast, when a homing search move is given in a motion program (e.g. **HOME1,2**), the motion program will keep track of completion by itself as part of its sequencing algorithms.

If there is an axis offset in the axis-definition statement for the motor, and/or following error in the motor servo loop, the reported position at the end of the homing search move will be equal to the axis offset minus the following error, not to zero.

Example

```
HOME ; Start homing search on the addressed motor
#1HM ; Start homing search on Motor 1
#3HM#4HM  ; Start homing search on Motors 3 and 4
```

See Also

Control Panel Port HOME/ Input (Connecting PMAC to the Machine)
Homing Moves (Basic Motor Moves)
I-variables Ix03, Ix19-Ix26, Encoder I-Variables 2 & 3
On-line command HOMEZ
Program command HOME{constant}, HOMEZ{constant}
JPAN Connector Pin 11

HOMEZ

Function: ..... Do a Zero-Move Homing
Scope: .....
..... Motor specific
Syntax: ..... **HOMEZ**

HMZ

Remarks

This command causes the addressed motor to perform a zero-move homing search. Instead of jogging until it finds a pre-defined trigger, and calling its position at the trigger the home position, with this command, the motor calls wherever it is (*commanded* position) at the time of the command the home position.

If there is an axis offset in the axis-definition statement for the motor, and/or following error in the motor servo loop, the reported position at the end of the homing operation will be equal to the axis offset minus the following error, not to zero.

Example

```
     ; On-line command Examples
HOMEZ      ; Do zero-move homing search on the addressed motor
#1HMZ      ; Do zero-move homing search on Motor 1
#3HMZ#4HMZ; Do zero-move  homing search on Motors 3 and 4
     ; Buffered motion program Examples
HOMEZ1
HOMEZ2,3
     ; On-line commands issued from PLC program
IF (P1=1) ;
  CMD"#5HOMEZ" ; Program issues on-line command
  P1=0     ; So command is not repeatedly issued
ENDIF
```

See Also

Homing Moves (Basic Motor Moves)

On-line command **HOME**

Program command **HOME{constant}**, **HOMEZ{constant}**

I{constant}

If a motion program buffer (including a rotary buffer) is open, ***I{constant}*** *will be entered into that buffer for later execution, to be interpreted as a full-circle move command with a vector to the center along the X-axis (see Circular Moves in the Writing a Motion Program section).*

Function: ..... Rep
Scope: .....
Syntax: ..... **I{**

Where:
{constant} is an integer from 0 to 1023 representing the number of the I-variable;
the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

Remarks

This command causes PMAC to report the current value of the specified I-variable or range of I-variables.
When I9 is 0 or 2, only the value of the I-variable itself is returned (e.g. *I10000*). When I9 is 1 or 3, the entire variable value assignment statement (e.g. *I130=10000*) is returned by PMAC.
When I9 is 0 or 1, the values of "address" I-variables are reported in decimal form. When I9 is 2 or 3, the values of these variables are reported in hexadecimal form.

Example

```
I5    ; Request the value of I5
2     ; PMAC responds
I130..135 ; Request the value of I130 through I135
60000     ; PMAC responds with 6 lines
5000
5000
50000
1
20000
To see the effect of I9 on the form of the response, observe the
following:
I9=0  I125
49152     ; Short form, decimal
I9=1  I125
I125=49152; Long form, decimal
I9=2  I125
$C000     ; Short form, hexadecimal
I9=3  I125
I125=$C000; Long form, hexadecimal
```

See Also

Initialization (I) Variables (Computational Features)
I-Variable Specifications
I-variable I9
On-line commands **I{constant}={expression}**, **M{constant}**, **P{constant}**, **Q{constant}**
Program commands **{axis}{data}{vector}{data}**, **I{data}**

I{constant}={expression}

Function: ..... Assign a value to an I-variable.
Scope: ..... Global
Syntax: ..... **I{constant}[..{constant}]={expression}**

Where:
{constant} is an integer from 0 to 1023 representing the number of the I-variable;
the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;
{expression} contains the value to be given to the specified I-variable(s)

Remarks

This command assigns the value on the right side of the equals sign to the specified I-variable or range of I-variables.
If a motion or PLC program buffer is open when this command is sent to PMAC, the command will be entered into the buffer for later execution.

Example

```
I5=2

I130=1.25*I130

I22..44=0

I102=$C003

I104=I103
```

See Also

Initialization (I) Variables (Computational Features)

I-Variable Specifications

On-line commands I{constant}, M{constant}={expression}, P{constant}={expression}, Q{constant}={expression}

I{constant}=*

Function: ..... Assign factory default value to an I-variable.
Scope: ..... Global
Syntax: ..... **I{constant}[..{constant}]=***

Where:

{constant} is an integer from 0 to 1023 representing the number of the I-variable;

the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

Remarks

This command sets the specified I-variable or range of I-variables to the factor default value. Each I-variable has its own factory default; these are shown in the I-Variable Specification section.

Example

```
I13=*
I100..199=*
```

See Also

Initialization (I) Variables (Computational Features)

I-Variable Specifications -- Default Values

On-line commands I{constant}, I{constant}={expression}

INC

Function: ..... Specify Incremental Move Mode
Scope: ..... Coordinate-system specific
Syntax: ..... **INC**

INC({axis}[,{axis}...])

Where:

{axis} is a letter (X, Y, Z, A, B, C, U, V, W) representing the axis to be specified, or the character R to specify radial vector mode

Remarks

The **INC** command without arguments causes all subsequent positions for all axes in position motion commands to be treated as incremental distances. An **INC** statement with arguments causes the specified axes to be in incremental mode, and all others stay the way they were before. The default axis specification is absolute.

If 'R' is specified as one of the 'axes', the I, J, and K terms of the circular move radius vector specification will be specified in incremental form (i.e. as a vector from the move start point, not from the origin). An **INC** command without any arguments does not affect this vector specification. The default vector specification is incremental. If a motion program buffer is open when this command is sent to PMAC, it will be entered into the buffer as a program statement.

Example

```
INC(A,B,C); A, B, & C axes made incremental --
     ; other axes and rad vector left as is
INC  ; All axes made incremental --
     ; radius vector left as is
INC(R)    ; Radius vector made incremental --
     ; all axes left as is
```

See Also

Circular Moves (Writing a Motion Program)
On-line command **ABS**
Program commands **ABS**, **INC**

J!

Function: ..... Adjust motor commanded position to nearest integer count

Scope: ..... Motor specific

Syntax: ..... **J!**

Remarks

This command causes the addressed motor, if the desired velocity is zero, to adjust its commanded position to the nearest integer count value. It can be valuable to stop dithering if the motor is stopped with its commanded position at a fractional value, and integral gain is causing oscillation about the commanded position.

Example

```
OPEN PLC 7 CLEAR
IF (M50=1); Condition to start branch
     CMD"#1J/" ; Tell motor to stop
     WHILE (M133=0) ; Wait for desired velocity to reach zero
     ENDWHILE
     CMD"#1J!" ; Adjust command position to integer value
```

```
    M50=0; To keep from repeated execution

ENDIF
```

See Also

On-line commands **J/**, **J={constant}**

J+

Function:.....Jog Positive
Scope: .....
..... Motor specific
Syntax: .....**J+**

Remarks

This command causes the addressed motor to jog in the positive direction indefinitely. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command.
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
J+    ; Jog addressed motor positive

#7J+ ; Jog Motor 7 positive

#2J+#3J+   ; Jog Motors 2 and 3 positive
```

See Also

Control Panel Port JOG+/ Input (Connecting PMAC to the Machine)

JPAN Connector Pin 6

Jogging Moves (Basic Motor Moves)

I-variables Ix19-Ix22

On-line commands **J-**, **J/**, **J=**, **J={constant}**, **J:{constant}**, **J^{constant}**

J-

Function: ..... Jog Negative
Scope: ..... Motor specific
Syntax: ..... **J-**

Remarks

This command causes the addressed motor to jog in the negative direction indefinitely. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command.
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
J-    ; Jog addressed motor negative

#5J-  ; Jog Motor 5 negative

#3J-#4J-   ; Jog Motors 3 and 4 negative
```

See Also

Control Panel Port JOG-/ Input (Connecting PMAC to the Machine)
JPAN Connector Pin 4
Jogging Moves (Basic Motor Moves)
I-variables Ix19-Ix22
On-line commands J+, J/, J=, J={constant}, J:{constant}, J^{constant}

J/

Function: ..... Jog Stop
Scope: ..... Motor specific
Syntax: ..... **J/**

Remarks

This command causes the addressed motor to stop jogging. It also restores position control if the motor's servo loop has been opened (enabled or killed), with the new commanded position set equal to the actual position at the time of the **J/** command. Jogging deceleration is determined by the values of Ix19-Ix21 in force at the time of this command.
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
#1J+ ; Jog Motor 1 positive
J/   ; Stop jogging Motor 1
```

```
O5    ; Open-loop output of 5% on Motor 1
O0    ; Open loop output of 0%
J/    ; Restore closed-loop control
K     ; Kill output
J/    ; Restore closed-loop control
```

See Also

Control Panel Port JOG+/, JOG-/ Inputs (Connecting PMAC to the Machine)
JPAN Connector Pin 4, 6
Jogging Moves (Basic Motor Moves)
I-variables Ix19-Ix22
On-line commands **<CTRL-A>**,**A**,**J+**,**J-**,**J=**,**J={constant}**,**J:{constant}**,**J^{constant}**,**K**,**O{constant}**

J:{constant}

Function: ..... Jog Relative to Commanded Position
Scope: ..... Motor specific
Syntax: ..... **J:{constant}**

Where:
{constant} is a floating point value specifying the distance to jog, in counts.

Remarks

This command causes a motor to jog the *distance* specified by **{constant}** relative to the present *commanded* position. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command. Compare to **J^{constant}**, which is a jog relative to the present *actual* position.
A variable incremental jog command can be executed with the **J:*** command
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
#1HM ; Do homing search move on Motor 1
J:2000    ; Jog a distance of 2000 counts (to 2000 counts)
J:2000    ; Jog a distance of 2000 counts (to 4000 counts)
```

See Also

Jogging Moves (Basic Motor Moves)
I-variables Ix19-Ix22
On-line commands **J+**, **J-**, **J/**, **J=**, **J={constant}**, **J^{constant}**

J:*

Function: ..... Jog to specified variable distance from present commanded position
Scope: ..... Motor specific

Syntax: ..... **J:***

Remarks

This command causes the addressed motor to jog the *distance* specified in the motor's "variable jog position/distance" register relative to the present *commanded* position. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command. Compare to **J^*** , which is a jog relative to the present *actual* position.

The "variable jog position/distance" register is a floating-point register with units of counts. It is best accessed with a floating-point M-variable. The register is located at PMAC address L:$082B for motor 1, L:$08EB for motor 2, etc. The usual procedure is to write the destination position to this register by assigning a value to the M-variable, then issuing the **J:*** command.

PMAC will reject this command if the motor is in a coordinate sytem that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
M172->L:$082B  ; Define #1 variable jog position/distance reg.

#1HMZ      ; Declare present position to be zero

M172=3000 ; Assign distance value to register

#1J:*      ; Jog Motor 1 this distance; end cmd. pos. will be 3000

#1J:*      ; Jog Motor 1 this distance; end cmd. pos. will be 6000

M172=P1*SIN(P2); Assign new distance value to register

#1J:*      ; Jog Motor 1 this distance

#1J= ; Return to prejog target position
```

See Also

Jogging Moves (Basic Motor Moves)

I-variables Ix19-Ix22

Memory map registers L:$082B, L:$08EB, etc.

Suggested M-variable definitions M172, M272, etc.

On-line commands **J=**, **J={constant}**, **J=***, **J^***

J=

Function: ..... Jog to Prejog Position

Scope: ..... Motor specific

Syntax: ..... **J=**

Remarks

This command causes the addressed motor to jog to the last pre-jog and pre-handwheel-move position (the most recent programmed position). Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command.

The register containing this position information for the motor is called the *target position register* (D:$080B for Motor 1, D:$08CB for Motor 2, etc.). Suggested M-variable definitions M163, M263, etc. can be used in programs to give access to these registers.

If the **/** or **** stop command has been used to suspend program execution, and one or more motors jogged away from the stop position, the **J=** command must be used to return the motor(s) back to the stop position before program execution can be resumed.
The **J=** command can also be useful if a program has been aborted in the middle of a move, because it will move the motor to the programmed move end position (provided I13=0 so PMAC is not in segmentation mode), so the program may be resumed properly from that point.
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
&1Q  ; Stop motion program at end of move

#1J+ ; Jog Motor 1 away from this position

J/   ; Stop jogging

J=   ; Jog back to position where program quit

R    ; Resume motion program


&1A  ; Stop motion program in middle of move

#1J=#2J=#3J=   ; Move all motors to original move end position

R    ; Resume motion program
```

See Also

Control Panel Port PREJ/ Input (Connecting PMAC to the Machine)
JPAN Connector Pin 7
Jogging Moves (Basic Motor Moves)
I-variables Ix19-Ix22
On-line commands **J+**, **J-**, **J/**, **J={constant}**, **J:{constant}**, **J^{constant}**

J={constant}

Function: ..... Jog to specified position

Scope: .....
..... Motor specific

Syntax: ..... **J={constant}**

Where:

{constant} is a floating point value specifying the location to which to jog, in encoder counts.

Remarks

This command causes the addressed motor to jog to the position specified by **{constant}**. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command.
A variable jog-to-position can be executed with the **J=*** command
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
J=0  ; Jog addressed motor to position 0

#4J=5000  ; Jog Motor 4 to 5000 counts

#8J=-32000; Jog Motor 8 to -32000 counts
```

See Also

Jogging Moves (Basic Motor Moves)
I-variables Ix19-Ix22
On-line commands **J+**, **J-**, **J/**, **J=**, **J:{constant}**, **J^{constant}**, **J=***, **J:***, **J^***

J=*

Function: ..... Jog to specified variable position

Scope: .....
..... Motor specific

Syntax: ..... **J=***

Remarks

This command causes the addressed motor to jog to the position specified in the motor's "variable jog position/distance" register. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command.

The "variable jog position/distance" register is a floating-point register with units of counts. It is best accessed with a floating-point M-variable. The register is located at PMAC address L:$082B for motor 1, L:$08EB for motor 2, etc. The usual procedure is to write the destination position to this register by assigning a value to the M-variable, then issuing the **J=*** command.

Virtually the same result can be obtained by writing to the motor "target position" register and issuing the **J=** command. However, using the **J=*** command permits you to return to the real target position afterwards without having to restore the target position register. Also, the **J=*** command uses a register whose value is scaled in counts, not fractions of a count.

PMAC will reject this command if the motor is in a coordinate sytem that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
M172->L:$082B  ; Define #1 variable jog position/distance reg.

M172=3000 ; Assign position value to register

#1J=*     ; Jog Motor 1 to this position

M172=P1*SIN(P2); Assign new position value to register

#1J=*     ; Jog Motor 1 to this position

#1J= ; Return to prejog target position
```

See Also

Jogging Moves (Basic Motor Moves)
I-variables Ix19-Ix22
Memory map registers L:$082B, L:$08EB, etc.
Suggested M-variable definitions M172, M272, etc.
On-line commands J=, J={constant}, J:*, J^*

J=={constant}

Function:.....Jog to specified motor position and make that position the “pre-jog” position

Scope: ..... Motor specific

Syntax: .....**J=={constant}**

Where:

{constant} is a floating point value specifying the location to which to jog, in encoder counts

Remarks

This command causes the addressed motor to jog the position specified by **{constant}**. It also makes this position the "pre-jog" position, so it will be the destination of subsequent **J=** commands. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command.
PMAC will reject this command if the motor is in a coordinate sytem that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
#1J==10000; Jog Motor 1 to 10000 counts and make that the pre-jog
     ; position.
J+   ; Jog indefinitely in the positive direction
J=   ; Return to 10000 counts
```

See Also

Jogging Moves (Basic Motor Moves)

I-variables Ix19-Ix22

On-line commands **J=**, **J={constant}**, **J=***, **J^***

J^{constant}

Function: ..... Jog Relative to Actual Position
Scope: ..... Motor specific
Syntax: ..... **J^{constant}**

Where:
{constant} is a floating point value specifying the distance to jog, in counts.

Remarks

This causes a motor to jog the *distance* specified by **{constant}** relative to the present *actual* position. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command. Compare to **J:{constant}**, which is a jog relative to the present *commanded* position.
Usually the **J:{constant}** command is more useful, because its destination is not dependent on the following error at the instant of the command. The **J^0** command can be useful for "swallowing" any existing following error.
A variable incremental jog can be executed with the **J^*** command
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
#1HM ; Do homing search move on Motor 1
J^2000    ; Jog a distance of 2000 counts from actual position
     ; If actual was -5 cts, new command pos is 1995 cts
J^2000    ; Jog a distance of 2000 counts from actual position
     ; If actual was 1992 cts, new cmd pos is 3992 cts
```

See Also

Jogging Moves (Basic Motor Moves)

I-variables Ix19-Ix22

On-line commands **J+**, **J-**, **J/**, **J=**, **J={constant}**, **J:{constant}**, **J=***, **J:***, **J^***

J^*

Function:.....Jog to specified variable distance from present actual position
Scope: ..... Motor specific
Syntax: .....**J^***

Remarks

This command causes the addressed motor to jog the *distance* specified in the motor's "variable jog position/distance" register relative to the present *actual* position. Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command. Compare to **J:*** , which is a jog relative to the present *commanded* position.
The "variable jog position/distance" register is a floating-point register with units of counts. It is best accessed with a floating-point M-variable. The register is located at PMAC address L:$082B for motor 1, L:$08EB for motor 2, etc. The usual procedure is to write the destination position to this register by assigning a value to the M-variable, then issuing the **J^*** command.
PMAC will reject this command if the motor is in a coordinate sytem that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
M172->L:$082B  ; Define #1 variable jog position/distance reg.

#1HMZ      ; Declare present position to be zero

M172=3000 ; Assign distance value to register

#1J^*      ; Jog Motor 1 this distance;  if following error at

     ;  command was 3, end cmd. pos. will be 2997

#1J^*      ; Jog Motor 1 this distance; if following error at

     ; command was 2, end cmd. pos. will be 5995

M172=P1*SIN(P2); Assign new distance value to register

#1J^*      ; Jog Motor 1 this distance

#1J= ; Return to prejog target position
```

See Also

Jogging Moves (Basic Motor Moves)
I-variables Ix19-Ix22
Memory map registers L:$082B, L:$08EB, etc.
Suggested M-variable definitions M172, M272, etc.
On-line commands J=, J={constant}, J=*, J^*

{jog command}^{constant}

Function:.....Jog until trigger
Scope: ..... Motor specific
Syntax: .....**J=^{constant}**

```
J={constant}^{constant}
J:{constant}^{constant}
J^{constant}^{constant}
J=*^{constant}
J:*^{constant}
J^*^{constant}
```

Where:

{constant} after the ^ is a floating point value specifying the distance from the trigger to which to jog after the trigger is found, in encoder counts

Remarks

This command format permits a "jog-until-trigger" Function. When the **^{constant}** structure is added to any "definite" jog command, the jog move can be interrupted by a pre-defined trigger condition, and the motor will move to a point relative to the trigger position as specified by the final value in the command. The "indefinite" jog commands **J+** and **J-** cannot be turned into jog-until-trigger moves. Jog-until-trigger moves are very similar to homing search moves, except they have a definite end position in the absence of a trigger, and they do not change the motor zero position.

The jog-until-trigger Function can be used with any jog command, whether the basic jog command is definite or indefinite. If the basic jog command is definite (e.g. **J=10000**), in the absence of a trigger the move will simply stop at the pre-defined position. If the basic jog command is indefinite (e.g. **J+**), in the absence of a trigger the motor will keep moving until stopped by another command or error condition.

The trigger condition for a jog-until-trigger move can either be an input flag, or a warning following error condition for the motor. If bit 17 of Ix03 is 0 (the default), the trigger is a transition of an input flag and/or encoder index channel from the set defined for the motor by Ix25. Encoder/flag variables 2 and 3 (e.g. I912 and I913) define which edges of which input signals create the trigger.

If bit 17 of Ix03 is 1, the trigger is the warning following error status bit of the motor becoming true. Ix12 for the motor sets the error threshold for this condition.

The trigger position can either be the hardware-captured position for the, or a software-read position. If bit 16 of Ix03 is 0 (the default), the encoder position latched by the trigger in the PMAC DSPGATE hardware is used as the trigger position. This is the most accurate option because it uses the position at the moment of the trigger, but it can only be used with incremental encoder feedback brought in on the same channel number as the triggering flag set. This option cannot be used for other types of feedback, or for triggering on following error.

If bit 16 of Ix03 is 1, PMAC reads the present sensor position after it sees the trigger. This can be used with any type of feedback and either trigger condition, but can be less accurate than the hardware capture because of software delays.

Jogging acceleration and velocity are determined by the values of Ix19-Ix22 in force at the time of this command. PMAC will reject this command if the motor is in a coordinate sytem that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
#1J=^1000 ; Jog to pre-jog position in the absence of a trigger,
          ; but if trigger is found, jog to +1000 counts from trigger.
```

```
#2J:5000^-100  ; Jog 5000 counts in the positive direction in the absence

     ; of a trigger,

     ; but if trigger is found, jog to -100 counts from trigger

     ; position.

#3J=20000^0    ; Jog to 20000 counts in the absence of a trigger, but if

     ; trigger is found, return to trigger position.
```

See Also

Jogging Moves (Basic Motor Moves)
I-variables Ix03, Ix19-Ix22, Ix25, Encoder/Flag I-variables 2 and 3
On-line commands J=, J={constant}, J:{constant}, J^{constant},
J=*, J:*, J^*
Program commands {axis}{data}^{data}

K

Function:.....Kill motor output

Scope: ..... Motor specific

Syntax: .....**K**

Remarks

This command causes PMAC to "kill" the outputs for the addressed motor. The servo loop is disabled, the DAC outputs are set to zero (Ix29 and/or Ix79 offsets are still in effect), and the AENA output for the motor is taken to the disable state (polarity is determined by E17).
Closed-loop control of this motor can be resumed with a **J/** command. The **A** command will re-establish closed-loop control for all motors in the addressed coordinate system, and the **<CTRL-A>** command will do so for all motors on PMAC.
The action on a **K** command is equivalent to what PMAC does automatically to the motor on an amplifier fault or a fatal following error fault.
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3). The program must be stopped first, usually with an **A** command. However, the global **<CTRL-K>** command will kill all motors immediately, regardless of whether any are running motion programs.

Example

```
K    ; Kill the addressed motor

#1K  ; Kill Motor 1

J/   ; Re-establish closed-loop control of Motor 1
```

See Also

Amplifier Fault, Following Error Limits, Stop Commands (Making Your Application Safe)
I-variables Ix29, Ix79
On-line commands <CTRL-A>, <CTRL-K>, A, Q, H, J/
Jumpers E17, E17A-E17H

LEARN



If a motor is closed loop, the learned position will differ from the actual position by the amount of the position following error because commanded position is used. If a motor is open-loop or killed, PMAC automatically sets motor commanded position equal to motor actual position, so the ***LEARN*** Function *can be used regardless of the state of the motor.*

Function: I

Scope: .....

Syntax: ..... **LEA**

```
                                        LRN[({axis}[,{ax
is}...]]
```

Remarks

This command causes PMAC to add a line to the end of the open motion program buffer containing axis position commands equal to the current commanded positions for some or all of themotors defined in the addressed coordinate system. In this way PMAC can "learn" a sequence of points to be repeated by subsequent execution of the motion program.
PMAC effectively performs a **PMATCH** Function, reading motor commanded positions and inverting the axis definition equations to compute axis positions.
If axis names are specified in the **LEARN** command, only position commands for those axes are used in the line added to the motion program. If no axis names are specified in the learn command, position commands for all nine possible axis names are used in the line added to the motion program. The position command for an axis with no motor attached ("phantom" axis) will be zero.

Example

```
&1   ; Address coordinate system 1

#1->10000X; Define motor 1 in C.S. 1

#2->10000Y; Define motor 2 in C.S. 1

OPEN PROG 1 CLEAR   ; Prepare program buffer for entry

F10 TA200 TS50 ; Enter required non-move commands

{move motors to a position, e.g. #1 to 13450 commanded, #2 to
29317 commanded}

LEARN(X,Y); Tell PMAC to learn these positions
```

```
X1.345 Y2.9317 ; This is the line that PMAC adds to PROG 1

{move motors to new position, e.g. #1 to 16752 cmd., #2 to 34726
cmd}

LEARN     ; Tell PMAC to learn positions

A0 B0 C0 U0 V0 W0 X1.6752 Y3.4726 Z0

     ; PMAC adds positions for all axes to PROG 1
```

See Also

Learning a Motion Program (Writing a Motion Program)

On-line command **PMATCH**

LIST

Function: ..... List the contents of the currently opened buffer.

Scope: ..... Global

Syntax: ..... **LIST**

Remarks

This command causes PMAC to report the contents of the currently opened buffer (PLC, PROG, or ROT) to the host. If no buffer is open, PMAC will report an error (ERR003 if I6=1 or 3). Notice that what is reported will not include any **OPEN**, **CLEAR**, or **CLOSE** statements (since these are *not* program commands).

You can list an unopened buffer by specifying the buffer name in the list command (e.g. **LIST PROG 1**). See further **LIST** commands, below.

Example

```
OPEN PROG 1    ; Open buffer for entry

LIST ; Request listing of open buffer

LINEAR    ; PMAC reports contents of open buffer

F10

X20 Y20

X0 Y0

RETURN

CLOSE     ; Close buffer

LIST ; Request listing of open buffer

<BELL>ERR003   ; PMAC reports error because no open buffer
```

See Also

On-line commands **OPEN, CLOSE, LIST PLC, LIST PROGRAM**

LIST COMP

Function: ..... List contents of addressed motor's compensation table
Scope: ..... Motor specific
Syntax: ..... **LIST COMP**

Remarks

This command causes PMAC to report to the host the contents of the compensation table belonging to the addressed motor. The values are reported in decimal ASCII form, multiple values to a line, with individual values separated by spaces.
The **LIST COMP DEF** command should be used to report the header information for this table.
If there is no table for the addressed motor, PMAC will reject the command (reporting ERR003 if I6=1 or 3).
The compensation table "belonging" to this motor may not affect this motor's position or be affected by it

Example

```
LIST COMP ; Request contents of compensation table

9 17 -3 6 35 87 65 24 18 -9 -16 -34; PMAC responds

-7 12 -3 -8 32 44 16 0 -20 -5 0    ; Continued response
```

See Also

Leadscrew Compensation Tables (Setting Up a Motor)
On-line commands **DEFINE COMP**, **DELETE COMP**, **LIST COMP DEF**

LIST COMP DEF

Function: ..... List definition of addressed motor's compensation table

Scope: ..... Motor specific

Syntax: ..... **LIST COMP DEF**

Remarks

This command causes PMAC to report to the host the definition of the compensation table that belongs to the addressed motor. The definition reported consists of the four items established in the **DEFINE COMP** command that set up the motor (even if some of those items were not specified explicitly):

1. The number of entries in the table (number of rows and number of columns for a two-dimensional table)

2. The number of the motor whose position provides the source data for the table (both source motors for a two-dimensional table)

3. The number of the motor whose position is modified by the table

4. The span of the table in counts of the source motor (in both dimensions for a two-dimensional table)

If there is no table for the addressed motor, PMAC will reject the command (reporting ERR003 if I6=1 or 3). The compensation table "belonging" to this motor may not affect this motor's position or be affected by it

Example

```
LIST COMP DEF  ; Request definition of compensation table
100,#2,#2,100000    ; PMAC responds; 100 entries in table,
     ; Motor 2 is source and target, span is 100,000
     ; counts
#3 LIST COMP DEF    ; Request def of comp table belonging to Motor 3
10,20,#4,#5,#6,50000,100000   ; PMAC responds
     ; A 2D 10x20 table, span of 50Kx100K counts
     ;#4 & #5 as source, #6 as target
```

See Also

Leadscrew Compensation Tables (Setting Up a Motor)
On-line commands **DEFINE COMP**, **DELETE COMP**, **LIST COMP**

LIST GATHER

Function:.....Report contents of the data gathering buffer.
Scope: .....Global
Syntax: .....**LIST GATHER [{start}] [,{length}]**
LIS GAT [{start}] [,{length}]

Where:

the optional **{start}** parameter is an integer constant specifying the distance from the start of the buffer (in words of memory) to begin the listing (0 is the default);
the optional **{length}** parameter (after a comma) is an integer constant specifying the number of words of the buffer to be sent to the host (to the end of the buffer is the default)

Remarks

This command causes PMAC to report the contents of the data gathering buffer to the host. The data is reported as 48-bit long words in hexadecimal format (12 characters per word) separated by spaces, 16 long words per line.
If neither **{start}** nor **{length}** is specified, the entire contents of the buffer will be reported. If **{start}** is specified, the reporting will begin **{start}** words from the beginning of the buffer. If **{length}** is specified, the reporting will continue for **{length}** words from the starting point.

Example

```
LIST GATHER    ; reports the whole buffer
LIST GATHER 256; skips the first 256 long words
```

```
LIST GATHER 0,32     ; reports the first 32 words
LIST GATHER ,32; does the same as above
LIST GATHER 64,128   ; skips the first 64 words, reports the next
128
```

See Also

Data Gathering Function (Analysis Features)

I-variables I19, I20, I21-I44.

On-line commands **GATHER**, **ENDGATHER**, **DEFINE GATHER**

Gathering and Plotting (PMAC Executive Program Manual)

LIST LDS

Function: ..... List Linking Addresses of Ladder Functions
Scope: ..... Global
Syntax: ..... **LIST LDS**

Remarks

This command causes PMAC to list the addresses of the internal routines that the special ladder-logic PLC cross-compiler needs to properly compile and link programs. This command is used automatically by the cross-compiler; a user only needs it directly for special debugging.

See Also

On-line commands **LIST LINK**

LIST LINK

Function: ..... List Linking Addresses of Internal PMAC Routines
Scope: ..... Global
Syntax: ..... **LIST LINK**

Remarks

This command causes PMAC to list the addresses of the internal routines that the PLC cross-compiler needs to properly compile and link its programs. This command is used automatically by the PLC cross-compiler in the Executive program.
For the standalone DOS cross-compiler, the ASCII characters of the PMAC response to this command must be contained in a file named LISTLINK.TXT in the same directory and subdirectory as the cross-compiler. Each separate version of the PMAC firmware potentially has different addresses for these routines, so a new LISTLINK.TXT file must be created any time the PMAC firmware is updated, even for a minor change such as from V1.15A to V1.15B.

Example

```
LIST LINK ; Request linking addresses
004532 004A97 005619 005F21 0062FE 0063A4
     ; PMAC responds
```

See Also

Compiled PLCs (Writing a PLC Program)

LIST PC

Function: ..... List Program at Program Counter

Scope: ..... Coordinate-system specific

Syntax: ..... **LIST PC[,[{constant}]]**

Where:

{constant} is a positive integer representing the number of words in the program to be listed

Remarks

This command causes PMAC to list the program line(s) that it is (are) about to calculate in the addressed coordinate system, with the first line preceded by the program number and each line preceded by the address offset. **LIST PC** just lists the next line to be calculated. **LIST PC,** lists from the next line to be calculated to the end of the program. **LIST PC,{constant}** lists the specified address range size starting at the next line to be calculated. To see the current line of execution, use the **LIST PE** command.

Because PMAC calculates ahead in a continuous sequence of moves, the **LIST PC** (Program Calculation) command will in general return a program line further down in the program than **LIST PE** will.

If the coordinate system is not pointing to any motion program, PMAC will return an error (ERR003 if I6=1 or 3). Initially the pointing must be done with the **B{constant}** command.

Example

```
LIST PC   ; List next line to be calculated

P1:22:X10Y20   ; PMAC responds

LIST PC,4 ; List next 4 words of program to be calculated

P1:22:X10Y20   ; PMAC responds

24:X15Y30

LIST PC,  ; List rest of program

P1:22:X10Y20   ; PMAC responds

24:X15Y30

26:M1=0

28:RETURN
```

See Also

On-line commands B{constant}, LIST, PC, LIST PE, PE

LIST PE

Function: ..... List Program at Program Execution
Scope: .....
.....Coordinate-system specific
Syntax: .....**LIST PE[,[{constant}]]**

Where:
{constant} is a positive integer representing the number of words in the program to be listed

Remarks

This command causes PMAC to list the program line(s) starting with the line containing the move that it is currently executing in the addressed coordinate system, with the first line preceded by the program number, and each line preceded by the address offset.

Because PMAC calculates ahead in a continuous sequence of moves, the **LIST PC** (Program Calculation) command will in general return a program line further down in the program than **LIST PE** will.

LIST PE returns only the currently executing line. **LIST PE,** returns from the currently executing line to the end of the program. **LIST PE,{constant}** returns the specified number of words in the program, starting at the currently executing line.

If the coordinate system is not pointing to any motion program, PMAC will return an error (ERR003 if I6=1 or 3). Initially the pointing must be done with the **B{constant}** command.

Example

```
LIST PE   ; List presently executing line

P5:35:X5Y30    ; PMAC responds

LIST PE,4 ; List 4 program words, starting with executing line

P5:35:X5Y30    ; PMAC responds

37:X12Y32

LIST PE,  ; List rest of program, starting with executing line

P5:35:X5Y30    ; PMAC responds

37:X12Y32

39:X0 Y10

41:RETURN
```

See Also

On-line commands `B{constant}`, `LIST`, `LIST PC`, `PC`, `PE`

LIST PLC

Function:.....List the contents of the specified PLC program.
Scope: .....
.....Global
Syntax: .....**LIST PLC {constant}**

Where:
{constant} is an integer from 0 to 31 representing the number of the PLC program

Remarks

This command causes PMAC to report the contents of the specified PLC program buffer to the host. The contents are reported in ASCII text form. If I9 is 0 or 2, the contents are reported in short form (e.g. *ENDW*). If I9 is 1 or 3, the contents are reported in long form (e.g. *ENDWHILE*).
PLCs 0-15 can be protected by password. If the PLC is protected by password, and the proper password has not been given, PMAC will reject this command (reporting an ERR002 if I6=1 or 3).

Example

```
LIST PLC 0

LIST PLC 5
```

See Also

PLC Program Features

I-variables I3, I4, I9

On-line commands **LIST**, **LIST PROG**, **PASSWORD={string}**

Program Command Specification

LIST PROGRAM

Function:..... List the contents of the specified motion program.

Scope: .....
.....Global

Syntax: .....**LIST PROGRAM {constant} [{start}] [,{length}]**

LIST PROG {constant} [{start}] [,{length}]

Where:

{constant} is an integer from 1 to 32767 specifying the number of the motion program

the optional **{start}** parameter is an integer constant specifying the distance from the start of the buffer (in words of memory) to begin the listing (0 is the default);

the optional **{length}** parameter (after a comma) is an integer constant specifying the number of words of the buffer to be sent to the host (to the end of the buffer is the default)

Remarks

This command causes PMAC to report the contents of the specified fixed motion program buffer (PROG) to the host. The contents are reported in ASCII text form. If I9 is 0 or 2, the contents are reported in short form (e.g. *LIN*). If I9 is 1 or 3, the contents are reported in long form (e.g. *LINEAR*).

If neither **{start}** nor **{length}** is specified, the entire contents of the buffer will be reported. If **{start}** is specified, the reporting will begin **{start}** words from the beginning of the buffer. If **{length}** is specified, the reporting will continue for **{length}** words from the starting point.

If either **{start}**, **{length}**, or both, or just the comma, is included in the command, the listing of the program will include the buffer address offsets with each line. Having a listing with these offsets can be useful in conjunction with later use of the **PC** (Program-Counter) and **LIST PC** commands.

If the motion program requested by this command does not exist in PMAC, PMAC will reject this command (reporting an ERR003 if I6=1 or 3).

PROGs 1000-32767 can be protected by password. If the PROG is protected by password, and the proper password has not been given, PMAC will reject this command (reporting an ERR002 if I6=1 or 3).

Example

```
LIST PROG 9     ; Request listing of all of motion program 9
LINEAR     ; PMAC responds
F10
X10Y10
X0Y0
RETURN
LIST PROG 9,    ; Request listing of program w/ address offsets
0:LINEAR
1:F10
2:X10Y10   ; Notice that a 2-axis command takes 2 addresses
4:X0Y0
6:RETURN
LIST PROG 9,4   ; Request listing starting at address 4
4:X0Y0
6:RETURN
LIST PROG 9,2,4; Request listing starting at 2, 4 words long
2:X10Y10
4:X0Y0
LIST PROG 9,,2 ; Request listing starting at top, 2 words long
0:LINEAR
1:F10
```

See Also

Writing a Motion Program
I-variables I3, I4, I9

On-line commands LIST, PC, LIST PC., PASSWORD={string}.

Program Command Specification

M{constant}



If a motion program buffer (including a rotary buffer) is open when this command is sent to PMAC it will be entered into the buffer for later execution, to be interpreted as an M-code subroutine call.

Function: .....Repo
Scope: .....
Syntax: .....**M{**

Where:
{constant} is an integer from 0 to 1023 representing the number of the M-variable;
the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

Remarks

This command causes PMAC to report the current value of the specified M-variable or range of M-variables. It does not cause PMAC to report the *definition* (address) of the M-variable(s); that is done with the **M{constant}->** command.

Example

```
M0    ; Host asks for value

3548976    ; the PMAC response

M165

5.75

M1..3

1

0

1
```

See Also

M-Variables (Computational Features)

On-line commands **M{constant}={expression}**, **M{constant}->**

Program commands **M{constant}**, **M{constant}={expression}**

M{constant}={expression}

Function: ..... Assign value to M-variable(s).

Scope: .....
..... Global

Syntax: ..... **M{constant}[..{constant}]={expression}**

Where:

{constant} is an integer from 0 to 1023 representing the number of the M-variable;

the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

{expression} contains the value to be given to the specified M-variable(s)

Remarks

This command assigns the value on the right side of the equals sign to the specified M-variable(s). It does not assign a *definition* (address) to the M-variable(s); that is done with the **M{constant}->{definition}** command.

If a motion or PLC program buffer is open when this command is sent to PMAC, it will be entered into the buffer for later execution.

Example

```
M1=1

M9=M9 & $20

M102=-16384

M1..8=0
```

See Also

M-Variables (Computational Features)

On-line commands **M{constant}**, **M{constant}->{definition}**

Program commands **M{constant}**, **M{constant}={expression}**

M{constant}->

Function: ..... Report current M-variable definition(s)
Scope: .....
..... Global
Syntax: ..... **M{constant}[..{constant}]->**

Where:

{constant} is an integer from 0 to 1023 representing the number of the M-variable;

the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

Remarks

This command causes PMAC to report the definition (address) of the specified M-variable or range of M-variables. It does not cause PMAC to report the *value* of the M-variable{s); that is done with the **M{constant}** command. When I9 is 0 or 2, only the definition itself (e.g. *Y:$FFC2,0*) is returned. When I9 is 1or 3, the entire definition statement (e.g. *M11->Y:$FFC2,0*) is returned.

Example

```
M1-> ; Host requests definition

Y:$FFC2,8 ; the PMAC response

M101..103->

X:$C001,24,S

Y:$C003,8,16,S

X:$C003,24,S
```

See Also

M-Variables (Computational Features)

On-line commands **M{constant}**, **M{constant}->{definition}**, **M{constant}={expression}**

Program command **M{constant}={expression}**

M{constant}->*

Function: ..... Self-Referenced M-Variable Definition
Scope: .....
..... Global
Syntax: ..... **M{constant}[..{constant}]->***

Where:

{constant} is an integer from 0 to 1023 representing the number of the M-variable;

the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

Remarks

This command causes PMAC to reference the specified M-variable or range of M-variables to its own definition word. If you just wish to use an M-variable as a flag, status bit, counter, or other simple variable, there is no need to find an open area of memory, because it is possible to use some of the definition space to hold the value. Simply define this form of the M-variable and you can use this M-variable much as you would a P-variable, except it only takes integer values in the range -1,048,576 to +1,048,575 (-2^{20} to $+2^{20}-1$).
When the definition is made, the value is automatically set to 0.
This command is also useful to "erase" an existing M-variable definition.

Example:

```
M100->*

M20..39->*
```

```
M0..1023->*     ; This erases all existing M-variable definitions
     ; It is a good idea to use this before loading new ones
```

See Also

M-Variables (Computational Features)

On-line commands **M{constant}**, **M{constant}->**, **..... M{constant}->{definition}**, **M{constant}={expression}**

Program command **M{constant}={expression}**

M{constant}->D:{address}

Function: ..... Long Fixed-Point M-Variable Definition
Scope: ..... Global
Syntax: ..... **M{constant}[..{constant}]->D[:]{address}**

Where:
{constant} is an integer from 0 to 1023 representing the number of the M-variable;
the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;
{address} is an integer constant from 0 to 65,535 ($0 to $FFFF if specified in hex).

Remarks

This command causes PMAC to define the specified M-variable or range of M-variables to a 48-bit double word (both X and Y memory; X more significant) at the specified location in the PMAC address space. The data is interpreted as a fixed-point signed (two's complement) integer.
The definition consists of the letter **D**, an optional colon (**:**), and the word address.
Memory locations for which this format is useful are labeled with **D:** in the memory map.

Example

```
M161->D:$0028  ; Motor 1 desired position register specified in
hex

M161->D40 ; Motor 1 desired position register specified in deci-
mal

M162->D$2C; Motor 1 actual position register specified in hex
```

See Also

M-Variables (Computational Features)

On-line commands **M{constant}**, **M{constant}->**, **M{constant}={expression}**

Program command **M{constant}={expression}**

M{constant}->DP:{address}

Function: ..... Dual-Ported RAM Fixed-Point M-Variable Definition

Scope: ..... Global

Syntax: ..... **M{constant}[..{constant}]->DP[:]{address}**

Where:

{constant} is an integer from 0 to 1023 representing the number of the M-variable;
the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;
{address} is an integer constant from 0 to 65,535 ($0 to $FFFF if specified in hex).

Remarks

This command causes PMAC to define the specified M-variable or range of M-variables to point to 32 bits of data in the low 16 bits of both X and Y memory at the specified location in the PMAC address space. The data is interpreted as a fixed-point signed (two's complement) integer.

The definition consists of the letters **DP**, an optional colon (**:**), and the word address.

This format is only useful for dual-ported RAM locations $D000 to $DFFF (Option 2 is required). With this format, the host can read or write to the corresponding location with a standard 32-bit integer data format. The data in the X word is the most significant word, which means on the host side the most significant word is in the higher of two consecutive addresses (standard Intel format).

Example

```
M150->DP:$D200

M250->DP$D201
```

See Also

M-Variables (Computational Features)

Dual-Ported RAM (Writing a Host Communications Program)

On-line commands **M{constant}**, **M{constant}->**, **..... M{constant}->F:{address}**, **M{constant}={expression}**

Program command **M{constant}={expression}**

M{constant}->F:{address}

Function: ..... Dual-Ported RAM Floating-Point M-Variable Definition
Scope: ..... Global
Syntax: ..... `M{constant}[..{constant}]->F[:]{address}`

Where:
`{constant}` is an integer from 0 to 1023 representing the number of the M-variable;
the optional second `{constant}` must be at least as great as the first `{constant}` -- it represents the number of the end of the range;
`{address}` is an integer constant from 0 to 65,535 ($0 to $FFFF if specified in hex).

Remarks

This command causes PMAC to define the specified M-variable or range of M-variables to point to 32 bits of data in the low 16 bits of both X and Y memory at the specified location in the PMAC address space. The data is interpreted as a floating-point value with the IEEE single-precision (32-bit) format.
The definition consists of the letter **F**, an optional colon (**:**), and the word address.
This format is only useful for dual-ported RAM locations $D000 to $DFFF (Option 2 required). With this format, the host can read or write to the corresponding location with the standard IEEE 32-bit floating-point data format.
The IEEE 32-bit floating point format has the sign bit in bit 31 (MSB); the biased exponent in bits 30 to 23 (the exponent is this value minus 127), and the fraction in bits 22 to 0 (there is an implied 1 added to the fraction in the mantissa). The words are arranged in the standard Intel format.

Example

```
M155->F:$D401

M255->F$D402
```

See Also

M-Variables (Computational Features)

Dual-Ported RAM (Writing a Host Communications Program)

On-line commands **M{constant}**, **M{constant}->**, **..... M{constant}->DP:{address}**, **M{constant}={expression}**

Program command **M{constant}={expression}**

M{constant}->L:{address}

Function: ..... Long Word Floating-Point M-Variable Definition
Scope: ..... Global
Syntax: ..... `M{constant}[..{constant}]->L[:]{address}`

Where:
`{constant}` is an integer from 0 to 1023 representing the number of the M-variable;
the optional second `{constant}` must be at least as great as the first `{constant}` -- it represents the number of the end of the range;
`{address}` is an integer constant from 0 to 65,535 ($0 to $FFFF if specified in hex).

Remarks

This command causes PMAC to define the specified M-variable or range of M-variables to point to a long word (48 bits) of data -- both X and Y memory -- at the specified location in the PMAC address space. The data is interpreted as a floating-point value with the PMAC own 48-bit floating-point format.
The definition consists of the letter `L`, an optional colon (`:`), and the word address.
Memory locations for which this format is useful are labeled with 'L:' in the memory map.

Example

```
M165->L:$081F

M265->L$0820

M265->L2080
```

See Also

M-Variables (Computational Features)

On-line commands `M{constant}`, `M{constant}->`, `..... M{constant}->D:{address}`, `M{constant}={expression}`

Program command `M{constant}={expression}`

M{constant}->TWB:{address}

Function: ..... Binary Thumbwheel-Multiplexer Definition
Scope: ..... Global
Syntax: ..... M{constant}[..{constant}]->TWB[:]{multiplex address},{offset},{size},{format}

Where:
{constant} is an integer from 0 to 1023 representing the number of the M-variable; the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;
{multiplex address} is an integer constant in the range 0 to 255, representing the byte address in the multiplexing scheme on the thumbwheel port of the least significant bit to be used in the M-variable(s);
{offset} is an integer constant from 0 to 7, representing which bit of this byte is the least significant bit to be used in the M-variable;
{size} is an integer constant from 1 to 32, representing the number of consecutive bits to be used in the M-variable(s);
{format} (optional) is either U for unsigned, or S for signed (two's complement). If no format is specified, U (unsigned) is assumed

Remarks

This command causes PMAC to define the specified M-variable or range of M-variables to a consecutive of input bits multiplexed on the thumbwheel port with Accessory 18 or compatible hardware.

Example:

```
M0->TWB:0,0,1

M1->TWB:0,1,1

M10->TWB:3,4,4,U

M745->TWB:4,0,16,S

M872->TWB:0,4,1
```

See Also

M-Variables (Computational Features)
On-line commands **M{constant}**, **M{constant}->**, ... **M{constant}->TWD:{address}**
Thumbwheel Multiplexer Board (ACC-18) Manual

M{constant}->TWD:{address}

Function: ..... BCD Thumbwheel-Multiplexer M-Variable Definition

Scope: ..... Global

Syntax: ..... **M{constant}[..{constant}]->TWD[:]{multiplex address},{offset},**
..... **{size}[.{dp}],{format}**

where:

{constant} is an integer from 0 to 1023 representing the number of the M-variable;

the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

{multiplex address} is an integer constant in the range 0 to 255, representing the address in the multiplexing scheme on the thumbwheel port of the most significant digit (lowest address) to be used in the M-variable(s);

{offset} is 0 or 4, representing whether the most significant digit is in the low nibble (left digit of pair) or high nibble (right digit of pair) of the pair of digits at **{multiplex address}**, respectively;

{size} is an integer constant from 1 to 12, representing the number of digits to be used in the M-variable(s);

{dp} (optional) is an integer constant from 0 to 8, representing the number of these digits to be interpreted as being to the right of the decimal point;

{format} (optional) is either U for unsigned, or S for signed. If it is signed, the least significant bit of the most significant digit is taken as the sign bit (the rest of the most significant digit is ignored). If no format is specified, U (unsigned) is assumed.

Remarks

This command causes PMAC to define the specified M-variable or range of M-variables to point to a set of binary-coded-decimal digits multiplexed on the thumbwheel port with Accessory 18 or compatible hardware. Thumbwheel-multiplexer M-variables are read-only, floating-point variables. Once defined, they are to be used in expressions, and queried. Each time one is used in an expression, the proper addresses on the multiplexer board(s) are read.

Example

M100->TWD:4,0,8.3,U means the most significant digit is at multiplex address 4, low nibble (left digit); there are 8 digits, 3 of which are fractional; and it is always interpreted as a positive value. This corresponds to eight thumbwheel digits along the bottom row of the lowest-addressed thumbwheel board, with the decimal point 3 digits in from the right.

M99->TWD:0,0,1,U means that a single digit is used, at multiplex address 0, low nibble (left digit). This corresponds to the upper left thumbwheel on the lowest-addressed thumbwheel board.

See Also

M-Variables (Computational Features)

On-line commands **M{constant}**, **M{constant}->**, **M{constant}->TWB:{address}**

Thumbwheel Multiplexer Board (ACC-18) Manual

M{constant}->TWR:{address},{offset}

Function: Resolver Thumbwheel-Multiplexer M-Variable Definition

Scope: Global

Syntax: **M{constant}[..{constant}]->TWR[:]{multiplex address}, {offset}**

Where:

{constant} is an integer from 0 to 1023 representing the number of the M-variable;

the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

{multiplex address} is an integer constant, divisible by 2, in the range 0 to 254, representing the address in the multiplexing scheme of the ACC-8D Option 7 resolver-to-digital converter board on the thumbwheel multiplexer port, as determined by the DIP switch settings on the board

{offset} is an integer constant from 0 to 7, representing the location of the device at the specified multiplexer address, as determined by in the buffer on the ACC-8D Option 7 and the actual pins to which the device was wired.

Remarks

It is not necessary to use an M-variable to access an R/D converter for actual servo or phasing feedback purposes. I-variables (Ix10, Ix81, I8x, I9x) are used for that purpose. However, even if this is your only use of the R/D converter, it is usually desirable to assign M-variables to the R/D converters for set-up and diagnostic purposes

This command causes PMAC to define the specified M-variable or range of M-variables to point to a 12-bit word from a resolver-to-digital (R/D) converter or similar device serially multiplexed on the "thumbwheel" port on an ACC-8D Option 7 or compatible board.
The address on the multiplex port specified here must match the address set by the DIP switches on board the ACC-8D Opt-7. The ACC-8D Opt-7 manual contains a table listing all of the possibilities.
One of the DIP switches on the ACC-8D Opt-7 board determines whether the R/D converters on board have offset values of 0 to 3 or 4 to 7. The **{offset}** specifier must match this DIP switch setting and the number of the R/D device on the board.

This is a read-only M-variable format. Use of this variable in an on-line query command or a program statement will cause PMAC to clock in 12 bits of unsigned data (range 0 to 4095) from the specified device through the multiplexer port.

Example

```
M100->TWR:0,0

M99->TWR:4,5
```

See Also

M-Variables (Computational Features)

On-line commands **M{constant}**, **M{constant}->**, **.....M{constant}->TWS:{address}**

Resolver-to-Digital Converter Board (ACC-8D Opt-7) Manual

M{constant}->TWS:{address}

Never use this M-variable form to write to a word that is set up for inputs.
Never use this M-variable form to read from a word that is set up for outputs.

The individual bits of the thumb-wheel port on an Accessory 34x board can not be directly assigned to an M-variable. Only 32-bit words (ports) of input or output can be accessed.

Function: ..... Serial Thumbwheel-M
Scope: .....
Syntax: ..... **M{constant}**{multiplex address}

Where:

{constant} is an integer from 0 to 1023 representing the number of the M-variable;

the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

{multiplex address} is an integer constant, divisible by 4, in the range 0 to 124, representing the address in the multiplexing scheme of the first of four bytes in the 32-bit input or output word. Adding 1 to the **{multiplex address}** designates it as a read-only variable and adding 2 designates it as a write-only variable.

Remarks

This command causes PMAC to define the specified M-variable or range of M-variables to point to a 32-bit word of input or output serially multiplexed on the "thumbwheel" port on an Accessory 34x board.
The address on the multiplex port specified here must match the address set by the DIP switch on board the ACC-34x. The ACC-34x manual contains a table listing all of the possibilities.
The entire word must either be all input or all output. On power-up/reset, all ACC-34x words are software-configured as inputs (if the hardware is configured for outputs, all outputs will be OFF -- pulled up to the supply voltage). Any subsequent write operation to an I/O word on the port with one of these M-variables automatically makes the entire word an output word, with individual bits ON or OFF, as determined by the value written to the word.
Any subsequent read operation of a word that has been set up for output configures, or tries to configure, the entire word into an input word, which turns any hardware outputs OFF. Therefore, it is important that the following rules be observed when working with these M-variables:
Because both reads and writes are enabled when a TWS type M-variable is used to point directly to the base address of an accessory 34x port (e.g. **M300->TWS:40**) their use is **very strongly discouraged**. Reads and writes are enabled when the least significant and the next least significant addresses bits are both zero (e.g. decimal 40 = 01000000 in binary).
In this situation, any accidental read of an output port (say via the Executive programs watch window) will cause all the output transistors to be turned off (outputs pulled to the supply voltage)! Alternatively, writing to an input port, will automatically reconfigure it to an output port! It is therefore safer and more predictable when bits 0 & 1 of the M-variable definition are intentionally used to disable either the read function or the write function. Setting one of these bits gives the **read-only** or **write-only** form of the TWS M-variable.
An M-variable pointing to an **input** port is defined as read-only by setting the **{multiplex address}** to a legal byte number (from column 2 of Table 1 of the Acc 34x manual) **plus 1**. Any attempt to write to a TWS type M-variable defined in this manner (with bit zero of its address set to 1) is automatically prevented by the PMAC firmware. For an **output** port, the **{multiplex address}** should be a legal byte number (from column 2 of Table 1 of the Acc 34x manual) **plus 2**. Any attempt to read a TWS type M-variable defined in this manner (with bit one of its address set to 1) returns zero and the actual read is prevented by the PMAC firmware.
Because you can not directly access the individual bits of the "thumbwheel" port on an Accessory 34x board and because of the relatively long time it takes to clock the data in or out of PMAC (A 32-bit Read or a 32-bit Write to an individual port takes approximately 64 microseconds of time in the the PMAC background time slot) it is best to keep an "image" of each M-variable of this type in internal memory. The image variable would preferably be a 32-bit or 48-bit fixed point M-variable, but it could also be a 48-bit floating point P or Q variable.
The best procedure for using TWS M-variables in a program is as follows. The input word (TWS M-variable) should be copied into its image variable at the beginning of a sequence of operations. The operations can then be done on the image variable without requiring PMAC to actually read or write to the I/O port for each operation. The output word is first "assembled" into its image variable, then copied to the actual output word once at the end of a sequence of operations. This procedure will allow the most efficient and flexible use of TWS M-variables.
This type of variable can only be used in background tasks (PLCs and PLCCs 1-31). They cannot be used in foreground tasks (motion programs and PLC and PLCC 0).

Example

To address Port B of board #1 as an output using M101, use the following definition. *This addressing format is not recommended because accidental reads of the port are not protected against* (Consider using the write-only format).

```
M100->TWS:4     ;Port B  (BIO 0-31) of an  ACC-34x
     ;with SW1 switches all ON
     ;NOT assigned for write-only
```

To address Port B as above using the write-only addressing format, use the following definition:

```
M100->TWS:6     ;Port B  (BIO 0-31) of an  ACC-34x
     ;with SW1 switches all ON
     ;assigned for write-only (6=4+2)
```

To address Port A of board #1 as an input using M99, use the following definition. *This addressing format is not recommended because accidental writing to the port is not protected against* (Consider using the read-only format).

```
M99->TWS:0;Port A (AIO 0-31) of an ACC-34x with
     ;SW1 switches all ON
     ;NOT assigned for read-only
```

To address Port A as above using the read-only addressing format, use the following definition:

```
M100->TWS:1    ;Port A (AIO 0-31) of an ACC-34x with
     ;SW1 switches all ON
     ;assigned for read-only (1=0+1)
```

Yet another Example: to address Port A of board #6 as an input using M300, we would use the following definition :

```
M300->TWS:41   ;Port A (AIO 0-31) of an ACC-34x with
     ;SW1 switches ON,ON,OFF,ON,ON
     ;assigned for read-only (41=40+1)
```

See Also

M-Variables (Computational Features)

On-line commands **M{constant}**, **M{constant}->**, **.....M{constant}->TWR:{address}**

Serial I/O Multiplexer Board (ACC-34) Manual

M{constant}->X/Y:{address}

Function: ..... Short Word M-Variable Definition

Scope: .....
.....Global

Syntax: .....**M{constant}[..{constant}]->**

```
X[:]{address},{offset}[,{width}[,{format}]]

                                M{constant}[..{constant}]->

Y[:]{address},{offset}[,{width}[,{format}]]
```

Where:

{constant} is an integer from 0 to 1023 representing the number of the M-variable;

the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

{address} is an integer constant from 0 to 65,535 ($0 to $FFFF if specified in hex);

{offset} is an integer constant from 0 to 23, representing the starting (least significant) bit of the word to be used in the M-variable(s), or 24 to specify the use of all 24 bits;

{width} (optional) is an integer constant from the set {1, 4, 8, 12, 16, 20, 24}, representing the number of bits from the word to be used in the M-variable(s); if **{width}** is not specifed, a value of 1 is assumed;

{format} (optional) is a letter from the set [U, S, D, C], specifying how PMAC is to interpret this value: (U=Unsigned integer, S=Signed integer, D=Binary-coded Decimal, C=Complementary binary-coded decimal); if **{format}** is not specified, U is assumed.

Remarks

This command causes PMAC to define the specified M-variable or range of M-variables to point to a location in one of the two halves (X or Y) of the PMAC data memory. In this form, the variable can have a width of 1 to 24 bits and can be decoded several different ways, so the bit offset, bit width, and decoding format must be specified (the bit width and decoding format do have defaults.
The definition consists of the letter **X** or **Y**, an optional colon (**:**), the word address, the starting bit number (offset), an optional bit width number, and an option format-specifying letter..
Legal values for bit width and bit offset are inter-related. The table below shows the possible values of **{width}**, and the corresponding legal values of **{offset}** for each setting of **{width}**.

{offset}

1	0 -- 23
4	0,4,8,12,16,20
8	0,4,8,12,16
12	0,4,8,12
16	0,4,8
20	
24	0

The format is irrelevant for 1-bit M-variables, and should not be included for them. If no format is specified, 'U' is assumed.

Example

```
     ; Machine Output 1
M1->Y:$FFC2,8,1; 1-bit (full spec.)
M1->Y$FFC2,8   ; 1-bit (short spec.)
     ; Encoder 1 Capture/Compare Register
M103->X:$C003,0,24,U; 24-bit (full spec.)
M103->X$C003,24; 24-bit (short spec.)
     ; DAC 1 Output Register
M102->Y:$C003,8,16,S; 16-bit value
M102->Y49155,8,16,S ; same, decimal address
```

See Also

M-Variables (Computational Features)

On-line commands **M{constant}**, **M{constant}->**, **..... M{constant}->D:{address}**, **M{constant}={expression}**

Program command **M{constant}={expression}**

MACROAUX

Function: ..... Report or write MACRO auxiliary parameter value
Scope: ..... Global
Syntax: ..... **MACROAUX{NodeNum}{ParamNum}[={constant}]**
MX{NodeNum}{ParamNum}[={constant}]

Where:
{NodeNum} is an integer constant from 0 to 15 specifying the slave number of the node
{ParamNum} is an integer constant from 0 to 65535 specifying the auxiliary parameter number for this node (2 to 254 required for a write operation)
{constant} is an integer constant from -32768 to +32767 representing the value to be written to the specified parameter

Remarks

This command permits PMAC to read or write auxiliary register values from slave nodes across the MACRO ring. The command must specify the node number of the slave node, the auxiliary parameter number at this node, and if a write command, the value to write into the register.
If used as a read command (no '**={constant}**' in the command), PMAC will report the value of the specified parameter back to the host as ASCII text, just as if the value of one of its own parameters had been requested.
Only one auxiliary access (read or write) of a single node can be done on one command line.
In order to access the auxiliary registers of a MACRO node *n*, bit *n* of I1000 must be set to 1.
If the slave node returns an error message or the slave node does not respond within 32 servo cycles, PMAC will report ERR008. Bit 5 of global status register X:$0003 is set to report such a MACRO auxiliary communications error. Register X:$0798 holds the error value. It is set to $010000 for a timeout error, or $*xxxx*FE if the slave node reports an error, where *xxxx* is the 16-bit error code reported by the slave node.

Example

```
MACROAUX1,24=2000   ; Set Node 1 Parameter 24 to 2000

MACROAUX1,24   ; Request value of Node 1 Parameter 24

2000 ; PMAC reports value
```

See Also

On-line commands **MACROAUXREAD, MACROAUXWRITE**

PLC Program commands **MACROAUXREAD, MACROAUXWRITE**

MACROAUXREAD

Function: ..... Read MACRO auxiliary parameter value
Scope: ..... Global
Syntax: ..... **MACROAUXREAD{NodeNum}{ParamNum}{Variable}**
MXR{NodeNum}{ParamNum}{Variable}

Where:
{NodeNum} is an integer constant from 0 to 15 specifying the slave number of the node
{ParamNum} is an integer constant from 0 to 65535 specifying the auxiliary parameter number for this node
{Variable} is the name of the PMAC variable (I, P, Q, or M) into which the parameter value is to be copied

Remarks

This command permits PMAC to read auxiliary register values from slave nodes across the MACRO ring. The command must specify the node number of the slave node, the auxiliary parameter number at this node, and the name of the PMAC variable to receive the value.
Only one auxiliary access (read or write) of a single node can be done on one command line.
In order to access the auxiliary registers of a MACRO node *n*, bit *n* of I1000 must be set to 1.
If the slave node returns an error message or the slave node does not respond within 32 servo cycles, PMAC will report ERR008. Bit 5 of global status register X:$0003 is set to report such a MACRO auxiliary communications error. Register X:$0798 holds the error value. It is set to $010000 for a timeout error, or $*xxxx*FE if the slave node reports an error, where *xxxx* is the 16-bit error code reported by the slave node.

Example

```
MACROAUXREAD1,24,P1 ; Read Node 1 Parameter 24 into P1

MXR5,128,M100  ; Read Node 5 Parameter 128 into M100
```

See Also

On-line commands **MACROAUX, MACROAUXWRITE**

PLC Program commands **MACROAUXREAD, MACROAUXWRITE**

MACROAUXWRITE

Function: ..... Write MACRO auxiliary parameter value
Scope: ..... Global
Syntax: ..... **MACROAUXWRITE{NodeNum}{ParamNum}{Variable}**
MXW{NodeNum}{ParamNum}{Variable}

Where:
{NodeNum} is an integer constant from 0 to 15 specifying the slave number of the node
{ParamNum} is an integer constant from 2 to 253 specifying the auxiliary parameter number for this node
{Variable} is the name of the PMAC variable (I, P, Q, or M) from which the parameter value is to be copied

Remarks

This command permits PMAC to write auxiliary register values to slave nodes across the MACRO ring. The command must specify the node number of the slave node, the auxiliary parameter number at this node, and the name of the PMAC variable from which the value comes.
Only one auxiliary access (read or write) of a single node can be done on one command line.
In order to access the auxiliary registers of a MACRO node *n*, bit *n* of I1000 must be set to 1.
If the slave node returns an error message or the slave node does not respond within 32 servo cycles, PMAC will report ERR008. Bit 5 of global status register X:$0003 is set to report such a MACRO auxiliary communications error. Register X:$0798 holds the error value. It is set to $010000 for a timeout error, or $*xxxx*FE if the slave node reports an error, where *xxxx* is the 16-bit error code reported by the slave node.

Example

```
MACROAUXWRITE1,24,P1; Write value of P1 to Node 1 Parameter 24

MXW5,128,M100  ; Write value of M100 to Node 5 Parameter 128
```

See Also

On-line commands **MACROAUX, MACROAUXREAD**

PLC Program commands **MACROAUXREAD, MACROAUXWRIT**E

MFLUSH

Function:.....Clear pending synchronous M-variable assignments

Scope: .....
.....Coordinate-system specific

Syntax: .....**MFLUSH**

Remarks

This command permits the user to clear synchronous M-variable assignment commands that have been put on the stack for intended execution with a subsequent move (without executing the commands). As an on-line command, it is useful for making sure pending outputs are not executed after a program has been stopped

Examples

```
/     ; Stop execution of a program

MFLUSH     ; Clear M-variable stack

B1R   ; Start another program; formerly pending M-variables will not
execute
```

See Also

Program commands **M{constant}=={expression},**
M{constant}&={expression},
M{constant}|={expression},
M{constant}^={expression}

O{constant}

Function:.....Open loop output
Scope: .....
..... Motor specific
Syntax: .....**O{constant}**

Where:
{constant} is a floating-point value representing the magnitude of the output as a percentage of Ix69 for the motor, with a range of +/-100

Remarks

This command causes PMAC to put the motor in open-loop mode and force an output of the specified magnitude, expressed as a percentage of the maximum output parameter for the motor (Ix69). This command is commonly used for set-up and diagnostic purposes (for instance, a positive **O** command must cause position to count in the positive direction, or closed-loop control cannot be established), but it can also be used in actual applications.
If the motor is *not* PMAC-commutated, this command will create a DC output voltage on the single DAC for the motor. If the motor is commutated by PMAC, the commutation algorithm is still active, and the specified magnitude of output is apportioned between the two DAC outputs for the motor according to the instantaneous commutation phase angle.
If the value specified is outside the range +/-100, the output will saturate at +/-100% of Ix69.

Closed-loop control for the motor can be re-established with the **J/** command. It is a good idea to stop the motor first with an **O0** command if it has been moving in open-loop mode.
To do a variable O-command, define an M-variable to the *filter result* register (X:$003A, etc.), command an **O0** to the motor to put it in open-loop mode, then assign a variable value to the M-variable. This technique will even work on PMAC-commutated motors.
PMAC will reject this command if the motor is in a coordinate system that is currently running a motion program (reporting ERR001 if I6 is 1 or 3).

Example

```
O50  ; Open-loop output 50% of Ix69 for addressed motor

#2O33.333 ; Open-loop output 1/3 of Ix69 for Motor 2

O0   ; Open-loop output of zero magnitude

J/   ; Re-establish closed-loop control
```

See Also

On-line commands **J/**, **K**

Memory-map registers **X:$003A, X:$0076,** etc.

Suggested M-variable definitions **Mx71**.

OPEN PLC

Function: Open a PLC program buffer for entry
Scope: Global
Syntax: **OPEN PLC {constant}**

Where:
{constant} is an integer from 0 to 31 representing the PLC program to be opened

Remarks

This command causes PMAC to open the specified PLC program buffer for entry and editing. This permits subsequent program lines that are valid for a PLC to be entered into this buffer. When entry of the program is finished, the **CLOSE** command should be used to prevent further lines from being put in the buffer.
No other program buffers (PLC, fixed or rotary motion) may be open when this command is sent (PMAC will report *ERR007* if I6=1 or 3). It is a good idea always to precede an **OPEN** command with a **CLOSE** command to make sure no other buffers have been left open.
PLCs 0-15 can be protected by password. If the PLC is protected by password, and the proper password has not been given, PMAC will reject this command (reporting an ERR002 if I6=1 or 3).
Opening a PLC program buffer automatically disables that PLC program. Other PLC programs and motion programs will keep executing. Closing the PLC program buffer after entry does not re-enable the program. To re-enable the program, the **ENABLE PLC** command must be used, or PMAC must be reset (with a saved value of I5 permitting this PLC program to execute).

Example

```
CLOSE      ; Make sure other buffers are closed

DELETE GATHER  ; Make sure memory is free

OPEN PLC 7; Open buffer for entry, disabling program

CLEAR      ; Erase existing contents
```

```
IF (M11=1); Enter new version of program...

     ...

CLOSE     ; Close buffer at end of program

ENABLE PLC 7   ; Re-enable program
```

See Also

PLC Program Features

I-variable I5

On-line commands **CLOSE, DELETE GATHER, ENABLE PLC**

OPEN PROGRAM

Function: ..... Open a fixed motion program buffer for entry
Scope: ..... Global
Syntax: ..... **OPEN PROGRAM {constant}**
OPEN PROG {constant}

Where:

{constant} is an integer from 1 to 32767 representing the motion program to be opened

Remarks

This command causes PMAC to open the specified fixed (non-rotary) motion program buffer for entry or editing. Subsequent program commands valid for motion programs will be entered into this buffer. When entry of the program is finished, the **CLOSE** command should be used to prevent further lines from being put in the buffer.
No other program buffers (PLC, fixed or rotary motion) may be open when this command is sent (PMAC will report *ERR007* if I6=1 or 3). It is a good idea always to precede an **OPEN** command with a **CLOSE** command to make sure no other buffers have been left open.
No motion programs may be running in any coordinate system when this command is sent (PMAC will report *ERR001* if I6=1 or 3). As long as a fixed motion program buffer is open, no motion program may be run in any coordinate system (PMAC will report *ERR015* if I6=1 or 3).
PROGs 1000-32767 can be protected by password. If the PROG is protected by password, and the proper password has not been given, PMAC will reject this command (reporting an ERR002 if I6=1 or 3).
After any fixed motion program buffer has been opened, each coordinate system must be commanded to point to a motion program with the **B{constant}** command before it can run a motion command (otherwise PMAC will report *ERR015* if I6=1 or 3)

Example

```
CLOSE     ; Make sure other buffers are closed

DELETE GATHER DELETE TRACE    ; Make sure memory is free

OPEN PROG 255  ; Open buffer for entry, disabling program

CLEAR     ; Erase existing contents
```

```
X10 Y20 F5; Enter new version of program...

     ...

CLOSE     ; Close buffer at end of program

&1B255R   ; Point to this program and run it
```

See Also

Writing a Motion Program

On-line commands **CLEAR**, **CLOSE**, **DELETE GATHER**, **DELETE TRACE**

Program Command Specification

OPEN ROTARY

Function: ..... Open all existing rotary motion program buffers for entry
Scope: ..... Global
Syntax: ..... **OPEN ROTARY**
OPEN ROT

Remarks

This command causes PMAC to open all existing rotary motion program buffers (created with the **DEFINE ROTARY** command) for entry. Subsequent program commands valid for rotary motion programs are entered into the rotary program buffer of the coordinate system addressed at the time of that command. (Branching and looping commands should not be used in a rotary program buffer.)
No other program buffers (PLC, fixed or rotary motion) may be open when this command is sent (PMAC will report *ERR007* if I6=1 or 3). It is a good idea always to precede an **OPEN** command with a **CLOSE** command to make sure no other buffers have been left open.
The **<CTRL-U>** command performs the same Function as **OPEN ROTARY**.
The **B0** command that points the coordinate system to the rotary buffer can not be given while the rotary buffers are open, because PMAC will interpret the command as a B-axis move command.

Example

```
&2 DEFINE ROT 100   ; Create C.S. 2 rotary buffer

&1 DEFINE ROT 100   ; Create C.S. 1 rotary buffer

&1 B0 &2 B0    ; Point both C.S.s to rotary buffers

OPEN ROT  ; Open buffers for entry

&1 X10 Y10 F5  ; Write to C.S. 1's buffer

&2 X30 Y30 F10 ; Write to C.S. 2's buffer

&1R &2R   ; Start executing both buffers
```

See Also

Rotary Motion Programs (Writing a Motion Program)
On-line commands **<CTRL-L>**, **<CTRL-U>**, **CLOSE**, **DEFINE ROT**, **B{constant}**, **R**

P

Function: ..... Report motor position

Scope: ..... Motor specific

Syntax: ..... **P**

Remarks

This command causes PMAC to report the present actual position for the addressed motor to the host, scaled in counts, rounded to the nearest tenth of a count.
PMAC reports the value of the *actual position register* plus the *position bias register* plus the *compensation correction register*, and if bit 16 of Ix05 is 1 (handwheel offset mode), minus the *master postion register*.

Example

```
P     ; Request the position of the addressed motor

1995 ; PMAC responds

#1P   ; Request position of Motor 1

-0.5 ; PMAC responds

#2P#4P     ; Request positions of Motors 2 and 4

9998 ; PMAC responds with Motor 2 position first

10002      ; PMAC responds with Motor 4 position next
```

See Also

On-line commands **<CTRL-P>**, **F**, **V**

Suggested M-variable definitions Mx62, Mx64, Mx67, Mx69

Memory map registers D:$002B, D:$0813, D:$002D, D:$0046, etc.

P{constant}

Function:.....Report the current P-variable value(s).
Scope: .....Global
Syntax: .....**P{constant}[..{constant}]**

Where:
{constant} is an integer from 0 to 1023 representing the number of the P-variable;
the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

Remarks

This command causes PMAC to report the current value of the specified P-variable or range of P-variables.

Example

```
P1    ; Host asks for value

25    ; PMAC responds

P1005

3.444444444

P100..102

17.5

-373

0.0005
```

See Also

P-Variables (Computational Features)

On-line commands **I{constant}**, **M{constant}**, **Q{constant}**, **P{constant}={expression}**

P{constant}={expression}

Function: ..... Assign a value to a P-variable.
Scope: .....
..... Global
Syntax: ..... **P{constant}[..{constant}]={expression}**

Where:
{constant} is an integer from 0 to 1023 representing the number of the P-variable;
the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;
{expression} contains the value to be given to the specified P-variable(s)

Remarks

This command causes PMAC to set the specified P-variable or range of P-variables equal to the value on the right side of the equals sign.

Example

```
P1=1

P75=P32+P10

P100..199=0

P10=$2000

P832=SIN(3.14159*Q10)
```

See Also

P-Variables (Computational Features)

On-line commands **I{constant}={expression}**, **M{constant}={expression}**, **Q{constant}={expression}**, **P{constant}**

Program command **P{constant}={expression}**

PASSWORD={string}

The password does not require quote marks. If you use quote marks when you enter the password string for the first time, you must use them every time you match this password string.

Function: .....
Scope: .....
Syntax: .....

Where:
{string} is a series of non-control ASCII characters ..... (values from 32 c

Remarks

This command permits the user to enter the card's password, or once entered properly, to change it. Without a properly entered password, PMAC will not open or list the contents of any motion program numbered 1000 or greater, or of PLC programs 0-15. If asked to do so, it will return an error (ERR002 reported if I6 is set to 1 or 3).

The default password is the null password (which means no password is needed to list the programs). This is how the card is shipped from the factory, and also after a **$$$***** re-initialization command. When there is a null password, you are automatically considered to have entered the correct password on power-up/reset.
If you have entered the correct password (which is always the case for the null password), PMAC interprets the **PASSWORD={string}** command as changing the password, and you can change it to anything you want. When the password is changed, it has automatically been "matched" and the host computer has access to the protected programs.
If you have not yet entered the correct password since the latest power-up/reset, PMAC interprets the **PASSWORD={string}** command as an attempt to match the existing password. If the command matches the existing password correctly, PMAC accepts it as a valid command, and the host computer has access to the protected programs until the PMAC is reset or has its power cycled. If the command does not match the existing password correctly, PMAC returns an error (reporting ERR002 if I6=1 or 3), and the host computer does not have access to the protected programs. The host computer is free to attempt to match the existing password.
There is no way to read the current password. If the password is forgotten and access to the protected programs is required, the card must be re-initialized with the **$$$*** command, which clears all program buffers as well as the password. Then the programs must be reloaded, and a new password entered.

Example

```
{Starting from power-up/reset with a null password}
LIST PLC 1; Request listing of protected program
P1=P1+1   ; PMAC responds because there is no password
RETURN
PASSWORD=Bush  ; This sets the password to "Bush"
LIST PLC 1; Request listing of protected program
P1=P1+1   ; PMAC responds because password has been
RETURN    ; matched by changing it.
$$$  ; Reset the card
LIST PLC 1; Request listing of protected program
ERR002    ; PMAC rejects because password not entered
PASSWORD=Reagan; Attempt to enter password
ERR002    ; PMAC rejects as incorrect password
PASSWORD=BUSH  ; Attempt to enter password
ERR002    ; PMAC rejects as incorrect (wrong case)
PASSWORD=Bush  ; Attempt to enter password
     ; PMAC accepts as correct password
LIST PLC 1; Request listing of protected program
P1=P1+1   ; PMAC responds because password matched
RETURN
PASSWORD=Clinton    ; This changes password to "Clinton"
LIST PLC 1; Request listing of protected program
P1=P1+1   ; PMAC responds because password has been
RETURN    ; matched by changing it.
$$$  ; Reset the card
PASSWORD=Clinton    ; Attempt to enter password
     ; PMAC accepts as correct password
LIST PLC 1; Request listing of protected program
P1=P1+1   ; PMAC responds because password matched
RETURN
```

See Also

On-line commands **LIST**, **LIST PC**, **LIST PE**, **OPEN**

PC

Function: ..... Report Program Counter
Scope: ..... Coordinate-system specific
Syntax: ..... **PC**

Remarks

This command causes PMAC to report the motion program number and address offset of the line in that program that it will next calculate (in the addressed coordinate system). It will also report the program number and address offset of any lines it must **RETURN** to if it is inside a **GOSUB** or **CALL** jump (up to 15 deep).

The number reported after the colon is not a line number; as an addres offset, it is the number of words of memory from the top of the program. The **LIST PROGRAM** command, when used with comma delimiters, shows the program or section of the program with address offsets for each line. The **LIST PC** command can show lines of the program with address offsets from the point of calculation.

Because PMAC calculates ahead in a continuous sequence of moves, the **PC** (Program Calculation) command will in general return a program line further down in the program than **PE** will.

If the coordinate system is not pointing to any motion program, PMAC will return an error (ERR003 if I6=1 or 3). Initially the pointing must be done with the **B{constant}** command.

Example

```
PC
P1:0 ; Ready to execute at the top of PROG 1
PC
P76:22    ; Ready to execute at 22nd word of PROG 76
LIST PC
P76:22:X10Y20  ; Program line at 22nd word of PROG 76
PC
P1001:35>P3.12 ; Execution will return to PROG 3, address 12
```

See Also

On-line commands **B{constant}, LIST, LIST PC, LIST PE, LIST PROGRAM, PE**

PE

Function: ..... Report Program Execution Pointer
Scope: ..... Coordinate-system specific
Syntax: ..... **PE**

Remarks

This command causes PMAC to report the motion program number and address offset of the currently *executing* programmed move in the addressed coordinate system. This is similar to the **PC** command, which reports the program number and address offset of the next move to be calculated. Since PMAC is calculating ahead in a continuous sequence of moves, **PC** will in general report a move line several moves ahead of **PE**.

If the coordinate system is not pointing to any motion program, PMAC will return an error (ERR003 if I6=1 or 3). Initially the pointing must be done with the **B{constant}** command.

Example

```
PE
```

```
P1:2

PE

P1:5
```

See Also

On-line commands **B{constant}, LIST, LIST PC, LIST PE, PC**

PMATCH

Function: ..... Re-match Axis Positions to Motor Positions
Scope: ..... Coordinate-system specific
Syntax: ..... **PMATCH**

Remarks

This command causes PMAC to recalculate the *axis* starting positions for the coordinate system to match the current *motor* commanded positions (by inverting the axis definition statement equations and solving for the axis position). Normally this does not need to be done. However, if a *motor* move function, such as a jog move, an open-loop move, or a stop on abort or limit, was done since the last *axis* move or home, PMAC will not automatically know that the *axis* position has changed. If an axis move is then attempted without the use of the **PMATCH** command, PMAC will use the wrong *axis* starting point in its calculations.

Also, with an absulute sensor, a **PMATCH** command should be executed before the first programmed move, so the starting axis position matches the (non-zero) motor position.

If the **PMATCH** function is not performed, PMAC will use the last axis destination position as the starting point for its upcoming axis move calculations, which is not necessarily the same position as the current commanded motor positions.

The **PMATCH** function can be executed from within a motion program using **CMD"PMATCH"** with **DWELL**s both before and after. This is useful if the coordinate system setup changes in the middle of the program (e.g. new axis brought in, or following mode changed).

If more than one motor is defined to a given axis (as in a gantry system), the commanded position of the lower-numbered motor is used in the PMAC calculations.

If I14 is set to 1, the **PMATCH** function will be executed automatically every time program execution is started. Most users will want to use I14=1 so they do not have to worry about when this needs to be done.

Example

```
#1J+ ; Jog motor 1

#1J/ ; Stop jogging

PMATCH    ; Match axis position to current motor position

B200R     ; Execute program 200

OPEN PROG 10 CLEAR

...

CMD"&1#4->100C"; Bring C-axis into coordinate system

DWELL100
```

```
CMD"PMATCH"     ; Issue PMATCH so C-axis has proper start position
DWELL100
C90
...
```

See Also

Further Position Processing (Setting Up a Motor)
Axes, Coordinate Systems (Setting Up a Coordinate System)
I-variables I14, Ix06

PR

Function: ..... Report Rotary Program Remaining
Scope: ..... Coordinate-system specific
Syntax: ..... **PR**

Remarks

This command causes PMAC to report the number of program lines that have been entered in the rotary buffer for the addressed coordinate system but have not yet been executed (program remaining). This command can be useful for finding out if it is time to send new lines to the buffer. The value returned is accurate only if the rotary program buffer is open.

Example

```
B0     ; Point to rotary buffer
OPEN ROT     ; Open rotary buffer
X10F10       ; Enter 1st line
X20    ; Enter 2nd line
X30    ; Enter 3rd line
X40    ; Enter 4th line
PR     ; Ask for  "program remaining"
4      ; PMAC responds that 4 lines remain
R      ; Start running the program
PR     ; Ask for "program remaining"
2      ; PMAC responds that 2 lines remain
```

See Also

Rotary Program Buffers (Writing a Motion Program)
BREQ interrupt (Using Interrupts -- Writing a Host Communications Program)
I-variables I16, I17

Q

Function: ..... Quit Program at End of Move
Scope: ..... Coordinate-system specific
Syntax: ..... **Q**

Remarks

This causes the currently addressed coordinate system to cease execution of the program at the end of the currently executing move or the next move if that has already been calculated. The program counter is set to the next line in the program, so execution may be resumed at that point with an **R** or **S** command.
Compare this to the similar **/** command, which always stops at the end of the currently executing move..

Example

```
B10R  ; Point to beginning of PROG 10 and run
Q     ; Quit execution
R     ; Resume execution
Q     ; Quit execution again
S     ; Resume execution for a single move
```

See Also

Stopping Commands (Making Your Application Safe)

Control-Panel Port STEP/ Input (Connecting PMAC to the Machine)

JPAN Connector Pin 9

On-line commands **<CTRL-Q>**, **A**, **H**, **K**, **/**, ****

Q{constant}

Function: ..... Report Q-Variable Value
Scope: ..... Coordinate-system specific
Syntax: ..... **Q{constant}[..{constant}]**

Where:
{constant} is an integer from 0 to 1023 representing the number of the Q-variable;
the optional second **{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;

Remarks

This command causes PMAC to report back the present value of the specified Q-variable or range of Q-variables *for the addressed coordinate system*.

Example

```
Q10

35

Q255

-3.4578

Q101..103
```

```
0

98.5

-0.333333333
```

See Also

Q-Variables (Computational Features)

On-line commands **I{constant}**, **M{constant}**, **P{constant}**, **Q{constant}={expression}**

Q{constant}={expression}

Function: ..... Q-Variable Value Assignment

Scope: .....
.....Coordinate-system specific

Syntax: .....**Q{constant}[..{constant}]={expression}**

Where:
{constant} is an integer from 0 to 1023 representing the number of the Q-variable;
the optional second**{constant}** must be at least as great as the first **{constant}** -- it represents the number of the end of the range;
{expression} contains the value to be given to the specified Q-variable(s)

Remarks

This command causes PMAC to assign the value of the expression to the specified Q-variable or range of Q-variables *for the addressed coordinate system*.
If a motion program buffer is open when this command is sent to PMAC it is entered into the buffer for later execution.

Example

```
Q100=2.5
Q1..10=0
```

See Also

Q-Variables (Computational Features)

On-line commands **I{constant}={expression}**, **M{constant}={expression}**, **P{constant}={expression}**, **Q{constant}**

Program command **Q{constant}={expression}**

R

Function: ..... Run Motion Program
Scope: .....
.....Coordinate-system specific
Syntax: .....**R**

Remarks

This command causes the addressed PMAC coordinate system to start continuous execution of the motion program addressed by the coordinate system's program counter from the location of the program counter. Alternatley, it will restore operation after a '****' or '**H**' command has been issued (even if a program was or is not running). Addressing of the program counter is done initially using the **B{constant}** command.
The coordinate system must be in a proper condition in order for PMAC to accept this command. Otherwise PMAC will reject this command with an error; if I6 is 1 or 3, it will report the error number. The following conditions can cause PMAC to reject this command (also listed are the remedies):

Both limits set for a motor in coordinate system (ERR010); clear limits

Another move in progress (ERR011); stop move (e.g. with **J/**)

Open-loop motor in coordinate system (ERR012); close loop with **J/** or **A**

Unactivated motor in coordinate system (ERR013); change Ix00 to 1 or remove motor from coordinate system

No motors in the coordinate system (ERR014); put at least 1 motor in C.S.

Fixed motion program buffer open (ERR015); close buffer and point to program

No program pointed to (ERR015); point to program with **B** command

Program structured improperly (ERR016); correct program structure

Motor(s) not at same position as stopped with **/** or **** command (ERR017); move back to stopped position with **J=**

Example

```
&1B1R ; C.S.1 point to PROG 1 and run
&2B200.06   ; C.S.2 point to N6000 of PROG 200 and run
Q     ; Quit this program
R     ; Resume running from point where stopped
H     ; Do a feed hold on this program
R     ; Resume running from point where stopped
```

See Also

Control Panel Port START/ Input (Connecting PMAC to the Machine)

Running a Motion Program (Writing a Motion Program)

I-variable I6

On-line commands **<CTRL-R>**, **A**, **H**, **Q**, **S**

JPAN Connector Pin 8

R[H]{*address*}

Function: ..... Report the contents of a specified memory address[es]
Scope: .....
.....Global
Syntax: .....**R[H]{address} [,{constant}]**

Where:
{address} consists of a letter **X**, **Y**, or **L**; an option colon (:); and an integer value from 0 to 65535 (in hex, $0000 to $FFFF); specifying the starting PMAC memory or I/O address to be read;
{constant} (optional) is an integer from 1 to 16 specifying the number of consecutive memory addresses to be read; if this is not specified, PMAC assumes a value of 1

Remarks

This command causes PMAC to report the contents of the specified memory word address or range of addresses to the host (it is essentially a PEEK command). The command can specify either short (24-bit) word(s) in the PMAC X-memory, short (24-bit) word(s) in the PMAC Y-memory, or long (48-bit) words covering both X and Y memory (X-word more significant). This choice is controlled by the use of the **X**, **Y**, or **L** address prefix in the command, respectively.
If the letter **H** is used after the **R** in the command, PMAC reports back the register contents in unsigned hexadecimal form, with 6 digits for a short word, and 12 digits for a long word. If the letter **H** is not used, PMAC reports the register contents in signed decimal form.

Example

```
RHX:49152    ; Request contents of X-register 49152 ($C000) in hex
8F4017       ; PMAC responds in unsigned hex (note no '$')
RHX:$C000    ; Request contents of X-reg $C000 (49152) in hex
8F4017       ; PMAC responds in unsigned hex
RX:49152     ; Request contents of same register in decimal
-7389161     ; PMAC responds in signed decimal
RX:$C000     ; Request contents of same register in decimal
-7389161     ; PMAC responds in signed decimal
RX0    ; Request contents of servo cycle counter in decimal
2953211      ; PMAC responds in signed decimal
RL$0028      ; Request contents of  #1 cmd. pos. reg in decimal
3072000      ; PMAC responds (=1000 counts)
RHY1824,12   ; Request set-up words of the conversion table
00C000 00C004 00C008 00C00C 00C010 00C014 00C018
00C01C 400723 0000295 000000 000000 ; PMAC responds in hex
```

See Also

PMAC Memory Mapping (Computational Features)

On-line command **W{address}**

Memory and I/O Map Description.

S

Function: .....Execute One Move ("Step") of Motion Program
Scope: .....
.....Coordinate-system specific
Syntax: .....**S**

Remarks

This command causes the addressed PMAC coordinate system to start single-step execution of the motion program addressed by the coordinate system's program counter from the location of the program counter. Addressing of the program counter is done initially using the **B{constant}** command.

At the default I53 value of zero, a Step command causes program execution through the next move or **DWELL** command in the program, even if this takes multiple program lines.
When I53 is set to 1, a Step command causes program execution of only a single program line, even if there is no move or **DWELL** command on that line. If there is more than one **DWELL** or **DELAY** command on a program line, a single Step command will only execute one of the **DWELL** or **DELAY** commands.
Regardless of the setting of I53, if program execution on a Step command encounters a **BLOCKSTART** statement in the program, execution will continue until a **BLOCKSTOP** statement is encountered.
If the coordinate system is already executing a motion program when this command is sent, the command puts the program in single-step mode, so execution will stop at the end of the latest calculated move. In this case, its action is the equivalent of the **Q** command.
The coordinate system must be in a proper condition in order for PMAC to accept this command. Otherwise PMAC will reject this command with an error; if I6 is 1 or 3, it will report the error number. The same conditions that cause PMAC to reject an **R** command will cause it to reject an **S** command; refer to those conditions under the **R** command specification.

Example

```
&3B20S      ; C.S.3 point to beginning of PROG 20 and step
P1     ; Ask for value of P1
1      ; PMAC responds
S      ; Do next step in program
P1     ; Ask for value of P1 again
-3472563     ; PMAC responds --probable problem
```

See Also

Control Panel Port STEP/ Input (Connecting PMAC to the Machine)

Running a Motion Program (Writing a Motion Program)

I-variable I6, I53

On-line commands **<CTRL-S>**, **A**, **H**, **Q**, **R**, **/**, ****

Program commands **{axis}{data}**, **BLOCKSTART**, **BLOCKSTOP**, **DWELL**, **DELAY**

SAVE

Function: ..... Copy setup parameters to non-volatile memory.
Scope: ..... Global
Syntax: ..... **SAVE**

Remarks

This command causes PMAC to copy setup information from active memory to non-volatile memory, so this information can be retained through power-down or reset. Its exact operation depends on the type of PMAC used.
For the "standard" PMACs with battery-backed RAM, only the basic setup information is stored with the SAVE command: I-variables, encoder conversion table entries, and VME/DPRAM address entries. This information is copied back from flash to active memory during a normal power-up/reset operation. User programs, buffers, and definitions are simply held in RAM by the battery backup; there is no need to save these.
For the "option" PMACs with flash-backed RAM, all user setup information, including programs, buffers, and definitions, is copied to flash memory with the **SAVE** command. This information is copied back from flash to active memory during a normal power-up/reset operation. *This means that anything changed in the PMAC active memory that is not saved to flash memory will be lost in a power-on/reset cycle.*
The **SAVE** operation can be inhibited by changing jumper E50 from its default state. If the **SAVE** command is issued with jumper E50 not in its default state, PMAC will report an error. The retrieval of information from non-volatile memory on power-up/reset can be inhibited by changing jumper E51 from its default state.

PMAC does not provide the acknowledging handshake character to the **SAVE** command until it has finished the saving operation, a significant fraction of a second later on PMACs with battery backup and about 5 to 10 seconds on the PMAC with flash backup. The host program should be prepared to wait much longer for this character than is necessary on most commands. For this reason, it is usually not a good idea to include the **SAVE** command as part of a "dump" download of a large file.
During execution of the **SAVE** command, PMAC will execute no other background tasks, including user PLCs and automatic safety checks, such as following error and overtravel limits. Particularly on boards with the flash backup where saving takes many seconds, you must make sure the system is not depending on these tasks for safety when the **SAVE** command is issued.

Example

```
I130=60000  ; Set Motor 1 proportional gain
SAVE  ; Save to non-volatile memory
I130=80000  ; Set new value
$$$   ; Reset card
I130  ; Request value of I130
60000 ; PMAC responds with saved value
```

See Also

On-line commands **$$$**, **$$$*****

Jumpers E50 and E51.

SIZE

Function: ..... Report the amount of unsed buffer memory in PMAC.

Scope: .....
..... Global

Syntax: ..... **SIZE**

Remarks

This command causes PMAC to report to the host the amount of unused long words of memory available for buffers. If no program buffer (motion, PLC or rotary buffer) is open, this value is reported as a positive number. If a buffer is currently open, the value is reported as a negative number.

Example

```
DEFINE GATHER  ; Reserve all remaining memory for gathering
SIZE ; Ask for amount of open memory
0    ; PMAC reports none available
DELETE GATHER  ; Free up memory from gathering buffer
SIZE ; Ask for amount of open memory
41301     ; PMAC reports number of words available
OPEN PROG 10   ; Open a motion program buffer
SIZE ; Ask for amount of open memory
-41302    ; The negative sign shows a buffer is open
```

See Also

I-Variable I18

On-line commands **DELETE GATHER**, **DELETE TRACE**

TYPE

Function: ..... Report type of PMAC
Scope: ..... Global
Syntax: ..... **TYPE**

Remarks

This command causes PMAC to return a string reporting the configuration of the card. It will report the configuration as a text string in the format:
{PMAC type},{Bus type},{Backup type},{Servo Type},{Ladder type},{Clock Multiplier}

where

{PMAC type}:

PMAC1	First generation PMAC (including PMAC"1.5")
PMAC2	Second generation PMAC
PMACUL	Ultra-lite (MACRO only PMAC2)

{Bus type}:

ISA	IBM-PC ISA bus
VME	VME bus
STD	STD bus
ISA/VME	PMAC1 firmware can support both busses
MINI	Special ISA bus for Mini PMAC (1)

{Backup type}:

BATTERY	Battery-backed RAM
FLASH	AMD-style flash-backed RAM
I-FLASH	Intel-style flash-backed RAM

{Servo type}:

PID	Standard PID servo algorithm
ESA	Option 6 Extended servo algorithm

{Ladder type}

{blank}	no ladder-logic diagram support
LDs	Ladder-logic diagram support

{Clock multiplier}:

CLK Xn where n is the multiplication of crystal frequency to CPU frequency

Examples

TYPE

```
PMAC1, ISA/VME, BATTERY, PID, CLK X1
```

TYPE

```
PMAC2, ISA, FLASH, ESA, CLK X3
```

TYPE

PMACUL, VME, FLASH, PID, LDs, CLK X2

See Also

On-line commands **VERSION, DATE**

UNDEFINE

Function:.....Erase Coordinate System Definition

Scope: .....
.....Coordinate-system specific

Syntax: .....**UNDEFINE**

UNDEF

Remarks

This command causes PMAC to erase all of the axis definition statements in the addressed coordinate system. It does not affect the axis definition statements in any other coordinate systems. It can be useful to use before making new axis definitions.

To erase the axis definition statement of a single motor only, use the **#{constant}->0** command; to erase all the axis definition statements in every coordinate system, use the **UNDEFINE ALL** command.

Example

```
&1    ; Address C.S.1
#1->  ; Ask for axis definition of Motor 1
10000X    ; PMAC responds
#2->  ; Ask for axis definition of Motor 2
10000Y    ; PMAC responds
UNDEFINE  ; Erase axis definitions
&2    ; Address C.S.2
#1->10000X; Redefine Motor 1 as X-axis in C.S.2
#2->10000Y; Redefine Motor 2 as Y-axis in C.S.2
```

See Also

Axes, Coordinate Systems (Setting Up a Coordinate System)

On-line commands **#{constant}->**, **#{constant}->0**, **UNDEFINE ALL**

UNDEFINE ALL

Function: ..... Erase coordinate definitions in all coordinate systems
Scope: ..... Global
Syntax: ..... **UNDEFINE ALL**
UNDEF ALL

Remarks

This command causes all of the axis definition statments in all coordinate systems to be cleared. It is a useful way of starting over on a reload of the PMAC coordinate system definitions.

Example

```
&1#1->     ; Request axis definition of Motor 1 in C.S. 1
1000X      ; PMAC responds
&2#5->     ; Request axis definition of Motor 5 in C.S. 2
1000X      ; PMAC responds
UNDEFINE ALL   ; Erase all axis definitions
&1#1->     ; Request axis definition of Motor 1 in C.S. 1
0    ; PMAC responds that there is no definition
&2#5->     ; Request axis definition of Motor 5 in C.S. 2
0    ; PMAC responds that there is no definition
```

See Also

Axes, Coordinate Systems (Setting Up a Coordinate System)

On-line commands **#{constant}->0**, **#{constant}->**, **UNDEFINE**.

V



The velocity values reported here are obtained by subtracting positions of consecutive servo cycles. As such, they can be very noisy. For purposes of display, it is probably better to use averaged velocity values held in registers Y:$082A, Y:$08EA, etc., accessed with M-variables

Function: .....
Scope: .....
Syntax: .....

Remarks

This command causes PMAC to report the present actual motor velocity to the host, scaled in counts/servo cycle, rounded to the nearest tenth. It is reporting the contents of the motor *actual velocity register* (divided by [Ix09*32]). To convert this reported value to counts/msec, multiply by 8,388,608*(Ix60+1) and divide by I10. It can be further converted to engineering units with additional scaling constants.

Example

```
V    ; Request actual velocity of addressed motor
21.9 ; PMAC responds with 21.9 cts/cycle
```

```
     ; (*8,388,608/3,713,707 = 49.5 cts/msec)
#6V  ; Request velocity of Motor 6
-4.2 ; PMAC responds
#5V#2V    ; Request velocities of Motors 5 and 2
0    ; PMAC responds with Motor 5 first
7.6  ; PMAC responds with Motor 2 second
```

See Also

I-variables **I10, Ix09, Ix60**

On-line commands **<CTRL-V>**, **F**, **P**

Memory map registers **X:$0033, X:$006F, etc., X:$082A, X:$08EA,** etc.

Suggested M-variable definitions **Mx66**

VERSION

Function: ..... Report PROM firmware version number

Scope: ..... Global

Syntax: ..... **VERSION**

VER

Remarks

This command causes PMAC to report the firmware version it is using.
When a flash-memory PMAC is in "bootstrap mode" (powering up with E51 ON), PMAC will report the version of the bootstrap firmware, not the operational firmware. Otherwise, it will report the operational firmware version. To change from bootstrap mode to normal operational mode, use the **<CTRL-R>** command.

Example

```
VERSION    ; Ask PMAC for firmware version
1.12D      ; PMAC responds
```

See Also

Resetting PMAC (Talking to PMAC)

On-line command **DATE**, **TYPE**

W{address}

Function: ..... Write value(s) to a specified address(es).
Scope: ..... Global
Syntax: ..... **W{address},{value} [,{value}...]**

Where:

{address} consists of a letter **X**, **Y**, or **L**; an option colon (:); and an integer value from 0 to 65535 (in hex, $0000 to $FFFF); specifying the starting PMAC memory or I/O address to be read;

{constant} is an integer, specified in decimal or hexadecimal, specifying the value to be written to the specified address;

further **{constants}** specify integer values to be written into subsequent consecutive higher addresses

Remarks

This command causes PMAC to write the specified **{constant}** value to the specified memory word address, or if a series of **{constant}** values is specified, to write them to consecutive memory locations starting at the specified address (it is essentially a memory POKE command). The command can specify either short (24-bit) word(s) in the PMAC X-memory, short (24-bit) word(s) in the PMAC Y-memory, or long (48-bit) words covering both X and Y memory (X-word more significant). This choice is controlled by the use of the **X**, **Y**, or **L** address prefix in the command, respectively.

Example

```
WY:$C002,4194304    ; This should put 5V on DAC2
     ; (provided I200=0 so servo does not overwrite)
WY$720,$00C000,$00C004,$00C008,$00C00C
     ; This writes the first four entries to the
     ; encoder conversion table
```

See Also

On-line command **R[H]{address}**

Memory and I/O Map.

Z

Function: ..... Make commanded axis positions zero.
Scope: ..... Coordinate-system specific
Syntax: ..... **Z**

Remarks

This command causes PMAC to re-label the current commanded axis positions for all axes in the coordinate system as zero. It does not cause any movement; it merely re-names the current position.
This command is simply a short way of executing **{axis}=0** for all axes in the coordinate system. **PSET X0 Y0** (etc.) is the equivalent motion program command.
This does not set the *motor* position registers to zero; it changes motor position bias registers to reflect the new offset between motor zero positions and axis zero positions. However, the motor reported positions will reflect the new bias, and report positions of zero (+/- the following error)

Example

```
<CTRL-P>  ; Ask for reported motor positions

2001 5002 3000 0 0 0 0 0 ; PMAC reports positions

Z    ; Zero axis positions

<CTRL-P>  ; Ask for reported motor positions again

1 2 -1 0 0 0 0 0    ; PMAC responds
```

See Also

On-line commands **{axis}={constant}**

Memory map registers D:$0813, D:$08D3, etc.

Suggested M-variable definitions Mx64.



PMAC PROGRAM COMMAND SPECIFICATION

Buffered Program Commands

PMAC has an extensive set of buffered program commands. These commands are not actually executed when they are sent to PMAC; they are merely placed in the open program buffer for later execution. PMAC has two types of program buffers: motion programs and PLC programs. Some buffered commands are valid for either type of buffer; others are valid only for one or the other. Motion programs can either be in "fixed" buffers (PROGRAM) or rotary buffers (ROTARY). Because the memory for rotary motion program buffers can be re-used for later parts of the program, certain types of branching, jumping, and looping commands are not permitted in these buffers. In the specification for each command in this section, the "Type" of the command – which type(s) of buffers it can be used for – is listed.

{axis}{data}[{axis}{data}...]

Function: Position-Only Move Specification

Type: Motion program (PROG and ROT)

Syntax: `{axis}{data}[{axis}{data}...]`

Where:

`{axis}` is the character specifying which axis (X, Y, Z, A, B, C, U, V, W);

`{data}` is a constant (no parentheses) or an expression (in parentheses) representing the end position or distance;

`[{axis}{data}...]` is the optional specification of simultaneous movement for more axes.

Remarks

This is the basic PMAC move specification statement. It consists of one or more groupings of an axis label and its associated value. The value for an axis is scaled (units determined by the axis definition statement); it represents a position if the axis is in absolute (ABS) mode, or a distance if the axis is in incremental (INC) mode. The order in which the axes are specified does not matter.
This command tells the axes *where* to move; it does not tell them *how* to move there. Other program commands and parameters define how; these must be set up ahead of time.
The type of motion a given motion command causes is dependent on the mode of motion and the state of the system at the beginning of the move.

Examples

```
X1000

X(P1+P2)

Y(Q100+500) Z35 C(P100)

X1000 Y1000

A(P1) B(P2) C(P3)

X(Q1*SIN(Q2/Q3)) U500
```

See Also

Axis Definition Statements (Setting Up a Coordinate System)

Motion program commands **LINEAR**, **CIRCLEn**, **RAPID**, **SPLINE1**, **PVT**

Motion program commands **TA**, **TS**, **TM**, **F**, **ABS**, **INC**

I-variables Ix87, Ix88, Ix89, Ix90.

{axis}{data}:{data} [{axis}{data}:{data}...]

Function: Position and Velocity Move Specification
Type: Motion program (PROG and ROT)
Syntax: **{axis}{data}:{data} [{axis}{data}:{data}...]**

Where:

{axis} is the character specifying which axis (X, Y, Z, A, B, C, U, V, W);

{data} is a constant (no parentheses) or an expression (in parentheses) representing the end position or distance;

:{data} represents the ending velocity

[{axis}{data}:{data}...] is the optional specification of simultaneous movement for more axes.

Remarks

In the case of PVT (position, velocity, time) motion mode, both the ending position and velocity are specified for each segment of each axis. The command consists of one or more groupings of axis labels with two data items separated by a colon character.
The first data item for each axis is the scaled ending position or distance (depending on whether the axis is in absolute (**ABS**) or incremental (**INC**) mode; position scaling is determined by the axis definition statement), and the second data item (after the colon) is the ending velocity.
The velocity units are the scaled position units as established by the axis definition statements divided by the time units as set by Ix90 for Coordinate System x. The velocity here is a signed quantity, not just a magnitude See the Examples in the PVT mode description of the Writing a Motion Program section.
The time for the segment is the argument for the most recently executed **PVT** or **TA** command, rounded to the nearest millisecond.
In PVT mode, if no velocity is given for the segment, PMAC assumes an ending velocity of zero for the segment.

Examples

```
X1000:50

Y500:-32 Z737.2:68.93

A(P1+P2):(P3) B(SIN(Q1)):0
```

See Also

PVT Mode Moves (Writing a Motion Program)

Axis Definition Statements (Setting Up a Coordinate System)

I-variables Ix87, Ix90

Motion program commands PVT, TA

{axis}{data}^{data}[{axis}{data}^{data}...]

Function:.....Move Until Trigger
Type:.....Motion program
Syntax: ..... **{axis}{data}^{data}[{axis}{data}^{data}...]**

Where:.....

{axis} is the character specifying which axis (X, Y, Z, A, B, C, U, V, W);

the first **{data}** is a constant (no parentheses) or expression (in parentheses) representing the end position or distance in the absence of a trigger;

the second **{data}** (after the ^ arrow) is a constant (no parentheses) or expression (in parentheses) representing the distance from the trigger position;

[{axis}{data}^{data}...] is the optional specification of simultaneous movement for more axes

Remarks

In the **RAPID** move mode, this move specification permits a move-until-trigger function. The first part of the move description for an axis -- before the ^ sign -- specifies where to move in the absence of a trigger. It is a position if the axis is in absolute mode; it is a distance if the axis is in incremental mode. In both cases, the units are the scaled axis user units. If no trigger is found before this destination is reached, the move is a standard **RAPID** move.
The second part of the move description for an axis -- after the ^ sign -- specifies the distance from the trigger position to end the post-trigger move if a trigger is found. The distance is expressed in the scaled axis user units.
Each motor assigned to an axis specified in the command executes a separate move-until-trigger. All the assigned motors will start together, but each can have its own trigger condition. If a common trigger is required, the trigger signal must be wired into all motor interfaces. Each motor can finish at a separate time; the next line in the program will not start to execute until all motors have finished their moves. No blending into the next move is possible.
The trigger for a motor can be either a hardware input trigger if bit 17 of Ix03 is 0, or the motor warning following error status bit if bit 17 of Ix03 is 1 (bit 16 of Ix03 should also be set to 1 in this case). If a hardware input trigger is used, Encoder/Flag I-variables 2 and 3 (e.g. I902 and I903) for the flag channel specified by Ix25 determine which edge(s) of which flag(s) cause the trigger. If the warning following error bit is used for "torque-limited" triggering, then Ix12 sets the size of the warning following error.
The speed of the move, both before the trigger and after, is set by Ix22 if I50=0 or by Ix16 if I50=1. The acceleration is set by Ix19 to Ix21.
On the same line, some axes may be specified for normal untriggered **RAPID** moves that will execute simultaneously
If the move ends for a motor without a trigger being found, the "trigger move" status bit (bit 7 of the second motor status word returned on a **?** command) is left set after the end of the move. If the trigger has been found, this bit is cleared to 0 at the end of the move.

Example

```
X1000^0

X10^-0.01 Y5.43^0.05

A(P1)^(P2) B10^200 C(P3)^0 X10
```

See Also

Move-Until-Trigger (Writing a Motion Program)
Torque-Limited Triggering (Setting Up a Motor)
RAPID-mode moves (Writing a Motion Program)

{axis}{data} [{axis}{data}...] {vector}{data} {vector}{data}...]

Function: ..... Circular Arc Move Specification
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **{axis}{data} [{axis}{data}...] {vector}{data} [{vector}{data}...]**

Where:.....

{axis} is a character specifying which axis (X, Y, Z, A, B, C, U, V, W);

{data} is a constant (no parentheses) or an expression (in parentheses) representing the end position or distance;

[{axis}{data}...] is the optional specification of simultaneous movement for more axes;

{vector} is a character (I, J, or K) specifying a vector component (parallel to the X, Y, or Z axis, respectively) to the center of the arc; or the character R specifying the magnitude of the vector;

{data} specifies the magnitude of the vector component;

[{vector}{data}...] is the optional specification of more vector components.

Remarks

For a blended circular mode move, both the move endpoint and the vector to the arc center are specified. The endpoint is specified just as in a **LINEAR** mode move, either by position (referenced to the coordinate system origin), or distance (referenced to the starting position).

The center of the arc for a circular move must also be specified in the move command. This is usually done by defining the vector to the center. This vector can either be referenced to the starting point of the move (incremental radial vector mode -- the default, or if an **INC(R)** command has been given), or it can be referenced to the coordinate system origin (absolute radial vector mode -- if an **ABS(R)** command has been given).

Alternatively, just the magnitude of the vector to the center can be specified with **R{data}** on the command line. If this is the case, PMAC will calculate the location of the center itself. If the value specified by **{data}** is positive, PMAC will compute the short arc path to the destination (<= 180°); if it is negative, PMAC will compute the long arc path (>= 180°). It is not possible to specify a full circle in one command with the R vector specifier.

The plane for the circular arc must have been defined by the **NORMAL** command (the default -- **NORMAL K-1** -- defines the XY plane). This command can only define planes in XYZ-space, which means that only the X, Y, and Z axes can be used for circular interpolation. Other axes specified in the same move command will be interpolated linearly to finish in the same time.

The direction of the arc to the destination point -- clockwise or counterclockwise -- is controlled by whether the card is in **CIRCLE1** (clockwise) or **CIRCLE2** (counterclockwise) mode. The sense of clockwise in the plane is determined by the direction of the **NORMAL** vector to the plane.

If the destination point is a different distance from the center point than is the starting point, the radius is changed smoothly through the course of the move, creating a spiral. This is useful in compensating for any round off errors in the specifications. However, if the distance from either the starting point or the destination point to the center point is zero, and error condition will be generated and the program will stop.

If the vector from the starting point to the center point does not lie in the circular interpolation plane, the projection of that vector into the plane is used. If the destination point does not lie in the same circular interpolation plane as the starting point, a helical move is done to the destination point.

If the destination point (or its projection into the circular interpolation plane containing the starting point) is the same as the starting point, a full 360° arc is made in the specified direction (provided the IJK vector specification is

used). In this case, only the vector needs to be specified in the move command, because for any axis whose destination is not specified, the destination point is automatically taken to be the same as the starting point.
If no vector, and no radial magnitude is specified in the move command, a linear move will be done to the destination point, even if the program is in circular mode
PMAC performs arc moves by segmenting the arc and performing the best cubic fit on each segment. I-variable I13 determines the time for each segment. I13 *must* be set greater than zero to put PMAC into this segmentation mode in order for arc moves to be done. If I13 is set to zero, circular arc moves will be done in linear fashion.

Examples

```
X5000 Y3000 I1000 J1000

X(P101) Z(P102) I(P201) K(P202)

X10 I5

X10 Y20 C5 I5 J5

Y5 Z3 R2

J10  ; Specifies a full circle of 10 unit radius
```

See Also

Circular Moves (Writing a Motion Program)

I-variables I13, Ix87, Ix88, Ix89, Ix90

Program commands **NORMAL**, **ABS**, **INC**, **CIRCLE1**, **CIRCLE2**, **TA**, **TS**, **TM**, **F**

A{data}

Function: ..... A-Axis Move
Type: .....
..... Motion program (PROG or ROT)
Syntax: ..... **A{data}**

Where:.....

{data} is a floating-point constant or expression representing the position or distance in user units for the U-axis

Remarks

This command causes a move of the A-axis. (See **{axis}{data}** descriptions, above.)

Examples

```
A10

A(P23)

A25 B10 Z35

A(20*SIN(Q5))
```

See Also

Program commands **{axis}{data}**, **B**, **C**, **U**, **V**, **W**, **X**, **Y**, **Z**, **CALL**, **READ**

ABS

Function: ..... Absolute Move Mode

Type: .....
..... Motion program (PROG and ROT)

Syntax: ..... **ABS [({axis}[,{axis}...])]**

Where:.....

{axis} is a character (X,Y,Z,A,B,C,U,V,W) representing the axis to be specified, or the character R to specify radial vector mode

Remarks

The **ABS** command without arguments causes all subsequent positions in motion commands for all axes in the coordinate system running the motion program to be treated as absolute positions. This is known as absolute mode, and it is the power-on default condition. An **ABS** statement with arguments causes the specified axes in the coordinate system running the program to be in absolute mode, and all others stay the way they were before.
If **R** is specified as one of the 'axes', the I, J, and K terms of the circular move radius vector specification will be specified in absolute form (i.e. as a vector from the origin, not from the move start point). An **ABS** command without any arguments does not affect this vector specification. The default radial vector specification is incremental.

If no motion program buffer is open when this command is sent to PMAC, it will be executed as an on-line coordinate system command.

Examples

```
ABS(X,Y)

ABS

ABS(V)

ABS(R)
```

See Also

Circular Moves (Writing a Motion Program)

On-line commands **ABS**, **INC**.

Program commands **{axis}{data}**, **{axis}{data}{vector}{data}**, **INC**.

ADDRESS

Function:.....Motor/Coordinate System Modal Addressing
Type:.....
..... PLC programs 1 to 31 **only**
Syntax: .....**ADDRESS [#{constant}][&{constant}]**
ADR [#{constant}][&{constant}]

Where:.....

{constant} is an integer constant from 1 to 8 representing the motor (#) number or the coordinate system (&) number to be addressed

Remarks

This statement, when executed, sets the motor and/or coordinate system that will be addressed by this particular PLC program when it **COMMAND**s motor- or coordinate-system-specific commands with no addressing in those commands. The addressed coordinate system also controls which set of Q-variables is accessed, even for ATAN2 functions, which automatically use Q0.
This command does not affect host addressing, the addressing of other PLC programs, or the selection of the control panel inputs. The addressing stays in effect until another **ADDRESS** statement supersedes it. Default addressing at power-on/reset is #1 and &1.
In motion programs, there is no modal addressing for **COMMAND** statements; each **COMMAND** statement must contain the motor or coordinate-system specifier within its quotation marks. A motion program automatically operates on the Q-variables of the coordinate system executing the program.

Examples

```
ADDRESS &4

ADR #2

ADDRESS &2#2

ADR#1      ; Modally address Motor 1

CMD"J+"    ; This will start Motor 1 jogging

CMD"#2J+"  ; This will start Motor 2 jogging

CMD"J/"    ; This will stop Motor 1
```

See Also

Addressing Modes (Talking To PMAC)
Q-Variables (Program Computational Features)
Program commands **COMMAND**, **Q{constant}={expression}**

ADIS{constant}

Function: ..... Absolute displacement of X, Y, and Z axes

Type: .....
..... Motion program (PROG and ROT)

Syntax: ..... **ADIS{constant}**

Where:.....

{constant} is an integer constant representing the number of the first of three consecutive Q-variables to be used in the displacement vector

Remarks

This command loads the currently selected (with **TSEL**) transformation matrix for the coordinate system with offset values contained in the three Q-variables starting with the specified one. This has the effect of renaming the current commanded X, Y, and Z axis positions (from the latest programmed move) to the values of these variables (X=Q{data}, Y=Q({data}+1), Z=Q({data}+2)).
This command does not cause any movement of any axes; it simply renames the present positions.
This command is almost equivalent to a **PSET X(Q{data}) Y(Q({data}+1)) Z(Q({data}+2))** command, except that **ADIS** does not force a stop between moves, as **PSET** does.

Examples

```
Q20=7.5

Q21=12.5

Q22=25
```

```
ADIS 20   ; This makes the current X position 7.5, Y 12.5, Z25
```

See Also

Axis Matrix Transformations (Writing a Motion Program)

On-line command **DEFINE TBUF**

Program commands **TSEL**, **AROT**, **IDIS**, **IROT**, **TINIT**

AND ({condition})

Function: ..... Conditional AND

Type: ..... PLC program only

Syntax: ..... **AND ({condition})**

Where:

{condition} is a simple or compound condition

Remarks

This statement forms part of an extended compound **IF** or **WHILE** condition to be evaluated in a PLC program. It must immediately follow an **IF**, **WHILE**, **AND**, or **OR** statement This **AND** is a Boolean operator logically combining the full conditions on its line and the program line immediately above. It takes lower precedence than **AND** or **OR** operators within a compound condition on a single line (the parentheses cause those to be executed first), but it takes higher precedence than an **OR** operator that starts a line.

In motion programs, there can be compound conditions within one program line, but not across multiple program lines, so this statement is not permitted in motion programs.

This logical **AND** command, which acts on *condition* should not be confused with the bit-by-bit **&** (ampersand) operator that acts on *values*.

Examples

```
IF (M11=1); This branch will start a motion program running

AND (M12=1)     ; on a cycle where inputs M11 and M12 are 1 and

AND (M21=0)    ; M21 is still zero.  Notice that M21 is immedi-
ately

CMD"R"    ; set to one so the run command will not be given

M21=1     ; again in the next cycle.

ENDIF
```

See Also

Writing a PLC Program

Conditions (Program Computational Features)

Program Commands **IF**, **WHILE**, **OR**

AROT{constant}

Function: ..... Absolute rotation/scaling of X, Y, and Z axes
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **AROT{constant}**

Where: .....

{constant} is an integer representing the number of the first of nine consecutive Q-variables to be used in the rotation/scaling matrix

Remarks



When using this command to scale the coordinate system, do not use the radius center specification for circle commands. The radius is not scaled. Use the ***I, J, K*** *vector specification instead.*

This command loads the currently selected (with **TSEL**) transformation matrix for the coordinate system with rotation/scaling values contained in the nine Q-variables starting with the specified one. This has the effect of renaming the current commanded X, Y, and Z axis positions (from the latest programmed move) by multiplying the XYZ vector by this matrix.

The rotation and scaling is done relative to the "base" XYZ coordinate system, defined by the axis definition statements. The math performed is:

$[Xrot\ Yrot\ Zrot]^T = [Rot\ Matrix]\ [Xbase\ Ybase\ Zbase]^T$

This command does not cause any movement of any axes; it simply renames the present positions.

Examples

Create a 3x3 matrix to rotate the XY plane by 30 degrees about the origin

```
Q40=COS(30)     Q41=SIN(30)     Q42=0

Q43=-SIN(30)    Q44=COS(30)     Q45=0

Q46=0      Q47=0           Q48=1

AROT 40    ; Implement the change
```

Create a 3x3 matrix to scale the XYZ space by a factor of 3

```
Q50=3      Q51=0           Q52=0

Q53=0      Q54=3           Q55=0

Q56=0      Q57=0           Q58=3

AROT 50    ; Implement the change
```

See Also

Axis Matrix Transformations (Writing a Motion Program)
On-line command **DEFINE TBUF**
Program commands **TSEL**, **ADIS**, **IDIS**, **IROT**, **TINIT**

B{data}

Function: ..... B-Axis Move

Type:.....Motion program (PROG and ROT)

Syntax: .....**B{data}**

Where:.....

{data} is a floating-point constant or expression representing the position or distance in user units for the U-axis

Remarks

This command causes a move of the B-axis. (See **{axis}{data}** description, above.)

See Also

Program commands **{axis}{data}, A, C, U, V, W, X, Y, Z, CALL, READ**

BLOCKSTART

Function:..... Mark Start of Stepping Block

Type:.....Motion program (PROG and ROT)

Syntax: .....**BLOCKSTART**

BSTART

Remarks

This statement allows multiple moves to be done on a single 'step' command. Execution on a 'step' command will proceed until the next **BLOCKSTOP** statement in the program (without **BLOCKSTART**, only a single servo command is executed on a 'step' command). Also, if Ix92=1 (move blending disabled), all moves between **BLOCKSTART** and **BLOCKSTOP** will be blended together. This does not affect how a program is executed from a 'run' command if Ix92=0.

This structure is particularly useful for executing a single sequence of PVT mode moves, because the individual segments do not end at zero velocity, making normal stepping very difficult.

Examples

For the program segment:

```
BLOCKSTART

INC

X10:100

X20:100

X20:100

X10:0

BLOCKSTOP
```

All four move segments will be executed on a single S command.

See Also

I-variable Ix92

On-line commands **<CONTROL-S>**, **R**, **S**.

Program commands **BLOCKSTOP**, **STOP**.

BLOCKSTOP

Function: ..... Mark End of Stepping Block
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **BLOCKSTOP**
BSTOP

Remarks

This statement marks the end of the block of statements, begun with a **BLOCKSTART**, to be done on a single 'step' command, or to be blended together even if Ix92=1 (move blending disabled). This does not affect how a program is executed from a 'run' command if Ix92=1.

Examples

See example under **BLOCKSTART**, above.

See Also

I-variable Ix92

On-line commands **<CONTROL-S>**, **R**, **S**.

Program commands **BLOCKSTART**, **STOP**.

C{data}

Function:.....C-Axis Move

Type:.....
.....Motion program (PROG and ROT)

Syntax: .....**C{data}**

Where: .....
{data} is a floating-point constant or expression representing the position or distance in user units for the U-axis

Remarks

This command causes a move of the C-axis. (See **{axis}{data}** description, above.)

See Also

Program commands {axis}{data}, A, B, U, V, W, X, Y, Z, CALL, READ

CALL

Function:.....Jump to Subprogram With Return

Type:.....
.....Motion program (PROG and ROT)

Syntax: .....**CALL{data} [{letter}{data}...]**

Where:.....

the first **{data}** is a floating-point constant or expression from 1.00000 to 32767.99999, with the integer part representing the motion program number to be called, and the fractional part representing the line label (**N** or **O**) within the program to be called (the line label number is equal to the fractional part multiplied by 100,000; every motion program has an implicit **N0** at the top);

{letter} is any letter of the English alphabet, except N or O, representing the variable into which the value following it will be placed (Q101 to Q126 for A to Z respectively);

following **{data}** is a floating-point constant or expression representing the value to be put into the variable

Remarks

This command allows the program to execute a subprogram and then return execution to the next line in the program. A subprogram is entered into PMAC the same as a program, and is labeled as PROGn (so one program can call another as a subprogram). The number n of the PROG heading is the one to which the value after **CALL** refers: **CALL7** would execute **PROG7** and return. Commanding execution of a non-existent subprogram will cause program execution to stop in an error condition.

The value immediately following **CALL** can take fractional values. If there is no fractional component, the called program starts at the beginning. If there is a fractional component, the called program is entered at a line label specified by the fractional component (if this label does not exist, PMAC will generate an error and stop execution). PMAC works with five fractional digits to specify the line label; if you use fewer, it automatically fills out the rest with zeros. For instance, **CALL 35.1** is interpreted as **CALL 35.10000**, which causes a jump to label **N10000** of program 35. **CALL 47.123** causes a jump to label **N12300** of program 47.

If letters and data (e.g. **X1000**) follow the **CALL{data}**, these can be arguments to be passed to the subprogram. If arguments are to be passed, the first line executed in the subroutine should be a **READ** statement. This statement will take the values associated with the specified letters and place them in the appropriate Q-variable. For instance, the data following **A** is placed in variable Q101 for the coordinate system executing the program; that following **B** is placed in Q102; and so on, to the data following **Z** being placed in Q126. The subprogram can then use these variables. If the subprogram calls another subprogram with arguments, the same Q-variables are used. Refer to **READ** for more details.

If there is no **READ** statement in the subroutine, or if not all the letter values in the **CALL** line are "read" (the **READ** statement stops as soon as it sees a letter in the calling line that is not in its list of letters to read), the remaining letter commands are executed upon return from the subroutine according to their normal function. For example, **G01 X10 Y10** is equivalent to a **CALL 1000.01 X10 Y10**. To implement the normal function for **G01** (linear move mode), there would be the following subroutine in PROG 1000:

```
N1000 LINEAR RETURN
```

Upon the return, **X10 Y10** would be executed as a move according to the move mode in force, which is **LINEAR**. If the specified program and line label do not exist, the **CALL** command is ignored, and the program continues as if it were not there.

Examples

```
CALL500   ; to Prog 500 at the top (N0)

CALL500.1 ; to Prog 500 label N10000

CALL500.12; to Prog 500 label N12000

CALL500.123    ; to Prog 500 label N12300

CALL500.1234   ; to Prog 500 label N12340

CALL500.12345  ; to Prog 500 label N12345

CALL700 D10 E20; to Prog 700 passing D and E
```

See Also

On-line command **B{constant}**

Program commands **GOTO**, **GOSUB**, **READ**, **RETURN**, **G{data}**, **M{data}**, **T{data}**, **D{data}**, **N{constant}**, **O{constant}**, **PRELUDE**

CC0

Function:.....Turn Off Cutter Radius Compensation

Type:.....Motion program (PROG and ROT)

Syntax: **CC0**

Remarks

This turns off the cutter radius compensation mode, reducing it gradually through the next move. This is equivalent to the **G40** command of the machine-tool standard RS-274 language.

Examples

```
CCR0.5     ; 1/2 unit cutter radius

CC1  ; Cutter compensation on to the left

X10 Y10   ; Compensation introduced during this move

X10 Y20

X20 Y20

X20 Y10

X10 Y10

CC0  ; Cutter compensation off

X0 Y0     ; Compensation eliminated during this move

OPEN PROG 1000 ; G-Code Subprogram

   ...

N40000 CC0 RETURN   ; To implement G40 directly in PMAC
```

See Also

Cutter (Tool) Radius Compensation

Program commands **CC1**, **CC2**, **CCR{data}**.

CC1

Function:.....Turn On Cutter Radius Compensation Left

Type:.....Motion program (PROG and ROT)

Syntax: **CC1**

Remarks

This turns on the cutter radius compensation mode, introducing the compensation gradually through the next move. The cutter is offset to the left of the programmed tool path, looking in the direction of cutter movement. The plane of the compensation is determined by the **NORMAL** command. This is equivalent to the **G41** command of the machine-tool standard RS-274 language.

Examples

```
CCR0.25   ; 1/4 unit cutter radius

CC1  ; Cutter compensation on to the left

X10 Y10   ; Compensation introduced during this move

X10 Y20

X20 Y20

X20 Y10

X10 Y10

CC0  ; Cutter compensation off

X0 Y0     ; Compensation eliminated during this move

OPEN PROG 1000 ; G-Code Subprogram

   ...

N41000 CC1 RETURN   ; To implement G41 directly in PMAC
```

See Also

Cutter (Tool) Radius Compensation

Program commands **CC2**, **CC0**, **CCR{data}**, **NORMAL**

CC2

Function: ..... Turn On Cutter Radius Compensation Right
Type: ..... Motion program (PROG and ROT)
Syntax: **CC2**

Remarks

This turns on the cutter radius compensation mode, introducing the compensation gradually through the next move. The cutter is offset to the right of the programmed tool path, looking in the direction of cutter movement. The plane of the compensation is determined by the **NORMAL** command. This is equivalent to the **G42** command of the machine-tool standard RS-274 language.

Examples

```
CCR1.5    ; 1-1/2 unit cutter radius

CC2  ; Cutter compensation on to the right

X10 Y10   ; Compensation introduced during this move

X10 Y20
```

```
X20 Y20

X20 Y10

X10 Y10

CC0  ; Cutter compensation off

X0 Y0     ; Compensation eliminated during this move

OPEN PROG 1000 ; G-Code Subprogram

   ...

N42000 CC2 RETURN   ; To implement G42 directly in PMAC
```

See Also

Tool Radius Compensation

Program commands **CC1**, **CC0**, **CCR**, **NORMAL**

CCR{data}

Function: ..... Set Cutter Compensation Radius
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **CCR{data}**

Remarks

This sets the radius for cutter compensation (when on) in user units. This function is often part of the **D** tool data used in the machine-tool standard RS-274 (G) code.

See Also

Program commands CC1, CC2, CC0, D{data}, NORMAL

CIRCLE1

Function: ..... Set Blended Clockwise Circular Move Mode
Type: .....
..... Motion program (PROG and ROT)
Syntax: **CIRCLE1**
CIR1

Remarks

By the most recent **NORMAL** command, which has also defined the sense of clockwise and counterclockwise in the plane.

The program is taken out of this circular move mode by another move mode command: the other **CIRCLE** mode, **LINEAR**, **PVT**, **RAPID** etc. Any circular move command must have either an **R** or an **IJK** vector specification; otherwise it will This command puts the program into clockwise circular move mode. The plane for the circular interpolation is defined be performed as a linear move even when in **CIRCLE** mode.

Note: PMAC must be in move segmentation mode (I13>0) in order to perform circular interpolation. If I13=0 (no move segmentation), the moves will be linearly interpolated.

Examples

```
LINEAR    ; Linear interpolation mode

X10Y10 F2 ; Linear move

CIRCLE1   ; Clockwise circular interpolation mode

X20 Y20 I10    ; Arc of 10-unit radius

X25 Y15 J-5    ; Arc of 5-unit radius

LINEAR    ; Go back to linear mode

X25 Y5    ; Linear move
```

See Also

Circular Moves (Writing a Motion Program)
I-variable I13
Program commands **NORMAL**, **CIRCLE2**, **LINEAR**, **PVT**, **RAPID**, **SPLINE1**, **{axis}{data}{vector}{data}**

CIRCLE2

Function: ..... Set Blended Counterclockwise Circular Move Mode

Type: ..... Motion program (PROG and ROT)

Syntax: **CIRCLE2**

CIR2

Remarks

The **CIRCLE2** command puts the program into counterclockwise circular move mode. The plane for the circular interpolation is defined by the most recent **NORMAL** command, which has also defined the sense of clockwise and counterclockwise in the plane.
The program is taken out of this circular move mode by another move mode command: the other **CIRCLE** mode, **LINEAR**, **PVT**, **RAPID** etc. Any circular move command must have either an **R** or an **IJK** vector specification; otherwise it will be performed as a linear move even when in **CIRCLE** mode.
Note: PMAC must be in move segmentation mode (I13>0) in order to perform circular interpolation. If I13=0 (no move segmentation), the moves will be linearly interpolated.

Examples

```
LINEAR    ; Linear interpolation mode

X10Y0 F2  ; Linear move

CIRCLE2   ; Counterclockwise circular interpolation

     ; mode

X20 Y10 J10    ; Arc of 10-unit radius
```

```
X15 Y15 I-5    ; Arc of 5-unit radius

CIRCLE1   ; Clockwise circle mode

X5 Y25 J10; Arc move of 10-unit radius
```

See Also

Circular Moves (Writing a Motion Program)

I-variable I13

Program commands **NORMAL**, **CIRCLE1**, **LINEAR**, **PVT**, **RAPID**, **SPLINE1**, **{axis}{data}{vector}{data}**

COMMAND"{command}"

Function: ..... Program Command Issuance
Type: .....
..... Motion program (PROG and ROT); PLC program
Syntax: ..... **COMMAND "{command}"**
. . . . . **CMD "{command}"**

Remarks

This statement causes the program to issue a command to PMAC as if it came from the host (except for addressing modes). If there is a motor- or coordinate-system-specifier (**#n** or **&n**) within the quoted string, a motor- or coordinate-system-specific command will be directed to that motor or coordinate system. If there is no specifier, a motor- or coordinate-system-specific command will be directed to the first motor or coordinate system. Any specifier within a **COMMAND** statement is not modal; it does not affect the host addressing specifications or the modal addressing of any program, including its own.

If I62=0, PMAC automatically issues a carriage-return (**<CR>**) character at the end of any data response to the command. If I62=1, PMAC does not issue a **<CR>** character at the end of the data response; a **SEND^M** must be used to issue a **<CR>** in this case.

Each PLC program has its own addressing mode for both motors and coordinate systems, independent of each other and independent of the host addressing modes. These are controlled by the PLC program **ADDRESS** command. This modal addressing affects commands issued from within a PLC program that do not have motor or coordinate-system specifiers. At power-up/reset, all PLC programs are addressing Motor 1 and C.S.1.

There is no modal **ADDRESS** command in motion programs. Any motor-specific or coordinate-system-specific command issued from within a motion program without a specifier is automatically addressed to Motor 1 or C.S.1, respectively.

Commands issued from within a program are placed in the command queue, to be parsed and acted upon at the appropriate time by the PMAC command interpreter, which operates in background, between other background tasks. If issued from a motion program, the command will not be interpreted before the next move or dwell command in the motion program is calculated. If issued from a PLC program, the command will not be interpreted before the end of the current scan of the PLC. This delay can make the action appear to execute out of sequence.

Because of the queuing of commands and the fact that command interpretation is a lower priority than command issuing, it is possible to overflow the queue. If there is no room for a new command, program execution is temporarily halted until the new command can be placed on the queue.

Also, commands that generate a response to the host (including errors if I6 is not equal to 2) potentially can fill up the response queue if there is no host or the host is not prepared to read the responses. This will temporarily halt program execution until the response queue is emptied. In standalone applications, it is a good idea to set I1 to 1, disabling the serial handshake, so any responses can be sent out the serial port (the default response port) at any time, even if there is no host to receive it.

In a PLC program, it is a good idea to have at least one of the conditions that caused the command issuance to occur set false immediately. This will prevent the same command from being issued again on succeeding scans of the PLC, overflowing the command and/or response queues. Typically in a motion program, the time between moves prevents this overflow unless there are many commands and the moves take a very short time.

PMAC will not issue an acknowledging character (**<ACK>** or **<LF>**) to a valid command issued from a program. It will issue a **<BELL>** character for an invalid command issued from a program unless I6 is set to 2. It is a good idea to have I6 not set to 2 in early development so you will know when PMAC has rejected such a command. Setting I6 to 2 in the actual application can prevent program hang-up from a full response queue, or from disturbing the normal host communications protocol.

Many otherwise valid commands will be rejected when issued from a motion program. For instance, you cannot jog any motor in the coordinate system executing the program, because all these motors are considered to be running in the program, even if the program is not requesting a move of the motors at that time.

When issuing commands from a program, be sure to include all the necessary syntax (motor and/or coordinate system specifiers) in the command statement or use the **ADDRESS** command. For example, use **CMD"#4HM"** and **CMD"&1A"** instead of **CMD"HM"** and **CMD"A"**. Otherwise, motor and coordinate system commands will be sent to the most recently addressed motor and coordinate system that may not always be as you intended.

Examples

```
COMMAND"#1J+"

CMD"#4HM"

CMD"&1B5R"

CMD"P1"

47.5

ADDRESS#3

COMMAND"J-"

IF(M40=1 AND M41=1)

     CMD"&4R"

     M41=0

ENDIF
```

See Also

Addressing Modes, On-Line Commands (Talking To PMAC)

I-variables I1, I3, I6.

Program commands **ADDRESS**, **COMMAND^{letter}**

Writing A PLC Program

COMMAND^{letter}

Function: ..... Program Control-Character Command Issuance
Type: ..... Motion program (PROG or ROT), PLC program
Syntax: ..... **COMMAND^{letter}**
..... **CMD^{letter}**

Where:

{letter} is a letter character from A to Z (upper or lowercase) representing the corresponding control character

Remarks



*Do not put the up-arrow character and the letter in quotes (do not use **COMMAND"^A"**) or PMAC will attempt to issue a command with the two non-control characters **^** and **A** for this example, instead of the control character.*

This statement causes the motion program to issue a control-character command as if it came from the host. All control-character commands are global, so there are no addressing concerns.
Commands issued from within a program are placed in the command queue, to be parsed and acted upon at the appropriate time by the PMAC command interpreter, which operates in background, between other background tasks. If issued from a motion program, the command will not be interpreted before the next move or dwell command in the motion program is calculated.
If issued from a PLC program, the command will not be interpreted before the end of the current scan of the PLC. This delay can make the action appear to execute out of sequence.
Because of the queuing of commands and the fact that command interpretation is a lower priority than command issuing, it is possible to overflow the queue. If there is no room for a new command, program execution is temporarily halted until the new command can be placed on the queue.
Also, commands that generate a response to the host (including errors if I6 is not equal to 2) potentially can fill up the response queue if there is no host or the host is not prepared to read the responses. This will temporarily halt program execution until the response queue is emptied. In standalone applications, it is a good idea to set I1 to 1, disabling the serial handshake, so any responses can be sent out the serial port (the default response port) at any time, even if there is no host to receive it.
In a PLC program, it is a good idea to have at least one of the conditions that caused the command issuance to occur set false immediately. This will prevent the same command from being issued again on succeeding scans of the PLC, overflowing the command and/or response queues. Typically in a motion program, the time between moves prevents this overflow unless there are many commands and the moves take a very short time.
PMAC will not issue an acknowledging character (**<ACK>** or **<LF>**) to a valid command issued from a program. It will issue a **<BELL>** character for an invalid command issued from a program unless I6 is set to 2. It is a good idea to have I6 not set to 2 in early development so you will know when PMAC has rejected such a command. Setting I6 to 2 in the actual application can prevent program hang-up from a full response queue, or from disturbing the normal host communications protocol

Examples

CMD^D would disable all PLC programs (equivalent to issuing a **<CONTROL-D>** from the host).

CMD^K would kill (disable) all motors on PMAC

CMD^A would stop all programs and moves on PMAC, also closing any loops that were open.

See Also

I-variables I1, I6
On-line commands **<CONTROL-A>** to **<CONTROL-Z>**

Program command **COMMAND"{command}"**

D{data}

Function:.....Tool Data (D-Code)
Type:.....
.....Motion program
Syntax: .....**D{data}**

Where:.....

{data} is a floating-point constant or expression in the range 0.000 to 999.999, specifying the program number and the line label to jump to.

Remarks

PMAC interprets this statement as a **CALL 10n3.({data'}*1000)** command, where n is the hundreds digit of **{data},** and **{data'}** is the value of **{data}** without the hundred's digit (modulo 100 in mathematical terms). That is, this statement causes a jump (with return) to motion program 10n3, and the specified line label. (Programs 10n3 are usually used to implement the tool data operations as the system designer sees fit.) The value of **{data'}** can be from 0.0 to 99.999, corresponding to line labels **N0** to **N99999**.

This structure permits the implementation of customizable D-Code routines for machine-tool style applications by the writing of subroutines in motion programs 10n3. Arguments can be passed to these subroutines by following the D-Code with one or more sets of **{letter}{data}**, as in **CALL** and **READ** statements.

Most users will have D-codes only in the range 0-99, which permits the use of PROG 1003 only, and allows **{data'}** to equal **{data}** for direct specification of the line label.

Example

D01 jumps to **N1000** of PROG 1003

D12 jumps to **N12000** of PROG 1003

D115 jumps to **N15000** of PROG 1013

See Also

Program commands CALL{data}, G{data}, M{data}, T{data}, RETURN

DELAY{data}

Function:.....Delay for Specified Time
Type:.....
.....Motion program
Syntax: .....**DELAY{data}**

DLY{data}

Where:.....

{data} is a floating-point constant or expression, specifying the delay time in milliseconds

Remarks

This command causes PMAC to keep the command positions of all axes in the coordinate system constant (no movement) for the time specified in **{data}**.
There are three differences between **DELAY** and **DWELL**. First, if **DELAY** comes after a blended move, the **TA** deceleration time from the move occurs within the **DELAY** time, not before it. Second, the actual time for **DELAY** does varies with a changing time base (current % value, from whatever source), whereas **DWELL** always uses the fixed time base (%100). Third, PMAC precomputes upcoming moves (and the lines preceding them) during a **DELAY**, but it does not do so during a **DWELL**.
A **DELAY** command is equivalent to a zero-distance move of the time specified in milliseconds. As for a move, if the specified **DELAY** time is less than the acceleration time currently in force (**TA** or 2***TS**), the delay will be for the acceleration time, not the specified **DELAY** time.

Examples

```
DELAY750

DELAY(Q1+100)
```

See Also

Time-Base Control (Synchronizing PMAC to External Events)

I-variables I10, Ix87, Ix88

On-line command **%{constant}**

Program commands **DWELL**, **TA**, **TS**

DISABLE PLC {constant}[,{constant}...]

Function:.....Disable PLC Program(s)
Type:.....
.....Motion program (PROG or ROT), PLC program

Syntax: .....**DISABLE PLC {constant}[,{constant}...]**

.....
.....DISABLE PLC {constant}[..{constant}]

.....
.....DIS PLC {constant}[,{constant}...]

```
.....DIS PLC {constant}[..{constant}]
```

Remarks

This command disables the operation of the specified PLC program[s]. The programs are specified by number, and can be used singly, in a list separated by commas, or in a continuous range.
Disabling a PLC cannot stop the PLC in the middle of a scan; it prevents it from starting the next scan.

Examples

```
DISABLE PLC 1

DISABLE PLC 4,5

DISABLE PLC 7..20

DIS PLC 3,8,11

DIS PLC 0..31
```

See Also

I-variable I5

On-line commands **ENABLE PLC**, **DISABLE PLC**, **ENABLE PLCC**, **DISABLE PLCC**, **<CONTROL-D>**

Program command **ENABLE PLC**, **DISABLE PLCC**, **ENABLE PLCC**

DISABLE PLCC {constant}[,{constant}...]

Function: ..... Disable Compiled PLC Program(s)

Type: ..... Motion program (PROG or ROT), PLC program (uncompiled or compiled), except for PLC 0 and PLCC 0

Syntax: .....**DISABLE PLCC {constant}[,{constant}...]**

```
.....DISABLE PLCC {constant}[..{constant}]

.....DIS PLCC {constant}[,{constant}...]

.....DIS PLCC {constant}[..{constant}]
```

Where:.....

{constant} is an integer from 0 to 31 representing the compiled PLC number

Remarks

This command disables the operation of the specified compiled PLC (PLCC) program[s]. The programs are specified by number, and can be used singly, in a list separated by commas, or in a continuous range.
This command should not be used in a foreground PLC -- either uncompiled PLC 0 or compiled PLCC 0 -- as its operation cannot be guaranteed in these programs.
I-variable I5 is a separate master control of PLC program operation. Think of the two bits of I5 as two master circuit breakers for a house, and the individual PLC and PLCC enable/disable bits as separate light switches within the house. Both the master breaker and the switch must be on for the PLC to operate. The breakers and the switches can be operated independently without affecting the setting of the others.

Examples

```
DISABLE PLCC 1

DISABLE PLCC 4,5

DISABLE PLCC 7..20

DIS PLCC 3,8,11

DIS PLC 0..31
```

See Also

I-variable I5

On-line commands **ENABLE PLC**, **DISABLE PLC**, **ENABLE PLCC**, **DISABLE PLCC**, **<CONTROL-D>**

Program command **ENABLE PLC**, **DISABLE PLC**, **ENABLE PLCC**

DISPLAY [{constant}] "{message}"

Function:.....Display Text to Display Port
Type:.....
.....Motion program (PROG and ROT), PLC program
Syntax: .....**DISPLAY [{constant}] "{message}"**

.....
.....**DISP [{constant}] "{message}"**

Where:.....

{constant} is an integer value between 0 and 79 specifying the starting character number on the display; if no value is specified, 0 is used

{message} is the ASCII text string to be displayed

Remarks

This command causes PMAC to send the string contained in **{message}** to the display port (J1 connector) for the liquid-crystal or vacuum-fluorescent display (Accessory 12 or equivalent).
The optional constant value specifies the starting point for the string on the display; it has a range of 0 to 79, where 0 is upper left, 39 is upper right, 40 is lower left, and 79 is lower right.

Examples

```
DISPLAY 10"Hello World"

DISP "VALUE OF P1 IS"

DISP 15, 8.3, P1
```

See Also

Display Port (Connecting PMAC to the Machine);

Accessory 12 (Basic Specifications)

Program commands **DISPLAY {variable}**, **SEND"{message}"**

DISPLAY ... {variable}

Function: ..... Formatted Display of Variable Value
Type: ..... Motion program (PROG and ROT), PLC program
Syntax: .....`DISPLAY {constant}, {constant}.{constant}, {variable}`
.....
.....`DISP {constant}, {constant}.{constant}, {variable}`

Where:.....

the first **`{constant}`** is an integer from 0 to 79 representing the starting location (character number) on the display;

the second **`{constant}`** is an integer from 2 to 16 representing the total number of characters to be used to display the value (integer digits, decimal point, and fractional digits);

the third **`{constant}`** is an integer from 0 to 9 (and at least two less than the second **`{constant}`**) representing the number of fractional digits to be displayed;

`{variable}` is the name of the variable to be displayed.

Remarks

This command causes PMAC to send a formatted string containing the value of the specified variable to the display port. The value of any I, P, Q, or M variable may be displayed with this command.
The first constant value specifies the starting point for the string on the display; it has a range of 0 to 79, where 0 is upper left, 39 is upper right, 40 is lower left, and 79 is lower right. The second constant specifies the number of characters to be used in displaying the value; it has a range of 2 to 16. The third constant specifies the number of places to the right of the decimal point; it has a range of 0 to 9, and must be at least 2 less than the number of characters. The last thing specified in the statement is the name of the variable -- I, P, Q, or M.

Examples

```
DISPLAY 0, 8.0, P50

DISPLAY 24, 2.0, M1

DISPLAY 40, 12.4, Q100
```

See Also

Display Port (Connecting PMAC to the Machine);
Accessory 12 (Basic Specifications)
Program commands **`DISPLAY"{message}"`**, **`COMMAND"{command}"`**

DWELL

Function:.....Dwell for Specified Time
Type:.....
.....Motion program (PROG and ROT)
Syntax: .....**DWELL{data}**
. . . . . .
. . . . . .**DWE{data}**

Where:.....

{data} is a non-negative floating point constant or expression representing the dwell time in milliseconds

Remarks

This command causes the card to keep the commanded positions of all axes in the coordinate system constant for the time specified in **{data}**.

There are three differences between **DWELL** and the similar **DELAY** command. First, if the previous servo command was a blended move, there will be a **TA** time deceleration to a stop before the dwell time starts. Second, **DWELL** is not sensitive to a varying time base -- it always operates in 'real time' (as defined by I10). Third, PMAC does not pre-compute upcoming moves (and the program lines before them during the **DWELL**; it waits until after it is done to start further calculations, which it performs in the time specified by I11 or I12.

Use of any **DWELL** command, even a **DWELL0**, while in external time base will cause a loss of synchronicity with the master signal.

Examples

```
DWELL250

DWELL(P1+P2)

DWE0
```

See Also

Dwell and Delay (Writing a Motion Program)

I-variables I10, I11, I12.

Program command **DELAY**

ELSE

Function:..... Start False Condition Branch

Type:.....
..... Motion program (PROG only), PLC program

Syntax: .....**ELSE**
. . . . . (Motion or PLC Program)

. . . . . .
. . . . . **ELSE {action}**
. . . . . (Motion Program only)

Remarks

This statement must be matched with an **IF** statement (**ELSE** requires a preceding **IF**, but **IF** does not require a following **ELSE**). It follows the statements executed upon a true **IF** condition. It is followed by the statements to be executed upon a false **IF** condition.

With nested **IF** branches, you must be careful to match the **ELSE** statements to the proper **IF** statement. In a motion program, it is possible to have a single-line **IF** statement (**IF({condition}) {action}**). An **ELSE** statement on the next program line is automatically matched to this **IF** statement, even if you wanted to match a previous IF statement. You must put a non-**ELSE** statement in between to make the next **ELSE** statement match a previous **IF** statement.

ELSE lines can take two forms (only the first of which is valid in a PLC program):

With no statement following on that line, all subsequent statements down to the next **ENDIF** statement will be executed provided that the preceding IF condition is false.

```
ELSE
      {statement}
      [{statement}
      ...]
ENDIF
```

With a statement or statements following on that line, the single statement will be executed provided that the preceding **IF** condition is false. No **ENDIF** statement should be used in this case

ELSE {statement} [{statement}...]

(This single-line **ELSE** branch form is valid only in motion programs. If you try this in a PLC program, PMAC will put the statement(s) on the next program line and expect an **ENDIF** to close the branch. Your logic will not be as you expect it.)

Examples

The first example has multi-line true and false branches. It could be used in either a motion program or a PLC program

```
IF (M11=1)
   P1=17
   P2=13
ELSE
   P1=13
   P2=17
ENDIF
```

This second example has a multi-line true branch, and a single-line false branch. This structure could only be used in a motion program.

```
IF (M11=0)
   X(P1)
   DWELL 1000
ELSE DWELL 500
```

This example has a single-line true branch, and a multi-line false branch. This structure could only be used in a motion program.

```
IF (SIN(P1)>0.5) Y(1000*SIN(P1))
ELSE
   P1=P1+5
   Y(1100*SIN(P1))
ENDIF
```

This example has single-line true and false branches. This structure could only be used in a motion program.

```
IF (P1 !< 5) X10
ELSE X-10
```

See Also

Program commands **IF**, **ENDIF**.

ENABLE PLC

Function:.....Enable PLC Buffer(s)

Type:.....
.....Motion program (PROG and ROT), PLC program

Syntax: .....**ENABLE PLC {constant}[,{constant}...]**

.....
.....**ENABLE PLC {constant}[..{constant}]**

.....
.....**ENA PLC {constant}[,{constant}...]**

.....
.....**ENA PLC {constant}[..{constant}]**

Remarks

This command enables the operation of the specified PLC buffer[s], provided I5 is set properly to allow their operation.

Examples

```
ENABLE PLC 0

ENABLE PLC 1,2,5

ENABLE PLC 1..16

ENA PLC 7
```

See Also

I-variable I5

On-line commands **ENABLE PLC**, **DISABLE PLC**, **<CONTROL-D>**

Program command **DISABLE PLC**

ENABLE PLCC

Function:.....Enable Compiled PLC Program(s)

Type:.....
..... Motion program (PROG and ROT), PLC program (uncompiled and compiled)

Syntax: .....**ENABLE PLCC {constant}[,{constant}...]**

.....
.....**ENABLE PLCC {constant}[..{constant}]**

.....
.....**ENA PLCC {constant}[,{constant}...]**

.....
.....**ENA PLCC {constant}[..{constant}]**

Where:.....

{constant} is an integer from 0 to 31 representing the compiled PLC number

Remarks

This command enables the operation of the specified compiled PLC (PLCC) buffer[s], provided I5 is set properly to allow their operation. The programs are specified by number, and can be used singly, in a list separated by commas, or in a continuous range.

I-variable I5 is a separate master control of PLC program operation. Think of the two bits of I5 as two master circuit breakers for a house, and the individual PLC and PLCC enable/disable bits as separate light switches within the house. Both the master breaker and the switch must be on for the PLC to operate. The breakers and the switches can be operated independently without affecting the setting of the others.

Examples

```
ENABLE PLCC 0

ENABLE PLCC 1,2,5

ENABLE PLCC 1..16

ENA PLCC 7
```

See Also

I-variable I5

On-line commands **ENABLE PLC**, **DISABLE PLC**, **ENABLE PLCC**, **DISABLE PLCC**, **<CONTROL-D>**

Program command **ENABLE PLC**, **DISABLE PLC**, **DISABLE PLCC**

ENDIF

Function:.....Mark End of Conditional Block

Type:.....
..... Motion program (PROG only), PLC program

Syntax: .....**ENDIF**

.....
.....**ENDI**

Remarks

This statement marks the end of a conditional block of statements begun by an **IF** statement. It can close out the "true" branch, following the **IF** statement, in which case there is no "false" branch, or it can close out the "false" branch, following the **ELSE** statement.

When nesting conditions, it is important to match this **ENDIF** with the proper **IF** or **ELSE** statement. In a PLC program, every **IF** or **IF/ELSE** pair must take an **ENDIF**, so the **ENDIF** always matches the most recent **IF** statement that does not already have a matching **ENDIF**. In a motion program an **IF** or **ELSE** statement with action on the same line does not require an **ENDIF**, so the **ENDIF** would be matched with a previous **IF** statement.

Examples

```
IF (P1>0)
```

```
   X1000

ENDIF

IF (P5=7)

   X1000

ELSE

   X2000

ENDIF
```

See Also

Logical Structures (Writing a Motion Program)
Conditional Statements (Writing a PLC Program)
Program commands **IF**, **ELSE**

ENDWHILE

Function: ..... Mark End of Conditional Loop

Type: ..... Motion program (PROG only), PLC program

Syntax: ..... **ENDWHILE**

```
. . . . . .
. . . . . ENDW
```

Remarks

This statement marks the end of a conditional loop of statements begun by a **WHILE** statement. **WHILE** loops can be nested, so an **ENDWHILE** statement matches the most recent **WHILE** statement not already matched by a previous **ENDWHILE** statement.

In a motion program a **WHILE** statement with an action on the same line does not require a matching **ENDWHILE**.

In the execution of a PLC program, when an **ENDWHILE** statement is encountered, that scan of the PLC is ended, and PMAC goes onto other tasks (communications, other PLCs). The next scan of this PLC will start at the matching **WHILE** statement.

In the execution of a motion program, if PMAC finds two jumps backward (toward the top) in the program while looking for the next move command, PMAC will pause execution of the program and not try to blend the moves together. It will go on to other tasks and resume execution of the motion program on a later scan. Two statements can cause such a jump back: **ENDWHILE** and **GOTO** (**RETURN** does not count).

The pertinent result is that PMAC will not blend moves when it hits two **ENDWHILE** statements (or the same **ENDWHILE** twice) between execution of move commands.

Examples

```
WHILE (Q10<10)

     Q10=Q10+1

ENDWHILE
```

See Also

Program commands **WHILE**, **ENDIF**

F{data}

Function: ..... Set Move Feedrate (Velocity)

Type: .....
..... Motion program (PROG and ROT)

Syntax: ..... **F{data}**

Where:.....

{data} is a positive floating-point constant or expression representing the vector velocity in user length units per user time units

Remarks

This statement sets the commanded velocity for upcoming **LINEAR** and **CIRCLE** mode blended moves. It will be ignored in other types of moves (**SPLINE**, **PVT**, and **RAPID**). It overrides any previous **TM** or **F** statement, and is overridden by any following **TM** or **F** statement.

The units of velocity specified in an **F** command are scaled position units (as set by the axis definition statements) per time unit (defined by "Feedrate Time Unit" I-variable for the coordinate system: Ix90).

The velocity specified here is the vector velocity of all of the feedrate axes of the coordinate system. That is, the move time is calculated as the vector distance of the feedrate axes (square root of the sum of the squares of the individual axes), divided by the feedrate value specified here. The minimum effective feedrate value will provide a move time of 2^{23} msec. The maximum effective feedrate value will provide a move time of 1 msec. Any non-feedrate axes commanded to move on the same move-command line will move at the speed necessary to finish in this same amount of time.

If the vector distance of a feedrate-specified move is so short that the computed move time (vector distance divided by feedrate) would be less than the acceleration time currently in force (**TA** or 2***TS**), the move will take the full acceleration time instead, and the axes will move more slowly than specified by the **F** command

Axes are designated as feedrate axes with the **FRAX** command. If no **FRAX** command is used, the default feedrate axes are the X, Y, and Z axes. Any axis involved in circular interpolation is automatically a feedrate axis, regardless of whether it was specified in the latest **FRAX** command. In multi-axis systems, feedrate specification of moves is really only useful for systems with Cartesian geometry, for which these moves give a constant velocity in the plane or in 3D space, regardless of movement direction.

If only non-feedrate axes are commanded to move in a feedrate-specified move, PMAC will compute the vector distance, and so the move time, as zero, and will attempt to do the move in the acceleration time (TA or 2*TS), possibly limited by the maximum velocity and/or acceleration parameters for the motor(s). This will probably be much faster than intended.

Examples

```
F100

F31.25

F(Q10)

F(SIN(P8*P9))
```

See Also

I-variables Ix87, Ix88, Ix89, Ix90

On-line commands **#{constant}->{axis definition}**, **FRAX**

Program commands **FRAX**, **LINEAR**, **CIRCLE**, **TM**, **TA**, **TS**

FRAX

Function: ..... Specify Feedrate Axes
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **FRAX [({axis}[,{axis}...])]**

Where:.....

{axis} is a character (X, Y, Z, A, B, C, U, V, W) specifying which axis is to be used in the vector feedrate calculations.

Remarks

This command specifies which axes are to be involved in the vector-feedrate (velocity) calculations for upcoming feedrate-specified (**F**) moves. PMAC calculates the time for these moves as the vector distance (square root of the sum of the squares of the axis distances) of all the feedrate axes divided by the feedrate. Any non-feedrate axes commanded on the same line will complete in the same amount of time, moving at whatever speed is necessary to cover the distance in that time.

Vector feedrate has obvious geometrical meaning only in a Cartesian system, for which it results in constant tool speed regardless of direction, but it is possible to specify for non-Cartesian systems, and for more than three axes.

If only non-feedrate axes are commanded to move in a feedrate-specified move, PMAC will compute the vector distance, and so the move time, as zero, and will attempt to do the move in the acceleration time (TA or 2*TS), possibly limited by the maximum velocity and/or acceleration parameters for the motor(s). This will probably be much faster than intended.

The **FRAX** command without arguments causes all axes in the coordinate system to be feedrate axes in subsequent move commands. The **FRAX** command with arguments causes the specified axes to be feedrate axes, and all axes not specified to be non-feedrate axes, in subsequent move commands.

If no motion program buffer is open when this command is sent to PMAC, it will be executed as an on-line coordinate system command.

Examples

For a three-axis Cartesian system scaled in millimeters:

```
FRAX(X,Y)

INC

X30 Y40 Z10 F100
```

Vector distance is SQRT($30^2 + 40^2$) = 50 mm. At a speed of 100 mm/sec, move time (unblended) is 0.5 sec. X-axis speed is 30/0.5 = 60 mm/sec; Y-axis speed is 40/0.5 = 80 mm/sec; Z-axis speed is 10/0.5 = 20 mm/sec.

```
Z20
```

Vector distance is SQRT(0^2 +0^2) = 0 mm. Move time (unblended) is 0.0 sec, so Z-axis speed is limited only by acceleration parameters.

```
FRAX(X,Y,Z)

INC

X-30 Y-40 Z120 F65
```

Vector distance is SQRT(-30^2 + -40^2 +120^2) = 130 mm. Move time is 130/65 = 2.0 sec. X-axis speed is 30/2.0 = 15 mm/sec; Y-axis speed is 40/2.0 = 20 mm/sec; Z-axis speed is 120/2.0 = 60 mm/sec.

See Also

I-variables Ix87, Ix88, Ix89, Ix90

On-line command **FRAX**

Program commands **F**, **LINEAR**, **CIRCLE**, **{axis}{data}**.

G{data}

Function: Preparatory Code (G-Code)
Type: Motion program
Syntax: **G{data}**

Where:

{data} is a floating-point constant or expression in the range 0.000 to 999.999, specifying the program number and the line label to jump to

Remarks

PMAC interprets this statement as a **CALL 10n0.({data'}*1000)** command, where n is the hundreds digit of **{data}**, and **{data'}** is the value of **{data}** without the hundred's digit (modulo 100 in mathematical terms). That is, this statement causes a jump (with return) to motion program 10n0, and the specified line label. (Programs 10n0 are usually used to implement the preparatory codes as the system designer sees fit.) The value of **{data'}** can be from 0.0 to 99.999, corresponding to line labels **N0** to **N99999**.

This structure permits the implementation of customizable G-Code routines for machine-tool style applications by the writing of subroutines in motion programs 10n0. Arguments can be passed to these subroutines by following the G-Code with one or more sets of **{letter}{data}**, as in **CALL** and **READ** statements.

Most users will have G-codes only in the range 0-99, which permits the use of PROG 1000 only, and allows **{data'}** to equal **{data}** for direct specification of the line label.

Example

```
G01 jumps to N1000 of  PROG 1000

G12 jumps to N12000 of PROG 1000

G115 jumps to N15000 of PROG 1010
```

See Also

Program commands **CALL{data}, D{data}, M{data}, T{data}, RETURN**

GOSUB

Function: ..... Unconditional Jump With Return
Type: ..... Motion program (PROG only)
Syntax: ..... **GOSUB{data}**

Where:.....

{data} is a constant or expression representing the line label to jump to;

{letter} (optional) is any letter character except N or O;

Remarks

This command causes the motion program execution to jump to the line label (**N** or **O**) of the same motion program specified in **{data}**, with a jump back to the commands immediately following the **GOSUB** upon encountering the next **RETURN** command.
If **{data}** is a constant, the path to the subroutine will have been linked before program run time, so the jump is very quick. If **{data}** is a variable expression, it must be evaluated at run time, and the appropriate label then searched for. The search starts downward in the program to the end, then continues (if necessary) from the top of the program down.
A variable **GOSUB** command permits the equivalent structure to the CASE statement found in many high-level languages.
If the specified line label is not found, the **GOSUB** command will be ignored, and the program will continue as if the command had not occurred.
The **CALL** command is similar, except that it can jump to another motion program.

Examples

GOSUB300 jumps to **N300** of this program, to jump back on **RETURN**

GOSUB8743 jumps to **N8743** of this program, to jump back on **RETURN**

GOSUB(P17) jumps to the line label of this program whose number matches the current value of P17, to jump back on **RETURN**

See Also

Writing a Motion Program
Program commands **CALL**, **GOTO**, **N**, **O**, **RETURN**

GOTO

Function: ..... Unconditional Jump Without Return

Type: ..... Motion program (PROG only)

Syntax: ..... **GOTO{data}**

Where:.....

{data} is an integer constant or expression with a value from 0 to 99,999.

Remarks

This command causes the motion program execution to jump to the line label (**N** or **O**) specified in **{data}**, with no jump back.

If **{data}** is a constant, the path to the label will have been linked before program run time, so the jump is very quick. If **{data}** is a variable expression, it must be evaluated at run time, and the appropriate label then searched for. The search starts downward in the program to the end, then continues (if necessary) from the top of the program down.

A variable **GOTO** command permits the equivalent structure to the CASE statement found in many high-level languages (see Examples, below).

If the specified line label is not found, the program will stop, and the coordinate system's Run-Time-Error bit will be set.

Modern philosophies of the proper structuring of computer code strongly discourage the use of **GOTO**, because of its tendency to make code undecipherable.

Examples

```
GOTO750

GOTO35000

GOTO1

GOTO(50+P1)

N51 P10=50*SIN(P11)

GOTO60

N52 P10=50*COS(P11)

GOTO60

N53 P10=50*TAN(P11)

N60 X(P10)
```

See Also

Writing a Motion Program;

Program commands **CALL**, **GOSUB**, **N**, **O**.

HOME

Function:.....Programmed Homing
Type:.....
.....Motion program
Syntax: .....**HOME {constant} [,{constant}...]**

.....
.....HOME {constant}..{constant} [,{constant}..{constant}...]

.....
.....**HM {constant} [,{constant}...]**

.....
.....**HM {constant}..{constant} [,{constant}..{constant}...]**

Where:.....

{constant} is an integer from 1 to 8 representing a motor number.

Remarks

This causes the specified motor(s) to go through their homing search cycle(s). Notice that the motors must be specified directly by number, not the matching axis letters. You must specify which motors are to be homed. All motors specified in a single **HOME** command (e.g. **HOME1,2**) will start their homing cycles simultaneously; if you wish some motors to home sequentially, specify them in consecutive commands (e.g. **HOME1 HOME2**), even if on the same line.

Any previous moves will come to a stop before the home moves start. No other program statement will be executed until all specified motors have finished homing. Homing direction, speed, acceleration, etc. are determined by motor I-variables. If a motor is specified that is not in the coordinate system running the program, the command or portion of the command will be ignored, but an error will not be generated.

The speed of the home search move is determined by Ix23. If Ix23=0 then the programmed home command for that axis is ignored.

Unlike an on-line homing command, the motor number(s) in a program homing command is (are) specified *after* the word **HOME** itself, not before. In addition, an on-line homing command simply *starts* the homing search -- it does not give any indication when the search is complete; but a program homing command automatically recognizes the end of the search, and then continues on in the program. A PLC program can only issue an on-line home command.

Examples

```
HOME1      ;These are motion program commands

HM1,2,3

HOME1..3,5..7

HM1..8

#1HOME     ;These are on-line commands

#1HM,#2HM,#3HM
```

See Also

Homing-Search Moves (Basic Motor Moves)

On-line motor commands **HOME**, **HOMEZ**

Program command **HOMEZ**

HOMEZ

Function:.....Programmed Zero-Move Homing

Type:.....
.....Motion program

Syntax: .....**HOMEZ {constant} [,{constant}...]**

```
.....................................................................
...................HOMEZ {constant}..{constant} [,{constant}..{constant}...]
      ..................................HMZ {constant} [,{constant}...]
.....................................................................
.....................HMZ {constant}..{constant} [,{constant}..{constant}...]
```

Where:.....

{constant} is an integer from 1 to 8 representing a motor number.

Remarks

This commands causes the specified motor(s) to go through pseudo-homing search cycle(s). In this operation, the present *commanded* position of the motor is made the zero position for the motor and the new *commanded* position for the motor.

If there is following error and/or an axis definition offset at the time of the **HOMEZ** command, the reported position after the command will be equal to the negative of the following error plus the axis definition offset.

Notice that the motors must be specified directly by number, not the matching axis letters. You must specify which motors are to be homed. All motors specified in a single **HOMEZ** command (e.g. **HOMEZ1,2**) will home simultaneously.

Unlike an on-line homing command, the motor number(s) in a program homing command is (are) specified *after* the word **HOMEZ** itself, not before.

Examples

```
HOMEZ1     ;These are motion program commands

HMZ1,2,3

HOMEZ1..3,5..7

HMZ1..8

#1HOMEZ    ;These are on-line commands

#1HMZ,#2HMZ,#3HMZ
```

See Also

Homing-Search Moves (Basic Motor Moves)

On-line motor command **HOME**, **HOMEZ**

Program command **HOME**

I{data}

Function:..... I-Vector Specification for Circular Moves or Normal Vectors

Type:.....
.....Motion program (PROG or ROT)

Syntax: ..... **I{data}**

Where:.....

{data} is a floating-point constant or expression representing the magnitude of the I-component of the vector in scaled user axis units.

Remarks

In circular moves, this specifies the component of the vector to the arc center that is parallel to the X-axis. The starting point of the vector is either the move start point (for **INC(R)** mode -- default) or the XYZ-origin (for **ABS(R)** mode).

In a **NORMAL** command, this specifies the component of the normal vector to the plane of circular interpolation and tool radius compensation that is parallel to the X-axis.

Examples

```
X10 Y20 I5 J5

X(2*P1) I(P1)

I33.333   ; specifies a full circle whose center is 33.333 units in

     ; the positive X-direction from the start and end point

NORMAL I-1     ; specifies a vector normal to the YZ plane
```

See Also

Circular Interpolation, Tool Radius Compensation (Writing a Motion Program)

On-line command **I{constant}**

Program Commands **{axis}{data}{vector}{data}**, **ABS**, **INC**, **NORMAL**, **J**, **K**, **I{constant}={expression}**

I{constant}={expression}

Function: ..... Set I-Variable Value

Type: .....
..... Motion program (PROG and ROT), PLC Program

Syntax: ..... **I{constant}={expression}**

Where:.....

{constant} is an integer value from 0 to 1023 representing the I-variable number;

{expression) represents the value to be assigned to the specified I-variable.

Remarks

This command sets the value of the specified I-variable to that of the expression on the right side of the equals sign. The assignment is done as the line is processed, which in a motion program is usually one or two moves ahead of the move actually executing at the time (because of the need to calculate ahead in the program).
If you desire the assignment of the I-variable value to be synchronous with the beginning of the next move in the program, you should assign an M-variable to the register of the I-variable, and use a synchronous M-variable assignment statement (**M{constant}=={expression}**).

Examples

```
I130=30000

I902=1

I131=P131+1000
```

See Also

How PMAC Executes a Motion Program (Writing a Motion Program)

On-line command **I{constant}={expression}**

Program commands **M{constant}={expression)**, **P{constant}={expression}**, **Q{constant}={expression}**, **M{constant}=={expression}**

IDIS{constant}

Function: ..... Incremental displacement of X, Y, and Z axes
Type: ..... Motion program (PROG and ROT)
Syntax: ..... **IDIS{constant}**

Where:

{constant} is an integer representing the number of the first of three consecutive Q-variables to be used in the displacement vector

Remarks



This command does not cause any movement of any axes; it simply renames the present positions.

This command adds to the offset values of the currently selected (with **TSEL**) transformation matrix for the coordinate system the values contained in the three Q-variables starting with the specified one. This has the effect of renaming the current commanded X, Y, and Z axis positions (from the latest programmed move) by adding the values of these variables (Xnew=Xold+Q{constant}, Ynew=Yold+Q({constant}+1), Znew=Zold+Q({constant}+2)).
This command is similar to a **PSET** command, except that **IDIS** is incremental and does not force a stop between moves, as **PSET** does.

Examples

```
X0 Y0 Z0

Q20=7.5

Q21=12.5

Q22=20
```

```
IDIS 20   ; This makes the current position X7.5, Y12.5, Z20

IDIS 20   ; This makes the current position X15 Y25 Z40
```

See Also

Axis Matrix Transformations (Writing a Motion Program)

On-line command **DEFINE TBUF**

Program commands **TSEL**, **ADIS**, **AROT**, **IROT**, **TINIT**

IF ({condition})

Function: ..... Conditional branch

Type: ..... Motion and PLC program

Syntax: ..... **IF ({condition})** (Valid in fixed motion (PROG) or PLC program only)

IF ({condition}) {action} [{action}...] (Valid in rotary or fixed motion program only)

Where:.....

{condition} consists of one or more sets of **{expression} {comparator} {expression}**, joined by logical operators **AND** or **OR**.

{action} is a program command

Remarks

This command allows conditional branching in the program.

With an action statement or statements following on that line, it will execute those statements provided the condition is true (this syntax is valid in motion programs only). If the condition is false, it will not execute those statements; it will only execute any statements on a false condition if the line immediately following begins with **ELSE**. If the next line does not begin with **ELSE**, there is an implied **ENDIF** at the end of the line.

When there is an **ELSE** statement on the motion-program line immediately following an **IF** statement with actions on the same line, that **ELSE** statement is automatically matched to this **IF** statement, not to any preceding **IF** statements under which this **IF** statement may be nested.

With no statement following on that line, if the condition is true, PMAC will execute all subsequent statements on following lines down to the next **ENDIF** or **ELSE** statement (this syntax is valid in motion and PLC programs). If the condition is false, it will skip to the **ENDIF** or **ELSE** statement and continue execution there.

In a rotary motion program, only the single-line version of the **IF** statement is permitted. No **ELSE** or **ENDIF** statements are allowed.

In a PLC program, compound conditions can be extended onto multiple program lines with subsequent **AND** and **OR** statements.

There is no limit on nesting of **IF** conditions and **WHILE** loops (other than total buffer size) in fixed motion and PLC programs. No nesting is allowed in rotary motion programs.

Examples

```
IF (P1>10) M1=1

IF (M11=0 AND M12!=0) M2=1 M3=1
```

```
IF (M1=0) P1=P1-1
ELSE P1=P1+1
IF (M11=0)
   P1=1000*SIN(P5)
   X(P1)
ENDIF
IF (P1<0 OR P2!<0)
AND (P50=1)
   X(P1)
   DWELL 1000
ELSE
   X(P1*2)
   DWELL 2000
ENDIF
```

See Also

Conditions (Program Computational Features)

Program commands **ELSE**, **ENDIF**, **WHILE**, **AND**, **OR**

INC

Function:.....Incremental Move Mode

Type:.....
.....Motion program

Syntax: .....**INC [({axis}[,{axis}...])]**

Where:.....

{axis} is a letter specifying a motion axis (X, Y, Z, A, B, C, U, V, W), or the letter R specifying the arc center radial vector.

Remarks

The **INC** command without arguments causes all subsequent command positions in motion commands for all axes in the coordinate system running the motion program to be treated as incremental distances from the latest command point. This is known as incremental mode, as opposed to the default absolute mode.
An **INC** statement with arguments causes the specified axes to be in incremental mode, and all others stay the way they were before.
If **R** is specified as one of the 'axes', the I, J, and K terms of the circular move radius vector specification will be specified in incremental form (i.e. as a vector from the move start point, not from the origin). An **INC** command without any arguments does not affect this vector specification. The default radial vector specification is incremental.
If no motion program buffer is open when this command is sent to PMAC, it will be executed as an on-line coordinate system command.

Examples

```
INC(A,B,C)

INC

INC(U)

INC(R)
```

See Also

Circular Moves (Writing a Motion Program)

On-line commands **ABS**, **INC**

Program commands **{axis}{data}**, **{axis}{data}{vector}{data}**, **ABS**.

IROT{constant}

Function: ..... Incremental rotation/scaling of X, Y, and Z axes
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **IROT{constant}**

Where:.....

{constant} is an integer representing the number of the first of nine consecutive Q-variables to be used in the rotation/scaling matrix

Remarks



This command does not cause any movement of any axes; it simply renames the present positions
When using this command to scale the coordinate system, do not use the radius center specification for circle commands. The radius does not get scaled. Use the ***I, J, K*** *vector specification instead.*

This command multiplies the currently selected (with **TSEL**) transformation matrix for the coordinate system by the rotation/scaling values contained in the nine Q-variables starting with the specified one. This has the effect of renaming the current commanded X, Y, and Z axis positions (from the latest programmed move) by multiplying the existing rotation/scaling matrix by the matrix containing these Q-variables, adding angles of rotation and multiplying scale factors.
The rotation and scaling is done relative to the latest rotation and scaling of the XYZ coordinate system, defined by the most recent **AROT** or **IROT** commands. The math performed is:
[New Rot Matrix] = [Old Rot Matrix] [Incremental Rot Matrix]
$[Xrot\ Yrot\ Zrot]^T = [New\ Rot\ Matrix]\ [Xbase\ Ybase\ Zbase]^T$

Examples

Create a 3x3 matrix to rotate the XY plane by 30 degrees about the origin

```
Q40=COS(30)     Q41=SIN(30)     Q42=0

Q43=-SIN(30)    Q44=COS(30)     Q45=0

Q46=0      Q47=0           Q48=1

IROT 40    ; Implement the change, rotating 30

     ; degrees from current

IROT 40    ; This rotates a further 30 degrees
```

Create a 3x3 matrix to scale the XYZ space by a factor of 3

```
Q50=3      Q51=0           Q52=0

Q53=0      Q54=3           Q55=0

Q56=0      Q57=0           Q58=3

IROT 50    ; Implement the change, scaling up by a factor of 3

IROT 50    ; Scale up by a further factor of 3 (total of 9x)
```

See Also

Axis Matrix Transformations (Writing a Motion Program)

On-line command **DEFINE TBUF**

Program commands **TSEL**, **ADIS**, **IDIS**, **AROT**, **TINIT**

J{data}

Function:.....J-Vector Specification for Circular Moves

Type:.....
.....Motion program (PROG and ROT)

Syntax: .....**J{data}**

Where:.....

{data} is a floating-point constant or expression representing the magnitude of the J-component of the vector in scaled user axis units.

Remarks

In circular moves, this specifies the component of the vector to the arc center that is parallel to the Y-axis. The starting point of the vector is either the move start point (for **INC(R)** mode -- default) or the XYZ-origin (for **ABS(R)** mode).

In a **NORMAL** command, this specifies the component of the normal vector to the plane of circular interpolation and tool radius compensation that is parallel to the Y-axis.

Examples

```
X10 Y20 I5 J5

Y(2*P1) J(P1)

J33.333   ; specifies a full circle whose center is 33.333 units
in

     ; the positive Y-direction  from the start and end point

NORMAL J-1     ; specifies a vector normal to the ZX plane
```

See Also

Circular Interpolation, Tool Radius Compensation (Writing a Motion Program)

Motion Program Commands **{axis}{data}{vector}{data}**, **ABS**, **INC**, **NORMAL**, **I**, **K**.

K{data}

Function:.....K-Vector Specification for Circular Moves

Type:.....
.....Motion program (PROG and ROT)

Syntax: ..... **K{data}**

Where:.....

{data} is a floating-point constant or expression representing the magnitude of the K-component of the vector in scaled user axis units.

Remarks

In circular moves, this specifies the component of the vector to the arc center that is parallel to the Z-axis. The starting point of the vector is either the move start point (for **INC(R)** mode -- default) or the XYZ-origin (for **ABS(R)** mode).

In a **NORMAL** command, this specifies the component of the normal vector to the plane of circular interpolation and tool radius compensation that is parallel to the Y-axis.

Examples

```
X10 Z20 I5 K5

Z(2*P1) K(P1)

K33.333   ; specifies a full circle whose center is 33.333 units
in

     ; the positive Z-direction from the start and end point

NORMAL K-1     ; specifies a vector normal to the XY plane
```

See Also

Circular Interpolation, Tool Radius Compensation (Writing a Motion Program)

Motion Program Commands **{axis}{data}{vector}{data}**, **ABS**, **INC**, **NORMAL**, **I**, **J**.

Linear

Function:..... Blended Linear Interpolation Move Mode

Type:.....
.....Motion program (PROG and ROT)

Syntax: ..... **LINEAR**

. . . . . **LIN**

Remarks

The **LINEAR** command puts the program in blended linear move mode (this is the default condition on power-up/reset). Subsequent move commands in the program will be processed according to the rules of this mode. On each axis, the card attempts to reach a constant velocity that is determined by the most recent feedrate (**F**) or move time (**TM**) command.

The **LINEAR** command takes the program out of any of the other move modes (**CIRCLE**, **PVT**, **RAPID**, **SPLINE**). A command for any of these other move modes takes the program out of **LINEAR** mode.

Examples

```
LINEAR ABS

CIRCLE1 X10 Y20 I5
```

```
LINEAR X10 Y0

OPEN PROG 1000 CLEAR

N1000 LINEAR RETURN
```

See Also

Linear Blended Moves (Writing a Motion Program);
I-variables Ix87, Ix88, Ix89, Ix90;
Program commands **CIRCLE**, **PVT**, **RAPID**, **SPLINE**, **TA**, **TS**, **TM**, **F**, **{axis}{data}**.

M{constant}={expression}

Function: ..... Set M-Variable Value

Type: .....
..... Motion program (PROG and ROT)

Syntax: ..... **M{constant}={expression}**

Where:.....

{constant} is an integer constant from 0 to 1023 representing the number of the M-variable;

{expression} is a mathematical expression representing the value to be assigned to this M-variable.

Remarks

This command sets the value of the specified M-variable to that of the expression on the right side of the equals sign.
In a motion program, the assignment is done as the line is processed, not necessarily in order with the actual execution of the move commands on either side of it. If it is in the middle of a continuous move sequence, the assignment occurs one or two moves ahead of its apparent place in the program (because of the need to calculate ahead in the program).
If you wish the actual assignment of the value to the variable to be synchronous with the beginning of the next move, use the synchronous M-variable assignment command **M{constant}=={expression}** instead.

Examples

```
M1=1

M102=$00FF

M161=P161*I108*32

M20=M20 & $0F
```

See Also

How PMAC Executes a Motion Program, Synchronous Variable Assignment (Writing a Motion Program)
Program Commands **I{constant}=**, **P{constant}=**, **Q{constant}=**, **M{constant}==**.

M{constant}=={expression}

Function:.....Synchronous M-Variable Value Assignment
Type:.....
.....Motion program
Syntax: .....**M{constant}=={expression}**

Where:.....

{constant} is an integer constant from 0 to 1023 representing the number of the M-variable;

{expression} is a mathematical expression representing the value to be assigned to this M-variable.

Remarks

This command allows the value of an M-variable to be set synchronously with the start of the next move or dwell. This is especially useful with M-variables assigned to outputs, so the output changes synchronously with beginning or end of the move. Non-synchronous calculations (with the single '=') are fully executed ahead of time, during previous moves.

This command may not be used with any of the thumbwheel-multiplexer forms of M-variables (TWB, TWD, TWR, TWS).

In this form, the expression on the right side is evaluated just as for a non-synchronous assignment, but the resulting value is not assigned to the specified M-variable until the start of the actual execution of the following motion command.

Remember that if you use this M-variable in further expressions before the next move in the program is started, you will not get the value assigned in this statement.

Examples

```
X10

M1==1              ; Set Output 1 at start of actual blending to
next move.

X20

M60==P1+P2
```

See Also

How PMAC Executes a Motion Program, Synchronous Variable Assignment (Writing a Motion Program)

Program Commands **I{constant}=**, **P{constant}=**, **Q{constant}=**, **M{constant}=**.

M{constant}&={expression}

Function:.....M-Variable 'And-Equals' Assignment

Type:.....
.....Motion program (PROG and ROT)

Syntax: .....`M{constant}&={expression}`

Where:.....

`{constant}` is an integer constant from 0 to 1023 representing the number of the M-variable;

`{expression}` is a mathematical expression representing the value to be 'ANDed' with this M-variable.

Remarks

This command is equivalent to `M{constant}=M{constant}&{expression}`, except that the bit-by-bit AND and the assignment of the resulting value to the M-variable do not happen until the start of the actual execution of the following motion command. The expression itself is evaluated when the program line is encountered, as in a non-synchronous statement.
This command may not be used with any of the thumbwheel-multiplexer forms of M-variables (TWB, TWD, TWR, or TWS), or with any of the double-word forms (L, D, or F).
Remember that if you use this M-variable in further expressions before the next move in the program is started, you will not get the value assigned in this statement.

Examples

```
M20&=$FE   ; Mask out LSB of byte M20

M346&=2    ; Clear all bits except bit 1
```

See Also

How PMAC Executes a Motion Program, Synchronous Variable Assignment (Writing a Motion Program)
Program Commands `M{constant}=`, `M{constant}==`, `M{constant}|=`, `M{constant}^=`

M{constant}|={expression}

Function: ..... M Variable 'Or-Equals' Assignment
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **M{constant}|={expression}**

Where:.....

{constant} is an integer constant from 0 to 1023 representing the number of the M-variable;

{expression} is a mathematical expression representing the value to be 'ORed' with this M-variable.

Remarks

This form is equivalent to **M{constant}=M{constant}|{expression}**, except that the bit-by-bit OR and the assignment of the resulting value to the M-variable do not happen until the start of the following servo command. The expression itself is evaluated when the program line is encountered, as in a non-synchronous statement. This command may not be used with any of the thumbwheel-multiplexer forms of M-variables (TWB, TWD, TWR, or TWS), or with any of the double-word forms (L, D, or F).
Remember that if you use this M-variable in further expressions before the next move in the program is started, you will not get the value assigned in this statement.

Examples

```
M20|=$01  ; Set low bit of byte M20, leave other bits

M875|=$FF00    ; Set high byte, leaving low byte as is
```

See Also

How PMAC Executes a Motion Program, Synchronous Variable Assignment (Writing a Motion Program)
Program Commands **M{constant}=**, **M{constant}==**, **M{constant}&=**, **M{constant}^=**

M{constant}^={expression}

Function: ..... M-Variable 'XOR-Equals' Assignment
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **M{data}^={expression}**

Where:.....

{constant} is an integer constant from 0 to 1023 representing the number of the M-variable;

{expression} is a mathematical expression representing the value to be 'XORed' with this M-variable.

Remarks

This form is equivalent to **M{constant}=M{constant}^{expression}**, except that the bit-by-bit XOR and the assignment of the resulting value to the M-variable do not happen until the start of the following servo command. The expression itself is evaluated when the program line is encountered, as in a non-synchronous statement. This command may not be used with any of the thumbwheel-multiplexer forms of M-variables (TWB, TWD, TWR, or TWS), or with any of the double-word forms (L, D, or F).

Remember that if you use this M-variable in further expressions before the next move in the program is started, you will not get the value assigned in this statement.

Examples

```
M20^=$FF  ; Toggle all bits of byte M20

M99^=$80  ; Toggle bit 7 of M99, leaving other bits as is
```

See Also

How PMAC Executes a Motion Program, Synchronous Variable Assignment (Writing a Motion Program)
Program Commands **M{constant}=**, **M{constant}==**, **M{constant}&=**, **M{constant}|=**

M{data}

Function: ..... Machine Code (M-Code)
Type: ..... Motion program
Syntax: ..... **M{data}**

Where:.....

{data} is a floating-point constant or expression in the range 0.000 to 999.999, specifying the program number and the line label to jump to

Remarks

PMAC interprets this statement as a **CALL 10n1.({data'}*1000)** command, where n is the hundreds digit of **{data},** and **{data'}** is the value of **{data}** without the hundred's digit (modulo 100 in mathematical terms). That is, this statement causes a jump (with return) to motion program 10n1, and the specified line label. (Programs 10n1 are usually used to implement the machine codes as the system designer sees fit.) The value of **{data'}** can be from 0.0 to 99.999, corresponding to line labels **N0** to **N99999**.

This structure permits the implementation of customizable M-Code routines for machine-tool style applications by the writing of subroutines in motion programs 10n1. Arguments can be passed to these subroutines by following the M-Code with one or more sets of **{letter}{data}**, as in **CALL** and **READ** statements.

Most users will have M-codes only in the range 0-99, which permits the use of PROG 1001 only, and allows **{data'}** to equal **{data}** for direct specification of the line label.

Example

```
M01 jumps to N1000 of  PROG 1001

M12 jumps to N12000 of PROG 1001

M115 jumps to N15000 of PROG 1011
```

See Also

Program commands **CALL{data}, D{data}, M{data}, T{data}, RETURN**

MACROAUXREAD

Function: ..... Read MACRO auxiliary parameter value
Type: .....
..... Background PLC (no motion program, PLC0, or compiled PLC)
Syntax: ..... **MACROAUXREAD{NodeNum}{ParamNum}{Variable}**
. . . . . .
. . . . . **MXR{NodeNum}{ParamNum}{Variable}**

Where:.....

{NodeNum} is an integer constant from 0 to 15 specifying the slave number of the node

{ParamNum} is an integer constant from 0 to 65535 specifying the auxiliary parameter number for this node

{Variable} is the name of the PMAC variable (I, P, Q, or M) into which the parameter value is to be copied

Remarks

This command permits PMAC to read auxiliary register values from slave nodes across the MACRO ring. The command must specify the node number of the slave node, the auxiliary parameter number at this node, and the name of the PMAC variable to receive the value.
Only one auxiliary access (read or write) of a single node can be done on one command line.
In order to access the auxiliary registers of a MACRO node *n*, bit *n* of I1000 must be set to 1.
If the slave node returns an error message or the slave node does not respond within 32 servo cycles, PMAC will note an error condition. Bit 5 of global status register X:$0003 is set to report such a MACRO auxiliary communications error. Register X:$0798 holds the error value. It is set to $010000 for a timeout error, or $*xxxx*FE if the slave node reports an error, where *xxxx* is the 16-bit error code reported by the slave node.

Examples

```
MACROAUXREAD1,24,P1 ; Read Node 1 Parameter 24 into P1

MXR5,128,M100  ; Read Node 5 Parameter 128 into M100
```

See Also

On-line commands **MACROAUX, MACROAUXREAD, MACROAUXWRITE**

Program commands **MACROAUXWRITE**

MACROAUXWRITE

Function: ..... Write MACRO auxiliary parameter value
Type: .....
..... Background PLC (no motion program, PLC0, or compiled PLC)
Syntax: ..... **MACROAUXWRITE{NodeNum}{ParamNum}{Variable}**
. . . . . .
. . . . . **MXW{NodeNum}{ParamNum}{Variable}**

Where:.....

{NodeNum} is an integer constant from 0 to 15 specifying the slave number of the node

{ParamNum} is an integer constant from 2 to 253 specifying the auxiliary parameter number for this node

{Variable} is the name of the PMAC variable (I, P, Q, or M) from which the parameter value is to be copied

Remarks

This command permits PMAC to write auxiliary register values to slave nodes across the MACRO ring. The command must specify the node number of the slave node, the auxiliary parameter number at this node, and the name of the PMAC variable from which the value comes.
Only one auxiliary access (read or write) of a single node can be done on one command line.
In order to access the auxiliary registers of a MACRO node *n*, bit *n* of I1000 must be set to 1.
If the slave node returns an error message or the slave node does not respond within 32 servo cycles, PMAC will note an error condition. Bit 5 of global status register X:$0003 is set to report such a MACRO auxiliary communications error. Register X:$0798 holds the error value. It is set to $010000 for a timeout error, or $*xxxx*FE if the slave node reports an error, where *xxxx* is the 16-bit error code reported by the slave node.

Examples

```
MACROAUXWRITE1,24,P1; Write value of P1 to Node 1 Parameter 24

MXW5,128,M100  ; Write value of M100 to Node 5 Parameter 128
```

See Also

On-line commands **MACROAUX, MACROAUXREAD, MACROAUXWRITE**

Program commands **MACROAUXREAD**

N{constant}

Function:..... Program Line Label
Type:.....
.....Motion program (PROG and ROT)
Syntax: .....**N{constant}**

Where:.....

{constant} is an integer from 0 to 262,143 (2^{18}-1)

Remarks

This is a label for a line in the program that allows the flow of execution to jump to that line with a **GOTO**, **GOSUB**, **CALL**, **G**, **M**, **T**, or **D** statement or a **B** command.
A line only needs a label if the user wishes to be able to jump to that line. Line labels do not have to be in any sort of numerical order. The label must be at the beginning of a line. Remember that each location label takes up space in PMAC memory.
There is always an implied **N0** at the beginning of every motion program. Putting an explicit **N0** at the beginning may be useful for people reading the program. Putting an **N0** anywhere else in the program is useless and may confuse people reading the program.

Examples

```
N1

N65537 X1000
```

See Also

Subroutines and Subprograms (Writing a Motion Program)

On-line command **B{constant}**

Program commands **O{constant}**, **GOTO**, **GOSUB**, **CALL**, **G**, **M**, **T**, **D**.

NORMAL

Function:.....Define Normal Vector to Plane of Circular Interpolation and Cutter Radius Compensation

Type:.....Motion program (PROG and ROT)

Syntax: .....**NORMAL {vector}{data} [{vector}{data}...]**

.....**NRM {vector}{data} [{vector}{data}...]**

Where:.....

{vector} is one of the letters I, J, and K, representing components of the total vector parallel to the X, Y, and Z axes, respectively

{data} is a constant or expression representing the magnitude of the particular vector component.

Remarks

This statement defines the orientation of the plane in XYZ-space in which circular interpolation and cutter radius compensation will take place by setting the normal (perpendicular) vector to that plane.
The vector components that can be specified are I (X-axis direction), J (Y-axis direction), and K (Z-axis direction). The ratio of the component magnitudes determines the orientation of the normal vector, and therefore, of the plane. The length of this vector does not matter -- it does not have to be a unit vector.
The direction sense of the vector does matter, because it defines the clockwise sense of an arc move, and the sense of cutter-compensation offset. PMAC uses a right-hand rule; that is, in a right-handed coordinate system (**I** x **J** = **K**), if your right thumb points in the direction of the normal vector specified here, your right fingers will curl in the direction of a clockwise arc in the circular plane, and in the direction of offset-right from direction of movement in the compensation plane.

Examples

The standard settings to produce circles in the principal planes will therefore be:

```
NORMAL K-1; XY plane -- equivalent to G17

NORMAL J-1; ZX plane -- equivalent to G18

NORMAL I-1; YZ plane -- equivalent to G19
```

By using more than one vector component, a circular plane skewed from the principal planes can be defined:

```
NORMAL I0.866 J0.500

NORMAL J25 K-25

NORMAL J(-SIN(Q1)) K(-COS(Q1))

NORMAL I(P101) J(P201) K(301)
```

See Also

Circular Blended Moves, Cutter Radius Compensation (Writing a Motion Program)

Cartesian Axes (Setting Up a Coordinate System)

Program Commands **CIRCLE1**, **CIRCLE2**, **CC0**, **CC1**, **CC2**

O{constant}

Function: Alternate Line Label
Type: Motion program (PROG and ROT)
Syntax: **O{constant}**

Where:

{constant} is an integer from 0 to 262,143 (2^{18}-1)

Remarks

This is an alternate form of label in the motion program. It allows the flow of execution to jump to that line with a **GOTO**, **GOSUB**, **CALL**, **G**, **M**, **T**, or **D** statement or a **B** command. PMAC will store and report this as an **N{constant}** statement, but **O** labels are legal to send to the program buffer. (**N10** and **O10** are identical labels to PMAC.)

A line only needs a label if the user wishes to be able to jump to that line. Line labels do not have to be in any sort of numerical order. The label must be at the beginning of a line. Remember that each location label takes up space in PMAC memory.

Examples

```
O1

O65537 X1000
```

See Also

Subroutines and Subprograms (Writing a Motion Program)

On-line command **B{constant}**

Program commands **O{constant}**, **GOTO**, **GOSUB**, **CALL**, **G**, **M**, **T**, **D**.

OR({condition})

Function: Conditional OR
Type: PLC program
Syntax: **OR ({condition})**

Remarks

This statement forms part of an extended compound condition to be evaluated in a PLC program. It must immediately follow an **IF**, **WHILE**, **AND**, or **OR** statement. This **OR** is a Boolean operator logically combining the condition on its line with the condition on the program line above.

It takes lower precedence than operators within a compound condition on a single line (those within parentheses), and also lower precedence than an **AND** operator that starts a line. (**OR**s operate on groups of **AND**ed conditions.)

In motion programs, there can be compound conditions within one program line, but not across multiple program lines, so this statement is not permitted in motion programs.
This logical **OR**, which acts on *conditions*, should not be confused with the bit-by-bit **|** (vertical bar) or-operator, which operates on *values.*

Examples

```
IF (M11=1); This branch increments P1 every cycle that
AND (M12=0)    ; inputs M11 and M12 are different, and decrements
OR (M11=0); them every cycle that they are the same.
AND (M12=1)
   P1=P1+1
ELSE
   P1=P1-1
ENDIF
IF (M11=1 AND M12=0); This does the same as above
OR (M11=0 AND M12=1)
   P1=P1+1
ELSE
   P1=P1-1
ENDIF
```

See Also

Conditions (Program Computational Features)
Writing a PLC Program
Program commands **IF**, **WHILE**, **AND**

P{constant}={expression}

Function:.....Set P-Variable Value

Type:.....
.....Motion program (PROG and ROT)

Syntax: .....**P{constant}={expression}**

Where:.....

{constant} is an integer constant from 0 to 1023 representing the P-variable number;

{expression} represents the value to be assigned to this P-variable.

Remarks

This command sets the value of the specified P-variable to that of the expression on the right side of the equals sign. The assignment is done as the line is processed, which in a motion program is usually one or two moves ahead of the move actually executing at the time (because of the need to calculate ahead in the program).

Examples

```
P1=0

P746=P20+P40

P893=SIN(Q100)-0.5
```

See Also

How PMAC Executes a Motion Program (Writing a Motion Program)

On-line command **P{constant}={expression}**

Program commands **I{constant}={expression}**, **M{constant}={expression}**, **Q{constant}={expression}**.

PRELUDE

Function: ..... Specify automatic subroutine call function
Type: .....
..... Motion program
Syntax: ..... **PRELUDE1{command}**
. . . . . **PRELUDE0**

Where:.....

{command} is a subprogram call from the set

. . . . . **CALL{constant},**

. . . . . **G{constant}, M{constant}, T{constant}, D{constant}**

Remarks

The **PRELUDE1** command permits automatic insertion of a subprogram call before each subsequent motion command (e.g. **X10Y10**) or other letter-number command (e.g. **L10**) other than a line label in the motion program. The action taken is equivalent to inserting the call into the program text before each subsequent motion command or letter-number command.

The subprogram call to be performed can be specified in the **PRELUDE1** command either as a **CALL** command, or as a **G**, **M**, **T**, or **D** code. The value following the **CALL** or code letter must be a constant; it cannot be a variable or expression. It does not have to be an integer. If the routine called in the subprogram starts with a **READ** statement, the motion or letter-number command itself can become arguments for the subprogram call. Any motion command within a **PRELUDE1** subroutine or subprogram call is executed directly as a motion command, without an automatic subroutine call in front of it..

PMAC will only execute the **PRELUDE1** function if the motion or letter-number command is found at the beginning of a program line or immediately after the line label. If another type of command occurs earlier on the program line, no **PRELUDE1** function will be executed before the motion or letter-number command. If the command is on a line that is already in a subroutine or subprogram reached by a **CALL** or **GOSUB** command, no **PRELUDE1** function will be executed.

Each **PRELUDE1** command supersedes the previous **PRELUDE1** command. It is not possible to nest automatic **PRELUDE1** calls, but an automatic **PRELUDE1** call can be nested within explicit subroutine and subprogram calls. **PRELUDE0** disables any automatic subroutine call.

Examples

```
PRELUDE1 CALL10; Insert a CALL10 before subsequent moves

X10 Y20   ; Implicit CALL10 before this move

X20 Y30   ; Implicit CALL10 before this move

...

OPEN PROG 10 CLEAR  ; Subprogram

Z-1  ; Move down

DWELL 500 ; Hold position

Z1   ; Move up

RETURN

...

G71 X7 Y15 P5  ; G71 calls PROG 1000 N71000

X8 Y16 P5 ; With PRELUDE, G71 is implied (modal)

X9 Y15 P8 ; With PRELUDE, G71 is implied (modal)

G70  ; Stop modal canned cycles

...

OPEN PROG 1000

...

N70000    ; G70 subroutine

PRELUDE0  ; Stop PRELUDE calls

RETURN

N71000    ; G71 subroutine

PRELUDE1 G71.1 ; Make G71 modal by using PRELUDE

RETURN

N71100    ; G71.1 routine is what executes G71 modally

READ(X,Y,P)     ; Read values associated with X, Y, and P {action
based on
     ;parameters}
```

RETURN

See Also

Subroutines and Subprograms (Writing a Motion Program)
Program commands **CALL**, **GOSUB**, **READ**, **G**, **M**, **T**, **D**

PSET

Function: ..... Redefine current axis positions (Position SET)
Type: ..... Motion program
Syntax: ..... **PSET{axis}{data} [{axis}{data}...]**

Where:.....

{axis} is the character specifying which axis (X, Y, Z, A, B, C, U, V, W);

{data} is a constant or an expression representing the new value for this axis position

Remarks

This command allows the user to re-define the value of an axis position in the middle of the program. It is equivalent to the RS-274 G-Code **G92**. No move is made on any axis as a result of this command -- the value of the present *commanded* position for the axis is merely set to the specified value.
Internally, this command changes the value of the "position bias" register for each motor attached to an axis named in the command. This register holds the difference between the axis zero point and the motor zero (home) point.
This command automatically forces a temporary pause in the motion of the axes; no moves are blended "through" a **PSET** command. For more powerful and flexible offsets that can be done on the fly (X, Y, and Z axes only), refer to the matrix manipulation commands such as **ADIS** and **IDIS**.

Examples

```
X10Y20

PSET X0 Y0; Call this position (0,0)

N92000 READ(X,Y,Z)        ; To implement G92 in PROG 1000

PSET X(Q124)Y(Q125)Z(Q126)    ; Equivalent of G92 X..Y..Z..
```

See Also

Axes (Setting Up a Coordinate System)
On-line command **{axis}={expression}**
Program commands **ADIS**, **AROT**, **IDIS**, **IROT**
Suggested M-variable definitions Mx64
Memory map registers D:$0813, D:$08D3, etc

PVT{data}

Function: ..... Set Position-Velocity-Time mode
Type: ..... Motion program (PROG and ROT)
Syntax: ..... **PVT{data}**

Where:

{data} is a positive constant or expression representing the time of a segment in milliseconds *(PMAC will round this value to the nearest integer in actual use)*.

Remarks

This command puts the motion program into Position-Velocity-Time move mode, and specifies the time for each segment of the move. In this mode, each move segment in the program must specify the ending position *and* velocity for the axis. Taking the starting position and velocity (from the previous segment), the ending position and velocity, and the segment time, PMAC computes the unique cubic position profile (parabolic velocity profile) to meet these constraints.
The segment time in a sequence of moves can be changed on the fly, either with another **PVT** command, or with a **TA** command. **TS**, **TM**, and **F** settings are irrelevant in this mode.
The **PVT** command takes the program out of any of the other move modes (**LINEAR**, **CIRCLE**, **SPLINE**, **RAPID**), and any of the other move mode commands takes the program out of **PVT** move mode.
Refer to the Writing a Motion Program section of the manual for more details of this mode.

Examples

```
INC  ; incremental mode, specify moves by distance

PVT200    ; enter this mode -- move time 200ms

X100:1500 ; cover 100 units ending at 1500 units/sec

X500:3000 ; cover 500 units ending at 3000 units/sec

X500:1500 ; cover 500 units ending at 1500 units/sec

X100:0    ; cover 100 units ending at 0 units/sec

PVT(P37)
```

See Also

Position-Velocity-Time Mode Moves (Writing a Motion Program)

Program commands **{axis}{data}:{data}...**, **TA**, **LINEAR**, **CIRCLEn**, **RAPID**, **SPLINE1**.

Q{constant}={expression}

Function: Set Q-Variable Value
Type: Motion program (PROG and ROT); PLC program
Syntax: **Q{constant}={expression}**

Where:

{constant} is an integer value from 0 to 1023 representing the Q-variable number;

{expression} represents the value to be assigned to the specified Q-variable.

Remarks

This command sets the value of the specified Q-variable to that of the expression on the right side of the equals sign. The assignment is done as the line is processed, which in a motion program performing a continuous move sequence is usually one or two moves ahead of the move actually executing at the time (because of the need to calculate ahead in the program).

Because each coordinate system has its own set of Q-variables, it is important to know which coordinate system's Q-variable is affected by this command. When executed from inside a motion program, this command affects the specified Q-variable of the coordinate system running the motion program.

When executed from inside a PLC program, this command affects the specified Q-variable of the coordinate system specified by the most recent **ADDRESS** command executed inside *that* PLC program. If there has been no **ADDRESS** command executed since power-on/reset, it affects the Q-variable of Coordinate System 1.

Examples

```
Q1=3

Q99=2.71828

Q124=P100+ATAN(Q120)
```

See Also

Q-Variables (Program Computational Features)

On-line command **Q{constant}={expression}**

Program commands **ADDRESS**, **I{constant}={expression}**, **M{constant}={expression}**, **P{constant}={expression}**

R{data}

Function:.....Set Circle Radius
Type:.....
.....Motion program (PROG or ROT)
Syntax: .....**R{data}**

Where:.....

{data} is a constant or expression representing the radius of the arc move specified in user length units.

Remarks

This partial command defines the magnitude of the radius for the circular move specified on that command line. It does not affect the moves on any other command lines.. (If there is no 'R' radius specification and no 'IJK' vector specification on a move command line, the move will be done linearly, even if the program is in **CIRCLE** mode.)
If the radius value specified in **{data}** is greater than zero, the circular move to the specified end point will describe an arc of less than or equal to 180° with a radial length of the specified value. If the radius value specified in **{data}** is less than zero, the circular move to the specified end point will describe an arc of greater than or equal to 180° with a radial length equal to the absolute value of **{data}**.
If you use the **AROT** or **IROT** commands to scale the coordinate system, do not use the radius center specification for circle commands. The radius does not get scaled. Use the **I, J, K** vector specification instead.
If the distance from the start point to the end point is more than twice the magnitude specified in **{data}**, there is no circular arc move possible. If the distance is greater than twice **{data}** by an amount less than Ix96 (expressed in user length units), PMAC will execute a spiral to the end point. If the distance is greater by more than Ix96, PMAC will stop the program with a run-time error.

Examples

```
RAPID X0 Y0     ; Move to origin

CIRCLE1     Clockwise circle mode

X10 Y10 R10     ; Quarter circle to (10, 10)

X0 Y0 R-10 Three-quarters circle back to (0, 0)

X(P101) R(P101/2)    ; Half circle to (P101, 0)
```

See Also

Circular Blended Moves (Writing a Motion Program);

I-variables I13, Ix96

Program commands **CIRCLE1**, **CIRCLE2**, **{axis}{data}{vector}{data}...**

RAPID

Function: ..... Set Rapid Traverse Mode
Type: ..... Motion program (PROG and ROT)
Syntax: ..... **RAPID**
. . . . . **RPD**

Remarks

This command puts the program into a mode in which all motors defined to the commanded axes move to their destination points in jog-style moves. This mode is intended to create the minimum-time move from one point to another. Successive moves are not blended together in this mode, and the different motors do not necessarily all reach their end points at the same time

The accelerations and decelerations in this mode are controlled by motor jog-acceleration I-variables Ix19, Ix20, and Ix21. If global I-variable I50 is set to 0, the velocities in this mode are controlled by the motor jog speed I-variables Ix22. If I50 is set to 1, they are controlled by the motor maximum speed I-variables Ix16. Only the motor with the greatest distance-to-speed ratio for the move actually moves at this speed; all other motors are slowed from the specified speed to complete the move in approximately the same time, so that the move is nearly linear.

The **RAPID** command takes the program out of any of the other move modes (**LINEAR**, **CIRCLE**, **PVT**, **SPLINE**); any of the other move-mode commands takes the program out of **RAPID** mode.

Examples

```
RAPID X10 Y20  ; Move quickly to starting cut position

M1=1 ; Turn on cutter

LINEAR X12 Y25 F2   ; Start cutting moves

...

M1=0 ; Turn off cutter

RAPID X0 Y0    ; Move quickly back to home position
```

See Also

Rapid Mode Moves (Writing a Motion Program)
I-variables I50, Ix16, Ix19, Ix22
Program commands **LINEAR**, **CIRCLE**, **PVT**, **SPLINE**

READ



*No space is allowed between **READ** and the left parenthesis.*

Function: .....
Type: .....
Syntax: ..... **READ(**
.....

{letter} is any letter of the English alphabet, except **N** or **O**, representing the letter on the calling program line whose following value is to be read into a variable

Remarks

This statement allows a subprogram or subroutine to take arguments from the calling routine. It looks at the remainder of the line calling this routine (**CALL**, **G**, **M**, **T**, **D**), takes the values following the specified letters and puts them into particular Q-variables for the coordinate system. For the Nth letter of the alphabet, the value is put in Q(100+N).

It scans the calling line until it sees a letter that is not in the list of letters to **READ**, or until the end of the calling line. Each letter value successfully "read" into a Q-variable causes a bit to be set in Q100, noting that it was read (bit N-1 for the Nth letter of the alphabet). For any letter not successfully read in the most recent **READ** command, the corresponding bit of Q100 is set to zero.

The Q-variable and flag bit of Q100 associated with each letter are shown in the following table:

LETTER	TARGET VARIABLE	Q100 BIT	BIT VALUE DECIMAL	BIT VALUE HEX
A	Q101	0	1	$01
B	Q102	1	2	$02
C	Q103	2	4	$04
D	Q104	3	8	$08
E	Q105	4	16	$10
F	Q106	5	32	$20
G	Q107	6	64	$40
H	Q108	7	128	$80
I	Q109	8	256	$100
J	Q110	9	512	$200
K	Q111	10	1,024	$400
L	Q112	11	2,048	$800
M	Q113	12	4,096	$1000
N*	Q114*	13*	8,192*	$2000*
O*	Q115*	14*	16,384*	$4000*
P	Q116	15	32,768	$8000
Q	Q117	16	65,536	$10000
R	Q118	17	131,072	$20000
S	Q119	18	262,144	$40000
T	Q120	19	524,288	$80000
U	Q121	20	1,048,57	$100000
V	Q122	21	2,097,15	$200000
W	Q123	22	4,194,304	$400000

LETTER	TARGET VARIABLE	Q100 BIT	BIT VALUE DECIMAL	BIT VALUE HEX
X	Q124	23	8,388,608	$800000
Y	Q125	24	16,777,216	$1000000
Z	Q126	25	33,554,432	$2000000

*Cannot be used

Any letter may be **READ** except **N** or **O**, which are reserved for line labels (and should only be at the beginning of a line anyway). If a letter value is read from the calling line, the normal function of the letter (e.g. an axis move) is overridden, so that letter serves merely to pass a parameter to the subroutine. If there are remaining letter values on the calling line that are not read, those will be executed according to their normal function after the return from the subroutine.

Examples

In standard machine tool code, a two-second **DWELL** would be commanded in the program as a **G04 X2000**, for instance. In PMAC, a **G04** is interpreted as a **CALL** to label **N04000** of PROG 1000, so to implement this function properly, PROG 1000 would contain the following code:

```
N04000 READ(X)
DWELL (Q124)
RETURN
```

Also, in standard machine tool code, the value assigned to the current position of the axis may be changed with the **G92** code, followed by the letters and the new assigned values of any axes (e.g. **G92 X20 Y30**). It is important only to assign new values to axes specified in this particular **G92** command, so the PMAC subroutine implementing **G92** with the **PSET** command must check to see if that particular axis is specified:

```
N92000 READ(X,Y)
IF (Q100 & $800000 > 0) PSET X(Q124)
IF (Q100 & $1000000 > 0) PSET Y(Q125)
IF (Q100 & $2000000 > 0) PSET Z(Q126)
RETURN
```

See Also

Subroutines and Subprograms (Writing a Motion Program)
Program commands **CALL**, **GOSUB**, **G**, **M**, **T**, **D**

RETURN

Function:.....Return From Subroutine Jump/End Main Program
Type:.....
.....Motion program (PROG only)
Syntax: .....**RETURN**
.....
.....**RET**

Remarks

The **RETURN** command tells the motion program to jump back to the routine that called the execution of this routine. If this routine was started from an on-line command (Run), program execution stops and the program pointer is reset to the top of this motion program -- control is returned to the PMAC "operating system".
If this routine was started from a **GOSUB**, **CALL**, **G**, **M**, **T**, or **D** command in a motion program, program execution jumps back to the command immediately following the calling command.
When the **CLOSE** command is sent to end the entry into a motion program buffer, PMAC automatically appends a **RETURN** command to the end of that program. When the **OPEN** command is sent to an existing motion program buffer, the final **RETURN** command is automatically removed.

Examples

```
OPEN PROG 1 CLEAR

X20 F10

X0

CLOSE     ; PMAC places a RETURN here

OPEN PROG 1000 CLEAR

N0 RAPID RETURN; Execution jumps back after one-line routine

N1000 LINEAR RETURN ; Ditto

N2000 CIRCLE1 RETURN; Ditto
```

```
...

CLOSE      ; PMAC places a RETURN here
```

See Also

Subroutines and Subprograms (Writing a Motion Program)

On-line commands **OPEN**, **CLOSE**

Program commands **CALL**, **GOSUB**, **G**, **M**, **T**, **D**

S{data}

Function: ..... Spindle data command
Type: ..... Motion program (PROG and ROT)
Syntax: ..... **S{data}**

Where:

{data} is a constant or expression representing the value to be passed to the storage variable for later use

Remarks

This command causes the value in **{data}** to be loaded in variable Q127 for the coordinate system executing the motion program. It takes no other action. It is intended to pass spindle speed data in machine tool programs. The algorithms that actually control the spindle would then use Q127 in their routines; for instance, to set jog speed, or voltage output.
This command is distinct from **S{data}** information passed as part of a subroutine call through a **READ(S)** command; in that form, the value is placed in Q119 for the coordinate system.

Examples

```
S1800      ; This puts a value of 1800 in Q127

S(P1)      ; This puts the value of P1 in Q127

G96 S50    ; Here the S-term is an argument in the G-code call

(PROG 1000)     ; This is the subroutine that executes G96

N96000 READ(S) ; This puts a value of 50 in Q119
```

See Also

Q-variables (Program Computational Features)

Implementing a Machine-Tool Style Program (Writing a Motion Program)

Motion program command **READ**

Example program SPINDLE.PMC

SEND

Function: ..... Cause PMAC to Send Message
Type: ..... Motion program (PROG and ROT); PLC program
Syntax: ..... **SEND"{message}"**
. . . . . .
. . . . . **SENDS"{message}"**
. . . . . .
. . . . . **SENDP"{message}"**

Remarks

This command causes PMAC to send the specified message out of one of the PMAC communications ports. This is particularly useful in the debugging of applications. It can also be used to prompt an operator, or to notify the host computer of certain conditions.

If I62=0, PMAC automatically issues a carriage-return (**<CR>**) character at the end of the message. If I62=1, PMAC does not issue a **<CR>** character at the end of the message; a **SEND^M** must be used to issue a **<CR>** in this case.

If there is no host on the port to which the message is sent, or the host is not ready to read the message, the message is left in the queue. If several messages back up in the queue this way, the program issuing the messages will halt execution until the messages are read. This is a common mistake when the SEND command is used outside of an Edge-Triggered condition in a PLC program. See Writing A PLC Program in chapter 3 for more details.

On the serial port, it is possible to send messages to a non-existent host by disabling the port handshaking with I1=1.

SEND transmits over the active communications response port, whether serial, parallel host port (PC-Bus or STD-Bus), VME-Bus port, or ASCII DPRAM buffer.

SENDS always transmits over the serial port regardless of what is the current active response port.

SENDP always transmits over the parallel host port (PC-Bus or STD-Bus), regardless of which port is the current active response port.

There is no **SENDV** command for the VME bus exclusively. The **SEND** command must be used with the VME port as the active response port.

When PMAC powers up or resets, the active response port is the serial port. When any command is received over a bus port, the active response port becomes the bus port. PMAC must then receive a **<CONTROL-Z>** command to cause the response port to revert back to the serial port.

If a program, particularly a PLC program, sends messages immediately on power-up/reset, it can confuse a host-computer program (such as the PMAC Executive Program) that is trying to "find" PMAC by querying it and looking for a particular response.

It is possible, particularly in PLC programs, to order the sending of messages faster than the port can handle them. This will almost always happen if the same **SEND** command is executed every scan through the PLC. For this reason, it is good practice to have at least one of the conditions that causes the **SEND** command to execute to be set false immediately to prevent execution of this **SEND** command on subsequent scans of the PLC.

To cause PMAC to send the *value* of a variable, use the **COMMAND** statement instead, specifying the name of the variable in quotes (e.g. **CMD"P1"**)

Examples

```
SEND"Motion Program Started"

SENDS"DONE"

SENDP"Spindle Command Given"

IF (M188=1)     ; C.S.1 Warning Following Error Bit set?

IF (P188=0)     ; But not set last scan? (P188 follows M188)
```

```
 SEND"Excessive Following Error"   ; Notify operator
 P188=1   ; To prevent repetition of message
ENDIF
ELSE ; F.E. Bit not set
 P188=0   ; To prepare for next time
ENDIF
SEND"THE VALUE OF P7 IS:"; PMAC to send the message string
CMD"P7"   ; PMAC to return the value of P7
```

See Also

I-variables I1, I62

Program commands **COMMAND**, **DISPLAY**, **SEND^{letter}**

Writing A PLC Program

SEND^{letter}

Function:.....Cause PMAC to Send Control Character

Type:.....
.....Motion program (PROG and ROT); PLC program

Syntax: .....**SEND^{letter}**

.....
.....**SENDS^{letter}**

.....
.....**SENDP^{letter}**

Where:.....

{letter} is one of the characters in the following set:

@ABC...XYZ[\]^_

Remarks

Do not put the up-arrow character and the letter in quotes (do not use ***SEND"^A"****) or PMAC will attempt to send the two non-control characters* ***^*** *and* ***A*** *for this example, instead of the control character.*

This command causes PMAC to send the specified control character over one of the communications ports. These can be used for printer and terminal control codes, or for special communications to a host computer
Control characters have ASCII byte values of 0 to 31 ($1F). The specified {letter} character determines which control character is sent when the statement is executed. The byte value of the control character sent is 64 ($40) less than the byte value of {letter}. The letters that can be used and their corresponding control characters are:

{LETTER}	LETTER VALUE	CONTROL CHARACTER	VALUE
@	64	NULL	0
A	65	<CTRL-A>	1
B	66	<CTRL-B>	2
C	67	<CTRL-C>	3
...			
X	88	<CTRL-X>	24
Y	89	<CTRL-Y>	25
Z	90	<CTRL-Z>	26
[	91	ESC	27
\	92		28
]	93		29
^	94		30
_	95		31

SEND transmits over the active communications response port, whether serial, parallel host port (PC-Bus or STD-Bus), or VME-Bus port.
SENDS always transmits over the serial port regardless of what is the current active response port.
SENDP always transmits over the parallel host port (PC-Bus or STD-Bus), regardless of which port is the current active response port.
There is no **SENDV** command for the VME bus exclusively. The **SEND** command must be used with the VME port as the active response port.

When PMAC powers up or resets, the active response port is the serial port. When any command is received over a bus port, the active response port becomes the bus port. PMAC must then receive a **<CONTROL-Z>** command to cause the response port to revert back to the serial port.
It is possible, particularly in PLC programs, to order the sending of messages faster than the port can handle them. This will almost always happen if the same **SEND** command is executed every scan through the PLC. For this reason, it is good practice to have at least one of the conditions that causes the **SEND** command to execute to be set false immediately to prevent execution of this **SEND** command on subsequent scans of the PLC.

See Also

On-line command **<CTRL-Z>**

Program commands **SEND"{message}", COMMAND"{command}", COMMAND^{letter}**

SPLINE1

Function: ..... Put program in uniform cubic spline motion mode
Type: ..... Motion program (PROG and ROT)
Syntax: **SPLINE1**

Remarks

This modal command puts the program in cubic spline mode. In **SPLINE1** mode, each programmed move takes **TA** time (Ix87 is default) -- there is no feedrate specification allowed. Each move on each axis is computed as a cubic position trajectory in which the intermediate positions are relaxed somewhat so there are no velocity or acceleration discontinuities in blending the moves together.
Before the first move in any series of consecutive moves, a starting move of **TA** time is added to blend smoothly from a stop. After the last move in any series of consecutive moves, an ending move of **TA** time is added to blend smoothly to a stop. If the **TA** time is changed in the middle of a series of moves, there will be a stop generated, with an extra $\mathbf{TA}_1$ move and an extra $\mathbf{TA}_2$ move added.

This command will take the program out of any of the other move modes (**LINEAR**, **CIRCLE**, **PVT**, **RAPID**). The program will stay in this mode until another move mode command is executed.

Examples

```
RAPID X10 Y10

SPLINE1 TA100

X20 Y15

X32 Y21

X43 Y26

X50 Y30

DWELL100

RAPID X0 Y0
```

See Also

Cubic Spline Mode (Writing a Motion Program)

I-variable Ix87

Program commands **LINEAR**, **CIRCLE**, **RAPID**, **PVT**, **SPLINE2**, **TA**

SPLINE2

Function:.....Put program in non-uniform cubic spline motion mode
Type:.....
.....Motion program (PROG and ROT)
Syntax: **SPLINE2**

Remarks

This modal command puts the program in non-uniform cubic spline mode. This mode is virtually identical to the **SPLINE1** uniform cubic spline mode described above, except that the TA segment time can vary in a continuous spline. This makes **SPLINE2** mode more flexible than **SPLINE1** mode, but it takes slightly more computation time.

Examples

```
RAPID X10 Y10

SPLINE2

X20 Y15 TA100

X32 Y21 TA120

X43 Y26 TA87

X50 Y30 TA62

DWELL100

RAPID X0 Y0
```

See Also

Cubic Spline Mode (Writing a Motion Program)

I-variable Ix87

Program commands **LINEAR**, **CIRCLE**, **RAPID**, **PVT**, **SPLINE1**, **TA**

STOP

Function: ..... Stop program execution
Type: ..... Motion program (PROG)
Syntax: ..... **STOP**

Remarks

This command suspends program execution, whether started by 'run' or 'step', keeping the program counter pointing to the next line in the program, so that execution may be resumed with a 'run' or 'step' command.

Examples

```
A10 B10

A20 B0

STOP

A0 B0
```

See Also

On-line commands **<CONTROL-Q>**, **Q**, **R**, **S**

Program commands **BLOCKSTART**, **BLOCKSTOP**

T{data}

Function: ..... Tool Select Code (T-Code)
Type: ..... Motion program
Syntax: ..... **T{data}**

Where:.....

{data} is a floating-point constant or expression in the range 0.000 to 999.999, specifying the program number and the line label to jump to

Remarks

PMAC interprets this statement as a **CALL 10n2.({data'}*1000)** command, where n is the hundreds digit of **{data},** and **{data'}** is the value of **{data}** without the hundred's digit (modulo 100 in mathematical terms). That is, this statement causes a jump (with return) to motion program 10n2, and the specified line label. (Programs 10n2 are usually used to implement the machine codes as the system designer sees fit.) The value of **{data'}** can be from 0.0 to 99.999, corresponding to line labels **N0** to **N99999**.

This structure permits the implementation of customizable T-Code routines for machine-tool style applications by the writing of subroutines in motion programs 10n2. Arguments can be passed to these subroutines by following the T-Code with one or more sets of **{letter}{data}**, as in **CALL** and **READ** statements.

Most users will have T-codes only in the range 0-99, which permits the use of PROG 1002 only, and allows **{data'}** to equal **{data}** for direct specification of the line label.

Example

T01 jumps to **N1000** of PROG 1002

T12 jumps to **N12000** of PROG 1002

T115 jumps to **N15000** of PROG 1012

See Also

Program commands CALL{data}, D{data}, M{data}, T{data}, RETURN

TA{data}

Function: ..... Set Acceleration Time

Type: ..... Motion program (PROG and ROT)

Syntax: ..... **TA{data}**

Where:.....

{data} is a constant or expression representing the acceleration time in milliseconds

Remarks

This statement specifies the commanded acceleration time between blended moves (**LINEAR** and **CIRCLE** mode), and from and to a stop for these moves. In **PVT** and **SPLINE1** mode moves, which are generally continually accelerating and decelerating, it specifies the actual move segment time. The units are milliseconds. *PMAC will round the specified value to the nearest integer number of milliseconds when executing this command* (no rounding is done in storing the value in the buffer).

Make sure the specified acceleration time (TA or 2*TS) is greater than zero, even if you are planning to rely on the maximum acceleration rate parameters (Ix17). A specified acceleration time of zero will cause a divide-by-zero error. The minimum specified time should be **TA1 TS0**.

If the specified S-curve time (from TS, or Ix88) is greater than half the TA time, the time used for the acceleration for blended moves will be twice the specified S-curve time.

The acceleration time is also the minimum time for a blended move; if the distance on a feedrate-specified (**F**) move is so short that the calculated move time is less than the acceleration time, or the time of a time-specified (**TM**) move is less than the acceleration time, the move will be done in the acceleration time instead. This will slow down the move. If **TA** controls the move time it must be greater than the I13 time and the I8 period.

The acceleration time will be extended automatically when any motor in the coordinate system is asked to exceed its maximum acceleration *rate* (Ix17) for a programmed **LINEAR** mode move with I13=0 (no move segmentation).

A move executed in a program before any **TA** statement will use the default acceleration time specified by coordinate system I-variable Ix87.

In executing the TA command, PMAC rounds the specified value to the nearest integer number of milliseconds (there is no rounding done when storing the command in the buffer).

Examples

```
TA100

TA(P20)

TA(45.3+SQRT(Q10))
```

See Also

Linear, Circular Blended Moves, Cubic Spline Moves, PVT Moves (Writing a Motion Program)

I-variables Ix17, Ix87, Ix88

Program commands **TS**, **TM**, **PVT**.

TINIT

Function: ..... Initialize selected transformation matrix

Type: .....
..... Motion program (PROG and ROT)

Syntax: ..... **TINIT**

Remarks

This command initializes the currently selected (with **TSEL**) transformation matrix for the coordinate system by setting it to the identity matrix. This makes the rotation angle 0, the scaling 1, and the displacement 0, so the XYZ points for the coordinate system are as the axis definition statements created them. PMAC will still perform the matrix calculations, even though they have no effect. **TSEL0** should be used to stop the matrix calculations

The matrix can subsequently be changed with the **ADIS**, **IDIS**, **AROT**, and **IROT** commands.

Examples

```
TSEL 4     ; Select transformation matrix 4

TINIT      ; Initialize it to the identity matrix

IROT 71    ;  Do incremental rotation/scaling with Q71-Q79
```

See Also

Axis Matrix Transformations (Writing a Motion Program)

On-line command **DEFINE TBUF**

Program commands **TSEL**, **ADIS**, **IDIS**, **AROT**, **IROT**

TM{data}

Function: ..... Set Move Time

Type: .....
..... Motion program

Syntax: ..... **TM{data}**

Where:.....

{data} is a floating-point constant or expression representing the move time in milliseconds. The maximum effective **TM** value is 2^{23} msec. The minimum effective **TM** value is 1 msec.

Remarks

This command establishes the time to be taken by subsequent **LINEAR** or **CIRCLE** mode (blended) motions. It overrides any previous **TM** or **F** statement, and is overridden by any subsequent **TM** or **F** statement. It is irrelevant in

RAPID, **SPLINE**, and **PVT** move modes, but the latest value will stay active through those modes for the next return to blended moves.

The acceleration time is the minimum time for a blended move; if the specified move time is shorter than the acceleration time, the move will be done in the acceleration time instead. This will slow down the move. If **TM** controls the move time it must be greater than the I13 time and the I8 period.

For **LINEAR** mode moves with I13=0 (no move segmentation), if the commanded velocity (distance/TM) of any motor in the move exceeds its maximum limit (Ix16), all motors in the coordinate system will be slowed down in proportion so that no motor exceeds its limit.

Examples

```
TM30

TM47.635

TM(P1/3)
```

See Also

Linear and Circular Blended Moves (Writing a Motion Program)

I-variable Ix16

Program commands **F**, **TA**, **TS**, **LINEAR**, **CIRCLE**

TS{data}

Function: ..... Set S-Curve Acceleration Time
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **TS{data}**
Where:.....

{data} is a positive constant or expression representing the S-curve time in milliseconds.

Remarks

This command specifies the time, at both the beginning and end of the total acceleration time, in **LINEAR** and **CIRCLE** mode blended moves that is spent in S-curve acceleration.
If **TS** is zero, the acceleration is constant throughout the **TA** time and the velocity profile is trapezoidal. If **TS** is greater than zero, the acceleration will start at zero and linearly increase through **TS** time, then stay constant (for time **TC**) until **TA-TS** time, and linearly decrease to zero at **TA** time (that is, **TA=2TS+TC**). If **TS** is equal to **TA/2**, the entire acceleration will be spent in S-curve form (**TS** values greater than **TA/2** override the **TA** value; total acceleration time will be **2TS**.
For **LINEAR** mode moves with PMAC not in segmentation mode (I13=0), if the *rate* of acceleration for any motor in the coordinate system exceeds that motor's maximum as specified by Ix17, the acceleration *time* for all motors is increased so that no motor exceeds its maximum acceleration *rate*.
TS does not affect **RAPID**, **PVT**, or **SPLINE** mode moves, but it stays valid for the next return to blended moves.
Make sure the specified acceleration time (TA or 2*TS) is greater than zero, even if you are planning to rely on the maximum acceleration rate parameters (Ix17). A specified acceleration time of zero will cause a divide-by-zero error. The minimum specified time should be **TA1 TS0**.
In executing the **TS** command, PMAC rounds the specified value to the nearest integer number of milliseconds (there is no rounding done when storing the command in the buffer).
A blended move executed in a program before any **TS** statement will use the default S-curve time specified by coordinate system I-variable Ix88.

Examples

```
TS20

TS(Q17)

TS(39.32+P43)
```

See Also

Linear and Circular Blended Moves (Writing a Motion Program)

I-variables I13, Ix17, Ix21, Ix87, Ix88

Program commands **TA**, **TM**, **F**, **LINEAR**, **CIRCLE**

TSELECT{constant}

Function: ..... Select active transformation matrix for X, Y, and Z axes
Type: .....
..... Motion program (PROG and ROT)
Syntax: ..... **TSELECT{constant}**
Where:.....

{constant} is an integer representing the number of the matrix to be used

Remarks

This command selects the specified matrix for use as the active transformation matrix for the X, Y, and Z axes of the coordinate system running the motion program. This matrix can then be modified using the **TINIT**, **ADIS**, **AROT**, **IDIS**, and **IROT** commands to perform translations, rotations, and scaling of the 3 axes. This matrix will be used until another one is selected.
This matrix must already have been created with the on-line **DEFINE TBUF** command. That command specifies the number of matrices to create, and it must have specified a number at least as high as the number used in **TSEL** (you cannot select a matrix that has not been created).
TSEL0 deselects all transformation matrices, saving calculation time.

Examples

```
DEFINE TBUF 5  ; Create 5 transformation matrices

OPEN PROG 10 CLEAR

...

TSEL 3    ; Select transformation matrix 3 (of 5)

TINIT     ; Make matrix 3 the identity matrix
```

See Also

Axis Matrix Transformations (Writing a Motion Program)
On-line command **DEFINE TBUF**
Program commands **AROT**, **IROT**, **ADIS**, **IDIS**, **TINIT**

U{data}

Function: ..... U-Axis Move
Type: .....
..... Motion program
Syntax: ..... **U{data}**

Where:.....

{data} is a floating point constant or expression representing the position or distance in user units for the U-axis.

Remarks

This command causes a move of the U-axis. (See **{axis}{data}** description, above.)

Examples

```
U10

U(P17+2.345)

X20 U20

U(COS(Q10)) V(SIN(Q10))
```

See Also

Program commands {axis}{data}, A, B, C, V, W, X, Y, Z, CALL, READ

V{data}

Function: ..... V-Axis Move

Type: .....
..... Motion program (PROG and ROT)

Syntax: ..... **V{data}**

Where:.....

{data} is a floating point constant or expression representing the position or distance in user units for the V-axis.

Remarks

This command causes a move of the V-axis. (See **{axis}{data}** description, above.)

Examples

```
V20

U56.5 V(P320)

Y10 V10

V(SQRT(Q20*Q20+Q21*Q21))
```

See Also

Program commands **{axis}{data}**, **A**, **B**, **C**, **U**, **W**, **X**, **Y**, **Z**, **CALL**, **READ**

W{data}

Function:.....W-Axis Move
Type:.....
.....Motion program
Syntax: .....**W{data}**

Where:.....

{data} is a floating point constant or expression representing the position or distance in user units for the W-axis.

Remarks

This command causes a move of the W-axis. (See **{axis}{data}** description, above.)

Examples

```
W5

W(P10+33.5)

Z10 W10

W(ABS(Q22*Q22))
```

See Also

Program commands **{axis}{data}**, **A**, **B**, **C**, **U**, **V**, **X**, **Y**, **Z**, **CALL**, **READ**

WAIT

Function:.....Suspend program execution
Type:.....
.....Motion program (PROG and ROT)
Syntax: .....**WAIT**

Remarks

This command may be used on the same line as a **WHILE** condition to hold up execution of the program until the condition goes false. When the condition goes false, program execution resumes on the next line. Use of the **WAIT** statement allows indefinite pauses without the need for repeated use of a servo command (e.g. **DWELL** or **DELAY**) to 'eat up' the time. However, it is impossible to predict how long the pause will be.

WAIT permits a faster resumption of the program upon the **WHILE** condition going false. Also, the program timer is halted when **WAIT**ing, which allows the "In-position" bit to go true (which can be used to trigger an action, or the next move).

Since PMAC executes a **WHILE ({condition}) WAIT** statement every Real Time Interrupt until the condition goes false, it is essentially the same as a PLC0. This could use excessive processor time and in severe cases trip the watchdog timer on the PMAC that simultaneously run several motion programs that use **WAIT** statements and or large PLC0 programs. For example, if the condition only needs to be checked every 20 msec and not every Real Time Interrupt, you should consider using a **DWELL** command to regulate the execution time of the **WHILE** loop.

```
WHILE ({condition})
     DWELL20
ENDW
```

Examples

```
WHILE (M11=0) WAIT  ; Pause here until Machine Input 1 set
WHILE (M187=0) WAIT ; Pause here until all axes in-position
M1=1                ; Turn on Output 1 to activate punch
```

See Also

I-variable Ix28

Program commands **DWELL**, **DELAY**, **STOP**

WHILE({condition})

Function: Conditional looping

Type: Motion program (PROG only); PLC program

Syntax: **WHILE ({condition})**

WHILE ({condition}) {action}

Where:

{condition} consists of one or more sets of **{expression} {comparator} {expression}**, joined by logical operators **AND** or **OR**.

{action} is a program command

Remarks

This statement allows repeated execution of a statement or series of statements as long as the condition is true. It is the PMAC only looping construct. It can take two forms:

(Valid in motion program only) With a statement following on the same line, it will repeatedly execute that statement as long as the condition is true. No **ENDWHILE** is used to terminate the loop.

```
WHILE ({condition}) {action}
```

(Valid in motion and PLC programs) With no statement following on the same line, it will execute statements on subsequent lines down to the next **ENDWHILE** statement.

```
WHILE ({condition})
     {statement}
     [{statement}
     ...]
ENDWHILE
```

If a **WHILE** loop in a motion program has no move, **DWELL**, or **DELAY** inside, PMAC will attempt to execute the loop twice (while true) each real-time interrupt cycle (stopped from more loops only by the "double-jump-back" rule), much like a PLC0. This can starve the background tasks for time, possibly even tripping the watchdog timer. PMAC will not attempt to blend moves through such an "empty" **WHILE** loop if it finds the loop condition true twice or more.

In PLC programs, extended compound **WHILE** conditions can be formed on multiple program lines through use of **AND** and **OR** commands on the program lines immediately following the **WHILE** command itself (this structure is not available in motion programs). Conditions in each program line can be either simple or compound. **AND** and **OR** operations within a program line take precedence over **AND** and **OR** operations between lines.

Examples

```
WHILE (P20=0)

   ...

ENDWHILE

WHILE (Q10<5 AND Q11>1)

   ...

ENDWHILE

WHILE (M11=0) WAIT  ; sit until input goes true

INC

WHILE (M11=0 OR M12=0) X100; increment until  2 inputs true
```

To do the equivalent of a For/Next loop:

```
P1=0 ; Initialize loop counter

WHILE (P1<10)  ; Loop until counter exceeds limit

         X1000 ; Perform action to be repeated

         P1=P1+1    ; Increment loop counter

ENDWHILE  ; Loop back
```

To do a timed wait in a PLC program, use the servo cycle counter as timer:

```
P90=16777216   ; Counter rollover value (2^24)

P91=M0    ; Store starting value of M0 (X:$0) counter

P92=0     ; Time elapsed so far

WHILE (P92<P93); Loop until past specified time

        P92=(M0-P91)%P90 ; Calculate time elapsed

     ; Modulo (%) operation to handle rollover

ENDWHILE  ; Loop back
```

To do extended compound conditions in a PLC program

```
WHILE (M11=1 AND M12=1)

OR (M13=1 AND M14=1)

AND (P1>0)

   ...

ENDWHILE
```

See Also

Program Logic (Writing a Motion Program, Writing a PLC Program)
How PMAC Executes a Motion Program (Writing a Motion Program)
Program commands **AND**, **OR**, **IF**, **ELSE**, **ENDIF**, **ENDWHILE**

X{data}

Function: ..... X-Axis Move
Type: ..... Motion program
Syntax: ..... **X{data}**

Where:

{data} is a floating point constant or expression representing the position or distance in user units for the X-axis.

Remarks

This command causes a move of the X-axis. (See **{axis}{data}** description, above.)

Examples

```
X10

X15 Y20

X(P1) Y30

X(Q10*COS(Q1)) Y(Q10*SIN(Q1))

X3.76 Z2.92 I0.075 K3.42
```

See Also

Program commands **{axis}{data}**, **A**, **B**, **C**, **U**, **V**, **W**, **Y**, **Z**, **CALL**, **READ**

Y{data}

Function: ..... Y-Axis Move

Type:.....
.....Motion program
Syntax: .....**Y{data}**

Where:.....

{data} is a floating point constant or expression representing the position or distance in user units for the Y-axis.

Remarks

This command causes a move of the Y-axis. (See **{axis}{data}** description, above.)

Examples

```
Y50

Y(P100)

X35 Y75

Y-0.221 Z3.475

Y(ABS(P3+P4)) A(INT(P3-P4))
```

See Also

Program commands **{axis}{data}**, **A**, **B**, **C**, **U**, **V**, **W**, **X**, **Z**, **CALL**, **READ**

Z{data}

Function:.....Z-Axis Move
Type:.....
.....Motion program
Syntax: .....**Z{data}**

Where:.....

{data} is a floating point constant or expression representing the position or distance in user units for the W-axis.

Remarks

This command causes a move of the Z-axis. (See **{axis}{data}** description, above.)

Examples

```
Z20

Z(Q25)

X10 Y20 Z30

Z23.4 R10.5

Z(P301+2*P302/P303)
```

See Also

Program commands **{axis}{data}**, **A**, **B**, **C**, **U**, **V**, **W**, **X**, **Y**, **CALL**, **READ**



PMAC I/O AND MEMORY MAP

Introduction

CAUTION *Certain registers that are under the PMAC automatic control can cause problems if the user writes to them directly -- particularly those used in the servo calculations.*

This guide to the PMAC memory and input and output registers is provided for user reference. The PMAC architecture is very open, allowing the user to examine and use many internal registers for his own use. Usually this is done through the use of M-variables, which point to locations in the memory-I/O space of the PMAC processor. Once defined to point to the proper location, an M-variable can be treated as any other variable for reading and writing.

The PMAC processor is the Motorola 56001 DSP. The 56001 has dual data buses, each 24 bits wide, so that both operands in a calculation may be brought in simultaneously. Each bus has access to a 16-bit address space (0000hex to FFFFhex), which provides 65,536 24-bit words. One bus and address space is called 'X', and the other is called 'Y'. Therefore, when specifying a single-word memory location, one must use 'X:' or 'Y:' with the 16-bit address. the PMAC input and output is mapped into the same address space with the memory.

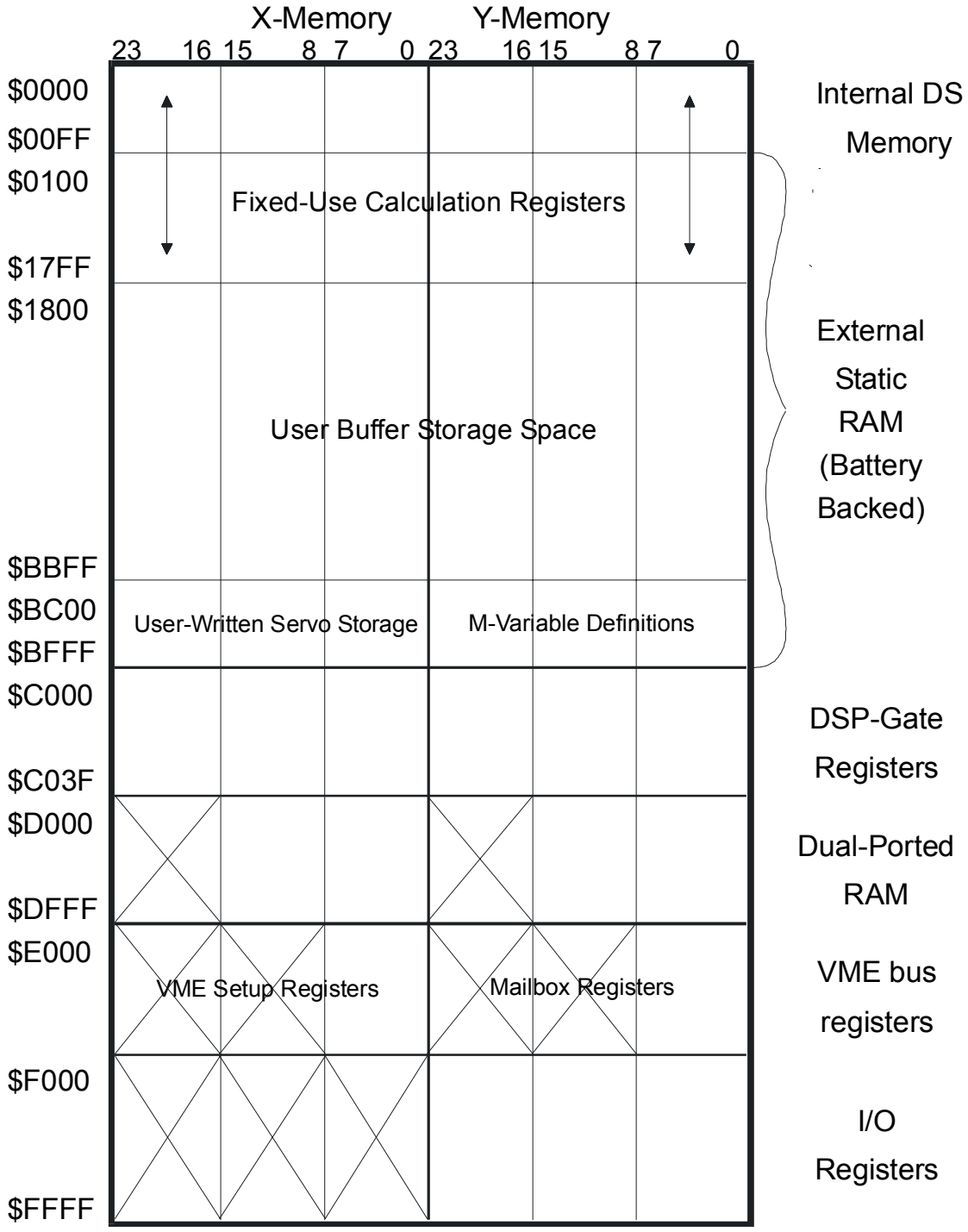
PMAC uses double-word memory for both extended fixed-point values and for floating-point values (single words are always fixed point). In this memory map, the fixed-point double-word locations are specified by a 'D:' (double), and the floating-point double-word locations are specified by an 'L:' (long). This matches the syntax of M-variable declarations for these registers. If an address is specified without any prefix, it means that a special internal format is used to hold data in that word.

The user may specify PMAC addresses with either decimal or hexadecimal values; the hex values must be preceded by a '$' to be interpreted as hex. For example, `Y:$FFC0` is the hexadecimal specification, and `Y:65472` is the decimal specification of the same word address.

M-variables are defined by providing the word address, the offset, the width, and the format (irrelevant for bits). Refer to M-variables in the manual for details of the definition. Several M-variables were defined at the factory to match to inputs and outputs. For instance, M11 thru M18 were assigned to Machine Inputs 1 thru 8 (MI1-MI8), and M1 to M8 were assigned to Machine Outputs 1 thru 8 (MO1-MO8).

Example statements that did this are as follows:

```
M11->Y:$FFC2,0,1   (Mach. In. 1: Offset 0, width 1)
M1->Y:$FFC2,8,1    (Mach. Out. 1: Offset 8, width 1)
M162->D:$002C      (Motor #1 Actual Position: fixed pt.)
M163->L:$081F      (C.S. 1 X-axis Position: floating pt.)
```



PMAC Memory Mapping

Global Servo Calculation Registers

In the listing below, the hexadecimal address is listed first, followed by the decimal address in parentheses.

X:$0000 (0)	Servo interrupt cycle counter (servo cycles)
Y:$0000	Servo count for next real time interrupt

X:$0001 (1) Global I-Variables

Bits	
0,1	I5
2	I2
4, 5	I9
6, 7	I3
10, 11	I4
14, 15	I6
18, 19	I1

Y:$0001 Real time interrupt period minus one (I8)
X:$0002 (2) Data gathering counter (time)
Y:$0002 Data gathering period (I19)
X:$0003 (3) Global servo status bits

(First word returned on **???** command. See **???** in the On-Line Commands section)

Bits	
0	This Card Addressed Serially
1	All Cards Addressed Serially
2-4	(Reserved for future use)
5	MACRO Auxiliary Communications Error
6	TWS Variable Parity Error
7-9	(For internal use)
10	EAROM Error
11	DPRAM Error
12	PROM Checksum Error
13	Any Memory Checksum Error
14	Compensation On
15	(Reserved for future use)
16	(Reserved for future use)
17	Prepared to gather on trigger
18	Prepared to gather next servo
19	Data gathering function on
20	Servo Error
21	Servo Active
22	Real Time Interrupt Re-entry
23	Real Time Interrupt Active

Y:$0003 Global status bits

(Second word returned on **???** command)

Bits	
07	(Reserved for future use)
8-10	(Internal use)
11	Fixed Buffer Full
12-14	(Internal use)
15	VME Communications Mode
16	PLC Command
17	PLC Buffer Open
18	Rotary Buffer Open
19	Motion Program Buffer Open
20-21	(Internal use)
22	Host communication mode
23	(internal use)

D:$0004-$001F Temporary calculation registers

Motor Calculation Registers (PMAC[1] Specific)

The addresses given are for Motor #1. For the registers for another motor x add (x-1)*$3C -- (x-1)*60 -- to the appropriate motor #1 address.) The address table shown every 16 addresses (every 10hex) gives the matching addresses for motors 1-8 to make address calculations easier.

Example

By table, Motor 6 actual position address is $014C + ($28-$20) = $0154. By formula, it is (6-1)*$003C+$0028 = $0154

MOTOR #	1	2	3	4	5	6	7	8
HEX	[$0020]	[$005C]	[$0098]	[$00D4]	[$0110]	[$014C]	[$0188]	[$01C4]
DECIMAL	32	92	152	212	272	332	392	452

D:$0020 (32)	Time left in move (X-register units msec*2 at %100)
D:$0021 (33)	Present desired jerk residual
D:$0022 (34)	Present desired jerk (dA/dt)
D:$0023 (35)	Present desired acceleration residual
D:$0024 (36)	Present desired acceleration (X-register units 6/[Ix08*32] cts/msec^2 at %100); Y is fractional
D:$0025 (37)	Present desired velocity residual
D:$0026 (38)	Present desired velocity (X-register units 3/[Ix08*32] cts/ms at %100); Y is fractional
D:$0027 (39)	Present desired position residual
D:$0028 (40)	Present desired position (1/[Ix08*32] counts)
X:$0029 (41)	Address of position feedback (Ix03)
Y:$0029	Position scaling factor (Ix08)
X:$002A (42)	Address of master (handwheel) register (Ix05)
Y:$002A	Previous actual position value
D:$002B (43)	Present actual position (1/[Ix08*32] counts)
X:$002C (44)	Master (handwheel) scale factor (Ix07)
Y:$002C	Previous master (handwheel) position
D:$002D (45)	Present master (handwheel) position (1/[Ix07*32]cts of the master or 1/[Ix08*32]cts of the slaved motor).
X:$002E (46)	Feedpot (timebase) pointer
D:$002F	Previous desired position

	1	2	3	4	5	6	7	8
HEX	[$0030]	[$006C]	[$00A8]	[$00E4]	[$0120]	[$015C]	[$0198]	[$01D4]
DECIMAL	48	108	168	228	288	348	408	468

X:$0030 (48)	Velocity feedforward gain (Ix32)
Y:$0030	Previous desired velocity
X:$0031 (49)	Address of "velocity" encoder (Ix04)
Y:$0031	Previous "velocity" position
X:$0032 (50)	Acceleration feedforward gain (Ix35)
Y:$0032	"Velocity" scaling factor (Ix09)
X:$0033 (51)	Actual velocity (1/[Ix09*32] cts/[Ix60+1]servo interrupts)

Y:$0033 Derivative gain (Ix31)
X:$0034 (52) Deadband size (Ix65) (1/16 count)
Y:$0034 Position error limit (Ix67) (1/16 count)
X:$0035 (53) "Deadband gain" (Ix64)
Y:$0035 Integral gain (Ix33)
X:$0036 (54) Integrated error residual
Y:$0036 Integrated error limit (Ix63)
D:$0037 (55) Integrated error
X:$0038 (56) Proportional gain (Ix30)
Y:$0038 Filter output (DAC) limit (Ix69)
D:$0039 (Filter intermediate values)
X:$003A (58) Filter result
D:$003B-$003C (Filter intermediate values)
X:$003D (61) Motor servo status bits

(First word returned on **?** command. See Y:$0814 for second word)
(Refer to ? description in on-line commands for detailed description of bits)

Bits	
0-9	(Internal use)
10	Home search in progress
11	Block request
12	Abort deceleration in progress
13	Desired velocity 0
14	Data block error
15	Dwell in progress
16	Integration mode (Ix34; 0 on always; 1 on when desired velocity zero) Running program
17	Open loop mode
18	Phased motor (Ix01)
19	Handwheel enabled (Ix06)
20	Positive end limit set (soft or hard [**-LIM**])
21	Negative end limit set (soft or hard [**+LIM**])
22	Motor activated (Ix00)
23	

Y:$003D Phase address pointer (Ix83)

Bits	
X:$003E (62) 0-7	Phase offset (Ix72)
8-23	2nd phase bias (Ix79)
Y:$003E 0-7	# of commutation cycles per rev (Ix70)
8-23	Filter output/1st phase bias (Ix29)

MOTOR #	1	2	3	4	5	6	7	8
HEX	[$0040]	[$007C]	[$00B8]	[$00F4]	[$0130]	[$016C]	[$01A8]	[$01E4]
DECIMAL	64	124	184	244	304	364	424	484

X:$003F (63) Magnetization current (Ix77)
Y:$003F Previous phase position
X:$0040 (64) Slip frequency
Y:$0040 (64) Counts per n commutation cycles (Ix71) Present phase position (X register units: counts*Ix70, range Ix71/2 - Ix71/2-1)
D:$0041 (65)
X:$0042 (66) Phase advance
Y:$0042 Phase advance gain (Ix76)
X:$0043 (67) Slip Gain (Ix78)
Y:$0043 Phased DAC amplitude

X:$0044 (68)	Command output address (Ix02)
Y:$0044	Velocity Phase Advance Gain (Ix76)
X:$0045 (69)	DAC command value
Y:$0045	Command internal offset
D:$0046 (70)	Compensation correction (1/[Ix08*32] cts)
$0047-$005B	(Reserved for future use)
**********	**********
$005C-$0097 (92-151)	Motor #2 registers (as above)
$0098-$00D3 (152-211)	Motor #3 registers (as above)
$00D4-$010F (212-271)	Motor #4 registers (as above)
$0110-$014B (272-331)	Motor #5 registers (as above)
$014C-$0187 (332-391)	Motor #6 registers (as above)
$0188-$01C3 (392-451)	Motor #7 registers (as above)
$01C4-$01FF (452-511)	Motor #8 registers (as above)
**********	**********
X:$0400-$04FF (1024-1279)	Commutation sine table [2^{23}*SIN((address-$400) *360°/256)]
X:$0600-$06FF (512-767)	Command character queue
Y:$0600-$06FF	Response character queue

Motor Calculation Registers (PMAC2 Specific)

The addresses given are for Motor #1. For the registers for another motor x add (x-1)*$3C -- (x-1)*60 -- to the appropriate motor #1 address.) The address table shown every 16 addresses (every 10hex) gives the matching addresses for motors 1-8 to make address calculations easier.

Example

By table, Motor 6 actual position address is $014C + ($28-$20) = $0154. By formula, it is (6-1)*$003C+$0028 = $0154

MOTOR #	1	2	3	4	5	6	7	8
HEX	[$0020]	[$005C]	[$0098]	[$00D4]	[$0110]	[$014C]	[$0188]	[$01C4]
DECIMAL	32	92	152	212	272	332	392	452

D:$0025 (37)	Present desired velocity residual
D:$0026 (38)	Present desired velocity (X-register units 3/[Ix08*32] cts/msec at %100); Y is fractional
D:$0027 (39)	Present desired position residual
D:$0028 (40)	Present desired position (1/[Ix08*32] counts)
X:$0029 (41)	Address of position feedback (Ix03)

Y:$0029	Position scaling factor (Ix08)
X:$002A (42)	Address of master (handwheel) register (Ix05)
Y:$002A	Previous actual position value
D:$002B (43)	Present actual position (1/[Ix08*32] counts)
X:$002C (44)	Master (handwheel) scale factor (Ix07)
Y:$002C	Previous master (handwheel) position
D:$002D (45)	Present master (handwheel) position (1/[Ix07*32]cts of the master or 1/[Ix08*32]cts of the slaved motor).
X:$002E (46)	Feedpot (timebase) pointer
D:$002F	Previous desired position
X:$0030 (48)	Velocity feedforward gain (Ix32)

MOTOR #	1	2	3	4	5	6	7	8
HEX	[$0030]	[$006C]	[$00A8]	[$00E4]	[$0120]	[$015C]	[$0198]	[$01D4]
DECIMAL	48	108	168	228	288	348	408	468

Y:$0030	Previous desired velocity
X:$0031 (49)	Address of "velocity" encoder (Ix04)
Y:$0031	Previous "velocity" position
X:$0032 (50)	Acceleration feedforward gain (Ix35)
Y:$0032	"Velocity" scaling factor (Ix09)
X:$0033 (51)	Actual velocity (1/[Ix09*32] cts/[Ix60+1]servo interrupts)
Y:$0033	Derivative gain (Ix31)
X:$0034 (52)	Deadband size (Ix65) (1/16 count)
Y:$0034	Position error limit (Ix67) (1/16 count)
X:$0035 (53)	"Deadband gain" (Ix64)
Y:$0035	Integral gain (Ix33)
X:$0036 (54)	Integrated error residual
Y:$0036	Integrated error limit (Ix63)
D:$0037 (55)	Integrated error
X:$0038 (56)	Proportional gain (Ix30)
Y:$0038	Filter output (DAC) limit (Ix69)
D:$0039	(Filter intermediate values)
X:$003A (58)	Filter result
D:$003B-$003C	(Filter intermediate values)
X:$003D (61)	Motor servo status bits

(First word returned on **?** command. See Y:$0814 for second word)
(Refer to ? description in on-line commands for detailed description of bits)

Bits	
0-9	(Internal use)
10	Home search in progress
11	Block request
12	Abort deceleration in progress
13	Desired velocity 0
14	Data block error
15	Dwell in progress
16	Integration mode (Ix34; 0 on always; 1 on when desired velocity zero)

	Bits	
	17	Running program
	18	Open loop mode
	19	Phased motor (Ix01)
	20	Handwheel enabled (Ix06)
	21	Positive end limit set (soft or hard [**-LIM**])
	22	Negative end limit set (soft or hard [**+LIM**])
	23	Motor activated (Ix00)
Y:$003D		Phase address pointer (Ix83)
X:$003E (62)		
	0-7	Phase offset (Ix72)
	8-23	2nd phase bias (Ix79)
Y:$003E		
	0-7	# of commutation cycles per rev (Ix70)
	8-23	Filter output/1st phase bias (Ix2)
X:$003F		Counts per commutation cycle (Ix71)
Y:$003F		Previous phase position

MOTOR #	1	2	3	4	5	6	7	8
HEX	[$0040]	[$007C]	[$00B8]	[$00F4]	[$0130]	[$016C]	[$01A8]	[$01E4]
DECIMAL	64	124	184	244	304	364	424	484

D:$0040	Present phase position (X-register units: counts*Ix70)
X:$0041	ADC mask word (Ix84)
Y:$0041	ADC address (Ix82)
X:$0042	Estimated rotor magnetization current
Y:$0042	Slip gain (Ix78)
X:$0043	Commanded quadrature current (servo command)
Y:$0043	Commanded direct current (Ix77)
X:$0044	Actual quadrature current
Y:$0044	Actual direct current
X:$0045	Current-loop back path proportional gain (Ix76)
Y:$0045	Current-loop forward path proportional gain (Ix62)
X:$0046	Quadrature current loop integrator output
Y:$0046	Direct current loop integrator output
X:$0047	PWM scale factor (Ix66)
Y:$0047	Current-loop integral gain (Ix61)
X:$0048	Command output address (Ix02)
Y:$0048	Velocity node bias
X:$0049	(Quadrature) command value
Y:$0049	(Quadrature) command bias
D:$004A	Compensation position correction (1/[Ix08*32] counts)
$004B-$005B	(Reserved for future use)

****

$005C-$0097 (92-151)	Motor #2 registers (as above)
$0098-$00D3 (152-211)	Motor #3 registers (as above)
$00D4-$010F (212-271)	Motor #4 registers (as above)
$0110-$014B (272-331)	Motor #5 registers (as above)
$014C-$0187	Motor #6 registers (as above)

(332-391)	
$0188-$01C3	Motor #7 registers (as above)
(392-451)	
$01C4-$01FF	Motor #8 registers (as above)
(452-511)	

Encoder Conversion (Interpolation) Table (PMAC[1] Specific)

X:$0700-$0701	User count-down timer registers (servo cycles)
(1792-1793)	
Y:$0700-$0701	User count-down timer registers (servo cycles)
X:$0708-$070F	Auto-converted ACC-36 ADC registers 9-16
(1800-1807)	
Y:$0708-$070F	Auto-converted ACC-36 ADC registers 1-8
X:$0720-$073F	Converted encoder and time base data
(1824-1855)	
Y:$0720-$073F	Encoder conversion source and format

The format of the conversion table is:

	Bits	
X:word	0-4	Fractional bits of converted data
	5-23	Integer bits of converted data (if last entry in conversion) Intermediate value if not last entry in conversion
	16-23	Conversion format: $00 = 1/T incremental encoder $10 = A/D register conversion $20 = Unfiltered parallel Y word source* $30 = Filtered parallel Y word source** $40 = Time base* $50 = Integrated A/D register conversion $60 = Unfiltered parallel X word source* $70 = Filtered parallel X word source** $80 = parallel interp. of incremental $90 = Triggered time base; frozen* $A0 = Triggered time base; running* $B0 = Triggered time base; armed* $C0 = no interp. of incremental encoder $D0 = Exponential filter**
	0-15	Address of source data
	16	

* Next Y word contains user-set constant for conversion (this is a double-entry conversion).

** Next two Y words contain user-set constants for conversion (this is a triple-entry conversion).

Refer to the detailed description of the encoder conversion table under **Feedback Features**.

Encoder Conversion (Interpolation) Table (PMAC2 Specific)

X:$0720-$073F (1824-1855)		Converted encoder and time base data
Y:$0720-$073F		Encoder conversion source and format

The format of the conversion table is:

	Bits	
Y:word	16-23	Conversion format: $00 = 1/T incremental encoder $10 = A/D register conversion $20 = Unfiltered parallel Y word source* $30 = Filtered parallel Y word source** $40 = Time base* $50 = Integrated A/D register conversion $60 = Unfiltered parallel X word source* $70 = Filtered parallel X word source** $80 = parallel interp. of incremental $90 = Triggered time base; frozen* $A0 = Triggered time base; running* $B0 = Triggered time base; armed*
	Bits	
		$C0 = no interp. of incremental encoder $D0 = Exponential filter**
	0-15	Address of source data
X:word	0-4	Fractional bits of converted data
	5-23	Integer bits of converted data (if last entry in conversion) Intermediate value if not last entry in conversion

Refer to the detailed description of the encoder conversion table under "Feedback Features".

General Global Registers

$0770 - $077F (1904-1919)		Open memory; cleared to 0 on power-on/reset (useful for 24-bit M-variables)
Y:$0780 - Y:$07D1 (1920-2001)		LCD Display character memory
X:$0780 (1920)		Dwell (fixed) feedpot (I10)
X:$0781 (1921)		Jog-to-position move delay time (I12)
X:$0782 (1922)		Programmed move delay time (I11)
X:$0783 - X:$078C (1923 - 1932)		VME address and vector values (See VME Inter-face Document for details)
X:$0786 - X:$0787 (1926 - 1927)		PC Dual-ported RAM host address registers (See Option 2 Manual for details)
X:$078D (1933)		Move segmentation time (I13)
X:$078E (1934)		Hold feedpot value (must be zero)
X:$078F (1935)		Addressed motor/coordinate system word
	Bits	
	0-2	Motor number minus 1 (#n-1)
	3-5	Coordinate system number minus 1 (&n-1)
X:$0790 (1936)		Card software address
X:$0794 (1940)		Running software checksum value (frozen if any error)

X:$07B1 (1969)	PROM (Firmware) Reference checksum value
X:$07B2 (1970)	User Program Reference checksum value
X:$07B3 (1971)	Defined buffer checksum value
X:$07B9 (1977)	Data gathering source mask (I20)
X:$07BA - X:$07D1 (1978 - 2001)	Data gathering source addresses (I21-I44)
$07F0 - $07FF (2032 - 2047)	Open registers (useful for creating 24-bit M-variables to handle roll-over of other 24-bit registers); on PMAC's without flash memory (no option CPU) these are stored in battery-backed RAM

* Next Y word contains user-set constant for conversion (this is a double-entry conversion).

** Next two Y words contain user-set constants for conversion (this is a triple-entry conversion).

General Global Registers (PMAC[1] Specific)

$0770 - $077F (1904-1919)	Open memory; cleared to 0 on power-on/reset (useful for 24-bit M-variables)
Y:$0780 - Y:$07D1 (1920-2001)	LCD Display character memory
X:$0780 (1920)	Dwell (fixed) feedpot (I10)
X:$0781 (1921)	Jog-to-position move delay time (I12)
X:$0782 (1922)	Programmed move delay time (I11)
X:$0783 - X:$078C (1923 - 1932)	VME address and vector values (See VME Interface Document for details)
X:$0786 - X:$0787 (1926 - 1927)	PC Dual-ported RAM host address registers (See Option 2 Manual for details)
X:$078D (1933)	Move segmentation time (I13)
X:$078E (1934)	Hold feedpot value (must be zero)
X:$078F (1935)	Addressed motor/coordinate system word

Bits	
0-2	Motor number minus 1 (#n-1)
3-5	Coordinate system number minus 1 (&n-1)

X:$0790 (1936)	Card software address
X:$0794 (1940)	Running software checksum value (frozen if any error)
X:$07B1 (1969)	PROM (Firmware) Reference checksum value
X:$07B2 (1970)	User Program Reference checksum value
X:$07B3 (1971)	Defined buffer checksum value
X:$07B9 (1977)	Data gathering source mask (I20)
X:$07BA - X:$07D1 (1978 - 2001)	Data gathering source addresses (I21-I44)
$07F0 - $07FF (2032 - 2047)	Open registers (useful for creating 24-bit M-variables to handle roll-over of other 24-bit registers); on the PMAC without flash memory (no option CPU) these are stored in battery-backed RAM

General Global Registers (PMAC2 Specific)

X:$0400-$04FF (1024-1279)	Commutation sine table [2^{23}*SIN((address-$400)*360°/256)]
X:$0600-$06FF (512-767)	Command character queue
Y:$0600-$06FF	Response character queue
X:$0700-$0701 (1792-1793)	User count-down timer registers (servo cycles)

Y:$0700-$0701	User count-down timer registers (servo cycles)
X:$0708-$070F (1800-1807)	Auto-converted ACC-36 ADC registers 9-16
Y:$0708-$070F	Auto-converted ACC-36 ADC registers 1-8
X:$0708 - X:$070F	Option 12 Analog data table configuration entries 0 - 7
Y:$0708 - Y:$070F	Option 12 Analog data table results
X:$0710 - X:$0717	Gate Array ADCnA input values (n = 1 to 8)
Y:$0710 - Y:$0717	Gate Array ADCnB input values (n = 1 to 8)

Motor and Coordinate System Status and Control Registers

The addresses given in this section are for Motor 1 and Coordinate System 1. Table headers for each set of 16 registers give the starting addresses of the set for each Motor/C.S. For the register of another Motor x or C.S. x, add (x-1)*$C0 or (x-1)*192 to the appropriate Motor 1 or C.S. 1 address.

Example

For the Motor 4 target position register, by the table compute $0A40 + ($080B-$0800) = $0A4B. By formula, compute (4-1)*$C0 + $080B = $0A4B

MOT/CS	1	2	3	4	5	6	7	8
HEX	[$0800]	[$08C0]	[$0980]	[$0A40]	[$0B00]	[$0BC0]	[$0C80]	[$0D40]
DECIMAL	2048	2240	2432	2624	2816	3008	3200	3392

L:$0800 (2048)	Motor #1 jog speed (Ix22) [floating point]
L:$0801 (2049)	Motor #1 jog max. accel (Ix19) [float. pt.]
X:$0802 (2050)	Motor #1 jog/home S-curve time (Ix21)
Y:$0802	Motor #1 jog/home accel time (Ix20)
X:$0803 (2051)	Motor #1 flag pointer and mode (Ix25)
Y:$0803	Motor #1 fatal following error limit (Ix11)
L:$0804 (2052)	Motor #1 limit/abort accel (Ix15) [flt. pt.]
X:$0805 (2053)	Motor #1 in-position band (Ix28)
Y:$0805	Motor #1 soft following error (Ix12)
X:$0806 (2054)	C.S. 1 present feedpot (units of I10 {1/8,388,608 msec})
X:$0807 (2055)	C.S. 1 desired feedpot pointer (Ix93)
Y:$0807	C.S. 1 present feedpot pointer
X:$0808 (2056)	C.S. 1 host command feedpot [100% when = I10]
Y:$0808	C.S. 1 present feedpot slew rate
X:$0809 (2057)	C.S. 1 feed-hold feedpot slew rate (Ix95)
Y:$0809	C.S. 1 feedpot slew rate (Ix94)
D:$080A (2058)	Motor #1 target velocity
D:$080B (2059)	Motor #1 target position (1/[Ix08*32] cts)

MOT/CS	1	2	3	4	5	6	7	8
HEX	[$0810]	[$08D0]	[$0990]	[$0A50]	[$0B10]	[$0BD0]	[$0C90]	[$0D50]
DECIMAL	2064	2256	2448	2640	2832	3024	3216	3408

X:$0810 (2064)	C.S. 1 move S-curve time
Y:$0810	C.S. 1 move accel time
L:$0811 (2065)	C.S. 1 move feedrate or time [flt. pt.]

L:$0812 (2066)		C.S. 1 feedrate time units (Ix90) [flt. pt.]
D:$0813 (2067)		Motor #1 position bias (1/[Ix08*32] cts)
X:$0814 (2068)		Motor #1 home offset position (Ix26)
Y:$0814		Motor #1 status bits (Second word returned on **?** command. See
Y:$0814		X:$003D for first word) (Refer to **?** specification in On-line Commands for detailed description of bit meanings)
	Bits	
	0	In-position true
	1	Warning following error limit exceeded
	2	Fatal following error limit exceeded
	3	Amplifier fault error
	4	Backlash direction flag
	5	I^2T Amplifier fault error
	6	Integrated fatal following error
	7	Trigger move
	8	Phasing search error
	9	(Reserved for future use)
	10	Home complete
	11	Stopped on position limit
	12-13	(Reserved for future use)
	Bits	
	14	Amplifier enabled
	15-19	(Reserved for future use)
	20-22	Number of C.S. defined in (-1)
	23	Assigned to C.S.
X:$0815 (2069)		Motor #1 phase finding torque and time (Ix73, Ix74)
Y:$0815		Motor #1 encoder home position offset (cts)
X:$0816 (2070)		C.S. 1 default program # (Ix91)
X:$0817 (2071)		C.S. 1 program execution address pointer
Y:$0817		C.S. 1 program execution status (Second word returned on **??** command. See X:$0818 for first word) (Refer to **??** specification in On-Line Commands for detailed description of bit meanings)
	Bits	
	0	CIRCLE/SPLINE move mode
	1	CCW circular move direction
	2	Cutter compensation on
	3	Cutter compensation left
	4	PVT/SPLINE move mode
	5	Segmented move stop request
	6	Segmented move acceleration in progress
	7	Segmented move in progress
	8	Pre Jog move flag
	9	Cutter compensation move buffered
	10	Cutter compensation move stop request
	11	Cutter compensation outside corner
	12	Dwell move buffered
	12	Synchronous M-variable one-shot
	14	End-of-block (/) stop in progress
	15	Delayed calculation flag
	16	Rotary buffer full
	17	In-position true (logical AND of motor bits)
	18	Warning following error (logical OR of motor bits)
	19	Fatal following error (logical OR of motor bits)
	20	Amplifier fault error (logical OR of motor bits)

	21	Circle radius error
	22	Run time error
	23	Program hold (\) in progress

`X:$0818` (2072) Coordinate System 1 Status/Control Bits (First word returned on **??** command. See `Y:$0817` for first word.) (Refer to **??** specification in On-Line Commands for detailed bit meanings)

Bits	
0	Program running
1	Single step mode
2	Continuous motion mode
3	Move-specified-by-time mode (not speed)
4	Continuous motion request
5	Radius vector incremental mode
6	A-axis incremental mode
7	A-axis used in feedrate calculations
8	B-axis incremental mode
9	B-axis used in feedrate calculations
10	C-axis incremental mode
11	C-axis used in feedrate calculations
12	U-axis incremental mode

Bits	
13	U-axis used in feedrate calculations
14	V-axis incremental mode
15	V-axis used in feedrate calculations
16	W-axis incremental mode
17	W-axis used in feedrate calculations
18	X-axis incremental mode
19	X-axis used in feedrate calculations
20	Y-axis incremental mode
21	Y-axis used in feedrate calculations
22	Z-axis incremental mode
23	Z-axis used in feedrate calculations

`Y:$0818` (2072) C.S. 1 Motor Definition word The Motor Definition word is divided into 8 groups of 3 bits. Each bit group reports a motor's assignment as shown below.

Bits	
0-2	Motor 1 assignment
3-5	Motor 2 assignment
6-8	Motor 3 assignment
9-11	Motor 4 assignment
12-14	Motor 5 assignment
15-17	Motor 6 assignment
18-20	Motor 7 assignment
21-23	Motor 8 assignment

Where the motor's assignment is determined by the value of its bit group (3 bits can have a value from 0 to 7).

Value	
0	Not Assigned
1	Assigned to A-axis
2	Assigned to B-axis
3	Assigned to C-axis
4	Assigned to UVW-axes
5	Reserved

	6	Reserved
	7	Assigned to XYZ-axes
L:$0819 (2073)		C.S. 1 A-Axis desired move position (f.p.)
L:$081A (2074)		C.S. 1 B-Axis desired move position (f.p.)
L:$081B (2075)		C.S. 1 C-Axis desired move position (f.p.)
L:$081C (2076)		C.S. 1 U-Axis desired move position (f.p.)
L:$081D (2077)		C.S. 1 V-Axis desired move position (f.p.)
L:$081E (2078)		C.S. 1 W-Axis desired move position (f.p.)
L:$081F (2079)		C.S. 1 X-Axis desired move position (f.p.)

MOT/CS	1	2	3	4	5	6	7	8
HEX	[$0820]	[$08E0]	[$09A0]	[$0A60]	[$0B20]	[$0BE0]	[$0CA0]	[$0D60]
DECIMAL	2080	2272	2462	2656	2848	3040	3232	3424

L:$0820 (2080)	C.S. 1 Y-Axis desired move position (f.p.)
L:$0821 (2081)	C.S. 1 Z-Axis desired move position (f.p.)
L:$0822 (2082)	Motor #1 X/U/A/B/C-Axis coefficient (f.p.)
L:$0823 (2083)	Motor #1 Y/V-Axis coefficient (f.p.)
L:$0824 (2084)	Motor #1 Z/W-Axis coefficient (f.p.)
L:$0825 (2085)	Motor #1 axis offset (f.p.)
L:$082A (2090)	Motor #1 filtered actual velocity (1/[Ix09*32] cts/servo cycle)
L:$082B (2091)	Motor #1 variable jog position/distance (for J=*, J^*, J:*)

MOT/CS	1	2	3	4	5	6	7	8
HEX	[$0830]	[$08F0]	[$09B0]	[$0A70]	[$0B30]	[$0BF0]	[$0CB0]	[$0D70]
DECIMAL	2096	2288	2480	2672	2864	3056	3248	3440

L:$083E (2110)	Cutter compensation radius
L:$083F (2111)	C.S. 1 Circle radius error limit (Ix96)

MOT/CS	1	2	3	4	5	6	7	8
HEX	[$0840]	[$0900]	[$09C0]	[$0A80]	[$0B40]	[$0C00]	[$0CC0]	[$0D80]
DECIMAL	2112	2304	2496	2688	2880	3072	3264	3456

D:$0840 (2112)	Motor #1 following error (1/[Ix08*32] cts)
L:$0841 (2113)	Motor #1 home speed (Ix23) [floating point]
D:$0842 (2114)	Motor #1 positive software overtravel limit (Ix13)
D:$0843 (2115)	Motor #1 negative software overtravel limit (Ix14)
D:$0844 (2116)	Motor #1 position rollover range (Ix27)
X:$0845 (2117)	Motor #1 backlash slew rate (Ix85)
Y:$0845 (2117)	Motor #1 backlash deadband (from I99)
D:$0846 (2118)	Motor #1 saved backlash position
X:$0847 (2119)	Motor #1 backlash size (Ix86)
Y:$0847 (2119)	Motor #1 present backlash

MOT/CS	1	2	3	4	5	6	7	8
HEX	[$08B0]	[$0970]	[$0A30]	[$0AF0]	[$0BB0]	[$0C70]	[$0D30]	[$0DF0]
DECIMAL	2224	2416	2608	2800	2992	3184	3376	3568

$08B0 - $08BF (2224 - 2239)	C.S. 1 subroutine stack
$08C0 - $097F (2240 - 2431)	Motor/Coordinate System 2 Status and Control Registers (equivalent to $0800 - $08BF above)
$0980 - $0A3F (2432 - 2623)	Motor/Coordinate System 3 Status and Control Registers (equivalent to $0800 - $08BF above)
$0A40 - $0AFF (2624 - 2815)	Motor/Coordinate System 4 Status and Control Registers (equivalent to $0800 - $08BF above)
$0B00 - $0BBF (2816 - 3007)	Motor/Coordinate System 5 Status and Control Registers (equivalent to $0800 - $08BF above)
$0BC0 - $0C7F (3008 - 3199)	Motor/Coordinate System 6 Status and Control Registers (equivalent to $0800 - $08BF above)
$0C80 - $0D3F (3200 - 3391)	Motor/Coordinate System 7 Status and Control Registers (equivalent to $0800 - $08BF above)
$0D40 - $0DFF (3392 - 3583)	Motor/Coordinate System 8 Status and Control Registers (equivalent to $0800 - $08BF above)

Buffer Management Registers (PMAC[1] Specific)

X:$0E00 (3584)	1st motion program number (low 16 bits) and entry status (high 8 bits)
Y:$0E00	1st motion program buffer storage address
X:$0E01 - X:$0EFF (3585 - 3839)	2nd to 256th program # and entry status
Y:$0E01 - Y:$0EFF	2nd to 256th program buffer storage address
X:$0F00 (3840)	PLC 0 execution address
Y:$0F00	PLC 0 buffer storage address
X:$0F01 - X:$0F1F	PLC 1 - 31 execution address
Y:$0F01 - Y:$0F1F (3841-3871)	PLC 1 - 31 storage pointer

Bits	
0-15	PLC base address
22	PLC disabled

X:$0F20 (3872)	Data gather buffer start address
Y:$0F20	Data gather buffer storage address
X:$0F21 - X:$0F28 (3873-3880)	Rotary buffer 1 - 8 start address
Y:$0F21 - Y:$0F28	Rotary buffer 1 - 8 storage address
X:$0F29 (3881)	Transformation matrix buffer start address
Y:$0F29	Transformation matrix buffer storage address
X:$0F2A - X:$0F31 (3882-3889)	Motor 1-8 backlash comp table start address
Y:$0F2A - Y:$0F31	Motor 1-8 backlash comp table storage address
X:$0F32 - X:$0F39 (3890-3897)	Motor 1-8 torque comp table start address
Y:$0F32 - Y:$0F39	Motor 1-8 torque comp table storage address
X:$0F3A - X:$0F41 (3898-3905)	Motor 1-8 leadscrew comp table start address

Y:$0F3A - Y:$0F41	Motor 1-8 leadscrew comp table storage address
X:$0F42	Start address of data gather buffer
L:$1000 (4096)	Variable P0 (floating point)
L:$1001 - L:$13FF (4097 - 5119)	Variables P1 - P1023 (floating point)
L:$1400 (5120)	Variable Q0 (floating point)
L:$1401 - L:$17FF (5121 - 6143)	Variables Q1 - Q1023 (floating point)
$1800 (6144)	Start of buffer storage
$09FF (40959)	End of buffer storage
X:$BC00 (48128)	User written servo storage
Y:$BC00 (48128)	Variable M0 definition
Y:$BC01 - $BFFF (48129-49151)	Variable M1 - M1023 definitions

Buffer Management Registers (PMAC2 Specific)

X:$0E00 (3584)	1st motion program number (low 16 bits)and entry status (high 8 bits)
Y:$0E00	1st motion program buffer storage address
X:$0E01 - X:$0EFF (3585 - 3839)	2nd to 256th program # and entry status
Y:$0E01 - Y:$0EFF	2nd to 256th program buffer storage address
X:$0F00 (3840)	PLC 0 execution address
Y:$0F00	PLC 0 buffer storage address
X:$0F01 - X:$0F1F	PLC 1 - 31 execution address
Y:$0F01 - Y:$0F1F (3841-3871)	PLC 1 - 31 storage pointer

Bits	
0-15	PLC base address
22	PLC disabled

X:$0F20 (3872)	Data gather buffer start address
Y:$0F20	Data gather buffer storage address
X:$0F21 - X:$0F28 (3873-3880)	Rotary buffer 1 - 8 start address
Y:$0F21 - Y:$0F28	Rotary buffer 1 - 8 storage address
X:$0F29 (3881)	Transformation matrix buffer start address
Y:$0F29	Transformation matrix buffer storage address
X:$0F2A - X:$0F31 (3882-3889)	Motor 1-8 backlash comp table start address
Y:$0F2A - Y:$0F31	Motor 1-8 backlash comp table storage address
X:$0F32 - X:$0F39 (3890-3897)	Motor 1-8 torque comp table start address
Y:$0F32 - Y:$0F39	Motor 1-8 torque comp table storage address
X:$0F3A - X:$0F41 (3898-3905)	Motor 1-8 leadscrew comp table start address
Y:$0F3A - Y:$0F41	Motor 1-8 leadscrew comp table storage address
X:$0F42	Start address of data gather buffer
X:$0F70 - X:$0F7F	MACRO Node Auxiliary Output Register Image (n= 0 to 15)
X:$0F70 - X:$0F7F	MACRO Node Auxiliary Output Register Image (n= 0 to 15)
L:$1000 (4096)	Variable P0 (floating point)
L:$1001 - L:$13FF (4097 - 5119)	Variables P1 - P1023 (floating point)
L:$1400 (5120)	Variable Q0 (floating point)
L:$1401 - L:$17FF (5121 - 6143)	Variables Q1 - Q1023 (floating point)

$1800 (6144)	Start of buffer storage
$09FF (40959)	End of buffer storage
X:$BC00 (48128)	User written servo storage
Y:$BC00 (48128)	Variable M0 definition
Y:$BC01 - $BFFF (48129-49151)	Variable M1 - M1023 definitions

Gate Array Registers (PMAC[1] Specific)

The registers in the PMAC "DSPGATE" Gate-Array ICs are mapped into the memory space of the PMAC processor. Each DSPGATE contains four consecutively numbered channels; there may be up to 4 DSPGATEs in a PMAC system, for up to 16 channels. Every PMAC contains DSPGATE 1, which has channels 1 through 4. If Option 1 is ordered (not available on PMAC-Lite), DSPGATE 2 is provided, which has channels 5 through 8. If Accessory 24 is ordered (not available on PMAC-STD), DSPGATE 3 is provided, which has channels 9 through 12. If Accessory 24 Option 1 is ordered as well (not available on PMAC-STD), DSPGATE 4 is provided, which has channels 13 through 16.

ENC #	1	2	3	4	5	6	7	8
HEX	[$C000]	[$C004]	[$C008]	[$C00C]	[$C010]	[$C014]	[$C018]	[$C01C]
DECIMAL	49152	49156	49160	49164	49168	49172	49176	49180
ENC #	9	10	11	12	13	14	15	16
HEX	[$C020]	[$C024]	[$C028]	[$C02C]	[$C030]	[$C034]	[$C038]	[$C03C]
DECIMAL	49184	49188	49192	49196	49200	49204	49208	49212

Y:$Cxxx — Time between last two encoder counts (SCLK cycles)

X:$Cxxx — Encoder Status/Control Bits

Bits 0-15: control;
Bits 16-23: status

0-3	Decode control (Encoder I-Variable 0)
4-7	Position capture control (Encoder I-Variable 2)
8-9	Flag select control (Encoder I-Variable 3)
10	Count write enable
11	Compare equal flag latch control
12	Compare-equal output enable
13	EQU output invert enable
14	OUT value (AENAn)
15	Digital delay filter disable (Encoder I-Variable1)
16	Compare-equal flag
17	Position-captured flag
18	Count error flag
19	Encoder C Channel Status
20	Flag 1 Status (HMFLn)
21	Flag 2 Status (-LIMn)
22	Flag 3 Status (+LIMn)
23	Flag 4 Status (FAULTn)

ENC #	1	2	3	4	5	6	7	8
HEX	[$C001]	[$C005]	[$C009]	[$C00D]	[$C011]	[$C015]	[$C019]	[$C01D]
DECIMAL	49153	49157	49161	49165	49169	49173	49177	49181

ENC #	9	10	11	12	13	14	15	16
HEX	[$C021]	[$C025]	[$C029]	[$C02D]	[$C031]	[$C035]	[$C039]	[$C03D]
DECIMAL	49185	49189	49193	49197	49201	49205	49209	49213

`Y:$Cxxx` Time since last encoder count (SCLK cycles)
`X:$Cxxx` Encoder phase position (counts)

DAC #	2	4	6	8	10	12	14	16
HEX	[$C002]	[$C00A]	[$C012]	[$C01A]	[$C022]	[$C02A]	[$C032]	[$C03A]
DECIMAL	49154	49162	49170	49178	49186	49194	49202	49210

`Y:$Cxxx` DAC output value (high 16 bits)

ADC #	1	3	5	7	9	11	13	15
HEX	[$C006]	[$C00E]	[$C016]	[$C01E]	[$C026]	[$C02E]	[$C036]	[$C03E]
DECIMAL	49158	49166	49174	49182	49190	49198	49206	49214

`Y:$Cxxx` ADC input value (high 16 bits)

ENC #	1	2	3	4	5	6	7	8
HEX	[$C002]	[$C006]	[$C00A]	[$C00E]	[$C012]	[$C016]	[$C01A]	[$C01E]
DECIMAL	49154	49158	49162	49166	49170	49174	49178	49182

ENC #	9	10	11	12	13	14	15	16
HEX	[$C022]	[$C026]	[$C02A]	[$C02E]	[$C032]	[$C036]	[$C03A]	[$C03E]
DECIMAL	49186	49190	49194	49198	49202	49206	49210	49214

`X:$Cxxx` Encoder servo position (2*counts; LSB is direction)

DAC #	1	3	5	7	9	11	13	15
HEX	[$C003]	[$C00B]	[$C013]	[$C01B]	[$C023]	[$C02B]	[$C033]	[$C03B]
DECIMAL	49155	49163	49171	49179	49187	49195	49203	49211

`Y:$Cxxx` DAC output value (high 16 bits)

ADC #	2	4	6	8	10	12	14	16
HEX	[$C007]	[$C00F]	[$C017]	[$C01F]	[$C027]	[$C02F]	[$C037]	[$C03F]
DECIMAL	49159	49167	49175	49183	49191	49199	49207	49215

`Y:$Cxxx` ADC input value (high 16 bits)

	1	2	3	4	5	6	7	8
HEX	[$C003]	[$C007]	[$C00B]	[$C00F]	[$C013]	[$C017]	[$C01B]	[$C01F]
DECIMAL	49155	49159	49163	49167	49171	49175	49179	49183

ENC #	9	10	11	12	13	14	15	16
HEX	[$C023]	[$C027]	[$C02B]	[$C02F]	[$C033]	[$C037]	[$C03B]	[$C03F]
DECIMAL	49187	49191	49195	49199	49203	49207	49211	49215

□`X:$Cxxx` Encoder Capture/Compare position (Capture register is read-only; compare register is write-only)

DSPGATE1 Registers (PMAC2 Specific)

Remarks

Channels 1-4, residing in the first DSPGATE1 IC, are present on all PMAC2 boards.

Channels 5-8, residing in the second DSPGATE1 IC, are present on PMAC2 boards with Option 1.

Channels 9-12, residing in the third DSPGATE1 IC, are present on PMAC2 systems with a 4-channel PMAC2 ACC-24 daughter board (not available as of this writing).

Channels 13-16, residing in the fourth DSPGATE1 IC, are present on on PMAC2 systems with an 8-channel ACC-24 daughter board (not available as of this writing).

ENC #	1	2	3	4	5	6	7	8
HEX	**[$C000]**	**[$C008]**	**[$C010]**	**[$C018]**	**[$C020]**	**[$C028]**	**[$C030]**	**[$C038]**
DECIMAL	49152	49160	49168	49176	49184	49192	49200	49208

ENC #	9	10	11	12	13	14	15	16
HEX	**[$C040]**	**[$C048]**	**[$C050]**	**[$C058]**	**[$C060]**	**[$C068]**	**[$C070]**	**[$C078]**
DECIMAL	49216	49224	49232	49240	49248	49256	49264	49272

`Y:$Cxxx` Channel n Time between last two encoder counts (SCLK cycles)

`X:$Cxxx` Channel n Status Word

Bits	
0-2	Capture Hall Effect Device State
3-7	Reserved for future use (reports as 0)
8	Encoder Count Error (0 on counter reset, 1 on illegal transition)
9	Position Compare (EQUn) output value
10	Position-Captured-On-Gated-Index Flag (=0 on read of captured position register, =1 on trigger capture)
11	Position-Captured Flag (on any trigger) (=0 on read of captured position register, =1 on trigger capture)

Bits

12	Encoder Channel A (CHAn) Input Value
13	Encoder Channel B (CHBn) Input Value
14	Encoder Channel C (Index, CHCn) Input Value (ungated)
15	Amplifier Fault (FAULTn) Input Value
16	Home Flag (HMFLn) Input Value
17	Positive End Limit (PLIMn) Input Value
18	Negative End Limit (MLIMn) Input Value
19	User Flag (USERn) Input Value
20	FlagWn Input Value
21	FlagVn Input Value
22	FlagUn Input Value
23	FlagTn Input Value

ENC #	1	2	3	4	5	6	7	8
HEX	[$C001]	[$C009]	[$C011]	[$C019]	[$C021]	[$C029]	[$C031]	[$C039]
DECIMAL	49153	49161	49169	49177	49185	49193	49201	49209

ENC #	9	10	11	12	13	14	15	16
HEX	[$C041]	[$C049]	[$C051]	[$C059]	[$C061]	[$C069]	[$C071]	[$C079]
DECIMAL	49217	49225	49233	49241	49249	49257	49265	49273

`Y:$Cxxx` Channel n Time since last encoder count (SCLK cycles)
`X:$Cxxx` Channel n Encoder phase position (counts)

ENC #	1	2	3	4	5	6	7	8
HEX	[$C002]	[$C00A]	[$C012]	[$C01A]	[$C022]	[$C02A]	[$C032]	[$C03A]
DECIMAL	49154	49162	49170	49178	49186	49194	49202	49210

ENC #	9	10	11	12	13	14	15	16
HEX	[$C042]	[$C04A]	[$C052]	[$C05A]	[$C062]	[$C06A]	[$C072]	[$C07A]
DECIMAL	49218	49226	49234	49242	49250	49258	49266	49274

`Y:$Cxxx` Channel n Output A Command Value

Bits

8-23:	PWM Command Value
6-23:	Serial DAC Command Value
0-5:	Not Used

`X:$Cxxx` Channel n Encoder Servo Position Capture Register

Bits

0:	Direction of last count (0=up, 1=down)
1-23:	Position counter (units of counts)

	1	2	3	4	5	6	7	8
HEX	[$C003]	[$C00B]	[$C013]	[$C01B]	[$C023]	[$C02B]	[$C033]	[$C03B]
DECIMAL	49155	49163	49171	49179	49187	49195	49203	49211

ENC #	9	10	11	12	13	14	15	16
HEX	[$C043]	[$C04B]	[$C053]	[$C05B]	[$C063]	[$C06B]	[$C073]	[$C07B]
DECIMAL	49219	49227	49235	49243	49251	49259	49267	49275

`Y:$Cxxx` Channel n Output B Command Value

Bits

8-23: PWM Command Value
6-23: Serial DAC Command Value
0-5: Not used

`X:$Cxxx` Channel n Flag Position Capture Value; 24 bits, units of counts

ENC #	1	2	3	4	5	6	7	8
HEX	[$C004]	[$C00C]	[$C014]	[$C01C]	[$C024]	[$C02C]	[$C034]	[$C03C]
DECIMAL	49156	49164	49172	49180	49188	49196	49204	49212

ENC #	9	10	11	12	13	14	15	16
HEX	[$C044]	[$C04C]	[$C054]	[$C05C]	[$C064]	[$C06C]	[$C074]	[$C07C]
DECIMAL	49220	49228	49236	49244	49252	49260	49268	49276

`Y:$Cxxx` Channel n Output C Command Value

Bits

8-23: PWM Command Value
0-23: PFM Command Value

`X:$Cxxx` IC Global Control Word. Channel 1: X:$C004; Channel 5: X:$C024; Channel 9: X:$C044; Channel 13: X:$C064: Clock Control Word (X:$C004 controls channels 1-4; X:$C024 controls channels 5-8) (X:$C044 controls channels 9-12; X:$C064 controls channels 13-16) (X:$C004 bits 0-11 is I903; X:$C024 bits 0-11 is I907)

0-2: SCLK Frequency Control n (f=39.3216MHz / 2^n, n=0-7)
3-5: PFM Clock Frequency Control n (f=39.3216MHz / 2^n, n=0-7)
6-8:
9-11: DAC Clock Frequency Control n (f=39.3216MHz / 2^n, n=0-7)
12: ADC Clock Frequency Control n (f=39.3216MHz / 2^n, n=0-7) Phase Clock Direction (0=output, 1=input) (This must be 0 in X:$C004; 1 in X:$C024--if 2nd ASIC is used)
13: Servo Clock Direction (0=output, 1=input) (This must be 0 in X:$C004; 1 in X:$C024--if 2nd ASIC is used)
14-15: Reserved for future use (report as zero) (X:$C004 bits 16-19 is I901)

Bits

16-19: Phase Clock Frequency Control n (f=MAXPHASE / [n+1], n=0-15) (value in X:$C024 not used) (X:$C004 bits 20-23 is I902)

20-23: Servo Clock Frequency Control n (f=PHASE / [n+1], n=0--15) (value in X:$C024 not used)
Channel 2: X:$C00C; Channel 6: X:$C02C;
Channel 10: X:$C04C; Channel 14: X:$C06C:
DAC Strobe Word, 24 bits
(X:$C00C controls channels 1-4 [I905]; X:$C02C controls channels 5-8 [I909])
(X:$C04C controls channels 9-12; X:$C06C controls channels 13-16)
(Shifted out MSB first one bit per DAC_CLK cycle, starting on rising edge of phase clock)
Channel 3: X:$C014; Channel 7: X:$C034;
Channel 11: X:$C054; Channel 15: X:$C074:
ADC Strobe Word, 24 bits
(X:$C014 controls channels 1-4; X:$C034 controls channels 5-8)
(X:$C054 controls channels 1-4; X:$C074 controls channels 5-8)
(Shifted out MSB first one bit per ADC_CLK cycle, starting on rising edge of phase clock)
Channel 4: X:$C01C; Channel 8: X:$C03C;
Channel 12: X:$C01C; Channel 16: X:$C03C:
PWM, PFM, MaxPhase Control Word
(X:$C01C controls channels 1-4; X:$C03C controls channels 5-8)
(X:$C05C controls channels 1-4; X:$C07C controls channels 5-8)
(X:$C01C bits 0-7 is I904; X:$C03C bits 0-7 is I908)

Bits

0-7: PWM Dead Time (16*PWM CLK cycles) also PFM pulse width (PFM CLK cycles) (X:$C01C bits 8-23 is I900; X:$C03C bits 8-23 is I906)

8-23: PWM MaxCount Value
PWM Frequency = 117.9648 MHz / [4*MaxCount + 6]
"MaxPhase" Frequency = 2*PWM Frequency = 117.9648 MHz / [2*MaxCount + 3]

	1	2	3	4	5	6	7	8
HEX	[$C005]	[$C00D]	[$C015]	[$C01D]	[$C025]	[$C02D]	[$C035]	[$C03D]
DECIMAL	49157	49165	49173	49181	49189	49197	49205	49213

ENC #	9	10	11	12	13	14	15	16
HEX	[$C045]	[$C04D]	[$C055]	[$C05D]	[$C065]	[$C06D]	[$C075]	[$C07D]
DECIMAL	49221	49229	49237	49245	49253	49261	49269	49277

Y:$Cxxx		Channel n ADC A Input Value
	Bits	
	6-23:	Serial ADC Value
	0-5:	Not used
X:$Cxxx		Channel n Control Word
	Bits 0-3:	I9n0
	0-1:	Encoder Decode Control
		00: Pulse and direction decode
		01: x1 quadrature decode
		10: x2 quadrature decode
		11: x4 quadrature decode
	2-3:	Direction & Timer Control
		00: Standard timer control, external signal source, no inversion
		01: Standard timer control, external signal source, invert direction
		10: Standard timer control, internal PFM source, no inversion
		11: Alternate timer control, external signal source
	Bits 4-7:	I9n2
	4-5:	Position Capture Control
		00: Software capture (by setting bit 6)
		01: Use encoder index alone
		10: Use capture flag alone
		11: Use encoder index and capture flag
	6:	Index Capture Invert Control (0=no inversion, 1=inversion)
	7:	Flag Capture Invert Control (0=no inversion, 1=inversion)
	8-9:	Capture Flag Select Control (I9n3)
		00: Home Flag (HMFLn)
		01: Positive End Limit (PLIMn)
		10: Negative End Limit (MLIMn)
		11: User Flag (USERn)
	10:	Encoder Counter Reset Control (1=reset)
	11:	Position Compare Initial State Write Enable
	12:	Position Compare Initial State Value
	13:	Position Compare Channel Select (I9n1) (0= use this channel's encoder; 1=use first encoder on IC)
	14:	AENAn output value
	15:	Gated Index Select for Position Capture (I9n4) (0=ungated index, 1=gated index)
	16:	Invert AB for Gated Index (I9n5) (0: Gated Signal=A&B&C; 1: Gated Signal=A/&B/&C)
	17-18:	Reserved for future use (report as 0)
	19:	Invert PFM Direction Control (0=no inversion, 1=invert) (I9n8)
	Bits 20-21:	I9n7
	20:	Invert A & B Output Control (0=no inversion, 1=invert)
	21:	Invert C Output Control (0=no inversion, 1=invert) (Bits 22-23: I9n6)
	22:	Output A & B Mode Select (0=PWM, 1=DAC)
	23:	Output C Mode Select (0=PWM, 1=PFM)

	1	2	3	4	5	6	7	8
HEX	**[$C006]**	**[$C00E]**	**[$C016]**	**[$C01E]**	**[$C026]**	**[$C02E]**	**[$C036]**	**[$C03E]**
DECIMAL	49158	49166	49174	49182	49190	49198	49206	49214

ENC #	9	10	11	12	13	14	15	16
HEX	**[$C046]**	**[$C04E]**	**[$C056]**	**[$C05E]**	**[$C066]**	**[$C06E]**	**[$C076]**	**[$C07E]**
DECIMAL	49222	49230	49238	49246	49254	49262	49270	49278

`Y:$Cxxx` Channel n ADC B Input Value

Bits

6-23: Serial ADC Value

0-5: Not used

`X:$Cxxx` Channel n Encoder Compare Auto-increment value (24 bits, units of counts)

	1	2	3	4	5	6	7	8
HEX	**[$C007]**	**[$C00F]**	**[$C017]**	**[$C01F]**	**[$C027]**	**[$C02F]**	**[$C037]**	**[$C03F]**
DECIMAL	49159	49167	49175	49183	49191	49199	49207	49215

ENC #	9	10	11	12	13	14	15	16
HEX	**[$C047]**	**[$C04F]**	**[$C057]**	**[$C05F]**	**[$C067]**	**[$C06F]**	**[$C077]**	**[$C07F]**
DECIMAL	49223	49231	49239	49247	49255	49263	49271	49279

`Y:$Cxxx` Channel n Encoder Compare A Value (24 bits, units of counts)

`X:$Cxxx` Channel n Encoder Compare B Value (24 bits, units of counts)

DSPGATE2 Registers (PMAC2 Specific)

`Y:$C080` JI/O Port Data Register (Input or output; when used as general I/O; see Y:$C084)

Bits

0 I/O00 Data Value

...

23 I/O23 Data Value

JI/O Port Data Direction Control Register (when used as general I/O; see Y:$C084)

0 I/O00 Direction Control

...

23 I/O23 Direction Control (All bits: 0=Input; 1=Output)

`Y:$C081` JI/O Port Data Register (Input or output; when used as general I/O; see Y:$C085)

	Bits	
	0	I/O24 Data Value
	...	
	7	I/O31 Data Value
	8	I/O24 Latched Data Value
	...	
	15	I/O31 Latched Data Value
	16-23	Not used
X:$C081		JI/O Port Data Direction Control Register (when used as general I/O; see Y:$C085)
	0	I/O24 Direction Control
	...	
	7	I/O31 Direction Control (All bits: 0=Input; 1=Output)
	8-23	Not used
Y:$C082		JTHW Port Data Register (Input or output; when used as general I/O; see Y:$C086)
	0	DAT0 Data Value
	...	
	7	DAT7 Data Value
	8	SEL0 Data Value
	...	
	Bits	
	15	SEL7 Data Value
	16-23	Not used
X:$C082		JTHW Port Data Direction Control Register (when used as general I/O; see Y:$C086)
	0	DAT0 Direction Control (Must be 0 to use standard port accessories)
	...	
	7	DAT7 Direction Control (Must be 0 to use standard port accessories)
	8	SEL0 Direction Control (Must be 1 to use standard port accessories)
	...	
	15	SEL7 Direction Control (Must be 1 to use standard port accessories) (All bits: 0=Input; 1=Output)
	16-23	Not used
Y:$C083		JDISP, J??? Port Data Register
	Bits	
	0	DISP0 Data Value
	...	
	7	DISP7 Data Value
	8	CTRL0 Data Value
	...	
	11	CTRL3 Data Value
	12-23	Not used

	Bits	
X:$C083		JDISP, J??? Port Data Direction Control Register
	0	DISP0 Direction Control (Must be 1 for display to function)
	...	
	7	DISP7 Direction Control (Must be 1 for display to function)
	8	CTRL0 Direction Control (Must be 1 for ??? to function)
	...	
	11	CTRL3 Direction Control (Must be 1 for ??? to function) (All bits: 0=Input; 1=Output)
	12-23	Not used
Y:$C084		JI/O Port Data Type Control Register
	Bits	
	0	I/O00 Data Type Control (0=FlagW1*; 1=I/O00)
	1	I/O01 Data Type Control (0=FlagV1*; 1=I/O01)
	2	I/O02 Data Type Control (0=FlagU1*; 1=I/O02)
	3	I/O03 Data Type Control (0=FlagT1*; 1=I/O03)
	4	I/O04 Data Type Control (0=USER1*; 1=I/O04)
	5	I/O05 Data Type Control (0=MLIM1*; 1=I/O05)
	6	I/O06 Data Type Control (0=PLIM1*; 1=I/O06)
	7	I/O07 Data Type Control (0=HMFL1*; 1=I/O07)
	8	I/O08 Data Type Control (0=PWM_B_BOT1*; 1=I/O08)
	9	I/O09 Data Type Control (0=PWM_B_TOP1*; 1=I/O09)
	10	I/O10 Data Type Control (0=PWM_A_BOT1*; 1=I/O10)
	11	I/O11 Data Type Control (0=PWM_A_TOP1*; 1=I/O11)
	12	I/O12 Data Type Control (0=PWM_B_BOT2*; 1=I/O12)
	13	I/O13 Data Type Control (0=PWM_B_TOP2*; 1=I/O13)
	14	I/O14 Data Type Control (0=PWM_A_BOT2*; 1=I/O14)
	15	I/O15 Data Type Control (0=PWM_A_TOP2*; 1=I/O15)
	16	I/O16 Data Type Control (0=HMFL2*; 1=I/O16)
	Bits	
	17	I/O17 Data Type Control (0=PLIM2*; 1=I/O17)
	18	I/O18 Data Type Control (0=MLIM2*; 1=I/O18)
	19	I/O19 Data Type Control (0=USER2*; 1=I/O19)
	20	I/O20 Data Type Control (0=FlagT2*; 1=I/O20)
	21	I/O21 Data Type Control (0=FlagU2*; 1=I/O21)
	22	I/O22 Data Type Control (0=FlagV2*; 1=I/O22)
	23	I/O23 Data Type Control (0=FlagW2*; 1=I/O23) (All bits: 0=dedicated hardware I/O; 1=general I/O) (All bits must be 1 for JI/O Port to function as general I/O)
X:$C084		JI/O Port Data Inversion Control Register (when used as general I/O; see Y:$C084)
	Bits	
	0	I/O00 Inversion Control
	...	
	23	I/O23 Inversion Control (All bits: 0=Non-inverting; 1=Inverting)

Y:$C085 JI/O Port Data Type Control Register

Bits	
0	I/O24 Data Type Control
...	
7	I/O31 Data Type Control

(These bits are always 1; there is no alternate mode for these lines)

Bits	
8-23	Not used

X:$C085 JI/O Port Data Inversion Control

Bits	
0	I/O24 Inversion Control
...	
7	I/O31 Inversion Control

(All bits: 0=Non-inverting; 1=Inverting)

Bits	
8-23	Not used

Y:$C086 JTHW Port Data Type Control Register

Bits	
0	DAT0 Data Type Control (0=HWC1; 1 =DAT0)
1	DAT1 Data Type Control (0=HWC2; 1 =DAT1)
2	DAT2 Data Type Control (0=Fault1*; 1 =DAT2)
3	DAT3 Data Type Control (0=Fault2*; 1 =DAT3)
4	DAT4 Data Type Control (0=EQU1*; 1 =DAT4)
5	DAT5 Data Type Control (0=EQU2*; 1 =DAT5)
6	DAT6 Data Type Control (0=AENA1*; 1 =DAT6)
7	DAT7 Data Type Control (0=AENA2*; 1 =DAT7)
8	SEL0 Data Type Control (0=ADC_STROB*; 1=SEL0)
9	SEL1 Data Type Control (0=ADC_CLK*; 1=SEL1)
10	SEL2 Data Type Control (0=ADC_A1*; 1=SEL2)
11	SEL3 Data Type Control (0=ADC_B1*; 1=SEL3)
12	SEL4 Data Type Control (0=ADC_A2*; 1=SEL4)
13	SEL5 Data Type Control (0=ADC_B2*; 1=SEL5)
14	SEL6 Data Type Control (0=SCLK*; 1=SEL6)
15	SEL7 Data Type Control (0=SCLK_DIR*; 1=SEL7)

(All bits: 0=dedicated hardware I/O; 1=general I/O)
(All bits must be 1 for JTHW Port to function as general I/O or with multiplexed accessories)

16-23	Not used

X:$C086 JTHW Port Data Inversion Control Register (when used as general I/O; see Y:$C086)

Bits	
0	DAT0 Inversion Control
...	
7	DAT7 Inversion Control
8	SEL0 Inversion Control
...	
15	SEL7 Inversion Control

(All bits: 0=Non-inverting; 1=Inverting)
(All bits must be 0 to use standard port accessories)

16-23	Not used

Y:$C087 JDISP, J??? Port Data Type Control Register

Bits	
0	DISP0 Data Type Control
...	
7	DISP7 Data Type Control
8	CTRL0 Data Type Control
...	
11	CTRL3 Data Type Control

(These bits are always 1; there is no alternate mode for these pins)

12-23	Not used

X:$C087		JDISP, J??? Port Data Inversion Control Register
	Bits	
	0	DISP0 Inversion Control
	...	
	7	DISP7 Inversion Control
	8	CTRL0 Inversion Control
	...	
	11	CTRL3 Inversion Control
	(All bits: 0=Non-inverting; 1=Inverting)	
	(All bits must be 0 to use standard port accessories)	
	12-23	Not used
Y:$C088-$C08B		Not used
X:$C088-$C08B		Not used
Y:$C08C		Pure binary conversion from gray code input on I/O00 to I/O23
X:$C08C		Not used
Y:$C08D		Gray-to-binary conversion bit-length control
	Bits	
	0-3	Bit length of less significant word portion (I/O00 - I/Onn)
	4	=1 specifies 16-bit lower / 8-bit upper conversion
	5-23	Not used
X:$C08D		Not used
Y:$C08E		MACRO Node Enable Control (I996)
	Bits	
	0	Node 0 enable control
	...	
	15	Node 15 enable control
	(0=node disable; 1=node enable)	
	16-19	Sync packet slave node number control
	20-23	Master number control
	(Note: Bits 16-23 not present on prototype PMAC2 boards with prototype "DSPGATE2" ICs; production "DSPGATE2A" ICs have full register)	

X:$C08E		Not used
Y:$C08F		MACRO Ring Status and Control
	Bits	
	0	Data overrun error (cleared when read)
	1	Byte violation error (cleared when read)
	2	Packet parity error (cleared when read)
	3	Data underrun error (cleared when read)
	4	Master station enable
	5	Synchronizing master station enable
	6	Sync packet received (cleared when read)
	7	Sync packet phase lock enable
	8	Node 8 master address check disable
	9	Node 9 master address check disable
	10	Node 10 master address check disable
	11	Node 11 master address check disable
	12	Node 12 master address check disable
	13	Node 13 master address check disable
	14	Node 14 master address check disable
	15	Node 15 master address check disable

(Note: On prototype PMAC2 boards with prototype "DSPGATE2" ICs, only bits 0,1,2 are present; equivalent to bits 4, 1, and 2, respectively, of production boards with "DSPGATE2A" ICs as listed above.)

X:$C08F DSPGATE2 clock control register

Bits

(Bits 0-11 comprise I993)

0-2	Handwheel SCLK* Frequency Control n (f=39.3216MHz / 2^n, n=0-7)
3-5	JHW/PD PFM Clock Frequency Control n (f=39.3216MHz / 2^n, n=0-7)
6-8	Not used
9-11	ADC Clock* Frequency Control n (f=39.3216MHz / 2^n, n=0-7)
12	Phase Clock Direction (0=output, 1=input) (This must be 1)
13	Servo Clock Direction (0=output, 1=input) (This must be 1)
14-15	Not used (report as zero) (I997 -- normally used on PMAC2 UltraLite only)
16-19	Phase Clock* Frequency Control n (f=MAXPHASE* / [n+1], n=0-15)
20-23	(I998 -- normally used on PMAC2 UltraLite only) Servo Clock* Frequency Control n (f=PHASE* / [n+1], n=0-15)

CHAN #	1	2
HEX	[$C090]	[$C098]
DECIMAL	49296	49304

Y:$C09x Handwheel n Time between last two encoder counts (SCLK cycles)

X:$C09x Supplementary Channel n* (Handwheel n) Status Word

Bits

0-2	Captured Hall Effect Device (UVW) State
3-7	Not used (reports as 0)

Bits

8	Encoder Count Error (0 on counter reset, 1 on illegal transition)
9	Position Compare (EQUn*) output value
10	Position-Captured-On-Gated-Index Flag (=0 on read of captured position register, =1 on trigger capture)
11	Position-Captured Flag (on any trigger) (=0 on read of captured position register, =1 on trigger capture)
12	Handwheel 1 Channel A (HWAn) Input Value
13	Handwheel 1 Channel B (HWBn) Input Value
14	Handwheel 1 Channel C (Index, HWCn) Input Value (ungated)
15	Amplifier Fault (FAULTn*) Input Value
16	Home Flag (HMFLn*) Input Value
17	Positive End Limit (PLIMn*) Input Value
18	Negative End Limit (MLIMn*) Input Value
19	User Flag (USERn*) Input Value
20	FlagWn* Input Value
21	FlagVn* Input Value
22	FlagUn* Input Value
23	FlagTn* Input Value

CHAN #	1	2
HEX	[$C091]	[$C099]
DECIMAL	49297	49305

Y:$C09x Handwheel n Time Since Last Encoder Count (SCLK cycles)
X:$C09x Handwheel n Phase Position Capture Register (counts)

CHAN #	1	2
HEX	[$C092]	[$C09A]
DECIMAL	49298	49306

Y:$C09x Supplementary Channel n* Output A Command Value

Bits	
8-23	PWM Command Value (appears on I/O10 & I/O11 if in dedicated mode)
0-5	Not Used

X:$C09x Handwheel n* Servo Position Capture Register

Bits	
0	Direction of last count (0=up, 1=down)
1-23	Position counter (units of counts)

CHAN #	1	2
HEX	[$C093]	[$C09B]
DECIMAL	49299	49307

Y:$C09x Supplementary Channel n* Output B Command Value

Bits	
8-23	PWM Command Value (appears on I/O08 & I/O09 if in dedicated mode)
0-5	Not used

X:$C09x Handwheel n Flag Position Capture Value; 24 bits, units of counts

CHAN #	1	2
HEX	[$C094]	[$C09C]
DECIMAL	49300	49308

Y:$C09x Supplementary Channel n* Output C Command Value

Bits	
8-23	PWM Command Value (appears on PUL1 & DIR1 if in PWM mode)
0-23	PFM Command Value (appears on PUL1 & DIR1 if in PFM mode)

X:$C094 Supplementary ADC Strobe Word, 24 bits
(Shifted out MSB first one bit per DAC_CLK cycle, starting on rising edge of phase clock; appears on SEL0 if in dedicated mode)

X:$C09C — Supplementary PWM, PFM, MaxPhase* Control Word

Bits	
0-7	PWM* Dead Time (16*PWM* CLK cycles) also PFM* pulse width (PFM* CLK cycles)
8-23	PWM* Max Count Value PWM* Frequency = 117.96MHz / 2*(MaxCount+1)] "MaxPhase*" Frequency = 2*PWM* Frequency

CHAN #	1	2
HEX	[$C095]	[$C09D]
DECIMAL	49301	49309

Y:$C09x — Supplementary Channel n* ADC A Input Value (uses SEL2 in dedicated mode)

Bits	
6-23	Serial ADC Value
0-5	Not used

X:$C09x — Supplementary Channel n* (Handwheel n) Control Word

Bits	
0-1	Encoder Decode Control 00: Pulse and direction decode 01: x1 quadrature decode 10: x2 quadrature decode 11: x4 quadrature decode

Bits	
2-3	Direction & Timer Control 00: Standard timer control, external signal source, no inversion 01: Standard timer control, external signal source, invert direction 10: Standard timer control, internal PFM source, no inversion 11: Alternate timer control, external signal source
4-5	Position Capture Control 00: Software capture (by setting bit 6) 01: Use encoder index alone 10: Use capture flag alone 11: Use encoder index and capture flag
6	Index Capture Invert Control (0=no inversion, 1=inversion)
7	Flag Capture Invert Control (0=no inversion, 1=inversion)
8-9	Capture Flag Select Control 00: Home Flag (HMFLn*) 01: Positive Limit (PLIMn*) 10: Negative Limit (MLIMn*) 11: User Flag (USERn*)
10	Encoder Counter Reset Control (1=reset)
11	Position Compare Initial State Write Enable
12	Position Compare Initial State Value
13	Position Compare Channel Select (0= use this channel's encoder; 1=use first encoder on IC)
14	AENAn* output value
15	Gated Index Select for Position Capture

Bits	(0=ungated index, 1=gated index)
16	Invert AB for Gated Index (0: Gated Signal=A&B&C; 1: Gated Signal=A/&B/&C)
17-18	Not used (report as 0)
19	Invert PFM Direction Control (0=no inversion, 1=invert)
20	Invert A & B Output Control (0=no inversion, 1=invert)
21	Invert C Output Control (0=no inversion, 1=invert)
22	Not used (reports as 0)
23	Output C Mode Select (0=PWM, 1=PFM)

CHAN #	1	2
HEX	[$C096]	[$C09E]
DECIMAL	49302	49310

`Y:$C09x` Supplementary Channel n* ADC B Input Value (uses SEL3 in dedicated mode)

Bits	
6-23	Serial ADC Value
0-5	Not used

`X:$C09x` Handwheel n Compare Auto-increment value (24 bits, units of counts)

CHAN #	1	2
HEX	[$C097]	[$C09F]
DECIMAL	49303	49311

`Y:$C09x` Handwheel n Compare A Value (24 bits, units of counts)
`X:$C09x` Handwheel n Compare B Value (24 bits, units of counts)
`Y:$C0A0` MACRO Node 0 24-bit command(write) and feedback (read) register
`X:$C0A0` MACRO Node 2 24-bit command(write) and feedback (read) register
`Y:$C0A1` MACRO Node 0 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`X:$C0A1` MACRO Node 2 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`Y:$C0A2` MACRO Node 0 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`X:$C0A2` MACRO Node 2 2nd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`Y:$C0A3` MACRO Node 0 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`X:$C0A3` MACRO Node 2 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`Y:$C0A4` MACRO Node 1 24-bit command(write) and feedback (read) register
`X:$C0A4` MACRO Node 3 24-bit command(write) and feedback (read) register
`Y:$C0A5` MACRO Node 1 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`X:$C0A5` MACRO Node 3 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`Y:$C0A6` MACRO Node 1 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
`X:$C0A6` MACRO Node 3 2nd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)

Y:$C0A7	MACRO Node 1 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0A7	MACRO Node 3 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0A8	MACRO Node 4 24-bit command(write) and feedback (read) register
X:$C0A8	MACRO Node 4 24-bit command(write) and feedback (read) register
Y:$C0A9	MACRO Node 6 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0A9	MACRO Node 6 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0AA	MACRO Node 4 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0AA	MACRO Node 6 2nd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0AB	MACRO Node 4 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0AB	MACRO Node 6 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0AC	MACRO Node 5 24-bit command(write) and feedback (read) register
X:$C0AC	MACRO Node 7 24-bit command(write) and feedback (read) register
Y:$C0AD	MACRO Node 5 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0AD	MACRO Node 7 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0AE	MACRO Node 5 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0AE	MACRO Node 7 2nd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0AF	MACRO Node 5 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0AF	MACRO Node 7 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0B0	MACRO Node 8 24-bit command(write) and feedback (read) register X:$C0B0 MACRO Node 10 24-bit command(write) and feedback (read) register
Y:$C0B1	MACRO Node 8 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0B1	MACRO Node 10 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0B2	MACRO Node 1 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0B2	MACRO Node 10 2nd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0B3	MACRO Node 8 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0B3	MACRO Node 10 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0B4	MACRO Node 9 24-bit command(write) and feedback (read) register
X:$C0B4	MACRO Node 11 24-bit command(write) and feedback (read) register
Y:$C0B5	MACRO Node 9 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0B5	MACRO Node 11 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0B6	MACRO Node 9 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0B6	MACRO Node 11 2nd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0B7	MACRO Node 9 3rd 16-bit command(write) and feedback (read) regis-

	ter (bits 8-23; bits 0-7 not used)
X:$C0B7	MACRO Node 11 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0B8	MACRO Node 12 24-bit command(write) and feedback (read) register
X:$C0B8	MACRO Node 14 24-bit command(write) and feedback (read) register
Y:$C0B9	MACRO Node 12 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0B9	MACRO Node 14 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0BA	MACRO Node 12 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0BA	MACRO Node 14 2nd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0BB	MACRO Node 12 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0BB	MACRO Node 14 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0BC	MACRO Node 13 24-bit command(write) and feedback (read) register
X:$C0BC	MACRO Node 15 24-bit command(write) and feedback (read) register
Y:$C0BD	MACRO Node 13 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0BD	MACRO Node 15 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0BE	MACRO Node 13 1st 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0BE	MACRO Node 15 2nd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
Y:$C0BF	MACRO Node 13 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)
X:$C0BF	MACRO Node 15 3rd 16-bit command(write) and feedback (read) register (bits 8-23; bits 0-7 not used)

Dual-Ported RAM (Option 2 Required)

$D000 - $D1FF (53248 - 53759)	Dedicated DPRAM functions (See Option 2 Manual)
$D200 - $DFFF	Open use

VME-Bus Registers (PMAC-VME and PMAC2-VME only)

$E000 - $EFFF (57344 - 61439)	Used for VME-bus functions
Y:$E000	VME Mailbox Register 0 (Bits 0-7)
Y:$E001	VME Mailbox Register 1 (Bits 0-7)
Y:$E002	VME Mailbox Register 2 (Bits 0-7)
Y:$E003	VME Mailbox Register 3 (Bits 0-7)
Y:$E004	VME Mailbox Register 4 (Bits 0-7)
Y:$E005	VME Mailbox Register 5 (Bits 0-7)
Y:$E006	VME Mailbox Register 6 (Bits 0-7)
Y:$E007	VME Mailbox Register 7 (Bits 0-7)
Y:$E008	VME Mailbox Register 8 (Bits 0-7)
Y:$E009	VME Mailbox Register 9 (Bits 0-7)
Y:$E00A	VME Mailbox Register A (Bits 0-7)
Y:$E00B	VME Mailbox Register B (Bits 0-7)
Y:$E00C	VME Mailbox Register C (Bits 0-7)

Y:$E00D	VME Mailbox Register D (Bits 0-7)
Y:$E00E	VME Mailbox Register E (Bits 0-7)
Y:$E00F	VME Mailbox Register F (Bits 0-7)

I/O Control Registers (PMAC2 Specific)

PC/ISA Bus PMAC2 Versions (PMAC2-PC, PMAC2-Lite, PMAC2-PC UltraLite)

X/Y:$E800 — I/O Buffer IC Direction Control

Bits	
0	OUT0: Buffer direction control for I/O00 to I/O07 on JIO
1	OUT1: Buffer direction control for I/O08 to I/O15 on JIO
2	OUT2: Buffer direction control for I/O16 to I/O23 on JIO
3	OUT3: Buffer direction control for I/O24 to I/O31 on JIO
4	OUT4: Buffer direction control for DAT0 to DAT7 on JTHW
5	OUT5: Buffer direction control for SEL0 to SEL7 on JTHW
6	OUT6: Buffer direction control for DISP0 to DISP7 on JDISP, inverted as R/W- output on JDISP (pin 5)
7	OUT7: Inverted as E output on JDISP

X/Y:$E801 — I/O Buffer IC Direction Control

Bits	
0	OUT8: Inverted as RS output on JDISP
1	XIN_1: Jumper E1 input; phase/servo clock direction status
2	XIN_2: Jumper E2 input (40/60 MHz control)
3	XIN_3: Jumper E3 input (re-initialize on reset)
4	XIN_4: Jumper E4 input
5	XIN_5: Jumper E5 input
6	XIN_6: Jumper E6 input
7	XIN_7: Option 12 ADC convert status

VME Bus PMAC2 Versions (PMAC2-VME, PMAC2-VME UltraLite):

X/Y:$E800 — I/O Buffer IC Direction Control

Bits	
0	OUT0: Buffer direction control for I/O00 to I/O07 on JIO
1	OUT1: Buffer direction control for I/O08 to I/O15 on JIO
2	OUT2: Buffer direction control for I/O16 to I/O23 on JIO
3	OUT3: Buffer direction control for I/O24 to I/O31 on JIO

X/Y:$E801 — I/O Buffer IC Direction Control

Bits	
0	OUT8: Inverted as RS output on JDISP
1	XIN_1: Jumper E1 input; phase/servo clock direction status
2	XIN_2: Jumper E2 input (40/60 MHz control)
3	XIN_3: Jumper E3 input (re-initialize on reset)

X/Y:$E802 — I/O Buffer IC Direction Control

Bits	
0	OUT4: Buffer direction control for DAT0 to DAT7 on JTHW
1	OUT5: Buffer direction control for SEL0 to SEL7 on JTHW
2	OUT6: Buffer direction control for DISP0 to DISP7 on JDISP, inverted as R/W- output on JDISP (pin 5)
3	OUT7: Inverted as E output on JDISP

X/Y:$E803		I/O Buffer IC Direction Control
	Bits	
	0	XIN_4: Jumper E4 input
	1	XIN_5: Jumper E5 input
	2	XIN_6: Jumper E6 input
	3	XIN_7: Option 12 ADC convert status

Option 12/12A Analog-to-Digital Converters

Y:$FFC0	Option 12 Analog-to-Digital Converters (low 12 bits) Write operation: channel select (Channels 0-7) and mode; Read operation: converted value of selected channel Option 12A Analog-to-Digital Converters (high 12 bits) Write operation: channel select (Channels 8-15) and mode; Read operation: converted value of selected channel

Inputs and Outputs (PMAC-PC, PMAC-VME, and PMAC-Lite Only)

	Bits	
Y:$FFC0	0-7	Display & EAROM I/O (dedicated)
(65472)	8	Jog Minus Input (J2-4)
	9	Jog Plus Input (J2-6)
	10	Prejog Input (J2-7)
	11	Start (Run) Input (J2-8)
	12	Step/Quit Input (J2-9)
	13	Stop (Abort) Input (J2-10)
	14	Home Command Input (J2-11)
	15	Feed Hold Input (J2-12)
	16	Motor/CoordSys Select Inp. Bit 0 (J2-3)
	17	Motor/CoordSys Select Inp. Bit 1 (J2-5)
	18	Motor/CoordSys Select Inp. Bit 2 (J2-13)
	19	Motor/CoordSys Select Inp. Bit 3 (J2-14)
	20	E51 Jumper (Reinitialize on power-on)
	21	E50 Jumper: EAROM Write Enable
	22	E49 Jumper: Wait State Control
	23	E48 Jumper: Parity Control
	Bits	
Y:$FFC1	0	Thumbwheel Port Inp. Bit 0 (DAT0:J3-3)
(65473)	1	Thumbwheel Port Inp. Bit 1 (DAT1:J3-5)
	2	Thumbwheel Port Inp. Bit 2 (DAT2:J3-7)
	3	Thumbwheel Port Inp. Bit 3 (DAT3:J3-9)
	4	Thumbwheel Port Inp. Bit 4 (DAT4:J3-11)
	5	Thumbwheel Port Inp. Bit 5 (DAT5:J3-13)
	6	Thumbwheel Port Inp. Bit 6 (DAT6:J3-15)
	7	Thumbwheel Port Inp. Bit 7 (DAT7:J3-17)
	8	Thumbwheel Port Out. Bit 0 (SEL0:J3-4)
	9	Thumbwheel Port Out. Bit 1 (SEL1:J3-6)
	10	Thumbwheel Port Out. Bit 2 (SEL2:J3-8)
	11	Thumbwheel Port Out. Bit 3 (SEL3:J3-10)
	12	Thumbwheel Port Out. Bit 4 (SEL4:J3-12)

	Bits	
	13	Thumbwheel Port Out. Bit 5 (SEL5:J3-14)
	14	Thumbwheel Port Out. Bit 6 (SEL6:J3-16)
	15	Thumbwheel Port Out. Bit 7 (SEL7:J3-18)
	16	Jumper E40: Software Card Address Bit 0
	17	Jumper E41: Software Card Address Bit 1
	18	Jumper E42: Software Card Address Bit 2
	19	Jumper E43: Software Card Address Bit 3
	20	Jumper E44: Baud Rate Select Bit 0
	21	Jumper E45: Baud Rate Select Bit 1
	22	Jumper E46: Baud Rate Select Bit 2
	23	Jumper E47: Baud Rate Select Bit 3
	Bits	
Y:$FFC2 (65474)	0	Machine Input 1 (MI1) (J5-15)
	1	Machine Input 2 (MI2) (J5-13)
	2	Machine Input 3 (MI3) (J5-11)
	3	Machine Input 4 (MI4) (J5-9)
	4	Machine Input 5 (MI5) (J5-7)
	5	Machine Input 6 (MI6) (J5-5)
	6	Machine Input 7 (MI7) (J5-3)
	7	Machine Input 8 (MI8) (J5-1)
	8	Machine Output 1 (MO1) (J5-31)
	9	Machine Output 2 (MO2) (J5-29)
	10	Machine Output 3 (MO3) (J5-27)
	11	Machine Output 4 (MO4) (J5-25)
	12	Machine Output 5 (MO5) (J5-23)
	13	Machine Output 6 (MO6) (J5-21)
	14	Machine Output 7 (MO7) (J5-19)
	15	Machine Output 8 (MO8) (J5-17)
	16	D_RS Line (dedicated use)
	17	Read/Write Line (dedicated use)
	18	EACLK (EAROM Clock -- dedicated)
	19	ENA422 (Enable RS422 -- dedicated)
	20	INPOS (In Position Status Line)
	21	BFUL (Buffer Full Status Line)
	22	EROR (Error Status Line)
	23	F1ER (Following Error Status Line)

Inputs and Outputs (PMAC-STD Only)

Y:$FFE0

Second Piggyback Board's J1,J2, & J3 I/O lines Address Map



All the I/O lines marked with the asterisk are used by the PMAC firmware for special functions. These lines should not be used for general-purpose I/O.

Bits			
0	MOD24/	Machine I/O 24	(J1-8)
1	MOD25/	Machine I/O 25	(J1-7)
2	MOD26/	Machine I/O 26	(J1-10)
3	MOD27/	Machine I/O 27	(J1-9)
4	MOD28/	Machine I/O 28	(J1-12)
5	MOD29/	Machine I/O 29	(J1-11)
6	MOD30/	Machine I/O 30	(J1-14)
7	MOD31/	Machine I/O 31	(J1-13)

	Bits			
Y:$FFE1	0	MOD32/	Machine I/O 32	(J3-3)
	1	MOD33/	Machine I/O 33	(J3-5)
	2	MOD34/	Machine I/O 34	(J3-7)
	3	MOD35/	Machine I/O 35	(J3-9)
	4	MOD36/	Machine I/O 36	(J3-11)
	5	MOD37/	Machine I/O 37	(J3-13)
	6	MOD38/	Machine I/O 38	(J3-15)
	7	MOD39/	Machine I/O 39	(J3-17)

	Bits			
Y:$FFE2	0	MOD40/	Machine I/O 40	(J3-4)
	1	MOD41/	Machine I/O 41	(J3-6)
	2	MOD42/	Machine I/O 42	(J3-8)
	3	MOD43/	Machine I/O 43	(J3-10)
	4	MOD44/	Machine I/O 44	(J3-12)
	5	MOD45/	Machine I/O 45	(J3-14)
	6	MOD46/	Machine I/O 46	(J3-16)
	7	MOD47/	Machine I/O 47	(J3-18)

	Bits			
Y:$FFE3	0	MOD48/	Machine I/O 48	(J2-4)
	1	MOD49/	Machine I/O 49	(J2-6)
	2	MOD50/	Machine I/O 50	(J2-7)
	3	MOD51/	Machine I/O 51	(J2-8)
	4	MOD52/	Machine I/O 52	(J2-9)
	5	MOD53/	Machine I/O 53	(J2-10)
	6	MOD54/	Machine I/O 54	(J2-11)
	7	MOD55/	Machine I/O 55	(J2-12)

	Bits			
Y:$FFE4	0	MOD56/	Machine I/O 56	(J2-3)
	1	MOD57/	Machine I/O 57	(J2-5
	2	MOD58/	Machine I/O 58	(J2-13)
	3	MOD59/	Machine I/O 59	(J2-14)
	4	Jumper E51	User Defined	
	5	Jumper E50	User Defined	
	6	Jumper E49	User Defined	
	7	Jumper E48	User Defined	

	Bits			
Y:$FFE5	0	MOD60/	Machine I/O 60	(J2-17)
	1	MOD61/	Machine I/O 61	(J2-18)
	2	MOD62/	Machine I/O 62	(J2-19)
	3	MOD63/	Machine I/O 63	(J2-23)
	4	MOD64/	Machine I/O 64	(J2-24)
	5	MOD65/	Machine I/O 65	(J1-3)
	6	MOD66/	Machine I/O 66	(J1-6)
	7	MOD67/	Machine I/O 67	(J1-5)

Y:$FFE8 JOPT (J4 on Bottom PCBA) I/O lines Address Map

	Bits			
	0	MOD00/	Machine I/O 0	(JOPT pin 22)
	1	MOD01/	Machine I/O 1	(JOPT pin 20)
	2	MOD02/	Machine I/O 2	(JOPT pin 18)
	3	MOD03/	Machine I/O 3	(JOPT pin 16)
	4	MOD04/	Machine I/O 4	(JOPT pin 14)
	5	MOD05/	Machine I/O 5	(JOPT pin 12)
	6	MOD06/	Machine I/O 6	(JOPT pin 10)
	7	MOD07/	Machine I/O 7	(JOPT pin 8)
	Bits			
Y:$FFE9	0	MOD08/	Machine I/O 8	(JOPT pin 6)
	1	MOD09/	Machine I/O 9	(JOPT pin 4)
	2	MOD10/	Machine I/O 10	(JOPT pin 2)
	3	MOD11/	Machine I/O 11	(JOPT pin 25)
	4	MOD12/	Machine I/O 12	(JOPT pin 23)
	5	MOD13/	Machine I/O 13	(JOPT pin 21)
	6	MOD14/	Machine I/O 14	(JOPT pin 19)
	7	MOD15/	Machine I/O 15	(JOPT pin 17)
	Bits			
Y:$FFEA	0	MOD16/	Machine I/O 16	(JOPT pin 15)
	1	MOD17/	Machine I/O 17	(JOPT pin 13)
	2	MOD18/	Machine I/O 18	(JOPT pin 11)
	3	MOD19/	Machine I/O 19	(JOPT pin 9)
	4	MOD20/	Machine I/O 20	(JOPT pin 7)
	5	MOD21/	Machine I/O 21	(JOPT pin 5)
	6	MOD22/	Machine I/O 22	(JOPT pin 3)
	7	MOD23/	Machine I/O 23	(JOPT pin 1)

Y:$FFF8 First Piggyback Board's J1,J2, & J3 I/O lines Address Map

	Bits			
	0	DB0	Display Data0*	(J1-8)
	1	DB1	Display Data1*	(J1-7)
	2	DB2	Display Data2*	(J1-10)
	3	DB3	Display Data3*	(J1-9)
	4	DB4	Display Data4*	(J1-12)
	5	DB5	Display Data5*	(J1-11)
	6	DB6	Display Data6*	(J1-14)
	7	DB7	Display Data7*	(J1-13)
	Bits			
Y:$FFF9	0	DAT0	THW DATA 0	(J3-3)
	1	DAT1	THW DATA 1	(J3-5)
	2	DAT2	THW DATA 2	(J3-7)
	3	DAT3	THW DATA 3	(J3-9)
	4	DAT4	THW DATA 4	(J3-11)
	5	DAT5	THW DATA 5	(J3-13)
	6	DAT6	THW DATA 6	(J3-15)
	7	DAT7	THW DATA 7	(J3-17)

	Bits			
Y:$FFFA	0	SEL0	THWSELECT0	(J3-4)
	1	SEL1	THWSELECT1	(J3-6)
	2	SEL2	THWSELECT2	(J3-8)
	3	SEL3	THWSELECT3	(J3-10)
	4	SEL4	THWSELECT4	(J3-12)
	5	SEL5	THWSELECT5	(J3-14)
	6	SEL6	THWSELECT6	(J3-16)
	7	SEL7	THWSELECT7	(J3-18)
	Bits			
Y:$FFFB	0	JOG-/	JOG IN - DIR.	(J2-4)
	1	JOG+/	JOG IN + DIR	(J2-6)
	2	PREJ/	RET. TO PRE.J	(J2-7)
	3	STRT/	START PROG.	(J2-8)
	4	STEP/	STEP THR. PRGM	(J2-9)
	5	STOP/	STOP PROG.	(J2-10)
	6	HOME/	HOME SEARCH	(J2-11)
	7	HOLD/	HOLD MOTION	(J2-12)
	Bits			
Y:$FFFC	0	FPD0/	SEL. BIT 0	(J2-3)
	1	FPD1/	SEL. BIT 1	(J2-5)
	2	FPD2/	SEL. BIT 2	(J2-13)
	3	FPD3/	SEL. BIT 3	(J2-14)
	4	Jumper E51	E-POINT*	
	5	Jumper E50	E-POINT*	
	6	Jumper E49	E-POINT*	
	7	Jumper E48	E-POINT*	
	Bits			
Y:$FFFD	0	IPLD/	In Position*	(J2-17)
	1	BREQ/	Buffer Request*	(J2-18)
	2	ERLD/	Error Indicator*	(J2-19)
	3	F1LD/	Following err 1*	(J2-23)
	4	F2LD/	Watch Dog Timer*	(J2-24)
	5	RS	DISP Read Strobe*	(J1-3)
	6	R/W	DISP Read/Write *	(J1-6)
	7	E	DISP Enable*	(J1-5)

Inputs and Outputs (PMAC1.5-STD Only)

Y:$FFC0 CPU Board J1 (JDISP) Outputs

All the I/O lines marked with the asterisk are used by the PMAC firmware for special functions. These lines should not be used for general-purpose I/O.

Bits			
0	DB0	Display Data0*	(J1-8)
1	DB1	Display Data1*	(J1-7)
2	DB2	Display Data2*	(J1-10)
3	DB3	Display Data3*	(J1-9)
4	DB4	Display Data4*	(J1-12)
5	DB5	Display Data5*	(J1-11)
6	DB6	Display Data6*	(J1-14)
7	DB7	Display Data7*	(J1-13)

Y:$FFC1 CPU Board J3 (JTHW) Outputs

Bits			
0	DAT0	THW DATA 0	(J3-3)
1	DAT1	THW DATA 1	(J3-5)
2	DAT2	THW DATA 2	(J3-7)
3	DAT3	THW DATA 3	(J3-9)
4	DAT4	THW DATA 4	(J3-11)
5	DAT5	THW DATA 5	(J3-13)
6	DAT6	THW DATA 6	(J3-15)
7	DAT7	THW DATA 7	(J3-17)

Y:$FFC2 CPU Board J3 (JTHW) Outputs

Bits			
0	SEL0	THWSELECT0	(J3-4)
1	SEL1	THWSELECT1	(J3-6)
2	SEL2	THWSELECT2	(J3-8)
3	SEL3	THWSELECT3	(J3-10)
4	SEL4	THWSELECT4	(J3-12)
5	SEL5	THWSELECT5	(J3-14)
6	SEL6	THWSELECT6	(J3-16)
7	SEL7	THWSELECT7	(J3-18)

Y:$FFC3 CPU Board Setup Jumpers

Bits		
0	E40	Jumper E40*
1	E41	Jumper E41*
2	E42	Jumper E42*
3	E43	Jumper E43*
4	E44	Jumper E44*
5	E45	Jumper E45*
6	E46	Jumper E46*
7	E47	Jumper E47*

Y:$FFC4 CPU Board Setup Jumpers and Interrupt Lines

Bits		
0	E48	Jumper E48*
1	E49	Jumper E49*
2	E50	Jumper E50*
3	E51	Jumper E51*
4	IPOS	In-Position Interrupt
5	BFUL	Buffer-Full Interrupt
6	EROR	Fatal F.E. Interrupt
7	F1ER	Warn. F.E. Interrupt

Y:$FFC5 CPU Board Display Control and Panel Indicators

Bits			
0	N.C.	No connect	
1	RS	Read Strobe*	(J1-3)
2	R/W	Read/Write*	(J1-6)
3	E	Enable*	(J1-5)
4	IPLD/	In-Position Indicator	(J3-23)
5	BFLD/	Buffer-Full Indicator	(J3-21)
6	ERLD/	Fatal F.E. Indicator	
7	F1LD/	Warn. F.E. Indicator	

Y:$FFC8 Axis Board 1 (E93A ON) JOPT port I/O

Bits			
0	MOD00/	Machine I/O 0	((J5-22)
1	MOD01/	Machine I/O 1	(J5-20)
2	MOD02/	Machine I/O 2	(J5-18)
3	MOD03/	Machine I/O 3	(J5-16)
4	MOD04/	Machine I/O 4	(J5-14)
5	MOD05/	Machine I/O 5	(J5-12)
6	MOD06/	Machine I/O 6	(J5-10)
7	MOD07/	Machine I/O 7	(J5-8)

Y:$FFC9 Axis Board 1 (E93A ON) JOPT port I/O

Bits			
0	MOD08/	Machine I/O 8	(J5-6)
1	MOD09/	Machine I/O 9	(J5-4)
2	MOD10/	Machine I/O 10	(J5 -2)
3	MOD11/	Machine I/O 11	(J5-25)
4	MOD12/	Machine I/O 12	(J5-23)
5	MOD13/	Machine I/O 13	(J5-21)
6	MOD14/	Machine I/O 14	(J5-19)
7	MOD15/	Machine I/O 15	(J5-17)

Y:$FFCA Axis Board 1 (E93A ON) JOPT port I/O

Bits			
0	MOD16/	Machine I/O 16	(J5-15)
1	MOD17/	Machine I/O 17	(J5-13)
2	MOD18/	Machine I/O 18	(J5-11)
3	MOD19/	Machine I/O 19	(J5-9)
4	MOD20/	Machine I/O 20	(J5-7)
5	MOD21/	Machine I/O 21	(J5-5)
6	MOD22/	Machine I/O 22	(J5-3)
7	MOD23/	Machine I/O 23	(J5-1)

Y:$FFCB Axis Board 1 (E93A ON) JOPT2 port I/O

Bits			
0	MOD24/	Machine I/O 24	(J6-1)
1	MOD25/	Machine I/O 25	(J6-2)
2	MOD26/	Machine I/O 26	(J6-3)
3	MOD27/	Machine I/O 27	(J6-4)
4	MOD28/	Machine I/O 28	(J6-5)
5	MOD29/	Machine I/O 29	(J6-6)
6	MOD30/	Machine I/O 30	(J6-7)
7	MOD31/	Machine I/O 31	(J6-8)

Y:$FFCC Axis Board 1 (E93A ON) JPAN port inputs

Bits			
0	JOG-/	JOG IN - DIR.	(J2-4)
1	JOG+/	JOG IN + DIR	(J2-6)
2	PREJ/	RET. TO PRE.J	(J2-7)
3	STRT/	START PROG.	(J2-8)
4	STEP/	STEP THR. PRGM	(J2-9)
5	STOP/	STOP PROG.	(J2-10)
6	HOME/	HOME SEARCH	(J2-11)
7	HOLD/	HOLD MOTION	(J2-12)

Y:$FFCD — Axis Board 1 (E93A ON) JPAN port I/O

Bits			
0	FPD0/	SEL. BIT 0	(J2-3)
1	FPD1/	SEL. BIT 1	(J2-5)
2	FPD2/	SEL. BIT 2	(J2-13)
3	FPD3/	SEL. BIT 3	(J2-14)
4	IPLD/	In Position*	(J2-17)
5	BRLD/	Buffer Request*	(J2-18)
6	ERLD/	Error Indicator*	(J2-19)
7	F1LD/	WarnFollowing Err*	(J2-23)

Y:$FFCF — Axis Board 1 (E93A ON) Input/Output Control (0=Output OK; 1=Input Only)

Bits		
0	MOD00/-MOD07/ Control	(Y:$FFC8)
1	MOD08/-MOD15/ Control	(Y:$FFC9)
2	MOD16/-MOD23/ Control	(Y:$FFCA)
3	MOD24/-MOD31/ Control	(Y:$FFCB)
4	JPAN Switch Control	(Y:$FFCC)
5	JPAN Selector/Indicator Control	(Y:$FFCD)
6	Reserved Control	(do not use)
7	Reserved Control	(do not use)

Y:$FFD0 — Axis Board 2 (E93B ON) JOPT port I/O

Bits			
0	MOD00/	Machine I/O 0	((J5-22)
1	MOD01/	Machine I/O 1	(J5-20)
2	MOD02/	Machine I/O 2	(J5-18)
3	MOD03/	Machine I/O 3	(J5-16)
4	MOD04/	Machine I/O 4	(J5-14)
5	MOD05/	Machine I/O 5	(J5-12)
6	MOD06/	Machine I/O 6	(J5-10)
7	MOD07/	Machine I/O 7	(J5-8)

Y:$FFD1 — Axis Board 2 (E93B ON) JOPT port I/O

Bits			
0	MOD08/	Machine I/O 8	(J5-6)
1	MOD09/	Machine I/O 9	(J5-4)
2	MOD10/	Machine I/O 10	(J5 -2)
3	MOD11/	Machine I/O 11	(J5-25)
4	MOD12/	Machine I/O 12	(J5-23)
5	MOD13/	Machine I/O 13	(J5-21)
6	MOD14/	Machine I/O 14	(J5-19)
7	MOD15/	Machine I/O 15	(J5-17)

Y:$FFD2 — Axis Board 2 (E93B ON) JOPT port I/O

Bits			
0	MOD16/	Machine I/O 16	(J5-15)
1	MOD17/	Machine I/O 17	(J5-13)
2	MOD18/	Machine I/O 18	(J5-11)
3	MOD19/	Machine I/O 19	(J5-9)
4	MOD20/	Machine I/O 20	(J5-7)
5	MOD21/	Machine I/O 21	(J5-5)
6	MOD22/	Machine I/O 22	(J5-3)
7	MOD23/	Machine I/O 23	(J5-1)

Y:$FFD3 Axis Board 2 (E93B ON) JOPT2 port I/O

Bits			
0	MOD24/	Machine I/O 24	(J6-1)
1	MOD25/	Machine I/O 25	(J6-2)
2	MOD26/	Machine I/O 26	(J6-3)
3	MOD27/	Machine I/O 27	(J6-4)
4	MOD28/	Machine I/O 28	(J6-5)
5	MOD29/	Machine I/O 29	(J6-6)
6	MOD30/	Machine I/O 30	(J6-7)
7	MOD31/	Machine I/O 31	(J6-8)

Y:$FFD4 Axis Board 2 (E93B ON) JPAN port inputs (general purpose here)

Bits			
0	JOG-/	(General Purpose)	(J2-4)
1	JOG+/	(General Purpose)	(J2-6)
2	PREJ/	(General Purpose)	(J2-7)
3	STRT/	(General Purpose)	(J2-8)
4	STEP/	(General Purpose)	(J2-9)
5	STOP/	(General Purpose)	(J2-10)
6	HOME/	(General Purpose)	(J2-11)
7	HOLD/	(General Purpose)	(J2-12)

Y:$FFD5 Axis Board 2 (E93B ON) JPAN port I/O (general purpose here)

Bits			
0	FPD0/	(General Purpose)	(J2-3)
1	FPD1/	(General Purpose)	(J2-5)
2	FPD2/	(General Purpose)	(J2-13)
3	FPD3/	(General Purpose)	(J2-14)
4	IPLD/	(General Purpose)	(J2-17)
5	BRLD/	(General Purpose)	(J2-18)
6	ERLD/	(General Purpose)	(J2-19)
7	F1LD/	(General Purpose)	(J2-23)

Y:$FFD7 Axis Board 2 (E93B ON) Input/Output Control (0=Output OK; 1=Input Only)

Bits		
0	MOD00/-MOD07/ Control	(Y:$FFD0)
1	MOD08/-MOD15/ Control	(Y:$FFD1)
2	MOD16/-MOD23/ Control	(Y:$FFD2)
3	MOD24/-MOD31/ Control	(Y:$FFD3)
4	JPAN Switch Control	(Y:$FFD4)
5	JPAN Selector/Indicator Control	(Y:$FFD5)
6	Reserved Control	(do not use)
7	Reserved Control	(do not use)

Y:$FFD8 Axis Board 3 (E93C ON) JOPT port I/O

Bits			
0	MOD00/	Machine I/O 0	(J5-22)
1	MOD01/	Machine I/O 1	(J5-20)
2	MOD02/	Machine I/O 2	(J5-18)
3	MOD03/	Machine I/O 3	(J5-16)
4	MOD04/	Machine I/O 4	(J5-14)
5	MOD05/	Machine I/O 5	(J5-12)
6	MOD06/	Machine I/O 6	(J5-10)
7	MOD07/	Machine I/O 7	(J5-8)

Y:$FFD9 Axis Board 3 (E93C ON) JOPT port I/O

Bits			
0	MOD08/	Machine I/O 8	(J5-6)
1	MOD09/	Machine I/O 9	(J5-4)
2	MOD10/	Machine I/O 10	(J5 -2)
3	MOD11/	Machine I/O 11	(J5-25)
4	MOD12/	Machine I/O 12	(J5-23)
5	MOD13/	Machine I/O 13	(J5-21)
6	MOD14/	Machine I/O 14	(J5-19)
7	MOD15/	Machine I/O 15	(J5-17)

Y:$FFDA Axis Board 3 (E93C ON) JOPT port I/O

Bits			
0	MOD16/	Machine I/O 16	(J5-15)
1	MOD17/	Machine I/O 17	(J5-13)
2	MOD18/	Machine I/O 18	(J5-11)
3	MOD19/	Machine I/O 19	(J5-9)
4	MOD20/	Machine I/O 20	(J5-7)
5	MOD21/	Machine I/O 21	(J5-5)
6	MOD22/	Machine I/O 22	(J5-3)
7	MOD23/	Machine I/O 23	(J5-1)

Y:$FFDB Axis Board 3 (E93C ON) JOPT2 port I/O

Bits			
0	MOD24/	Machine I/O 24	(J6-1)
1	MOD25/	Machine I/O 25	(J6-2)
2	MOD26/	Machine I/O 26	(J6-3)
3	MOD27/	Machine I/O 27	(J6-4)
4	MOD28/	Machine I/O 28	(J6-5)
5	MOD29/	Machine I/O 29	(J6-6)
6	MOD30/	Machine I/O 30	(J6-7)
7	MOD31/	Machine I/O 31	(J6-8)

Y:$FFDC Axis Board 3 (E93C ON) JPAN port inputs (general purpose here)

Bits			
0	JOG-/	(General Purpose)	(J2-4)
1	JOG+/	(General Purpose)	(J2-6)
2	PREJ/	(General Purpose)	(J2-7)
3	STRT/	(General Purpose)	(J2-8)
4	STEP/	(General Purpose)	(J2-9)
5	STOP/	(General Purpose)	(J2-10)
6	HOME/	(General Purpose)	(J2-11)
7	HOLD/	(General Purpose)	(J2-12)

Y:$FFDD Axis Board 3 (E93C ON) JPAN port I/O (general purpose here)

Bits			
0	FPD0/	(General Purpose)	(J2-3)
1	FPD1/	(General Purpose)	(J2-5)
2	FPD2/	(General Purpose)	(J2-13)
3	FPD3/	(General Purpose)	(J2-14)
4	IPLD/	(General Purpose)	(J2-17)
5	BRLD/	(General Purpose)	(J2-18)
6	ERLD/	(General Purpose)	(J2-19)
7	F1LD/	(General Purpose)	(J2-23)

Y:$FFDF — Axis Board 3 (E93C ON) Input/Output Control
(0=Output OK; 1=Input Only)

Bits		
0	MOD00/-MOD07/ Control	(Y:$FFD8)
1	MOD08/-MOD15/ Control	(Y:$FFD9)
2	MOD16/-MOD23/ Control	(Y:$FFDA)
3	MOD24/-MOD31/ Control	(Y:$FFDB)
4	JPAN Switch Control	(Y:$FFDC)
5	JPAN Selector/Indicator Control	(Y:$FFDD)
6	Reserved Control	(do not use)
7	Reserved Control	(do not use)

Y:$FFE0 — Axis Board 4 (E93D ON) JOPT port I/O

Bits			
0	MOD00/	Machine I/O 0	(J5 pin 22)
1	MOD01/	Machine I/O 1	(J5 pin 20)
2	MOD02/	Machine I/O 2	(J5 pin 18)
3	MOD03/	Machine I/O 3	(J5 pin 16)
4	MOD04/	Machine I/O 4	(J5 pin 14)
5	MOD05/	Machine I/O 5	(J5 pin 12)
6	MOD06/	Machine I/O 6	(J5 pin 10)
7	MOD07/	Machine I/O 7	(J5 pin 8)

Y:$FFE1 — Axis Board 4 (E93D ON) JOPT port I/O

Bits			
0	MOD08/	Machine I/O 8	(J5-6)
1	MOD09/	Machine I/O 9	(J5-4)
2	MOD10/	Machine I/O 10	(J5 -2)
3	MOD11/	Machine I/O 11	(J5-25)
4	MOD12/	Machine I/O 12	(J5-23)
5	MOD13/	Machine I/O 13	(J5-21)
6	MOD14/	Machine I/O 14	(J5-19)
7	MOD15/	Machine I/O 15	(J5-17)

Y:$FFE2 — Axis Board 4 (E93D ON) JOPT port I/O

Bits			
0	MOD16/	Machine I/O 16	(J5-15)
1	MOD17/	Machine I/O 17	(J5-13)
2	MOD18/	Machine I/O 18	(J5-11)
3	MOD19/	Machine I/O 19	(J5-9)
4	MOD20/	Machine I/O 20	(J5-7)
5	MOD21/	Machine I/O 21	(J5-5)
6	MOD22/	Machine I/O 22	(J5-3)
7	MOD23/	Machine I/O 23	(J5-1)

Y:$FFE3 — Axis Board 4 (E93D ON) JOPT2 port I/O

Bits			
0	MOD24/	Machine I/O 24	(J6-1)
1	MOD25/	Machine I/O 25	(J6-2)
2	MOD26/	Machine I/O 26	(J6-3)
3	MOD27/	Machine I/O 27	(J6-4)
4	MOD28/	Machine I/O 28	(J6-5)
5	MOD29/	Machine I/O 29	(J6-6)
6	MOD30/	Machine I/O 30	(J6-7)
7	MOD31/	Machine I/O 31	(J6-8)

Y:$FFE4 Axis Board 4 (E93D ON) JPAN port inputs (general purpose here)

Bits			
0	JOG-/	(General Purpose)	(J2-4)
1	JOG+/	(General Purpose)	(J2-6)
2	PREJ/	(General Purpose)	(J2-7)
3	STRT/	(General Purpose)	(J2-8)
4	STEP/	(General Purpose)	(J2-9)
5	STOP/	(General Purpose)	(J2-10)
6	HOME/	(General Purpose)	(J2-11)
7	HOLD/	(General Purpose)	(J2-12)

Y:$FFE5 Axis Board 4 (E93D ON) JPAN port I/O (general purpose here)

Bits			
0	FPD0/	(General Purpose)	(J2-3)
1	FPD1/	(General Purpose)	(J2-5)
2	FPD2/	(General Purpose)	(J2-13)
3	FPD3/	(General Purpose)	(J2-14)
4	IPLD/	(General Purpose)	(J2-17)
5	BREQ/	(General Purpose)	(J2-18)
6	ERLD/	(General Purpose)	(J2-19)
7	F1LD/	(General Purpose)	(J2-23)

Y:$FFE7 Axis Board 4 (E93D ON) Input/Output Control (0=Output OK; 1=Input Only)

Bits		
0	MOD00/-MOD07/ Control	(Y:$FFE0)
1	MOD08/-MOD15/ Control	(Y:$FFE1)
2	MOD16/-MOD23/ Control	(Y:$FFE2)
3	MOD24/-MOD31/ Control	(Y:$FFE3)
4	JPAN Switch Control	(Y:$FFE4)
5	JPAN Selector/Indicator Control	(Y:$FFE5)
6	Reserved Control	(do not use)
7	Reserved Control	(do not use)

Expansion I/O (PMAC-PC, PMAC-VME, and PMAC-Lite Only)

Y:$FFD0 (65488)	1st Accessory-14 Port A
Y:$FFD1 (65489)	1st Accessory-14 Port B
Y:$FFD2 (65490)	1st Accessory-14 Multimodule
Y:$FFD3 (65491)	1st Accessory-14 Control Word
Y:$FFD8 (65496)	2nd Accessory-14 Port A
Y:$FFD9 (65497)	2nd Accessory-14 Port B
Y:$FFDA (65498)	2nd Accessory-14 Multimodule
Y:$FFDB (65499)	2nd Accessory-14 Control Word
Y:$FFE0 (65504)	3rd Accessory-14 Port A
Y:$FFE1 (65505)	3rd Accessory-14 Port B
Y:$FFE2 (65506)	3rd Accessory-14 Multimodule
Y:$FFE3 (65507)	3rd Accessory-14 Control Word
Y:$FFE8 (65512)	4th Accessory-14 Port A
Y:$FFE9 (65513)	4th Accessory-14 Port B

Y:$FFEA (65514)	4th Accessory-14 Multimodule
Y:$FFEB (65515)	4th Accessory-14 Control Word
Y:$FFF0 (65520)	5th Accessory-14 Port A
Y:$FFF1 (65521)	5th Accessory-14 Port B
Y:$FFF2 (65522)	5th Accessory-14 Multimodule
Y:$FFF3 (65523)	5th Accessory-14 Control Word
Y:$FFF8 (65528)	6th Accessory-14 Port A
Y:$FFF9 (65529)	6th Accessory-14 Port B
Y:$FFFA (65530)	6th Accessory-14 Multimodule
Y:$FFFB (65531)	6th Accessory-14 Control Word



EXAMPLE PROGRAMS

PMAC Example Programs

Example 1: Suggested M-variable Definitions (PMAC[1] Specific)

This file contains a suggested set of M-Variable definitions. These definitions can be very useful for getting access to important I/O and registers. It is not required that you use these particular definitions. Most example programs will use these definitions.

```
CLOSE                   ; To ensure commands are on-line
M0..1023->*             ; To clear all existing definitions
M0->X:$0,0,24,U         ; Servo cycle counter
                        ; General Purpose inputs and outputs
                        ; (PMAC-PC,-Lite,-VME)
M1->Y:$FFC2,8,1         ; Machine Output 1
M2->Y:$FFC2,9,1         ; Machine Output 2
M3->Y:$FFC2,10,1        ; Machine Output 3
M4->Y:$FFC2,11,1        ; Machine Output 4
M5->Y:$FFC2,12,1        ; Machine Output 5
M6->Y:$FFC2,13,1        ; Machine Output 6
M7->Y:$FFC2,14,1        ; Machine Output 7
M8->Y:$FFC2,15,1        ; Machine Output 8
M9->Y:$FFC2,8,8,U       ; Machine Outputs 1-8 treated as byte
M11->Y:$FFC2,0,1        ; Machine Input 1
M12->Y:$FFC2,1,1        ; Machine Input 2
M13->Y:$FFC2,2,1        ; Machine Input 3
M14->Y:$FFC2,3,1        ; Machine Input 4
M15->Y:$FFC2,4,1        ; Machine Input 5
M16->Y:$FFC2,5,1        ; Machine Input 6
M17->Y:$FFC2,6,1        ; Machine Input 7
M18->Y:$FFC2,7,1        ; Machine Input 8
M19->Y:$FFC2,0,8,U      ; Machine Inputs 1-8 treated as byte
                        ; Control-Panel Port Input Bits (so can be used as general
                        ; I/O if I2=1) (These definitions valid for PMAC-PC,
                        ; -Lite, & -VME)
M20->Y:$FFC0,8,1        ; Jog Minus Input
M21->Y:$FFC0,9,1        ; Jog Plus Input
M22->Y:$FFC0,10,1       ; Prejog Input
M23->Y:$FFC0,11,1       ; Start (Run) Input
```

```
M24->Y:$FFC0,12,1       ; Step/Quit Input
M25->Y:$FFC0,13,1       ; Stop (Abort) Input
M26->Y:$FFC0,14,1       ; Home Command Input
M27->Y:$FFC0,15,1       ; Feed Hold Input
M28->Y:$FFC0,16,1       ; Motor/C.S. Select Input Bit 0
M29->Y:$FFC0,17,1       ; Motor/C.S. Select Input Bit 1
M30->Y:$FFC0,18,1       ; Motor/C.S. Select Input Bit 2
M31->Y:$FFC0,19,1       ; Motor/C.S. Select Input Bit 3
M32->Y:$FFC0,16,4,C     ; Selected Motor/C.S. Number
                        ; Thumbwheel Port Bits (So they can be used as general
                        ; purpose I/O) (These definitions valid for PMAC-PC,
                        ; -Lite, & -VME)
M40->Y:$FFC1,8,1        ; SEL0 Output
M41->Y:$FFC1,9,1        ; SEL1 Output
M42->Y:$FFC1,10,1       ; SEL2 Output
M43->Y:$FFC1,11,1       ; SEL3 Output
M44->Y:$FFC1,12,1       ; SEL4 Output
M45->Y:$FFC1,13,1       ; SEL5 Output
M46->Y:$FFC1,14,1       ; SEL6 Output
M47->Y:$FFC1,15,1       ; SEL7 Output
M48->Y:$FFC1,8,8,U      ; SEL0-7 Outputs treated as a byte
M50->Y:$FFC1,0,1        ; DAT0 Input
M51->Y:$FFC1,1,1        ; DAT1 Input
M52->Y:$FFC1,2,1        ; DAT2 Input
M53->Y:$FFC1,3,1        ; DAT3 Input
M54->Y:$FFC1,4,1        ; DAT4 Input
M55->Y:$FFC1,5,1        ; DAT5 Input
M56->Y:$FFC1,6,1        ; DAT6 Input
M57->Y:$FFC1,7,1        ; DAT7 Input
M58->Y:$FFC1,0,8,U      ; DAT0-7 Inputs treated as a byte
                        ; Registers associated with Encoder/DAC1
                        ; (Usually Motor #1)
M101->X:$C001,0,24,S    ; ENC1 24-bit counter position
M102->Y:$C003,8,16,S    ; DAC1 16-bit analog output
M103->X:$C003,0,24,S    ; ENC1 capture/compare position register
M104->X:$0720,0,24,S    ; ENC1 interpolated position (1/32 ct)
M105->Y:$C006,8,16,S    ; ADC1 16-bit analog input
M106->Y:$C000,0,24,U    ; ENC1 time between counts (SCLK cycles)
M110->X:$C000,10,1      ; ENC1 count-write enable control
M111->X:$C000,11,1      ; EQU1 compare flag latch control
M112->X:$C000,12,1      ; EQU1 compare output enable
M113->X:$C000,13,1      ; EQU1 compare invert enable
M114->X:$C000,14,1      ; AENA1/DIR1 Output
M116->X:$C000,16,1      ; EQU1 compare flag
M117->X:$C000,17,1      ; ENC1 position-captured flag
M118->X:$C000,18,1      ; ENC1 Count-error flag
M119->X:$C000,19,1      ; ENC1 3rd channel input status
M120->X:$C000,20,1      ; HMFL1 input status
M121->X:$C000,21,1      ; -LIM1 input status
M122->X:$C000,22,1      ; +LIM1 input status
M123->X:$C000,23,1      ; FAULT1 input status
                        ; Motor #1 Status Bits
M130->Y:$0814,11,1      ; #1 Stopped-on-position-limit bit
M131->X:$003D,21,1      ; #1 Positive-end-limit-set bit
M132->X:$003D,22,1      ; #1 Negative-end-limit-set bit
M133->X:$003D,13,1      ; #1 Desired-velocity-zero bit
M135->X:$003D,15,1      ; #1 Dwell-in-progress bit
```

```
M137->X:$003D,17,1      ; #1 Running-program bit
M138->X:$003D,18,1      ; #1 Open-loop-mode bit
M139->Y:$0814,14,1      ; #1 Amplifier-enabled status bit
M140->Y:$0814,0,1       ; #1 In-position bit
M141->Y:$0814,1,1       ; #1 Warning-following error bit
M142->Y:$0814,2,1       ; #1 Fatal-following-error bit
M143->Y:$0814,3,1       ; #1 Amplifier-fault-error bit
M145->Y:$0814,10,1      ; #1 Home-complete bit
                        ; Motor #1 Move Registers
M161->D:$0028           ; #1 Commanded position (1/[Ix08*32] cts)
M162->D:$002B           ; #1 Actual position (1/[Ix08*32] cts)
M163->D:$080B           ; #1 Target (end) position (1/[Ix08*32] cts)
M164->D:$0813           ; #1 Position bias (1/[Ix08*32] cts)
M165->L:$081F           ; &1 X-axis target position (engineering units)
M166->X:$0033,0,24,S    ; #1 Actual velocity (1/[Ix09*32] cts/cyc)
M167->D:$002D           ; #1 Present master ((handwheel) pos (1/[Ix07*32] cts
                        ; of master or (1/[Ix08*32] cts of slaved motor)
M168->X:$0045,8,16,S    ; #1 Filter Output (DAC bits)
M169->D:$0046           ; #1 Compensation correction
M170->D:$0041           ; #1 Present phase position; includes fraction in
                        ; Y-register
M171->X:$0041,0,24,S    ; #1 Present phase position (counts*Ix70)
M172->L:$082B           ; #1 Variable jog position/distance (counts)
M173->Y:$0815,0,24,S    ; #1 Encoder home capture offset (counts)
                        ; Coordinate System &1 Status Bits
M180->X:$0818,0,1       ; &1 Program-running bit
M181->Y:$0817,21,1      ; &1 Circle-radius-error bit
M182->Y:$0817,22,1      ; &1 Run-time-error bit
M184->X:$0818,4,1       ; &1 Continuous motion request
M187->Y:$0817,17,1      ; &1 In-position bit (AND of motors)
M188->Y:$0817,18,1      ; &1 Warning-following-error bit (OR)
M189->Y:$0817,19,1      ; &1 Fatal-following-error bit (OR)
M190->Y:$0817,20,1      ; &1 Amp-fault-error bit (OR of motors)
                        ; Motor #1 Axis Definition Registers
M191->L:$0822           ; #1 X/U/A/B/C-Axis scale factor (cts/unit)
M192->L:$0823           ; #1 Y/V-Axis scale factor (cts/unit)
M193->L:$0824           ; #1 Z/W-Axis scale factor (cts/unit)
M194->L:$0825           ; #1 Axis offset (cts)
                        ; Coordinate System &1 Variables
M197->X:$0806,0,24,S    ; &1 Host commanded time base (I10 units)
M198->X:$0808,0,24,S    ; &1 Present time base (I10 units)
                        ; Registers associated with Encoder/DAC2
                        ; (Usually Motor #2)
M201->X:$C005,0,24,S    ; ENC2 24-bit counter position
M202->Y:$C002,8,16,S    ; DAC2 16-bit analog output
M203->X:$C007,0,24,S    ; ENC2 capture/compare position register
M204->X:$0721,0,24,S    ; ENC2 interpolated position (1/32 ct)
M205->Y:$C007,8,16,S    ; ADC2 16-bit analog input
M206->Y:$C004,0,24,U    ; ENC2 time between counts (SCLK cycles)
M210->X:$C004,10,1      ; ENC2 count-write enable control
M211->X:$C004,11,1      ; EQU2 compare flag latch control
M212->X:$C004,12,1      ; EQU2 compare output enable
M213->X:$C004,13,1      ; EQU2 compare invert enable
M214->X:$C004,14,1      ; AENA2/DIR2 Output
M216->X:$C004,16,1      ; EQU2 compare flag
M217->X:$C004,17,1      ; ENC2 position-captured flag
M218->X:$C004,18,1      ; ENC2 count-error flag
```

```
M219->X:$C004,19,1      ; ENC2 3rd channel input status
M220->X:$C004,20,1      ; HMFL2 input status
M221->X:$C004,21,1      ; -LIM2 input status
M222->X:$C004,22,1      ; +LIM2 input status
M223->X:$C004,23,1      ; FAULT2 input status
                        ; Motor #2 Status Bits
M230->Y:$08D4,11,1      ; #2 Stopped-on-position-limit bit
M231->X:$0079,21,1      ; #2 Positive-end-limit-set bit
M232->X:$0079,22,1      ; #2 Negative-end-limit-set bit
M233->X:$0079,13,1      ; #2 Desired-velocity-zero bit
M235->X:$0079,15,1      ; #2 Dwell-in-progress bit
M237->X:$0079,17,1      ; #2 Running-program bit
M238->X:$0079,18,1      ; #2 Open-loop-mode bit
M239->Y:$08D4,14,1      ; #2 Amplifier-enabled status bit
M240->Y:$08D4,0,1       ; #2 In-position bit
M241->Y:$08D4,1,1       ; #2 Warning-following error bit
M242->Y:$08D4,2,1       ; #2 Fatal-following-error bit
M243->Y:$08D4,3,1       ; #2 Amplifier-fault-error bit
M245->Y:$08D4,10,1      ; #2 Home-complete bit
                        ; Motor #2 Move Registers
M261->D:$0064           ; #2 Commanded position (1/[Ix08*32] cts)
M262->D:$0067           ; #2 Actual position (1/[Ix08*32] cts)
M263->D:$08CB           ; #2 Target (end) position (1/[Ix08*32] cts)
M264->D:$08D3           ; #2 Position bias (1/[Ix08*32] cts)
M265->L:$0820           ; &1 Y-axis target position (engineering units)
M266->X:$006F,0,24,S    ; #2 Actual velocity (1/[Ix09*32] cts/cyc)
M267->D:$0069           ; #2 Present master (handwheel) pos (1/[Ix07*32] cts
                        ; of master or (1/[Ix08*32] cts of slaved motor)
M268->X:$0081,8,16,S    ; #2 Filter Output (DAC bits)
M269->D:$0082           ; #2 Compensation correction
M270->D:$007D           ; #2 Present phase position; includes fraction in
                        ; Y-register
M271->X:$007D,0,24,S    ; #2 Present phase position (counts*Ix70)
M272->L:$08EB           ; #2 Variable jog position/distance (counts)
M273->Y:$08D5,0,24,S    ; #2 Encoder home capture offset (counts)
                        ; Coordinate System &2 Status Bits
M280->X:$08D8,0,1       ; &2 Program-running bit
M281->Y:$08D7,21,1      ; &2 Circle-radius-error bit
M282->Y:$08D7,22,1      ; &2 Run-time-error bit
M284->X:$08D8,4,1       ; &2 Continuous motion request
M287->Y:$08D7,17,1      ; &2 In-position bit (AND of motors)
M288->Y:$08D7,18,1      ; &2 Warning-following-error bit (OR)
M289->Y:$08D7,19,1      ; &2 Fatal-following-error bit (OR)
M290->Y:$08D7,20,1      ; &2 Amp-fault-error bit (OR of motors)
                        ; Motor #2 Axis Definition Registers
M291->L:$08E2           ; #2 X/U/A/B/C-Axis scale factor (cts/unit)
M292->L:$08E3           ; #2 Y/V-Axis scale factor (cts/unit)
M293->L:$08E4           ; #2 Z/W-Axis scale factor (cts/unit)
M294->L:$08E5           ; #2 Axis offset (cts)
                        ; Coordinate System &2 Variables
M297->X:$08C6,0,24,S    ; &2 Host commanded time base (I10 units)
M298->X:$08C8,0,24,S    ; &2 Present time base (I10 units)
                        ; Registers associated with Encoder/DAC3
                        ; (Usually Motor #3)
M301->X:$C009,0,24,S    ; ENC3 24-bit counter position
M302->Y:$C00B,8,16,S    ; DAC3 16-bit analog output
M303->X:$C00B,0,24,S    ; ENC3 capture/compare position register
```

```
M304->X:$0722,0,24,S    ; ENC3 interpolated position (1/32 ct)
M305->Y:$C00E,8,16,S    ; ADC3 16-bit analog input
M306->Y:$C008,0,24,U    ; ENC3 time between counts (SCLK cycles)
M310->X:$C008,10,1      ; ENC3 count-write enable control
M311->X:$C008,11,1      ; EQU3 compare flag latch control
M312->X:$C008,12,1      ; EQU3 compare output enable
M313->X:$C008,13,1      ; EQU3 compare invert enable
M314->X:$C008,14,1      ; AENA3/DIR3 Output
M316->X:$C008,16,1      ; EQU3 compare flag
M317->X:$C008,17,1      ; ENC3 position-captured flag
M318->X:$C008,18,1      ; ENC3 count-error flag
M319->X:$C008,19,1      ; ENC3 3rd channel input status
M320->X:$C008,20,1      ; HMFL3 input status
M321->X:$C008,21,1      ; -LIM3 input status
M322->X:$C008,22,1      ; +LIM3 input status
M323->X:$C008,23,1      ; FAULT3 input status
                        ; Motor #3 Status Bits
M330->Y:$0994,11,1      ; #3 Stopped-on-position-limit bit
M331->X:$00B5,21,1      ; #3 Positive-end-limit-set bit
M332->X:$00B5,22,1      ; #3 Negative-end-limit-set bit
M333->X:$00B5,13,1      ; #3 Desired-velocity-zero bit
M335->X:$00B5,15,1      ; #3 Dwell-in-progress bit
M337->X:$00B5,17,1      ; #3 Running-program bit
M338->X:$00B5,18,1      ; #3 Open-loop-mode bit
M339->Y:$0994,14,1      ; #3 Amplifier-enabled status bit
M340->Y:$0994,0,1       ; #3 In-position bit
M341->Y:$0994,1,1       ; #3 Warning-following error bit
M342->Y:$0994,2,1       ; #3 Fatal-following-error bit
M343->Y:$0994,3,1       ; #3 Amplifier-fault-error bit
M345->Y:$0994,10,1      ; #3 Home-complete bit
                        ; Motor #3 Move Registers
M361->D:$00A0           ; #3 Commanded position (1/[Ix08*32] cts)
M362->D:$00A3           ; #3 Actual position (1/[Ix08*32] cts)
M363->D:$098B           ; #3 Target (end) position (1/[Ix08*32] cts)
M364->D:$0993           ; #3 Position bias (1/[Ix08*32] cts)
M365->L:$0821           ; &1 Z-axis target position (engineering units)
M366->X:$00AB,0,24,S    ; #3 Actual velocity (1/[Ix09*32] cts/cyc)
M367->D:$00A5           ; #3 Present master (handwheel) pos (1/[Ix07*32] cts
                        ; of master or (1/[Ix08*32] cts of slaved motor)
M368->X:$00BD,8,16,S    ; #3 Filter Output (DAC bits)
M369->D:$00BE           ; #3 Compensation correction
M370->D:$00B9           ; #3 Present phase position; includes fraction in
                        ; Y-register
M371->X:$00B9,0,24,S    ; #3 Present phase position (counts*Ix70)
M372->L:$09AB           ; #3 Variable jog position/distance (counts)
M373->Y:$0995,0,24,S    ; #3 Encoder home capture offset (counts)
                        ; Coordinate System &3 Status Bits
M380->X:$0998,0,1       ; &3 Program-running bit
M381->Y:$0997,21,1      ; &3 Circle-radius-error bit
M382->Y:$0997,22,1      ; &3 Run-time-error bit
M384->X:$0998,4,1       ; &3 Continuous motion request
M387->Y:$0997,17,1      ; &3 In-position bit (AND of motors)
M388->Y:$0997,18,1      ; &3 Warning-following-error bit (OR)
M389->Y:$0997,19,1      ; &3 Fatal-following-error bit (OR)
M390->Y:$0997,20,1      ; &3 Amp-fault-error bit (OR of motors)
                        ; Motor #3 Axis Definition Registers
M391->L:$09A2           ; #3 X/U/A/B/C-Axis scale factor (cts/unit)
```

```
M392->L:$09A3           ; #3 Y/V-Axis scale factor (cts/unit)
M393->L:$09A4           ; #3 Z/W-Axis scale factor (cts/unit)
M394->L:$09A5           ; #3 Axis offset (cts)
                        ; Coordinate System &3 Variables
M397->X:$0986,0,24,S    ; &3 Host commanded time base (I10 units)
M398->X:$0988,0,24,S    ; &3 Present time base (I10 units)
                        ; Registers associated with Encoder/DAC4
                        ; (Usually Motor #4)
M401->X:$C00D,0,24,S    ; ENC4 24-bit counter position
M402->Y:$C00A,8,16,S    ; DAC4 16-bit analog output
M403->X:$C00F,0,24,S    ; ENC4 capture/compare position register
M404->X:$0723,0,24,S    ; ENC4 interpolated position (1/32 ct)
M405->Y:$C00F,8,16,S    ; ADC4 16-bit analog input
M406->Y:$C00C,0,24,U    ; ENC4 time between counts (SCLK cycles)
M410->X:$C00C,10,1      ; ENC4 count-write enable control
M411->X:$C00C,11,1      ; EQU4 compare flag latch control
M412->X:$C00C,12,1      ; EQU4 compare output enable
M413->X:$C00C,13,1      ; EQU4 compare invert enable
M414->X:$C00C,14,1      ; AENA4/DIR4 Output
M416->X:$C00C,16,1      ; EQU4 compare flag
M417->X:$C00C,17,1      ; ENC4 position-captured flag
M418->X:$C00C,18,1      ; ENC4 count-error flag
M419->X:$C00C,19,1      ; ENC4 3rd channel input status
M420->X:$C00C,20,1      ; HMFL4 input status
M421->X:$C00C,21,1      ; -LIM4 input status
M422->X:$C00C,22,1      ; +LIM4 input status
M423->X:$C00C,23,1      ; FAULT4 input status
                        ; Motor #4 Status Bits
M430->Y:$0A54,11,1      ; #4 Stopped-on-position-limit bit
M431->X:$00F1,21,1      ; #4 Positive-end-limit-set bit
M432->X:$00F1,22,1      ; #4 Negative-end-limit-set bit
M433->X:$00F1,13,1      ; #4 Desired-velocity-zero bit
M435->X:$00F1,15,1      ; #4 Dwell-in-progress bit
M437->X:$00F1,17,1      ; #4 Running-program bit
M438->X:$00F1,18,1      ; #4 Open-loop-mode bit
M439->Y:$0A54,14,1      ; #4 Amplifier-enabled status bit
M440->Y:$0A54,0,1       ; #4 In-position bit
M441->Y:$0A54,1,1       ; #4 Warning-following error bit
M442->Y:$0A54,2,1       ; #4 Fatal-following-error bit
M443->Y:$0A54,3,1       ; #4 Amplifier-fault-error bit
M445->Y:$0A54,10,1      ; #4 Home-complete bit
                        ; Motor #4 Move Registers
M461->D:$00DC           ; #4 Commanded position (1/[Ix08*32] cts)
M462->D:$00DF           ; #4 Actual position (1/[Ix08*32] cts)
M463->D:$0A4B           ; #4 Target (end) position (1/[Ix08*32] cts)
M464->D:$0A53           ; #4 Position bias (1/[Ix08*32] cts)
M465->L:$0819           ; &1 A-axis target position (engineering units)
M466->X:$00E7,0,24,S    ; #4 Actual velocity (1/[Ix09*32] cts/cyc)
M467->D:$00E1           ; #4 Present master (handwheel) pos (1/[Ix07*32] cts
                        ; of master or (1/[Ix08*32] cts of slaved motor)
M468->X:$00F9,8,16,S    ; #4 Filter Output (DAC bits)
M469->D:$00FA           ; #4 Compensation correction
M470->D:$00F5           ; #4 Present phase position; includes fraction in
                        ; Y-register
M471->X:$00F5,0,24,S    ; #4 Present phase position (counts*Ix70)
M472->L:$0A6B           ; #4 Variable jog position/distance (counts)
M473->Y:$0A55,0,24,S    ; #4 Encoder home capture offset (counts)
```

```
                        ; Coordinate System &4 Status Bits
M480->X:$0A58,0,1       ; &4 Program-running bit
M481->Y:$0A57,21,1      ; &4 Circle-radius-error bit
M482->Y:$0A57,22,1      ; &4 Run-time-error bit
M484->X:$0A58,4,1       ; &4 Continuous motion request
M487->Y:$0A57,17,1      ; &4 In-position bit (AND of motors)
M488->Y:$0A57,18,1      ; &4 Warning-following-error bit (OR)
M489->Y:$0A57,19,1      ; &4 Fatal-following-error bit (OR)
M490->Y:$0A57,20,1      ; &4 Amp-fault-error bit (OR of motors)
                        ; Motor #4 Axis Definition Registers
M491->L:$0A62           ; #4 X/U/A/B/C-Axis scale factor (cts/unit)
M492->L:$0A63           ; #4 Y/V-Axis scale factor (cts/unit)
M493->L:$0A64           ; #4 Z/W-Axis scale factor (cts/unit)
M494->L:$0A65           ; #4 Axis offset (cts)
                        ; Coordinate System &4 Variables
M497->X:$0A46,0,24,S    ; &4 Host commanded time base (I10 units)
M498->X:$0A48,0,24,S    ; &4 Present time base (I10 units)
                        ; Registers associated with Encoder/DAC5
                        ; (Usually Motor #5)
M501->X:$C011,0,24,S    ; ENC5 24-bit counter position
M502->Y:$C013,8,16,S    ; DAC5 16-bit analog output
M503->X:$C013,0,24,S    ; ENC5 capture/compare position register
M504->X:$0724,0,24,S    ; ENC5 interpolated position (1/32 ct)
M505->Y:$C016,8,16,S    ; ADC5 16-bit analog input
M506->Y:$C010,0,24,U    ; ENC5 time between counts (SCLK cycles)
M510->X:$C010,10,1      ; ENC5 count-write enable control
M511->X:$C010,11,1      ; EQU5 compare flag latch control
M512->X:$C010,12,1      ; EQU5 compare output enable
M513->X:$C010,13,1      ; EQU5 compare invert enable
M514->X:$C010,14,1      ; AENA5/DIR5 Output
M516->X:$C010,16,1      ; EQU5 compare flag
M517->X:$C010,17,1      ; ENC5 position-captured flag
M518->X:$C010,18,1      ; ENC5 count-error flag
M519->X:$C010,19,1      ; ENC5 3rd channel input status
M520->X:$C010,20,1      ; HMFL5 input status
M521->X:$C010,21,1      ; -LIM5 input status
M522->X:$C010,22,1      ; +LIM5 input status
M523->X:$C010,23,1      ; FAULT5 input status
                        ; Motor #5 Status Bits
M530->Y:$0B14,11,1      ; #5 Stopped-on-position-limit bit
M531->X:$012D,21,1      ; #5 Positive-end-limit-set bit
M532->X:$012D,22,1      ; #5 Negative-end-limit-set bit
M533->X:$012D,13,1      ; #5 Desired-velocity-zero bit
M535->X:$012D,15,1      ; #5 Dwell-in-progress bit
M537->X:$012D,17,1      ; #5 Running-program bit
M538->X:$012D,18,1      ; #5 Open-loop-mode bit
M539->Y:$0B14,14,1      ; #5 Amplifier-enabled status bit
M540->Y:$0B14,0,1       ; #5 In-position bit
M541->Y:$0B14,1,1       ; #5 Warning-following error bit
M542->Y:$0B14,2,1       ; #5 Fatal-following-error bit
M543->Y:$0B14,3,1       ; #5 Amplifier-fault-error bit
M545->Y:$0B14,10,1      ; #5 Home-complete bit
                        ; Motor #5 Move Registers
M561->D:$0118           ; #5 Commanded position (1/[Ix08*32] cts)
M562->D:$011B           ; #5 Actual position (1/[Ix08*32] cts)
M563->D:$0B0B           ; #5 Target (end) position (1/[Ix08*32] cts)
M564->D:$0B13           ; #5 Position bias (1/[Ix08*32] cts)
```

```
M565->L:$081A           ; &1 B-axis target position (engineering units)
M566->X:$0123,0,24,S    ; #5 Actual velocity (1/[Ix09*32] cts/cyc)
M567->D:$011D           ; #5 Present master (handwheel) pos (1/[Ix07*32] cts
                        ; of master or (1/[Ix08*32] cts of slaved motor)
M568->X:$0135,8,16,S    ; #5 Filter Output (DAC bits)
M569->D:$0136           ; #5 Compensation correction
M570->D:$0131           ; #5 Present phase position; includes fraction in
                        ; Y-register
M571->X:$0131,0,24,S    ; #5 Present phase position (counts*Ix70)
M572->L:$0B2B           ; #5 Variable jog position/distance (counts)
M573->Y:$0B15,0,24,S    ; #5 Encoder home capture offset (counts)
                        ; Coordinate System &5 Status Bits
M580->X:$0B18,0,1       ; &5 Program-running bit
M581->Y:$0B17,21,1      ; &5 Circle-radius-error bit
M582->Y:$0B17,22,1      ; &5 Run-time-error bit
M584->X:$0B18,4,1       ; &5 Continuous motion request
M587->Y:$0B17,17,1      ; &5 In-position bit (AND of motors)
M588->Y:$0B17,18,1      ; &5 Warning-following-error bit (OR)
M589->Y:$0B17,19,1      ; &5 Fatal-following-error bit (OR)
M590->Y:$0B17,20,1      ; &5 Amp-fault-error bit (OR of motors)
                        ; Motor #5 Axis Definition Registers
M591->L:$0B22           ; #5 X/U/A/B/C-Axis scale factor (cts/unit)
M592->L:$0B23           ; #5 Y/V-Axis scale factor (cts/unit)
M593->L:$0B24           ; #5 Z/W-Axis scale factor (cts/unit)
M594->L:$0B25           ; #5 Axis offset (cts)
                        ; Coordinate System &5 Variables
M597->X:$0B06,0,24,S    ; &5 Host commanded time base (I10 units)
M598->X:$0B08,0,24,S    ; &5 Present time base (I10 units)
                        ; Registers associated with Encoder/DAC6
                        ; (Usually Motor #6)
M601->X:$C015,0,24,S    ; ENC6 24-bit counter position
M602->Y:$C012,8,16,S    ; DAC6 16-bit analog output
M603->X:$C017,0,24,S    ; ENC6 capture/compare position register
M604->X:$0725,0,24,S    ; ENC6 interpolated position (1/32 ct)
M605->Y:$C017,8,16,S    ; ADC6 16-bit analog input
M606->Y:$C014,0,24,U    ; ENC6 time between counts (SCLK cycles)
M610->X:$C014,10,1      ; ENC6 count-write enable control
M611->X:$C014,11,1      ; EQU6 compare flag latch control
M612->X:$C014,12,1      ; EQU6 compare output enable
M613->X:$C014,13,1      ; EQU6 compare invert enable
M614->X:$C014,14,1      ; AENA6/DIR6 Output
M616->X:$C014,16,1      ; EQU6 compare flag
M617->X:$C014,17,1      ; ENC6 position-captured flag
M618->X:$C014,18,1      ; ENC6 count-error flag
M619->X:$C014,19,1      ; ENC6 3rd channel input status
M620->X:$C014,20,1      ; HMFL6 input status
M621->X:$C014,21,1      ; -LIM6 input status
M622->X:$C014,22,1      ; +LIM6 input status
M623->X:$C014,23,1      ; FAULT6 input status
                        ; Motor #6 Status Bits
M630->Y:$0BD4,11,1      ; #6 Stopped-on-position-limit bit
M631->X:$0169,21,1      ; #6 Positive-end-limit-set bit
M632->X:$0169,22,1      ; #6 Negative-end-limit-set bit
M633->X:$0169,13,1      ; #6 Desired-velocity-zero bit
M635->X:$0169,15,1      ; #6 Dwell-in-progress bit
M637->X:$0169,17,1      ; #6 Running-program bit
M638->X:$0169,18,1      ; #6 Open-loop-mode bit
```

```
M639->Y:$0BD4,14,1      ; #6 Amplifier-enabled status bit
M640->Y:$0BD4,0,1       ; #6 In-position bit
M641->Y:$0BD4,1,1       ; #6 Warning-following error bit
M642->Y:$0BD4,2,1       ; #6 Fatal-following-error bit
M643->Y:$0BD4,3,1       ; #6 Amplifier-fault-error bit
M645->Y:$0BD4,10,1      ; #6 Home-complete bit
                        ; Motor #6 Move Registers
M661->D:$0154           ; #6 Commanded position (1/[Ix08*32] cts)
M662->D:$0157           ; #6 Actual position (1/[Ix08*32] cts)
M663->D:$0BCB           ; #6 Target (end) position (1/[Ix08*32] cts)
M664->D:$0BD3           ; #6 Position bias (1/[Ix08*32] cts)
M665->L:$081B           ; &1 C-axis target position (engineering units)
M666->X:$015F,0,24,S    ; #6 Actual velocity (1/[Ix09*32] cts/cyc)
M667->D:$0159           ; #6 Present master (handwheel) pos (1/[Ix07*32] cts
                        ; of master or (1/[Ix08*32] cts of slaved motor)
M668->X:$0171,8,16,S    ; #6 Filter Output (DAC bits)
M669->D:$0172           ; #6 Compensation correction
M670->D:$016D           ; #6 Present phase position; includes fraction in
                        ; Y-register
M671->X:$016D,0,24,S    ; #6 Present phase position (counts*Ix70)
M672->L:$0BEB           ; #6 Variable jog position/distance (counts)
M673->Y:$0BD5,0,24,S    ; #6 Encoder home capture offset (counts)
                        ; Coordinate System &6 Status Bits
M680->X:$0BD8,0,1       ; &6 Program-running bit
M681->Y:$0BD7,21,1      ; &6 Circle-radius-error bit
M682->Y:$0BD7,22,1      ; &6 Run-time-error bit
M684->X:$0BD8,4,1       ; &6 Continuous motion request
M687->Y:$0BD7,17,1      ; &6 In-position bit (AND of motors)
M688->Y:$0BD7,18,1      ; &6 Warning-following-error bit (OR)
M689->Y:$0BD7,19,1      ; &6 Fatal-following-error bit (OR)
M690->Y:$0BD7,20,1      ; &6 Amp-fault-error bit (OR of motors)
                        ; Motor #6 Axis Definition Registers
M691->L:$0BE2           ; #6 X/U/A/B/C-Axis scale factor (cts/unit)
M692->L:$0BE3           ; #6 Y/V-Axis scale factor (cts/unit)
M693->L:$0BE4           ; #6 Z/W-Axis scale factor (cts/unit)
M694->L:$0BE5           ; #6 Axis offset (cts)
                        ; Coordinate System &6 Variables
M697->X:$0BC6,0,24,S    ; &6 Host commanded time base (I10 units)
M698->X:$0BC8,0,24,S    ; &6 Present time base (I10 units)
                        ; Registers associated with Encoder/DAC7
                        ; (Usually Motor #7)
M701->X:$C019,0,24,S    ; ENC7 24-bit counter position
M702->Y:$C01B,8,16,S    ; DAC7 16-bit analog output
M703->X:$C01B,0,24,S    ; ENC7 capture/compare position register
M704->X:$0726,0,24,S    ; ENC7 interpolated position (1/32 ct)
M705->Y:$C01E,8,16,S    ; ADC7 16-bit analog input
M706->Y:$C018,0,24,U    ; ENC7 time between counts (SCLK cycles)
M710->X:$C018,10,1      ; ENC7 count-write enable control
M711->X:$C018,11,1      ; EQU7 compare flag latch control
M712->X:$C018,12,1      ; EQU7 compare output enable
M713->X:$C018,13,1      ; EQU7 compare invert enable
M714->X:$C018,14,1      ; AENA7/DIR7 Output
M716->X:$C018,16,1      ; EQU7 compare flag
M717->X:$C018,17,1      ; ENC7 position-captured flag
M718->X:$C018,18,1      ; ENC7 count-error flag
M719->X:$C018,19,1      ; ENC7 3rd channel input status
M720->X:$C018,20,1      ; HMFL7 input status
```

```
M721->X:$C018,21,1      ; -LIM7 input status
M722->X:$C018,22,1      ; +LIM7 input status
M723->X:$C018,23,1      ; FAULT7 input status
                        ; Motor #7 Status Bits
M730->Y:$0C94,11,1      ; #7 Stopped-on-position-limit bit
M731->X:$01A5,21,1      ; #7 Positive-end-limit-set bit
M732->X:$01A5,22,1      ; #7 Negative-end-limit-set bit
M733->X:$01A5,13,1      ; #7 Desired-velocity-zero bit
M735->X:$01A5,15,1      ; #7 Dwell-in-progress bit
M737->X:$01A5,17,1      ; #7 Running-program bit
M738->X:$01A5,18,1      ; #7 Open-loop-mode bit
M739->Y:$0C94,14,1      ; #7 Amplifier-enabled status bit
M740->Y:$0C94,0,1       ; #7 In-position bit
M741->Y:$0C94,1,1       ; #7 Warning-following error bit
M742->Y:$0C94,2,1       ; #7 Fatal-following-error bit
M743->Y:$0C94,3,1       ; #7 Amplifier-fault-error bit
M745->Y:$0C94,10,1      ; #7 Home-complete bit
                        ; Motor #7 Move Registers
M761->D:$0190           ; #7 Commanded position (1/[Ix08*32] cts)
M762->D:$0193           ; #7 Actual position (1/[Ix08*32] cts)
M763->D:$0C8B           ; #7 Target (end) position (1/[Ix08*32] cts)
M764->D:$0C93           ; #7 Position bias (1/[Ix08*32] cts)
M765->L:$081C           ; &1 U-axis target position (engineering units)
M766->X:$019B,0,24,S    ; #7 Actual velocity (1/[Ix09*32] cts/cyc)
M767->D:$0195           ; #7 Present master (handwheel) pos (1/[Ix07*32] cts
                        ; of master or (1/[Ix08*32] cts of slaved motor)
M768->X:$01AD,8,16,S    ; #7 Filter Output (DAC bits)
M769->D:$01AE           ; #7 Compensation correction
M770->D:$01A9           ; #7 Present phase position; includes fraction in
                        ; Y-register
M771->X:$01A9,0,24,S    ; #7 Present phase position (counts*Ix70)
M772->L:$0CAB           ; #7 Variable jog position/distance (counts)
M773->Y:$0C95,0,24,S    ; #7 Encoder home capture offset (counts)
                        ; Coordinate System &7 Status Bits
M780->X:$0C98,0,1       ; &7 Program-running bit
M781->Y:$0C97,21,1      ; &7 Circle-radius-error bit
M782->Y:$0C97,22,1      ; &7 Run-time-error bit
M784->X:$0C98,4,1       ; &7Continuous motion request
M787->Y:$0C97,17,1      ; &7 In-position bit (AND of motors)
M788->Y:$0C97,18,1      ; &7 Warning-following-error bit (OR)
M789->Y:$0C97,19,1      ; &7 Fatal-following-error bit (OR)
M790->Y:$0C97,20,1      ; &7 Amp-fault-error bit (OR of motors)
                        ; Motor #7 Axis Definition Registers
M791->L:$0CA2           ; #7 X/U/A/B/C-Axis scale factor (cts/unit)
M792->L:$0CA3           ; #7 Y/V-Axis scale factor (cts/unit)
M793->L:$0CA4           ; #7 Z/W-Axis scale factor (cts/unit)
M794->L:$0CA5           ; #7 Axis offset (cts)
                        ; Coordinate System &7 Variables
M797->X:$0C86,0,24,S    ; &7 Host commanded time base (I10 units)
M798->X:$0C88,0,24,S    ; &7 Present time base (I10 units)
                        ; Registers associated with Encoder/DAC8
                        ; (Usually Motor #8)
M801->X:$C01D,0,24,S    ; ENC8 24-bit counter position
M802->Y:$C01A,8,16,S    ; DAC8 16-bit analog output
M803->X:$C01F,0,24,S    ; ENC8 capture/compare position register
M804->X:$0727,0,24,S    ; ENC8 interpolated position (1/32 ct)
M805->Y:$C01F,8,16,S    ; ADC8 16-bit analog input
```

```
M806->Y:$C01C,0,24,U    ; ENC8 time between counts (SCLK cycles)
M810->X:$C01C,10,1      ; ENC8 count-write enable control
M811->X:$C01C,11,1      ; EQU8 compare flag latch control
M812->X:$C01C,12,1      ; EQU8 compare output enable
M813->X:$C01C,13,1      ; EQU8 compare invert enable
M814->X:$C01C,14,1      ; AENA8/DIR8 Output
M816->X:$C01C,16,1      ; EQU8 compare flag
M817->X:$C01C,17,1      ; ENC8 position-captured flag
M818->X:$C01C,18,1      ; ENC8 count-error flag
M819->X:$C01C,19,1      ; ENC8 3rd channel input status
M820->X:$C01C,20,1      ; HMFL8 input status
M821->X:$C01C,21,1      ; -LIM8 input status
M822->X:$C01C,22,1      ; +LIM8 input status
M823->X:$C01C,23,1      ; FAULT8 input status
                        ; Motor #8 Status Bits
M830->Y:$0D54,11,1      ; #8 Stopped-on-position-limit bit
M831->X:$01E1,21,1      ; #8 Positive-end-limit-set bit
M832->X:$01E1,22,1      ; #8 Negative-end-limit-set bit
M833->X:$01E1,13,1      ; #8 Desired-velocity-zero bit
M835->X:$01E1,15,1      ; #8 Dwell-in-progress bit
M837->X:$01E1,17,1      ; #8 Running-program bit
M838->X:$01E1,18,1      ; #8 Open-loop-mode bit
M839->Y:$0D54,14,1      ; #8 Amplifier-enabled status bit
M840->Y:$0D54,0,1       ; #8 In-position bit
M841->Y:$0D54,1,1       ; #8 Warning-following error bit
M842->Y:$0D54,2,1       ; #8 Fatal-following-error bit
M843->Y:$0D54,3,1       ; #8 Amplifier-fault-error bit
M845->Y:$0D54,10,1      ; #8 Home-complete bit
                        ; Motor #8 Move Registers
M861->D:$01CC           ; #8 Commanded position (1/[Ix08*32] cts)
M862->D:$01CF           ; #8 Actual position (1/[Ix08*32] cts)
M863->D:$0D4B           ; #8 Target (end) position (1/[Ix08*32] cts)
M864->D:$0D53           ; #8 Position bias (1/[Ix08*32] cts)
M865->L:$081D           ; &1 V-axis target position (engineering units)
M866->X:$01D7,0,24,S    ; #8 Actual velocity (1/[Ix09*32] cts/cyc)
M867->D:$01D1           ; #8 Present master (handwheel) pos (1/[Ix07*32] cts
                        ; of master or (1/[Ix08*32] cts of slaved motor)
M868->X:$01E9,8,16,S    ; #8 Filter Output (DAC bits)
M869->D:$01EA           ; #8 Compensation correction
M870->D:$01E5           ; #8 Present phase position; includes fraction in
                        ; Y-register
M871->X:$01E5,0,24,S    ; #8 Present phase position (counts*Ix70)
M872->L:$0D6B           ; #8 Variable jog position/distance (counts)
M873->Y:$0D55,0,24,S    ; #8 Encoder home capture offset (counts)
                        ; Coordinate System &8 Status Bits
M880->X:$0D58,0,1       ; &8 Program-running bit
M881->Y:$0D57,21,1      ; &8 Circle-radius-error bit
M882->Y:$0D57,22,1      ; &8 Run-time-error bit
M884->X:$0D58,4,1       ; &8 Continuous motion request
M887->Y:$0D57,17,1      ; &8 In-position bit (AND of motors)
M888->Y:$0D57,18,1      ; &8 Warning-following-error bit (OR)
M889->Y:$0D57,19,1      ; &8 Fatal-following-error bit (OR)
M890->Y:$0D57,20,1      ; &8 Amp-fault-error bit (OR of motors)
                        ; Motor #8 Axis Definition Registers
M891->L:$0D62           ; #8 X/U/A/B/C-Axis scale factor (cts/unit)
M892->L:$0D63           ; #8 Y/V-Axis scale factor (cts/unit)
M893->L:$0D64           ; #8 Z/W-Axis scale factor (cts/unit)
```

```
M894->L:$0D65           ; #8 Axis offset (cts)
                        ; Coordinate System &8 Variables
M897->X:$0D46,0,24,S    ; &8 Host commanded time base (I10 units)
M898->X:$0D48,0,24,S    ; &8 Present time base (I10 units)
                        ; Accessory 14 I/O M-Variables (1st ACC-14)
M900->Y:$FFD0,0,1       ; MI/O0
M901->Y:$FFD0,1,1       ; MI/O1
M902->Y:$FFD0,2,1       ; MI/O2
M903->Y:$FFD0,3,1       ; MI/O3
M904->Y:$FFD0,4,1       ; MI/O4
M905->Y:$FFD0,5,1       ; MI/O5
M906->Y:$FFD0,6,1       ; MI/O6
M907->Y:$FFD0,7,1       ; MI/O7
M908->Y:$FFD0,8,1       ; MI/O8
M909->Y:$FFD0,9,1       ; MI/O9
M910->Y:$FFD0,10,1      ; MI/O10
M911->Y:$FFD0,11,1      ; MI/O11
M912->Y:$FFD0,12,1      ; MI/O12
M913->Y:$FFD0,13,1      ; MI/O13
M914->Y:$FFD0,14,1      ; MI/O14
M915->Y:$FFD0,15,1      ; MI/O15
M916->Y:$FFD0,16,1      ; MI/O16
M917->Y:$FFD0,17,1      ; MI/O17
M918->Y:$FFD0,18,1      ; MI/O18
M919->Y:$FFD0,19,1      ; MI/O19
M920->Y:$FFD0,20,1      ; MI/O20
M921->Y:$FFD0,21,1      ; MI/O21
M922->Y:$FFD0,22,1      ; MI/O22
M923->Y:$FFD0,23,1      ; MI/O23
M924->Y:$FFD1,0,1       ; MI/O24
M925->Y:$FFD1,1,1       ; MI/O25
M926->Y:$FFD1,2,1       ; MI/O26
M927->Y:$FFD1,3,1       ; MI/O27
M928->Y:$FFD1,4,1       ; MI/O28
M929->Y:$FFD1,5,1       ; MI/O29
M930->Y:$FFD1,6,1       ; MI/O30
M931->Y:$FFD1,7,1       ; MI/O31
M932->Y:$FFD1,8,1       ; MI/O32
M933->Y:$FFD1,9,1       ; MI/O33
M934->Y:$FFD1,10,1      ; MI/O34
M935->Y:$FFD1,11,1      ; MI/O35
M936->Y:$FFD1,12,1      ; MI/O36
M937->Y:$FFD1,13,1      ; MI/O37
M938->Y:$FFD1,14,1      ; MI/O38
M939->Y:$FFD1,15,1      ; MI/O39
M940->Y:$FFD1,16,1      ; MI/O40
M941->Y:$FFD1,17,1      ; MI/O41
M942->Y:$FFD1,18,1      ; MI/O42
M943->Y:$FFD1,19,1      ; MI/O43
M944->Y:$FFD1,20,1      ; MI/O44
M945->Y:$FFD1,21,1      ; MI/O45
M946->Y:$FFD1,22,1      ; MI/O46
M947->Y:$FFD1,23,1      ; MI/O47
```

Example 1: Suggested M-Variable Definitions (PMAC2 Specific)



These are only suggestions; the user is free to make whatever definitions are desired.

This file contains suggested definitions for M-variables on the PMAC2. It is similar to the file for the PMAC(1) family of boards, but there are significant differences in the input/output definitions, both for servo registers and general-purpose I/O.

```
; Clear existing definitions
CLOSE                          ; Make sure no buffer is open on PMAC2
M0..1023->*                    ; All M-variables are now self-referenced
; JI/O Port M-variables
M0->Y:$C080,0                  ; I/O00 Data Line; J3 Pin 1
M1->Y:$C080,1                  ; I/O01 Data Line; J3 Pin 2
M2->Y:$C080,2                  ; I/O02 Data Line; J3 Pin 3
M3->Y:$C080,3                  ; I/O03 Data Line; J3 Pin 4
M4->Y:$C080,4                  ; I/O04 Data Line; J3 Pin 5
M5->Y:$C080,5                  ; I/O05 Data Line; J3 Pin 6
M6->Y:$C080,6                  ; I/O06 Data Line; J3 Pin 7
M7->Y:$C080,7                  ; I/O07 Data Line; J3 Pin 8
M8->Y:$C080,8                  ; I/O08 Data Line; J3 Pin 9
M9->Y:$C080,9                  ; I/O09 Data Line; J3 Pin 10
M10->Y:$C080,10                ; I/O10 Data Line; J3 Pin 11
M11->Y:$C080,11                ; I/O11 Data Line; J3 Pin 12
M12->Y:$C080,12                ; I/O12 Data Line; J3 Pin 13
M13->Y:$C080,13                ; I/O13 Data Line; J3 Pin 14
M14->Y:$C080,14                ; I/O14 Data Line; J3 Pin 15
M15->Y:$C080,15                ; I/O15 Data Line; J3 Pin 16
M16->Y:$C080,16                ; I/O16 Data Line; J3 Pin 17
M17->Y:$C080,17                ; I/O17 Data Line; J3 Pin 18
M18->Y:$C080,18                ; I/O18 Data Line; J3 Pin 19
M19->Y:$C080,19                ; I/O19 Data Line; J3 Pin 20
M20->Y:$C080,20                ; I/O20 Data Line; J3 Pin 21
M21->Y:$C080,21                ; I/O21 Data Line; J3 Pin 22
M22->Y:$C080,22                ; I/O22 Data Line; J3 Pin 23
M23->Y:$C080,23                ; I/O23 Data Line; J3 Pin 24
M24->Y:$C081,0                 ; I/O24 Data Line; J3 Pin 25
M25->Y:$C081,1                 ; I/O25 Data Line; J3 Pin 26
M26->Y:$C081,2                 ; I/O26 Data Line; J3 Pin 27
M27->Y:$C081,3                 ; I/O27 Data Line; J3 Pin 28
M28->Y:$C081,4                 ; I/O28 Data Line; J3 Pin 29
M29->Y:$C081,5                 ; I/O29 Data Line; J3 Pin 30
M30->Y:$C081,6                 ; I/O30 Data Line; J3 Pin 31
M31->Y:$C081,7                 ; I/O31 Data Line; J3 Pin 32
M32->X:$C080,0,8               ; Direction control for I/O00 to I/O07
M33->Y:$E800,0                 ; Buffer direction control for I/O00 to I/O07
M34->X:$C080,8,8               ; Direction control for I/O08 to I/O15
M35->Y:$E800,1                 ; Buffer direction control for I/O08 to I/O15
M36->X:$C080,16,8              ; Direction control for I/O16 to I/O23
M37->Y:$E800,2                 ; Buffer direction control for I/O16 to I/O23
M38->X:$C081,0,8               ; Direction control for I/O24 to I/O31
M39->Y:$E800,3                 ; Buffer direction control for I/O24 to I/O31
```

```
; JTHW Thumbwheel Multiplexer Port M-variables
M40->Y:$C082,8                ; SEL0 Line; J2 Pin 4
M41->Y:$C082,9                ; SEL1 Line; J2 Pin 6
M42->Y:$C082,10               ; SEL2 Line; J2 Pin 8
M43->Y:$C082,11               ; SEL3 Line; J2 Pin 10
M44->Y:$C082,12               ; SEL4 Line; J2 Pin 12
M45->Y:$C082,13               ; SEL5 Line; J2 Pin 14
M46->Y:$C082,14               ; SEL6 Line; J2 Pin 16
M47->Y:$C082,15               ; SEL7 Line; J2 Pin 18
M48->Y:$C082,8,8,U            ; SEL0-7 Lines treated as a byte
M50->Y:$C082,0                ; DAT0 Line; J2 Pin 3
M51->Y:$C082,1                ; DAT1 Line; J2 Pin 5
M52->Y:$C082,2                ; DAT2 Line; J2 Pin 7
M53->Y:$C082,3                ; DAT3 Line; J2 Pin 9
M54->Y:$C082,4                ; DAT4 Line; J2 Pin 11
M55->Y:$C082,5                ; DAT5 Line; J2 Pin 13
M56->Y:$C082,6                ; DAT6 Line; J2 Pin 15
M57->Y:$C082,7                ; DAT7 Line; J2 Pin 17
M58->Y:$C082,0,8,U            ; DAT0-7 Lines treated as a byte
M60->X:$C082,0,8              ; Direction control for DAT0 to DAT7
M61->Y:$E800,4                ; Buffer direction control for DAT0 to DAT7, PCbus
;M61->Y:$E802,0               ; Buffer direction control for DAT0 to DAT7,
                              ; VMEbus
M62->X:$C080,8,8              ; Direction control for SEL0 to SEL7
M63->Y:$E800,5                ; Buffer direction control for SEL0 to SEL7, PCbus
;M63->Y:$E802,1               ; Buffer direction control for SEL0 to SEL7,
                              ; VMEbus
; User timer registers -- count down once per servo cycle
M70->Y:$0700,0,24,S           ; 24-bit countdown user timer
M71->X:$0700,0,24,S           ; 24-bit countdown user timer
M72->Y:$0701,0,24,S           ; 24-bit countdown user timer
M73->X:$0701,0,24,S           ; 24-bit countdown user timer
; Servo cycle counter (read only) -- counts up once per servo cycle
M100->X:$0000,0,24,S          ; 24-bit servo cycle counter
; Gate Array Registers for Channel 1
M101->X:$C001,0,24,S          ; ENC1 24-bit counter position
M102->Y:$C002,8,16,S          ; OUT1A command value; DAC or PWM
M103->X:$C003,0,24,S          ; ENC1 captured position
M104->Y:$C003,8,16,S          ; OUT1B command value; DAC or PWM
M105->X:$0710,8,16,S          ; ADC1A input image value
M106->Y:$0710,8,16,S          ; ADC1B input image value
M107->Y:$C004,8,16,S          ; OUT1C command value; PFM or PWM
M108->Y:$C007,0,24,S          ; ENC1 compare A position
M109->X:$C007,0,24,S          ; ENC1 compare B position
M110->X:$C006,0,24,S          ; ENC1 compare autoincrement value
M111->X:$C005,11              ; ENC1 compare initial state write enable
M112->X:$C005,12              ; ENC1 compare initial state
M114->X:$C005,14              ; AENA1 output status
M115->X:$C000,19              ; USER1 flag input status
M116->X:$C000,9               ; ENC1 compare output value
M117->X:$C000,11              ; ENC1 capture flag
M118->X:$C000,8               ; ENC1 count error flag
M119->X:$C000,14              ; CHC1 input status
M120->X:$C000,16              ; HMFL1 flag input status
M121->X:$C000,17              ; PLIM1 flag input status
M122->X:$C000,18              ; MLIM1 flag input status
M123->X:$C000,15              ; FAULT1 flag input status
```

```
M124->X:$C000,20              ; Channel 1 W flag input status
M125->X:$C000,21              ; Channel 1 V flag input status
M126->X:$C000,22              ; Channel 1 U flag input status
M127->X:$C000,23              ; Channel 1 T flag input status
M128->X:$C000,20,4            ; Channel 1 TUVW inputs as 4-bit value
; Motor #1 Status Bits
M130->Y:$0814,11,1            ; #1 Stopped-on-position-limit bit
M131->X:$003D,21,1            ; #1 Positive-end-limit-set bit
M132->X:$003D,22,1            ; #1 Negative-end-limit-set bit
M133->X:$003D,13,1            ; #1 Desired-velocity-zero bit
M135->X:$003D,15,1            ; #1 Dwell-in-progress bit
M137->X:$003D,17,1            ; #1 Running-program bit
M138->X:$003D,18,1            ; #1 Open-loop-mode bit
M139->Y:$0814,14,1            ; #1 Amplifier-enabled status bit
M140->Y:$0814,0,1             ; #1 In-position bit
M141->Y:$0814,1,1             ; #1 Warning-following error bit
M142->Y:$0814,2,1             ; #1 Fatal-following-error bit
M143->Y:$0814,3,1             ; #1 Amplifier-fault-error bit
M145->Y:$0814,10,1            ; #1 Home-complete bit
; Motor #1 Move Registers
M161->D:$0028                 ; #1 Commanded position (1/[Ix08*32] cts)
M162->D:$002B                 ; #1 Actual position (1/[Ix08*32] cts)
M163->D:$080B                 ; #1 Target (end) position (1/[Ix08*32] cts)
M164->D:$0813                 ; #1 Position bias (1/[Ix08*32] cts)
M165->L:$081F                 ; &1 X-axis target position (engineering units)
M166->X:$0033,0,24,S          ; #1 Actual velocity (1/[Ix09*32] cts/cyc)
M167->D:$002D                 ; #1 Present master pos (1/[Ix07*32] cts)
M168->X:$0043,8,16,S          ; #1 Filter Output (DAC bits)
M169->D:$004A                 ; #1 Compensation correction (1/[Ix08*32] cts)
M170->D:$0040                 ; #1 Present phase position (including fraction)
M171->X:$0040,24,S            ; #1 Present phase position (counts *Ix70)
M172->L:$082B                 ; #1 Variable jog position/distance (cts)
M173->Y:$0815,24,S            ; #1 Encoder home capture position (cts)
M174->Y:$082A,24,S            ; #1 Averaged actual velocity
                              ; (1/[Ix09*32] cts/cyc)
; Coordinate System &1 Status Bits
M180->X:$0818,0,1             ; &1 Program-running bit
M181->Y:$0817,21,1            ; &1 Circle-radius-error bit
M182->Y:$0817,22,1            ; &1 Run-time-error bit
M184->X:$0818,0,4             ; &1 Continuous motion request
M187->Y:$0817,17,1            ; &1 In-position bit (AND of motors)
M188->Y:$0817,18,1            ; &1 Warning-following-error bit (OR)
M189->Y:$0817,19,1            ; &1 Fatal-following-error bit (OR)
M190->Y:$0817,20,1            ; &1 Amp-fault-error bit (OR of motors)
; Motor #1 Axis Definition Registers
M191->L:$0822                 ; #1 X/U/A/B/C-Axis scale factor (cts/unit)
M192->L:$0823                 ; #1 Y/V-Axis scale factor (cts/unit)
M193->L:$0824                 ; #1 Z/W-Axis scale factor (cts/unit)
M194->L:$0825                 ; #1 Axis offset (cts)
; Coordinate System &1 Variables
M197->X:$0806,0,24,S          ; &1 Host commanded time base (I10 units)
M198->X:$0808,0,24,S          ; &1 Present time base (I10 units)
; Gate Array Registers for Channel 2
M201->X:$C009,0,24,S          ; ENC2 24-bit counter position
M202->Y:$C00A,8,16,S          ; OUT2A command value; DAC or PWM
M203->X:$C00B,0,24,S          ; ENC2 captured position
M204->Y:$C00B,8,16,S          ; OUT2B command value; DAC or PWM
```

```
M205->X:$0711,8,16,S          ; ADC2A input image value
M206->Y:$0711,8,16,S          ; ADC2B input image value
M207->Y:$C00C,8,16,S          ; OUT2C command value; PFM or PWM
M208->Y:$C00F,0,24,S          ; ENC2 compare A position
M209->X:$C00F,0,24,S          ; ENC2 compare B position
M210->X:$C00E,0,24,S          ; ENC2 compare autoincrement value
M211->X:$C00D,11              ; ENC2 compare initial state write enable
M212->X:$C00D,12              ; ENC2 compare initial state
M214->X:$C00D,14              ; AENA2 output status
M215->X:$C008,19              ; USER2 flag input status
M216->X:$C008,9               ; ENC2 compare output value
M217->X:$C008,11              ; ENC2 capture flag
M218->X:$C008,8               ; ENC2 count error flag
M219->X:$C008,14              ; CHC2 input status
M220->X:$C008,16              ; HMFL2 flag input status
M221->X:$C008,17              ; PLIM2 flag input status
M222->X:$C008,18              ; MLIM2 flag input status
M223->X:$C008,15              ; FAULT2 flag input status
M224->X:$C008,20              ; Channel 2 W flag input status
M225->X:$C008,21              ; Channel 2 V flag input status
M226->X:$C008,22              ; Channel 2 U flag input status
M227->X:$C008,23              ; Channel 2 T flag input status
M228->X:$C008,20,4            ; Channel 2 TUVW inputs as 4-bit value
; Motor #2 Status Bits
M230->Y:$08D4,11,1            ; #2 Stopped-on-position-limit bit
M231->X:$0079,21,1            ; #2 Positive-end-limit-set bit
M232->X:$0079,22,1            ; #2 Negative-end-limit-set bit
M233->X:$0079,13,1            ; #2 Desired-velocity-zero bit
M235->X:$0079,15,1            ; #2 Dwell-in-progress bit
M237->X:$0079,17,1            ; #2 Running-program bit
M238->X:$0079,18,1            ; #2 Open-loop-mode bit
M239->Y:$08D4,14,1            ; #2 Amplifier-enabled status bit
M240->Y:$08D4,0,1             ; #2 In-position bit
M241->Y:$08D4,1,1             ; #2 Warning-following error bit
M242->Y:$08D4,2,1             ; #2 Fatal-following-error bit
M243->Y:$08D4,3,1             ; #2 Amplifier-fault-error bit
M245->Y:$08D4,10,1            ; #2 Home-complete bit
; Motor #2 Move Registers
M261->D:$0064                 ; #2 Commanded position (1/[Ix08*32] cts)
M262->D:$0067                 ; #2 Actual position (1/[Ix08*32] cts)
M263->D:$08CB                 ; #2 Target (end) position (1/[Ix08*32] cts)
M264->D:$08D3                 ; #2 Position bias (1/[Ix08*32] cts)
M265->L:$0820                 ; &1 Y-axis target position (engineering units)
M266->X:$006F,0,24,S          ; #2 Actual velocity (1/[Ix09*32] cts/cyc)
M267->D:$0069                 ; #2 Present master pos (1/[Ix07*32] cts)
M268->X:$007F,8,16,S          ; #2 Filter Output (DAC bits)
M269->D:$0086                 ; #2 Compensation correction (1/[Ix08*32] cts)
M270->D:$007C                 ; #2 Present phase position (including fraction)
M271->X:$007C,24,S            ; #2 Present phase position (counts *Ix70)
M272->L:$08EB                 ; #2 Variable jog position/distance (cts)
M273->Y:$08D5,24,S            ; #2 Encoder home capture position (cts)
M274->Y:$08EA,24,S            ; #2 Averaged actual velocity
                              ; (1/[Ix09*32] cts/cyc)
; Coordinate System &2 Status Bits
M280->X:$08D8,0,1             ; &2 Program-running bit
M281->Y:$08D7,21,1            ; &2 Circle-radius-error bit
M282->Y:$08D7,22,1            ; &2 Run-time-error bit
```

```
M284->X:$08D8,0,4             ; &2 Continuous motion request
M287->Y:$08D7,17,1            ; &2 In-position bit (AND of motors)
M288->Y:$08D7,18,1            ; &2 Warning-following-error bit (OR)
M289->Y:$08D7,19,1            ; &2 Fatal-following-error bit (OR)
M290->Y:$08D7,20,1            ; &2 Amp-fault-error bit (OR of motors)
; Motor #2 Axis Definition Registers
M291->L:$08E2                 ; #2 X/U/A/B/C-Axis scale factor (cts/unit)
M292->L:$08E3                 ; #2 Y/V-Axis scale factor (cts/unit)
M293->L:$08E4                 ; #2 Z/W-Axis scale factor (cts/unit)
M294->L:$08E5                 ; #2 Axis offset (cts)
; Coordinate System &2 Variables
M297->X:$08C6,0,24,S          ; &2 Host commanded time base (I10 units)
M298->X:$08C8,0,24,S          ; &2 Present time base (I10 units)
; Gate Array Registers for Channel 3
M301->X:$C011,0,24,S          ; ENC3 24-bit counter position
M302->Y:$C012,8,16,S          ; OUT3A command value; DAC or PWM
M303->X:$C013,0,24,S          ; ENC3 captured position
M304->Y:$C013,8,16,S          ; OUT3B command value; DAC or PWM
M305->X:$0712,8,16,S          ; ADC3A input image value
M306->Y:$0712,8,16,S          ; ADC3B input image value
M307->Y:$C014,8,16,S          ; OUT3C command value; PFM or PWM
M308->Y:$C017,0,24,S          ; ENC3 compare A position
M309->X:$C017,0,24,S          ; ENC3 compare B position
M310->X:$C016,0,24,S          ; ENC3 compare autoincrement value
M311->X:$C015,11              ; ENC3 compare initial state write enable
M312->X:$C015,12              ; ENC3 compare initial state
M314->X:$C015,14              ; AENA3 output status
M315->X:$C010,19              ; USER3 flag input status
M316->X:$C010,9               ; ENC3 compare output value
M317->X:$C010,11              ; ENC3 capture flag
M318->X:$C010,8               ; ENC3 count error flag
M319->X:$C010,14              ; CHC3 input status
M320->X:$C010,16              ; HMFL3 flag input status
M321->X:$C010,17              ; PLIM3 flag input status
M322->X:$C010,18              ; MLIM3 flag input status
M323->X:$C010,15              ; FAULT3 flag input status
M324->X:$C010,20              ; Channel 3 W flag input status
M325->X:$C010,21              ; Channel 3 V flag input status
M326->X:$C010,22              ; Channel 3 U flag input status
M327->X:$C010,23              ; Channel 3 T flag input status
M328->X:$C010,20,4            ; Channel 3 TUVW inputs as 4-bit value
; Motor #3 Status Bits
M330->Y:$0994,11,1            ; #3 Stopped-on-position-limit bit
M331->X:$00B5,21,1            ; #3 Positive-end-limit-set bit
M332->X:$00B5,22,1            ; #3 Negative-end-limit-set bit
M333->X:$00B5,13,1            ; #3 Desired-velocity-zero bit
M335->X:$00B5,15,1            ; #3 Dwell-in-progress bit
M337->X:$00B5,17,1            ; #3 Running-program bit
M338->X:$00B5,18,1            ; #3 Open-loop-mode bit
M339->Y:$0994,14,1            ; #3 Amplifier-enabled status bit
M340->Y:$0994,0,1             ; #3 In-position bit
M341->Y:$0994,1,1             ; #3 Warning-following error bit
M342->Y:$0994,2,1             ; #3 Fatal-following-error bit
M343->Y:$0994,3,1             ; #3 Amplifier-fault-error bit
M345->Y:$0994,10,1            ; #3 Home-complete bit
; Motor #3 Move Registers
M361->D:$00A0                 ; #3 Commanded position (1/[Ix08*32] cts)
```

```
M362->D:$00A3                ; #3 Actual position (1/[Ix08*32] cts)
M363->D:$098B                ; #3 Target (end) position (1/[Ix08*32] cts)
M364->D:$0993                ; #3 Position bias (1/[Ix08*32] cts)
M365->L:$0821                ; &1 Z-axis target position (engineering units)
M366->X:$00AB,0,24,S         ; #3 Actual velocity (1/[Ix09*32] cts/cyc)
M367->D:$00A5                ; #3 Present master pos (1/[Ix07*32] cts)
M368->X:$00BB,8,16,S         ; #3 Filter Output (DAC bits)
M369->X:$00C2                ; #3 Compensation correction (1/[Ix08*32] cts)
M370->D:$00B8                ; #3 Present phase position (including fraction)
M371->X:$00B8,24,S           ; #3 Present phase position (counts*Ix70)
M372->L:$09AB                ; #3 Variable jog position/distance (cts)
M373->Y:$0995,24,S           ; #3 Encoder home capture position (cts)
M374->Y:$09AA,24,S           ; #3 Averaged actual velocity
                             ; (1/[Ix09*32] cts/cyc)
; Coordinate System &3 Status Bits
M380->X:$0998,0,1            ; &3 Program-running bit
M381->Y:$0997,21,1           ; &3 Circle-radius-error bit
M382->Y:$0997,22,1           ; &3 Run-time-error bit
M384->X:$0998,0,4            ; &3 Continuous motion request
M387->Y:$0997,17,1           ; &3 In-position bit (AND of motors)
M388->Y:$0997,18,1           ; &3 Warning-following-error bit (OR)
M389->Y:$0997,19,1           ; &3 Fatal-following-error bit (OR)
M390->Y:$0997,20,1           ; &3 Amp-fault-error bit (OR of motors)
; Motor #3 Axis Definition Registers
M391->L:$09A2                ; #3 X/U/A/B/C-Axis scale factor (cts/unit)
M392->L:$09A3                ; #3 Y/V-Axis scale factor (cts/unit)
M393->L:$09A4                ; #3 Z/W-Axis scale factor (cts/unit)
M394->L:$09A5                ; #3 Axis offset (cts)
; Coordinate System &3 Variables
M397->X:$0986,0,24,S         ; &3 Host commanded time base (I10 units)
M398->X:$0988,0,24,S         ; &3 Present time base (I10 units)
; Gate Array Registers for Channel 4
M401->X:$C019,0,24,S         ; ENC4 24-bit counter position
M402->Y:$C01A,8,16,S         ; OUT4A command value; DAC or PWM
M403->X:$C01B,0,24,S         ; ENC4 captured position
M404->Y:$C01B,8,16,S         ; OUT4B command value; DAC or PWM
M405->X:$0713,8,16,S         ; ADC4A input image value
M406->Y:$0713,8,16,S         ; ADC4B input image value
M407->Y:$C01C,8,16,S         ; OUT4C command value; PFM or PWM
M408->Y:$C01F,0,24,S         ; ENC4 compare A position
M409->X:$C01F,0,24,S         ; ENC4 compare B position
M410->X:$C01E,0,24,S         ; ENC4 compare autoincrement value
M411->X:$C01D,11             ; ENC4 compare initial state write enable
M412->X:$C01D,12             ; ENC4 compare initial state
M414->X:$C01D,14             ; AENA4 output status
M415->X:$C018,19             ; USER4 flag input status
M416->X:$C018,9              ; ENC4 compare output value
M417->X:$C018,11             ; ENC4 capture flag
M418->X:$C018,8              ; ENC4 count error flag
M419->X:$C018,14             ; HMFL4 flag input status
M420->X:$C018,16             ; CHC4 input status
M421->X:$C018,17             ; PLIM4 flag input status
M422->X:$C018,18             ; MLIM4 flag input status
M423->X:$C018,15             ; FAULT4 flag input status
M424->X:$C018,20             ; Channel 4 W flag input status
M425->X:$C018,21             ; Channel 4 V flag input status
M426->X:$C018,22             ; Channel 4 U flag input status
```

```
M427->X:$C018,23              ; Channel 4 T flag input status
M428->X:$C018,20,4            ; Channel 4 TUVW inputs as 4-bit value
; Motor #4 Status Bits
M430->Y:$0A54,11,1            ; #4 Stopped-on-position-limit bit
M431->X:$00F1,21,1            ; #4 Positive-end-limit-set bit
M432->X:$00F1,22,1            ; #4 Negative-end-limit-set bit
M433->X:$00F1,13,1            ; #4 Desired-velocity-zero bit
M435->X:$00F1,15,1            ; #4 Dwell-in-progress bit
M437->X:$00F1,17,1            ; #4 Running-program bit
M438->X:$00F1,18,1            ; #4 Open-loop-mode bit
M439->Y:$0A54,14,1            ; #4 Amplifier-enabled status bit
M440->Y:$0A54,0,1             ; #4 In-position bit
M441->Y:$0A54,1,1             ; #4 Warning-following error bit
M442->Y:$0A54,2,1             ; #4 Fatal-following-error bit
M443->Y:$0A54,3,1             ; #4 Amplifier-fault-error bit
M445->Y:$0A54,10,1            ; #4 Home-complete bit
; Motor #4 Move Registers
M461->D:$00DC                 ; #4 Commanded position (1/[Ix08*32] cts)
M462->D:$00DF                 ; #4 Actual position (1/[Ix08*32] cts)
M463->D:$0A4B                 ; #4 Target (end) position (1/[Ix08*32] cts)
M464->D:$0A53                 ; #4 Position bias (1/[Ix08*32] cts)
M465->L:$0819                 ; &1 A-axis target position (engineering units)
M466->X:$00E7,0,24,S          ; #4 Actual velocity (1/[Ix09*32] cts/cyc)
M467->D:$00E1                 ; #4 Present master pos (1/[Ix07*32] cts)
M468->X:$00F7,8,16,S          ; #4 Filter Output (DAC bits)
M469->D:$00FE                 ; #4 Compensation correction (1/[Ix08*32] cts)
M470->D:$00F4                 ; #4 Present phase position (including fraction)
M471->X:$00F4,24,S            ; #4 Present phase position (counts*Ix70)
M472->L:$0A6B                 ; #4 Variable jog position/distance (cts)
M473->Y:$0A55,24,S            ; #4 Encoder home capture position (cts)
M474->Y:$0A6A,24,S            ; #4 Averaged actual velocity
                              ; (1/[Ix09*32] cts/cyc)
; Coordinate System &4 Status Bits
M480->X:$0A58,0,1             ; &4 Program-running bit
M481->Y:$0A57,21,1            ; &4 Circle-radius-error bit
M482->Y:$0A57,22,1            ; &4 Run-time-error bit
M484->X:$0A58,0,4             ; &4 Continuous motion request
M487->Y:$0A57,17,1            ; &4 In-position bit (AND of motors)
M488->Y:$0A57,18,1            ; &4 Warning-following-error bit (OR)
M489->Y:$0A57,19,1            ; &4 Fatal-following-error bit (OR)
M490->Y:$0A57,20,1            ; &4 Amp-fault-error bit (OR of motors)
; Motor #4 Axis Definition Registers
M491->L:$0A62                 ; #4 X/U/A/B/C-Axis scale factor (cts/unit)
M492->L:$0A63                 ; #4 Y/V-Axis scale factor (cts/unit)
M493->L:$0A64                 ; #4 Z/W-Axis scale factor (cts/unit)
M494->L:$0A65                 ; #4 Axis offset (cts)
; Coordinate System &4 Variables
M497->X:$0A46,0,24,S          ; &4 Host commanded time base (I10 units)
M498->X:$0A48,0,24,S          ; &4 Present time base (I10 units)
; Gate Array Registers for Channel 5
M501->X:$C021,0,24,S          ; ENC5 24-bit counter position
M502->Y:$C022,8,16,S          ; OUT5A command value; DAC or PWM
M503->X:$C023,0,24,S          ; ENC5 captured position
M504->Y:$C023,8,16,S          ; OUT5B command value; DAC or PWM
M505->X:$0714,8,16,S          ; ADC5A input image value
M506->Y:$0714,8,16,S          ; ADC5B input image value
M507->Y:$C024,8,16,S          ; OUT5C command value; PFM or PWM
```

```
M508->Y:$C027,0,24,S          ; ENC5 compare A position
M509->X:$C027,0,24,S          ; ENC5 compare B position
M510->X:$C026,0,24,S          ; ENC5 compare autoincrement value
M511->X:$C025,11              ; ENC5 compare initial state write enable
M512->X:$C025,12              ; ENC5 compare initial state
M514->X:$C025,14              ; AENA5 output status
M515->X:$C020,19              ; USER5 flag input status
M516->X:$C020,9               ; ENC5 compare output value
M517->X:$C020,11              ; ENC5 capture flag
M518->X:$C020,8               ; ENC5 count error flag
M519->X:$C020,14              ; CHC5 input status
M520->X:$C020,16              ; HMFL5 flag input status
M521->X:$C020,17              ; PLIM5 flag input status
M522->X:$C020,18              ; MLIM5 flag input status
M523->X:$C020,15              ; FAULT5 flag input status
M524->X:$C020,20              ; Channel 5 W flag input status
M525->X:$C020,21              ; Channel 5 V flag input status
M526->X:$C020,22              ; Channel 5 U flag input status
M527->X:$C020,23              ; Channel 5 T flag input status
M528->X:$C020,20,4            ; Channel 5 TUVW inputs as 4-bit value
; Motor #5 Status Bits
M530->Y:$0B14,11,1            ; #5 Stopped-on-position-limit bit
M531->X:$012D,21,1            ; #5 Positive-end-limit-set bit
M532->X:$012D,22,1            ; #5 Negative-end-limit-set bit
M533->X:$012D,13,1            ; #5 Desired-velocity-zero bit
M535->X:$012D,15,1            ; #5 Dwell-in-progress bit
M537->X:$012D,17,1            ; #5 Running-program bit
M538->X:$012D,18,1            ; #5 Open-loop-mode bit
M539->Y:$0B14,14,1            ; #5 Amplifier-enabled status bit
M540->Y:$0B14,0,1             ; #5 In-position bit
M541->Y:$0B14,1,1             ; #5 Warning-following error bit
M542->Y:$0B14,2,1             ; #5 Fatal-following-error bit
M543->Y:$0B14,3,1             ; #5 Amplifier-fault-error bit
M545->Y:$0B14,10,1            ; #5 Home-complete bit
; Motor #5 Move Registers
M561->D:$0118                 ; #5 Commanded position (1/[Ix08*32] cts)
M562->D:$011B                 ; #5 Actual position (1/[Ix08*32] cts)
M563->D:$0B0B                 ; #5 Target (end) position (1/[Ix08*32] cts)
M564->D:$0B13                 ; #5 Position bias (1/[Ix08*32] cts)
M565->L:$081A                 ; &1 B-axis target position (engineering units)
M566->X:$0123,0,24,S          ; #5 Actual velocity (1/[Ix09*32] cts/cyc)
M567->D:$011D                 ; #5 Present master pos (1/[Ix07*32] cts)
M568->X:$0133,8,16,S          ; #5 Filter Output (DAC bits)
M569->D:$013A                 ; #5 Compensation correction (1/[Ix08*32] cts)
M570->D:$0130                 ; #5 Present phase position (including fraction)
M571->X:$0130,24,S            ; #5 Present phase position (counts*Ix70)
M572->L:$0B2B                 ; #5 Variable jog position/distance (cts)
M573->Y:$0B15,24,S            ; #5 Encoder home capture position (cts)
M574->Y:$0B2A,24,S            ; #5 Averaged actual velocity
                              ; (1/[Ix09*32] cts/cyc)
; Coordinate System &5 Status Bits
M580->X:$0B18,0,1             ; &5 Program-running bit
M581->Y:$0B17,21,1            ; &5 Circle-radius-error bit
M582->Y:$0B17,22,1            ; &5 Run-time-error bit
M584->X:$0B18,0,4             ; &5 Continuous motion request
M587->Y:$0B17,17,1            ; &5 In-position bit (AND of motors)
M588->Y:$0B17,18,1            ; &5 Warning-following-error bit (OR)
```

```
M589->Y:$0B17,19,1            ; &5 Fatal-following-error bit (OR)
M590->Y:$0B17,20,1            ; &5 Amp-fault-error bit (OR of motors)
; Motor #5 Axis Definition Registers
M591->L:$0B22                 ; #5 X/U/A/B/C-Axis scale factor (cts/unit)
M592->L:$0B23                 ; #5 Y/V-Axis scale factor (cts/unit)
M593->L:$0B24                 ; #5 Z/W-Axis scale factor (cts/unit)
M594->L:$0B25                 ; #5 Axis offset (cts)
; Coordinate System &5 Variables
M597->X:$0B06,0,24,S          ; &5 Host commanded time base (I10 units)
M598->X:$0B08,0,24,S          ; &5 Present time base (I10 units)
; Gate Array Registers for Channel 6
M601->X:$C029,0,24,S          ; ENC6 24-bit counter position
M602->Y:$C02A,8,16,S          ; OUT6A command value; DAC or PWM
M603->X:$C02B,0,24,S          ; ENC6 captured position
M604->Y:$C02B,8,16,S          ; OUT6B command value; DAC or PWM
M605->X:$0715,8,16,S          ; ADC6A input image value
M606->Y:$0715,8,16,S          ; ADC6B input image value
M607->Y:$C02C,8,16,S          ; OUT6C command value; PFM or PWM
M608->Y:$C02F,0,24,S          ; ENC6 compare A position
M609->X:$C02F,0,24,S          ; ENC6 compare B position
M610->X:$C02E,0,24,S          ; ENC6 compare autoincrement value
M611->X:$C02D,11              ; ENC6 compare initial state write enable
M612->X:$C02D,12              ; ENC6 compare initial state
M614->X:$C02D,14              ; AENA6 output status
M615->X:$C028,19              ; USER6 flag input status
M616->X:$C028,9               ; ENC6 compare output value
M617->X:$C028,11              ; ENC6 capture flag
M618->X:$C028,8               ; ENC6 count error flag
M619->X:$C028,14              ; CHC6 input status
M620->X:$C028,16              ; HMFL6 flag input status
M621->X:$C028,17              ; PLIM6 flag input status
M622->X:$C028,18              ; MLIM6 flag input status
M623->X:$C028,15              ; FAULT6 flag input status
M624->X:$C028,20              ; Channel 6 W flag input status
M625->X:$C028,21              ; Channel 6 V flag input status
M626->X:$C028,22              ; Channel 6 U flag input status
M627->X:$C028,23              ; Channel 6 T flag input status
M628->X:$C028,20,4            ; Channel 6 TUVW inputs as 4-bit value
; Motor #6 Status Bits
M630->Y:$0BD4,11,1            ; #6 Stopped-on-position-limit bit
M631->X:$0169,21,1            ; #6 Positive-end-limit-set bit
M632->X:$0169,22,1            ; #6 Negative-end-limit-set bit
M633->X:$0169,13,1            ; #6 Desired-velocity-zero bit
M635->X:$0169,15,1            ; #6 Dwell-in-progress bit
M637->X:$0169,17,1            ; #6 Running-program bit
M638->X:$0169,18,1            ; #6 Open-loop-mode bit
M639->Y:$0BD4,14,1            ; #6 Amplifier-enabled status bit
M640->Y:$0BD4,0,1             ; #6 In-position bit
M641->Y:$0BD4,1,1             ; #6 Warning-following error bit
M642->Y:$0BD4,2,1             ; #6 Fatal-following-error bit
M643->Y:$0BD4,3,1             ; #6 Amplifier-fault-error bit
M645->Y:$0BD4,10,1            ; #6 Home-complete bit
; Motor #6 Move Registers
M661->D:$0154                 ; #6 Commanded position (1/[Ix08*32] cts)
M662->D:$0157                 ; #6 Actual position (1/[Ix08*32] cts)
M663->D:$0BCB                 ; #6 Target (end) position (1/[Ix08*32] cts)
M664->D:$0BD3                 ; #6 Position bias (1/[Ix08*32] cts)
```

```
M665->L:$081B                  ; &1 C-axis target position (engineering units)
M666->X:$015F,0,24,S           ; #6 Actual velocity (1/[Ix09*32] cts/cyc)
M667->D:$0159                  ; #6 Present master pos (1/[Ix07*32] cts)
M668->X:$016F,8,16,S           ; #6 Filter Output (DAC bits)
M669->D:$0176                  ; #6 Compensation correction (1/[Ix08*32] cts)
M670->D:$016C                  ; #6 Present phase position (including fraction)
M671->X:$016C,24,S             ; #6 Present phase position (counts*Ix70)
M672->L:$0BEB                  ; #6 Variable jog position/distance (cts)
M673->Y:$0BD5,24,S             ; #6 Encoder home capture position (cts)
M674->Y:$0BEA,24,S             ; #6 Averaged actual velocity
                               ; (1/[Ix09*32] cts/cyc)
; Coordinate System &6 Status Bits
M680->X:$0BD8,0,1              ; &6 Program-running bit
M681->Y:$0BD7,21,1             ; &6 Circle-radius-error bit
M682->Y:$0BD7,22,1             ; &6 Run-time-error bit
M684->X:$0BD8,0,4              ; &6 Continuous motion request
M687->Y:$0BD7,17,1             ; &6 In-position bit (AND of motors)
M688->Y:$0BD7,18,1             ; &6 Warning-following-error bit (OR)
M689->Y:$0BD7,19,1             ; &6 Fatal-following-error bit (OR)
M690->Y:$0BD7,20,1             ; &6 Amp-fault-error bit (OR of motors)
; Motor #6 Axis Definition Registers
M691->L:$0BE2                  ; #6 X/U/A/B/C-Axis scale factor (cts/unit)
M692->L:$0BE3                  ; #6 Y/V-Axis scale factor (cts/unit)
M693->L:$0BE4                  ; #6 Z/W-Axis scale factor (cts/unit)
M694->L:$0BE5                  ; #6 Axis offset (cts)
; Coordinate System &6 Variables
M697->X:$0BC6,0,24,S           ; &6 Host commanded time base (I10 units)
M698->X:$0BC8,0,24,S           ; &6 Present time base (I10 units)
; Gate Array Registers for Channel 7
M701->X:$C031,0,24,S           ; ENC7 24-bit counter position
M702->Y:$C032,8,16,S           ; OUT7A command value; DAC or PWM
M703->X:$C033,0,24,S           ; ENC7 captured position
M704->Y:$C033,8,16,S           ; OUT7B command value; DAC or PWM
M705->X:$0716,8,16,S           ; ADC7A input image value
M706->Y:$0716,8,16,S           ; ADC7B input image value
M707->Y:$C034,8,16,S           ; OUT7C command value; PFM or PWM
M708->Y:$C037,0,24,S           ; ENC7 compare A position
M709->X:$C037,0,24,S           ; ENC7 compare B position
M710->X:$C036,0,24,S           ; ENC7 compare autoincrement value
M711->X:$C035,11               ; ENC7 compare initial state write enable
M712->X:$C035,12               ; ENC7 compare initial state
M714->X:$C035,14               ; AENA7 output status
M715->X:$C030,19               ; CHC7 input status
M716->X:$C030,9                ; ENC7 compare output value
M717->X:$C030,11               ; ENC7 capture flag
M718->X:$C030,8                ; ENC7 count error flag
M719->X:$C030,14               ; CHC7 input status
M720->X:$C030,16               ; HMFL7 flag input status
M721->X:$C030,17               ; PLIM7 flag input status
M722->X:$C030,18               ; MLIM7 flag input status
M723->X:$C030,15               ; FAULT7 flag input status
M724->X:$C030,20               ; Channel 7 W flag input status
M725->X:$C030,21               ; Channel 7 V flag input status
M726->X:$C030,22               ; Channel 7 U flag input status
M727->X:$C030,23               ; Channel 7 T flag input status
M728->X:$C030,20,4             ; Channel 7 TUVW inputs as 4-bit value
; Motor #7 Status Bits
```

```
M730->Y:$0C94,11,1            ; #7 Stopped-on-position-limit bit
M731->X:$01A5,21,1            ; #7 Positive-end-limit-set bit
M732->X:$01A5,22,1            ; #7 Negative-end-limit-set bit
M733->X:$01A5,13,1            ; #7 Desired-velocity-zero bit
M735->X:$01A5,15,1            ; #7 Dwell-in-progress bit
M737->X:$01A5,17,1            ; #7 Running-program bit
M738->X:$01A5,18,1            ; #7 Open-loop-mode bit
M739->Y:$0C94,14,1            ; #7 Amplifier-enabled status bit
M740->Y:$0C94,0,1             ; #7 In-position bit
M741->Y:$0C94,1,1             ; #7 Warning-following error bit
M742->Y:$0C94,2,1             ; #7 Fatal-following-error bit
M743->Y:$0C94,3,1             ; #7 Amplifier-fault-error bit
M745->Y:$0C94,10,1            ; #7 Home-complete bit
; Motor #7 Move Registers
M761->D:$0190                 ; #7 Commanded position (1/[Ix08*32] cts)
M762->D:$0193                 ; #7 Actual position (1/[Ix08*32] cts)
M763->D:$0C8B                 ; #7 Target (end) position (1/[Ix08*32] cts)
M764->D:$0C93                 ; #7 Position bias (1/[Ix08*32] cts)
M765->L:$081C                 ; &1 U-axis target position (engineering units)
M766->X:$019B,0,24,S          ; #7 Actual velocity (1/[Ix09*32] cts/cyc)
M767->D:$0195                 ; #7 Present master pos (1/[Ix07*32] cts)
M768->X:$01AB,8,16,S          ; #7 Filter Output (DAC bits)
M769->D:$01B2                 ; #7 Compensation correction (1/[Ix08*32] cts)
M770->D:$01A8                 ; #7 Present phase position (including fraction)
M771->X:$01A8,24,S            ; #7 Present phase position (counts*Ix70)
M772->L:$0CAB                 ; #7 Variable jog position/distance (cts)
M773->Y:$0C95,24,S            ; #7 Encoder home capture position (cts)
M774->Y:$0CAA,24,S            ; #7 Averaged actual velocity
                              ; (1/[Ix09*32] cts/cyc)
; Coordinate System &7 Status Bits
M780->X:$0C98,0,1             ; &7 Program-running bit
M781->Y:$0C97,21,1            ; &7 Circle-radius-error bit
M782->Y:$0C97,22,1            ; &7 Run-time-error bit
M784->X:$0C98,0,4             ; &7 Continuous motion request
M787->Y:$0C97,17,1            ; &7 In-position bit (AND of motors)
M788->Y:$0C97,18,1            ; &7 Warning-following-error bit (OR)
M789->Y:$0C97,19,1            ; &7 Fatal-following-error bit (OR)
M790->Y:$0C97,20,1            ; &7 Amp-fault-error bit (OR of motors)
; Motor #7 Axis Definition Registers
M791->L:$0CA2                 ; #7 X/U/A/B/C-Axis scale factor (cts/unit)
M792->L:$0CA3                 ; #7 Y/V-Axis scale factor (cts/unit)
M793->L:$0CA4                 ; #7 Z/W-Axis scale factor (cts/unit)
M794->L:$0CA5                 ; #7 Axis offset (cts)
; Coordinate System &7 Variables
M797->X:$0C86,0,24,S          ; &7 Host commanded time base (I10 units)
M798->X:$0C88,0,24,S          ; &7 Present time base (I10 units)
; Gate Array Registers for Channel 8
M801->X:$C039,0,24,S          ; ENC8 24-bit counter position
M802->Y:$C03A,8,16,S          ; OUT8A command value; DAC or PWM
M803->X:$C03B,0,24,S          ; ENC8 captured position
M804->Y:$C03B,8,16,S          ; OUT8B command value; DAC or PWM
M805->X:$0717,8,16,S          ; ADC8A input image value
M806->Y:$0717,8,16,S          ; ADC8B input image value
M807->Y:$C03C,8,16,S          ; OUT8C command value; PFM or PWM
M808->Y:$C03F,0,24,S          ; ENC8 compare A position
M809->X:$C03F,0,24,S          ; ENC8 compare B position
M810->X:$C03E,0,24,S          ; ENC8 compare autoincrement value
```

```
M811->X:$C03D,11              ; ENC8 compare initial state write enable
M812->X:$C03D,12              ; ENC8 compare initial state
M814->X:$C03D,14              ; AENA8 output status
M815->X:$C038,19              ; USER8 flag input status
M816->X:$C038,9               ; ENC8 compare output value
M817->X:$C038,11              ; ENC8 capture flag
M818->X:$C038,8               ; ENC8 count error flag
M819->X:$C038,14              ; CHC8 input status
M820->X:$C038,16              ; HMFL8 flag input status
M821->X:$C038,17              ; PLIM8 flag input status
M822->X:$C038,18              ; MLIM8 flag input status
M823->X:$C038,15              ; FAULT8 flag input status
M824->X:$C038,20              ; Channel 8 W flag input status
M825->X:$C038,21              ; Channel 8 V flag input status
M826->X:$C038,22              ; Channel 8 U flag input status
M827->X:$C038,23              ; Channel 8 T flag input status
M828->X:$C038,20,4            ; Channel 8 TUVW inputs as 4-bit value
; Motor #8 Status Bits
M830->Y:$0D54,11,1            ; #8 Stopped-on-position-limit bit
M831->X:$01E1,21,1            ; #8 Positive-end-limit-set bit
M832->X:$01E1,22,1            ; #8 Negative-end-limit-set bit
M833->X:$01E1,13,1            ; #8 Desired-velocity-zero bit
M835->X:$01E1,15,1            ; #8 Dwell-in-progress bit
M837->X:$01E1,17,1            ; #8 Running-program bit
M838->X:$01E1,18,1            ; #8 Open-loop-mode bit
M839->Y:$0D54,14,1            ; #8 Amplifier-enabled status bit
M840->Y:$0D54,0,1             ; #8 In-position bit
M841->Y:$0D54,1,1             ; #8 Warning-following error bit
M842->Y:$0D54,2,1             ; #8 Fatal-following-error bit
M843->Y:$0D54,3,1             ; #8 Amplifier-fault-error bit
M845->Y:$0D54,10,1            ; #8 Home-complete bit
; Motor #8 Move Registers
M861->D:$01CC                 ; #8 Commanded position (1/[Ix08*32] cts)
M862->D:$01CF                 ; #8 Actual position (1/[Ix08*32] cts)
M863->D:$0D4B                 ; #8 Target (end) position (1/[Ix08*32] cts)
M864->D:$0D53                 ; #8 Position bias (1/[Ix08*32] cts)
M865->L:$081D                 ; &1 V-axis target position (engineering units)
M866->X:$01D7,0,24,S          ; #8 Actual velocity (1/[Ix09*32] cts/cyc)
M867->D:$01D1                 ; #8 Present master pos (1/[Ix07*32] cts)
M868->X:$01E7,8,16,S          ; #8 Filter Output (DAC bits)
M869->D:$01EE                 ; #8 Compensation correction (1/[Ix08*32] cts)
M870->D:$01E4                 ; #8 Present phase position (including fraction)
M871->X:$01E4,24,S            ; #8 Present phase position (counts*Ix70)
M872->L:$0D6B                 ; #8 Variable jog position/distance (cts)
M873->Y:$0D55,24,S            ; #8 Encoder home capture position (cts)
M874->Y:$0D6A,24,S            ; #8 Averaged actual velocity (1/[Ix09*32]
                              cts/cyc)
; Coordinate System &8 Status Bits
M880->X:$0D58,0,1             ; &8 Program-running bit
M881->Y:$0D57,21,1            ; &8 Circle-radius-error bit
M882->Y:$0D57,22,1            ; &8 Run-time-error bit
M884->X:$0D58,0,4             ; &8 Continuous motion request
M887->Y:$0D57,17,1            ; &8 In-position bit (AND of motors)
M888->Y:$0D57,18,1            ; &8 Warning-following-error bit (OR)
M889->Y:$0D57,19,1            ; &8 Fatal-following-error bit (OR)
M890->Y:$0D57,20,1            ; &8 Amp-fault-error bit (OR of motors)
; Motor #8 Axis Definition Registers
```

```
M891->L:$0D62                ; #8 X/U/A/B/C-Axis scale factor (cts/unit)
M892->L:$0D63                ; #8 Y/V-Axis scale factor (cts/unit)
M893->L:$0D64                ; #8 Z/W-Axis scale factor (cts/unit)
M894->L:$0D65                ; #8 Axis offset (cts)
; Coordinate System &8 Variables
M897->X:$0D46,0,24,S         ; &8 Host commanded time base (I10 units)
M898->X:$0D48,0,24,S         ; &8 Present time base (I10 units)
; Accessory 14 I/O M-Variables (1st ACC-14)
M900->Y:$FFD0,0,1            ; MI/O0
M901->Y:$FFD0,1,1            ; MI/O1
M902->Y:$FFD0,2,1            ; MI/O2
M903->Y:$FFD0,3,1            ; MI/O3
M904->Y:$FFD0,4,1            ; MI/O4
M905->Y:$FFD0,5,1            ; MI/O5
M906->Y:$FFD0,6,1            ; MI/O6
M907->Y:$FFD0,7,1            ; MI/O7
M908->Y:$FFD0,8,1            ; MI/O8
M909->Y:$FFD0,9,1            ; MI/O9
M910->Y:$FFD0,10,1           ; MI/O10
M911->Y:$FFD0,11,1           ; MI/O11
M912->Y:$FFD0,12,1           ; MI/O12
M913->Y:$FFD0,13,1           ; MI/O13
M914->Y:$FFD0,14,1           ; MI/O14
M915->Y:$FFD0,15,1           ; MI/O15
M916->Y:$FFD0,16,1           ; MI/O16
M917->Y:$FFD0,17,1           ; MI/O17
M918->Y:$FFD0,18,1           ; MI/O18
M919->Y:$FFD0,19,1           ; MI/O19
M920->Y:$FFD0,20,1           ; MI/O20
M921->Y:$FFD0,21,1           ; MI/O21
M922->Y:$FFD0,22,1           ; MI/O22
M923->Y:$FFD0,23,1           ; MI/O23
M924->Y:$FFD1,0,1            ; MI/O24
M925->Y:$FFD1,1,1            ; MI/O25
M926->Y:$FFD1,2,1            ; MI/O26
M927->Y:$FFD1,3,1            ; MI/O27
M928->Y:$FFD1,4,1            ; MI/O28
M929->Y:$FFD1,5,1            ; MI/O29
M930->Y:$FFD1,6,1            ; MI/O30
M931->Y:$FFD1,7,1            ; MI/O31
M932->Y:$FFD1,8,1            ; MI/O32
M933->Y:$FFD1,9,1            ; MI/O33
M934->Y:$FFD1,10,1           ; MI/O34
M935->Y:$FFD1,11,1           ; MI/O35
M936->Y:$FFD1,12,1           ; MI/O36
M937->Y:$FFD1,13,1           ; MI/O37
M938->Y:$FFD1,14,1           ; MI/O38
M939->Y:$FFD1,15,1           ; MI/O39
M940->Y:$FFD1,16,1           ; MI/O40
M941->Y:$FFD1,17,1           ; MI/O41
M942->Y:$FFD1,18,1           ; MI/O42
M943->Y:$FFD1,19,1           ; MI/O43
M944->Y:$FFD1,20,1           ; MI/O44
M945->Y:$FFD1,21,1           ; MI/O45
M946->Y:$FFD1,22,1           ; MI/O46
M947->Y:$FFD1,23,1           ; MI/O47
; JANA Analog Input Port M-Variables
```

```
M990->X:$0708,0,24,U          ; 1st CONFIG_W1 and CONFIG_W2
M991->X:$0709,0,24,U          ; 2nd CONFIG_W1 and CONFIG_W2
M992->X:$070A,0,24,U          ; 3rd CONFIG_W1 and CONFIG_W2
M993->X:$070B,0,24,U          ; 4th CONFIG_W1 and CONFIG_W2
M994->X:$070C,0,24,U          ; 5th CONFIG_W1 and CONFIG_W2
M995->X:$070D,0,24,U          ; 6th CONFIG_W1 and CONFIG_W2
M996->X:$070E,0,24,U          ; 7th CONFIG_W1 and CONFIG_W2
M997->X:$070F,0,24,U          ; 8th CONFIG_W1 and CONFIG_W2
M1000->Y:$0708,0,12,{f}       ; ANAI00 image register; from J1 pin 1
M1001->Y:$0709,0,12,{f}       ; ANAI01 image register; from J1 pin 2
M1002->Y:$070A,0,12,{f}       ; ANAI02 image register; from J1 pin 3
M1003->Y:$070B,0,12,{f}       ; ANAI03 image register; from J1 pin 4
M1004->Y:$070C,0,12,{f}       ; ANAI04 image register; from J1 pin 5
M1005->Y:$070D,0,12,{f}       ; ANAI05 image register; from J1 pin 6
M1006->Y:$070E,0,12,{f}       ; ANAI06 image register; from J1 pin 7
M1007->Y:$070F,0,12,{f}       ; ANAI07 image register; from J1 pin 8
M1008->Y:$0708,12,12,{f}      ; ANAI08 image register; from J1 pin 9
M1009->Y:$0709,12,12,{f}      ; ANAI09 image register; from J1 pin 10
M1010->Y:$070A,12,12,{f}      ; ANAI10 image register; from J1 pin 11
M1011->Y:$070B,12,12,{f}      ; ANAI11 image register; from J1 pin 12
M1012->Y:$070C,12,12,{f}      ; ANAI12 image register; from J1 pin 13
M1013->Y:$070D,12,12,{f}      ; ANAI13 image register; from J1 pin 14
M1014->Y:$070E,12,12,{f}      ; ANAI14 image register; from J1 pin 15
M1015->Y:$070F,12,12,{f}      ; ANAI15 image register; from J1 pin 16
```

where {f} should be a U if the channel is read as an unsigned quantity, or an S if the channel is read as a signed quantity.

Example 2: A Simple Move

This example shows how to program a simple move on the PMAC. First the program specifies how to do the move, then commands the move.

```
;******************** Set-up and Definitions ********************
&1                            ; Coordinate System 1
CLOSE                         ; Make sure all buffers are closed
#1->X                         ; Assign motor 1 to the X-axis - 1 program unit
                              ; of X is 1 encoder count of motor #1

;******************** Motion Program Text ************************
OPEN PROG 1                   ; Open buffer for program entry, Program #1
CLEAR                         ; Erase existing contents of buffer
LINEAR                        ; Blended linear interpolation move mode
ABS                           ; Absolute mode - moves specified by position
TA500                         ; Set 1/2 sec (500 msec) acceleration time
TS0                           ; Set no S-curve acceleration time
F5000                         ; Set feedrate (speed) of 5000 units (cts)/sec
X10000                        ; Move X-axis to position 10000
DWELL500                      ; Stay in position for 1/2 sec (500 msec)
X0                            ; Move X-axis to position 0
CLOSE                         ; Close buffer - end of program
To run this program:
&1 B1 R                       ; Coordinate System 1, point to Beginning of
                              ; Program 1, Run
```

Example 3: A More Complex Move

This example introduces incremental and time-specification of moves, looping logic, using variables, scaling of axes, and simple arithmetic. Note that logical and mathematical operations do not delay moves.

```
;******************** Set-up and Definitions ********************
&2                              ; Coordinate system 2
                                ; NOTE: No motor can be simultaneously defined
                                ; in more than one coordinate system
CLOSE                           ; Make sure all buffers are closed
#5->1000X                       ; 1 unit (cm) of X is 1000 counts of motor 5
;******************** Motion Program Text ************************
 OPEN PROG 2                     ; Open buffer for entry, Program #2
 CLEAR                           ; Erase existing contents of buffer
 LINEAR                          ; Blended linear interpolation move mode
 INC                             ; Incremental mode - moves specified by distance
 TA500                           ; 1/2 sec (500 msec) acceleration time
 TS250                           ; 1/4 sec in each half of S-curve
 P1=0                            ; Initialize a loop counter variable
 WHILE (P1<10)                   ; Loop until condition is false (10 times) X10
                                 ; Move X-axis 10 cm (=10,000 cts) positive
                                 ; DWELL500
 Hold position for 1/2 sec X-10
                                 ; Move X-axis back 10 cm negative DWELL500
                                 ; Hold position for 1/2 sec
 P1=P1+1
                                 ; Increment loop counter
 ENDWHILE                        ; End of loop
 CLOSE                           ; Close buffer - end of program
 To run this program:
 &2 B2 R                         ; Coordinate System 2, point to Beginning of
                                 ; Program 2, Run
```

Example 4: Conditional Branching

This program introduces conditional branching, calculated move distances, home finding, and I/O addressing.

```
;******************** Set-up and Definitions *********************
CLOSE                           ; Make sure all buffers are closed
&1                              ; Coordinate System 1
#2->27.77777778A                ; A-axis is programmed in degrees
                                ; 10,000 (cts/rev) / 360 (deg/rev)
M1->Y:$FFC2,8,1                 ; Variable M1 assigned to Machine Output 1
M11->Y:$FFC2,0,1                ; Variable M11 assigned to Machine Input 1
I190=60000                      ; Feedrate (speed) units are per minute
                                ; (1 min=60000 msec)
```

```
;******************** Motion Program Text ***********************
OPEN PROG 3 CLEAR              ; Prepare buffer for entry
HOME2                          ; Find home position for motor
LINEAR                         ; Blended linear interpolation move mode
F20                            ; Speed of 20 degrees per minute
Q50=0                          ; Initialize a loop counter variable
WHILE (Q50<36)                 ; Loop until condition is false (36 times)
                               ; IF (M11=1)
                               ; Machine Input 1 on? A((Q50+1)*10)
                               ; Positive move to calculated position
ELSE                           ; Do the following branch for false IF
   A(-(Q50+1)*10)              ; Negative move to calculated position
   ENDIF
   DWELL20                     ; Hold position for 20 msec
   M1=1 M1=0                   ; Pulse output on and off quickly
   DWELL20                     ; Hold position for 20 msec
   A0                          ; Return to home position
   Q50=Q50+1                   ; Increment loop counter
ENDWHILE                       ; End of loop
CLOSE                          ; Close buffer - end of program
                               ;To run this program:
&1 B3 R                        ; Coordinate System 1, point to Beginning of
                               ; Program 3, Run
```

Example 5: Linear and Circular interpolation

This example introduces linear and circular interpolation in a Cartesian X-Y application

```
;********************* Set-up and Definitions *********************
CLOSE                          ; Make sure all buffers are closed
&1                             ; Coordinate System 1
#3->10000X                     ; Use motor 3 for X-axis
#4->10000Y                     ; Use motor 4 for Y-axis
;*************** Motion Program Text for Rotary Axis ***************
OPEN PROG 4 CLEAR              ; Prepare buffer for entry
RAPID X1 Y4                    ; Fast move to starting point
F500                           ; Speed for linear and circular moves
LINEAR Y13                     ; Straight move
CIRCLE1 X2 Y14 I1 J0           ; CW arc
LINEAR X3                      ; Straight move
CIRCLE1 X4 Y13 I0 J-1          ; CW arc
LINEAR Y7                      ; Straight move
CIRCLE1 X7 Y4 I3 J0            ; CCW arc
LINEAR X13                     ; Straight move
CIRCLE1 X14 Y3 I0 J-1          ; CW arc
LINEAR Y2                      ; Straight move
CIRCLE1 X13 Y1 I-1 J0          ; CW arc
LINEAR X4                      ; Straight move
CIRCLE1 X1 Y4 I0 J3            ; CW arc
DWELL100                       ; Hold 100 ms
RAPID X0 Y0                    ; Return home
CLOSE
                               ;To run this program:
&1 B4 R                        ; Coordinate System 1, point to Beginning
                               ; of Program 4, Run
```

Example 6: A Simple G-Code Program



The part program does not need to know how the G-codes and M-codes are executed.

This is a simple example of a PMAC G-Code program. `Gxx` is interpreted as a call to label `Nxx000` of `PROG 1000` (below). Mxx is interpreted as a call to label `Nxx000` of `PROG 1001` (bottom). Far more extensive codes are possible; standard forms of the common codes are provided on diskette.

```
;******************** Part Program Text *************************
OPEN PROG 5 CLEAR             ; Prepare motion program 5 for entry
G17 G90                       ; XY plane, absolute move spec
G97 S1800                     ; Set spindle speed of 1800 rpm
F500                          ; Cutting speed 500 mm/min
G00 X10.00 Y5.00              ; Rapid move to (10, 5)
M03                           ; Start spindle
G04 P2.0                      ; Wait 2 seconds
G01 Z0                        ; Lower cutter
X30.25 Y5.00                  ; Linear XY move
G03 X35.25 Y10.00 J5          ; CCW arc move
G01 X35.25 Y50.10             ; Linear move
G03 X30.25 Y55.10 I-5         ; CCW arc move
G01 X10.00 Y55.10             ; Linear move
G03 X5.00 Y50.10 J-5          ; CCW arc move
G01 X5.00 Y10.00              ; Linear move
G03 X10.00 Y5.00 I5           ; CCW arc move
G01 Z5 M05                    ; Cutter up, stop
G00 X0 Y0                     ; Back to home
CLOSE
;*****************************************************************
Motion program 1000 contains the G-Code subroutines
OPEN PROG 1000 CLEAR          ; Prepare buffer 1000 for entry
RAPID RETURN                  ; G00 Rapid mode (N0 is implied)
N01000 LINEAR RETURN          ; G01 Linear interpolation mode
N02000 CIRCLE1 RETURN         ; G02 Clockwise circle mode
N03000 CIRCLE2 RETURN         ; G03 Counterclockwise circle mode
N04000 READ(P)                ; G04 Dwell for P seconds
IF (Q100 & 32768 > 0)         ; P parameter specified?
   DWELL (Q116*1000)          ; PMAC specifies dwell time in msec
ENDIF
RETURN
N17000 NORMAL K-1 RET         ; G17 Specify XY plane
N18000 NORMAL J-1 RET         ; G18 Specify ZX plane
N19000 NORMAL I-1 RET         ; G19 Specify YZ plane
N90000 ABS RET                ; G90 Absolute mode
N91000 INC RET                ; G91 Incremental mode
N97000 READ(S)                ; G97 Spindle speed set
IF (Q100 & 262144 > 0)        ; S parameter specified?
   I422=Q119/30               ; #4 jog speed in cts/msec
ENDIF
RETURN
CLOSE
```

```
;*****************************************************************
Motion program 1001 contains the M-Code subroutines
OPEN PROG 1001 CLEAR          ; Prepare buffer 1001 for entry
N03000 CMD "#4J+" RET         ; Start spindle clockwise (closed loop)
N04000 CMD "#4J-" RET         ; Start spindle counterclockwise (ditto)
N05000 CMD "#4J/" RET         ; Stop spindle
CLOSE
                              ; To run this program:
&1 B5 R                       ; Coordinate System 1, point to Beginning of
                              ;  Program 5, Run
```

Example 7: A Robotic Application

This example shows how a robotic style application can be implemented very quickly and easily on the PMAC. This example uses the Option 2 dual-ported RAM to pass values in the fastest possible manner. (The normal communications port could be used at a slight cost in performance.) Here, the host PC is performing the inverse kinematic calculations every 5 msec, converting tip positions to joint positions and passing the joint positions to the DPRAM, where they are accessed as M-variables. PMAC interpolates between these intermediate points on each axis with a cubic spline that is updated every servo cycle.

```
;******************** Set-up and Definitions ********************
CLOSE                         ; Make sure all buffers are closed
&1                            ; Define axes (joints) in degrees in
                              ; Coordinate System 1
#1->1024X                     ; 1024 cts/deg
#2->2000Y                     ; 2000 cts/deg
#3->2000Z                     ; 2000 cts/deg
#4->5000A                     ; 5000 cts/deg
#5->512B                      ; 512 cts/deg
#6->277.77778C                ; 277.77778 cts/deg
M70->F:$D200                  ; 32-bit  IEEE floating-point format
M71->F:$D201                  ; variables in DPRAM for passing joint
M72->F:$D202                  ; positions to PMAC.  The host writes
M73->F:$D203                  ; to these locations through the use
M74->F:$D204                  ; of pointer variables.
M75->F:$D205                  ;
M113->X:$C000,13,1            ; Interrupt bit to signal host for next set of points
;******************** Motion Program Text ************************
OPEN PROG 75 CLEAR
SPLINE1                       ; Cubic spline mode
TA5                           ; Set spline segment time of 5 msec
INC                           ; Incremental mode
X0 Y0 Z0 A0 B0 C0             ; First zero-distance move
X0 Y0 Z0 A0 B0 C0             ; Second zero-distance move
ABS                           ; Further moves will be absolute
WHILE (P1>0)                  ; Host sets P1 to exit this mode
   X(M70) Y(M71) Z(M72) A(M73) B(M74) C(M75) M113=1 M113=0
                              ; Calculate next segment, signal host
                              ; thru interrupt for more data
ENDWHILE
CLOSE
                              To run this program:
&1 B75 R                      ;Coordinate System 1, point to Beginning of
                              ;  Program 75, Run
```

Example 8: Rotary Tangent to X-Y System

This example shows how easy it is to keep a rotary axis tangent (or normal) to the movement of an X-Y system. The rotary axis is controlled by a separate program running simultaneously that monitors X-Y variables. The XY part program *DOES NOT* need to know anything about the rotary axis - it is all automatic.

```
;*********************  Set-up and Definitions ********************
CLOSE
&1 #1->10000X                  ; Motor 1 is X axis in Coordinate system 1
#2->10000Y                     ; Motor 2 is Y axis in Coordinate system 1
&2 #3->40C                     ; Motor 3 is C axis in Coordinate system 2
I15=0                          ; Do trig calculations in degrees
I327=14400                     ; C axis rolls over every 14400 cts
M161->D:$0028                  ; Motor 1 present commanded position
M163->D:$080B                  ; Motor 1 target position
M261->D:$0064                  ; Motor 2 present commanded position
M263->D:$08CB                  ; Motor 2 target position
P10=3072                       ; 1 count in Mx61, Mx63 units (div by 0 check)
;************** Motion Program Text for Rotary Axis ***************
OPEN PROG 10 CLEAR
SPLINE1 TA20                   ; Cubic spline segments of 20 msec
WHILE (P1>0)                   ; As long as in this mode
   Q0=M163-M161                ; X target position - X current position
   Q1=M263-M261                ; Y target position - Y current position
   IF (ABS(Q0)>P10 OR ABS(Q1)>P10) ; Target position < > current position?
   Q2=ATAN2(Q1)                ; Calculate angle - ATAN2 (Q1, Q2)
   ENDIF
   C(Q2)                       ; Move rotary axis to new angle
ENDWHILE
CLOSE
```

Example 9: PLC Input Programming

This example shows how to use a PLC program to create customized inputs. As the PLC program repeatedly cycles in background, it checks the input lines and issues commands as appropriate. This program shows how to use a thumbwheel input line to create a jog switch dedicated to a particular motor. Note the use of a "latching flag" so that commands are only given on a change of state, making them edge triggered.

```
;********************* Set-up and Definitions *********************
CLOSE
M50->Y:$FFC1,0                 ; Thumbwheel port input bit 0
M60->*                         ; Latching bit for M50
;********************* PLC Program Text ***************************
OPEN PLC 16
CLEAR
IF (M50=1)                     ; Motor 1 jog plus switch on
   IF (M60=0)                  ; But not on last time
   COMMAND"#1J+"               ; Issue command
   M60=1                       ; Set latching flag
   ENDIF
ELSE                           ; Motor 1 jog plus switch off
   IF (M60=1)                  ; But not off last time
   COMMAND"#1J/"               ; Issue stop command
   M60=0                       ; Set latching flag
   ENDIF
ENDIF
CLOSE
```

Example 10: PLC Display Programming

This example shows how a PLC can be used to continually update a display with values derived from the current status. Here, the program takes the actual motor positions on an R-Theta table, converts them to X-Y positions, puts them in the dual-ported RAM for the host, and on the LCD display.

```
;********************* Set up and Definitions *********************
CLOSE
#1->-100U+15000               ; Radial arm at 100 cts/mm; offset 150 mm
#2->40C                       ; Rotary axis at 40 cts/deg
M162->D:$002B                 ; Motor 1 actual position
M262->D:$0067                 ; Motor 2 actual position
M100->F:$D400                 ; DPRAM register for X position reporting
M200->F:$D401                 ; DPRAM register for Y position reporting
;********************* PLC Program Text ***************************
OPEN PLC 6 CLEAR
P162=150-M162/(I108*32*100)   ; Radial distance from center in mm
P262=M262/(I208*32*40)        ; Rotary position in degrees
M100=P162*COS(P262)           ; X position in mm => DPRAM register
M200=P162*SIN(P262)           ; Y position in mm => DPRAM register
DISPLAY 5,10.2,M100           ; Formatted display to LCD
DISPLAY 15,10.2,M200          ; Formatted display to LCD
CLOSE
```

Example 11: Precision Slaving

This example shows how the PMAC can use an external time-base source for sophisticated precision slaving applications. Here it is used for multiple-pass screw threading on a lathe with an open-loop spindle. Coil winding applications are virtually identical as are cut-on-the-fly applications for conveyorized or extruded material. The PMAC slaves its cutter axes (X&Z) to an encoder on the spindle so that cutter speed tracks spindle speed, resulting in constant pitch. This is done simply in time-base mode. Define a "real-time" speed for the master (spindle) and set the time-base constant based on this. Write the program for the slave axes as if the master were always operating at the real-time speed. The time base control will compensate automatically for changes in the spindle speed.

```
;********************* Set-up and Definitions *********************
 &1                            ; Define axes in Coordinate System 1
 #1->10000X                    ; Motor 1 is X at 10000 cts/in (radial)
 #2->10000Z                    ; Motor 2 is Z at 10000 cts/in (traverse)

Spindle encoder has 1024 lines/rev or 4096 cts/rev.  At real-time speed of
3000 rpm (50 rps), encoder frequency is 204.8 cts/msec.  By formula, the time
base constant is 131,072/204.8 = 640.


WY:1833,640                   ; Set time base constant (at address 1833)
I193=1833                     ; Tell Coordinate System 1 to use this
                              ; address as the time-base source.

;********************* Motion Program Text ************************

This program is written to cut a 5-pitch (5 threads per inch) screw.  It as-
sumes the spindle speed is 3000 rpm, or 50 rps.  Cutting speed is therefore
50(rev/sec)/5(rev/in)=10 in/sec.  Programmed time for each loop must be an
exact multiple of 20 msec for cutter to match thread each pass (50rev/sec <=>
20msec/rev).
```

```
OPEN PROG 77 CLEAR            ; Prepare buffer 77 for entry
P100=3.00                     ; X (radial) dimension of outside of stock
P101=P100                     ; Starting location of cut
RAPID X(P100-0.1) Z2          ; Move quickly to starting position
WHILE (P101<3.10)             ; Loop until 0.1 inches deep
   P101=P101+0.01             ; Increase depth 0.01 inch
   TM100                      ; 100 msec "plunge" time
   X(P101)                    ; Radial axis into stock ("plunge")
   TM(24*1000/10)             ; 24 inches at 10 in/sec (in msec)
   Z26                        ; Make threading pass for 24 inches (26-2)
   TM60                       ; 60 msec retract time
   X(P100-0.1)                ; Retract to just outside of stock
   TM(24*1000/30)             ; 24 inches at 30 in/sec (in msec)
   Z2                         ; Reverse move for next pass
                              ; Total time for moves in loop is
                              ; 100+2400+60+800=3360 msec, which is an
                              ; exact multiple of 20 msec, or 1 spindle rev.
ENDWHILE                      ; End of loop
RAPID X0 Z0                   ; Return to home position
CLOSE
```

Example 12: Matching Positions on the Fly

This example shows how the position-capture function of PMAC can be used to match positions on the fly with a moving system not under PMAC control. This particular example has a short belt that takes items arriving at un-even intervals and evens out the spacing to place them between the lugs of a conveyor belt driven by a variable-speed drive. The same technique is applicable to any application requiring the matching of position of two moving devices when the spacing is not exactly known ahead of time.

The application makes use of PMAC's hardware position-capture registers, which register the position on an external trigger to the count, regardless of speed. This permits very high speed registration and correction. The belt in this application would probably also be slaved to the lugged conveyor in time-base mode (see Example 10) for speed matching -- this would keep any position corrections relatively small.

```
;***************** Set-up and Definitions ******************************
CLOSE
&1 #1->X                      ; Motor 1 is X-axis scaled in counts
M11->Y:$FFC2,0,1              ; M11 is Machine Input 1
M203->X:$C007,0,24,S          ; Encoder 2 captured-position register
M217->X:$C004,17,1            ; Encoder 2 position-capture flag
I907=2                        ; Capture ENC2 on rising edge of flag
I908=0                        ; Use HMFL2 to capture ENC2
P1=5000                       ; 5000 counts of ENC2 per lug
P2=1325                       ; "No-correction" position within lug
                              ; cycle, when captured
;***************** Motion Program Text ********************************
OPEN PROG 346 CLEAR
INC                           ; Incremental move mode
SPLINE1 TA10                  ; Cubic spline segments of 10 msec
WHILE (M11=1)                 ; Loop once per item as long as input set
  WHILE (M217=0)              ; Loop until photocell blocked
  X(P10)                      ; P10 is nominal distance in 10 msec
```

```
  ENDWHILE                    ; Loop back to check for trigger again
                              ; Found trigger, so calculate and adjust
  P3=M203 % P1                ; Read captured position of lug conveyor
                              ; and do modulo by counts per lug to find
                              ; out where in lug cycle we are.
  P4=P3-P2                    ; Compare to no-correction position --
                              ; difference is error to make up
  X(P10+P4/4)                 ; Make up first 1/4 of error in 10 msec
  X(P10+P4/2)                 ; Make up 1/2 of error in 10 msec
  X(P10+P4/4)                 ; Make up last 1/4 of error in 10 msec
ENDWHILE                      ; Loop back for next item
```

Example 13: Spindle Operation

This program provides for sophisticated spindle operation, including constant surface speed (CSS) mode. It breaks the spindle move into very small pieces (25 msec in this case), so the spindle can react very quickly to the need for a change in velocity. This example requires the use of Motor #1 for the X-axis as the controlling dimension for CSS mode. In CSS mode, the spindle speed is varied with the radial dimension of the tool tip so that the surface speed of the spindle past the tool tip is constant. Essentially, the rotational velocity of the spindle is inversely proportional to the radial dimension of the tool tip (the X-axis position +/- an offset). Since this could lead to a request for infinite rotational velocity, there is a maximum rotational velocity -- specified by G92.

The program relies on the following variables:

M55 a flag denoting whether the spindle is ON CW (= +1), ON CCW (= -1), or OFF (= 0) set by part program

P97 desired spindle speed in RPM set by part program -- absolute value

P96 desired spindle surface speed in ft/min or m/min, set by part program (in this case P97 is calculated from P96) -- absolute value

P95 maximum spindle acceleration magnitude in RPM/sec set as system constant

P92 maximum permitted spindle velocity in RPM set in part program (G92S__)

P93 number of segments per second for spindle program set as system constant

M96 a flag denoting CSS mode (=1) or not (=0) set by part program

M70 a flag denoting metric (=1) or English (=0) set by part program

M95 a flag denoting positive accel (= +1), negative accel (= -1), or no accel (= 0) set in spindle program

P197 commanded spindle speed for current segment in RPM not necessarily same as desired spindle speed P97, but always approaching magnitude of P97 signed value

M162 the register holding Motor #1 (X-axis) actual position units are 1/(I108*32) encoder counts

M164 the register holding Motor #1 position offset bias

P162 variable holding Motor #1 position converted to counts

M191 the register holding the scaling between X-axis position in user units and Motor #1 position in counts set in axis definition statement

M194 the register holding the offset between X-axis zero position and Motor #1 zero position, in counts set in axis definition statement

P98 variable holding the difference between X-axis zero and spindle center (if any) set in part program (G92R__)

P163 variable holding the X-axis radial distance from the spindle center first motor #1 position is converted to axis position [(P162-M194)/M191], then the spindle center offset (P98) is subtracted

```
;********************* Set-up and definitions ***************************
P1=3.14159272                          ; PI -- for angular to surface speed
P95=1000                               ; Maximum spindle accel in RPM/sec
P93=40                                 ; 40 segments per second
M55->*                                 ; Defined as self-referenced flag
M95->*                                 ; Ditto
M96->*                                 ; Ditto
M70->*                                 ; Ditto
M95->*                                 ; Ditto
M162->D:$002B                          ; Motor #1 actual position register
                                       ; (was D:$002C in V1.09 and older)
M164->D:$0813                          ; Motor #1 position bias register
M191->L:$0822                          ; Motor #1 X-axis coefficient
M194->L:$0825                          ; Motor #1 offset coefficient
;********************* Actual spindle program ***************************
CLOSE
OPEN PROG 1010
CLEAR
LINEAR                                 ; Linear blended moves
INC(C)                                 ; Incremental move specification
TM(1000/P93)                           ; Move segment size in msec
TA(1000/P93)                           ; Accel thru entire segment
TS0                                    ; Straight ramp accel
P197=0                                 ; Starting RPM is zero
WHILE (M55!=0 OR P197!=0)
  IF (M96=1)                           ; Constant surface speed mode?
    P162=ABS((M162+M164)/(I108*32      )); Motor #1 actual pos in cts
    P163=(P162-M194)/M191-P98          ; X-axis radial distance
    IF (ABS(P163)>0.1)                 ; Not  ~zero?
      P97= P96/(2*P1*P163)             ; Convert to rotational speed
      IF (M70=0) P97=12*P97            ; English conversion (in->ft)
    ELSE P97=1000*P97                  ; Metric conversion (mm->m)
      IF (P97>P92)                     ; Past limit?
        P97=P92                        ; Set to limit
      ENDIF
  ELSE
    P97=P92                            ; Set to RPM limit
  ENDIF
  ENDIF
  IF (M55*P97<P197) M95=-1             ; Negative accel required
    IF (M55*P97>P197) M95=1            ; Positive accel required
      IF (M95!=0)                      ; Change of speed required?
        P197=P197+M95*P95/P93          ; Slew to new RPM for this segment
        IF ((P197-M55*P97)*M95>0)      ; Past desired RPM?
          P197=M55*P97                 ; Set to desired RPM
          M95=0                        ; Mark as no longer accelerating
        ENDIF
      ENDIF
  C(P197/(60*P93))                     ; Actual incremental move segment
ENDWHILE
DWELL50
M95=0                                  ; Clear accel flag to end &1 wait
CLOSE
```

Example 14: Torque-limited Velocity Move

This program performs a torque-limited velocity move. Axis moves at constant velocity as long as torque is below a preset limit (here measured by the DAC register, but could be an actual torque/force sensor). If the limit is exceeded, the commanded velocity is reduced.

```
CLOSE DELETE GATHER DELETE TRACE
M102->Y:$C003,8,16,S          ; DAC1 register
P102=3200                     ; Torque limit
M1=1
OPEN PROG 47
CLEAR
TA10
TS0
TM10
LINEAR
INC
P1=100                        ; Nominal distance (cts) in 10 msec
WHILE (M1=1)                  ; As long as in this mode
  IF (M102<P102)              ; Within torque limit (current amp)
  P2=P1                       ; Move nominal distance
  ELSE                        ; Exceeded torque limit
  IF (P2>0)
  P2=P2-5                     ; Reduce distance
  ELSE
  P2=0                        ; Don't let go negative
  ENDIF
  ENDIF
  X(P2)                       ; Make next segment of move
ENDWHILE
CLOSE
```

Example 15: A Torque Limited Screwing Operation

This example shows how to do a torque limited screwing operation. The key is to steadily increment the screwing motion in small pieces until some measure of the torque exceeds a preset limit. Here the DAC output (into a torque-mode amplifier) is used, but an ADC input from motor current or an actual torque sensor could also be used.

```
CLOSE
&1
#3->2000Z                     ; Vertical axis (in mm)
#4->2000C                     ; Rotary axis (in turns)
M102->Y:$C00A,8,16,S          ; DAC4 output register
                              ; (indirect measure of torque)
P102=4000                     ; Torque limit (in DAC bits)
OPEN PROG 508 CLEAR
ABS(Z,C)                      ; Both axes absolute
PSET C0                       ; Call current rotary position zero
P1=-5                         ; End Z position of fast move
TA50                          ; Fast move accel time
F10                           ; Fast move speed
Z(P1)                         ; Command fast move
DWELL5                        ; Momentary pause (optional)
TA10 TM10                     ; Move segment time of 10 msec in loop
```

```
P2=0                              ; C-axis position index
WHILE (P2<3.0 AND M102<P102)      ; Loop until 3 turns or high torque
  P1=P1-0.05                      ; Increment Z-axis position
  P2=P2+0.1                       ; Increment C-axis position
  Z(P1) C(P2)                     ; Next piece of screwing move
ENDWHILE                          ; Loop back
DWELL 10                          ; Momentary pause (optional)
TA50                              ; Retract move accel
F10                               ; Retract move speed
Z0                                ; Command retract move
CLOSE
```

Example 16: Altering Destination in Moves

This program allows altered destination in moves. The card is always moving toward an X position of P1 and a Y position of P2, but it does it in small increments so when P1 or P2 is changed, it will change direction within a few milliseconds. P1 and P2 can be changed from the host on the fly.

```
CLOSE
OPEN PROG 2
CLEAR
P9=100                            ; Vector distance per increment
P11=0                             ; Starting X position
P12=0                             ; Starting Y position
I187=10                           ; C.S. 1 accel time
I188=0                            ; C.S. 1 S-curve time
TM10                              ; Increment time in msec
WHILE (M1=1)                      ; Whatever condition you like
  P21=P1-P11                      ; X distance to go
  P22=P2-P12                      ; Y distance to go
P0=SQRT(P21*P21+P22*P22)          ; Vector distance to go
IF(P0>P9)                         ; If longer than increment
  P0=P9/P0                        ; Fraction of distance
  P11=P11+P21*P0                  ; X increment
  P12=P12+P22*P0                  ; Y increment
  ELSE                            ; Distance < increment
  P11=P1                          ; Just go to destination
  P12=P2
  ENDIF
  X(P11) Y(P12)                   ; Make move
ENDWHILE
CLOSE
```

Example 17: Changing Speed

This program is an example of moving at high speed until some sort of external command, then reacting very quickly to the command, but changing speed very slowly. It works by splitting the move into very small segments, and using WHILE loops to continue in one mode until the signal is received.

```
CLOSE
OPEN PROG 10
CLEAR
P4=150000
P5=3000
P6=600
```

```
INC(X)                          ; X-axis in incremental mode
TM(P5/P6)                       ; move segment time (5 msec here!)
TA(P5/P6)                       ; accel time (<= TM)
P3=P4/(P6*P6)                   ; multiplier to get distance
P1=0                            ; start at zero speed
P2=0                            ; start at zero distance
N10 WHILE (P0 != 0)             ; P0=0 is signal to decelerate
  IF (P1<P6)                    ; not up to full speed?
  P1=P1+1                       ; increment to increase speed
  P2=P1*P3                      ; distance to travel in segment
  ENDIF
  X(P2)                         ; incremental move segment
ENDWHILE
WHILE (P0=0)                    ; P0=1 is signal to accelerate
  IF (P1>0)                     ; still moving?
  P1=P1-1                       ; decrement to decrease speed
  P2=P1*P3                      ; distance to travel in segment
  ENDIF
  X(P2)                         ; incremental move segment
ENDWHILE
GOTO 10                         ; loops back up when P0=1 again
CLOSE
```

Example 18: Circular Interpolation with Linear Moves

This example contains subroutines that perform circular interpolation with blended moves. The subroutine starting at N10000 does all the calculations for the circular move. If the host program has not specified the center, the routine calls the subroutine at N20000 to compute the center point. For each circularly interpolated move, the main program must specify the starting point, the ending point, the radius, the direction, and the increment size. Also, either the long-arc/short-arc toggle or the center point must be specified. Since these specifications are variables that hold their value, any that do not change between moves do not have to be re-specified. The main program in this example provides a thorough exercise of the subroutines.

```
CLOSE
OPEN PROG 1
CLEAR
                                ; Main section of program is to exercise
                                ; subroutine
TA50                            ; should be shorter than time for
                                ; increment
TS0                             ; zero for best blending
F1000                           ; feedrate specification is easiest
X10000Y0                        ; linear move
                                ; Long arc clockwise
Q1=10000 Q2=0 Q3=5000 Q4=-5000 Q5=5000
Q6=1 Q7=1 Q8=2 GOSUB 10000
                                ; Linear move
X5000Y0
                                ; Short arc clockwise
Q1=5000 Q2=0 Q3=10000 Q4=5000 Q5=5000
Q6=1 Q7=0 Q8=2 GOSUB 10000
                                ; Short arc counterclockwise
Q1=Q3 Q2=Q4 Q3=12500 Q4=7500 Q5=2500
Q6=0 Q7=0 Q8=2 GOSUB 10000
                                ; Long arc counterclockwise
Q1=Q3 Q2=Q4 Q3=10000 Q4=5000 Q5=2500
```

```
Q6=0 Q7=1 Q8=2 GOSUB 10000
                                        ; Half-circle clockwise
Q1=Q3 Q2=Q4 Q3=10000 Q4=0 Q5=2500
Q6=1 Q7=1 Q8=2 GOSUB 10000
                                        ; Half-circle counterclockwise
                                        ; dist > 2R
Q1=Q3 Q2=Q4 Q3=10000 Q4=-5005 Q5=2500
Q6=0 Q7=0 Q8=2 GOSUB 10000
DELAY 500
X0Y-5000
DELAY 500
                                        ;Now an multi-part circle to test off
                                        ; angles
Q1=0 Q2=-5000 Q3=3535.5 Q4=-3535.5 Q5=5000
Q6=0 Q7=0 Q8=2 GOSUB 10000
Q1=Q3 Q2=Q4 Q3=3535.5 Q4=3535.5 GOSUB10000
Q1=Q3 Q2=Q4 Q3=-3535.5 Q4=3535.5 GOSUB10000
Q1=Q3 Q2=Q4 Q3=-3535.5 Q4=-3535.5 GOSUB10000
Q1=Q3 Q2=Q4 Q3=0 Q4=-5000 GOSUB10000
DELAY 500
X0Y0
RETURN
```

PMAC Circular Arc Move Subroutine

This subroutine performs moves on a circular arc automatically. The section of the program calling this subroutine simply needs to set the following variables as controlling parameters for the move:

```
Q1:   X-coordinate of the starting point of arc
Q2:   Y-coordinate of the starting point of arc
Q3:   X-coordinate of the ending point of arc
Q4:   Y-coordinate of the ending point of arc
Q5:   Radius of the arc
Q6:   0 for counterclockwise 1 for clockwise
Q7:   0 for short arc 1 for long arc            ;-1 if have specified center
Q8:   subdivide size in degrees
```

If host has the center location, the following can be specified also (with Q7=-1) otherwise they will be calculated by the routine

```
Q9:    X-coordinate of the center
Q10:   Y-coordinate of the center
```

```
N10000                                  ; Circular arc move subroutine
IF (Q7>-1) GOSUB 20000                  ; Must compute center
Q0=Q1-Q9                                ; Prep for ATAN2
Q26=ATAN2(Q2-Q10)                       ; Angle from center to start
Q0=Q3-Q9                                ; Prep for ATAN2
Q27=ATAN2(Q4-Q10)                       ; Angle from center to end
IF (Q6=0 AND Q27<Q26) Q27=Q27+360       ; Proper direct.
IF (Q6=1 AND Q27>Q26) Q27=Q27-360       ; Proper direct.
IF (Q6=0)                               ; If counterclockwise
   Q28=Q26+Q8                           ; Increasing angle
   WHILE (Q28<Q27)                      ; Not past end angle
   Q11=Q9+Q5*COS(Q28)                   ; X-coord on arc
   Q12=Q10+Q5*SIN(Q28)                  ; Y-coord on arc
```

```
   X(Q11)Y(Q12)                        ; Make partial move
   Q28=Q28+Q8                          ; Increment angle
   ENDWHILE                            ; Loop until end angle
   X(Q3)Y(Q4)                          ; Make final move
ELSE                                   ; Clockwise arc
   Q28=Q26-Q8                          ; Decreasing angle
   WHILE (Q28>Q27)                     ; Not past end angle
   Q11=Q9+Q5*COS(Q28)                  ; X-coord on arc
   Q12=Q10+Q5*SIN(Q28)                 ; Y-coord on arc
   X(Q11)Y(Q12)                        ; Make partial move
   Q28=Q28-Q8                          ; Decrement angle
   ENDWHILE                            ; Loop until end angle
   X(Q3)Y(Q4)                          ; Make final move
ENDIF
RETURN
N20000                                 ; Subroutine to calculate circle center

Q20=SQRT((Q3-Q1)*(Q3-Q1)+(Q4-Q2)*(Q4-Q2))  ; Distance
Q21=Q5*Q5-Q20*Q20/4
IF (Q21<0)                             ; Move distance > 2R: must adjust
   Q21=0                               ; Center on line FR start to end
   Q5=Q20/2                            ; Adjust radius to half distance
ENDIF
Q23=SQRT(Q21)                          ; Distance FR midpoint to center
Q0=Q3-Q1                               ; Prep for ATAN2
Q24=ATAN2(Q4-Q2)                       ; Angle from start to end
Q0=Q20/2                               ; Prep for ATAN2
Q25=ATAN2(Q23)                         ; Angle of center off mid-line
IF (Q6=1 AND Q7=1 OR Q6=0 AND Q7=0)
   Q26=Q24+Q25                         ; Add to get ang FR start to cent

ELSE Q26=Q24-Q25                       ; Sub to get ang FR start to cent
Q9=Q1+Q5*COS(Q26)                      ; Center X-coord
Q10=Q2+Q5*SIN(Q26)                     ; Center Y-coord
RETURN
CLOSE
```

Example 19: Sending Information to a Display



After loading these programs you must set `I5=2` *or* `3` *and* `ENABLE PLC 1` *and* `17` *or* `18`. *Do not run* `PLC 17` *and* `18` *at the same time.*

These PLC programs monitor the motor position registers, scales them into counts, and sends the resulting values to the LCD display. This example illustrates several important features:

1. The use of D: M-variables for double-word values
2. The use of the servo cycle counter as a timer
3. The use of PMAC's built in timers
4. The use of an open 24-bit M-variable containing difference information to handle rollover properly
5. The display of messages on the LCD
6. The display of variable values on the LCD

7. The disabling of a condition that caused a display command so that the next scan through the PLC, the display command will not automatically be done again

8. PLC program 1 can be used with either PLC program 17 or 18. The difference between PLC program 18 and 17 is that PLC program 18 uses PMAC's built in timers instead of the servo clock to create a delay.

```
;********************* Setup and Definitions ***************************
CLOSE
M0->X:$0,24                    ; Servo cycle counter (timer)
M85->X:$07F0,24                ; Open memory for start time
M86->X:$07F1,24                ; Open memory for elapsed time
M87->X:$0770,1                 ; Bit that is zero on power on/reset
M90->X:$0700,0,24,S            ; PMAC timer register
M130->D:$2B                    ; #1 actual position (double word)
M230->D:$67                    ; #2 actual position (double word)
M330->D:$A3                    ; #3 actual position (double word)
M430->D:$DF                    ; #4 actual position (double word)
;******************* PLC programs to do the task ********************
OPEN PLC 1
CLEAR
DISPLAY 0 "#1 POS =           "
; Display labels
DISPLAY 20 "#2 POS =          "
DISPLAY 40 "#3 POS =          "
DISPLAY 60 "#4 POS =          "
P86=333*8388608/I10
DISABLE PLC 1                  ; Only need to write the labels once
CLOSE
OPEN PLC 17
CLEAR
IF (M87=0)                     ; Need to mark starting time
   M85=M0                      ; Record starting time
   M87=1                       ; No longer first time through
ENDIF
P130=M130/(I108*32)            ; These statements take the actual position,
P230=M230/(I208*32)            ; add in the position bias, then scale into
P330=M330/(I308*32)            ; counts and user units
P430=M430/(I408*32)
M86=M0-M85                     ; Compute time since last display
IF (M86>P86)                   ; More than update time?
   DISPLAY  9, 10.4, P130      ; Display the positions
   DISPLAY 29, 10.4, P230
   DISPLAY 49, 10.4, P330
   DISPLAY 69, 10.4, P430
   M87=0                       ; So program will mark starting time
ENDIF
CLOSE
OPEN PLC 18
CLEAR
IF (M87=0)                     ; Need to mark starting time
   M90=P86*8388608/I10         ; Load timer register for P86 milliseconds
   M87=1                       ; No longer first time through
ENDIF
P130=M130/(I108*32)            ; These statements take the actual position,
```

```
P230=M230/(I208*32)           ; add in the position bias, then scale into
P330=M330/(I308*32)           ; counts and user units
P430=M430/(I408*32)



IF (M90<0)                    ; Has time expired?
   DISPLAY  9, 10.4, P130     ; Display the positions
   DISPLAY 29, 10.4, P230
   DISPLAY 49, 10.4, P330
   DISPLAY 69, 10.4, P430
   M87=0                      ; So program will mark starting time
ENDIF
CLOSE
```

Example 20: In Position Confirming Routines

While the PMAC has an automatic "in-position" function, it may not meet the exact needs of many users. It is possible for these users to construct more sophisticated functions in PLC. The routines here provide a variety of suggested methods of determining whether a system is "in-position". Since each user may have a different idea of what constitutes a system settled in position, a variety of techniques are presented here. These examples are intended to provide ideas on how to perform this function both the actual in-position conditions and the action taken on being in-position will probably be different in every application (here an output bit is set).

```
; ************************ Setup and Definitions ************************
                              ; M-Variable Definitions for Output Bits
M1->Y:$FFC2,8,1               ; Machine Output 1
M2->Y:$FFC2,9,1               ; Machine Output 2
M3->Y:$FFC2,10,1              ; Machine Output 3
M4->Y:$FFC2,11,1              ; Machine Output 4
                              ; Address bits that show no move is commanded
M140->Y:$0814,0               ; #1 In-Position Bit
M240->Y:$08D4,0               ; #2 In-Position Bit
M340->Y:$0994,0               ; #3 In-Position Bit
M440->Y:$0A54,0               ; #4 In-Position Bit
M187->Y:$0817,17              ; &1 In-Position Bit
M177->*                       ; Self-referenced variable
M233->X:$0079,13              ; #2 desired-velocity-zero bit
M366->X:$00AB,24              ; Motor #3 actual velocity
M466->X:$00E7,24              ; Motor #4 actual velocity
;*************** PLC Program to execute In-Position Function************
OPEN PLC 10
CLEAR
```

This first branch is a simple in-position function for motor #1. Setting an output when the motor becomes "in position". The shadow bit (M177) is used so that the output is set only when the in-position bit first goes true, and will not get set again as long as the motor stays in position.

```
IF (M140=1)                   ; Motor in position?
   IF (M177=0)                ; Not in position last scan?
   M1=1                       ; Set output
   M177=1                     ; Latch output
   ENDIF
ELSE
   M177=0
ENDIF
```

```
This second branch requires ten consecutive cycles within the in-position
band to report an "in-position" condition for motor #2

IF (M233=1)                    ; No commanded move
   IF (M240=1)
   P240=P240+1                 ; Increment counter
   IF (P240!<10)               ; Ten times in band?
   P240=10                     ; To prevent overflow
   M2=1                        ; Set in-position signal
   ELSE
   M2=0                        ; Clear in-position signal
   ENDIF
   ELSE                        ; Not in band
   P240=0                      ; Reset counter
   M2=0                        ; Clear in-position signal
   ENDIF
ELSE                           ; Is commanding a move
   P240=0                      ; Reset counter
   M2=0                        ; Clear in-position signal
ENDIF
```

This third branch requires that both motors #3 & #4 (in C.S. 1) be in-position and both have moved less than a quarter of the band since the last cycle to set the output.

```
IF (M187=1)                          ; All C.S. 1 motors in position?
  P366=M366/(I309*32)                ; #3 velocity in cts/servo cycle
  P466=M466/(I409*32)                ; #4 velocity in cts/servo cycle
  IF (P366<(I328/64 AND P466<I428/64)     ; Ix28 in 1/16 ct
  M3=1
  ENDIF
ELSE
  M3=0
ENDIF
CLOSE
I5=3                                 ; So PLCs can be enabled
ENABLE PLC 10
```

Example 21: Encoder Modulo Check

This PLC program uses the position-capture feature of PMAC to do a once-around check of Encoder 1 by triggering the capture with the third channel and checking the number of counts that have gone by since the last triggered position. This action should not be started until one C-channel has already been captured.

```
;****************** Definitions and setup ****************************
CLOSE                          ; To ensure these are on-line
P50=2000                       ; Counts per revolution
I902=1                         ; Capture on rising C-channel
M40->*                         ; Flag to control operation
M41->*                         ; Error flag
M103->X:$C003,0,24,S           ; Encoder 1 captured pos reg
M117->X:$C000,17,1             ; Enc.1 position-captured flag
CLOSE                          ; Make sure others are closed
;******************** PLC Program ************************************
OPEN PLC 4                     ; Open for editing
CLEAR                          ; Erase existing contents
IF (M40>0)                     ; User sets M40 to 1 to start
IF (M40>1 AND M117=1)          ; Not 1st time; captured
```

```
  P12=P11                       ; Store from last time thru
  P11=M103                      ; Most recently captured
  P13=P11-P12                   ; Difference between the two
  IF (P13>P50/2)                ; A positive revolution
  IF (ABS(P13-P50)>1)           ; More than 1 off ?
  M41=1                         ; Set error flag
  ENDIF
  ELSE
  IF (P13<-P50/2)               ; A positive revolution
  IF (ABS(P13+P50)>1)           ; More than one off?
  M41=1                         ; Set error flag
  ENDIF
  ELSE                          ; 2 readings from same index
  IF (ABS(P13)>6)               ; Error band must allow for pulse width
  M41=1                         ; Set error flag
  ENDIF
  ENDIF
  ENDIF
  ELSE                          ; First time thru
  IF (M117=1)                   ; Has position been captured?
  P11=M103                      ; Grab captured position
  M40=2                         ; To mark first time done
  ENDIF
  ENDIF
ENDIF
CLOSE
M40=0                           ; So will not start right away
M41=0                           ; Clear error flag to start
ENABLE PLC 4
```

Example 22: Store The Home-Position Encoder Offset

This program stores the home-position encoder offset for future use. The encoder position is zero at power-up/reset when the home position is found, the motor position is set to zero, but the encoder position is not. Therefore we will want to have an offset value to relate motor and encoder positions if we will be using encoder registers directly for position-compare and capture functions (the servo handles the offset automatically for most purposes). This PLC program stores the offset in a P-variable as soon as the position has been found (the capture register will be re-used).

```
;***************** Set-up and definitions ****************************
CLOSE
M103->X:$C003,0,24,S            ; Enc 1 captured-pos register
M140->X:$003D,10,1              ; #1 home-search-in-progress
M141->*
;****************** PLC program listing ******************************
OPEN PLC 3
CLEAR
IF (M140=1)                     ; Searching for home
      M141=1                    ; Set flag
ENDIF
IF (M140=0 AND M141=1)          ; Have just found home flag
      P103=M103                 ; Save capture home offset
      M141=0                    ; Mark as not searching
ENDIF
CLOSE
ENABLE PLC 3
```

Example 23: Pure Position Step

This program performs a pure position step on Motor #1. It is done with the PLC writing directly to the command position registers, because motion programs would blend this. The step size in counts must be in variable P80, and the step time (each way) in milliseconds must be in P85. To run this on a different motor, only the definition of M81 must be changed.

```
CLOSE                          ; Ensure following are on-line
DELETE GATHER                  ; Make sure there is room
M88->X3,18                     ; Data gather prep bit
M89->X3,19                     ; Data gather on bit
M0->X0,24                      ; Servo cycle counter (timer)
M85->X$07F0,24                 ; Open memory for start time
M86->X$07F1,24                 ; Open memory for time elapsed
M81->D$28                      ; #1 cmd. pos. (double word)
OPEN PLC 2
CLEAR
CMD"GAT"                       ; Turn on data gathering
P81=P80*I108*32                ; Compute increment to pos reg
P86=P85*8388608/I10            ; Step time in servo cycle
M81=M81+P81                    ; Increment commanded position
M85=M0                         ; Start of timer
M86=M0-M85                     ; Time elapsed so far
WHILE (M86<P86)                ; Still waiting for specified time?
  M86=M0-M85                   ; Time elapsed so far
ENDWHILE
M81=M81-P81                    ; Decrement commanded position
M85=M0                         ; Restart timer
M86=M0-M85                     ; Time elapsed so far
WHILE (M86<P86)                ; Past specified time?
  M86=M0-M85                   ; Time elapsed so far
ENDWHILE
CMD"ENDGATHER"                 ; Turn off data gathering
DISABLE PLC 2                  ; Turn self off so no repeat
CLOSE
I5=3                           ; Make sure PLCs can be enabled
                               ; First, reserve memory for data gathering
                               ; with the on-line DEFINE GATHER command (
                               ; DEF GAT for short).  The step action will be
                               ; triggered with an "ENABLE PLC 2" command
                               ; ("ENAPLC2" for short)
```

Example 24: Pulse Output with Motors

These sample programs combine to provide a series of output pulses occurring when the motor using Encoder 1 reaches set positions at even increments apart. It uses the position-compare feature in the DSP-GATE IC to generate the pulses. PLC 0 is used to update the preloaded compare position (using the fast PLC 0 is only necessary if you need very fast -- near 1 kHz -- pulses). PLC 1 is used to prepare the function it executes only once, then disables itself.

```
;****************** Definitions and Setup ********************************
CLOSE
M101->X:$C001,0,24,S           ; Encoder 1 actual position
M103->X:$C003,0,24,S           ; Encoder 1 compare position
```

```
M105->X:$07F0,0,24,S          ; Scratch 24-bit register (to handle rollover)
M111->X:$C000,11,1            ; ENC1 EQU flag latch control
M112->X:$C000,12,1            ; ENC1 EQU output enable
M116->X:$C000,16,1            ; ENC1 EQU flag bit
P101=50                       ; Increment in counts
;********************* PLC Program ************************************
OPEN PLC 1
CLEAR
M112=1                        ; Enable EQU output
M105=M101+P101                ; Put 1st EQU position in scratch register
                              ; (this can be any starting position you wish
                              ; here it is relative to starting position)
M103=M105                     ; Put value in compare reg
M105=M105+P101                ; Prepare next value
ENABLE PLC 0                  ; To start up repetitive fn.
DISABLE PLC 1                 ; This is one time only
CLOSE
OPEN PLC 0
CLEAR
IF (M116=1)                   ; Reached last EQU position?
  M103=M105                   ; Load next EQU position
  M105=M105+P101              ; Prepare following EQU pos
  M111=0                      ; Clear and set control bit
  M111=1                      ;  to clear latched flag
ENDIF
CLOSE
```

Example 25: PLC I/O

These examples show how you can use PLC programs with PMAC's general-purpose I/O

```
;********************** Definitions and Setup ***********************
CLOSE                         ; To ensure these are on-line
M1->Y:$FFC2,8,1               ; Machine Output 1
M2->Y:$FFC2,9,1               ; Machine Output 2
M3->Y:$FFC2,10,1              ; Machine Output 3
M4->Y:$FFC2,11,1              ; Machine Output 4
M11->Y:$FFC2,0,1              ; Machine Input 1
M12->Y:$FFC2,1,1              ; Machine Input 2
M13->Y:$FFC2,2,1              ; Machine Input 3
M14->Y:$FFC2,3,1              ; Machine Input 4
M20..39->*                    ; Self-referenced flag variables
;************************ PLC Program *******************************
CLOSE                          ; Make sure other buffers closed
OPEN PLC 5                     ; Open buffer for editing
CLEAR                          ; Erase existing contents
```

This first branch sets a variable based on the state of Machine Input 1. This variable could be a destination, speed, time, etc.

```
IF (M11=1)                    ; Machine Input 1 true?
  P1000=5000                  ; If so, set to this value
ELSE
  P1000=500                   ; If not set to this value
ENDIF
```

This next branch increments a counter every time Machine Input 2 goes true (edge triggered)

```
IF (M12=1)                      ; Machine Input 2 true?
  IF (M22=0)                    ; Not true last time thru?
  P8=P8+1                       ; Have rise edge, so increment
  M22=1                         ; Note as true for next time thru
  ENDIF
ELSE                            ; MI2 is not true
  M22=0                         ; Note as not true for next time thru
ENDIF
```

This next branch sets an output only if all four of a set of flag variables are true

```
IF (M23=1 AND M24=1)            ; First two flags true?
AND (M25=1 AND M26=1)           ; Second two flags true?
  M1=1                          ; Then set output
ELSE
  M1=0                          ; Otherwise clear output
ENDIF
CLOSE                           ; Close the buffer
ENABLE PLC 5                    ; Enable operation of program
```

Example 26: Circular Interpolation

This program "draws" a daisy-shaped figure in the XY-plane. It illustrates circular interpolation, the use of subroutines, and output setting.

```
OPEN PROG 10 CLEAR
HM1..8                          ; Home all motors in C.S.
F2000                           ; Set feedrate
TA 10                           ; Accel time
NORMAL K-1                      ; Define XY plane for circles
LIN                             ; Start with linear moves
GOTO 0                          ; Jump over subroutines
N62                             ; Label to start subroutine
DWELL01
M1=0                            ; Turn off output
F2000                           ; Faster feedrate
DWELL 100
RET                             ; Return to line after GOSUB
N63                             ; Label to turn on subroutine
DWELL01
M1=1                            ; Turn on output
F1000                           ; Slower feedrate
DWELL 100
RET                             ; Return to line after GOSUB
N0                              ; Start of main program area
GOSUB 62
X2082.4000 Y1778.0000           ; Linear offset move
GOSUB 63                        ; To turn on output (e.g. laser)
                                ; Full circle move (end point = start point)
CIR 1 X2082.4000 Y1778.0000 I-606.2000 J0.0000 LIN
GOSUB 62                        ; To turn off output
X1880.9716 Y2232.2616
GOSUB 63
```

```
                              ; First "petal"
CIR 2 X1070.7003 Y2233.1163 I-404.7716 J345.5384 LIN
GOSUB 62
X1930.4225 Y1383.0427
GOSUB 63
                              ; Second "petal"
CIR 2 X1937.9537 Y2179.4222
I345.5775 J394.9573 LIN
GOSUB 62
X1012.0638 Y2163.0278
GOSUB 63
                              ; Third "petal"
CIR 2 X1013.4259 Y1394.1638
I-335.6638 J-385.0278 LIN
GOSUB 62
X1071.4245 Y1323.7339
GOSUB 63
                              ; Fourth "petal"
CIR 2 X1885.0458 Y1318.9081 I404.7755 J-345.5339 LIN
GOSUB 62
X74.0000 Y178.2000
GOSUB 63
X2870.6000 Y178.2000
X2870.6000 Y3391.2000
X74.0000 Y3391.2000
X74.0000 Y178.2000
GOSUB 62
X0.0000 Y0.0000
CLOSE
```

Example 27: Proportional Voltage Output



Make sure no activated motors use these DACs.

This example utilizes extra analog outputs to provide a voltage proportional to axis positions.

```
;************************ Definitions and setup ************************
M101->X:$C001,0,24,S          ; Encoder 1 position
M201->X:$C005,0,24,S          ; Encoder 2 position
M302->Y:$C00B,8,16,S          ; DAC 3 output
M402->Y:$C00A,8,16,S          ; DAC 4 output
M30->*                        ; Self-referenced flag
;P10=?                        ; (constant of proportionality)
;************************ PLC program contents *************************
CLOSE                         ; Close any open buffer
OPEN PLC 3                    ; Open buffer for editing
CLEAR                         ; Erase contents of buffer
IF (M30=1)                    ; M30 sets mode
  M302=P10*M101               ; DAC3 proportional to Pos 1
  M402=P10*M201               ; DAC4 proportional to Pos 2
ELSE                          ; Not in this mode
  M302=0                      ; DAC3 is zero output
  M402=0                      ; DAC4 is zero output
ENDIF
CLOSE                         ; Close the buffer
ENABLE PLC 3                  ; Enable this PLC program
```

Example 28: Pulsing a Machine Output

This example uses a PLC program to pulse Machine Output 1 every time a distance of 250 counts is passed (vector distance) by the axes using Encoders 1 and 2. The pulse is turned off the next time through the PLC (~1 msec).

```
;******************* Definitions and setup ******************************
M1->Y:$FFC2,8,1               ; Machine Output 1
M20->*                        ; Self-referenced flag variable
M101->X:C001,0,24,S           ; 24-bit encoder 1 position
M201->X:C005,0,24,S           ; 24-bit encoder 2 position
;******************* PLC program contents ******************************
CLOSE                         ; Close any open buffer
OPEN PLC 2                    ; Open buffer for editing
CLEAR                         ; Erase old contents
IF (M20=1)                    ; M20 sets the mode
  IF (M1=0)                   ; Output now off
  P102=(M101-P101)*(M101-P101) ; X-distance squared
  P202=(M201-P201)*(M201-P201) ; Y-distance squared
  IF (SQRT(P102+P202)>250)    ; distance > 250?
  M1=1                        ; Set output
  P101=M101                   ; Reset 'last pos' variable
  P201=M201                   ; Ditto
  ENDIF
  ELSE
  M1=0                        ; Clear output next time thru
  ENDIF
ENDIF
CLOSE                         ; Close the buffer
ENABLE PLC 2                  ; Enable the action
```

Example 29: Jogging Motors From A PLC Program

This program shows how to use the thumbwheel input lines to create jog switches that are dedicated to a particular motor. Note in particular the use of "latching flags" so that commands are only given on a change of state of the input line, making them edge-triggered.

```
;********************* Setup and definitions ************************
CLOSE
M50->Y:$FFC1,0                ; Thumbwheel port input bit 0
M51->Y:$FFC1,1                ; Thumbwheel port input bit 1
M52->Y:$FFC1,2                ; Thumbwheel port input bit 2
M53->Y:$FFC1,3                ; Thumbwheel port input bit 3
M60->*                        ; Latching bit for M50
M61->*                        ; Latching bit for M51
M62->*                        ; Latching bit for M52
M63->*                        ; Latching bit for M53
;********************* PLC Program to implement function ***********
OPEN PLC 16
CLEAR
IF (M50=1)                    ; Motor 1 jog plus switch on
  IF (M60=0)                  ; But not on last time
  COMMAND"#1J+"               ; Issue command
  M60=1                       ; Set latching flag
  ENDIF
```

```
ELSE                            ; Motor 1 jog plus switch off
  IF (M60=1)                    ; But not off last time
  COMMAND"#1J/"                 ; Issue stop command
  M60=0                         ; Set latching flag
  ENDIF
ENDIF
IF (M51=1)                      ; Motor 1 jog minus switch on
  IF (M61=0)                    ; But not on last time
  COMMAND"#1J-"                 ; Issue command
  M61=1                         ; Set latching flag
  ENDIF
ELSE                            ; Motor 1 jog minus switch off
  IF (M61=1)                    ; But not off last time
  COMMAND"#1J/"                 ; Issue stop command
  M61=0                         ; Set latching flag
  ENDIF
ENDIF
IF (M52=1)                      ; Motor 2 jog plus switch on
  IF (M62=0)                    ; But not on last time
  COMMAND"#2J+"                 ; Issue command
  M62=1                         ; Set latching flag
  ENDIF
ELSE                            ; Motor 2 jog plus switch off
  IF (M62=1)                    ; But not off last time
  COMMAND"#2J/"                 ; Issue stop command
  M62=0                         ; Set latching flag
  ENDIF
ENDIF
IF (M53=1)                      ; Motor 2 jog minus switch on
  IF (M63=0)                    ; But not on last time
  COMMAND"#2J-"                 ; Issue command
  M63=1                         ; Set latching flag
  ENDIF
ELSE                            ; Motor 2 jog minus switch off
  IF (M63=1)                    ; But not off last time
  COMMAND"#2J/"                 ; Issue stop command
  M63=0                         ; Set latching flag
  ENDIF
ENDIF
```

Example 30: Open-loop Movement

This example shows how you can get effective open-loop action during a PMAC motion program. In open-loop mode, the DAC output can be commanded directly. When used with a current-loop amplifier, this output is a torque or force command. When used with a velocity-loop amplifier, this output is a velocity command. During this "open-loop" action, PMAC is still technically trying to close the position loop. However, by setting the proportional gain to zero, the output of the PID is always zero. This means that any value written to the output offset I-variable (Ix29) is put directly on the DAC. This is only valid for motors not commutated by PMAC.

What must be handled carefully is the transition in and out of this mode. Going into "open-loop", the active DAC output is copied into the offset variable, and the proportional gain is set to zero (for the smoothest possible transition). In addition, the integral gain (Ix33) and integration limit (Ix63) are set to zero so they do not build up, and the following error limit (Ix11) is set to zero to disable it. Coming back into closed loop mode, the actual motor position must be copied into a couple of command position registers, one for the motor and one for the axis (scaled), so there is no lurch on closing the loop again.

In this example, the loop is closed again in ten steps, ramping down the offset and ramping up the proportional gain. This may not be necessary in a particular application -- but the smoothest possible transition was the intent of doing the ramping. Also integral gain and following error limits are restored -- if desired. This example just uses Motor #1 as the X-axis in Coordinate System 1. The technique is extendible to all motors in all coordinate systems.
In many cases, this program can be avoided by using the program command "CMD" to send open loop commands directly to a motor. This will be possible if the open motor can be assigned to a coordinate system not running a motion program.

```
;********************* Set-up and Definitions **********************
CLOSE
M102->Y:$C003,8,16,S          ; DAC 1 output register
M162->D:$002C                 ; Motor #1 actual position register
M163->D:$080B                 ; Motor #1 target position register
M165->L:$081F                 ; C.S. 1 X-axis scaled target position
;********************* Example motion program **********************
PROG 12
CLEAR
WHILE (1<2)                   ; Infinite loop
```

This first section of the program is a "dummy" routine to show some closed-loop action. It does a constant speed move. The condition that keeps it in this mode is arbitrary, and would almost certainly be different in an actual application.

```
WHILE (M1=0)                  ; Stays closed loop until condition false
INC                           ; Incremental moves
TM50
X100
ENDWHILE
```

This portion of the program converts to the "open-loop" mode. It could easily be put in a subroutine.

```
DWELL50                       ; Make sure stopped
DWELL50                       ; So variables not changed too early
I129=M102                     ; Output offset = DAC output
I130=0                        ; Proportional gain set to zero
I163=0                        ; Integration limit
I133=0                        ; Integral gain
I111=0                        ; Disable following error limit
```

This portion of the program constitutes the "open-loop" mode. Note that PMAC thinks it is DWELLing. During this time the host or the PLC can write to I129, changing the open loop output.

```
WHILE (M1=1)                  ; Stays in loop until false
M163=M162                     ; Keep making target pos = actual pos
DWELL10                       ; To take up time; could also be other axis moves
ENDWHILE
```

This portion of the program converts back to real closed-loop. It could be put in a subroutine.

```
M163=M162                     ; Motor target position = actual position
M165=M162/(I108*32)           ; Axis scaled target pos = actual pos
DWELL5                        ; Make sure above statements "take"
DWELL5                        ;  before gain is reintroduced
P129=I129                     ; Hold present output value
P130=8000                     ; Increment for prop gain (1/10 of total)
P131=0                        ; Increment counter
I163=8000000                  ; Restore integration limit value
```

```
I133=50000                    ; Restore integral gain
WHILE (P131<10)               ; Loop ten times
I129=I129-P129/10             ; Decrease by 1/10 of starting value
I130=I130+P130                ; Increase by 1/10 of target
P131=P131+1                   ; Count the cycle
DWELL20                       ; Control the speed of transition
ENDWHILE
I129=0                        ; Make sure got proper final value
I111=32000                    ; Restore following. error limit (2000 cts)
```

This final part of the program DWELLs in real closed loop so the transition can be judged. Again, the condition keeping it in this portion is arbitrary here.

```
WHILE (M2=0)
DWELL50
ENDWHILE
M2=0
ENDWHILE
CLOSE
```

Example 31: Inverse Kinematics

This example shows how PMAC can be used to perform the inverse kinematic problem -- the conversion of user coordinates (usually cartesian) to motor or joint coordinates. This program works only in two axes -- it shows the conversion of XY data into radial and angular axes. This would permit the cartesian specification of points on a rotary table with a radial arm. The concept can be extended to 6 or even 8 axes -- the calculations simply get more involved. Note that logical branching, not just pure computation, can be done in the conversion (here to handle C-axis rollover).

```
;********************* Set-up and definitions ***********************
#3->5000U                     ; Radial axis definition; units here should
                              ; be the same as for X and Y user units
#4->13.889C                   ; Angular (theta) axis definition; units are
                              ; degrees or radians, but must match I15.
                              ; Here 5000 cts/rev yields 13.889 cts/degree.
I15=0                         ; Use degrees in trig calculations.
Q21=5                         ; X zero position in U-axis coordinates
Q22=0                         ; C-axis offset
```

Program that performs the inverse kinematic problem and actually does the move in joint coordinates. This is called as a subprogram and expects 'X' data (in Q124) and 'Y' data (in Q125). It converts this data into radius and angle coordinates (the inverse kinematic problem) and commands the move -- here of the U (radius) and C (angle) axes. Q21 and Q22 should be preset as offsets for the U and C axes, respectively (if needed). The move is broken into small segments and the conversion is done at each segment boundary. The resulting joint trajectories are then splined together so the movement is smooth and accurate.

```
CLOSE
OPEN PROG 500
CLEAR
READ(X,Y)                     ; X endpoint->Q124; Y endpoint->Q125
Q12=Q10*Q11/1000              ; Distance per segment (user units)
Q14=Q24                       ; X working point is X starting point
Q15=Q25                       ; Y working point is Y starting point
SPLINE1                       ; Cubic spline mode
TA(Q11)                       ; Segment time
Q16=0                         ; To start loop
```

```
WHILE (Q16<1)
```
This section breaks the linear move into small segments. The coordinate transformation is done at each segment boundary. This next calculation should only be done on "feedrate axes".

```
Q34=Q124-Q14                    ; X distance to go
Q35=Q125-Q15                    ; Y distance to go
Q36=SQRT(Q34*Q34+Q35*Q35)       ; Vector distance to go
IF (Q36>Q12)                    ; Longer than segment size?
Q37=Q12/Q36                     ; Fraction of distance to go
```

The interpolation below should be done on all axes, feedrate or not

```
Q14=Q14+Q34*Q37                 ; X location at end of next segment
Q15=Q15+Q35*Q37                 ; Y location at end of next segment
ELSE                            ; Last segment of move
Q14=Q124                        ; End of segment is end of move
Q15=Q125                        ; End of segment is end of move
Q24=Q124                        ; X starting point for next move
Q25=Q125                        ; Starting point for next move
Q16=1                           ; To mark last segment
ENDIF
```
Now convert to joint coordinates

```
Q17=Q21-SQRT(Q14*Q14+Q15*Q15)   ; Radial axis coordinate
Q28=Q18                         ; Last angular coordinate
Q0=Q14                          ; Second argument for ATAN2
Q18=ATAN2(Q15)+Q22              ; Angular axis coordinate
IF (Q18-Q28>180) Q8=Q8-360      ; Rollover going negative
IF (Q18-Q28<-180) Q8=Q8+360     ; Rollover going positive
U(Q17)C(Q18+Q8)                 ; Make segment move
ENDWHILE
CLOSE
```

The following is a sample main program that can use the above program. This program simply specifies a square in user coordinates. The inverse kinematic problem is basically invisible to it. This shows that the actual inverse kinematics can be largely hidden from the final parts programmer.

```
OPEN PROG 25
CLEAR
Q10=2                           ; Desired feedrate (XY units/sec)
Q11=30                          ; Spline piece time
CALL 500 X2 Y2                  ; Move to start of square
DWELL100
M1=1                            ; Turn on output (e.g. laser)
CALL 500 X2 Y-2                 ; Do first side of square
DWELL100
CALL 500 X-2 Y-2                ; Do second side of square
DWELL100
CALL 500 X-2 Y2                 ; Do third side of square
DWELL100
CALL 500 X2 Y2                  ; Do last side of square
DWELL100
M1=0                            ; Turn off output
CALL 500 X-5 Y0                 ; Return to retract position
CLOSE
```

```
Start-up data: needed only
before the first running.
Q24=-Q21                        ; Starting X-value
Q25=-Q22                        ; Starting Y-value
Q8=0                            ; Starting C-axis rollover
Q18=0                           ; Starting C-axis
```

Example 32: Orientation of a Rotary Head

This example shows how to orient a rotary head to the angle of movement of an XY system. The key concept here is the use of the ATAN2 function, which returns a value between -180 degrees and +180 degrees. The ATAN2 function here takes two arguments: the first is a Y-axis value, and the second is an X-axis value, which is placed in the variable Q0 for the coordinate system.

The approach puts the cutting head in a different PMAC coordinate system from the X and Y axes. This allows it to run a separate motion program from the XY system. The head program can stay the same regardless of what the XY program is -- it simply uses the real-time data from the XY system to plan its moves. This permits the XY program to be generated automatically without concern for the details of the head movement. With the head program resident in PMAC, head movements are taken care of automatically, working from cues in the XY program.

This example head program looks at two specific cues from the XY program: DWELLs and M1 status. During an XY DWELL, it brings up the head (if it is not up already). It also issues a '%0' command to the XY coordinate system, so the next XY move will not start before the head program is ready. When the XY DWELL is complete (DWELLs are not affected by % commands), the head program looks to see if M1 has been set (the XY program's signal for cutter down). If it has been set, the head program uses the already-calculated XY target positions to compute the required head orientation, moves the head to that angle, lowers the cutter head, then re-starts the XY program (with a %100 command).

```
;********************* Set-up and definitions **********************
CLOSE
I13=10                          ; Move segmentation time (msec)
I15=0                           ; Trig functions use degrees
I427=14400                      ; Motor #4 has 14400 cts/rev and will
                                ; therefore rollover every 14400 cts
I194=2000000                    ; C.S. 1 time base slew rate
M161->D:$0028                   ; Motor #1 present commanded position
M163->D:$080B                   ; Motor #1 target position
M261->D:$0064                   ; Motor #2 present commanded position
M263->D:$08CB                   ; Motor #2 target position
M150->X:$0818,0                 ; C.S. 1 program-running status bit
M151->X:$0038,15                ; Motor #1 dwell-in-progress bit
M1->Y:$FFC2,8                   ; Machine Output 1 (PMAC A to D up/down
                                ; signal)
&1                              ; XY coordinate system
#1->10000X                      ; Motor 1 is X axis in C.S. 1
#2->10000Y                      ; Motor 2 is Y axis in C.S. 1
&2
#3->1000Z                       ; Motor 3 is Z axis (vertical) in C.S. 2
#4->40C                         ; Motor 4 is rotary axis in C.S. 2 (in deg)
OPEN PROG 2
CLEAR
FRAX(Z,C)                       ; Both vertical and rotary are feedrate
ABS                             ; Absolute mode for rollover to work properly
Q10=I108*32                     ; 1 encoder count in Mx61, Mx63 units
CMD"&1%0"                       ; Halt C.S. 1 moves temporarily
F10
```

```
C0                                ; Move rotary axis to starting position
DWELL5
Q2=0                              ; Starting angle value
CMD"&1%100"                       ; Restore C.S. 1 movement
WHILE (M150=1)                    ; As long as XY program is executing
  IF (M151=1)                     ; XY dwell in progress
  CMD "&1%0"                      ; Keep next XY move from starting
  F5
  Z0                              ; Pull up cutting head
  WHILE (M151=1) WAIT             ; Wait for XY dwell to finish
  IF (M1=1)                       ; XY program requesting head down?
  GOSUB 1000                      ; Calculate head angle
  F10
  C(Q2)                           ; Rotate head to starting angle
  Z10                             ; Lower head
  ENDIF
  DWELL5
  DWELL5                          ; So head up-down move finished
  CMD"&1%100"                     ; Allow XY moves to continue
  ENDIF
  TM20                            ; Set up 20 msec moves
  WHILE (M150=1 AND M151=0)       ; XY program running; no dwell
  GOSUB 1000                      ; Calculate head angle
  C(Q2)                           ; Move head to new angle
  ENDWHILE
ENDWHILE
RETURN                            ; Head program ends when XY program ends
N1000                             ; Start of head angle calculation routine
Q0=M163-M161                      ; X target pos - X current pos
Q1=M263-M261                      ; Y target pos - Y current pos
IF (ABS(Q0)>Q10 OR ABS(Q1)>Q10)   ; target not = current pos ?
  Q2=ATAN2(Q1)                    ; Calculate angle -- ATAN2(Q1,Q0)
ENDIF                             ; Note that if target pos = current pos
                                  ; Q2 does not change
RETURN
```

Example 33: Time Base Following

This example is for a coil winding machine. It shows how one axis on PMAC can be used as a master for another (or several others) through PMAC's time base following mode. In this application, Motor #1 (in Coordinate System 1) is the spindle axis, and is the master axis. Coordinate System 2 is slaved to the frequency from Motor #1's encoder (this encoder is used both for Motor #1's feedback and for Coordinate System 2's master frequency). Here, there is only one axis in C.S. 2: the traversing axis (Motor #3), which moves back and forth axially along the spindle to lay down the wire.

The key to proper winding (as for proper threading of screws or other similar applications) is to keep good control of the pitch, which requires a proper ratio of traverse speed to spindle speed. This could be done by writing a single coordinated program, but it is difficult to write such a program to keep the axes properly ratioed through the acceleration and deceleration of the spindle. Time-base following lets the program be much simpler: the traverse axis can be programmed pretending that the spindle axis is always turning at a fixed ("real-time") frequency, but the program executes at a rate proportional to the actual frequency. What must be done first is to set up the scaling factor to set the proper ratio between axes.

```
;********************* Example system parameters ************************
;Spindle axis:
                              ; Resolution: 100 lines/rev = 400 cts/rev
                              ; Max speed: 15,000 rpm = 100 cts/msec
                              ; Min slew speed: 100 rpm = 0.667 cts/msec
                              ; "Real-time" frequency = 64 cts/msec (want
                              ; power of 2; want max less than 200%) Scale-
                              ; factor: 131,072/64 = 2048 At "real-time":
                              ; 6.25 msec/rev Traverse axis:  Resolution:
                              ; 1000 lines/in = 4000 cts/in
;********************* PMAC Setup and Definitions ************************
CLOSE
I103=$720                     ; Motor 1 uses Encoder 1 as feedback (=1824D)
WY$728,$400720                ; Create time-base info from Encoder 1
WY$729,2048                   ; Scaling factor for time base
&1
#1->400X                      ; Motor 1 is X-axis in C.S. 1 X units are revs
                              I190=60000
                              ; Time units for C.S. 1 feedrate in minutes
I193=$0806                    ; C.S. 1 uses fixed time base (=2054D)
I303=$722                     ; Motor 3 uses Encoder 3 as feedback (=1825D)
&2
#3->4000Y                     ; Motor 3 is Y-axis in C.S. 2; Y units are inches
I293=$08C6                    ; C.S. 2 uses fixed time base (=2246D)
                              ; This is to start -- for homing etc.
                              ; When program starts, this will change to $0729
                              ; (=1833D) -- variable time base from Enc 1
I294=8388607                  ; For immediate changes in &2 % value
Q3=6.25                       ; msec/turn at "real-time"
CLOSE
;************************** Spindle Program ****************************
OPEN PROG 1                   ; Spindle program: one long move
CLEAR
I293=$729                     ; Slave C.S. 2 to Encoder 1 frequency
TS0                           ; No S-Curve acceleration
TA(Q5*1000)                   ; Accel time (sec) set by parameter Q5 of C.S. 1
F(Q6)                         ; Feedrate (rpm) set by parameter Q6 of C.S. 1
INC                           ; Incremental so don't worry about start point
X(Q7)                         ; # of turns set by Q7 of C.S. 1
DWELL10                       ; Allow to come to stop
DWELL10                       ; before releasing C.S. 2 from slavery
I293=$08C6                    ; Return C.S. 2 to internal time base
CMD"&2Q"                      ; Abort traverse program
DWELL500                      ; Let stabilize
M1=1                          ; Turn on output to cut wire
CMD"#3HM"                     ; Make traverse motor do home
CLOSE
;********************* Traverse axis program ************************
CLOSE
OPEN PROG 2
CLEAR
I293=$729
TS0
TA3                           ; Short accel/decel time
Q1=0                          ; Counter starts at zero
Q2=Q519                       ; C.S.2 Q519 same as C.S.2 Q7 (# of turns)
TM(Q3*Q12)                    ; Time/layer=(time/turn) * (turns/layer)
```

```
WHILE (Q1<Q2)
  Y(Q11*Q12)                    ; Distance is pitch times number of turns
  DELAY0                        ; To reach full distance
  Q1=Q1+Q12                     ; Turns counter
  IF (Q1<Q2)
  Y0                            ; To come back
  DELAY0                        ; To reach full distance
  Q1=Q1+Q12                     ; Two turns took place
  ENDIF
ENDWHILE
CLOSE
;*********** Parameters to set up a specific winding application *********
CLOSE
&1                              ; Spindle parameters
Q5=5                            ; 5 second accel/decel
Q6=2000                         ; 2000 rpm
Q7=3000                         ; 3000 turns total
&2                              ; Traverse parameters
Q11=0.005                       ; 5/1000 inch pitch
Q12=200                         ; 200 turns/layer (1 inch wide)
```

Example 34: Move Until Trigger

This example shows how to do a registration move, commonly known as a "move until trigger", on PMAC. It uses a position capture register to latch the trigger position immediately. The program can react to the trigger some time later without any loss of accuracy. In reacting to the trigger, PMAC must take the captured encoder position, convert it to motor position by adding in the home offset (see HOMOFFST.PMC), then scale that to axis position, adding in the desired distance from the trigger position that we wish to stop.

```
;********************* Set-up and Definitions ***********************
M103->X:$C003,0,24,S            ; Enc 1 captured-position register
M117->X:$C000,17,1              ; Enc 1 position-captured flag
I902=2                          ; Enc 1 capture on rising flag edge
I903=0                          ; Enc 1 use HMFL1 for capture
I190=1000                       ; Feedrate in units/sec
M191->L:$0822                   ; Motor 1 X-axis coefficient (S.F.)
M194->L:$0825                   ; Motor 1 axis offset
;************************** Motion Program ****************************
OPEN PROG 3
CLEAR
ABS                             ; Absolute move specification
TA50                            ; Accel time to speed
TS0                             ; No S-Curve accel
F10                             ; 10 user units/sec
P100=5                          ; position short of trigger
X(P100)                         ; first part of move -- get up to speed
                                ; Now go into loop to look for trigger

TM10                            ; Loop time is 10 msec
TA10                            ; Accel time must be <= TM
P99=M103                        ; Ensure it is cleared
WHILE (M117=0)                  ; Loop until position captured
  P100=P100+0.1                 ; Increment commanded position
  X(P100)                       ; Keep moving
ENDWHILE                        ; Found trigger -- calculate destination
```

```
P104=M103-P103                ; MotorPos=EncPos-HomeOffset
P105=(P104-M194)/M191         ; AxisPos=(MotorPos-AxisOffset)/S.F.
                              ; Perform final registration move
TA50
F10                           ; 10 units per second matches old speed
X(P105+0.75)                  ; Stop 3/4 unit from trigger
DWELL100                      ; Hold there
                              ; Further action could be taken here
CLOSE
```

Example 35: Linear and Circular Moves

This motion program is a simple example of linear and circular moves in an XY plane. It draws a rectangle with round corners.

```
CLOSE
I13=10                        ; Move segmentation time for circles
OPEN PROG 55
CLEAR
NORMAL K-1                    ; Circles in XY plane
RAPID                         ; Rapid move mode
X10Y5                         ; Move to starting position
M1=1                          ; Turn on output (e.g. laser, waterjet)
LINEAR                        ; Linear interpolation mode
F10                           ; Specified speed
X30Y5                         ; Bottom side of rectangle
CIRCLE2                       ; Counterclockwise circle
X35Y10J5                      ; Lower right corner
LINEAR
X35Y50                        ; Right side
CIRCLE2
X30Y55I-5                     ; Upper right corner
LINEAR
X10Y55                        ; Top side
CIRCLE2
X5Y50J-5                      ; Upper left corner
LINEAR
X5Y10                         ; Left side
CIRCLE2
X10Y5I5                       ; Lower left corner
DWELL10                       ; To make sure got there
M1=0                          ; Turn off output
RAPID
X0Y0                          ; Return to origin
CLOSE
```

Example 36: Motion Program Execution

This program demonstrates how the SAME motion program can be executed asynchronously by 8 different coordinate systems. It's like having 8 different programs running at the same time, independent of each other. A PLC program monitors the values of M1 through M8. When any of these variables are set to 1, the corresponding coordinate system executes the program, independent of the other coordinate systems. When any of these variables set to 0, the corresponding coordinate system stops executing, not affecting the other coordinate systems. In this example, the M-variables are self defined, making them behave just like P-variables. However, you can define these M-variables to point to machine inputs for reading switches, which can start and stop program execution.

```
CLOSE                          ; Close any potentially opened buffer
UNDEFINE ALL                   ; Clear out all current coordinate definitions
&1                             ; Select coordinate system 1
#1->X                          ; Define motor 1 as X axis
&2                             ; Select coordinate system 2
#2->X                          ; Define motor 2 as X axis
&3                             ; Select coordinate system 3
#3->X                          ; Define motor 3 as X axis
&4                             ; Select coordinate system 4
#4->X                          ; Define motor 4 as X axis
&5                             ; Select coordinate system 5
#5->X                          ; Define motor 5 as X axis
&6                             ; Select coordinate system 6
#6->X                          ; Define motor 6 as X axis
&7                             ; Select coordinate system 7
#7->X                          ; Define motor 7 as X axis
&8                             ; Select coordinate system 8
#8->X                          ; Define motor 8 as X axis
M1..8->*                       ; Define M1 through M8 as self defined M-variables
M1..8=0                        ; Set them all equal to zero
P1..8=0                        ; Set RUN/QUIT flag to zero
OPEN PLC 1 CLEAR               ; Open PLC buffer #1 (note:  CS = coordinate system)
IF(M1=1ANDP1=0)                ; Switch is on & this CS is not currently running
  CMD"&1B1R"                   ; Issue the commands to start the program
  P1=1                         ; This lets us know that the program is now running
ENDIF
IF(M1=0ANDP1=1)                ; If switch is off & this CS is currently running
  CMD"&1Q"                     ; Issue the commands to stop the program
  P1=0                         ; This lets us know that the program has stopped
ENDIF
IF(M2=1ANDP2=0)                ; Switch is on & this CS is not currently running
  CMD"&2B1R"                   ; Issue the commands to start the program
  P2=1                         ; This lets us know that the program is now running
ENDIF
IF(M2=0ANDP2=1)                ; If switch is off & this CS is currently running
  CMD"&2Q"                     ; Issue the commands to stop the program
  P2=0                         ; This lets us know that the program has stopped
ENDIF
IF(M3=1ANDP3=0)                ; Switch is on & this CS is not currently running
  CMD"&3B1R"                   ; Issue the commands to start the program
  P3=1                         ; This lets us know that the program is now running
ENDIF
IF(M3=0ANDP3=1)                ; If switch is off & this CS is currently running
  CMD"&3Q"                     ; Issue the commands to stop the program
  P3=0                         ; This lets us know that the program has stopped
ENDIF
IF(M4=1ANDP4=0)                ; Switch is on & this CS is not currently running
  CMD"&4B1R"                   ; Issue the commands to start the program
  P4=1                         ; This lets us know that the program is now running
ENDIF                          ; If switch is off & this CS is currently running
IF(M4=0ANDP4=1)
  CMD"&4Q"                     ; Issue the commands to stop the program
  P4=0                         ; This lets us know that the program has stopped
ENDIF
IF(M5=1ANDP5=0)                ; Switch is on & this CS is not currently running
  CMD"&5B1R"                   ; Issue the commands to start the program
```

```
  P5=1                          ; This lets us know that the program is now running
ENDIF
IF(M5=0ANDP5=1)                 ; If switch is off & this CS is currently running
  CMD"&5Q"                      ; Issue the commands to stop the program
  P5=0                          ; This lets us know that the program has stopped
ENDIF
IF(M6=1ANDP6=0)                 ; Switch is on & this CS is not currently running
  CMD"&6B1R"                    ; Issue the commands to start the program
  P6=1                          ; This lets us know that the program is now running
ENDIF
IF(M6=0ANDP6=1)                 ; If switch is off & this CS is currently running
  CMD"&6Q"                      ; Issue the commands to stop the program
  P6=0                          ; This lets us know that the program has stopped
ENDIF
IF(M7=1ANDP7=0)                 ; Switch is on & this CS is not currently running
  CMD"&7B1R"                    ; Issue the commands to start the program
  P7=1                          ; This lets us know that the program is now running
ENDIF
IF(M7=0ANDP7=1)                 ; If switch is off & this CS is currently running
  CMD"&7Q"                      ; Issue the commands to stop the program
  P7=0                          ; This lets us know that the program has stopped
ENDIF
IF(M8=1ANDP8=0)                 ; Switch is on & this CS is not currently running
  CMD"&8B1R"                    ; Issue the commands to start the program
  P8=1                          ; This lets us know that the program is now running
ENDIF
IF(M8=0ANDP8=1)                 ; If switch is off & this CS is currently running
  CMD"&8Q"                      ; Issue the commands to stop the program
  P8=0                          ; This lets us know that the program has stopped
ENDIF
CLOSE                           ; Close the PLC buffer
OPEN PROG 1 CLEAR               ; Open program buffer 5 and clear it out
ABS                             ; Set absolute mode
TA500                           ; Set accel/decel time of 500 msec
TS10                            ; With 20 msec spent in S-curve accel/decel
F10000                          ; Set feedrate to 10000 counts per Ix90 msec
N10                             ; Program label-- we will jump to here later on...
X10000                          ; Move X axis to position 10000
DWELL400                        ; Sit there for 400 msec
X0                              ; Move back to position 0
DWELL400                        ; Wait another 400 msec
GOTO 10                         ; Repeat! (until the PLC program tells us to stop)
CLOSE                           ; Close the motion program buffer
I5=2                            ; Allow PLC 1 to be enabled
ENABLE PLC1                     ; Turn on PLC 1!
```

Example 37: G-code Programming

This program demonstrates the use of G-codes in a PMAC program. The program uses a few commonly used G-codes to cut a part (an L-shaped bracket) using 3 axes. In PMAC, G-codes actually treated as subroutine calls which contain true PMAC commands. However, you use the G-codes exactly the same way you would in a true G-code machine, such as a CNC machine. The fact that they are subroutine calls gives you the added flexibility to customize your G-codes to perform whatever tasks you need.

```
CLOSE                          ; Close any previously opened buffer
OPEN PROG 6 CLEAR              ; Open motion program buffer 6 and clear it out
G17                            ; Select the plane for doing circles
G20                            ; Select inches for units
G90                            ; Select absolute mode
G94                            ; Select inches per minute mode
F100                           ; Set feedrate of 100 inches per minute
G00 X1 Y4                      ; Go RAPIDly to starting position
M3 S1800                       ; Start spindle at 1800 RPM clock wise
G01 Z.1                        ; Lower the cutting tool
M08                            ; Turn coolant on
Z0                             ; Plunge cutting tool into material
Y12                            ; Cut
G02 X2 Y13 I1 J0               ; Do rounded corner
G01 X3                         ; Cut across
G02 X4 Y12 I0 J-1              ; Do rounded corner
G01 Y6                         ; Cut downward
G03 X7 Y4 I3 J0
G01 X13                        ; Cut across
G02 X14 Y3 I0 J-1
G01 Y2                         ; Cut downward
G02 X13 Y1 I-1 J0
G01 X4                         ; Cut across backward
G02 X1 Y4 I0 J4                ; Last rounded edge
G01 Z.1                        ; Remove cutting tool from material
X3 Y3                          ; Move to next point
Z0                             ; Plunge cutting tool into material
G02 I.5 J.5                    ; Cut first hole
G01 Z.1                        ; Remove cutting tool from material
X2 Y11                         ; Move to next point
Z0                             ; Plunge cutting tool into material
G02 I.5 J.5                    ; Cut first hole
G01 Z.1                        ; Remove cutting tool from material
X12 Y2                         ; Move to next point
Z0                             ; Plunge cutting tool into material
G02 I.5 J.5                    ; Cut first hole
G01 Z.1                        ; Remove cutting tool from material
M09                            ; Turn off coolant
Z1                             ; Lift cutting tool
G00 X0 Y0                      ; Return to home position
M05                            ; Turn off spindle
CLOSE                          ; Close this program buffer
; Here are some M-variable definitions for the G-code routines...
M163->D:$080B                  ; Motor 1 move-end target position
M165->L:$081F                  ; C.S. 1 X-axis move-end target position
M191->L:$0822                  ; Motor 1 X-axis coefficient
M194->L:$0825                  ; Motor 1 axis offset
M263->D:$080B                  ; Motor 2 move-end target position
M265->L:$081F                  ; C.S. 1 Y-axis move-end target position
M292->L:$0823                  ; Motor 2 Y-axis coefficient
M294->L:$0824                  ; Motor 2 axis offset
M363->D:$098B                  ; Motor 3 move-end target position
M365->L:$0821                  ; C.S. 1 Z-axis move-end target position
M393->L:$09A4                  ; Motor 3 Z-axis coefficient
M394->L:$09A5                  ; Motor 3 axis offset
OPEN PROG 1000 CLEAR           ; G-Code definition subroutine program
```

```
RAPID RET                       ; G00
N1000 LINEAR RET                ; G01
N2000                           ; G02
CIRCLE1 RET                     ; Clockwise arcs
N3000                           ; G03
CIRCLE2 RET                     ; Counterclockwise arcs
N17000 NORMAL K-1               ; G17 -- define X-Y plane
RET
N20000                          ; G20 -- inch mode
M191=2000                       ; X axis scale factor 2000 cts/in
M292=2000                       ; Y axis scale factor 2000 cts/in
M393=2000                       ; Z axis scale factor 2000 cts/in
P163=M163/(I108*32)             ; Scale motor 1 pos into counts
M165=(P163-M194)/M191           ; Change X axis position
P263=M263/(I208*32)             ; Scale motor 2 pos into counts
M265=(P263-M294)/M292           ; Change Y axis position
P363=M363/(I308*32)             ; Scale motor 3 pos into counts
M365=(P363-M394)/M393           ; Change Z axis position
M70=0                           ; Flag to note not metric
RET
N90000 ABS                      ; G90 -- absolute mode
M90=1                           ; Flag for mode
RET
N94000 I190=60000               ; G94 -- feedrate is per minute
I193=2054                       ; Use internal timebase
RET
CLOSE                           ; Close this program buffer
OPEN PROG 1001 CLEAR            ; M-Code definition subroutine program
N03000 READ(S)                  ; M03 -- turn on spindle CW
IF(Q100&262144>0)               ; If S parameter specified,...
  I422=Q119/30                  ; Set jog (spindle) speed!
ENDIF
CMD"#4J+"                       ; Jog motor #4 for spindle
RET
N05000                          ; M05 -- turn off spindle
CMD"#4J/"                       ; Stop spindle
RET N08000                      ; M08 -- coolant on
M1=0                            ; Machine output = 1
RET
N09000                          ; M09 -- coolant off
M1=0                            ; Machine output = 0
RET
CLOSE                           ; Close this program buffer
```

Example 38: Delayed Program Execution

This program demonstrates how two separate coordinate systems can run the same program with a specified offset delay. In this example, motor 1 in coordinate system 1 will start first, and then motor #2 will follow, lagging by 400 milliseconds as specified by Q1. Q-variables are special in PMAC because the same Q-variable can contain unique values in different coordinate systems. Here, Q1=0 in CS 1, and Q1=400 in CS 2. Also, since both coordinate systems are running the same program, Q2 is used as the loop counter since it is incremented by both CS 1 AND CS 2--we could not used a P-variable for either the delay variable or the loop counter. To run the program, Type:.....
&1B1R&2B1R

```
UNDEFINE ALL                    ; Clear out all coordinate system definitions
&1                              ; Select coordinate system 1
#1->X                           ; Define motor #1 as X
I190=1000                       ; Set feedrate time units to seconds
Q1=0                            ; Set delay variable to 0 msec
&2                              ; Select coordinate system 2
#2->X                           ; Select coordinate system 2
I290=1000                       ; Set feedrate time units to seconds
Q1=400                          ; Set delay variable to 400 msec
OPEN PROG 1 CLEAR               ; Open program buffer 1 and clear it out
Q2=0                            ; Set our counter to zero
ABS                             ; Set absolute mode
TA100                           ; Set acceleration time to 100 msec
TS0                             ; With no S-curve time
F10000                          ; Set feedrate to 10000 counts/sec
DELAY (Q1)                      ; Delay an amount specified by Q1 in msec
WHILE (Q2<10)                   ; Begin a loop of 10 repetitions
  X2000                         ; Move X 2000 counts
  DELAY250                      ; Delay 250 msec
  X4000                         ; Move X another 2000 counts
  DELAY250                      ; Delay 250 msec
  X6000                         ; Move another 2000 counts
  DELAY250                      ; Delay 250 msec
  X0                            ; Return to position 0
  DELAY250                      ; Delay 250 msec
  Q2=Q2+1                       ; Bump our repetition counter
ENDWHILE                        ; End of loop
CLOSE                           ; Close the program buffer
```

Example 39: Integrated Current Limiting

This example shows how a PMAC PLC program can be used to create integrated-current limiting (I2T, "Eye-Squared-Tee" protection) to supplement or replace that in the amplifier. This type of protection, as opposed to instantaneous current limiting, is intended to prevent thermal overload of the amplifier and/or motor. As such, it uses a running time average of the square of the current (the digital equivalent of a capacitor charging up and bleeding down). The square function is used because power dissipation is proportional to the square of the current.

This example uses the commanded current output, and so is only valid for current-mode amplifiers. A more robust and flexible version would use the measured current, but that would require an analog to digital converter accessory. However, the math and logic are identical in both cases.

```
;********************* Set-up and definitions **********************
CLOSE
M132->X:0045,8,16,S             ; Motor #1 servo loop 16-bit output command -- before
                                ; sent to DACs.  Use of this register permits this
                                ; technique even for PMAC commutated motors.
P130=16384*16384                ; Squared current limit:  16K*16K bits^2.  When the
                                ; time-averaged current exceeds 16K bits (half of max)
                                ; routine will shut down drive.
P131=512                        ; Number of PLC scans over which the current is averaged.
                                ; This number, multiplied by the typical PLC scan cycle
                                ; time, should be approximately equal to the thermal time
                                ; constant of the motor.
```

```
;****************** Actual PLC Program ******************************
OPEN PLC 1 CLEAR
P132=0                              ; Make sure average is clear to start
DISABLE PLC 1                       ; So only runs once on power-up/reset
CLOSE
OPEN PLC 5 CLEAR
P132=((P131-1)*P132+M132*M132)/P131 ; Compute running average of square
IF (P132>P130)                      ; Exceeded limit
  CMD"&1A"                          ; Stop any motion in Coordinate System
  CMD"#1K"                          ; Kill this motor
  SEND"Motor1I2T Limit Trapped"     ; Message to host
  M1=1                              ; Turn on indicator
  P132=0                            ; Clear running average
ENDIF
CLOSE
```

Host Example Programs

PMACPOLL.C

```
/*    Sample program using read-ready bit polling.
*/
#include <stdio.h>
#include <dos.h>
#include <conio.h>
#define  SERIAL              555
#define  PCBUS               123
#define  COM1                1016
#define  COM2                760
#define  PMAC_PIC02          528+10
#define  PMAC_PIC03          528+12
int   combase,               /*base address of the card */
      baudrate,              /*baudrate for serial communication */
      speed,                 /*used for calculating timeout */
      able_to_talk,          /*flag for communications established */
      timeout;               /*limits how long to look for handshake */
char  buf [256];             /*holds input string */
int config_card_for (int port,int address) {
```

/* Clock speed multiplier sets limit on how long to look for handshake so read or write can occur. Software looks 700 times the number entered for proper handshake before giving up. The faster the CPU the higher this value should be. For PMAC the only thing this controls is how long to check to see if PMAC is going to send a line or not. For baud rate settings, use the following values:

DESIRED BAUD RATE	VALUE TO USE FOR VARIABLE BAUD RATE
300	Baud rate = 384
1200	Baud rate = 96
2400	Baud rate = 48
4800	Baud rate = 24
9600	Baud rate = 12
19200	Baud rate = 6
38400	Baud rate = 3



PMAC is fast enough that the Read Ready bit may only need to be checked at the beginning of the line (with timeout provisions) -- once the line read has started, this status check may be superfluous

Make sure you set the E-point jumpers to match your baudrate! */

```
speed = 9;
    combase = address;
    if (port == SERIAL) {
        baudrate = 12;                  //  select 9600 baudrate
        timeout = 7 * speed * (baudrate + 100);
        outportb (combase + 3, 131);    /*  set up the PC's serial port for communication */
        outportb (combase, baudrate);
        outportb (combase + 1, baudrate/256);
        outportb (combase + 3, 3);
    }
    else {            /*  PC-Bus will be used for communication */
        baudrate = 0;  /*  when baudrate is set to 0, PC-Bus communication is assumed */
        timeout = 7 * speed * 100;
        outportb (combase + 5,0) ;
        outportb (combase + 6,0) ;
    }
}
int sendline (char *outchar) {   /*  calls send char to send a string of characters to PMAC
                                     followed by a CR */
    int i = 0;
    while (outchar [i] != 0)
        sendchar (outchar [i++]);
    sendchar (13);
}
int sendchar (char outchar) {    /*  sends a character to the PMAC-PC card Note: even a 25 MHz
                                     386 PC has not been able to 'outrun' PMAC, so looking for
                                     Write Ready bit may be superfluous */
    int i;
    i = 0;
```

```
    if (baudrate == 0) {     /* Send data over PC-Bus interface */
                                    /* look for Write Ready bit (bit 1 of base address + 2)
*/
        while (i++ < timeout  &&  !(inportb (combase+2) & 2)) ;
        if (i < timeout)
            outportb (combase+7, outchar) ;
    }
    else {            /*  Send data over RS232 interface (serial) */
                         /*  look for Write Ready bit (bit 6 & 5 of base address + 5 & 6) */
        while (i++ < timeout && (inportb (combase+5) & 32) == 0);
        while (i++ < timeout && (inportb (combase+6) & 16) == 0);
        if (i < timeout)
            outportb (combase,outchar);
    }
    return (0);
}
int getline (char *linebuf) {
/* looks for a line of character data from the PMAC-PC card
    char   ic;
    int i,nc;
    if (able_to_talk) {
        ic = nc = i = 0;
        if (baudrate == 0) {     /*  Read data over the PC-Bus interface */
            while (i++ < timeout && ic != 13 && nc < 255)
/* look for Read Ready bit (bit 0 of base address + 2) */
                if ((inportb (combase+2) & 1) == 1) {
/* found it.. read a character and add on to string */
                    ic = inportb (combase+7) ;
                    linebuf [nc++] = ic ;
                    i = 0 ;
                }
        }
        else {                              /*  Read data over the RS-232 interface */
            outportb (combase+4,2);         //  release "hold off" signal
            while (i++ < timeout && ic != 13 && nc < 255) {
                   if ((inportb (combase+5) & 1) == 1) {
                    ic = inportb (combase);
                    enable ();
                    linebuf [nc++] = ic;
                    i = 0;
                    disable();
                    }
            }
            outportb (combase+4,0);      //  set the "hold off" signal
            enable ();
```

```
            while ((i += 2) < timeout);
        }
        linebuf [nc] = 0;
        return (nc);                    /*  Return number of characters received */
    }
    else {
        linebuf [0] = 0;
        return (0);                     /*  No characters received! */
    }
}
int Cards_On_Line () {                  /*  verify PMAC is there */
    int i,n;
    able_to_talk = 1;
    n = i = 0;
    while (n < 3  && i < 11) {
        if (baudrate != 0) {            /*  serial communications only */
            sendchar (24);              /*  send ^X */
            delay (20);
            sendchar (26);              /*  send ^Z  for serial mode */
        }
        sendline ("RHL:$720") ;
        i = getline (buf) ;
        n++ ;
    }
    if (i > 11  &&  i < 17) {
        cprintf ("Found PMAC...\r\n");
        return (1);
    }
    else {
        sound (1000); delay (250); nosound ();
        cprintf ("No PMAC found!\r\n") ;
        able_to_talk = 0;
        return (0);
    }
}
void show_printf (char *lbuf) {
    int i;
    for (i = 0; i < strlen (lbuf); i++) {
        if (lbuf [i] == 13 /* ||lbuf [i] == 7* / || lbuf [i] == 6 ||lbuf [i] == 10)
            switch (lbuf [i]) {
                    case 13: printf ("<CR>\r\n"); break;
                    case 7:  printf ("<BELL>\r\n"); break;
                    case 6:  printf ("<ACK>"); break;
                    case 10: printf ("<LF>\r\n"); break;
            }
```

```
        else {
            if (isprint (lbuf [i]))
                    putch (lbuf [i]);
            else
                    printf ("<%u>",lbuf [i]&255);
        }
    }
}
void main () {
    int done = 0;
    char   ic;
//  config_card_for (PCBUS,528); // Use this line to talk over PC-BUS!
    config_card_for (SERIAL,COM1);
    Cards_On_Line ();
    cprintf ("Now acting as a terminal to PMAC.\r\n");
    cprintf ("Press <ESC> to abort.\r\n");
    while (!done && able_to_talk) {
        while (!kbhit()) {       /*  no key pressed, so check card for data */
            getline (buf);
//          if (show_codes)
//                  show_printf (buf);
//          else
                    printf (buf);
        }
        switch (ic = getch ()) {
            case 1:
                    outp (PMAC_PIC03,0x40);
                    delay (4000);
                    outp (PMAC_PIC02,0x40);
                    break;
            case 27:             /*  ESC key pressed, so exit */
                    done = 1; break;
            case  0:             /*  swallow 2nd part of special keys (i.e. F-keys) */
                    getch (); break;
            default:
                    sendchar (ic);    /*  send character to PMAC */
                    if (ic == 13)     /*  ENTER pressed */
                       putch (10);
                    putch (ic);
                    break;
        }
    }
}
```

PMACINT.C

```
/*  Sample program to talk to PMAC using interrupts.  Make sure jumper E83 for IRQ 5 is in-
stalled to allow PMAC to interrupt the PC at IRQ 5.

*/

/* ---------------- PMAC bus port addresses --------------- */
#define PMACBUFFERSIZE   1024          /*  this our queue buffer size */
#define PMAC                528        /*  bus address of PMAC */
#define PMAC_STATUS      PMAC+2     /*  bus status byte */
#define PMAC_DATA        PMAC+7     /*  transmit/receive data */
#define PMAC_PIC00       PMAC+8     /*  8259 interrupt controller */
#define PMAC_PIC01       PMAC+9     /*     "      "          "    */
#define PMAC_PIC02       PMAC+10    /*     "      "          "    */
#define PMAC_RECEIVE     2             /*  ready to receive bit */
#define PMAC_SEND        1             /*  ready to send bit */
#define PMAC_IS_SENDING (inp (PMAC_STATUS) & PMAC_SEND)

/* ------------ PMAC communication interrupt values ----------------*/
#define PMAC_IRQ     4                    /*  0-7  IRQ0 - IRQ7 */
#define PMAC_MASK    (~(1<<PMAC_IRQ))  /*  for unmask of PMAC 8259 */
#define PMAC_EOI     0x20                 /*  Pmac end of interrupt */
#define PMAC_IPOS    1                    /*  IR0 for in position */
#define PMAC_BREQ    2                    /*  IR1 for buffer request */
#define PMAC_ERROR   4                    /*  IR2 for general error */
#define PMAC_FERROR  8                    /*  IR3 for following error */
#define PMAC_HREQ    16                   /*  IR4 for communication */
#define PMAC_IR5     32                   /*  IR5 */
#define PMAC_IR6     64                   /*  IR6 */
#define PMAC_IR7     128               /*  IR7 */
#define PMAC_NOP     0x40                 /*  8259 no operation code */

/*     -------------- PC interrupt values -------------------     */
#define IRQ          5                 //  PC interrupt used for PMAC:
                                          //  0 - 15 correspond to IRQ0 - IRQ15
#if (IRQ < 8)                         //  interrupt level for XT/AT
//#define  PC_INT        (8+IRQ)
#define PC_INT       0x77
#define PC_MASK      (~(1<<IRQ))       //  for unmask of PC's 8259
#define PC_PIC01     0x21              //  address for 8259 (master PIC in AT)
#define PC_PIC00     0x20
#else                                 //  interrupt vector for AT only
#define PC_INT       (104+IRQ)
#define PC_MASK      (~(1<<(IRQ-8)))// for unmask of PC's 8259
```

```
#definePC_PIC01      0xA1             //  address for second 8259 (slave PIC in AT)
#definePC_PIC00      0xA0
#endif
#define PC_EOI       0x20                //  end of interrupt
#defineACK           0x06
#defineLF               0x0c
#define EOL                 0x0d
#include <stdio.h>
#include <dos.h>
#include <conio.h>
#include <bios.h>
static void       (interrupt far *old_comm_int) ();
static void       interrupt far pmac_comm ();
static unsigned   combase,speed,able_to_talk,timeout,old_int_stat;
static char          ipos_flag,breq_flag,error_flag,ferror_flag,hreq_flag,
                  ir5_flag,buf [256],comm_buffer [PMACBUFFERSIZE],
                  *current_buffer,*next_in,*next_out;
void init_pmac_comm () {     /*  initialize the PMAC port for interrupt comm. */
   setup_pmac_queue (comm_buffer);     /*  setup input PMAC queue */
   disable ();                         /*  disable interrupts until we're done */
   old_comm_int = getvect (PC_INT);    /*  save original interrupt vector */
   setvect (PC_INT, pmac_comm);        /*  write in our new interrupt vector! */
/* -------  setup PC's 8259...  -------- */
   old_int_stat = inp (PC_PIC01);
   outp (PC_PIC01,(old_int_stat & PC_MASK));
/* -------  now setup PMAC's 8259...  -------*/
   outp (PMAC, 0);                 /*  flush PMAC interrupt */
   outp (PMAC_PIC00, 0x17);     /*  0x17 = edge, 0x1f = level trigger (ICW1) */
   outp (PMAC_PIC01, 0x08);     /*  vector for data bus (ICW2) */
   outp (PMAC_PIC01, 0x03);     /*  set up for 8086 mode (ICW4) */
   outp (PMAC_PIC01, 0x00);     /*  mask out interrupts IR6,IR7 */
/* If you wish to enable all interrupt inputs use 0x00 instead of 0xE0 */
   outp (PMAC, 0x81);              /*  DSP enable read */
   enable ();                      /*  enable interrupts because we're done! */
}
void restore_pmac_comm () {     /*   restore original interrupt vector */
   disable ();
   setvect (PC_INT, old_comm_int); /*  replace old PC vectors */
   outp (PC_PIC01, old_int_stat);  /*  restore old status */
   outp (PMAC_PIC01, 0xff);        /*  turn off PMAC's 8259 */
   enable ();
}
static void interrupt far pmac_comm (void) {   /*  PMAC interrupt service routine */
   int     ch;
   char    isr;
```

```
    sound (3000); delay (10); nosound ();   /* beep when interrupted! */
    disable();                  /* don't be interrupted unitl done */
    outp (PMAC_PIC02,PMAC_NOP); /* acknowledge interrupt via INTA/ */
    outp (PMAC_PIC00,0x0A);     /* setup to read IRR register */
    isr = inp (PMAC_PIC00);     /* read IRR register */
    if (isr & PMAC_IPOS)        /* if IPOS interrupt set, set a flag */
       ipos_flag = 1;
    if (isr & PMAC_BREQ)        /* if BREQ interrupt set, set a flag */
       breq_flag = 1;
    if (isr & PMAC_ERROR)       /* if ERROR interrupt set, set a flag */
       error_flag = 1;
    if (isr & PMAC_FERROR)      /* if FERROR interrupt set, set a flag */
       ferror_flag = 1;
    if (isr & PMAC_HREQ)        /* if HREQ interrupt set, set a flag */
       hreq_flag = 1;
//  if (isr & PMAC_IR5) {                /* if IR5 interrupt set, set a flag */
       ir5_flag = 1;
//     sound (3000); delay (10); nosound ();
//  }
    outp(PMAC_PIC02,PMAC_NOP);      /* acknowledge interrupt again! */
    outp(PMAC_PIC00,PMAC_EOI);      /* send end-of-intrpt to PMAC's 8259 */
    outp(PC_PIC00, PC_EOI);         /* send end-of-intrpt to PC's 8259 */
    enable();                       /* re-enable interrupts! */
    (*old_comm_int) ();             // jump to replaced routine
}
int setup_pmac_queue (char *buf) {
    next_in = next_out = current_buffer = buf;
    return (0);
}
int read_pmac_queue (char *buf) {   /* circular queue buffer for PMAC responses */
    int  nc = 0, done = 0;
    while (!done && (next_in != next_out)) {
       if (next_out == current_buffer + PMACBUFFERSIZE)
          next_out = current_buffer;
        if ((*next_out == EOL) || (*next_out == ACK)) {
          next_out++;
          done = 1;
       }
        else
          buf[nc++] = *next_out++;
    }
    buf[nc] = 0;
    return (nc);
}
void config_card_for (unsigned address) {
```

```
    speed = 9;
    combase = address;
    timeout = 7 * speed * 100;
    outportb (combase + 5,0) ;
    outportb (combase + 6,0) ;
}
void sendchar (char outchar) {
    int i = 0;
    while (i++ < timeout  &&  !(inportb(combase+2) & 2));
    if (i < timeout)
        outportb (combase+7, outchar) ;
}
void sendline (char *outchar) {
    int i = 0;
    while (outchar [i] != 0)
        sendchar (outchar [i++]);
    sendchar(13);
}
int getline (char *linebuf) {
    char   ic;
    int i,nc;
    if (able_to_talk) {
        ic = nc = i = 0;
        while (i++ < timeout && ic != 13 && nc < 255)
            if ((inportb (combase+2) & 1) == 1) {
                    ic = inportb (combase+7) ;
                    linebuf [nc++] = ic ;
                    i = 0 ;
            }
        linebuf[nc] = 0;
        return (nc);          /*  PMAC had something to say */
    }
    else {
        linebuf [0] = 0;
        return (0);           /*  PMAC had nothing to say */
    }
}
int Cards_On_Line () {
    int i,n;
    sendchar (24);            //  send ^ X
    delay (500);
    able_to_talk = 1;
    n = i = 0;
    while (n < 3  && i < 11) {
        sendline ("RHL:$720") ;
```

```
        i = getline (buf) ;
        n++ ;
    }
    if (i > 11  &&  i < 17) {
        cprintf ("\r\nFound PMAC...\r\n") ;
        while (getline (buf));       /* clear out PMAC's output buffer */
        return (1);
    }
    else {
        sound (1000); delay (250); nosound ();
        cprintf ("\r\nNo PMAC found!\r\n") ;
        able_to_talk = 0;
        return (0);
    }
}
void do_interrupt_terminal () {
    int    key,done = 0,pos;
    char   str [255],ch;
    init_pmac_comm ();
    sendline ("i3=0");
    while (!done) {
/*            Now process the interrupt(s) */
        if (hreq_flag) {             /* does PMAC have something to say? */
            hreq_flag = 0;
            while (PMAC_IS_SENDING) {/* actually, a polled read of data */
                   if (next_in == current_buffer + PMACBUFFERSIZE)
                       next_in = current_buffer;    /* read PMAC and store the */
                   *next_in++ = (char) inp (PMAC_DATA);    /* data in the queue! */
            }
        }
        if (ferror_flag) {
            ferror_flag = 0;
            cprintf ("\r\n*** Following Error! ***\r\n");
        }
        if (error_flag) {
            error_flag = 0;
            cprintf ("\r\n*** General Error! ***\r\n");
        }
        if (breq_flag) {
            breq_flag = 0;
            cprintf ("\r\n*** Buffer Request ***\r\n");
        }
        if (ipos_flag) {
            cprintf ("\r\n*** In Position *** %x\r\n",(int) ipos_flag);
            ipos_flag = 0;
```

```
        }
/*    Include this code for each additional interrupt enabled */
/*     if (ir5_flag) {
          cprintf ("\r\n*** Interrupt IR5 %x ***\r\n",(int) ir5_flag);
sendline ("p"); getline (buf);
sscanf (buf+1,"%d",&pos);
printf ("%d %d\r\n",pos,pos%360);
          ir5_flag = 0;
       }
*/     if (kbhit ()) {
          key = getche ();         /*  get a character from keyboard */
          switch (key) {
                 case 27:              /*  ESC key pressed, so exit */
                    done = 1; break;
                 case 13:              /*  ENTER pressed, so print LF */
                    cprintf ("\n");
                 default:
                    sendchar (key);
          }
       }
       if (read_pmac_queue (str) != 0)
          puts (str);              /*  print out anything in the queue */
   }
   restore_pmac_comm();
   sendline ("i3=1");
}
void main () {
   clrscr ();
   config_card_for (0xF090);
//  config_card_for (528);
   Cards_On_Line ();
   ipos_flag=breq_flag=error_flag=ferror_flag=hreq_flag=ir5_flag = 0;
   if (able_to_talk) {
      cprintf ("Make sure a jumper is installed on E80.\r\n");
      cprintf ("Now acting as an interrupt driven terminal to PMAC.\r\n");
      cprintf ("Press <ESC> to abort.\r\n");
      do_interrupt_terminal ();
   }
}
```

PMACPROT.C

```
/* PMAC polled rotary buffer dump program for PMAC.
*/
#include <stdio.h>
#include <dos.h>
#include <conio.h>
#include <dir.h>
#define  SERIAL   321
#define  PCBUS    123
#define  COM1     1016
#define  COM2     760
#define  TRUE     1
#define  FALSE    0
FILE     *tempfile;
int      combase,baudrate,speed,able_to_talk,timeout;
char     buf [256];
struct   ffblk ffblk;
int config_card_for (int port,int address) {
   speed = 9;
   combase = address;
   if (port == SERIAL) {
      baudrate = 12;                    /* select 9600 baud */
      timeout = 7 * speed * (baudrate + 100);
      outportb (combase + 3, 131);      /* set up the PC's serial port for
                                           communication */
      outportb (combase, baudrate);
      outportb (combase + 1, baudrate/256);
      outportb (combase + 3, 3);
   }
   else {            /* PC-Bus will be used for communication */
      baudrate = 0;  /* when baudrate is set to 0, PC-Bus communication is
assumed */
      timeout = 7 * speed * 100;
      outportb (combase + 5,0) ;
      outportb (combase + 6,0) ;
   }
}
int sendline (char *outchar) {   /* calls send char to send a string of char-
                                    acters to PMAC followed by a CR */
   int   i = 0;
   while (outchar [i] != 0)
      sendchar (outchar [i++]);
   sendchar (13);
}
int sendchar (char outchar) {    /* sends a character to PMAC */
```

```
   int   i;
   i = 0;
   if (baudrate == 0) {          /* Send data over PC-Bus interface */
      while (i++ < timeout  &&  !(inportb (combase+2) & 2)) ;
      if (i < timeout)
         outportb (combase+7, outchar) ;
   }
   else {                        /* Send data over RS232 interface (serial)
*/
      while (i++ < timeout && (inportb (combase+5) & 32) == 0);
      while (i++ < timeout && (inportb (combase+6) & 16) == 0);
      if (i < timeout)
         outportb (combase,outchar);
   }
   return (0);
}
int getline (char *linebuf) {
   char  ic;
   int   i,nc;
   if (able_to_talk) {
      ic = nc = i = 0;
      if (baudrate == 0) {          /* Read data over the PC-Bus interface */
         while (i++ < timeout && ic != 13 && nc < 255)
               if ((inportb (combase+2) & 1) == 1) {
                  ic = inportb (combase+7) ;
                  linebuf [nc++] = ic ;
                  i = 0 ;
               }
      }
      else {                        /* Read data over the RS-232 interface */
         outportb (combase+4,2);
         while (i++ < timeout && ic != 13 && nc < 255) {
               if ((inportb (combase+5) & 1) == 1) {
                  ic = inportb (combase);
                  enable ();
                  linebuf [nc++] = ic;
                  i = 0;
                  disable();  }
         }
         outportb (combase+4,0);
         enable ();
         while ((i += 2) < timeout);
      }
      linebuf [nc] = 0;
      return (nc);               /* Return number of characters received */
```

```
   }
   else {
      linebuf [0] = 0;
      return (0);                    /* No characters received! */
   }
}
int getchare () {    /* modified getline routine optimized for high speed
                        downloading of data to PMAC */
   char  ic;
   int   i,nc;
   if (able_to_talk) {
      ic = nc = i = 0;
      if (baudrate == 0) {    /* not RS-232 */
         while (nc < 1)
               if ((inportb (combase+2) & 1) == 1) {
                  ic = inportb (combase+7) ;
                  nc++;
               }
      }
      else {                  /* RS-232 */
         outportb (combase+4,2);
         while (nc < 1) {
               if ((inportb (combase+5) & 1) == 1) {
                  ic = inportb (combase);
                  enable();
                  nc++;
                  i = 0;
                  disable();  }
         }
         outportb(combase+4,0);
         enable();
         while ((i += 2) < timeout);
      }
      if (ic == 7)
         return (1);
      else
         return (0);
   }
   else
      return (0);
}
int Cards_On_Line () {
   int   i,n,repeat;
   able_to_talk = 1;
   for (repeat = 1;repeat <=3;repeat++) {
```

```
      n = i = 0;
    while (n < 3  && i < 11) {
         if (baudrate > 0) {     /* serial communications only */
               sendchar (24);       /* send ^X */
               delay (20);
               sendchar (26);       /* send ^Z  for serial mode */
         }
         sendline ("RHL:$720") ;
         i = getline (buf) ;
         n++ ;
      }
   }
   if (i > 11  &&  i < 17) {
      cprintf ("Found PMAC...\r\n") ;
      while (getline (buf));     /* clear out PMAC's output buffer */
      return (1);
   }
   else {
      sound (1000); delay (250); nosound ();
      cprintf ("No PMAC found!\r\n") ;
      able_to_talk = 0;
      return (0);
   }
}
 int lines_in_buffer () {      /* sends the PR command to PMAC which returns
                                  the number of lines in the rotary buffer not
                                  yet executed */
   int   lines_remaining;
   sendline ("PR");
   getline (buf);
   sscanf (buf,"%d",&lines_remaining);
   return (lines_remaining);
}
int main (int argc, char *argv []) {
   int         byte,i,aborted,done,key;
   long        line;
   char        filename [80],line_buf [256];
   cprintf ("\r\n\nPMAC Rotary File Program\r\n");
/* config_card_for (PCBUS,528); */
   config_card_for (SERIAL,COM1);
   Cards_On_Line ();
   if ((argv [1][0] >= ' ' && argv [1][0] <= 'z') && able_to_talk) {
      sendline ("Q CLOSE I9=0 I3=1");
      sendline ("&1");                        /* select coordinate system 1 */
      sendline ("DELETE TRACE DELETE GATHER");
```

```
      sendline ("DEFINE ROT 250 CLEAR");     /* setup rotary buffer */
      while (getline (buf));
      strcpy (filename,argv [1]);
      done = aborted = FALSE;
      while (!aborted && !done) {
         if ((tempfile = fopen (filename,"rt")) != NULL) {
               findfirst (filename,&ffblk,0);   /* open file to dump to ro-
                                                   tary buffer */
               i = line_buf [0] = 0;
               line = 1;
               while (fscanf (tempfile,"%c",&byte) != EOF && !aborted) {
                  if (kbhit ())
                     if (getch () == 27)  /* pressing ESC aborts dump */
                        aborted = TRUE;
                  byte = byte & 255;      /* strip off high order bits */
                  if (byte == 10) {       /* end of line reached */
                     while ((lines_in_buffer () > 250) && ((line - 1) % 250)
< 2 && !aborted);
                        if (kbhit ())
                           if (getch () == 27) /* pressing ESC aborts dump */
                              aborted = TRUE;
                     sendline (line_buf); getchare ();
                     if (line > 250)
                        cprintf ("*Sending line: %ld
\r",line);
                     else
                        cprintf ("Sending line: %ld\r",line);
                     i = line_buf [0] = 0;
                     line ++;
                     if (line == 250) {      /* ask only the first time
                                                through */
                        cprintf ("\n\n\nPress <ENTER> to begin execu-
tion:\r");
                        key = getch ();
                        while (key != 13 && key != 27)
                           key = getch ();   /* wait for ESC or ENTER */
                        if (key == 13)
                           sendline ("R");
                        else {
                           sendline ("CLOSE");
                           aborted = TRUE;
                        }
                     }
                  }
                  else {   /* since not at end of line yet, add it to our
                              string */
```

```
                    line_buf [i++] = byte;
                    line_buf [i] = 0;
                 }
              }
              done = TRUE;
              if (line < 250) {             /* ask only the first time */
                 cprintf ("Press <ENTER> to begin execution:");
                 key = getch ();
                 while (key != 13 && key != 27)
                    key = getch ();
                 if (key == 13)
                    sendline ("CLOSE R");   /* issue R if small file */
                 else                       /* (less than 250 lines) */
                    aborted = TRUE;
              }
              if (!aborted)
                 cprintf ("\r\nDone.\r\n");
              else {
                 cprintf ("\r\nAborted.\r\n");
                 sendline ("CLOSE");
              }
           }
           else {
                 aborted = TRUE;
                 sound (1000); delay (300); nosound ();
                 cprintf ("\r\nUnable to open file %s",filename);
           }
        }
        fclose (tempfile);
        sendline ("CLOSE");        /* close rotary buffer */
     }
}
```

PMACIROT.C

```
/* PMAC interrupt driven rotary buffer program for PMAC.  Make sure jumper
E83 for IRQ 5 is installed to allow PMAC to interrupt the PC at IRQ 5.

*/
/* ---------------- PMAC bus port addresses --------------- */
#define PMACBUFFERSIZE  1024        /* this our queue buffer size */
#define PMAC            528         /* bus address of PMAC */
#define PMAC_STATUS     PMAC+2      /* bus status byte */
#define PMAC_DATA       PMAC+7      /* transmit/receive data */
#define PMAC_PIC00      PMAC+8      /* 8259 interrupt controller */
#define PMAC_PIC01      PMAC+9      /*    "      "          "    */
```

```
#define PMAC_PIC02      PMAC+10     /*    "      "          "    */
#define  PMAC_RECEIVE   2           /* ready to receive bit */
#define PMAC_SEND       1           /* ready to send bit */
#define PMAC_IS_SENDING (inp (PMAC_STATUS) & PMAC_SEND)

/* ------------ PMAC communication interrupt values ----------------*/
#define PMAC_IRQ     4                 /* 0-7  IRQ0 - IRQ7 */
#define PMAC_MASK    (~(1<<PMAC_IRQ))  /* for unmask of PMAC 8259 */
#define PMAC_EOI     0x20              /* Pmac end of interrupt */
#define PMAC_IPOS    1                 /* IR0 for in position */
#define PMAC_BREQ    2                 /* IR1 for buffer request */
#define PMAC_ERROR   4                 /* IR2 for general error */
#define PMAC_FERROR  8                 /* IR3 for following error */
#define PMAC_HREQ    16                /* IR4 for communication */
#define PMAC_EQU1    32                /* IR5 for DP RAM acknowledge */
#define PMAC_NOP     0x40              /* 8259 no operation code */

/*     -------------- PC interrupt values -------------------     */
#define IRQ          5              /* PC interrupt used for PMAC */
#define PC_INT    (8+IRQ)
#define PC_MASK   (~(1<<IRQ))    /* for unmask of PC 8259 */
#define PC_PIC01  0x21           /* 8259 interrupt controller XT */
#define PC_PIC00  0x20           /* " 0-7 interrupts  "    */
#define PC_EOI    0x20           /* end of interrupt */
#define ACK          0x06
#define LF           0x0C
#define EOL          0x0D
#define PCBUS     123
#define TRUE      1
#define FALSE     0
#include <stdio.h>
#include <dos.h>
#include <conio.h>
#include <stdlib.h>
static void    (interrupt far *old_comm_int) ();
static void    interrupt far pmac_comm ();
static int     combase,speed,able_to_talk,timeout,old_int_stat,breq_flag;
static long       x,y,z;
static char       buf [256];
void init_pmac_comm () {      /* initialize the PMAC port for interrupt comm.
*/
   disable ();                      /* disable interrupts until we're done */
   old_comm_int = getvect (PC_INT);    /* save original interrupt vector */
   setvect (PC_INT, pmac_comm);     /* write in our new interrupt vector! */
```

```
/* -------  setup PC's 8259...  -------- */
   old_int_stat = inp (PC_PIC01);
   outp (PC_PIC01,(old_int_stat & PC_MASK));

/* -------  now setup PMAC's 8259...  -------*/
   outp (PMAC, 0);            /* flush PMAC interrupt */
   outp (PMAC_PIC00, 0x17);   /* 0x17 = edge, 0x1f = level trigger (ICW1) */
   outp (PMAC_PIC01, 0x08);   /* vector for data bus               (ICW2) */
   outp (PMAC_PIC01, 0x03);   /* set up for 8086 mode              (ICW4) */
   outp (PMAC_PIC01, 0xFD);   /* mask out ferr,err,ipos,hreq       (OCW1) */
   outp (PMAC, 0x81);            /* DSP enable read */
   enable ();                    /* enable interrupts because we're done! */
   breq_flag = 0;                /* clear our interrupt flags */
}
void restore_pmac_comm () {      /* restore original interrupt vector */
   disable ();
   setvect (PC_INT, old_comm_int);     /* replace old PC vectors */
   outp (PC_PIC01, old_int_stat);      /* restore old status */
   outp (PMAC_PIC01, 0xff);            /* turn off PMAC's 8259 */
   enable ();
}
   static void interrupt far pmac_comm (void) {/*  PMAC interrupt service
                                                   routine */
   int  ch;
   char isr;
/* sound (3000); delay (10); nosound (); /* beep when interrupted! */
   outp(PMAC_PIC02,PMAC_NOP);       /* rising edge of 1st INTA/ pulse */
   outp(PMAC_PIC00,PMAC_NOP);       /* trailing edge of 1st INTA/ pulse */
   outp(PMAC_PIC02,0x0B);           /* setup to read ISR register */
   isr = inp (PMAC_PIC00);          /* read ISR register */
   if (isr & PMAC_BREQ)             /* if BREQ interrupt set, set a flag */
      breq_flag = 1;
   outp(PMAC_PIC00,PMAC_NOP);       /* trailing adge of 2nd INTA/ pulse */
   outp(PC_PIC00, PC_EOI);          /* send end-of-intrpt to PC's 8259 */
   enable();                        /* re-enable interrupts! */
}
void config_card_for (int address) {
   speed = 9;
   combase = address;
   timeout = 7 * speed * 100;
   outportb (combase + 5,0) ;
   outportb (combase + 6,0) ;
}
void sendchar (char outchar) {   /* sends a character to PMAC */
   int   i = 0;
```

```
   while (i++ < timeout  &&  !(inportb(combase+2) & 2));
   if (i < timeout)
      outportb (combase+7, outchar) ;
}
void sendline (char *outchar) {  /* sends a string with CR to PMAC */
   int   i = 0;
   while (outchar [i] != 0)
      sendchar (outchar [i++]);
   sendchar(13);
}
int getline (char *linebuf) {    /* This is a polled read of PMAC */
   char  ic;
   int   i,nc;
   if (able_to_talk) {
      ic = nc = i = 0;
      while (i++ < timeout && ic != 13 && nc < 255)
         if ((inportb (combase+2) & 1) == 1) {
               ic = inportb (combase+7) ;
               linebuf [nc++] = ic ;
               i = 0 ;
         }
      linebuf[nc] = 0;
      return (nc);               /* PMAC had something to say */
   }
   else {
      linebuf [0] = 0;
      return (0);                /* PMAC had nothing to say */
   }
}
int Cards_On_Line () {           /* verify PMAC is there */
   int   i,n;
   able_to_talk = 1;
   n = i = 0;
   while (n < 3  && i < 11) {
      sendline ("RHL:$720") ;
      i = getline (buf) ;
      n++ ;
   }
   if (i > 11  &&  i < 17) {
      cprintf ("\r\nFound PMAC...\r\n") ;
      while (getline (buf));     /* clear out PMAC's output buffer */
      return (1);
   }
   else {
      sound (1000); delay (250); nosound ();
```

```
      cprintf ("\r\nNo PMAC found!\r\n") ;
      able_to_talk = 0;
      return (0);
   }
}
void send_command_to_PMAC () {   /* send calculated positions to PMAC */
   char  command [80];
   sprintf (command,"X%ld Y%ld Z%ld",x,y,z);
   sendline (command);           /* send previously calculated move */
   cprintf ("Sending X%ld Y%ld Z%ld\r",x,y,z);
   x+= 2000;                     /* calculate positions for next move! */
   y+= 2000;
   z+= 2000;
}
void main () {
   int   done = FALSE;
   char  str [256];
   config_card_for (528);           /* setup for PC-bus at 528 */
   Cards_On_Line ();                /* verify card it there */
   if (able_to_talk) {
      cprintf ("Now downloading positions to rotary buffer...\r\n\n");
      sendline ("Q I16=1 I17=2"); /* set rotary buffer request parameters */
      sendline ("I3=0");
      sendline ("#1hm#3hm#4hm");
      init_pmac_comm ();          /* setup interrupt vector for our routine
*/
      sendline ("DEFINE ROT 50");   /* allocate space for rotary buffer */
      sendline ("OPEN ROT");        /* open it...
      sendline ("F20000");
      sendline ("LINEAR");
      while (getline (buf));
      sendline ("R");               /* and start running it */
      while (!done) {
/*            Now process the interrupt(s) */
         if(breq_flag) {               /* if BREQ interrupt set, */
               breq_flag = 0;             /* clear the flag */
               send_command_to_PMAC ();   /* &download a calculated move to
                                             PMAC */
         }
/*   Background tasks can be inserted here! */
         if (kbhit ())
               if (getch () == 27)        /* pressing ESC aborts program */
                  done = TRUE;
      }
      sendline ("CLOSE Q");         /* close rotary buffer since we're done
*/
```

```
      sendline ("I3=1");
      restore_pmac_comm ();          /* restore original interrupt setup */
   }
   while (getline (buf));
}
```

PMACLERN.C

```
/*  Sample program to implement a learn mode in PMAC.  This will generate a motion program
(PROG 123) which contain a series of X moves which are created each time you press CTRL-B.

*/
#include <stdio.h>
#include <dos.h>
#include <conio.h>
#define  SERIAL   555
#define  PCBUS       123
#define  COM1        1016
#define  COM2        760
int combase,                  //  base address of the card
      baudrate,               //  baudrate for serial communication
      speed,                  //  used for calculating timeout
      able_to_talk,           //  flag for communications established
      timeout;                //  limits how long to look for handshake
char   buf [256];             //  holds input string
int config_card_for (int port,int address) {
   speed = 9;
   combase = address;
   if (port == SERIAL) {
      baudrate = 12;              //  select 38400 baud
      timeout = 7 * speed * (baudrate + 100);
      outportb (combase + 3, 131); //  set up the PC's serial port for communication
      outportb (combase, baudrate);
      outportb (combase + 1, baudrate/256);
      outportb (combase + 3, 3);
   }
   else {          /*    PC-Bus will be used for communication */
      baudrate = 0;/*   when baudrate is set to 0, PC-Bus communication is assumed */
      timeout = 7 * speed * 100;
      outportb (combase + 5,0) ;
      outportb (combase + 6,0) ;
   }
}
int sendline (char *outchar) {   /*  calls send char to send a string of characters to PMAC
                                     followed by a CR */
```

```
    int i = 0;
    while (outchar [i] != 0)
        sendchar (outchar [i++]);
    sendchar (13);
}
int sendchar (char outchar) {    //  sends a character to the PMAC-PC card
    int i;
    i = 0;
    if (baudrate == 0) {     /* Send data over PC-Bus interface */
                                /*  look for Write Ready bit (bit 1 of base address + 2) */
        while (i++ < timeout  &&  !(inportb (combase+2) & 2)) ;
        if (i < timeout)
            outportb (combase+7, outchar) ;
    }
    else {            /*  Send data over RS232 interface (serial) */
                        /*  look for Write Ready bit (bit 6 & 5 of base address + 5 & 6) */
        while (i++ < timeout && (inportb (combase+6) & 16) == 0);
        if (i < timeout)
            outportb (combase,outchar);
    }
    return (0);
}
int getline (char *linebuf) {
    char   ic;
    int i,nc;
    if (able_to_talk) {
        ic = nc = i = 0;
        if (baudrate == 0) {       /*   Read data over the PC-Bus interface */
            while (i++ < timeout && ic != 13 && nc < 255)
/* look for Read Ready bit (bit 0 of base address + 2) */
                if ((inportb (combase+2) & 1) == 1) {
/* found it.. read a character and add on to string */
                    ic = inportb (combase+7) ;
                    linebuf [nc++] = ic ;
                    i = 0 ;
                }
        }
        else {                          /*  Read data over the RS-232 interface */
            outportb (combase+4,2);      //  release "hold off" signal
            while (i++ < timeout && ic != 13 && nc < 255) {
                if ((inportb (combase+5) & 1) == 1) {
                    ic = inportb (combase);
                    enable ();
                    linebuf [nc++] = ic;
                    i = 0;
```

```
                    disable();
                    }
            }
            outportb (combase+4,0);      //  set the "hold off" signal
            enable ();
            while ((i += 2) < timeout);
        }
        linebuf [nc] = 0;
        return (nc);                     /*  Return number of characters received */
    }
    else {
        linebuf [0] = 0;
        return (0);                      /*  No characters received! */
    }
}
int Cards_On_Line () {                   /*  verify PMAC is there */
    int i,n;
    able_to_talk = 1;
    n = i = 0;
    while (n < 3  && i < 11) {
        if (baudrate != 0) {         /*  serial communications only */
            sendchar (24);               /*  send ^X */
            delay (20);
            sendchar (26);           /*  send ^Z  for serial mode */
        }
        sendline ("RHL:$720") ;
        i = getline (buf) ;
        n++ ;
    }
    if (i > 11  &&  i < 17) {
        cprintf ("Found PMAC...\r\n") ;
        while (getline (buf));           /*  clear out PMAC's output buffer */
        return (1);
    }
    else {
        sound (1000); delay (250); nosound ();
        cprintf ("No PMAC found!\r\n") ;
        able_to_talk = 0;
        return (0);
    }
}
void main () {
    int done = 0,learning = 1;
    long   position;
    char   ic,program_line [80];
```

```
    config_card_for (PCBUS,528);
//  config_card_for (SERIAL,COM1); //Use this line to talk over PC-BUS!
    Cards_On_Line ();
    if (able_to_talk) {
        cprintf ("Now acting as a terminal to PMAC.\r\n");
        cprintf ("Press <ESC> to abort.\r\n");
        cprintf ("\r\nNow creating program 123.\r\n");
        cprintf ("Press CTRL-B to learn current position of motor 1.\r\n");
        cprintf ("Press CTRL-D to see learned moves so far.\r\n");
        sendline ("OPEN PROG 123 CLEAR CLOSE");
        while (!done) {
            while (!kbhit()) {          //  no key pressed, so check card for data
                   getline (buf);
                   cprintf (buf);
            }
            switch (ic = getch ()) {
                   case 2:                  //  CTRL-B pressed, so add 'X' move to program
                       sendline ("#1p");    //  ask for position of motor 1
                       getline (buf);       //  get the position information...
                       sscanf (buf,"%ld",&position);
                       sprintf (program_line,"X%ld",position);
                       sendline ("OPEN PROG 123");
                       sendline (program_line);
                       sendline ("CLOSE");
                       while (getline (buf));
                       break;
                   case 4:                  //  CTRL-D pressed
                       cprintf ("Learned moves in PROG 123:\r");
                       sendline ("OPEN PROG 123 LIST CLOSE");
                       break;
                   case 27:                 //  ESC key pressed, so exit
                       done = 1; break;
                   case  0:                 //  swallow 2nd part of special keys (i.e. F-keys)
                       getch (); break;
                   default:
                       sendchar (ic);       //  send character to PMAC
                       putch (ic);
                       break;
            }
        }
    }
    else
        cprintf ("Can't talk to PMAC!\r\n");
}
```

PMACRAM.C

```
/*     PMAC interrupt driven program for PMAC using OPTION 2
Dual Ported RAM.  Make sure a jumper on E83 for IRQ 5 is installed to allow
PMAC to interrupt the PC at IRQ 5, and a jumper on E65 is installed to allow
EQU1 to interrupt PMAC's 8259 at IR5.
*/
/* ---------------- PMAC bus port addresses --------------- */
#define PMACBUFFERSIZE   1024          /*  this our queue buffer size */
#define PMAC             528        /*  bus address of PMAC */
#define PMAC_STATUS      PMAC+2     /*  bus status byte */
#define PMAC_DATA        PMAC+7     /*  transmit/receive data */
#define PMAC_PIC00       PMAC+8     /*  8259 interrupt controller */
#define PMAC_PIC01       PMAC+9     /*  8259 interrupt controller */
#define PMAC_PIC02       PMAC+10    /*  8259 interrupt controller */
#define PMAC_RECEIVE     2             /*  ready to receive bit */
#define PMAC_SEND        1             /*  ready to send bit */
#define PMAC_IS_SENDING (inp (PMAC_STATUS) & PMAC_SEND)
/* ------------ PMAC communication interrupt values ----------------*/
#define PMAC_IRQ      4          /*  0-7  IRQ0 - IRQ7 */
#define PMAC_MASK     (~(1<<PMAC_IRQ))  /*  for unmask of PMAC 8259 */
#define PMAC_EOI      0x20           /*  Pmac end of interrupt */
#define PMAC_IPOS     1          /*  IR0 for in position */
#define PMAC_BREQ     2          /*  IR1 for buffer request */
#define PMAC_ERROR    4             /*  IR2 for general error */
#define PMAC_FERROR   8             /*  IR3 for following error */
#define PMAC_HREQ     16         /*  IR4 for communication */
#define PMAC_EQU1     32         //  IR5 for DP RAM acknowledge
#define PMAC_NOP      0x40           /*  8259 no operation code */

/*     -------------- PC interrupt values -------------------     */
#define IRQ              5      /*  PC interrupt used for PMAC */
#define PC_INT        (8+IRQ)
#define PC_MASK       (~(1<<IRQ))       /*  for unmask of PC 8259 */
#define PC_PIC01      0x21           /*  8259 interrupt controller XT */
#define PC_PIC00      0x20           /*  " 0-7 interrupts  "    */
#define PC_EOI        0x20           /*  end of interrupt */
#define ACK           0x06
#define LF         0x0C
#define EOL           0x0D
#define PCBUS         123
#define TRUE          1
#define FALSE         0
#include <stdio.h>
#include <dos.h>
#include <conio.h>
#include <stdlib.h>
static void    (interrupt far *old_comm_int) ();
static void    interrupt far pmac_comm ();
static int combase,speed,able_to_talk,timeout,old_int_stat,equ1_flag;
static long far   *X,*Y,*Z;
static int far *M155,*M154;
static char    buf [256];
void init_pmac_comm () {   /*   initialize the PMAC port for interrupt comm. */

    disable ();                     /*  disable interrupts until we're done */
    old_comm_int = getvect (PC_INT);/*  save original interrupt vector */
```

```
    setvect (PC_INT, pmac_comm);    /*  write in our new interrupt vector! */

/* -------  setup PC's 8259...  -------- */
    old_int_stat = inp (PC_PIC01);
    outp (PC_PIC01,(old_int_stat & PC_MASK));

/* -------  now setup PMAC's 8259...  -------*/
    outp (PMAC, 0);          /*  flush PMAC interrupt */
    outp (PMAC_PIC00, 0x17); /*  0x17 = edge, 0x1f = level trigger (ICW1) */
    outp (PMAC_PIC01, 0x08); /*  vector for data bus               (ICW2) */
    outp (PMAC_PIC01, 0x03); /*  set up for 8086 mode              (ICW4) */
    outp (PMAC_PIC01, 0xDF); /*  mask out ferr,err,inpos,breq      (OCW1) */
    outp (PMAC, 0x81);       /*  DSP enable read */
    enable ();                  /*  enable interrupts because we're done! */
}
void restore_pmac_comm () {  /*  restore original interrupt vector */
    disable ();
    setvect (PC_INT, old_comm_int);     /*  replace old PC vectors */
    outp (PC_PIC01, old_int_stat);      /*  restore old status */
    outp (PMAC_PIC01, 0xff);            /*  turn off PMAC's 8259 */
    enable ();
}
static void interrupt far pmac_comm (void) {    /* PMAC interrupt service routine */
    int    ch;
    char   isr;
    disable();                              /*  don't be interrupted unitl done */
    outp(PMAC_PIC02,PMAC_NOP);          /*  rising edge of 1st INTA/ pulse */
    outp(PMAC_PIC00,PMAC_NOP);          /*  trailing edge of 1st INTA/ pulse */
    outp(PMAC_PIC02,0x0B);                 /*  setup to read ISR register */
    isr = inp (PMAC_PIC00);                /*  read ISR register */
    if (isr & PMAC_EQU1)                   /*  if EQU1 interrupt set, set a flag */
       equ1_flag = 1;
    outp(PMAC_PIC00,PMAC_NOP);          /*  trailing adge of 2nd INTA/ pulse */
    outp(PC_PIC00, PC_EOI);                /*  send end-of-intrpt to PC's 8259 */
    enable();                              /*  re-enable interrupts! */
}
void config_card_for (int address) {
    speed = 9;
    combase = address;
    timeout = 7 * speed * 100;
    outportb (combase + 5,0) ;
    outportb (combase + 6,0) ;
}
void sendchar (char outchar) {                 /*  sends a character to PMAC */
    int i = 0;
    while (i++ < timeout  &&  !(inportb(combase+2) & 2));
    if (i < timeout)
       outportb (combase+7, outchar) ;
}
void sendline (char *outchar) {                /*  sends a string with CR to PMAC */
    int i = 0;
    while (outchar [i] != 0)
       sendchar (outchar [i++]);
    sendchar(13);
}
int getline (char *linebuf) {              /*  This is a polled read of PMAC */
    char   ic;
    int i,nc;
```

```
    if (able_to_talk) {
        ic = nc = i = 0;
        while (i++ < timeout && ic != 13 && nc < 255)
            if ((inportb (combase+2) & 1) == 1) {
                ic = inportb (combase+7) ;
                linebuf [nc++] = ic ;
                i = 0 ;
            }
        linebuf[nc] = 0;
        return (nc);                /*  PMAC had something to say */
    }
    else {
        linebuf [0] = 0;
        return (0);                 /*  PMAC had nothing to say */
    }
}
int Cards_On_Line () {                  /*  verify PMAC is there */
    int i,n;
    able_to_talk = 1;
    n = i = 0;
    while (n < 3  && i < 11) {
        sendline ("RHL:$720") ;
        i = getline (buf) ;
        n++ ;
    }
    if (i > 11  &&  i < 17) {
        cprintf ("\r\nFound PMAC...\r\n") ;
        while (getline (buf));      /*  clear out PMAC's output buffer */
        return (1);
    }
    else {
        sound (1000); delay (250); nosound ();
        cprintf ("\r\nNo PMAC found!\r\n") ;
        able_to_talk = 0;
        return (0);
    }
}
void send_command_to_PMAC () {          /*  send calculated positions to PMAC */
    *X-= 100;                       /*  calculate new positions for next move! */
    *Y-= 100;
    *Z-= 100;
    *M155 = 1;                      /*  signal PMAC to read positions */
    cprintf ("Sending:  X%ld Y%ld Z%ld\r",*X,*Y,*Z);
}
void main () {
    int done = FALSE;
    char   str [256];
    config_card_for (528);          /*  setup for PC-bus at 528 */
    Cards_On_Line ();               /*  verify card is there */
    equ1_flag = 0;
    if (able_to_talk) {
        sendline ("WX:$786,$D,$43"); /*  configure DPRAM to start at $D4000 in PC */
        sendline ("save");
        sendline ("$$$");           /*  reinitialize the card to enable DPRAM */
        delay (1000);               /*  make sure I180,I280,I380, etc. are set to 1 */
        cprintf ("Now sending positions via dual ported RAM & interrupts...\r\n\n");
        sendline ("Q &1 #1->X");    /*  set up coordinate system */
        sendline ("#3->Y");
```

```
        sendline ("#4->Z");
        sendline ("#1hm #3hm #4hm"); /*  home the motors... */
        sendline ("i8=0");           /*  set real time int for every servo int */
        sendline ("M112->X:$C000,12,1");   /*   disable EQU1 output enable */
        sendline ("M113->X:$C000,13,1");          /*  point to EQU1 output invert enable */
        sendline ("M116->X:$C000,16,1");          /*  point to EQU1 compare equal flag */
        sendline ("M151->DP:$D201"); /*  used for motor X's commanded position */
        sendline ("M152->DP:$D202"); /*  used for motor Y's commanded position */
        sendline ("M153->DP:$D203"); /*  used for motor Z's commanded position */
        sendline ("M154->Y:$D200,0,1"); /* flag used to stop motion program */
        sendline ("M155->X:$D200,0,1"); /* handshake flag for PMAC to read pos */
        getline (buf);
        sendline ("OPEN PROG 10 CLEAR");
        sendline ("TA2");            /*  set acceleration time to 2 miliseconds(!) */
        sendline ("SPLINE1");        /*  set up to do splined moves */
        sendline ("X0Y0Z0");         /*     need to have 3 moves (2 done ahead of time) */
        sendline ("X0Y0Z0");         /*  to do 3rd order spline moves */
        sendline ("WHILE(M154=1)");  /*  loop until told to stop */
        sendline ("IF(M155=1)X(M151)Y(M152)Z(M153)M155=0M113=0M113=1");
        sendline ("ENDWHILE");                  /*  M155 is set to 1 by PC telling PMAC that
                                                    the newly calculated positions can
                                                    now be taken.  M113 set set to 0
                                                    and then to 1 to produce the inter-
                                                    rupt pulse which will interrupt the
                                                    PC, telling it to send down new po-
                                                    sitions! */
        sendline ("CLOSE");
        getline (buf);
        sendline ("M113=0");             /* clear IR5 interrupt input */
        sendline ("M151=0");             /* init motor X's position */
        sendline ("M152=0");             /* init motor Y's position */
        sendline ("M153=0");             /* init motor Z's position */
        sendline ("M154=1");             /* let motion program run */
        sendline ("M155=1");            /*  read first position immediatley */
        while (getline (buf));
        X = MK_FP (0xD400,0x0804);       /* make these pointer variables point to */
        Y = MK_FP (0xD400,0x0808);      /*  the proper locations in th DP RAM */
        Z = MK_FP (0xD400,0x080C);       /* X,Y, & Z are our position variables */
        M155 = MK_FP (0xD400,0x0802); /* out flag to tell PMAC to take the data */
        M154 = MK_FP (0xD400,0x0800); /* our flag to stop the program in PMAC */
        init_pmac_comm ();              /*  setup interrupt vector for our routine */
        sendline ("B10 R");                 /* and start running it */
        while (!done) {
/*            Now process the interrupt(s) */
           if (equ1_flag) {             /*  if EQU1 interrupt set, */
               equ1_flag = 0;      /*   clear the our flag and */
        send_command_to_PMAC (); //  download calculated move to PMAC
           }
/*   Background tasks can be inserted here!...   */
           if (kbhit ())                /*  ...like scanning the keyboard... */
           if (getch () == 27)   /*  pressing ESC aborts program */
                 done = TRUE;
        }
        *M154 = 0;                   /*  stop motion program */
        restore_pmac_comm ();        /*  restore original interrupt setup */
        sendline ("i8=2");           /*  set real time int for every 3rd servo int */
    }
}
```

7 PMAC PROM SOFTWARE UPDATE LISTING

Instructions to Change a PROM

The algorithms (firmware) for standard PMAC's and those with Option 5 are stored in a Programmable Read-Only Memory (PROM) IC. If you need to update to a newer version of the firmware, you can do so in the field by removing the old PROM chip (the one with the label) from its socket, and replacing it with the new PROM chip.
On PMAC boards with the new flash memory (Options 4A, 5A, and 5B), the firmware is stored in the flash memory. In this case updating to a newer version of the firmware does not involve replacing any chips. The update is accomplished by putting the PMAC board into "bootstrap mode" and loading the new firmware (software) from a host computer into the PMAC.

Backing Up Your Data

Before you change the PROM or update the firmware, make sure you have any configuration information that you wish to keep (I-variable values, M-variable definitions, coordinate-system definitions, programs) stored on disk in your host computer. The PMAC Executive Program has a *Save Full Configuration* utility for this purpose. Alternatively, portions of this information can be stored by either selecting the *Setup* button on this dialog box and choosing what information should be saved or by uploading them into the Exuctive Program editor, then saving the editor contents to disk.

Standard and Option 5 PMACs

In replacing the IC, some static protection is advisable, although chances are low of destroying the ICs. If you do not have a real grounding strap, at least touch some metal with a fingertip to draw off any stored charge before removing the new PROM from its package. With power off to the card, remove the old PROM and replace it with the new PROM, making sure that the notch is on the same end as before.

Reloading Data

Now restore power and communications to the card. If the change in firmware was small, you may be able to proceed immediately with your work at this point, with all variables and programs retained.
However, a more major change may result in the loss of your I-variable values and/or your stored programs. If this is the case, you will need to download your old configuration data that you have saved in your host computer. The *Restore Full Configuration* utility in the Executive program can be used for this purpose, as can the *Download Editor* and *Download File* utilities.
If you have problems getting going after changing the PROM, you may wish to command the card with a **$$$***** to reset and reinitialize the card, setting all I-variables to their factory defaults, and empty the buffers. If you cannot

talk to the card at all after the update, you may want to change the setting of jumper E51 to its non-default state (ON for PMAC-PC, -Lite, and -VME; OFF for PMAC-STD) to reinitialize the card when it is powered up.
Remember to **SAVE** your I-variable values once you have downloaded them to the PMAC.

PMACs with Options 4A, 5A, and 5B



For any change in the operational firmware, the compiled PLCs will have to be re-compiled with the ***LIST LINK*** *file for the new firmware version. It is important to delete all compiled PLCs (****DELETE PLCC n****) before attempting to change the operational firmware version. Compiled PLC programs running under a firmware version other than that which they were compiled for can have unpredictable consequences.*

If the jumper E51 is ON when a PMAC with the Option CPU executes its reset cycle, PMAC enters a special re-initialization mode that permits the downloading of new firmware. In this mode, the PMAC can communicate over the serial port only at a baudrate of 38,400, regardless of the setting of the baud rate jumpers. Bus communications is also possible on PMAC's with bootstrap version 1.01 and newer (most PMACs have one of these versions). To verify the bootstrap version you should type the command VER while communicating to PMAC in bootstrap mode.
In this bootstrap mode, there are very few command options. PMAC will respond to any of the status-bit query commands (**?**, **??**, or **???**) with the response *BOOTSTRAP PROM*. This permits the host to know whether PMAC is in this mode or not. PMAC will respond to the **VERSION** query command with the number of the bootstrap firmware (e.g. *1.01*) which will probably be different than the operational firmware version.
To bypass the download operation in this mode, send a **<CONTROL-R>** character to PMAC. This puts PMAC in the normal operational mode with the existing firmware. Factory default values for I-variables, conversion table settings, and bus addresses for DPRAM and VME are copied from the firmware section of flash memory into active memory. The saved values of these values are not used, but they are still kept in the user section of flash memory.

Downloading New Firmware

To download new operational firmware to the PMAC, send a **<CONTROL-O>** character to PMAC over the serial port. The bootstrap firmware interprets this as a signal to prepare for downloading of new operational firmware. All subsequent bytes received over the serial port will be considered as binary-coded bytes of machine-code firmware, and will be written into the flash memory.
The host computer should wait at least 5 seconds after the **<CONTROL-O>** command before starting to download the operational firmware. This delay ensures that the flash memory is ready to be written to. After downloading, the PMAC should be powered down; no other communications should be attempted with PMAC at this time.
After turning off power to PMAC, the E51 jumper should be removed. When power is re-applied to PMAC, it should operate normally with the new firmware. The user settings stored in other segments of the flash memory with the **SAVE** command are not affected by the downloading of new firmware (unless the new firmware has a different user memory map).
The PMAC Executive program V3.x and newer, when it establishes communications with a PMAC in this re-initialization mode, will automatically notice that PMAC is in this mode. In this mode, the menu selection "Download binary firmware file.." in the File menu can be selected to take a binary file from disk and copy over the serial port to PMAC. The program then forces you to exit to the operating system. At this point, you should turn off power to PMAC and remove the E51 jumper.
With older versions of the PMAC Executive program, or with a terminal emulator program running on a PC, the procedure for downloading new firmware is as follows (remember to back up your PMAC software and delete any compiled PLC programs first):

9. Establish communications to PMAC over the serial port at 38400 baud. Confirm that PMAC is in this re-initialization mode by seeing that it responds to the **?** command with *BOOTSTRAP PROM*. If you are in the Executive program, make sure that all windows other than the Terminal window (such as the position window) are closed, so no other commands are being sent to PMAC.

10. Type a **<CONTROL-O>** character in the terminal window and immediately exit to DOS. Do not send any other characters to PMAC here.

11. Use the binary version (/B) of the DOS `COPY` command to download the file containing the new firmware to PMAC. The command typed at the DOS prompt will look something like

```
COPY/B B:V115A.BIN COM1:
```

where `B:V115A.BIN` is the directory and name of the file containing the operational firmware in binary machine code format, and `COM1:` is the serial port being used for communications.

12. Shut off power to PMAC and remove the E51 jumper.

13. Restore power to PMAC and resume normal operation with the new firmware.

14. If you want to update your firmware checksum reference value so PMAC does not report a firmware checksum error, the easiest method is to send the **$$$***** command, which causes PMAC to compute the new reference value automatically (but it also clears all of your programs and buffers from active memory). As an alternative method, send the command **RHX:$0794** several times. If you get the same value each time, PMAC has stopped its checksum calculations on an error, and the reported value is the value it calculated for the firmware checksum. Write this value into reference register X:$07B1. For example, if **RHX:$0794** returns *9A3B12* several times, send the command **WX:$07B1,$9A3B12** to PMAC. Remember that for either method, you will need to store this reference value to flash memory with the **SAVE** command.

15. If you need to re-compile PLC programs, either use the LISTLINK.TXT file that corresponds to the new firmware version, or create this file by sending the **LIST LINK** command to the PMAC with the new firmware and storing the response in a text file of this name.

From V1.08 to V1.09 (2/13/91)

Additions

Encoder conversion table formats (feedback types)

$10 -- A/D register feedback.

$20 -- Parallel encoder Y-word source, no filtering.

$30 -- Parallel encoder Y-word source, with filtering.

$60 -- Parallel encoder X-word source, no filtering.

$70 -- Parallel encoder X-word source, with filtering.

I-variables

I45 -- permits data gathering into dual-ported RAM and gathering raparound.

Ix02 -- extended to permit magnitude and sign output (required for V/F conversion).

Ix03 -- extended to permit home capture from other than incremental encoder.

Ix05 -- extended to permit handwheel following that does not change reported position.

Ix25 -- extended to permit use of AENA/DIR line for direction bit, and to control FAULT input use, polarity, and effect.

Ix85 -- Motor x Backlash size (1/16 count).

Ix86 -- Motor x Backlash takeup rate (1/16 count/msec).

Ix96 -- C.S. x Circle Error Limit (user units).

M-variables

DP -- 32-bit signed integer format for dual-ported RAM.

F -- 32-bit IEEE floating-point format for dual-ported RAM.

On-line commands

<CTRL-Y> -- Reloads last command line ready to execute on a <CR>; also sends same line to host (w/o <CR>).

{constant} -- Equivalent to "P0={constant}".

Motion buffer commands

RAPID -- Move mode command so all motors in coordinate system move to destination at maximum velocity and acceleration (as set by Ix16 and Ix17). This is the equivalent of RS-274 G00.

R{data} -- Radius specification for circular moves.

From V1.09 to V1.11 (7/30/91)

Changes



V1.10 was never officially

Jumpers

E49 now selects serial communications parity (no change is required with no

released, only beta-site sampled. V1.11 is a 1Mbit EPROM that fills the entire socket. It should be oriented the same as before (notch -- pin 1 end -- toward the center of the board).
This update requires a reloading of I-variable values. It also requires setting new values for Ix04 and Ix09 for all motors (set Ix04 equal to Ix03, and Ix09 equal to Ix08 for backward compatibility). For PMAC-commutated motors new values must be set for Ix02 and Ix83 as well.

parity).

Memory map

$0000-$01FF substantially changed (important only for those gathering or using M-variables in this area).

I-Variables

Ix02 (DAC address) now must be specified for PMAC -commutated motors as well as non-PMAC-commutated.

Ix04 ("velocity" feedback address) must be specified (equal to Ix03 for backward compatibility).

Ix09 ("velocity" scale factor) must be specified (equal to Ix08 for backward compatibility).

Ix83 (commutation feedback address) must be specified {for PMAC-commutated motors only}.

Ix85,Ix86 backlash parameters are correctly located (were accidently at Ix61, Ix62 in V1.09).

Additions

I-Variables

I4 -- Communications Checksum Enable.

I14 -- Coord. Sys. Align on Program Start Enable.

Dual (motor/load) feedback -- requiring;

Ix04, Ix09 -- See under changes, above.

Ix25 bit 17 -- allows disabling of position limits.

Ix27 -- Motor x position rollover range.

Notch Filter -- requiring;

Ix36-Ix39 -- Motor x notch filter gains.

Ix60 -- Motor x Servo Cycle Extension.

Ix83 -- See under changes, above.

On-line commands

<CTRL-B> -- Report all motor status words.

<CTRL-C> -- Report all coordinate system status words.

<CTRL-G> -- Report global status words.

<CTRL-W> -- Execute ASCII command from DPRAM buffer.

PMATCH -- Re-matches Coord. Sys. axis positions to motor commanded positions (used in case axis definition or motor position changed since last axis move); not valid if axis-motor cross-coupling.

{variable}={expression} -- provides on-line calculation.

DEFINE COMP {constant} -- establishes leadscrew compensation table of specified size for addressed motor.

Data Integrity Checks

Serial Parity Check -- (Optional -- See E49).

Communications Line -- Checksum (Optional -- See I4).

Firmware Checksum -- (Automatic in background).

User Program Checksum -- (Automatic in background).

Commutation Features

Direct microstepping commutation.

From V1.11 to V1.12 (4/13/92)

Changes



In updating from V1.11 to V1.12, those parameters stored in the EAROM (I-variables, conversion table, VME addressing, DPRAM addressing) must be downloaded again from the host, then SAVEd. Programs and axis definitions should be retained, but it is strongly advised to do a full backup using the Executive Program beforehand.

Leadscrew Compensation Table

Leadscrew compensation table values must be entered as integer values representing 1/16 count; span of table must be entered as argument in **DEFINE COMP** command; compensation now performed inside servo loop (regardless of value of I13).

Direct Microstepping Commutation

Ix30 -- (Prop Gain) should be set to 8192.

Ix69 -- (DAC limit) should be set to 32512.

Ix78 -- (Slip gain) should be set to 4,194,304/N, where N is the number of phasing cycles per servo cycle as set by E3-E6 (default N=4).

User-Written Servo

On entry, user routine will find necessary data as follows:

Desired position in DSP register B10;

Desired velocity in A10;

Actual position in X;

Position scale factor in Y1;

On exit, user should leave filter output in A1, in units of 1/64 DAC bit (i.e. A1 should hold DAC output * 64).

Fixes

Fixed on-the-fly **TA** changes in move-segmentation mode (I13>0) Machine Output 1 no longer cleared on a **SAVE** command.

Additions

Operators

% -- Modulo operator extended to permit modulo by negative number.

X%-N -- returns a value from -N to +N-epsilon.

On-line commands

PMATCH -- extended to work for cross-coupled axes (i.e. rotated coordinate systems).

HOMEZ -- (short form **HMZ**) -- Do a "no-movement" homing n the addressed motor make current actual position the commanded position and call that motor position zero.

ZZ -- Do a **HOMEZ** on all motors in the addressed coordinate system.

J{constant} -- Jog addressed motor specified distance from current commanded position.

DEFINE COMP -- extended for cross-axis compensation.

Motion Programs

CC0 -- Turn off cutter radius compensation.

CC1 -- Turn on cutter radius compensation left.

CC2 -- Turn on cutter radius compensation right.

CCR{data} -- Defines cutter compensation radius.

HOMEZ{motor(s)} -- Do a "no-movement" homing on the specified motor(s) make current actual position the commanded position and call that motor position zero.

SEND^{letter} -- Send specified control character to host.

I-Variables

I1 -- Serial Handshake Line Disable. Permits PMAC to send serial data without CTS/CS handshake.

I6 -- Error Reporting Mode. Permits PMAC to report number of error

I50 -- **RAPID** mode control (for all motors and C.S.s);

=0 use jog speed parameter. (Ix22).

=1 use max speed parameter (Ix16).

I51 -- Leadscrew Compensation Enable.

Motor Status Bits

Amp Fault Error -- 2nd motor status word, bit 3.

Home Complete -- 2nd motor status word, bit 10.

Stop on Position Limit -- 2nd motor status word, bit 11.

Amplifier Disabled -- 2nd motor status word, bit 14.

Coordinate System Status Bits

Cutter Compensation On -- 2nd C.S. status word, bit 2.

Cutter Compensation Left -- 2nd C.S. status word, bit 3.

Pre-Jog Move Flag -- 2nd C.S. status word, bit 8.

Cutter Comp Move Buffered -- 2nd C.S. status word, bit 9.

Cutter Comp Move Stop Request -- 2nd C.S. status word, bit 10.

Cutter Comp Outside Corner -- 2nd C.S. status word, bit 11.

Amp Fault Error (OR of motors) -- 2nd C.S. status word, bit 20.

Circle Radius Error -- 2nd C.S. status word, bit 21.

From V1.12 to V1.13 (10/21/92)



In updating from V1.12 to V1.13, it should be possible simply to turn off power to the card, remove the V1.12 PROM from its socket, install the V1.13 PROM in its place, turn on power, and continue with all parameters and programs in place. However, it is strongly recommended that you do a full backup of the parameters and programs using the PMAC Executive Program before this.

In updating from V1.11 or older to V1.13, those parameters stored in the EAROM (I-variables, conversion table, VME addressing, DPRAM addressing) will revert to factory default values when power is turned on. Their previous values must be downloaded again from the host, then SAVEd. Programs and axis definitions should be retained, but, again, it is strongly advised to do a full backup using the Executive Program beforehand.

Additions

On-Line Command

@{constant} -- Address selected card on serial daisychain.

@ -- Report addressed card on serial daisychain.

DEFINE TBUF{constant} -- Create buffer for axis transformation matrices.

DELETE TBUF -- Erase axis transformation matrices.

M{constant}->TWS{address} -- Define M-variable for serial I/O board (ACC-34).

Motion Program commands

TSEL{constant} -- Select axis transformation matrix.

ADIS -- Absolute displacement of axes with selected transformation matrix.

AROT -- Absolute rotation of axes with selected transformation matrix.

IDIS -- Incremental displacement of axes with selected transformation matrix.

IROT -- Incremental rotation of axis with selected transformation matrix.

M{constant}=={expression} -- Synchronous M-variable value assignment.

M{constant}&={expression} -- Synchronous M-variable "AND-equals" assignment.

M{constant}|={expression} -- Synchronous M-variable "OR-equals" assignment.

M{constant}^={expression} -- Synchronous M-variable "XOR-equals" assignment.

I-Variables

I1 -- Serial Port Communications Mode extended to permit daisychained serial communications.

Update Summary: From V1.13 To V1.14 (July 93)

Changes

In updating from an older PROM version to V1.14, it will be necessary to re-initialize the card, then to reload all variable values and definitions, plus programs and other buffers. Without re-initializing, you will have a random unknown password, so will be unable to open or list the protected programs. Before you change the PROM, you should do a backup with the "Save Full PMAC Configuration" option in the Executive program. After powering down, changing the PROM, then powering up again, re-establish communications through the Executive program. Issue the ***$$$****** *command to re-initialize the card by typing in the terminal window. Then select the "Load Full PMAC Configuration" option to restore your variables and programs. Finally, type the* ***SAVE*** *command to copy the I-variable values to non-volatile EAROM.*

Rotary Buffer Blend On-The-Fly

If the rotary buffer is executing the last move in the buffer, and a new move is entered into the buffer before the start of deceleration of the current move, PMAC will blend into the new move without stopping (valid only for I13>0).

Abort Command

When in feed hold mode now causes no movement.

Power-on/reset position

When using any type of absolute sensor for servo position feedback, whether analog input, parallel digital input, or MLDT input, Ix10 must be used to establish the power-on/reset position. If Ix10 = 0, PMAC will treat the power-on/reset position as zero.

Long PLC 0 Programs

Those that consistently take more than one real-time interrupt period to execute -- which formerly only intermittently tripped the watchdog timer, now will trip the timer quickly.

Block Execution Rate

Motion program block execution rate improved. Benchmarks for V1.14 SPLINE1 mode are given below:

```
Test #1: 3 active motors; 2-axis SPLINE1
moves.

Test #2: 4 active motors; 3-axis SPLINE1
moves.

Test #3: 4 active motors; 4-axis SPLINE1
moves.
```

CPU SPEED	20MHZ	20MHZ	30MHZ
Wait States	1	0 (Opt 4)	0 (Opt 5)
Test #1	1600 b/s	2259 b/s*	3388 b/s*
Test #2	1100 b/s	1500 b/s	2250 b/s
Test #3	880 b/s	1250 b/s	1880 b/s

b/s = blocks per second.
* - one block every servo cycle.

Additions

On-line commands

<CTRL-N> -- Report checksum of current command line.

/ -- Halt program execution at end of currently *executing* move (I13>0 only).

**** -- Feed hold in manner permitting jogging while in hold (I13>0 only).

DEFINE COMP (two-dimensional) -- 2D compensation table.

LIST PE -- Report program line of currently *executing* move (I13>0 only).

M{constant}->TWR:{address},{offset} -- Assign M-variable to R/D.

converter on thumbwheel multiplexer port.

PASSWORD={string} -- Enter/set password.

PE -- Report program address of currently *executing* move (I13>0 only).

SEND"{message}" -- This command has been extended to permit **{message}.** to contain characters with ASCII values > 127 (non-English characters).

Motion Program commands

PMATCH -- This existing on-line command can now be executed from within a motion program with **CMD"PMATCH"**, provided that there are **DWELL** commands on both sides of it. This is useful for bringing an axis into a coordinate system in the middle of a program (e.g. making a spindle axis a rotary positioning axis).

S{data} -- Intended as spindle data command -- puts **{data}** in Q127.

I-Variables

I2 -- Control Panel Disable: Range extended; value of 3 disables JPAN port automatic functions, enables DPRAM control panel functions.

I48 -- DPRAM Servo Data Enable.

I49 -- DPRAM Background Data Enable.

I52 -- '\' Feed Hold Slew Rate.

I53 -- Program Step Mode Control (single-line or single-move).

I56 -- DPRAM ASCII Interrupt Enable.

I57 -- DPRAM Binary Rotary Buffer Enable.

I58 -- DPRAM ASCII Communications Enable.

I59 -- DPRAM Buffer Maximum Motor Number.

I8x -- Motor x Third-Resolver Gear Ratio.

I9x -- Motor x Second Resolver Gear Ratio.

Ix10 -- Motor x Power-on Position Address.

Ix60 -- Motor x Servo Cycle Period Extension (extended to support all non-negative integers).

Ix75 -- Motor x Phase Position Offset.

Ix81 -- Motor x Power-on Phase Position Address.

Conversion Table Entries

$50 -- Integrated analog with bias.

$28, $38, $68, $78 -- Unshifted parallel feedback (no fractional bits).

$90, $A0, $B0 -- Triggered time-base conversion.

Registers

Motor Encoder Position Offset Registers -- Y:$0815 (#1), :$08D5 (#2), etc. Difference in counts between encoder zero and motor zero (home). See *Storing the Home Position* under *Basic Motor Moves.*

Timer Registers -- X:$0700, Y:$0700, X:$0701, Y:$0701 User-writeable; decrement once per servo cycle. See *Timers* under *Computational Features.*

Dual Ported RAM Functions (Option 2 required)

1. DPRAM Control Panel Function.
2. DPRAM Servo Data Function.
3. DPRAM Background Fixed Data Reporting Function.
4. DPRAM Background Variable Data Reporting Function.
5. DPRAM ASCII Communications.
6. DPRAM Binary Rotary Program Buffer.

Removals

On-line commands



DELETE TRACE is still a valid command, even though it does not perform any action.

DEFINE TRACE

TRACE

ENDTRACE

Update Summary: From V1.14 To V1.15 (April 94)

In upgrading from some of the V1.15T test PROM versions, it may be necessary to put the E51 jumper on the first time power is applied with the released V1.15 PROM.

Changes



In updating from an older PROM version to V1.15, it is necessary to re-initialize the card, then reload all variable values and definitions, plus programs and other buffers. Before you change the PROM, you should do a backup with the "Save Full PMAC Configuration" option in the Executive program. After powering down, changing the PROM, then powering up again, re-establish communications through the Executive program. Issue the ***$$$****** *command to re-initialize the card by typing in the terminal window. Then select the "Load Full PMAC Configuration" option to restore your variables and programs. Finally, type the* ***SAVE*** *command to copy the I-variable values to non-volatile EAROM, or all settings to flash ROM*

User Buffer Space

8K 48-bit words of user buffer space now reserved for compiled PLCs, reducing space available for other programs and buffers from 43K words to 35K words.

User-Written Servo

The memory location of the user-written servo has changed due to the space reserved for the compiled PLCs. The starting address is now P:$B800, not P:$9C00. User-written servos will have to be re-assembled using the command `ORG P:$B800` to replace the existing `ORG P:$9C00`.

Ix07, Ix08, Ix09

Now reported as signed 24-bit values, not unsigned (action has always been as signed variable).

Ix25

Setting bit 17 to 1 now disables only the hardware overtravel limits, not the software overtravel limits (these can be disabled by setting Ix13 and Ix14 to 0).

Open Buffers

Any open buffers automatically closed on power-on/reset.

Partial Commands

Any partial commands in the command queue automatically erased on power-on/reset.

Additions

Flash memory support

On boards with the new flash memory backup (Options 4A, 5A, and 5B), programs, buffers, and definitions are copied from flash memory to RAM on power-on/reset. For these boards, after loading programs, buffers, and/or definitions, the **SAVE** command must be used to store these in flash memory if it is desired to retain them through power-down or reset.

On-line commands

<CTRL-L> -- Equivalent to **CLOSE** (quick form of command).

<CTRL-U> -- Equivalent to **OPEN ROTARY** (quick form of command).

DEFINE UBUFFER {size} -- Reserve memory space of specified size for user discretionary use.

DELETE PLCC{constant} -- Erase specified compiled PLC program.

DISABLE PLCC... -- Disable specified compiled PLC program(s).

DISPLAY"{message}" -- This command has been extended to permit **{message}** to contain characters with ASCII values > 127 (non-English characters).

ENABLE PLCC... -- Enable specified compiled PLC program(s).

J! -- Adjust command position of addressed motor to nearest integer count.

LIST LINK -- Causes PMAC to report addresses of internal routines for linking with PLC cross-compiler code.

Error Reporting

ERR003 now reported after **CLOSE** command when an **IF** statement does not have a matching **ENDIF** statement, or a **WHILE** statement does not have a matching **ENDWHILE** statement.

Motion Program commands

DISABLE PLCC... -- Disable specified compiled PLC program(s).

ENABLE PLCC... -- Enable specified compiled PLC program(s).

IF{condition}{action} -- This single-line condition now permitted in the rotary buffer.

WHILE{condition}{action} -- This single-line loop now permitted in the rotary buffer.

PLC Program commands

DISABLE PLCC... -- Disable specified compiled PLC program(s).

ENABLE PLCC... -- Enable specified compiled PLC program(s).

I-Variables

I7 -- In-Position Number of Consecutive Cycles (Minus One).

I60 -- Auto-converted ACC-36 ADC Address.

I61 -- Number of Auto-converted ADC Pairs (Minus One).

Ix10 -- Motor x Absolute Power-On Position Address extended to support power-on read of Yaskawa absolute encoders through ACC-8D Option 9.

Ix68 -- Direction-Sensitive Output Bias.

Conversion Table Entries

$D0 -- Exponential filter.

Registers

Auto-converted ACC-36 ADC Registers -- Y:$0708, X:$0708 to Y:$070F, X:$070F; if enabled with I60 and I61, these registers in RAM will contain converted analog values from the ACC-36 at the specified address, updated every (I61+1) phase cycles.

Averaged actual motor velocity registers -- Y:$082A, Y:$08EA, etc. These registers contain the actual velocities averaged over the previous 80 real-time interrupt periods (80*[I8+1] servo cycles); good for display purposes. The units are 1/(Ix09*32) counts per servo cycle.

Dual Ported RAM Functions (Option 2 required)

1. DPRAM ASCII Communications enhancements:
2. Permits I3 setting of 0 as well as 2.
3. Protects against overflow of PMAC's internal rotary program buffer.
4. Supports command lines up to 200 characters long.
5. Permits quicker sending of control-character commands.

Update Summary: From V1.15 to V1.16 (July 96)

Changes

In updating from an older PROM version to V1.16, it will be necessary to re-initialize the card, then to reload all variable values and definitions, plus programs and other buffers. Before you change the PROM, you should do a backup with the "Save Full PMAC Configuration" option in the Executive program. After powering down, changing the PROM, then powering up again, re-establish communications through the Executive program. Issue the ***$$$****** *command to re-initialize the card by typing in the terminal window. Then select the "Load Full PMAC Configuration" option to restore your variables and programs. Finally, type the* ***SAVE*** *command to copy the I-variable values to non-volatile EAROM, or all settings to flash ROM.*

I99 (backlash hysteresis)

With I99=0 backlash hysteresis is 0 counts, not 4 counts.

I89

With I89=0 cutter comp outside corners only add arc if change in directed angle is greater than 90°; formerly 1°.

I11

Default changed from 10 to 0.

I52

Default changed from 0 to 37137 (1 second slew by default).

Full Circles

Full circles executed on any arc command (with IJK center specification) smaller than 2^{-20} part of circle (0.5 arc second); formerly 2^{-34} part of circle.

Cutter Compensation

Boundary of cutter compensation lead-in and lead-out moves on inside corners changed slightly; now offset directly from uncompensated destination point perpendicular to fully compensated move only.

Additions

Features

User-written phase commutation routines (Ix59).

“Stepper-motor” phasing search method for synchronous motors (Ix80).

Hall-Effect power-on phase read (Ix81).

Integrated current (I^2T) limiting protection (Ix57, Ix58).

Integrated position error limiting protection (Ix63).

Programmable backlash hysteresis (I99).

Backlash compensation tables (DEFINE BLCOMP).

Torque compensation tables (DEFINE TCOMP).

MACRO ring support enhancements.

Teach/Learn functionality (LEARN).

Non-uniform spline mode (SPLINE2).

Variable jog commands (J=*, J:*, J^*).

Jog-until-trigger commands ({jog}^{constant}).

Program move-until-trigger commands ({axis}{data}^{data}).

"Torque-limiting" triggering (trigger on warning following error) (Ix03).

Extended G, M, T, and D code ranges.

Better control of outside corners with cutter compensation (I89).

PRELUDE functions in motion programs.

Ability to suppress carriage return after SEND and CMD responses (I62).

Unsigned A/D feedback ($18, $58 conversions).

Parallel feedback in high 16 bits ($2C, $3C, $6C, $7C conversions).

Support for ladder-logic compiler in special firmware version.

Variables

I55 -- DPRAM Background Variable Buffer Control (V1.15G).

I62 -- Internal Message Carriage Return Control.

I89 -- Cutter Comp Outside Angle Break Point.

I99 -- Backlash Hysteresis.

Ix02 -- Range extended: bit 19=1 specifies X-register output.

Ix25 -- Range extended: bit 18=1 specifies MACRO node for flags.

Ix30 -- Range extended: negative values can be used to invert output polarity.

Ix57 -- Continuous Current Limit for I^2T current fault.

Ix58 -- Integrated Current Fault Level for I^2T current fault.

Ix59 -- Range extended to 0-3; values of 2&3 enable user-written commutation.

Ix63 -- Range extended; negative values permit integrated following error fault.

Ix80 -- Range extended to support "stepper" power-on phasing search.

Ix81 -- Range extended to support hall-effect phase read.

Ix83 -- Range extended: bit 19=1 specifies Y-register feedback.

I1000 -- MACRO node auxiliary register enable.

I1001 -- MACRO Ring Check Time Period.

TWS-format M-variables -- Extended to add parity bits in backward-compatible format; bit 6 of global status word X:$0003 is set to 1 if the most recent TWS read or write operation resulted in a parity error.

Conversion table entries

$2C, $3C, $6C, $7C -- parallel feedback shifted right 3 bits (meant for feedback appearing in high 16 bits); supports rollover.

$18, $58 -- unsigned 16-bit A/D feedback, no rollover of source; to support ACC-28B.

On-line commands

DEFINE BLCOMP -- Establishes backlash table for addressed motor.

DEFINE TCOMP -- Establishes torque compensation table for addressed Motor.

DELETE BLCOMP -- Erases backlash table for addressed motor.

DELETE TCOMP -- Erases torque compensation table for addressed motor.

J=* -- Variable jog-to-position; destination in L:$082B, etc. (V1.15E).

J:* -- Variable incremental jog; distance in L:$082B, etc. (V1.15E).

J^* -- Variable incremental jog, distance in L:$082B, etc. (V1.15E).

J=={constant} -- Jog to specified position, make that "pre-jog" position.

{jog command}^{constant} -- Jog until trigger, final value specifies distance from trigger position to stop.

LEARN -- Reads present commanded positions for all motors in coordinate system, converts to axis positions, adds axis commands to open motion program.

MFLUSH -- Clears synchronous M-variable stack without executing.

LIST LDS -- PMAC reports addresses of special compiled PLC ladder-logic routines for cross-compiler (special firmware required).

TYPE -- PMAC reports hardware and software configuration.

Motion program commands

{axis}{data}^{data} -- Move-until-trigger for RAPID mode moves.

SPLINE2 -- Puts program in non-uniform B-spline mode.

G, M, T, D codes -- Extended to support range of 0-999.99999; formerly 0-99.99999.

PRELUDE1{command} -- Adds **{command}** before subsequent move commands.

PRELUDE0 -- turns off prelude function.

DPRAM Structures

Second binary rotary program download buffer added (V1.15G).

Refinements

Full circle moves always executed after starting position recalculated due to PMATCH or axis matrix transformation (more tolerance between start and end positions permitted; see Changes, above).

DISPLAY of I-variables fixed.

Fast versions of PMAC can interface to ACC-8D Opt 9.

Fast versions of PMAC can interface to NC control panel and ACC-34C.

O-commands now saturate at +/-100% of Ix69.

I52 default set to 37137 for default 1 second stop on '\' program hold command.

I11 default set to 0 so move sequences start as soon as calculations are finished.

Ix77, Ix86 reported as signed values.

RAPID declaration alone on a motion program line no longer causes zero-distance move taking 2*TA time.

Ix10 absolute servo-position read copies axis definition offset into position bias register.

Maximum commanded speed increased by factor of 3 to 768M/Ix08 cts/sec (8M cts/sec at default Ix08 value).

Cutter comp lead-in move destination, lead-out move origin, changed slightly.

Ix30 now can take negative values, permitting output polarity reversal.

(PMAC2) ADC1A,B to ADC8A,B registers in DSPGATE1 copied into RAM registers $0710 - $0717 one pair per phase cycle for reliable servo & program access.

$$$*** command kills all motors before re-initialization.

Index

'

'\' Program Hold Slew Rate (I52) 2-26

#

(Report Addressed Motor) 1-2, 3-18
#{constant} (Set Addressed Motor) 3-18
#{constant}-> (Report Axis Definition) 3-19
#{constant}->{axis definition}(Set Axis Definition) 3-20, 3-21
#{constant}->0 (Delete Axis Definition) 3-20

$

$ (Motor Reset) 3-23
$$$ (Global Reset) 3-23
$$$*** (Global Reset and Reinitialize) 3-24, 3-25

%

% (Report Feedrate Override) 3-25, 3-26
%{constant} (Set Feedrate Override) 3-27

&

& (Report Coordinate System) 1-2, 3-29
&{constant} (Address Coordinate System) 3-28

.

Decode Control "Encoder I-Variable 0" for Encoder n (I900; I905 1-17
Filter Disable "Encoder I-Variable 1" for Encoder n (I901; I906 1-17
Position Capture Control "Encoder I-Variable 2" for Encoder n (I902; I907 1-17
Flag Select Control "Encoder I-Variable 3" for Encoder n (I903; I908 1-17
Reserved for future use (I0; I904; I909 1-17

/

/ (Halt program execution at end of currently executing move) 3-29
/ (hold permitting jogging) 3-43

?

?(Report Motor Status) 1-7, 3-30
?? (Report Coordinate System Status) 3-34
??? (Report Global Status) 3-38

@

@ (Report Addressed PMAC Board) 1-2, 3-41
@{card} (Set Addressed PMAC Board) 3-42

{

{axis}{data}^{data}[{axis}{data}^{data}...] 4-3
{axis}={constant} (Re-define Axis Location) 3-46
{constant} (Assign to P0) 1-4, 3-49
{jog command}^{constant} (Jog Until Trigger) . 3-90

A

A (Abort) 3-44
A{data} (A-axis Move) 4-7
A-axis Move 4-7
Abort all programs and moves 3-1
ABS (Absolute position mode) 3-45, 4-7
ABS Function 1-2, 3-45
Acceleration
  Default Program S-Curve Time for Coordinate System x (Ix88) 2-111
  Default Program Time for Coordinate System x (Ix87) 2-110
  Jogging and Homing Motor x (Ix20) 2-57
  Jogging and Homing S-Curve Time for Motor x (Ix21) 2-58
  Maximum Permitted for Motor x (Ix17) 2-54
  Maximum Permitted for Motor x (Ix19) 2-57
  Parameters
    Set S-curve Time 4-84
    TA{data} 4-81
Acceleration Feedforward Gain for Motor x (Ix35) 1-15, 2-72
ACOS Function 1-2
Activate Variable for Motor x (Ix00) 1-13, 2-9, 2-38
ADDRESS (Address Command) 4-8
Address Coordinate System 3-28
Address of Pointer for Control-W Command Variable (I47) 1-13, 2-24
Addressed Motor 3-18
Addressed PMAC Board
  Report 3-41
ADIS{constant} (Absolute Axis Displacement) ... 4-9
Adjust motor commanded position to nearest integer count 3-80
Analysis Features
  Data Gathering
    DEFINE GATHER 3-57
    DELETE GATHER 3-63
    END GATHER 3-71
    GATHER 3-73
AROT{constant} (Absolute Axis Rotation) 4-11
ASIN Function 1-2
Assign to P0 3-49
ATAN Function 1-2

ATAN2 Function 1-2
Auto Position Match on Run Enable Variable (I14) 1-13, 7-5
Auto-Converted ADC Register Address (I60) 2-31
Axis Definition
  Report 3-19
Axis Definition Statements 3-21
Axis Move
  A-axis 4-7
  B-axis 4-11
  C-axis 4-14
Axis Position Scaling
  Delete Axis Definition 3-20
  Report Axis Definition 3-19
  Set Axis Definition 3-20
Axis Transformation Matrices
  Absolute Axis Displacement 4-9
  Absolute Axis Rotation 4-11
  Define Axis Transformation Buffer 3-59
  Delete Axis Transformation Buffer 3-65
  Incremental Axis Displacement 4-45
  Incremental Axis Rotation 4-49
  TINIT 4-82
  TSELECT{constant} 4-84

B

B{constant} (Point to program) 3-47
B{data} (B-axis Move) 4-11
Backlash Hysteresis (I99) 2-36
Backlash Size for Motor x (Ix86) 1-16, 7-4
Backlash Takeup Rate for Motor x (Ix85) 1-16, 7-4
B-axis Move 4-11
Begin execution of motion programs in all coordinate systems 3-11
BLOCKSTOP 4-13

C

C{data} (C-axis Move) 4-14
Calculating Ahead 4-30
CALL (Call Subprogram) 4-14
Cancel in-process communications 3-16
C-axis Move 4-14
CC0 (Cutter Compensation Off) 1-9, 4-15, 7-7
CC1 (Cutter Compensation On Left) 1-9, 4-16, 7-7
CC2 (Cutter Compensation On Right) 1-9, 4-17, 7-7
CCR{data} (Set Cutter Compensation Radius) 1-9, 4-18, 7-7
Checksum 1-13, 2-7, 7-5
  command line 3-9
Circle CCW 4-20
Circle Center Vector
  I{data} 4-43
  J{data} 4-50
  K{data} 4-50
Circle CW 4-18
Circle Error Limit for Coordinate System x (Ix96) . 1-16, 7-4
Circle Modes
  CIRCLE1 (Circle CW) 4-18
  CIRCLE2 (Circle CCW) 4-20
CIRCLE1 (Circle CW) 4-18
CIRCLE2 (Circle CCW) 3-37, 4-20
Circular Arc Move Specification 4-5
Circular Blended Moves
  Center Vector
    I{data} 4-43
    J{data} 4-50
    K{data} 4-50
  Circle Modes
    CIRCLE1 (Circle CW) 4-18
    CIRCLE2 (Circle CCW) 4-20
  Circular Arc Move Specification 4-5
  Radius Size Specification
    Set Circle Radius 4-69
  Specifying the Interpolation Plane
    NORMAL 4-59
CLEAR (Clear Buffer) 2-9, 3-48
Clear All Coordinate System Definitions 3-139
Clear Buffer 3-48
Clear Coordinate System Definition 3-138
Clear pending synchronous M-Variable assignments 3-120
CLOSE (Close Buffer) 2-9, 3-48
Close Buffer 3-48
Close open rotary buffer 3-7
Command Output (DAC) Address for Motor x (Ix02) . 1-13, 7-4
COMMAND"{command}" 4-21
COMMAND^{letter} 4-23
Commands
  Notes 1-1
Checksum 1-13, 2-7, 7-5
Communications
  Active Response Port
    Set to serial port mode 3-17
Communications Integrity Mode Variable (I4) . 1-13, 2-6, 2-7, 7-5
Commutation Cycles
  Encoder Counts per (Ix71) 2-87
  Number of (Ix70) 2-86
Commutation I-Variables 2-86
Commutation Phase Angle for Motor x (Ix72) ... 1-15, 2-88
Compiled PLC Programs
  DELETE PLCC 3-64
  Disable 4-26
  Enable 4-34
Control Panel Disable Variable (I2) 1-13, 2-3
CONTROL-A Command (Abort all programs and moves) 3-1
CONTROL-B Command (Report status word for all motors) 3-2
CONTROL-C Command (Report all coordinate system status words) 3-3

Control-Character Commands 2-7
CONTROL-D Command (Disable all PLC programs) 3-3
CONTROL-E Command (Report configured address contents in binary) 3-4
CONTROL-F Command (Report following errors for all motors) 3-5
CONTROL-G Command (Report global status word) 3-5
CONTROL-H Command (Erase last character) 3-6
CONTROL-I Command (Repeat last command line) 3-6
CONTROL-K Command (Kill all motors) 3-7
CONTROL-L Command (Close open rotary buffer) 3-7
CONTROL-M Command (Enter command line) .. 3-9
CONTROL-N Command (Report command line checksum) 3-9
CONTROL-O Command (Feedhold on all coordinate systems) 3-10
CONTROL-P Command (Report positions of all motors) 3-10
CONTROL-Q Command (Quit all executing motion programs) 3-11
CONTROL-R Command (Begin execution of motion programs in all coordinate systems) 3-11
CONTROL-S Command (Step working motion programs in all coordinate systems) 3-12
CONTROL-T Command (Toggle serial port half/full duplex mode) 3-12
CONTROL-U Command (Open rotary program buffer(s)) 3-13
CONTROL-V Command (Report velocity of all motors) 3-13
CONTROL-W Command 1-13
CONTROL-W Command (Take command line from dual-ported RAM) 3-14
Control-W Command Variable (I47) 2-24
CONTROL-X Command (Cancel in-process communications) 3-16
CONTROL-Y Command (Report last command line) 3-16
CONTROL-Z Command (Set PMAC in serial port communications mode) 3-17
Coordinate System
  Report 3-29
  Report Status 3-34
coordinate system status words 3-3
Coordinate-System-Specific Commands
  % (Report Feedrate Override) 3-25
  %{constant} (Set Feedrate Override) 3-27
  & (Report Coordinate System) 3-29
  &{constant} (Address Coordinate System) 3-28
  / (Halt program execution at end of currently executing move) 3-29
  / (hold permitting jogging) 3-43
  ?? (Report Coordinate System Status) 3-34
  {axis}={constant} (Re-define Axis Location). 3-46
  A (Abort) 3-44
  ABS Function 3-45
  B{constant} (Point to program) 3-47
  DEFINE ROTARY (Define Rotary Buffer) 3-58
  DELETE ROTARY (Delete Rotary Buffer) 3-65
  DELETE TBUF (Delete Axis Transformation Matrices) 3-65
  FRAX (Set Feedrate Axes) 3-72
  H (FeedHold) 3-74
  INC (Incremental Move Mode) 3-79
  MFLUSH (Clear pending synchronous M-Variable assignments) 3-120
  OPEN ROTARY (Open Rotary Buffer) 3-123
  PC (Report Program Counter) 3-128
  PE (Report Program Execution Pointer) 3-128
  PMATCH (Match Present and Commanded Position) 3-129
  PR (Report Rotary Buffer Size) 3-130
  Q (Quit) 3-131
  Q{constant} (Report Q-Variable Value) 3-131
  Q{constant}={expression} (Set Q-Variable Value) 3-132
  R (Run) 3-132
  S (Step) 3-134
  Summary 1-5
  UNDEFINE (Clear Coordinate System Definition) 3-138
  Z (Make commanded axis positions zero) 3-142
Copy setup parameters to non-volatile memory 3-135
COS 1-2
Current Limit
  Continuous (Ix57) 2-74
  Integrated (Ix58) 2-75
Cutter Comp Outside Corner Break Point (I89) . 2-34
Cutter Compensation Off 4-15
Cutter Compensation On Left 4-16
Cutter Compensation On Right 4-17
Cutter Compensation Radius 7-7
  Set CCR 4-18
Cutter Radius Compensation 1-9, 7-7
  Turning Off Compensation
    CC0 4-15
  Turning On Compensation
    CC1 4-16
    CC2 4-17

D

DAC Bias for Motor x (Ix29) 2-68
DAC Limit for Motor x (Ix69) 2-84
Data Gathering 1-4, 1-13, 2-23, 7-4
  DEFINE GATHER 3-57
  DELETE GATHER 3-63
  END GATHER 3-71
  GATHER 3-73

Data Gathering Buffer Location and Mode Variable (I45) . 1-13, 2-23, 7-4
Data Gathering I-Variables 2-21
Data Gathering Period (in Servo Cycles) Variable (I19). 1-13
Data Gathering Selection Mask Variable (I20) 1-13
Data Gathering Source 1 Address Variable (I21) 1-13
Data Integrity
Report command line checksum 3-9
DATE (Report PROM Date) 3-50
Deceleration Feed Hold Rate for Coordinate System x (Ix95) 2-115
Deceleration Rate on Position Limit or Abort for Motor x (Ix15) 1-14
Default Program Acceleration Time for Coordinate System x (Ix87) 1-16, 2-110
Default Program Feedrate for Coordinate System x (Ix89) 1-16, 2-111
Default Program S-Curve Time for Coordinate System x (Ix88) 1-16, 2-110, 2-111
Default Working Program Number for Coordinate System x (Ix91) 1-16, 2-112
Define A User Variable Buffer 3-60
Define Axis Transformation Buffer 3-59
Define Backlash Compensation Table 3-50
DEFINE BLCOMP (Define Backlash Compensation Table) 3-50
DEFINE COMP (Define Leadscrew Compensation)
one-dimensional 3-51
two-dimensional 3-55
DEFINE GATHER (Define Gather Buffer) 3-57
Define Gather Buffer 3-57
Define Leadscrew Compensation
one-dimensional 3-51
two-dimensional 3-55
DEFINE ROTARY (Define Rotary Buffer) 3-58
Define Rotary Buffer 3-58
DEFINE TBUF (Define Axis Transformation Buffer) 3-59
DEFINE TCOMP (Define Torque Compensation Table) 3-60
Define Torque Compensation Table 3-60
DEFINE UBUFFER (Define A User Variable Buffer) 3-60
Degree/Radian Control for User Trig Functions Variable (I15) 1-13
DELAY 4-25
Delete Axis Definition 3-20
Delete Axis Transformation Matrices 3-65
Delete Backlash Compensation Table 3-62
DELETE BLCOMP (Delete Backlash Compensation Table) 3-62
DELETE COMP (Delete Leadscrew Compensation) 1-7, 3-62
Delete Compiled PLC Program 3-64
DELETE GATHER (Delete Gather Buffer) 1-4, 3-63
Delete Gather Buffer 3-63
Delete Leadscrew Compensation 3-62
DELETE PLCC (Delete Compiled PLC Program) . 3-64
DELETE ROTARY (Delete Rotary Buffer) 3-65
Delete Rotary Buffer 3-65
DELETE TBUF (Delete Axis Transformation Matrices) 3-65
DELETE TCOMP (Delete Torque Compensation Table) 3-66
Delete Torque Compensation Table 3-66
DELETE TRACE (Delete Trace Buffer) 3-66
Delete Trace Buffer 3-66
Derivative Gain for Motor x (Ix31) 1-14
Disable a compiled PLC program 3-68
Disable a PLC program 3-67
Disable all PLC programs 3-3
DISABLE PLC 1-4
DISABLE PLC (Disable a PLC program) 3-67
DISABLE PLCC 4-26
DISABLE PLCC (Disable a compiled PLC program) 3-68
DISPLAY ... {variable} 4-29
DISPLAY [{constant}] "{message}" 4-27
Display a message 4-27
Display a variable 4-29
DPRAM ASCII Communications Enable (I58) . 2-30
DPRAM ASCII Communications Interrupt Enable (I56) 2-29
DPRAM Background Data Enable (I49) 2-25
DPRAM Background Variable Buffers Enable (I55) 2-28
DPRAM Binary Rotary Buffer Enable (I57) 2-30
DPRAM Buffer Maximum Motor/C.S. Number (I59) 2-30
DPRAM Servo Data Enable (I48) 2-25
Dual-Ported RAM 7-4
Registers 5-35
DWELL 4-30

E

ELSE 4-31
Enable a compiled PLC program 3-70
Enable a PLC program 3-69
ENABLE PLC 1-4, 4-34
ENABLE PLC (Enable a PLC program) 3-69
ENABLE PLCC 4-34
ENABLE PLCC (Enable a compiled PLC program) 3-70
Encoder Conversion Table 2-106, 5-8, 5-9
Encoder Counts per N Commutation Cycles for Motor x (Ix71) 1-15, 2-87
End Gathering 3-71
ENDGATHER (End Gathering) 3-71
ENDIF 2-9, 4-35
ENDWHILE 2-9, 4-36
Enter command line 3-9
Enter/Set Program Password 3-126

Erase last character 3-6
Error Codes
Summary 1-18
Error Reporting Mode Variable (I6) 1-13, 2-8
Examples
A More Complex Move 6-27
A Robotic Application 6-30
A Simple G-Code Program 6-29
A Simple Move 6-26
A Torque Limited Screwing Operation 6-36
Altering Destination in Moves 6-37
Changing Speed 6-37
Circular Interpolation 6-47
Circular Interpolation with Linear Moves 6-38
Conditional Branching 6-27
Delayed Program Execution 6-62
Encoder Modulo Check 6-43
G-code Programming 6-60
In Position Confirming Routines 6-42
Integrated Current Limiting 6-63
Inverse Kinematics 6-52
Jogging Motors From A PLC Program 6-49
Linear and Circular interpolation 6-28
Linear and Circular Moves 6-58
Matching Positions on the Fly 6-33
Motion Program Execution 6-58
Move Until Trigger 6-57
Open-loop Movement 6-50
Orientation of a Rotary Head 6-54
PLC Display Programming 6-32
PLC I/O 6-46
PLC Input Programming 6-31
PMAC Example Programs 6-1
Precision Slaving 6-32
Proportional Voltage Output 6-48
Pulse Output with Motors 6-45
Pulsing a Machine Output 6-49
Pure Position Step 6-45
Rotary Tangent to X-Y System 6-31
Sending Information to a Display 6-40
Spindle Operation 6-34
Store The Home-Position Encoder Offset 6-44
Suggested M-variable Definitions 6-1
Time Base Following 6-55
Torque-limited Velocity Move 6-36
EXP Function 1-2
Expansion I/O
PMAC-Lite 5-48
PMAC-PC 5-48
PMAC-VME 5-48
Servo Algorithm I-variables 2-74
Extended Servo Algorithm I-variables (Ix40 - Ix56) 2-74

F

F (Report Motor Following Error) 3-71
F (Set Feedrate) 1-7
Fatal (Shutdown) Following Error Limit for Motor x (Ix11) 1-14, 2-51, 2-83
Feed Hold Deceleration Rate for Coordinate System x (Ix95) 1-16, 2-115
Feedforward
Acceleration (Ix35) 2-72
Friction (Ix68) 2-84
Velocity (Ix32) 2-71
Feedhold 3-74
Feedhold on all coordinate systems 3-10
Feedrate
Default Program for Coordinate System x (Ix89) 2-111
Feedrate Axis 4-38
Feedrate Override
Report 3-25
Feedrate Specification 4-37
Feedrate Time Units for Coordinate System x (Ix90) 1-16
Fixed Buffer Full Warning Point Variable (I18) 1-13
Flag Address 2-59
Following Error
Report for all motors 3-5
Report for Motor 3-71
Following Error Limits 2-83
Fatal 2-51
Warning 2-52
FRAX (Set Feedrate Axes) 3-72
Friction Feedforward (Ix68) 2-84
Full/Abbreviated Program Listing Form Variable (I9) ... 1-13
Further Motor I-Variables 2-108

G

G{data} 4-39
Gate Array Registers 5-17
GATHER (Gather Data) 3-73
Gather Data 3-73
General Global Registers 5-10, 5-11
Global Commands
$$$ (Global Reset) 3-23
$$$*** (Global Reset and Reinitialize) 3-24
??? (Report Global Status) 3-38
CONTROL-A Command (Abort all programs and moves) 3-1
CONTROL-D Command (Disable all PLC programs) 3-3
CONTROL-F Command (Report following errors for all motors) 3-5
CONTROL-K Command (Kill all motors) 3-7
CONTROL-O Command (Feedhold on all coordinate systems) 3-10
CONTROL-P Command (Report positions of all motors) 3-10
CONTROL-Q Command (Quit all executing motion programs) 3-11

CONTROL-R Command (Begin execution of motion programs in all coordinate systems) 3-11
CONTROL-S Command (Step working motion programs in all coordinate systems) 3-12
CONTROL-V Command (Report velocity of all motors)..... 3-13
DEFINE GATHER (Define Gather Buffer) 3-57
DEFINE UBUFFER (Define A User Variable Buffer) 3-60
DELETE TRACE (Delete Trace Buffer)..... 3-66
DISABLE PLC (Disable a PLC program)..... 3-67
DISABLE PLCC (Disable a compiled PLC program) 3-68
ENABLE PLC (Enable a PLC program)..... 3-69
ENABLE PLCC (Enable a compiled PLC program) 3-70
ENDGATHER (End Gathering)..... 3-71
GATHER (Gather Data)..... 3-73
I{constant} (Report I-Variable Value) 3-76
I{constant}=* (Set Default Value) 3-79
I{constant}={expression} (Set I-Variable Value) 3-77
LIST GATHER (List Gathered Data)..... 3-95
LIST LDS (List Linking Addresses of Ladder Functions)..... 3-97
LIST LINK (List Linking Addresses of Internal PMAC Routines) 3-97
LIST PROGRAM (List Motion Program)..... 3-100
MACROAUX (Report or write MACRO auxiliary parameter value) 3-117
MACROAUXREAD (Read MACRO auxiliary parameter value) 3-117
MACROAUXWRITE (Write MACRO auxiliary parameter value) 3-119
OPEN PLC (Open PLC Buffer) 3-121
OPEN PROGRAM (Open Program Buffer).. 3-122
P{constant} (Report P-Variable Value)..... 3-125
P{constant}={expression} (Set P-Variable Value) 3-126
PASSWORD={string} (Enter/Set Program Password) 3-126
R[H]{address} (Read Address) 3-134
SAVE (Copy setup parameters to non-volatile memory) 3-135
SIZE (Report Available Memory) 3-136
TYPE (Report type of PMAC) 3-137
UNDEFINE ALL (Clear All Coordinate System Definitions)..... 3-139
VERSION (Report PROM Version)..... 3-140
W{address} (Write to Address)..... 3-141
Global I-Variables 2-2
Data Gathering I-Variables..... 2-21
Global Reset 3-23
Global Reset and Reinitialize 3-24
Global Servo Calculation Registers..... 5-2
Global Status
Report 3-38
Global status word..... 3-5
GOSUB (Goto Subprogram) 4-40
GOTO..... 4-40

H

H (FeedHold)..... 3-74
Halt program execution at end of currently executing move 3-29
hold permitting jogging 3-43
HOME (Home Command) 3-74
Home Command 1-7, 3-74
HOME 3-74, 4-41
HOMEZ..... 3-75, 4-42
Home Flag 1-14, 2-59
Home Offset for Motor x (Ix26)..... 1-14
Home Speed for Motor x (Ix23) 1-14, 2-59
HOMEZ (No Motion Home)..... 1-7, 1-8, 3-75, 7-7
Homing Search Move
Home Command..... 4-41
HOME 3-74
HOMEZ..... 3-75
Zero-Move Homing..... 4-42
Homing Speed 1-14, 2-59

I

I/O Handshake Control Variable (I3) 1-13, 2-4
I{constant} (Report I-variable Value)..... 1-4
I{constant} (Report I-Variable Value) 3-76
I{constant}=* (Set Default Value) 3-79
I{constant}={expression} (Set I-variable Value)..... 1-4, 4-44
I{constant}={expression} (Set I-Variable Value) 3-77
I10 Servo Interrupt Time 2-13
I1000 MACRO Node Auxiliary Register Enable ... 2-121
I1001 MACRO Ring Check Period..... 2-122
I11 Programmed Move Calculation Time 2-16
I12 Jog-to-Position Calculation Time..... 2-16
I13 Programmed Move Segmentation Time..... 2-17
I14 Auto Position Match on Run Enable 2-17
I15 Degree/Radian Control for User Trig Functions 2-18
I16 Rotary Buffer Request On Point..... 2-18
I17 Rotary Buffer Request Off Point..... 2-20
I18 Fixed Buffer Full Warning Point..... 2-21
I19 Data Gathering Period..... 2-21
I20 Data Gathering Selection Mask..... 2-21
I21 Data Gathering Source 1 Address 2-22
I22--I44 Data Gathering Source 2 thru 24 Addresses 2-23
I45 Data Gathering Buffer Location and Mode.. 2-23
I46 Reserved for future use..... 2-24
I47 Address of Pointer for Control-W Command 2-24
I48 DPRAM Servo Data Enable..... 2-25
I49 DPRAM Background Data Enable..... 2-25
I50 Rapid Move Mode Control..... 2-26

I51 Leadscrew Compensation Enable..... 2-26
I52 '\' Program Hold Slew Rate 2-26
I53 Program Step Mode Control..... 2-27
I55 DPRAM Background Variable Buffers Enable 2-28
I56 DPRAM ASCII Communications Interrupt Enable 2-29
I57 DPRAM Binary Rotary Buffer Enable..... 2-30
I58 DPRAM ASCII Communications Enable 2-30
I59 DPRAM Buffer Maximum Motor/C.S. Number 2-30
I60 Auto-Converted ADC Register Address 2-31
I61 Number of Auto-Converted ADC Registers 2-32
I62 Internal Message Carriage Return Control... 2-33
I63--I79 Reserved for future use..... 2-33
I7 In-Position Number of Cycles..... 2-9
I8 Real Time Interrupt Period..... 2-10
I89 Cutter Comp Outside Corner Break Point.... 2-34
I8x Motor x Third-Resolver Gear Ratio 2-34
I9 Full/Abbreviated Program Listing Form 2-10
I900, I905, ... I975 Encoder n Decode Control. 2-117
I901, I906, ... I976 Encoder n Filter Disable 2-118
I902, I907, ... I977 Encoder n Position Capture Control..... 2-119
I903, I908, ... I978 Encoder n Flag Select Control . 2-121
I904, I909, ... I979 -- Reserved for future use ... 2-121
I99 Backlash Hysteresis..... 2-36
I9x Motor x Second-Resolver Gear Ratio 2-36
IDIS{constant} (Incremental Axis Displacement) .. 4-45
IF ({condition}) 4-46
INC (Incremental Move Mode) 3-79, 4-48
Incremental Move Mode 3-79
Induction Motor Magnetization Current for Motor x (Ix77) 1-15, 2-94
Induction Motor Slip Gain for Motor x (Ix78) 1-16
In-position Band for Motor x (Ix28)..... 1-14
In-Position Number of Cycles (I7) 2-9
Inputs and Outputs
PMAC-Lite 5-37
PMAC-PC..... 5-37
PMAC-VME..... 5-37
INT Function 1-2
Integral Gain..... 1-15, 2-71
Integral Gain for Motor x (Ix33) 1-15, 2-71
Integration Mode for Motor x (Ix34)..... 1-15
Internal Message Carriage Return Control (I62) 2-33
IROT{constant} (Incremental Axis Rotation)..... 4-49
I-Variables 2-2, 7-5
Commutation I-Variables 2-86
Coordinate System Specific Summary..... 1-16
Data Gathering I-Variables..... 2-21
Encoder Specific Summary..... 1-17
Further Motor I-Variables..... 2-108
Global 2-2
MACRO Specific Summary..... 1-17
Motor Definition I-Variables..... 2-38
Motor I-Variables 2-38
Motor Movement I-Variables 2-57
Motor Safety I-Variables 2-51
Motor Servo Loop Modifiers..... 2-74
Report Value..... 3-76
Servo Control I-Variables..... 2-69
Set Default I-Variable Value 3-79
Set I-Variable Value 3-77
Summary..... 1-16
Basic Servo Control..... 1-14
Global 1-13
Motor 1-13
Motor Commutation 1-15
Motor Definition..... 1-13
Motor Movement..... 1-14
Motor Safety 1-14
Servo Loop 1-15
Ix00 Motor x Activate 2-38
Ix01 Motor x PMAC-Commutation Enable 2-38
Ix02 Motor x Command Output (DAC) Address 2-38
Ix03 Motor x Position Loop Feedback Address . 2-43
Ix04 Motor x Velocity Loop Feedback Address. 2-44
Ix05 Motor x Master (Handwheel) Position Address 2-46
Ix06 Motor x Master (Handwheel) Following Enable 2-47
Ix07 Motor x Master (Handwheel) Scale Factor 2-47
Ix08 Motor x Position Scale Factor 2-48
Ix09 Motor x Velocity Loop Scale Factor 2-48
Ix10 Motor x Power-On Servo Position Address 2-49
Ix11 Motor x Fatal (Shutdown) Following Error Limit 2-51
Ix12 Motor x Warning Following Error Limit ... 2-52
Ix13 Motor x Positive Software Position Limit.. 2-53
Ix14 Motor x Negative Software Position Limit 2-53
Ix15 Motor x Deceleration Rate on Position Limit or Abort..... 2-54
Ix16 Motor x Maximum Permitted Motor Program Velocity 2-54
Ix17 Motor x Maximum Permitted Motor Program Acceleration..... 2-54
Ix19 Motor x Maximum Permitted Motor Jog/Home Acceleration..... 2-57
Ix20 Motor x Jog/Home Acceleration Time..... 2-57
Ix21 Motor x Jog/Home S-Curve Time..... 2-58
Ix22 Motor x Jog Speed..... 2-58
Ix23 Motor x Homing Speed and Direction 2-59
Ix25 Motor x Limit/Home Flag 2-59
Ix26 Motor x Home Offset 2-65
Ix27 Motor x Position Rollover Range..... 2-66
Ix28 Motor x In-position Band 2-66
Ix29 Motor x Output - or First Phase - DAC Bias.. 2-68
Ix30 Motor x PID Proportional Gain..... 2-70

Ix31 Motor x PID Derivative Gain 2-70
Ix32 Motor x PID Velocity Feedforward Gain ... 2-71
Ix33 Motor x PID Integral Gain 2-71
Ix34 Motor x PID Integration Mode 2-72
Ix35 Motor x PID Acceleration Feedforward Gain 2-72
Ix36 Motor x PID Notch Filter Coefficient N1 .. 2-73
Ix37 Motor x PID Notch Filter Coefficient N2 .. 2-73
Ix38 Motor x PID Notch Filter Coefficient D1 .. 2-73
Ix39 Motor x PID Notch Filter Coefficient D2 .. 2-74
Ix40 - Ix56 Motor x Extended Servo Algorithm I-variables 2-74
Ix57 Motor x Continuous Current Limit 2-74
Ix58 Motor x Integrated Current Limit 2-75
Ix59 Motor x User-Written Servo/Phase Enable 2-78
Ix60 Motor x Servo Cycle Period Extension 2-78
Ix63 Motor x Integration Limit 2-80
Ix64 Motor x Deadband Gain Factor 2-81
Ix65 Motor x Deadband Size 2-83
Ix67 Motor x Linear Position Error Limit 2-83
Ix68 Motor x Friction Feedforward 2-84
Ix69 Motor x Output Command (DAC) Limit ... 2-84
Ix70 Motor x Number of Commutation Cycles (N) 2-86
Ix71 Motor x Encoder Counts per N Commutation Cycles 2-87
Ix72 Motor x Commutation Phase Angle 2-88
Ix73 Motor x Phase Finding Output Value 2-89
Ix74 Motor x Phase Finding Time 2-91
Ix75 Motor x Power-On Phase Position Offset .. 2-92
Ix76 Motor x Velocity Phase Advance 2-93
Ix77 Motor x Induction Motor Magnetization Current 2-94
Ix78 Motor x Induction Motor Slip Gain 2-95
Ix79 Motor x A Phase Output (DAC) Bias 2-96
Ix80 Motor x Power-Up Mode 2-97
Ix81 Motor x Power-On Phase Position Address ... 2-100
Ix83 Motor x Ongoing Phasing Position Address .. 2-106
Ix85 Motor x Backlash Takeup Rate 2-108
Ix86 Motor x Backlash Size 2-109
Ix87 Coordinate System x Default Program Acceleration Time 2-110
Ix88 Coordinate System x Default Program S-Curve Time 2-111
Ix89 Coordinate System x Default Program Feedrate/Move Time 2-111
Ix90 Coordinate System x Feedrate Time Units 2-112
Ix91 Coordinate System x Default Working Program Number 2-112
Ix92 Coordinate System x Move Blend Disable 2-113
IX93 COORDINATE SYSTEM X TIME BASE CONTROL REGISTER ADDRESS 2-113
Ix94 Coordinate System x Time Base Slew Rate (and Limit) ... 2-114
Ix95 Coordinate System x Feed Hold Deceleration Rate ... 2-115
Ix96 Coordinate System x Circle Error Limit ... 2-115

J

J
* (Jog Incremental Specified by a Variable) 3-83
{constant} (Jog Incremental) 3-83
J- (Jog Negative) 3-82
J! (Adjust motor commanded position to nearest integer count) 3-80
J/ (Jog Stop) 3-82
J^* (Jog Incremental Specified by a Variable) 3-89
J^{constant} (Jog Incremental) 3-88
J+ (Jog Positive) 3-81
J= (Jog to Previous) 3-84
J=* (Jog to Position Specified by a Variable) 3-86
J={constant} (Jog to Position) 3-86
J=={constant} (Jog to position and make it the ... 3-87
Jog Commands
{jog command}^{constant} (Jog Until Trigger) . 3-90
J
{constant} (Jog Incremental Specified by a Variable) 3-83
{constant} (Jog Incremental) 3-83
J- (Jog Negative) 3-82
J/ (Jog Stop) 3-82
J^* (Jog Incremental Specified by a Variable) 3-89
J^{constant} (Jog Incremental) 3-88
J+ (Jog Positive) 3-81
J= (Jog to Previous) 3-84
J=* (Jog to Position Specified by a Variable) .. 3-86
J={constant} (Jog to Position) 3-86
J=={constant} (Jog to position and make it the 3-87
Jog Incremental 3-83, 3-88
Jog Incremental Specified by a Variable 3-83, 3-89
Jog Negative 3-82
Jog Positive 3-81
Jog Speed 1-14, 2-58
Jog Speed for Motor x (Ix22) 1-14, 2-58
Jog Stop 3-82
Jog to Position 3-86
Jog to position and make it the 3-87
Jog to Position Specified by a Variable 3-86
Jog to Previous 3-84
Jog Until Trigger 3-90
Jogging and Homing Acceleration Time for Motor x (Ix20) 1-14, 2-57
Jogging and Homing S-Curve Time for Motor x (Ix21) 1-14, 2-57, 2-58
Jogging Move Control
Jog Commands
{jog command}^{constant} (Jog Until Trigger) 3-90
J
{constant} (Jog Incremental Specified by a Variable) 3-83
{constant} (Jog Incremental) 3-83

J- (Jog Negative) ... 3-82
J! 3-80
J/ (Jog Stop) ... 3-82
J^* (Jog Incremental Specified by a Variable) 3-89
J^{constant} (Jog Incremental) ... 3-88
J+ (Jog Positive) ... 3-81
J= (Jog to Previous) ... 3-84
J=* (Jog to Position Specified by a Variable) .. 3-86
J={constant} (Jog to Position) ... 3-86
J=={constant} (Jog to position and make it the 3-87
Jog-to-Position Calculation Time Variable (I12) ... 1-13
JTHW Port
BCD Thumbwheel-Multiplexer M-Variable Definition ... 3-109
Binary Thumbwheel-Multiplexer Definition . 3-109
Resolver Thumbwheel-Multiplexer M-Variable Definition ... 3-111
Serial Thumbwheel-Multiplexer M-Variable Definition ... 3-112

K

K (Kill Motor) ... 3-91
Kill all motors ... 3-7
Kill Motor ... 3-91

L

Leadscrew Compensation ... 1-7, 1-13, 7-6
Define Leadscrew Compensation (one-dimensional) ... 3-51
Define Leadscrew Compensation (two-dimensional) ... 3-55
Delete Leadscrew Compensation ... 3-62
Leadscrew Compensation Enable Variable (I51) ... 1-13
LEARN (Learn A Motion Program) ... 3-92
Learn A Motion Program ... 3-92
Limit/Home Flag/Amp Flag Address for Motor x (Ix25) ... 1-14, 2-59, 7-4
LINEAR (Linear Mode) ... 3-37, 4-51
Acceleration Parameters
Set S-curve Time ... 4-84
TA{data} ... 4-81
Feedrate or Move-Time Specification 4-37, 4-38
Set Move Time ... 4-82
LIST (List Buffer) ... 3-93
List Buffer ... 3-93
LIST COMP (List Leadscrew Compensation Table) ... 3-94
LIST COMP DEF (List Definition of Addressed Motor's Leadscrew Compensation Table) ... 3-94
List Definition of Addressed Motor's Leadscrew Compensation Table ... 3-94
LIST GATHER (List Gathered Data) ... 3-95
List Gathered Data ... 3-95
LIST LDS (List Linking Addresses of Ladder Functions) ... 3-97
List Leadscrew Compensation Table ... 3-94
LIST LINK (List Linking Addresses of Internal PMAC Routines) ... 3-97
List Linking Addresses of Internal PMAC Routines 3-97
List Linking Addresses of Ladder Functions ... 3-97
List Motion Program ... 3-100
LIST PC (List Program at Counter) ... 3-97
LIST PE (List Program at Execution Point) ... 3-99
LIST PLC (List PLC Program) ... 3-100
List PLC Program ... 3-100
LIST PROGRAM (List Motion Program) ... 3-100
List Program at Counter ... 3-97
List Program at Execution Point ... 3-99
LN Function ... 1-2

M

M{constant} (Report M-variable Value) ... 1-4
M{constant} (Report M-Variable Value) ... 3-102
M{constant}&={expression} ... 4-54
M{constant}={expression} (Set M-variable Value) 4-52
M{constant}={expression} (Set M-Variable Value) 3-103
M{constant}=={expression} (Set Synchronous M-variable Value) ... 4-53
M{constant}-> (Report M-Variable Definition) 3-103
M{constant}->* (Self Reference M-variable) ... 1-4
M{constant}->* (Self Reference M-Variable) .. 3-104
M{constant}->D
{address} (Long Fixed-Point M-Variable Definition) ... 3-105
M{constant}->DP
{address} (Dual-Ported RAM Fixed-Point M-Variable Definition) ... 3-105
M{constant}->F
{address} (Dual-Ported RAM Floating-Point M-Variable Definition) ... 3-107
M{constant}->L
{address} (Long Word Floating-Point M-Variable Definition) ... 3-108
M{constant}->TWB
{address} (Binary Thumbwheel-Multiplexer Definition) ... 3-109
M{constant}->TWD
{address} (BCD Thumbwheel-Multiplexer M-Variable Definition) ... 3-109
M{constant}->TWR
{address},{offset} (Resolver Thumbwheel-Multiplexer M-Variable Definition) ... 3-111
M{constant}->TWS
{address} (Serial Thumbwheel-Multiplexer M-Variable Definition) ... 3-112
M{constant}->X/Y

{address} (Short Word M-Variable Definition) .. 3-115
Machine-Tool Style Program
G, M, T, and D-Codes
D{data} ... 4-24
G{data} ... 4-39
M{data} ... 4-56
S{data} (Spindle Function) ... 4-74
Standard M-Codes
M{data} ... 4-56
T{data} ... 4-80
MACRO
Read Auxiliary Register ... 4-57
Report auxiliary parameter value ... 3-117
Write to Auxiliary Register ... 4-57
MACRO Node Auxiliary Register Enable (I1000) 2-121
MACRO Ring Check Period (I1001) ... 2-122
MACROAUX (Report or write MACRO auxiliary parameter value) ... 3-117
MACROAUXREAD (Read MACRO auxiliary parameter value) ... 3-117
MACROAUXWRITE (Write MACRO auxiliary parameter value) ... 3-119
Magnetization Current ... 2-94
Make commanded axis positions zero ... 3-142
Master (Handwheel) Following Enable for Motor x (Ix06) 1-13
Master (Handwheel) Position Address for Motor x (Ix05) .. 1-13, 2-5, 2-106, 7-4
Master (Handwheel) Scale Factor for Motor x (Ix07) 1-13
Match Present and Commanded Position ... 3-129
Matrices
Absolute Axis Displacement ... 4-9
Absolute Axis Rotation ... 4-11
Define Axis Transformation Buffer ... 3-59
Delete Axis Transformation Buffer ... 3-65
Incremental Axis Displacement ... 4-45
Incremental Axis Rotation ... 4-49
TINIT ... 4-82
TSELECT{constant} ... 4-84
Maximum Permitted Motor Jog Acceleration for Motor x (Ix19) ... 1-14, 2-57
Maximum Permitted Motor Programmed Acceleration for Motor x (Ix17) ... 1-14, 2-54
Maximum Permitted Motor Velocity for Motor x (Ix16) ... 1-14, 2-54
M-Codes ... 4-56
Memory
Report Available ... 3-136
Memory Map ... 5-1
Dual-Ported RAM Registers ... 5-35
Encoder Conversion Table ... 5-8, 5-9
Expansion I/O
PMAC-Lite ... 5-48
PMAC-PC ... 5-48
PMAC-VME ... 5-48
Gate Array Registers ... 5-17
General Global Registers ... 5-10, 5-11
Global Servo Calculation Registers ... 5-2
Inputs and Outputs
PMAC-Lite ... 5-37
PMAC-PC ... 5-37
PMAC-VME ... 5-37
Motor and Coordinate System Status and Control Registers ... 5-11
Motor Calculation Registers ... 5-3, 5-6
VME-Bus Registers ... 5-35
MFLUSH (Clear pending synchronous M-Variable assignments) ... 3-120
Modal Commands
ABS Function ... 4-7
INC ... 4-48
Motion Program Calculation and Execution
Calculating Ahead ... 4-30
Motion Program Structure
Adding Logic
ELSE ... 4-31
ENDIF ... 4-35
ENDWHILE ... 4-36
IF ({condition}) ... 4-46
WHILE({condition}) ... 4-88
Adding Variables and Calculations
M{constant}&={expression} ... 4-54
M{constant}^={expression} ... 4-55
M{constant}|={expression} ... 4-55
Set I-variable Value ... 4-44
Set M-variable Value ... 4-52
Set P-variable Value ... 4-62
Set Q-variable Value ... 4-68
Set Synchronous M-variable Value ... 4-53
PRELUDE Command ... 4-63
Subroutines and Subprograms
Alternate Location ... 4-60
Call Subprogram ... 4-14
GOSUB ... 4-40
GOTO ... 4-40
Location Label ... 4-58
Read Arguments ... 4-70
RETURN ... 4-73
Motion Programs
Adding Logic
ELSE ... 4-31
ENDIF ... 4-35
ENDWHILE ... 4-36
IF ({condition}) ... 4-46
WHILE({condition}) ... 4-88
Adding Variables and Calculations
M{constant}&={expression} ... 4-54
M{constant}^={expression} ... 4-55
M{constant}|={expression} ... 4-55
Set I-variable Value ... 4-44
Set M-variable Value ... 4-52

Set P-variable Value 4-62
Set Q-variable Value..... 4-68
Set Synchronous M-variable Value 4-53
Axis Transformation Matrices
Absolute Axis Displacement 4-9
Absolute Axis Rotation..... 4-11
Define Axis Transformation Buffer..... 3-59
Delete Axis Transformation Buffer 3-65
Incremental Axis Displacement..... 4-45
Incremental Axis Rotation 4-49
TINIT..... 4-82
TSELECT{constant} 4-84
CLEAR (Clear Buffer) 3-48
CLOSE (Close Buffer) 3-48
Examples
A More Complex Move..... 6-27
A Robotic Application..... 6-30
A Simple G-Code Program..... 6-29
A Simple Move..... 6-26
A Torque Limited Screwing Operation 6-36
Altering Destination in Moves..... 6-37
Changing Speed..... 6-37
Circular Interpolation..... 6-47
Circular Interpolation with Linear Moves 6-38
Conditional Branching..... 6-27
Delayed Program Execution 6-62
G-code Programming..... 6-60
Linear and Circular interpolation..... 6-28
Linear and Circular Moves 6-58
Matching Positions on the Fly 6-33
Motion Program Execution..... 6-58
Move Until Trigger..... 6-57
Open-loop Movement..... 6-50
Orientation of a Rotary Head..... 6-54
Rotary Tangent to X-Y System 6-31
Spindle Operation 6-34
Torque-limited Velocity Move 6-36
Linear Blended Moves
LINEAR 4-51
Machine-Tool Style
T{data} 4-80
Machine-Tool Style Program
S{data} (Spindle Function)..... 4-74
Modal Commands
ABS Function 4-7
INC 4-48
Move Commands
A-axis Move 4-7
B-axis Move 4-11
C-axis Move 4-14
Circular Arc Move Specification 4-5
Move-Until-Trigger 4-3
Position and Velocity Move Specification 4-2
Position-Only Move Specification..... 4-1
PVT{data} 4-67
STOP 4-80
U-axis Move 4-85
V-axis Move 4-86
W-axis Move 4-87
X-axis Move 4-90
Y-axis Move 4-91
Z-axis Move..... 4-91
OPEN PROGRAM (Open Program Buffer).. 3-122
Pointing to the Program
B{constant} 3-47
PRELUDE Command..... 4-63
PVT-Mode Moves
PVT{data} 4-67
Rapid-Mode Moves
RAPID 4-70
Report Execution Pointer..... 3-128
Rotary Motion Program Buffers
OPEN ROTARY (Open Rotary Buffer).... 3-123
Running the Program
R (Run) 3-132
Splined Moves 4-78, 4-79
Stepping the Program 4-12
BLOCKSTOP..... 4-13
S (Step) 3-134
Subroutines and Subprograms
Alternate Location 4-60
Call Subprogram..... 4-14
GOSUB..... 4-40
GOTO 4-40
Location Label 4-58
Read Arguments 4-70
RETURN 4-73
Motor and Coordinate System Status and Control Registers..... 5-11
Motor Calculation Registers 5-3, 5-6
Motor Definition I-Variables..... 2-38
Motor Following Error
Report 3-71
Motor I-variables 2-38
Commutation I-Variables 2-86
Further Motor I-Variables..... 2-108
Motor Definition I-Variables..... 2-38
Motor Movement I-Variables 2-57
Motor Safety I-Variables 2-51
Motor Servo Loop Modifiers..... 2-74
Servo Control I-Variables..... 2-69
Summary..... 1-13
Motor Movement I-Variables 2-57
Motor Position
Report 3-124
Motor Reset 3-23
Motor Safety I-Variables 2-51
Motor Servo Loop Modifiers..... 2-74
Motor Status
Report 3-30
Motor Velocity
Report 3-139

Motor x Power-On Servo Position Address (Ix10). 2-49
Motor x Second-Resolver Gear Ratio (I9x)..... 2-36
Motor x Third-Resolver Gear Ratio (I8x)..... 2-34
Motors
Addressed 3-18
Positions 3-10
Velocity 3-13
Motor-Specific Commands
(Report Addressed Motor)..... 3-18
#{constant} (Set Addressed Motor)..... 3-18
#{constant}-> (Report Axis Definition) 3-19
#{constant}->{axis definition}(Set Axis Definition) 3-20
#{constant}->0 (Delete Axis Definition)..... 3-20
$ (Motor Reset)..... 3-23
? (Report Motor Status) 3-30
{jog command}^{constant} (Jog Until Trigger) . 3-90
DEFINE BLCOMP (Define Backlash Compensation Table)..... 3-50
DEFINE COMP (Define Leadscrew Compensation)
one-dimensional..... 3-51
two-dimensional 3-55
DEFINE TCOMP (Define Torque Compensation Table)..... 3-60
DELETE BLCOMP (Delete Backlash Compensation Table)..... 3-62
DELETE COMP (Delete Leadscrew Compensation)..... 3-62
F (Report Motor Following Error)..... 3-71
HOME (Home Command) 3-74
HOMEZ (No Motion Home)..... 3-75
J
{constant} (Jog Incremental)..... 3-83
J- (Jog Negative)..... 3-82
J! (Adjust motor commanded position to nearest integer count)..... 3-80
J* (Jog Incremental Specified by a Variable) .. 3-83
J/ (Jog Stop)..... 3-82
J^* (Jog Incremental Specified by a Variable) 3-89
J+ (Jog Positive) 3-81
J= (Jog to Previous) 3-84
J=* (Jog to Position Specified by a Variable).. 3-86
J={constant} (Jog to Position)..... 3-86
J=={constant} (Jog to position and make it the3-87
K (Kill Motor) 3-91
LIST COMP (List Leadscrew Compensation Table)..... 3-94
LIST COMP DEF (List Definition of Addressed Motor's Leadscrew Compensation Table) ... 3-94
O{constant} (Open Loop Move) 3-120
P (Report Motor Position) 3-124
V (Report Motor Velocity)..... 3-139
Move Blend Disable for Coordinate System x (Ix92) 1-16
Move Commands
A-axis Move 4-7
B-axis Move 4-11
C-axis Move 4-14
Circular Arc Move Specification..... 4-5
Move-Until-Trigger..... 4-3
Position and Velocity Move Specification 4-2
Position-Only Move Specification..... 4-1
PVT{data} 4-67
U-axis Move 4-85
V-axis Move 4-86
W-axis Move 4-87
X-axis Move 4-90
Y-axis Move 4-91
Z-axis Move..... 4-91
Move Segmentation 1-13, 2-17
Move Until Trigger..... 4-3
Move-Time Specification 4-82
Move-Until-Trigger 4-3
M-Variables..... 7-4
BCD Thumbwheel-Multiplexer Definition.... 3-109
Binary Thumbwheel-Multiplexer Definition. 3-109
Dual-Ported RAM Fixed-Point Definition..... 3-105
Dual-Ported RAM Floating-Point Definition 3-107
Long Fixed-Point Definition..... 3-105
Long Word Floating-Point Definition 3-108
Report Definition..... 3-103
Report Value..... 3-102
Resolver Thumbwheel-Multiplexer Definition.... 3-111
Self Reference M-Variable 3-104
Serial Thumbwheel-Multiplexer Definition .. 3-112
Set Value 3-103
Short Word Definition 3-115
Suggested Definitions..... 6-1

N

N{constant} (Location Label) 1-10, 4-58
Negative Software Position Limit for Motor x (Ix14) 1-14, 2-53
No Motion Home..... 3-75
Notch Filter Coefficient D1 for Motor x (Ix38)... 1-14
Notch Filter Coefficient D2 for Motor x (Ix39)... 1-15
Notch Filter Coefficient N1 for Motor x (Ix36)... 1-14
Notch Filter Coefficient N2 for Motor x (Ix37)... 1-14
Notch Filters 1-14, 1-15
Number of Auto-Converted ADC Registers (I61).. 2-32
Number of Commutation Cycles (N) for Motor x (Ix70) 1-15, 2-86

O

O{constant} (Alternate Location)..... 4-60
O{constant} (Open Loop Move) 3-120

Ongoing Phasing Position Address for Motor x (Ix83) 1-15, 2-106
On-line Commands
Sending from a program 4-21, 4-23
On-Line Commands 7-5
Open Loop Move 3-120
OPEN PLC (Open PLC Buffer) 3-121
Open PLC Buffer 3-121
OPEN PROGRAM (Open Program Buffer) 3-122
Open Program Buffer 3-122
OPEN ROTARY (Open Rotary Buffer) 3-123
Open Rotary Buffer 3-123
Open rotary program buffer(s) 3-13
Opening A PLC Buffer
OPEN PLC (Open PLC Buffer) 3-121
Output - or B Phase - DAC Bias for Motor x (Ix29) 2-68
Output - or First Phase - DAC Bias for Motor x (Ix29) 1-14
Output Command (DAC) Limit for Motor x (Ix69) 2-84

P

P (Report Motor Position) 1-7, 3-124
P{constant} (Report P-variable Value) 1-4
P{constant} (Report P-Variable Value) 3-125
P{constant}={expression} (Set P-variable Value) .. 4-62
P{constant}={expression} (Set P-Variable Value) . 3-126
PASSWORD={string} (Enter/Set Program Password) 3-126
PC (Program Counter) 3-128
PC (Report Program Counter) 3-128
PE (Report Program Execution Pointer) 3-128
Phase Advance 1-15
Velocity 2-93
Phase Finding Output (DAC) Value for Motor x (Ix73) 1-15, 2-89
Phase Finding Time for Motor x (Ix74) 1-15, 2-91
Phase Output (DAC) Bias for Motor x (Ix79) 2-96
Phase Position Address
Power-On (Ix81) 2-100
Phase Position Offset (Ix75) 2-92
Phasing Offset for Motor x (Ix75) 1-15
Phasing Position Address for Motor x (Ix83) 2-106
PID Servo Filter 1-14, 1-15
Acceleration Feedforward Gain 2-72
Derivative Gain 2-70
Integral Gain 2-71
Integration Mode 2-72
Notch Filter Coefficient D1 2-73
Notch Filter Coefficient D2 2-74
Notch Filter Coefficient N1 2-73
Notch Filter Coefficient N2 2-73
Proportional Gain 2-70
Velocity Feedforward Gain 2-71
PLC ENABLE 4-34
PLC Programs
Address Command 4-8
CLEAR (Clear Buffer) 3-48
CLOSE (Close Buffer) 3-48
Compiled PLC Programs
DELETE PLCC 3-64
Conditional Statements
AND ({condition}) 4-10
OR({condition}) 4-60
Erasing the Program
CLEAR (Clear Buffer) 3-48
Examples
Encoder Modulo Check 6-43
In Position Confirming Routines 6-42
Integrated Current Limiting 6-63
Jogging Motors From A PLC Program 6-49
PLC Display Programming 6-32
PLC I/O 6-46
PLC Input Programming 6-31
Proportional Voltage Output 6-48
Pulse Output with Motors 6-45
Pulsing a Machine Output 6-49
Pure Position Step 6-45
Sending Information to a Display 6-40
Store The Home-Position Encoder Offset 6-44
Opening the Buffer
OPEN PLC (Open PLC Buffer) 3-121
PLC Programs On/Off Variable (I5) 1-13, 2-8
PLCC DISABLE 4-26
PLCC ENABLE 4-34
PMAC
Report type of 3-137
PMAC-Commutation Enable for Motor x (Ix01) 1-13
PMATCH (Match Present and Commanded Position) 3-129
Point to program 3-47
Position and Velocity Move Specification 4-2
Position Limit
Negative Software for Motor x (Ix14) 2-53
Positive Software for Motor x (Ix13) 2-53
Position Loop Feedback Address for Motor x (Ix03) 1-13, 2-44, 2-106, 7-4
Position Processing
Axis Position Scaling
Delete Axis Definition 3-20
Report Axis Definition 3-19
Set Axis Definition 3-20
Backlash Compensation
Define Backlash Compensation Table 3-50
Delete Backlash Compensation Table 3-62
Leadscrew Compensation
Define Leadscrew Compensation (one-dimensional) 3-51

Define Leadscrew Compensation (two-dimensional) 3-55
Delete Leadscrew Compensation..... 3-62
Torque Compensation Tables
Define Torque Compensation Table..... 3-60
Delete Torque Compensation Table 3-66
Position Rollover Range for Motor x (Ix27) 1-14
Position Scale Factor for Motor x (Ix08)..... 1-13
Position-Only Move Specification 4-1
Positions of all motors 3-10
Positive Software Position Limit for Motor x (Ix13)1-14, 2-53
Power-On Phase Position Address (Ix81) 2-100
Power-On Phase Position Offset (Ix75)..... 2-92
Power-Up Mode 1-15
Power-Up Mode for Motor x (Ix80)..... 1-15
PR (Report Rotary Buffer Size)..... 3-130
Precalculation
Stopping..... 4-30
PRELUDE Subprogram Calls 4-63
Program Counter..... 3-128
Program Number for Coordinate System x (Ix91)..... 2-112
Program Step Mode Control (I53)..... 2-27
Programmed Move Calculation Time Variable (I11) 1-13
Programmed Move Segmentation Time Variable (I13) 1-13, 2-17, 3-37
PROM
Report Verrsion 3-140
PROM Date
Report 3-50
Proportional Gain for Motor x (Ix30)..... 1-14, 2-70
PSET (Position Set)..... 4-66
P-Variables
Report P-Variable Value..... 3-125
Set P-Variable Value 3-126
PVT-Mode Moves
Move Statements 4-2
PVT{data} 4-67

Q

Q (Quit) 3-131
Q{constant} (Report Q-Variable Value) 3-131
Q{constant}={expression} (Set Q-variable Value). 4-68
Q{constant}={expression} (Set Q-Variable Value) 3-132
Quit..... 3-131
Quit all executing motion programs 3-11
Q-Variables
Report Q-Variable Value..... 3-131
Set Q-Variable Value..... 3-132

R

R (Run)..... 3-132
R[H]{address} (Read Address) 3-134
Radius Size Specification
Set Circle Radius 4-69
RAPID 3-37, 4-70
Rapid Move Mode Control Variable (I50)..... 1-13
READ (Read Arguments)..... 4-70
Read Address..... 3-134
Read MACRO auxiliary parameter value..... 3-117
Real Time Interrupt Period (I8) 2-10
Real Time Interrupt Period Variable (I8) 1-13
Re-define Axis Location..... 3-46
Re-initialization Actions
Global Reset 3-23
Global Reset and Reinitialize 3-24
Repeat last command line..... 3-6
Report Addressed Motor 3-18
Report Addressed PMAC Board 3-41
Report all coordinate system status words..... 3-3
Report Available Memory 3-136
Report Axis Definition 3-19
Report command line checksum..... 3-9
Report configured address contents in binary..... 3-4
Report Coordinate System..... 3-29
Report Coordinate System Status 3-34
Report Feedrate Override 3-25
Report following errors for all motors..... 3-5
Report Global Status..... 3-38
Report global status word 3-5
Report I-Variable Value 3-76
Report last command line 3-16
Report Motor Following Error..... 3-71
Report Motor Position 3-124
Report Motor Status..... 3-30
Report Motor Velocity..... 3-139
Report M-Variable Definition 3-103
Report M-Variable Value 3-102
Report or write MACRO auxiliary parameter value 3-117
Report positions of all motors..... 3-10
Report Program Execution Pointer..... 3-128
Report PROM Date 3-50
Report P-Variable Value 3-125
Report Q-Variable Value..... 3-131
Report Rotary Buffer Size 3-130
Report status word for all motors 3-2
Report type of PMAC..... 3-137
Report velocity of all motors 3-13
Reserved for future use Variable (I46) 2-24
Reset Actions
Global Reset 3-23
Global Reset and Reinitialize 3-24
Motor Reset 3-23
Reset and Reinitialize 3-24
Response to Commands..... 2-7
RETURN 4-73
Rotary Buffer Request Off Point Variable (I17)..... 1-13
Rotary Buffer Request On Point Variable (I16) 1-13
Rotary Buffers

Report Size 3-130
Rotary Motion Program Buffers
OPEN ROTARY (Open Rotary Buffer)..... 3-123
Run 3-132
Running the Program
R (Run)..... 3-132

S

S (Step)..... 3-134
S{data} (Spindle Function) 4-74
Safety Features
Host-Generated Stop Commands
A (Abort) 3-44
CONTROL-A Command..... 3-1
CONTROL-K Command..... 3-7
CONTROL-O Command..... 3-10
CONTROL-Q Command..... 3-11
H (FeedHold)..... 3-74
K (Kill Motor) 3-91
Q (Quit) 3-131
SAVE (Copy setup parameters to non-volatile memory) 3-135
Resolver Gear Ratio..... 2-36
Second Phase Output (DAC) Bias for Motor x (Ix79) 1-15, 2-96
SEND (Message Issuance) 4-75, 7-7
Serial Communications Mode Variable (I1)...1-13, 2-3, 7-8
Serial Port Communications 1-13, 7-8
Servo Control I-Variables..... 2-69
Servo Cycle Period Extension for Motor x (Ix60)... 2-13, 2-78
Servo Interrupt Time Variable (I10)..... 1-13, 2-13
Set Addressed Motor 3-18
Set Addressed PMAC Board 3-42
Set Axis Definition 3-20
Set Feedrate Axes 3-72
Set Feedrate Override 3-27
Set I-variable Value 4-44
Set M-Variable Value..... 3-103
Set PMAC in serial port communications mode.. 3-17
Set P-Variable Value 3-126
Set Q-Variable Value..... 3-132
SIN Function 1-2
SIZE (Report Available Memory) 3-136
Slew Rate for Coordinate System x (Ix94)..... 2-114
Slip Gain..... 1-16
Splined Moves 4-78, 4-79
SQRT Function..... 1-2
Status word for all motors..... 3-2
Step 3-134
Step working motion programs in all coordinate systems 3-12
Stepping the Program 4-12
BLOCKSTOP..... 4-13
S (Step)..... 3-134
STOP 4-80
Stop Commands
A (Abort) 3-44
CONTROL-A Command..... 3-1
CONTROL-K Command..... 3-7
CONTROL-O Command..... 3-10
CONTROL-Q Command..... 3-11
H (FeedHold)..... 3-74
K (Kill Motor) 3-91
Q (Quit) 3-131
Subprogram
GOSUB..... 4-40
Subroutines and Subprograms
Alternate Location 4-60
Call Subprogram..... 4-14
GOSUB..... 4-40
GOTO 4-40
Location Label..... 4-58
Read Arguments 4-70
RETURN 4-73

T

Take command line from dual-ported RAM 3-14
TAN Function..... 1-2
Resolver Gear Ratio..... 2-34
Thumbwheel Multiplexer Port I/O
BCD Thumbwheel-Multiplexer M-Variable Definition..... 3-109
Binary Thumbwheel-Multiplexer Definition. 3-109
Resolver Thumbwheel-Multiplexer M-Variable Definition..... 3-111
Serial Thumbwheel-Multiplexer M-Variable Definition..... 3-112
Time Base Control Register Address for Coordinate System x (Ix93) 1-16
Time Base Slew Rate (and Limit) for Coordinate System x (Ix94) 1-16, 2-114
Toggle serial port half/full duplex mode 3-12
TYPE (Report type of PMAC) 3-137

U

U{data} (U-axis Move) 4-85
UNDEFINE (Clear Coordinate System Definition) 3-138
UNDEFINE ALL (Clear All Coordinate System Definitions)..... 1-3, 3-139

V

V (Report Motor Velocity)..... 1-7, 3-139
V{data} (V-axis Move) 4-86
Variables
I-Variables
Report Value..... 3-76
Set Default I-Variable Value 3-79
Set I-Variable Value 3-77
M-Variables
BCD Thumbwheel-Multiplexer Definition 3-109

Binary Thumbwheel-Multiplexer Definition ... 3-109
Dual-Ported RAM Fixed-Point Definition. 3-105
Dual-Ported RAM Floating-Point Definition .. 3-107
Long Fixed-Point Definition ... 3-105
Long Word Floating-Point Definition ... 3-108
Report Definition ... 3-103
Report Value ... 3-102
Resolver Thumbwheel-Multiplexer Definition 3-111
Self Reference M-Variable ... 3-104
Serial Thumbwheel-Multiplexer Definition 3-112
Set Value ... 3-103
Short Word Definition ... 3-115
P-Variables
Report P-Variable Value ... 3-125
Set P-Variable Value ... 3-126
Q-Variables
Report Q-Variable Value ... 3-131
Set Q-Variable Value ... 3-132
Velocity
Home Speed ... 2-59
Jog Speed ... 2-58
Maximum Permitted for Motor x (Ix16) ... 2-54
Velocity Feedforward Gain ... 1-15, 2-71
Velocity Feedforward Gain (F0) for Motor x (Ix32) 1-14
Velocity Feedforward Gain for Motor x (Ix32) ... 2-71
Velocity Loop Feedback Address for Motor x (Ix04) 1-13, 2-106
Velocity Loop Scale Factor for Motor x (Ix09) ... 1-13
velocity of all motors ... 3-13
Velocity Phase Advance Gain for Motor x (Ix76) ... 1-15, 2-93
VERSION (Report PROM Version) ... 3-140
VME-Bus Registers ... 5-35

W

W{address} (Write to Address) ... 1-4, 3-141
W{data} (W-axis Move) ... 4-87
WAIT ... 4-87
Warning Following Error Limit ... 1-14
Warning Following Error Limit for Motor x (Ix12) 1-14, 2-52, 2-83
Write MACRO auxiliary parameter value ... 3-119
Write to Address ... 3-141

X

X{data} (X-axis Move) ... 4-90

Y

Y{data} (Y-axis Move) ... 4-91

Z

Z (Make commanded axis positions zero) ... 3-142
Z{data} (Z-axis Move) ... 4-91
Zero-Move Homing ... 4-42